

грокаем

функциональное мышление

Эрик Норманд



grokking **Simplicity**

**taming complex software with
functional thinking**

Eric Normand

Foreword by Guy Steele and Jessica Kerr



MANNING
SHELTER ISLAND

играем
функциональное
мышление

Эрик Норманд



Санкт-Петербург • Москва • Минск

2023

ББК 32.973.2-018
УДК 004.42
Н83

Норманд Эрик

Н83 Прокаем функциональное мышление. — СПб.: Питер, 2023. — 608 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-1887-8

Кодовые базы разрастаются, становясь все сложнее и запутаннее, что не может не пугать разработчиков. Как обнаружить код, изменяющий состояние вашей системы? Как сделать код таким, чтобы он не увеличивал сложность и запутанность кодовой базы?

Большую часть «действий», изменяющих состояние, можно превратить в «вычисления», чтобы ваш код стал проще и логичнее.

Вы научитесь бороться со сложными ошибками синхронизации, которые неизбежно проникают в асинхронный и многопоточный код, узнаете, как компонуемые абстракции предотвращают дублирование кода, и откроете для себя новые уровни его выразительности.

Книга предназначена для разработчиков среднего и высокого уровня, создающих сложный код. Примеры, иллюстрации, вопросы для самопроверки и практические задания помогут надежно закрепить новые знания.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.973.2-018
УДК 004.42

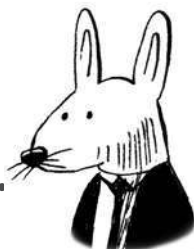
Права на издание получены по соглашению с Manning Publications. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1617296208 англ.
ISBN 978-5-4461-1887-8

© 2021 by Manning Publications Co. All rights reserved
© Перевод на русский язык ООО «Прогресс книга», 2022
© Издание на русском языке, оформление ООО «Прогресс книга», 2022
© Серия «Библиотека программиста», 2022

Оглавление



.....

https://t.me/it_boooks

Предисловие	15
Вступление	20
Благодарности	22
О книге	23
Об авторе	25
От издательства	26
Глава 1. Добро пожаловать в мир функционального мышления	27
Что такое функциональное программирование	28
Недостатки определения при практическом применении	30
Определение ФП сбивает с толку руководителей	31
Функциональное программирование рассматривается как совокупность навыков и концепций	32
Действия, вычисления и данные	33
Функциональные программисты особо выделяют код, для которого важен момент вызова	34
Функциональное программирование отличает инертные данные от работающего кода	35
Функциональные программисты разделяют действия, вычисления и данные	36
Три категории кода в ФП	37
Как нам помогают различия между действиями, вычислениями и данными	38
Чем эта книга отличается от других книг о ФП	39
Что такое функциональное мышление	40
Основные правила для идей и навыков, представленных в книге	41

Итоги главы	44
Резюме	44
Что дальше?	44
Глава 2. Функциональное мышление в действии	45
Добро пожаловать в пиццерию Тони!	46
Часть 1. Проведение различий между действиями, вычислениями и данными	47
Организация кода по частоте изменений	48
Часть 2. Использование первоклассных абстракций	49
Временные линии наглядно представляют работу распределенных систем	50
Действия на временных линиях могут выполняться в разном порядке	51
Особенности распределенных систем: урок, полученный дорогой ценой	52
Сегментация временной линии: заставляем роботов ожидать друг друга	54
Положительные уроки	55
Итоги главы	56
Резюме	56
Что дальше?	56
 ЧАСТЬ I. ДЕЙСТВИЯ, ВЫЧИСЛЕНИЯ И ДАННЫЕ	
Глава 3. Действия, вычисления и данные	58
Действия, вычисления и данные	59
Действия, вычисления и данные применимы в любых ситуациях	60
Что мы узнали при моделировании процесса покупки	63
Применение функционального мышления в новом коде	68
Наглядное представление процесса рассылки купонов по электронной почте	71
Реализация процесса отправки купонов	75
Применение функционального мышления в существующем коде	84
Распространение действий в коде	86
Действия могут принимать разные формы	87
Итоги главы	91
Резюме	91
Что дальше?	91
Глава 4. Извлечение вычислений из действий	92
Добро пожаловать в MegaMart.com!	93
Вычисление бесплатной доставки	94
Вычисление налога	95
Необходимо упростить тестирование	96
Необходимо улучшить возможности повторного использования кода	97
Различия между действиями, вычислениями и данными	98
У функций есть ввод и вывод	99

Тестирование и повторное использование связаны с вводом и выводом	100
Извлечение вычислений из действий	102
Извлечение другого вычисления из действия	105
Весь код в одном месте	117
Итоги главы	118
Резюме	118
Что дальше?	118
Глава 5. Улучшение структуры действий	119
Согласование структуры с бизнес-требованиями	120
Приведение функции в соответствие с бизнес-требованиями	121
Принцип: минимизация неявного ввода и вывода	123
Сокращение неявного ввода и вывода	124
Проверим код еще раз	127
Классификация наших расчетов	129
Принцип: суть проектирования в разделении	130
Улучшение структуры за счет разделения <code>add_item()</code>	131
Выделение паттерна копирования при записи	132
Использование функции <code>add_item()</code>	133
Классификация вычислений	134
Уменьшение функций и новые вычисления	138
Итоги главы	139
Резюме	139
Что дальше?	139
Глава 6. Неизменяемость в изменяемых языках	140
Может ли неизменяемость применяться повсеместно	141
Классификация операций чтения и/или записи	142
Три этапа выполнения копирования при записи	143
Преобразование записи в чтение с использованием копирования при записи	144
Сравнение двух версий	148
Операции копирования при записи обобщаются	149
Знакомство с массивами в JavaScript	150
Что делать с операциями чтения/записи	154
Разделение функции, выполняющей чтение и запись	154
Возвращение двух значений одной функцией	155
Операции чтения неизменяемых структур данных являются вычислениями	163
Приложения обладают состоянием, которое изменяется во времени	164
Неизменяемые структуры данных достаточно быстры	165
Операции с копированием при записи для объектов	166
Кратко об объектах JavaScript	167
Преобразование вложенных операций записи в чтение	172

Что же копируется?	173
Наглядное представление поверхностного копирования и структурного совместного использования	174
Итоги главы	178
Резюме	178
Что дальше?	178
Глава 7. Сохранение неизменяемости при взаимодействии с ненадежным кодом	179
Неизменяемость при работе с унаследованным кодом	180
Наш код копирования при записи должен взаимодействовать с ненадежным кодом	181
Защитное копирование позволяет сохранить неизменяемый оригинал	182
Реализация защитного копирования	184
Правила защитного копирования	185
Упаковка ненадежного кода	187
Защитное копирование, которое вам может быть знакомо	190
Сравнение копирования при записи с защитным копированием	192
Глубокое копирование затратнее поверхностного	193
Трудности реализации глубокого копирования в JavaScript	194
Диалог между копированием при записи и защитным копированием	196
Итоги главы	199
Резюме	199
Что дальше?	199
Глава 8. Многоуровневое проектирование: часть 1	200
Что такое проектирование программной системы	201
Что такое многоуровневое проектирование	202
Развитие чувства проектирования	203
Паттерны многоуровневого проектирования	204
Паттерн 1. Прямолинейная реализация	205
Три уровня детализации	219
Выделение цикла for	223
Обзор паттерна 1. Прямолинейная реализация	232
Итоги главы	234
Резюме	234
Что дальше?	234
Глава 9. Многоуровневое проектирование: часть 2	235
Паттерны многоуровневого проектирования	236
Паттерн 2. Абстрактный барьер	237
Абстрактные барьеры скрывают реализацию	238

Игнорирование подробностей симметрично	239
Замена структуры данных корзины	240
Повторная реализация корзины в виде объекта	242
Абстрактный барьер позволяет игнорировать подробности	243
Когда следует (или не следует!) использовать абстрактные барьеры	244
Обзор паттерна 2. Абстрактный барьер	245
Код становится более прямолинейным	246
Паттерн 3. Минимальный интерфейс	247
Обзор паттерна 3. Минимальный интерфейс	253
Паттерн 4. Удобные уровни	254
Паттерны многоуровневой архитектуры	255
Что можно узнать из графа о коде?	256
Код в верхней части графа проще изменять	257
Важность тестирования кода нижних уровней	259
Код нижних уровней лучше подходит для повторного использования	262
Итоги: что можно узнать о коде по графу вызовов	263
Итоги главы	264
Резюме	264
Что дальше?	264

ЧАСТЬ II. ПЕРВОКЛАССНЫЕ АБСТРАКЦИИ

Глава 10. Первоклассные функции: часть 1	266
Отдел маркетинга все еще должен согласовываться с разработчиками	268
Признак «кода с душком»: неявный аргумент в имени функции	269
Рефакторинг: явное выражение неявного аргумента	271
Определение того, что является и что не является первоклассным значением	273
Не приведут ли строки с именами полей к новым ошибкам?	274
Усложнят ли первоклассные поля изменения API?	276
Мы будем использовать множество объектов и массивов	281
Первоклассные функции могут заменить любой синтаксис	284
Пример цикла for: еда и уборка	287
Рефакторинг: замена тела обратным вызовом	293
Что это за синтаксис	296
Почему мы упаковали код в функцию	297
Итоги главы	300
Резюме	300
Что дальше?	300
Глава 11. Первоклассные функции: часть 2	301
Одна проблема, два метода рефакторинга	302
Рефакторинг копирования при записи	303

Рефакторинг копирования при записи для массивов	304
Возвращение функций функциями	314
Итоги главы	324
Резюме	324
Что дальше?	324
Глава 12. Функциональные итерации	325
Один признак «кода с душком» и два рефакторинга	326
MegaMart создает группу взаимодействия с клиентами	327
map() в примерах	331
Инструмент функционального программирования: map()	332
Три способа передачи функций	334
Пример: адреса всех клиентов	336
filter() в примерах	339
Инструмент функционального программирования: filter()	340
Пример: клиенты без покупок	341
reduce() в примерах	344
Инструмент функционального программирования: reduce()	345
Пример: конкатенация строк	346
Что можно сделать с reduce()	350
Сравнение трех инструментов функционального программирования	352
Итоги главы	353
Резюме	353
Что дальше?	353
Глава 13. Сцепление функциональных инструментов	354
Группа взаимодействия с клиентами продолжает работу	355
Улучшение цепочек, способ 1: присваивание имен шагам	361
Улучшение цепочек, способ 2: присваивание имен обратным вызовам	362
Улучшение цепочек: сравнение двух способов	363
Пример: адреса клиентов с одной покупкой	364
Преобразование существующих циклов for в инструменты функционального программирования	369
Совет 1. Создавайте данные	370
Совет 2. Работайте с целыми массивами	371
Совет 3. Используйте много мелких шагов	372
Совет 3. Используйте много мелких шагов	373
Сравнение функционального кода с императивным	375
Советы по сцеплению	376
Советы по отладке	378
Другие функциональные инструменты	378
reduce() для построения значений	383

Творческий подход к представлению данных	385
О выравнивании точек	390
Итоги главы	391
Резюме	392
Что дальше?	392

Глава 14. Функциональные инструменты для работы с вложенными данными .. 393

Функции высшего порядка для значений в объектах	394
Явное выражение имени поля	395
Построение update()	396
Использование update() для изменения значений	397
Рефакторинг: замена схемы «чтение — изменение — запись» функцией update() ..	398
Функциональный инструмент: update()	399
Наглядное представление значений в объектах	400
Наглядное представление обновлений вложенных данных	406
Применение update() к вложенным данным	407
Построение updateOption()	408
Построение update2()	409
Наглядное представление update2() с вложенными объектами	410
Четыре варианта реализации incrementSizeByName()	413
Построение update3()	414
Построение nestedUpdate()	416
Анатомия безопасной рекурсии	421
Наглядное представление nestedUpdate()	422
Сила рекурсии	423
Конструктивные особенности при глубоком вложении	425
Абстрактные барьеры для глубоко вложенных данных	426
Сводка использования функций высшего порядка	427
Итоги главы	428
Резюме	428
Что дальше?	429

Глава 15. Изоляция временных линий 430

Осторожно, ошибка!	431
Пробуем кликать вдвое чаще	432
Временные диаграммы показывают, что происходит с течением времени	434
Два фундаментальных принципа временных диаграмм	435
Две неочевидные детали порядка действий	439
Построение временной линии добавления товара в корзину: шаг 1	440
Асинхронным вызовам необходимы новые временные линии	441
Разные языки, разные потоковые модели	442
Поэтапное построение временной линии	443

Изображение временной линии добавления товара в корзину: шаг 2	445
Временные диаграммы отражают две разновидности последовательного кода ...	447
Временные диаграммы отражают неопределенность в упорядочении параллельного кода	448
Принципы работы с временными линиями	449
Однопоточная модель в JavaScript	450
Асинхронная очередь JavaScript	452
AJAX и очередь событий	453
Полный пример с асинхронными вызовами	454
Упрощение временной линии	455
Чтение завершенной временной линии	461
Упрощение временной диаграммы добавления товара в корзину: шаг 3	463
Рисование временной линии (шаги 1–3)	465
Резюме: построение временных диаграмм	468
Сопоставление временных диаграмм помогает выявить проблемы	469
Два медленных клика приводят к правильному результату	470
Два быстрых клика приводят к неправильному результату	471
Временные линии с совместным использованием ресурсов создают проблемы ...	472
Преобразование глобальной переменной в локальную	473
Преобразование глобальной переменной в аргумент	474
Расширение возможностей повторного использования кода	477
Принцип: в асинхронном контексте в качестве явного вывода вместо возвращаемого значения используется обратный вызов	478
Итоги главы	483
Резюме	483
Что дальше?	483

Глава 16. Совместное использование ресурсов между временными линиями ... 484

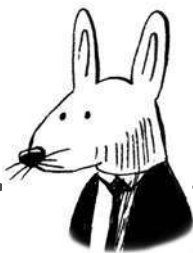
Принципы работы с временными линиями	485
Корзина все еще содержит ошибку	486
Необходимо гарантировать порядок обновлений DOM	489
Реализация очереди в JavaScript	492
Принцип: берите за образец решения по совместному использованию из реального мира	501
Совместное использование очереди	502
Анализ временной линии	507
Принцип: чтобы узнать о возможных проблемах, проанализируйте временную диаграмму	510
Пропуск задач в очереди	511
Итоги главы	515
Резюме	515
Что дальше?	515

Глава 17. Координация временных линий	516
Принципы работы с временными линиями	517
Ошибка!	518
Как изменился код	520
Идентификация действий: шаг 1	521
Представление каждого действия: шаг 2	522
Упрощение диаграммы: шаг 3	526
Анализ возможных вариантов упорядочения	529
Почему эта временная линия выполняется быстрее	530
Ожидание двух параллельных обратных вызовов	532
Примитив синхронизации для нарезки временных линий	533
Использование Cut() в коде	535
Анализ неопределенных упорядочений	537
Анализ параллельного выполнения	538
Анализ для нескольких кликов	539
Примитив для однократного вызова	546
Неявная и явная модель времени	548
Резюме: манипулирование временными линиями	553
Итоги главы	553
Резюме	554
Что дальше?	554
Глава 18. Реактивные и многослойные архитектуры	555
Два архитектурных паттерна	556
Связывание причин и эффектов изменений	557
Что такое реактивная архитектура	558
Плюсы и минусы реактивной архитектуры	559
Ячейки как первоклассное состояние	560
Переменную ValueCell можно сделать реактивной	561
Обновление значков доставки при изменении ячейки	562
FormulaCell и вычисление производных значений	563
Изменяемое состояние в функциональном программировании	564
Как реактивная архитектура изменяет конфигурацию систем	565
Отделение эффектов от причин	566
Центр связей между причинами и эффектами	568
Интерпретация последовательности шагов как конвейера	569
Гибкость временной линии	570
Второй архитектурный паттерн	574
Что такое многослойная архитектура	575
Краткий обзор: действия, вычисления и данные	576
Краткий обзор: многоуровневое проектирование	577

Традиционная многоуровневая архитектура	578
Функциональная архитектура	579
Упрощение изменений и повторного использования	580
Понятия, используемые для размещения правила в слое	583
Анализ удобочитаемости и громоздкости решения	584
Итоги главы	588
Резюме	588
Что дальше?	588

Глава 19. Путешествие в мир функционального программирования	
продолжается	589
План главы	590
Полученные профессиональные навыки	591
Главные выводы	592
Приобретение навыков: победы и разочарования	593
Параллельные пути к мастерству	594
Песочница: создание побочного проекта	596
Песочница: практические упражнения	597
Реальный код: устранение ошибок	598
Реальный код: постепенное улучшение проекта	599
Популярные функциональные языки	600
Функциональные языки с наибольшим количеством вакансий	601
Функциональные языки на разных платформах	602
Возможность получения знаний	602
Математическая основа	603
Литература	605
Итоги главы	606
Резюме	606
Что дальше?	606

Предисловие



Гай Стил

Я пишу программы уже более 52 лет. И мне это все еще интересно, потому что всегда появляются новые проблемы, которые нужно решить, и новые штуки, которые нужно изучать. За прошедшие годы мой стиль программирования серьезно менялся в процессе изучения новых алгоритмов, новых языков программирования и новых методов реализации кода.

Когда я учился программировать в 1960-х годах, считалось, что перед написанием кода следует нарисовать блок-схему программы. Каждое вычисление в программе изображалось прямоугольным блоком, каждое решение — ромбом, а каждая операция ввода/вывода — еще какой-нибудь фигурой. Блоки соединялись стрелками, представляющими передачу управления между блоками. Написание программы сводилось к написанию кода для каждого блока в определенном порядке. Каждый раз, когда стрелка указывала куда-либо за пределами следующего блока, который вы должны были запрограммировать, программист также писал команду `goto` для обозначения необходимой передачи управления. Проблема была в том, что блок-схемы были двумерными, а код — одномерным, и даже если структура блок-схемы на бумаге была красивой и аккуратной, понять ее после записи в виде кода могло быть достаточно сложно. Если провести в коде стрелку от каждой команды `goto` к ее точке передачи, результат часто напоминал клубок спагетти, поэтому в те дни часто говорили о сложностях понимания и сопровождения «спагетти-кода».

Первые изменения в моем стиле программирования были вызваны движением «структурного программирования» в начале 1970-х. Обращаясь к прошлому, я вижу *две* важные идеи, выработанные в ходе обсуждения в сообществе. Обе относятся к средствам **организации потока управления**.

Первая идея получила большую популярность. Она заключалась в том, что большая часть логики передачи управления может быть выражена несколькими

простыми паттернами: последовательным выполнением, многовариантными решениями (например, командами `if-then-else` и `switch`) и повторным выполнением (например, циклы `for` и `while`). Иногда эта идея чрезмерно упрощается до девиза «Никаких команд `goto`!», но здесь важны паттерны, при последовательном использовании которых выяснялось, что необходимость в `goto` возникает крайне редко. Вторая идея, не столь популярная, но не менее важная, гласила, что последовательные команды могут группироваться в блоки, которые должны правильно вкладываться друг в друга, а нелокальная передача управления может осуществляться к концу блока или из блока (команды `break` и `continue`), но не может входить в блок извне.

Когда я впервые ознакомился с идеями структурного программирования, у меня не было доступа к подобному языку. Однако я обнаружил, что стал писать код на Fortran чуть более аккуратно, организуя его по принципам структурного программирования. Даже низкоуровневый код на ассемблере я писал так, словно был компилятором, преобразующим структурный язык программирования в машинные команды. Оказалось, что эта дисциплина упрощает написание и сопровождение моих программ. Да, я все еще писал команды `goto` и команды ветвления, но почти всегда делал это по одному из стандартных паттернов. Код получался намного более понятным.

В старые недобрые времена, когда я программировал на Fortran, все переменные, которые могли понадобиться программе, приходилось объявлять заранее в одном месте, а за ними следовал исполняемый код. (В языке COBOL эта конкретная организация была жестко формализована: переменные объявлялись в «разделе данных» программы, который начинался со слов DATA DIVISION. Далее следовал код, который всегда начинался со слов PROCEDURE DIVISION.) К любой переменной можно было обратиться из любой точки кода. Программисту было труднее понять, как именно можно обратиться к каждой конкретной переменной и изменить ее.

На мой стиль программирования также сильно повлияло «объектно-ориентированное программирование», которое в моем представлении является сочетанием и высшей точкой развития более ранних идей объектов, классов, «сокрытия информации» и «абстрактных типов данных». Оглядываясь назад, я снова вижу, как этот грандиозный синтез породил две выдающиеся идеи, и *обе* были связаны с **организацией доступа к данным**. Первая идея заключается в том, что переменные должны каким-то образом инкапсулироваться, чтобы программисту было проще видеть, что их чтение или запись возможны только из определенных частей кода. Возможны разные варианты инкапсуляции, от простого объявления локальных переменных в блоке (а не в начале программы) до объявления переменных в классе (или модуле), чтобы они были доступны только для методов этого класса (или процедур внутри модуля). Классы или модули могут гарантировать, что группы переменных обладают определенными характеристиками целостности — методы или процедуры могут быть запрограммированы так, чтобы при обновлении одной переменной связанные с ней переменные также обновлялись соответ-

ствующим образом. Вторая идея, наследование, обозначает, что программист может определить более сложный объект путем расширения более простых объектов через добавление новых переменных или методов и, возможно, переопределение существующих методов. Вторая идея становится возможной благодаря первой.

В то время, когда я узнал об объектах и абстрактных типах данных, я писал много кода на языке Lisp. И хотя Lisp не является чистым объектно-ориентированным языком, на нем достаточно просто реализуются структуры данных и обращения к ним только через проверенные методы (реализованные как функции Lisp). Уделяя внимание организации данных, я мог пользоваться многими преимуществами объектно-ориентированного программирования даже с учетом того, что программировал на языке, который не принуждал к соблюдению этой дисциплины.

Третьим фактором, повлиявшим на мой стиль программирования, стало «функциональное программирование», суть которого иногда упрощается до девиза «Никаких побочных эффектов!». Впрочем, это невозможно. На самом деле функциональное программирование предоставляет средства для **упорядочения побочных эффектов**, чтобы они не возникали *где угодно*, — **именно это и является темой книги**.

И здесь мы снова имеем дело с двумя главными идеями, которые работают в сочетании друг с другом. Первая: отделение *вычислений*, которые не влияют на внешнее окружение и выдают один и тот же результат при многократном выполнении, от *действий*, которые могут выдавать разные результаты при каждом выполнении и могут создавать побочные эффекты для внешнего окружения (например, выводить текст на экран или запускать ракеты). Программу будет проще понять, если она организована на базе стандартных паттернов. С такими паттернами программист быстро разберется, какие части могут иметь побочные эффекты, а какие являются «всего лишь вычислениями». Стандартные паттерны можно разделить на две подкатегории: типичные для однопоточных программ (последовательное выполнение) и типичные для многопоточных программ (параллельное выполнение).

Вторая глобальная идея включает набор методов обработки больших коллекций данных (массивов, списков, баз данных) по принципу «все сразу», а не элемент за элементом. Такие методы оказываются наиболее эффективными тогда, когда элементы могут обрабатываться независимо друг от друга, когда на них не влияют побочные эффекты. Получается, что вторая идея снова работает лучше благодаря первой.

В 1995 году я помогал в написании первой полной спецификации языка программирования Java, а на следующий год уже участвовал в создании первого стандарта JavaScript (ECMAScript). Оба этих языка очевидно являются объектно-ориентированными: в Java вообще нет такого понятия, как глобальная переменная, — каждая переменная должна быть объявлена внутри некоторого класса или метода. Кроме того, в обоих языках нет команды **goto**: разработчики языка пришли к выводу, что идеология структурного программирования достиг-

ла успеха и в `goto` более нет надобности. В наши дни миллионы программистов прекрасно обходятся без `goto` и глобальных переменных.

Но как насчет функционального программирования? Существуют чисто функциональные языки, например Haskell, которые широко применяются на практике. Haskell можно использовать для вывода текста на экран или для запуска ракет, но для использования побочных эффектов в Haskell устанавливаются очень жесткие ограничения. Как следствие, вы не можете просто вставить команду вывода в любую точку в программе Haskell, чтобы понять, что в ней происходит.

С другой стороны, Java, JavaScript, C++, Python и многие другие языки программирования не являются чисто функциональными, но они взяли на вооружение многие идеи из функционального программирования, упрощающие их использование. В этом и суть: если вы понимаете ключевые принципы организации побочных эффектов, эти простые идеи можно использовать в любом языке программирования. Книга показывает, как это делается. На первый взгляд она кажется длинной, но это доступная литература, наполненная практическими примерами и врезками с объяснением технических терминов. Я обратил на нее внимание, получил большое удовольствие и узнал пару новых фишек, которые мне не терпится применить в своем коде.

Надеюсь, вам она тоже понравится!

Джессика Керр

Когда я только начинала изучать программирование, оно нравилось мне своей предсказуемостью. Каждая из моих программ была незатейливой: небольшая, работающая на одной машине и простая для использования одним человеком (мной). Разработка современных программных продуктов — совсем другое дело. Программные продукты огромны. Они не пишутся одним человеком. Они работают на многих машинах и во многих процессах. Ими пользуются разные люди, в том числе и те, которых вообще не интересует, как работает программа.

Полезные программы не могут быть простыми.

Что же делать программисту?

Методы функционального программирования в течение 60 лет развивались в умах теоретиков. Исследователи вообще любят делать позитивные заявления относительно того, чего в принципе быть не может.

Последнее десятилетие или два разработчики применяли эти методы в коммерческих программных продуктах. И это было своевременно, потому что соответствовало доминированию веб-приложений: каждое приложение представляет собой распределенную систему, загружаемую на неизвестные компьютеры, с которой работают неизвестные люди. Функциональное программирование отлично подходит для таких целей. Целые категории трудноуловимых ошибок становятся в принципе невозможными.

Однако перенос функционального программирования из академической области в коммерческую разработку не проходит легко и мирно. Мы не работаем на Haskell и не начинаем с нуля. Мы зависим от библиотек и исполнительных сред, которые нам неподконтрольны. Наши программные продукты взаимодействуют с множеством других систем — просто вывести ответ недостаточно. Путь от мира ФП до коммерческого программирования долог.

Эрик прошел этот путь за нас. Он глубоко изучил функциональное программирование, выявил в нем все самое полезное и делится этим с нами.

В прошлом осталась строгая типизация, «чистые» языки, теория категорий. Вместо них мы видим код, который взаимодействует с окружающим миром; разработчик сознательно оставляет данные без изменений и осуществляет логическое разбиение кода для большей ясности. На смену возвышенным абстракциям приходят степени и уровни абстракции. На смену отказу от состояния приходят способы безопасного хранения состояния.

Эрик предлагает взглянуть на существующий код с новых точек зрения. Новые диаграммы, новые «запахи кода», новые эвристики. Да, все это вышло из мира функционального программирования, но когда он находит новые способы выражения своих мыслей, чтобы мы тоже могли ими воспользоваться, он создает нечто новое. Нечто такое, что поможет всем нам в работе над нашими творениями.

Мои простые программы были бесполезными для окружающего мира. Полезные программы никогда не будут простыми. Тем не менее мы можем сделать код проще, чем он был до этого. И мы можем управлять критическими частями, которые взаимодействуют с миром. Эрик распутывает все эти противоречия за нас.

После прочтения этой книги вы будете лучше разбираться в программировании — и более того, в разработке программного обеспечения.

Вступление



Я впервые познакомился с функциональным программированием (ФП) при изучении языка Common Lisp в 2000 году, во время прохождения курса по искусственному интеллекту в университете. По сравнению с другими объектно-ориентированными языками, к которым я привык, Lisp поначалу казался каким-то непривычным и нестандартным. Но к концу семестра я реализовал на Lisp уже достаточно учебных заданий и перестал ощущать дискомфорт. Я ощутил вкус ФП, хотя только начинал понимать его.

С годами я все больше разбирался в функциональном программировании. Написал собственную реализацию Lisp. Читал книги о Lisp. Начал писать на Lisp учебные задания из других курсов. Вскоре я познакомился с Haskell, а в 2008 году и с Clojure. И в Clojure я обрел свою музу. Он строился на базе 50-летней традиции Lisp, но на современной и практичной платформе. А сообщество щедро выдавало идеи о вычислениях, природе данных и практике разработки больших программных систем. Оно было благоприятной почвой для философии, компьютерной теории и технического проектирования. И я был поглощен этим. Я вел блог, посвященный Clojure, и в конечном итоге основал компанию по обучению Clojure.

Тем временем также повышалась популярность Haskell. Я несколько лет профессионально работал на Haskell. У него много общего с Clojure, но существует и ряд различий. Как определить *функциональное программирование*, чтобы определение включало как Clojure, так и Haskell? С формулирования этого вопроса и началась история появления данной книги.

Основной была идея действий, вычислений и данных как главного отличия парадигмы функционального программирования. Если вы спросите любого функционального программиста, он согласится с тем, что это отличие критично для практики ФП, хотя лишь немногие согласятся с тем, что оно является определяющим аспектом парадигмы. Все это отдает когнитивным диссонансом. Как известно, люди склонны учить так, как когда-то учили их. В этом когни-

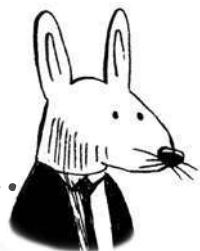
тивном диссонансе я увидел возможность помочь людям изучать ФП по новым принципам.

Я проработал много черновых вариантов этой книги. Один был излишне теоретическим. В другом демонстрировались впечатляющие возможности ФП. Третий отличался чрезмерной дидактичностью. Четвертый был полностью повествовательным. Но в конечном итоге после всех наставлений со стороны редактора я пришел к нынешнему варианту, в котором функциональное программирование рассматривается как совокупность навыков. По сути, речь идет о навыках, стандартных в кругах ФП, но редко встречающихся за их пределами. И когда я определился с этим подходом, планирование книги свелось к нахождению таких навыков, их упорядочению и расстановке приоритетов. После этого работа пошла очень быстро.

Конечно, в книге не получится рассмотреть все возможные навыки. Функциональному программированию не менее 60 лет. Мне не хватило места для описания многих методов, которые определенно стоило бы описать. Надеюсь, что навыки, рассмотренные в книге, станут отправной точкой для обсуждения и дальнейшего обучения для других авторов.

Моя главная цель при написании книги заключалась в том, чтобы по крайней мере запустить процесс легитимизации функционального программирования как прагматичного варианта для профессиональных программистов. Когда программист хочет изучить объектно-ориентированное программирование, он найдет множество книг по теме, написанных именно для него — начинающего профессионала. В этих книгах описываются паттерны, принципы и практики, на основе которых учащийся может формировать свои навыки. У функционального программирования такой учебной литературы нет. Существующие книги в основном имеют академическую природу, а тем, которые пытаются ориентироваться больше на практику, по моему мнению, не удастся объяснить основные концепции. Однако все необходимые знания и опыт есть у тысяч функциональных программистов. Надеюсь, эта книга будет способствовать расцвету литературы о функциональном программировании.

Благодарности



Прежде всего, мне хотелось бы поблагодарить Рика Хики и все сообщество Clojure — неиссякаемый источник философских, научных и технических идей, относящихся к программированию. Вы вдохновляли меня и были моим стимулом.

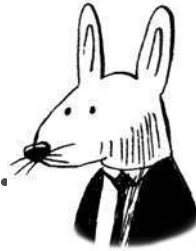
Я должен поблагодарить свою семью, особенно Вирджинию Мединилья, Оливию Норманд и Изабеллу Норманд, поддерживавших меня во время написания книги своим терпением и любовью. Также я благодарю Лиз Уильямс, которая постоянно помогала мне своими советами.

Спасибо Гаю Стиллу и Джесси Керр за их внимание к книге. Видеть суть вещей дано не каждому, но я считаю, что вы правильно поняли замысел этой книги. И конечно, спасибо за личные впечатления, которыми вы поделились во вступлении.

Наконец, хочу поблагодарить сотрудников издательства Manning. Берт Бэйтс, спасибо за бесчисленные часы обсуждений, часто менявших направление, — благодаря им эта книга все-таки была завершена. Спасибо за постоянные наставления относительно того, как лучше учить. Спасибо за терпение и поддержку в то время, когда я разбирался, какой должна быть эта книга. Благодарю Джени Стаут за то, что она вела проект в нужном направлении.

Спасибо Дженифер Хаул за прекрасный дизайн книги. Спасибо всем остальным работникам Manning, которые участвовали в работе. Я знаю, что эта книга была непростой во многих отношениях. Хочу упомянуть всех рецензентов: Майкл Эйдинбас, Джеймс Дж. Билецки, Хавьер Колладо, Тео Деспудис, Фернандо Гарсиа, Клайв Харбер, Фред Хит, Колин Джойс, Оливье Кортен, Джоэл Лукка, Филипп Мешан, Брайан Миллер, Орландо Мендес, Нага Паван Кумар Т., Роб Пачеко, Дэн Поззи, Аншуман Пурухит, Конор Редмонд, Эдвард Рибейро, Дэвид Ринк, Армин Сейдлин, Кай Стрем, Кент Спиллнер, Серж Симон, Ричард Таттл, Айвен Фелизо и Грег Райт — ваши предложения помогли улучшить эту книгу.

О книге



Для кого написана эта книга

Книга написана для программистов с практическим опытом от 2 до 5 лет. Предполагается, что вы уже знаете хотя бы один язык программирования. Также желательно, чтобы вы построили хотя бы одну достаточно крупную систему, чтобы представлять, с какими проблемами разработчики сталкиваются при масштабировании. Примеры написаны в стиле JavaScript, направленном на читаемость кода. Если вы понимаете код C, C#, C++ или Java, у вас не будет особых сложностей.

Структура издания

Книга состоит из двух частей и 19 глав. В каждой части описан некоторый фундаментальный навык, а затем исследуются другие связанные с ним навыки. Каждая часть завершается описанием принципов проектирования и архитектуры в контексте функционального программирования. В части I, начинающейся с главы 3, вводятся различия между действиями, вычислениями и данными. Часть II, начинающаяся с главы 10, знакомит читателя с идеей первоклассных значений.

- В главе 1 приводится общая информация о книге и основных концепциях функционального программирования.
- В главе 2 приведен краткий обзор возможностей, которые откроет перед вами использование этих навыков.

Часть I. Действия, вычисления и данные.

- Глава 3 открывает первую часть книги: в ней представлены практические навыки, которые позволят вам различать действия, вычисления и данные.
- Глава 4 показывает, как проводить рефакторинг кода в вычисления.

- Из главы 5 вы узнаете, как усовершенствовать действия в том случае, если они не могут быть преобразованы в вычисления посредством рефакторинга.
- В главе 6 представлен важный механизм неизменяемости — так называемое копирование при записи.
- В главе 7 описан дополняющий механизм неизменяемости, называемый защитным копированием.
- В главе 8 представлен способ организации кода в соответствии со смысловыми уровнями.
- Глава 9 помогает анализировать уровни в соответствии с принципами сопровождения кода, тестирования и повторного использования.

Часть II. Первокласные абстракции.

- Глава 10 открывает вторую часть книги описанием концепции первокласных значений.
- Глава 11 показывает, как наделить любую функцию суперспособностями.
- Из главы 12 вы узнаете, как создавать и использовать средства перебора массивов.
- Глава 13 помогает строить сложные вычисления из средств, описанных в главе 12.
- В главе 14 представлены функциональные средства для работы с вложенными данными, а также кратко затрагивается тема рекурсии.
- Глава 15 знакомит читателя с концепцией временных диаграмм как средства анализа выполнения вашего кода.
- Глава 16 показывает, как организовать безопасное совместное использование ресурсов между временными линиями без создания ошибок.
- Глава 17 показывает, как манипуляции с порядком и повторением действий могут использоваться для предотвращения ошибок.
- Глава 18 завершает часть II обсуждением двух архитектурных паттернов для построения сервисов в функциональном программировании.
- Глава 19 завершает книгу ретроспективным обзором и рекомендациями по дальнейшему обучению.

Книгу следует читать с самого начала и по порядку. Каждая глава строится на материале предыдущей. Не пропускайте упражнения. Думайте над упражнениями, пока не найдете ответ. Упражнения включены для того, чтобы помочь вам формулировать собственное мнение по субъективным вопросам. У упражнений «Ваш ход!» есть ответы. Они включены в книгу для того, чтобы дать вам возможность потренироваться и укрепить усвоенные навыки на реалистичных сценариях. Ничто не мешает вам прервать чтение в любой момент — еще никто не освоил ФП одним чтением книг. Если вы узнали что-то важное, отложите

книгу и дайте материалу закрепиться в памяти. Книга подождет до того момента, когда вы будете готовы к ней вернуться.

О примерах кода

В книге встречаются примеры кода. Код пишется на JavaScript в стиле, который на первое место ставит ясность. Я использую JavaScript вовсе не потому, чтобы показать вам, что на JavaScript можно заниматься функциональным программированием. Собственно, JavaScript не блещет в области ФП. Но именно потому, что в нем не реализована серьезная поддержка ФП, этот язык отлично подходит для обучения. Многие функциональные конструкции приходится строить самостоятельно, что позволит нам глубже понять их. Кроме того, вы будете больше ценить такие конструкции, предоставляемые языком (таким, как Haskell или Clojure).

Части текста, содержащие элементы кода, сразу видны. Для ссылок на переменные и другие фрагменты синтаксиса, встроенные в текст, используется **моноширинный шрифт** — так они выделяются в обычном тексте. Тот же шрифт используется в листингах. Иногда код выделяется **цветом**, чтобы показать изменения по сравнению с предыдущим шагом. Переменные верхнего уровня и имена функций выделяются **жирным** шрифтом. Я также использую подчеркивание для привлечения внимания к важным фрагментам кода.

Исходный код примеров этой книги можно загрузить по адресу <https://www.manning.com/books/grokking-simplicity>.

Другие ресурсы в интернете

Сетевых и автономных ресурсов, посвященных функциональному программированию, слишком много, чтобы перечислить их здесь. И никакие ресурсы не каноничны настолько, чтобы заслуживать особого упоминания. Я рекомендую поискать локальные группы по функциональному программированию. Мы лучше учимся тогда, когда делаем это вместе с другими.

Если вам нужны дополнительные материалы, относящиеся конкретно к этой книге, ссылки на ресурсы и другие материалы доступны по адресу <https://grokking simplicity.com>.

Об авторе

Эрик Норманд — опытный функциональный программист, преподаватель, докладчик и автор книг, посвященных функциональному программированию. Он родился в Новом Орлеане и начал программировать на Lisp в 2000 году. Эрик создает учебные материалы по Clojure на сайте PurelyFunctional.tv. Он также консультирует компании, желающие использовать функциональное программирование для более эффективного решения своих бизнес-задач. Вы можете ознакомиться с его докладами на международных конференциях по

программированию. Его статьи, доклады, обучающие материалы и информация о консультациях доступны на сайте LispCast.com.

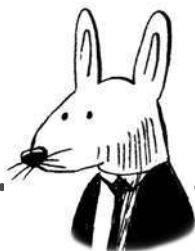
От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

https://t.me/it_boooks



В этой главе вы

- ✓ Узнаете определение функционального мышления.
- ✓ Поймете, чем эта книга отличается от других книг по функциональному программированию.
- ✓ Познакомитесь с главной отличительной особенностью кода с точки зрения функциональных программистов.
- ✓ Решите, насколько эта книга подходит лично вам.

Мы начнем с того, что определим функциональное мышление и расскажем, как его главная отличительная особенность помогает программистам-практикам строить более качественные продукты. Кроме того, будет приведен обзор предстоящего путешествия в контексте двух главных идей, осознанных функциональными программистами.

Что такое функциональное программирование

Программисты постоянно спрашивают меня, что такое функциональное программирование (ФП) и для чего его лучше использовать. Мне трудно объяснить, для чего лучше подойдет ФП, потому что это парадигма общего назначения. Оно хорошо подходит для всего. А о том, в каких областях оно раскрывается по-настоящему, вы узнаете через несколько страниц.

Дать определение функциональному программированию тоже непросто. Мир функционального программирования огромен. Оно применяется в промышленном программировании и в академических кругах. Впрочем, большая часть книг о функциональном программировании написана именно теоретиками.

Эта книга отличается от типичных книг о функциональном программировании. Она безусловно ориентирована на промышленное применение ФП. Весь материал книги должен приносить практическую пользу для профессиональных программистов.

Возможно, вам встретятся определения из других источников, и важно понимать, как они связаны с тем, что мы обсуждаем в этой книге. Типичное определение, приведенное в Википедии, в слегка перефразированном виде выглядит так:

функциональное программирование (ФП), *сущ.*

1. Парадигма программирования, для которой характерно использование математических функций и предотвращение побочных эффектов.
2. Стиль программирования, использующий только чистые функции без побочных эффектов.

Подчеркнутым терминам необходимо дать определение

Разберем подчеркнутые термины.

К *побочным эффектам* относится все, что делает функция помимо возвращения значения: например, отправка электронной почты или изменение глобального состояния. Побочные эффекты могут создавать проблемы, потому что они происходят при каждом вызове вашей функции. Если вас интересует только возвращаемое значение, а не побочные эффекты, значит, в программе происходит что-то непреднамеренное.

Согласно типичному определению функциональное программирование выглядит совершенно непрактичным.



Функциональные программисты обычно стараются избегать побочных эффектов, которые не являются абсолютно необходимыми.

Чистые функции — функции, которые зависят только от своих аргументов и не имеют побочных эффектов. С одними и теми же аргументами они всегда выдают одно возвращаемое значение. Можно называть их *математическими функциями*, чтобы отличить от функций как элемента в программировании. Функциональные программисты уделяют особое внимание использованию чистых функций, потому что они более просты для понимания и управления.

Определение подразумевает, что функциональные программисты полностью избегают побочных эффектов и используют только чистые функции. Тем не менее это не так. Функциональные программисты, работающие над реальными программами, используют побочные эффекты и функции с побочными эффектами.

Обычно программы запускаются как раз ради этого!

Типичные побочные эффекты

1. Отправка электронной почты.
2. Чтение файла.
3. Переключение светового индикатора.
4. Отправка веб-запроса.
5. Включение тормоза в машине.

Недостатки определения при практическом применении

Такое определение хорошо подойдет для академических кругов, но у него есть ряд недостатков с точки зрения программиста-практика. Еще раз присмотримся к определению:

функциональное программирование (ФП), *сущ.*

1. Парадигма программирования, для которой характерно использование математических функций и предотвращение побочных эффектов.
2. Стиль программирования, использующий только чистые функции без побочных эффектов.

Для наших целей такое определение создает три основные проблемы.

Проблема 1: ФП необходимы побочные эффекты

В определении сказано, что ФП стремится избегать побочных эффектов, но программы обычно запускаются как раз ради побочных эффектов. Какой прок от почтового клиента, который не отправляет электронную почту? Определение подразумевает, что мы должны полностью избегать их, тогда как на практике побочные эффекты используются там, где это необходимо.

Проблема 2: ФП неплохо справляется с побочными эффектами

Функциональные программисты знают, что побочные эффекты необходимы, хоть и проблематичны, поэтому существует значительное количество инструментов для работы с ними. Определение подразумевает, что мы используем только чистые функции. Напротив, мы достаточно часто применяем функции с побочными эффектами. Существует великое множество функциональных методов, которые упрощают их использование.



Загляни в словарь

Побочные эффекты — это любое поведение функций, кроме возврата значения.

Чистые функции зависят только от своих аргументов и не имеют побочных эффектов.

Проблема 3: Практичность ФП

Из определения может сложиться впечатление, что ФП имеет в основном математическую природу и для реальных программ оно непрактично. Тем не менее многие важные программные системы написаны с применением функционального программирования.

Определение особенно сильно сбивает с толку людей, которые знакомятся с ФП именно по нему. Представьте руководителя, действующего из лучших побуждений, который прочитал определение в Википедии.

Определение ФП сбивает с толку руководителей

Представьте, что Дженна, программист-энтузиаст, хочет использовать ФП для сервиса отправки электронной почты. Она знает, что ФП поможет спроектировать систему для улучшения общей надежности. Ее начальник не знает, что такое ФП, поэтому он находит определение в Википедии.



Программист-энтузиаст

А мы можем применить ФП для написания нового сервиса отправки электронной почты?



Ее начальник

Эм-м... Я тебе перезвоню.

Начальник ищет «функциональное программирование» в Википедии:

... предотвращение побочных эффектов ...

Он находит «побочный эффект» в Google. Типичные побочные эффекты:

- отправка электронной почты
- ...



Хм-м, а что такое «побочный эффект»?



Избегать отправки электронной почты?

Но мы же пишем сервис отправки электронной почты!

Позднее в тот же день...

По поводу ФП: нет, мы не можем его использовать. Это слишком рискованно.



Но я думала, что ФП идеально подойдет для нашего сервиса.

Функциональное программирование рассматривается как совокупность навыков и концепций

Мы не будем использовать типичное определение в этой книге. ФП каждый понимает по-своему, это огромная область для научных исследований и практики.

Я говорил со многими функциональными программистами относительно того, какие свойства ФП кажутся им наиболее полезными. Книга «Грокаем функциональное мышление» стала квинтэссенцией навыков, логических умозаключений и перспектив функциональных программистов-практиков. В книгу были включены только самые практичные и эффективные идеи.

В книге вы не найдете данных последних исследований или эзотерических идей. Мы будем изучать только навыки и концепции, применяемые в наши дни. В ходе исследований я обнаружил, что самые важные концепции ФП применимы даже в объектно-ориентированном и процедурном коде, а также в разных языках программирования. Красота ФП по-настоящему проявляется в том, что ФП имеет дело с полезными, универсальными практиками программирования.

Взглянем на навык, важность которого не будет отрицать ни один функциональный программист: разграничение действий, вычислений и данных.

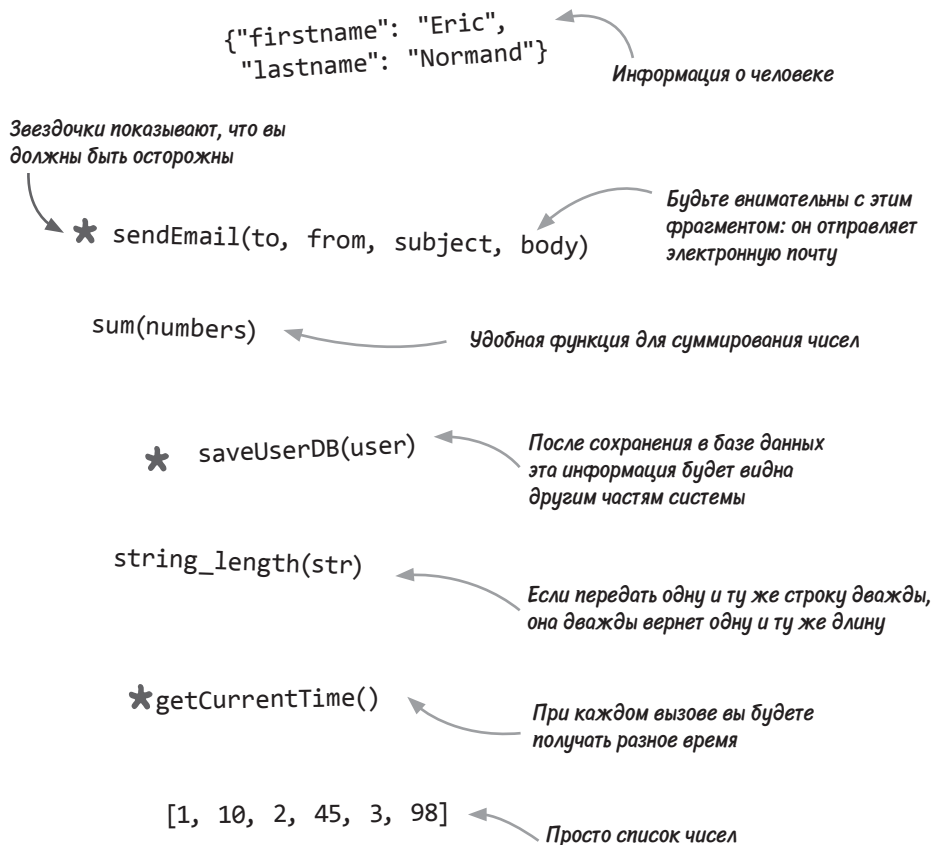


Действия, вычисления и данные

Когда функциональный программист смотрит на код, он немедленно классифицирует его на три категории:

1. Действия (actions — A).
2. Вычисления (calculations — C).
3. Данные (data — D).

Рассмотрим несколько примеров кода из существующей базы данных. Будьте особенно внимательны с фрагментами, помеченными звездочкой.



Функции со звездочкой требуют особого внимания, потому что они зависят от того, когда или сколько раз они вызываются. Например, важная электронная почта не должна отправляться дважды или отправляться ноль раз.

Фрагменты, помеченные звездочкой, относятся к *действиям*. Отделим их от остальных.

Функциональные программисты особо выделяют код, для которого важен момент вызова

Проведем линию и переместим все функции, зависящие от момента вызова, по одну сторону от линии:

*Действия зависят от того,
когда они вызываются*

*sendEmail(to, from, subject, body)

*saveUserDB(user)

*getCurrentTime()

Действия

{"firstname": "Eric",
"lastname": "Normand"}

*Все остальное не зависит
от момента вызова*

sum(numbers)

string_length(str)

[1, 10, 2, 45, 3, 98]

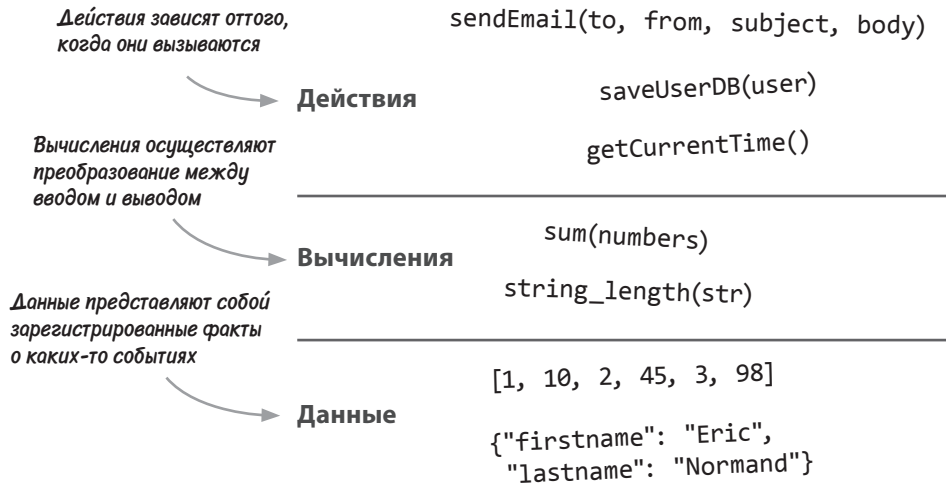
Это очень важный момент. Действия (все, что находится над линией) зависят от того, когда или сколько раз они вызываются. Они требуют особого внимания.

Однако с тем, что находится под линией, работать намного проще. Неважно, когда вы вызовете функцию `sum`, — она будет каждый раз вызывать правильный ответ. Неважно и то, сколько раз вы ее вызовете. Она не повлияет на остальные части программы или окружающий мир за пределами программы.

Однако существует еще одно различие: одна часть кода может выполняться, другая остается инертной. Проведем еще одну линию на следующей странице.

Функциональное программирование отличает инертные данные от работающего кода

Еще одна линия отделяет вычисления от данных. Ни вычисления, ни данные не зависят от того, сколько раз они будут использоваться. Отличие заключается в том, что вычисления могут выполняться, а данные выполняться не могут. Данные инертны и прозрачны. Вычисления непрозрачны в том смысле, что вы не знаете, что именно сделает вычисление до его запуска.



Различия между действиями, вычислениями и данными являются фундаментальными для ФП. Любой функциональный программист согласится с тем, что уметь различать их исключительно важно. Многие другие концепции и навыки в ФП строятся на базе этого навыка.

Важно подчеркнуть, что функциональные программисты не питают отвращения к использованию кода в любой из трех категорий, потому что все они важны. Однако при этом они понимают плюсы и минусы и пытаются выбрать оптимальный инструмент для своей работы. В общем случае они предпочитают данные вычислениям, а вычисления — действиям. Работать с данными проще всего.

Стоит еще раз подчеркнуть: **функциональные программисты видят эти категории каждый раз, когда они смотрят на любой код.** Это главная особенность точки зрения ФП. Эта классификация лежит в основе целого ряда навыков и концепций. Мы займемся их изучением в оставшихся главах части I.

Посмотрим, что эта классификация скажет о простом сервисе управления задачами.

.....

Функциональные программисты предпочитают данные вычислениям, а вычисления — действиям.

.....

Функциональные программисты разделяют действия, вычисления и данные

Различия между действиями, вычислениями и данными фундаментальны для ФП. Без них практиковать ФП просто невозможно. Кому-то это покажется очевидным, но просто чтобы убедиться в том, что все мы находимся на одной волне, рассмотрим простой сценарий, поясняющий смысл этих трех понятий.

Представьте облачный сервис для управления проектами. Когда клиенты помечают свои задачи как завершенные, центральный сервер отправляет уведомления по электронной почте.

Где же в этой картине действия, вычисления и данные? Иначе говоря, как функциональный программист определяет, что где происходит?

Шаг 1. Пользователь помечает задачу как завершенную

В результате инициируется UI-событие, которое является *действием*, так как результат зависит от того, сколько раз оно происходит.

Шаг 2. Клиент отправляет сообщение серверу

Отправка сообщения является *действием*, но само сообщение представляет собой *данные* (инертные байты, которые должны интерпретироваться программой).

Шаг 3. Сервер получает сообщение

Получение сообщения является *действием*, так как результат зависит от того, сколько раз оно происходит.

Шаг 4. Сервер вносит изменение в базу данных

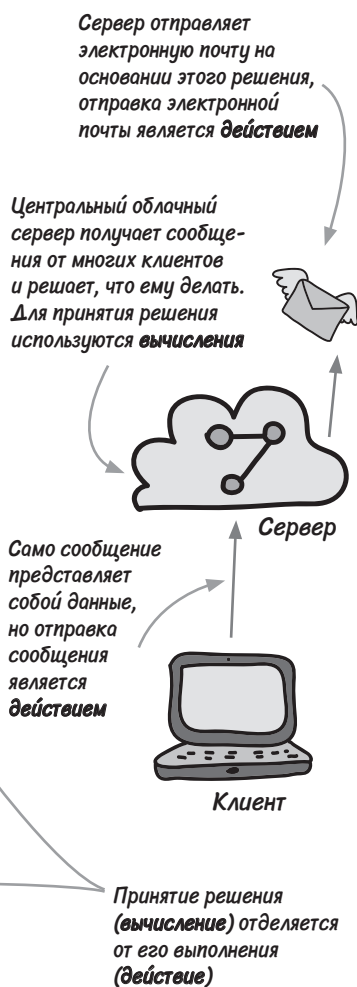
Изменение внутреннего состояния является *действием*.

Шаг 5. Сервер принимает решение относительно того, кого следует уведомить

Принятие решения является *вычислением*. Для одних и тех же исходных данных сервер будет принимать одно и то же решение.

Шаг 6. Сервер отправляет уведомление по электронной почте

Отправка электронной почты является *действием*, так как отправка одного сообщения дважды не эквивалентна его однократной отправке.



Если вам это пока кажется не очень логичным, не беспокойтесь, потому что вся первая часть книги будет посвящена пониманию того, как выявляются такие различия, почему мы их делаем и как они помогают в улучшении кода. Как говорилось ранее, различия между действиями, вычислениями и данными — первая большая идея, представленная в книге.

Три категории кода в ФП

Рассмотрим основные характеристики трех категорий.

В ФП имеются средства для использования каждой категории

1. Действия (A)

Все, что зависит от времени выполнения или количества выполнений, является *действием*. Если я отправляю неотложенную электронную почту сегодня, это совсем не то же самое, как если бы я отправил ее через неделю. И конечно, отправка одного сообщения электронной почты десять раз сильно отличается от его однократной отправки.

Действия

- Средства для безопасного изменения состояния во времени.
- Средства, обеспечивающие упорядочение.
- Средства, гарантирующие, что действия выполняются только один раз.

2. Вычисления (C)

Вычисления — это расчеты, преобразующие ввод в вывод. Они всегда дают один и тот же результат при получении одних и тех же данных. Их можно вызвать когда угодно и где угодно — это никак не повлияет на что-либо за их пределами. Это сильно упрощает тестирование и безопасное использование, а вам не нужно беспокоиться о том, сколько раз и когда они выполняются.

Вычисления

- Статический анализ, направленный на обеспечение правильности.
- Математические средства, хорошо подходящие для программных продуктов.
- Стратегии тестирования.

3. Данные (D)

Данные представляют собой зарегистрированные факты, относящиеся к событиям. Мы особо выделяем данные, потому что они менее сложны, чем исполняемый код. Они обладают хорошо понятными свойствами. Данные интересны тем, что они обладают смыслом без выполнения. Они могут интерпретироваться разными способами. Возьмем для примера чек за обед в ресторане: на его основании менеджер ресторана может определить наиболее популярные позиции меню. А посетитель может воспользоваться чеком для анализа бюджета.

*Это различие
лежит у истоков
функционального мышления,
и оно станет отправной точкой
для всего, что вы узнаете
в этой книге.*

Данные

- Способы организации данных для эффективного доступа.
- Механизмы долгосрочного хранения данных.
- Принципы отражения важной информации с использованием данных.



Как нам помогают различия между действиями, вычислениями и данными

Функциональное программирование отлично подходит для распределенных систем, и многие программные продукты, созданные сегодня, являются распределенными.

ФП сейчас у всех на слуху. Важно понять, что это: мода, которая однажды умрет, или в этом есть что-то существенное.

Функциональное программирование — это не просто модное течение. Это одна из самых старых парадигм программирования, которая уходит своими корнями к еще более старым математическим дисциплинам. Причина, по которой оно обретает популярность только сейчас, связана с тем, что благодаря интернету и массовому распространению таких устройств, как телефоны, портативные компьютеры и облачные серверы, возникла необходимость в новой точке зрения на программные продукты, которая учитывает существование многих программных компонентов, взаимодействующих по сети.

Когда компьютеры начинают общаться по сети, возникает хаос. Сообщения приходят не по порядку, дублируются или не приходят вообще. Разобраться в том, что и когда происходит, с помощью воспроизведения изменений во времени очень важно, но также сложно. Чем больше мы можем сделать для исключения зависимости от того, когда или сколько раз выполняется некоторый код, тем проще будет избежать серьезных ошибок в дальнейшем.

Данные и вычисления не зависят от того, сколько раз они выполняются (или от количества обращений в программу). Перемещая большую часть кода в данные и вычисления, мы избавляем код от проблем, присущих распределенным системам.

Проблемы с действиями все еще остаются, но эти проблемы идентифицированы и изолированы. Кроме того, в ФП существует набор инструментов для работы с действиями, которые делают их более безопасными даже в условиях неопределенности распределенных систем. При этом перемещение кода из действий в вычисления позволит уделить больше внимания тем действиям, которые нуждаются в нем более всего.

Три правила распределенных систем

1. Сообщения поступают не по порядку.
2. Каждое сообщение может поступить один или несколько раз или не поступить вовсе.
3. Не получив ответа, вы не имеете никакой информации о том, что же случилось.

С переходом на распределенную архитектуру ситуация сильно усложняется

Чем эта книга отличается от других книг о ФП

Используется практический подход к программированию

Большая часть обсуждений ФП имеет академическую природу: ученые исследуют теорию. Теория — замечательная штука, пока мы не пытаемся применить ее на практике.

Многие книги о ФП сосредоточены на академической стороне происходящего. Они обучают читателя рекурсии и стилю передачи продолжений (continuation-passing style). С этой книгой дело обстоит иначе. В ней собран преимущественно практический опыт многих профессиональных функциональных программистов. Эти люди уважают теорию, но научились придерживаться того, что реально работает.

Описываются реальные ситуации

Вы не найдете в этой книге определений метода Фибоначчи или сортировки слиянием. Вместо этого в рассматриваемых сценариях воспроизводятся ситуации, которые вполне могут встретиться вам в ходе работы. Мы применяем функциональное мышление к существующему коду, к новому коду и к архитектуре.

Основное внимание уделяется проектированию программных продуктов

Легко взять маленькую задачу и найти элегантное решение. При написании учебных программ архитектура не нужна. Необходимость в принципах проектирования возникает только в масштабных программах.

Многие книги по ФП вообще обходятся без проектирования, потому что программы очень маленькие. В реальном мире архитектура системы должна быть простой в обслуживании в будущем. Книга учит принципам функционального проектирования на каждом уровне масштабирования, от строки кода до целого приложения.

Книга передает богатство возможностей ФП

Функциональные программисты накапливали принципы и методы с 1950-х годов. За эти годы в компьютерной теории многое изменилось, но многое прошло проверку временем. Автор книги прикладывает значительные усилия, чтобы показать, что функциональное мышление сейчас более актуально, чем когда-либо.

Книга нейтральна по отношению к языку

Многие книги описывают возможности конкретного функционального языка. Часто это означает, что люди, работающие на других языках, не получают пользы от чтения.

В этой книге для примеров кода используется JavaScript, который не слишком подходит для *реализации* ФП. Но как выяснилось, JavaScript оказался отличным языком для *обучения* ФП именно из-за своего несовершенства. Ограничения языка заставляют нас остановиться и задуматься.

И хотя примеры написаны на JavaScript, эта книга написана не о ФП на JavaScript. Анализируйте их логику, а не язык.

Примеры кода были написаны с расчетом на максимальную ясность, а не на конкретный стиль JavaScript. Если вы можете читать код C, Java, C# или C++, то здесь вы легко разберетесь.

Что такое функциональное мышление

Функциональное мышление — совокупность навыков и идей, используемых функциональными программистами для решения задач при написании программного кода. Этот набор навыков весьма обширен. В этой книге я постараюсь изложить две выдающиеся идеи, которые играют очень важную роль в функциональном программировании: 1) проведение различий между действиями, вычислениями и данными и 2) использование первоклассных абстракций. Они не исчерпывают весь арсенал идей ФП, но закладывают прочную и практичную основу для дальнейших построений. Они проведут вас по пути от новичка до профессионального функционального программиста.

С каждой идеей связаны определенные навыки. Они также соответствуют двум частям этой книги. В каждой части представлены практические навыки, которые подробно разбираются строка за строкой, функция за функцией. Каждая часть завершается одной-двумя главами с описанием архитектуры; они дают общую картину происходящего.

Рассмотрим две основные идеи и навыки, описанные в каждой главе.

Часть I. Различия между действиями, вычислениями и данными

Как было показано ранее, функциональные программисты разделяют весь код на три категории: действия, вычисления и данные. Иногда встречаются другие термины, но в этой книге я буду называть их именно так. Эти категории соответствуют разным уровням сложности понимания, тестирования и повторного использования кода.

В этой главе мы уже начали обсуждать это важное различие. В части I вы научитесь определять категорию любого фрагмента кода, преобразовывать действия в вычисления и изменять действия для упрощения работы с ними. В главах, посвященных архитектуре, рассказано, как выявление уровней в коде упрощает его сопровождение, тестирование и повторное использование.

Часть II. Первоклассные абстракции

Программисты всех специальностей находят общие процедуры и присваивают им имена, упрощающие их повторное использование в будущем. Функциональные программисты делают то же самое, но часто им удается повысить уровень повторного использования процедур за счет передачи процедур процедурам. Идея звучит странно, но она в высшей степени практична. Вы узнаете, как это делается и как избежать злоупотребления этой возможностью. В завершение будут рассмотрены две стандартные архитектуры: реактивная и многослойная.

Путешествие будет захватывающим. Впрочем, не будем забегать вперед. Ведь мы все еще находимся в части I! Прежде чем отправляться в путь, установим некоторые основные правила, касающиеся того, что вам предстоит узнать.

Основные правила для идей и навыков, представленных в книге

Функциональное программирование — необъятная тема. Вы не сможете узнать все от начала и до конца. Придется выбирать, что изучать. Следующие основные правила помогут с выбором практического знания для программиста.

1. Навыки не могут базироваться на возможностях языка

Существует множество языков функционального программирования, возможности которых специально создавались для поддержки ФП. Например, во многих функциональных языках реализованы очень мощные системы типов.

Если вы работаете на одном из таких языков — прекрасно! Но даже без этого функциональное мышление может вам помочь. Мы сосредоточимся только на навыках, не зависящих от языковых возможностей. Это означает, что, хотя системы типов очень полезны, в тексте книги они будут упоминаться не так часто.

2. Навыки должны иметь непосредственную практическую ценность

Функциональное программирование используется как в промышленности, так и в академических кругах. Ученые иногда концентрируются на идеях, имеющих теоретическую важность. И это хорошо, но нам хотелось бы сохранить практическую направленность. В книге будут представлены только те навыки, которые пригодятся вам здесь и сейчас. Собственно, идеи, которые будут излагаться в книге, пригодятся даже в том случае, если они не встретятся в вашем коде. Например, умение идентифицировать действия с самого начала поможет предупредить некоторые ошибки.

3. Навыки должны применяться независимо от текущего состояния кода

Одни из нас начинают работу над совершенно новыми проектами, в которых еще не написано ни строки кода. Другие работают над существующими кодовыми базами из сотен тысяч строк. Третьи находятся где-то посередине. Идеи и навыки, представленные в книге, помогут каждому независимо от ситуации. Речь идет не о переписывании кода в функциональном стиле. Необходимо принимать практичные решения и работать с тем кодом, который у вас есть.



Мы неплохо подгото-
вились к предстоящему
путешествию.
Присоединяйтесь!



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Я использую объектно-ориентированный (ОО) язык. Пригодится ли мне эта книга?

О: Да, книга будет полезна. Принципы, описанные в книге, универсальны. Некоторые из них похожи на принципы ОО-проектирования, которые могут быть вам известны. Другие отличаются от них, так как происходят от другой фундаментальной концепции. Фундаментальное мышление обладает ценностью независимо от того, на каком языке вы работаете.

В: Каждый раз, когда я заглядывал в книги о ФП, они были слишком отвлеченными и перегруженными математикой. Эта книга из их числа?

О: Нет! Теоретики любят ФП, потому что вычисления абстрактны и их удобно анализировать в статьях. К сожалению, ФП обсуждают в основном ученые-теоретики.

Однако многие люди продуктивно работают, применяя ФП. И хотя мы следим за академической литературой, функциональные программисты точно так же пишут код, как и большинство профессиональных программистов. Мы делимся знаниями друг с другом относительно того, как решать повседневные задачи. Часть этой информации вы найдете в книге.

В: Почему JavaScript?

О: Очень хороший вопрос. Язык JavaScript широко известен и доступен. Если вы программируете в Сети, то хотя бы немного его знаете. Синтаксис JavaScript знаком большинству программистов. И хотите верить, хотите нет, но в JavaScript есть все необходимое для ФП, включая функции и некоторые базовые структуры данных.

JavaScript далеко не идеален для ФП. Тем не менее эти несовершенства позволяют нам вывести на первый план принципы ФП. Реализация принципов на языке, в котором они не реализованы по умолчанию, — полезный навык (особенно если учесть, что по умолчанию они не реализованы в большинстве языков).

В: Почему существующего определения ФП недостаточно? Для чего использовать термин «функциональное мышление»?

О: Еще один хороший вопрос. Стандартное определение — полезная крайность для поиска новых путей теоретических исследований. Фактически оно задает вопрос: «Что можно сделать, если полностью отказаться от побочных эффектов?» Оказывается, сделать можно достаточно много, причем такого, что представляет интерес для инженерно-технических разработок.

Впрочем, в стандартном определении делаются некоторые неявные допущения, которые весьма неочевидны. В этой книге мы обязательно их разберем. «Функциональное мышление» и «функциональное программирование» по своей сути являются синонимами. Новый термин просто подразумевает более свежий подход.

Итоги главы

Функциональное программирование — это большая, богатая методами и принципами область. Тем не менее все начинается с разделения действий, вычислений и данных. Книга учит практической стороне ФП. Материал применим к любому языку и любой задаче. Существуют тысячи функциональных программистов, и я надеюсь, что эта книга убедит вас присоединиться к их числу.

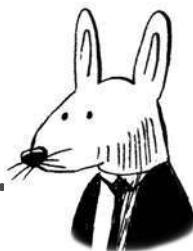
Резюме

- Книга разделена на две части, соответствующие двум фундаментальным идеям и сопутствующим группам навыков: классификации кода на действия, вычисления и данные и использованию абстракций более высокого порядка.
- Типичное определение функционального программирования устраивало ученых-исследователей, но до настоящего времени не существовало подходящего определения для программистов-практиков. Это объясняет, почему ФП может показаться абстрактным и непрacticным.
- Функциональное мышление — совокупность навыков и концепций ФП. Оно является главной темой этой книги.
- Функциональные программисты различают три категории кода: действия, вычисления и данные.
- Действия зависят от времени, поэтому их сложнее всего реализовать правильно. Мы отделяем их, чтобы уделить им больше внимания.
- Вычисления от времени не зависят. Мы стараемся писать побольше кода в этой категории, потому что правильно реализовать их относительно несложно.
- Данные инертны, они требуют интерпретации. Данные понятны и просты в хранении и передаче.
- Для примеров в книге используется JavaScript, потому что этот язык имеет знакомый синтаксис. Некоторые концепции JavaScript будут представлены там, где они понадобятся.

Что дальше?

Итак, мы сделали уверенный первый шаг в области функционального мышления. Возможно, вам уже хочется узнать, как выглядит программирование на базе функционального мышления. В следующей главе рассматриваются решения задач с использованием фундаментальных идей, определяющих структуру этой книги.

https://t.me/it_boooks



В этой главе

- ✓ Примеры применения функционального мышления в реальных задачах.
- ✓ Как многоуровневое проектирование улучшает структуру программного продукта?
- ✓ Наглядное представление действий на временных диаграммах.
- ✓ Обнаружение и решение проблем, связанных с хронометражем, с помощью временных диаграмм.

В этой главе приведен широкий обзор двух масштабных идей, которые будут рассматриваться в книге. Главная цель главы — дать читателю представление о стиле мышления, применяемом в ФП. Не огорчайтесь, если что-то покажется непонятным. Помните, что каждая из этих идей будет более подробно рассмотрена через несколько глав. А главу стоит рассматривать как головокружительный обзор функционального мышления в действии.

Добро пожаловать в пиццерию Тони!

Добро пожаловать в пиццерию Тони! Наступил 2118 год. Оказывается, в будущем тоже любят пиццу, но теперь ее готовят роботы. И роботы программируются на JavaScript. Кто бы мог подумать!

Тони использует функциональное мышление в коде, управляющем работой ее ресторанов. Мы кратко познакомимся с некоторыми системами, включая кухню и склад, и посмотрим, как в них применяются два уровня функционального мышления.

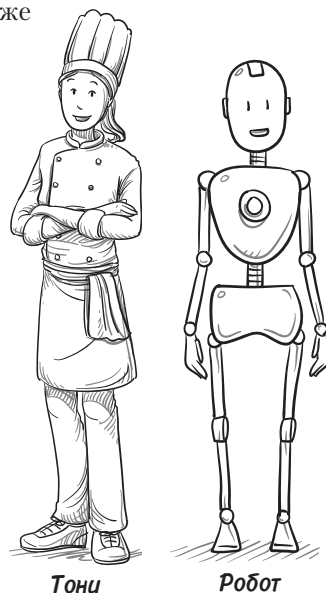
Чтобы вам не пришлось возвращаться на несколько страниц назад, напомним вам эти два уровня. Вот как они используются в пиццериях Тони.

Часть 1. Проведение различий между действиями, вычислениями и данными

Тони сознательно отличает код, использующий ингредиенты и другие ресурсы (действия), от кода, в котором они не используются (вычисления). В этой главе будут рассмотрены примеры всех категорий, а затем мы увидим, как Тони формирует уровни в своем коде, используя принципы многоуровневого проектирования.

Часть 2. Использование первоклассных абстракций

Кухня Тони представляет собой распределенную систему, потому что несколько роботов совместно готовят пиццу. Вы узнаете, как Тони с помощью *временных диаграмм (timeline diagram)* пытается понять принцип работы этой системы (не всегда успешно). Вы также увидите, как она использует *первоклассные функции* (функции, получающие функции в аргументах) для координации своих роботов, чтобы масштабировать процесс выпечки.



Часть 1. Проведение различий между действиями, вычислениями и данными

Бизнес Тони заметно расширился, и возникла проблема масштабирования. Тем не менее ей удалось справиться с этой проблемой благодаря функциональному мышлению. Основное направление, по которому она применяла функциональное мышление, было и самым фундаментальным: я говорю о проведении различий между действиями, вычислениями и данными. Каждый функциональный программист должен различать эти категории, которые сообщают нам, какие части кода (вычисления и данные) просты, а какие требуют больше внимания. Заглянув в код Тони, вы увидите в нем все три основные категории.

1. Действия

К действиям относится весь код, который зависит от того, когда или сколько раз он будет вызван. В действиях используются ингредиенты или другие ресурсы (например, фургон для доставки), поэтому Тони приходится тщательно следить за их выполнением.

Примеры действий

- Раскатка теста.
- Доставка пиццы.
- Заказ ингредиентов.

Примеры
категорий

2. Вычисления

Вычисления представляют решения или планирование. Их выполнение не влияет на окружающий мир. Тони любит вычисления, потому что она может выполнять их в любое время и в любом месте, не беспокоясь о том, что они устроят хаос на ее кухне.

Примеры вычислений

- Копирование рецепта.
- Определение списка закупок.

3. Данные

Тони старается как можно шире использовать неизменяемые данные. К этой категории относятся бухгалтерские отчеты, данные склада и рецепты пиццы. Данные обладают исключительной гибкостью, потому что их можно хранить, передавать по сети и использовать разными способами.

Примеры данных

- Заказы от клиентов.
- Чеки.
- Рецепты.

Все эти примеры актуальны для бизнеса Тони по доставке пиццы. Но различия действуют на всех уровнях, от выражений JavaScript на самом нижнем уровне до самых больших функций. О том, как эти три категории взаимодействуют при обращении друг к другу, рассказано в главе 3.

Проведение различий между тремя категориями жизненно важно для ФП, хотя функциональные программисты могут использовать другие слова для их описания. **К концу части I вы будете уверенно идентифицировать действия и вычисления и перемещать код между ними.** Посмотрим, как Тони использует многоуровневое проектирование в своей кодовой базе.

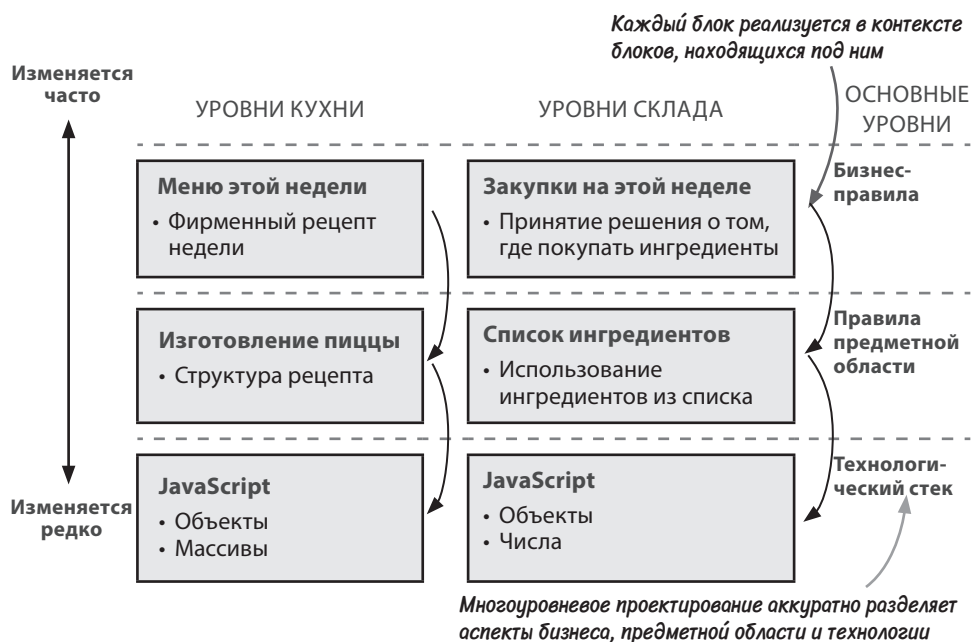
Организация кода по частоте изменений

Первый взгляд на многоуровневое проектирование

Со временем программное обеспечение Тони должно изменяться и расширяться вместе с ее бизнесом. Применяя функциональное мышление, она научилась структурировать свой код для минимизации затрат на внесение изменений.

Представьте, что вы рисуете диаграмму. В нижней части размещается то, что изменяется реже, а в верхней — то, что изменяется чаще всего.

Какие части программной системы Тони окажутся в нижней части диаграммы, а какие попадут наверх? Несомненно, сам язык JavaScript изменяется реже всего. На нижнем уровне будут находиться все встроенные языковые конструкции (например, массивы и объекты). В середине будет находиться весь код, связанный с приготовлением пиццы. Наверху будет располагаться вся конкретика, относящаяся к бизнесу, например содержимое меню на эту неделю.



Каждый уровень строится на уровнях под ним. Это означает, что каждый блок кода строится на более стабильном фундаменте. Создавая программную систему таким образом, мы гарантируем, что изменения в коде будут настолько простыми, насколько это возможно. Код наверху изменяется легко, потому что от него зависит минимум другого кода. Конечно, содержимое нижних уровней может измениться, но вероятность этого намного меньше.

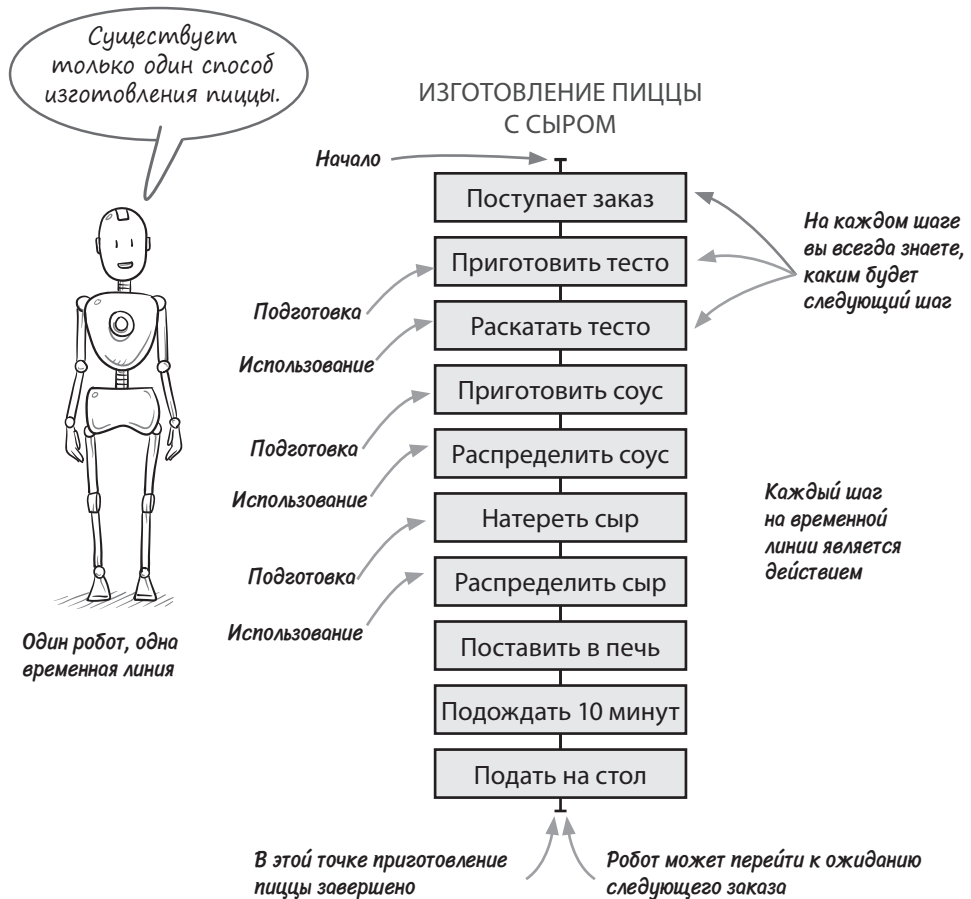
Функциональные программисты называют этот архитектурный паттерн *многоуровневым проектированием* (stratified design), потому что он делит систему на уровни. В общем случае выделяются три основных уровня: бизнес-правила, правила предметной области и технологический стек.

Многоуровневое проектирование будет более подробно рассмотрено в главах 8 и 9. Это отличный способ организации кода, упрощающий его тестирование, повторное использование и сопровождение.

Часть 2. Использование первоклассных абстракций

Применительно к роботизированной кухне

На кухне Тони сейчас трудится всего один робот, и у нее начинают возникать проблемы с масштабированием. Она не успевает выпекать пиццу достаточно быстро, чтобы удовлетворять запросы клиентов. Ниже приведена *временная диаграмма* (последовательный план) действий робота по изготовлению пиццы.

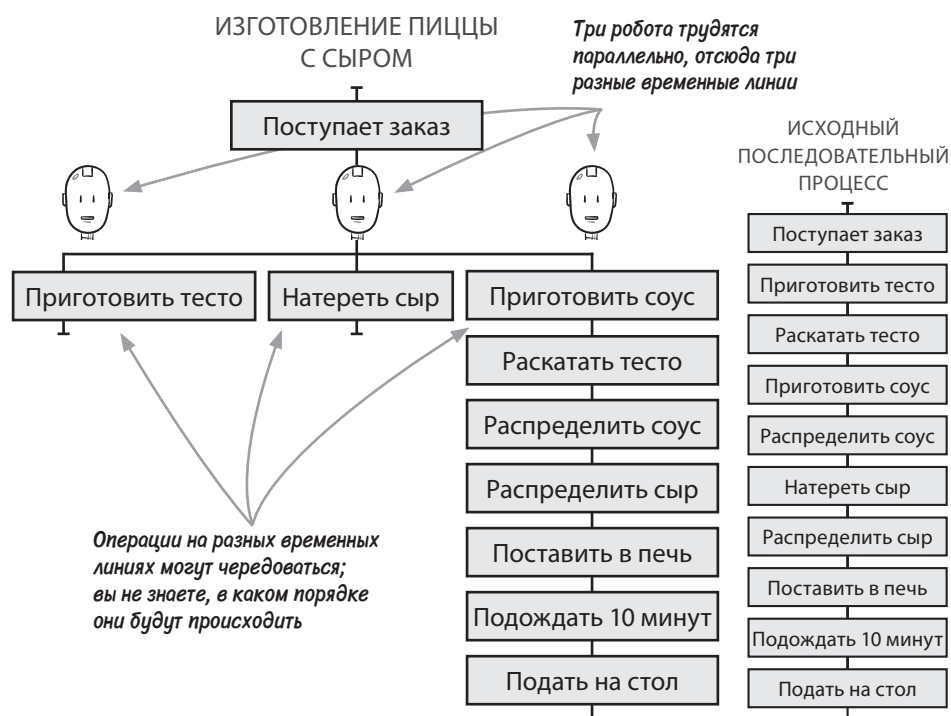


Временная диаграмма помогает понять, как действия будут выполняться с течением времени. Помните: действия зависят от того, когда они выполняются, поэтому важно проследить за тем, чтобы они выполнялись в правильном порядке. Вы увидите, как Тони работает со своей диаграммой, чтобы ее пizzeria работала более эффективно. Все, что она делает (включая построение самих диаграмм), будет рассмотрено, начиная с главы 15. А пока просто сядьте поудобнее и понаблюдайте за ее работой. Понимать все прямо сейчас нам не нужно.

Временные линии наглядно представляют работу распределенных систем

Единственный робот Тони хорошо готовит пиццу, но он не успевает делать все. Он работает по последовательному принципу. Тони уверена, что можно сделать так, чтобы три робота вместе работали над одной пиццей. Работу можно разделить на три фазы, которые выполняются параллельно: приготовление теста, приготовление соуса и натирание сыра.

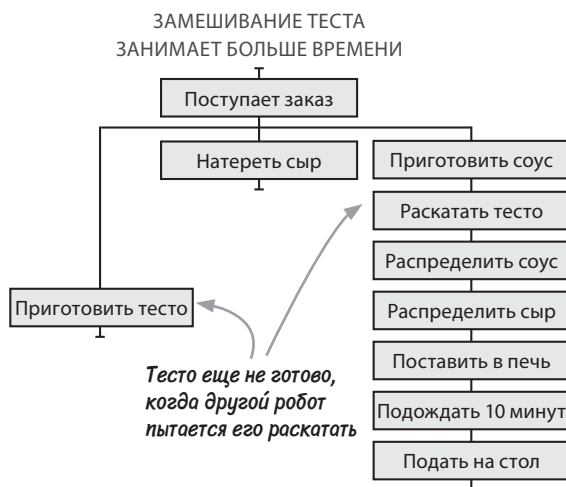
Но как только на кухне начинают работать несколько роботов, возникает распределенная система. Исходный порядок действий может быть нарушен. Тони рисует временную диаграмму, чтобы понять, что будут делать роботы при выполнении своих программ. У каждого робота появляется собственная временная линия (timeline), которая выглядит примерно так (вы научитесь рисовать такие линии в части II).



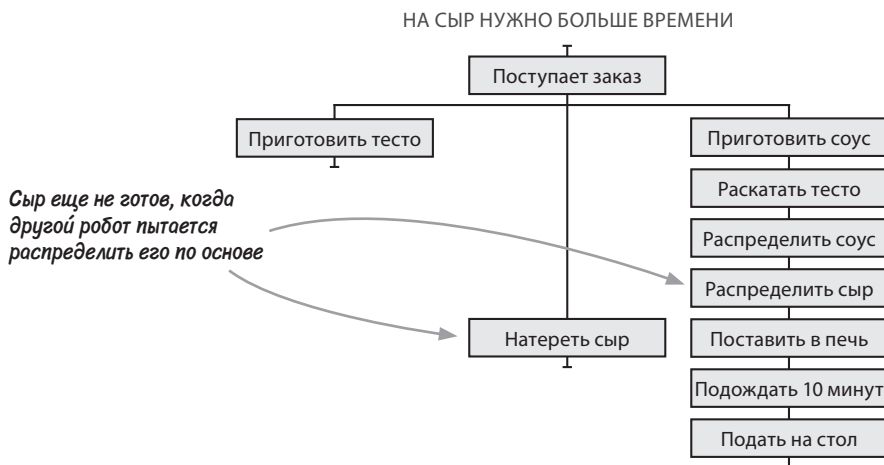
Временная диаграмма поможет Тони понять суть проблем с ее программой, но она не учла изменения в порядке действий. Когда три робота работали на кухне в ресторане этой ночью, произошла катастрофа: большинство пицц было приготовлено неправильно. Существует много вариантов отработки этих диаграмм, которые приведут к нарушению рецепта.

Действия на временных линиях могут выполняться в разном порядке

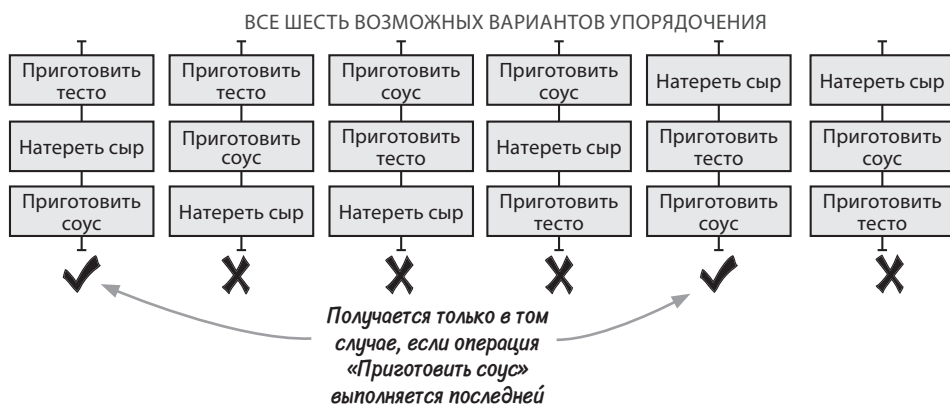
На временной диаграмме разные временные линии по умолчанию не координировались. На диаграмме нет ничего, что бы приказывало текущей временной линии ожидать завершения другой временной линии, поэтому ожидание отсутствует. Действия разных временных линий также могут выполняться с нарушением предполагаемого порядка. Например, тесто может быть готово позже соуса. В таком случае робот, занимающийся соусом, начнет раскатывать тесто до того, как оно будет готово.



Возможен и другой вариант: последним будет готов натертый сыр. Робот, занимающийся соусом, начнет распределять сыр по основе еще до того, как он будет готов.



Собственно, существует шесть способов чередования подготовительных операций, и соус готовится последним только в двух из них. Только в том случае, когда соус готовится в конце, получится нормальная пицца.



Неопровержимый факт: если в распределенных системах временные линии не скоординированы, действия выполняются в неожиданном порядке. Тони должна координировать роботов, чтобы они не начинали собирать пиццу, пока не будут готовы все три ингредиента.

Особенности распределенных систем: урок, полученный дорогой ценой

Тони проводит работу над ошибками

Тони на собственном горьком опыте узнает, что перейти от последовательной программы к распределенной системе непросто. Ей придется сосредоточиться на действиях (операциях, которые зависят от времени) и позаботиться о том, чтобы они выполнялись в правильном порядке.

1. Временные линии не координируются по умолчанию.

Тесто может быть еще не готово, а работы по другим временным линиям просто продолжают выполняться. Они должны быть скоординированы.



2. На продолжительность действий полагаться нельзя.

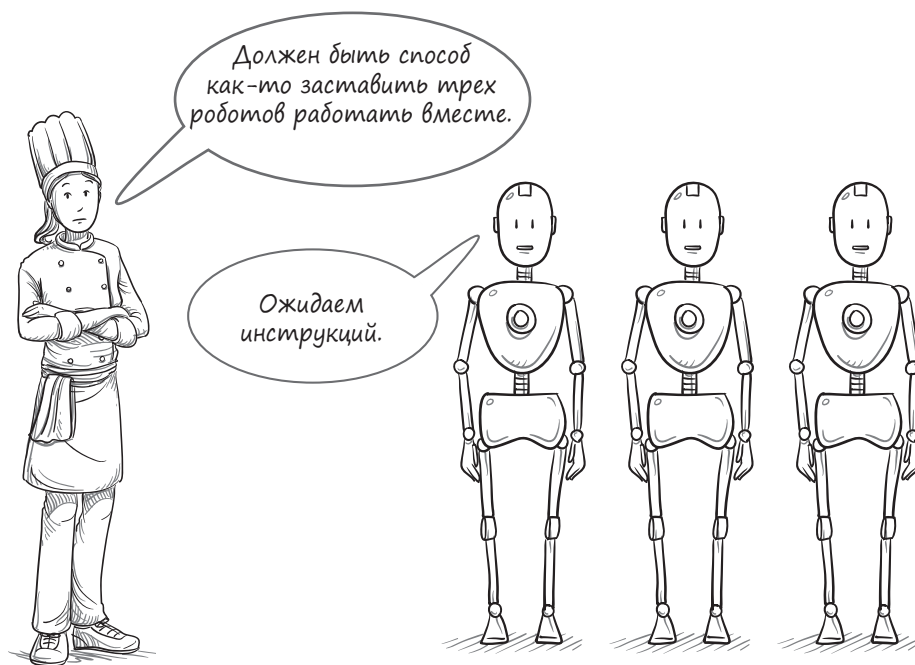
Тот факт, что приготовление соуса обычно занимает больше всего времени, не означает, что так происходит *всегда*. Временные линии не должны зависеть от продолжительности действий.

3. Нарушения хронометража редки, но они возможны в рабочей обстановке.

В тестах все работало нормально, но когда наступает час пик, маловероятные события становятся обыденными. Временные линии должны каждый раз обеспечивать хороший результат.

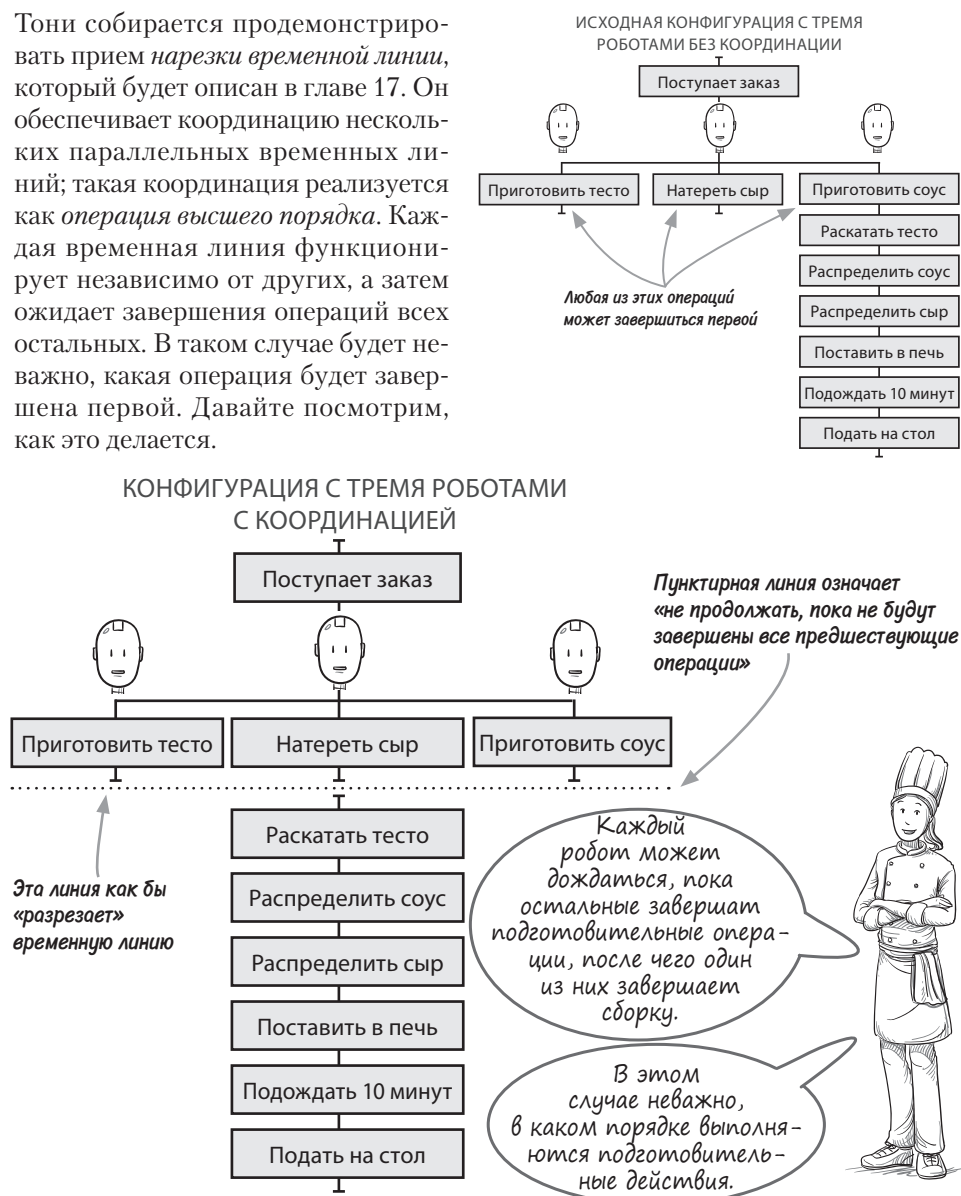
4. Временные диаграммы выявляют проблемы в системе.

Из диаграммы должно быть видно, что сыр может быть не готов вовремя. Используйте временную диаграмму для того, чтобы понять систему.



Сегментация временной линии: заставляем роботов ожидать друг друга

Тони собирается продемонстрировать прием *нарезки временной линии*, который будет описан в главе 17. Он обеспечивает координацию нескольких параллельных временных линий; такая координация реализуется как *операция высшего порядка*. Каждая временная линия функционирует независимо от других, а затем ожидает завершения операций всех остальных. В таком случае будет неважно, какая операция будет завершена первой. Давайте посмотрим, как это делается.



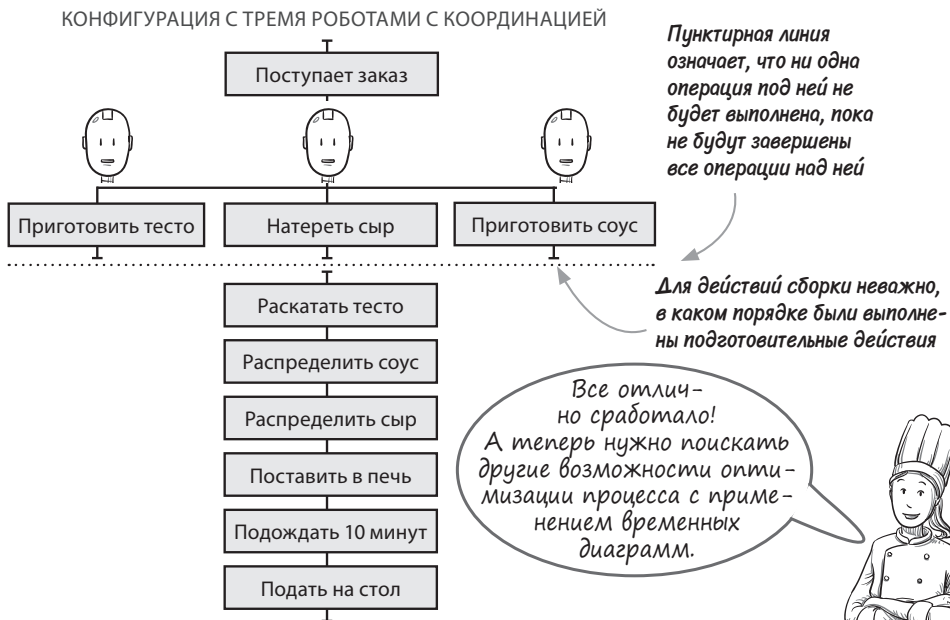
Будем называть эту операцию *нарезкой временной линии*. Вы научитесь реализовывать ее в главе 17.

Тони использует ее в ресторане на следующий вечер, и это приводит к потрясающему результату.

Положительные уроки

Координация роботов в ретроспективе

Система с тремя роботами сработала идеально. Пицца готовилась за рекордное время и идеально соответствовала рецепту. Метод нарезки временных линий гарантировал, что все действия будут выполняться в правильном порядке.



1. Нарезка временной линии упрощает анализ изолированных частей.

Нарезка позволяет Тони отделить подготовительные операции, которые могут выполняться параллельно, от операций сборки, которые выполняются строго последовательно. Метод нарезки позволяет рассматривать более короткие временные линии, не беспокоясь о порядке операций.

2. Работа с временными линиями помогает понять поведение системы во времени.

Теперь, когда Тони понимает временные линии, она больше доверяет своему рабочему процессу. Временные диаграммы — полезный инструмент для визуализации параллельных и распределенных систем.

3. Временные диаграммы обладают гибкостью.

Тони определила, какой результат она хочет получить, и нарисовала временную линию. После этого ей осталось только найти простой способ закодировать ее. Временные диаграммы предоставляют возможность моделировать координацию между отдельными временными линиями.

Нарезка и другие операции высокого порядка будут рассматриваться в части II. А пока наша экскурсия по миру функционального мышления подошла к концу!

Итоги главы

В этой книге кратко рассмотрены некоторые функциональные идеи. Подробнее мы разберем их позже.

Вы видели, какую пользу принесло функциональное мышление в программном обеспечении для пиццерии. Тони выстроила действия и вычисления в соответствии с многоуровневым проектированием, чтобы свести к минимуму затраты на сопровождение. О том, как это делается, вы узнаете в главах 3–9. Кроме того, Тони смогла масштабировать кухонную систему для нескольких роботов и избежать некоторых неприятных ошибок, связанных с последовательностью выполнения. Вы научитесь рисовать временные диаграммы и работать с ними в главах 15–17.

К сожалению, в пиццерию Тони мы уже не вернемся. Но в книге будут продемонстрированы очень похожие сценарии, и вы освоите точно такие же приемы, которые использовала она.

Резюме

- Действия, вычисления и данные — первая и самая важная классификация кода, используемая функциональными программистами. Необходимо научиться видеть их во всем коде, который вы читаете. Мы начнем применять эту классификацию в главе 3.
- Функциональные программисты используют многоуровневое проектирование с целью упрощения сопровождения кода. Уровни помогают организовывать код по скорости изменений. Процесс построения многоуровневых архитектур подробно описывается в главах 8 и 9.
- Временные диаграммы наглядно представляют выполнение действий во времени. Они помогают разработчику увидеть, где действия могут нарушить работу друг друга. Процесс построения временных линий рассматривается в главе 15.
- В этой главе вы научились применять нарезку временных линий для координации их действий. Координация позволяет гарантировать, что действия будут выполняться в правильном порядке. Очень похожий сценарий нарезки временных диаграмм представлен в главе 17.

Что дальше?

Мы рассмотрели пример применения функционального мышления в практической ситуации. После краткого обзора спустимся на уровень будничных, но жизненно необходимых деталей функционального мышления: обсудим различия между действиями, вычислениями и данными.

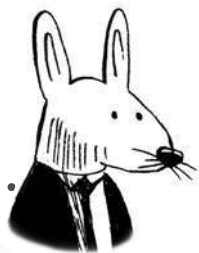
Часть I

Действия, вычисления и данные

В своем путешествии в мир функционального программирования вы освоите много полезных навыков. Но сначала необходимо вооружиться самым фундаментальным навыком — умением различать три категории кода: действия, вычисления и данные. А когда вы освоите эти различия, вы научитесь проводить рефакторинг действий для преобразования их в вычисления, чтобы упростить чтение и тестирование кода. Мы займемся усовершенствованием проектирования действий для того, чтобы повысить уровень их повторного использования. Данные будут преобразовываться в неизменяемые, чтобы на них можно было положиться для протоколирования. Вы также узнаете, как организовать и понимать код на смысловых уровнях. Но сначала необходимо научиться различать вычисления, действия и данные.

3

Действия, вычисления и данные



https://t.me/it_boooks

В этой главе

- ✓ Различия между действиями, вычислениями и данными.
- ✓ Проведение различий между действиями, вычислениями и данными при анализе проблемы, программировании и чтении существующего кода.
- ✓ Отслеживание действий при их распространении в коде.
- ✓ Умение обнаруживать действия в существующем коде.

Категории действий, вычислений и данных уже были кратко охарактеризованы ранее. В этой главе вы научитесь распознавать их в реальной жизни и в коде. Как я уже упоминал, это всего лишь первый шаг функционального программирования. Распознавая эти категории, вы увидите, что вычисления часто упускаются из виду, а действия распространяются в коде, словно заразная болезнь.

Действия, вычисления и данные

Функциональные программисты различают действия, вычисления и данные (actions, calculations, data — ACD).

Действия	Вычисления	Данные
Зависят от того, сколько раз или когда выполняются.	Преобразуют ввод в вывод.	Факты, относящиеся к событиям.
Также называются функциями с побочными эффектами.	Также называются чистыми функциями или математическими функциями	Примеры: адрес электронной почты, предоставленный пользователем; денежная сумма, прочитанная через API банка
Примеры: отправка электронной почты; чтение из базы данных	Примеры: поиск максимального числа в множестве; проверка действительности адреса электронной почты	

Это различие применяется в процессе разработки. Например, функциональные программисты применяют эти концепции в следующих ситуациях.

1. Анализ задачи

Еще до того, как переходить к написанию кода, функциональные программисты стараются разбить задачу на действия, вычисления и данные. Такая классификация помогает выявить части, которым необходимо уделить особое внимание (действия), данные, которые должны храниться в программе, и решения, которые необходимо принять (вычисления).

2. Программирование решения

Во время написания кода функциональный программист отражает три категории в своем коде. Например, данные отделяются от вычислений, которые, в свою очередь, отделяются от действий. Кроме того, мы всегда ищем возмож-



Загляни в словарь

Вычисления обладают *ссылочной прозрачностью*, то есть вызов кода, выполняющего вычисления, может быть заменен его результатом.

Например, $+$ обозначает вычисление; $2 + 3$ всегда дает результат 5, а это значит, что $2 + 3$ всегда можно заменить на 5 с сохранением эквивалентности программы. Кроме того, вычисление $2 + 3$ можно выполнить ноль, один или много раз — каждый раз вы будете получать один и тот же результат.

ности переписать действие в виде вычисления или преобразовать вычисления в данные.

3. Чтение кода

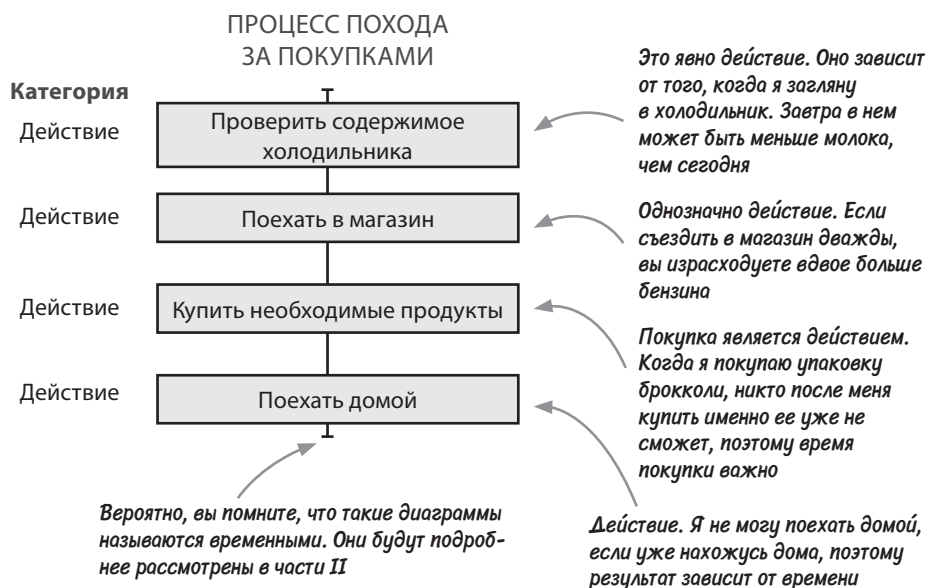
Читая код, мы всегда осознаем, что входит в ту или иную категорию (особенно в категорию действий). Известно, что действия нередко создают проблемы из-за своей зависимости от времени, поэтому мы всегда стараемся найти скрытые действия. В общем случае функциональные программисты всегда ищут возможности для проведения рефакторинга, который бы улучшил разделение действий, вычислений и данных.

В этой главе будут рассмотрены все три категории. Вы также узнаете, как применять их в трех перечисленных ситуациях. Итак, за дело!

Действия, вычисления и данные применимы в любых ситуациях

Представьте ситуацию, с которой мы часто сталкиваемся в жизни: поход в магазин за продуктами.

Если предложить нефункциональному программисту составить схему процесса покупок, она будет выглядеть приблизительно так, как показано ниже. Слева обозначена принадлежность каждой фазы к одной из категорий: действиям, вычислениям или данным.





Джордж
из отдела тестирования

Похоже, в этой схеме чего-то не хватает. Присмотримся к ней подробнее и по-ищем вычисления и данные.

Нельзя согласиться с тем, что все отраженное на схеме является действиями. Такое возможно разве что в самых простых ситуациях, но эта ситуация не настолько проста. Общая схема процесса покупок обрисована неплохо. Рассмотрим каждую фазу и посмотрим, не было ли в ней что-то упущено.

Проверить содержимое холодильника

Проверка содержимого холодильника является действием, потому что результат зависит от того, *когда* она выполняется. Информация об имеющихся продуктах является данными. Назовем эти данные *текущим запасом*.

Данные



Поехать в магазин

Поездка в магазин — сложное действие. Оно безусловно является действием, однако в нем также задействованы некоторые данные, например местонахождение магазина и маршрут до него. Так как мы не строим беспилотный автомобиль, проигнорируем этот шаг.

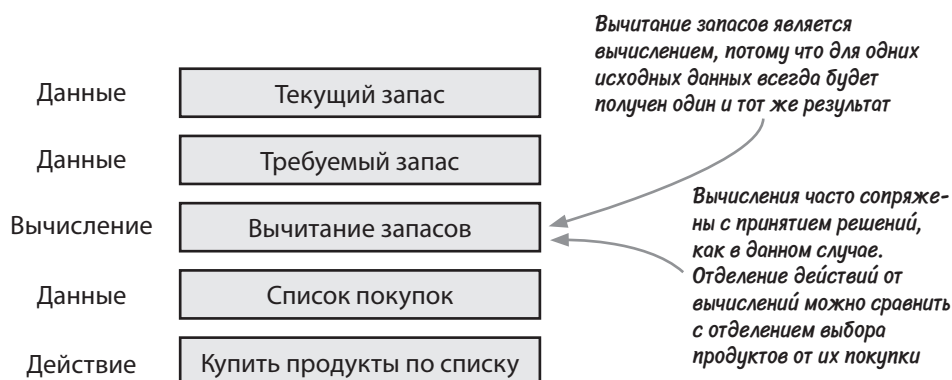


Купить необходимые продукты

Приобретение продуктов определенно является действием, но это действие также можно разбить на меньшие составляющие. Как узнать, что нужно купить? Это зависит от ваших привычек, но проще всего составить список всего, что вам нужно, но чего у вас еще нет.

$$\text{требуемый запас} - \text{текущий запас} = \text{список покупок}$$

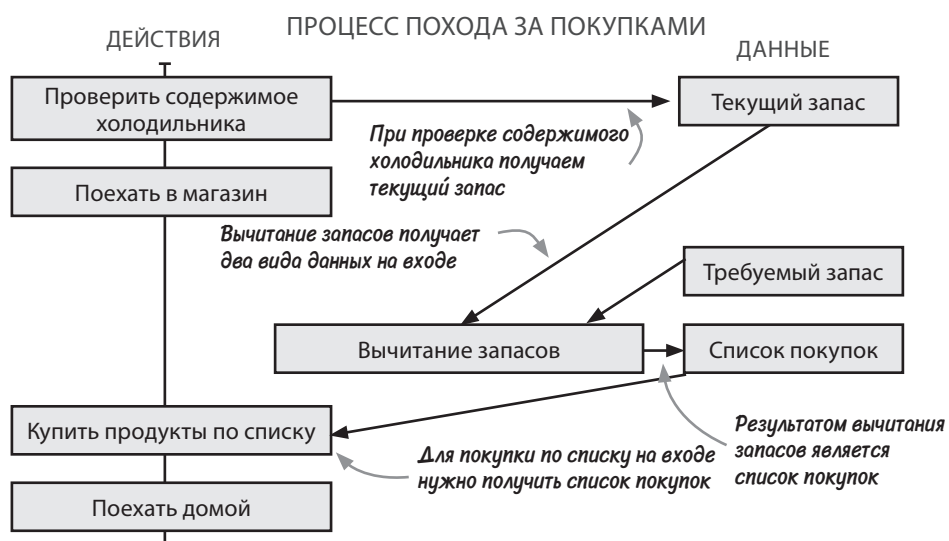
Здесь используются **данные**, сгенерированные на первом шаге: *текущий запас*. Действие «купить необходимые продукты» можно разбить на части:



Поехать домой

Поездку домой тоже можно разбить на составляющие, но это выходит за рамки упражнения.

Перестроим процесс, руководствуясь новыми знаниями.



Итак, мы нарисовали более сложную диаграмму процесса. Действия, вычисления и данные для ясности распределены по столбцам. Стрелки изображают ввод и вывод данных в процессе действий и вычислений.

Конечно, можно пойти еще дальше и продолжить разбиение этих блоков на действия, вычисления и данные. Чем сильнее вы их разделяете, тем богаче ваша модель.

Например, можно дополнительно разделить проверку холодильника на проверку холодильника и морозильной камеры. Каждое действие генерирует отдельный вид данных, которые затем будут объединены. Также можно разбить действие «Купить продукты по списку» на действия «Положить в корзину» и «Оформить покупку».

Конечно, схему можно сделать сколь угодно сложной. Для функциональных программистов важно понимать, что действия могут быть сложными переплетениями действий, вычислений и данных. Не успокаивайтесь, пока не разделите их.

Что мы узнали при моделировании процесса покупки

1. Классификация «действия/вычисления/данные» (ACD) может быть применена в любой ситуации

Поначалу ее бывает трудно рассмотреть, но чем больше вы тренируетесь, тем лучше у вас будет получаться.

2. Действия могут скрывать другие действия, вычисления и данные

То, что кажется простым действием, в действительности может состоять из нескольких частей, относящихся к разным категориям. Один из аспектов функционального программирования как раз и заключается в том, чтобы уметь разбивать действия на меньшие действия, вычисления и данные и знать, когда следует остановиться.

3. Вычисления могут строиться из меньших вычислений и данных

Хороший пример такого рода еще не рассматривался, но данные также могут скрываться в вычислениях. Часто это удобно, но иногда бывает лучше разделить их. Обычно при этом одно вычисление делится на два, а данные, полученные в результате первого вычисления, передаются второму в качестве ввода.

4. Данные могут содержать только другие данные

К счастью, данные — это всего лишь данные. Это одна из причин, по которым мы так активно ищем данные при проектировании. Если у вас имеются данные, то к ним также прилагаются значительные гарантии относительно их поведения.

5. Вычисления часто выполняются «в голове»

Одна из причин, по которым мы воспринимаем вычисления как нечто само собой разумеющееся, в том, что они часто принимают форму мыслительных процессов. Например, вы можете мысленно прикинуть, что нужно купить при посещении магазина. При этом вы не садитесь и не составляете список того, что нужно купить. Все происходит у вас в голове.

Но когда вы это осознаете, выявить вычисления становится легче. Вы спрашиваете себя: «Нужно ли принимать какие-либо решения? Есть ли что-то такое, что можно планировать заранее?» Решения и планирование становятся хорошими кандидатами для вычислений.

Сценарий с покупкой продуктов хорошо демонстрировал применение метода «ДВД» в задаче еще до того, как она была запрограммирована. Но как выглядит применение ДВД к коду, который вы пишете? Чтобы ответить на этот вопрос, необходимо перейти к задаче, связанной с реальным программированием. Но сначала скажу несколько слов о данных.



Глубокое погружение: данные

Что такое данные?

Данные — факты, относящиеся к событиям. По сути, это запись, сохраненная информация о том, что что-то произошло. У функциональных программистов тоже богатая традиция ведения записей, насчитывающая уже тысячи лет.

Как реализуются данные?

В JavaScript данные реализуются с использованием встроенных типов: чисел, строк, массивов, объектов и т. д. В других языках предусмотрены более сложные способы реализации данных. Например, Haskell позволяет определять новые типы данных, отражающие важные аспекты структуры предметной области.

Как в данных кодируется смысловая нагрузка?

Данные кодируют смысловую нагрузку в структурном виде. Структура данных должна отражать структуру предметной области задачи, которую вы реализуете. Например, если порядок имен в списке важен, следует выбрать структуру данных, которая поддерживает этот порядок.

Неизменяемость

Функциональные программисты применяют два основных подхода для реализации неизменяемых данных:

1. **Копирование при записи.** Данные копируются перед их изменением.
2. **Защитное копирование.** Создание копии данных, которые должны остаться в программе.

Эти подходы будут рассмотрены в главах 6 и 7.

Примеры

- Список продуктов, которые нужно купить.
- Ваше имя.
- Мой номер телефона.
- Рецепт блюда.

Какими преимуществами обладают данные?

Полезность данных связана прежде всего с тем, чего они *не могут* сделать. В отличие от действий и вычислений, они не могут выполняться. Данные инертны. Именно благодаря этой особенности они хорошо понятны.

1. **Последовательность обращений.** У вычислений и действий обычно возникают проблемы с последовательностью выполнения, которые удается предотвратить лишь ценой немалых хлопот. С другой стороны, у данных не возникает проблем с передачей по каналу связи или сохранением на диске и последующим чтением. Надежно сохраненные данные могут просуществовать тысячи лет. А ваши данные столько протянут? Трудно сказать. Но они наверняка проживут дольше, чем код функции.
2. **Проверка равенства.** Два фрагмента данных можно проверить на равенство. С вычислениями и действиями это невозможно.
3. **Открытость для интерпретации.** Данные могут интерпретироваться разными способами. Например, журналы обращений к серверу можно анализировать для отладки. С другой стороны, по этим журналам также можно определить, откуда к вам поступает трафик. Обе задачи используют одни и те же данные с разными интерпретациями.

Недостатки

Интерпретация — палка о двух концах. Хотя возможность разной интерпретации данных является преимуществом, сама необходимость интерпретации данных для получения полезной информации рассматривается как недостаток. Вычисление может выполняться и приносить пользу, даже если вы его не понимаете. Данные же не имеют смысла без интерпретации — это просто набор байтов.

.....

Функциональному программисту очень важно уметь представить данные, чтобы их можно было интерпретировать сейчас, а потом заново интерпретировать в будущем.

.....



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Все данные являются фактами, относящимися к событиям? Как насчет фактов, относящихся к человеку или другому субъекту?

О: Очень хороший вопрос. Даже информация о человеке поступает в систему в определенное время. Например, можно сохранить в базе данных имя и фамилию пользователя. Безусловно, это данные. Но откуда они взялись? Если проследить их происхождение, может оказаться, что они были получены как часть веб-запроса «Создать пользователя». Получение веб-запроса является событием. Веб-запрос был обработан и интерпретирован, а некоторые части его были сохранены в базе данных. Итак, имя может *интерпретироваться* как факт, относящийся к человеку, но оно происходит от конкретного события: веб-запроса.

данные, сущ.

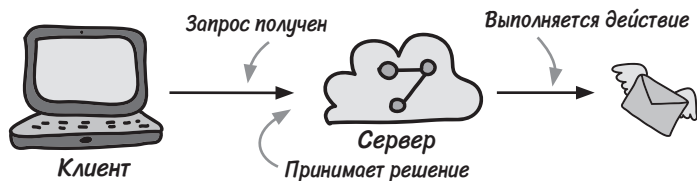
1. Факты, относящиеся к событиям.
2. Фактическая информация, которая становится основой для анализа, обсуждения или вычислений.
3. Информация, полученная с входного устройства, которая становится содержательной в результате обработки.

Определение «факты, относящиеся к событиям» взято прямо из словаря. Конечно, в разных словарях приводятся различные определения. Но это определение идеально подходит для функционального программирования, потому что оно подчеркивает два важных момента. Во-первых, оно подчеркивает необходимость интерпретации. Большинство данных проходит через многочисленные уровни интерпретации: от байтов к символам, разметке JSON и информации о пользователе.

МНОЖЕСТВЕННЫЕ УРОВНИ ИНТЕРПРЕТАЦИИ ВЕБ-ЗАПРОСА



Во-вторых, определение подчеркивает, что программисты строят информационные системы. Наши системы получают и обрабатывают информацию (которая может содержать ошибки), принимают решения на ее основе (например, какие данные следует сохранить и кому отправить сообщение электронной почты), а затем выполняют действие, основанное на этих решениях (фактическая отправка электронной почты).



Применение функционального мышления в новом коде

Новая маркетинговая тактика в CouponDog

Компания CouponDog ведет огромный список людей, интересующихся купонами. Она рассылает по электронной почте еженедельный бюллетень с купонами. Люди обожают купоны!

Для расширения списка директор по маркетингу разработал хитрый план. Если кто-то порекомендует CouponDog десяти своим друзьям, то он получит более выгодные купоны.

Компания создала в базе данных таблицу с адресами электронной почты. Также в базе данных хранится счетчик, показывающий, сколько раз каждый человек порекомендовал CouponDog своим друзьям.

Кроме того, существует отдельная база данных с купонами. Каждый купон помечен одним из трех типов: «плохой» (bad), «хороший» (good) и «лучший» (best). «Лучшие» купоны зарезервированы для людей, которые часто рекомендуют сервис. Всем остальным предоставляются «хорошие» купоны. «Плохие» купоны вообще не рассылаются.



Партнерская программа

Приведи десятерых друзей и получи более выгодные купоны.

Ваша задача — реализовать программу рассылки купонов пользователям. К пятнице успеете?

Директор по маркетингу



Ваш ход

Не бойтесь ответить неправильно. Пока мы только исследуем идеи.

Новый маркетинговый план выглядит просто, но так ли он прост на самом деле? Что наш код должен знать, решать и делать, чтобы он успешно работал? Запишите столько предложений, сколько сможете. Подробные описания не нужны. Также не нужно перечислять их по порядку. Ниже приведены примеры, которые помогут вам начать. Помните: правильных и неправильных ответов здесь нет. Классификация ответов будет приведена на следующей странице.

ТАБЛИЦА АДРЕСОВ
ЭЛЕКТРОННОЙ ПОЧТЫ

email	rec_count
john@coldmail.com	2
sam@pmail.co	16
linda1989@oal.com	1
jan1940@ahoy.com	0
mrbig@pmail.co	25
lol@lol.lol	0

ТАБЛИЦА КУПОНОВ

coupon	rank
MAYDISCOUNT	good
10PERCENT	bad
PROMOTION45	best
IHEARTYOU	bad
GETADEAL	best
ILIKEDISCOUNTS	good

ОБЛАЧНЫЙ
ПОЧТОВЫЙ
СЕРВИС



Партнерская программа

Приведи десятерых друзей и получи более выгодные купоны.

Примеры, которыми
вы можете
руководствоваться

отправка электронной почты

считывание абонентов из базы данных

определение категории каждого купона

Запишите
здесь свои
идеи



Ваш ход

Ниже перечислены некоторые предложения, выдвинутые группой CouponDog. Теперь их необходимо разделить на категории. Поставьте пометку (действие — А, вычисление — С, данные — D) рядом с каждым вариантом, чтобы отнести его к одной из трех основных категорий: действия, вычисления или данные.

- Отправка электронной почты **A** ← *Действие (пример)*
- Чтение информации о подписчиках из базы данных
- Определение категории каждого купона
- Чтение информации о купонах из базы данных
- Тема сообщения
- Адрес электронной почты
- Счетчик рекомендаций
- Принятие решения о том, какое сообщение получает пользователь
- Запись подписчика
- Запись купона
- Список записей купонов
- Список записей подписчиков
- Тело сообщения

*Зависит от того, когда
и сколько раз вызывается*

Три основные категории

- A** Действие
- C** Вычисление
- D** Данные

*Вычисление преобразует
ввод в вывод*



Ваш ход

- A** Отправка электронной почты
- A** Чтение информации о подписчиках из базы данных
- D** Определение категории каждого купона
- A** Чтение информации о купонах из базы данных
- D** Тема сообщения
- D** Адрес электронной почты
- D** Счетчик рекомендаций
- C** Принятие решения о том, какое сообщение получает пользователь
- D** Запись подписчика
- D** Запись купона
- D** Список записей купонов
- D** Список записей подписчиков
- D** Тело сообщения

Если хотите, вернитесь и классифицируйте части, предложенные вами на предыдущей странице.

Наглядное представление процесса рассылки купонов по электронной почте

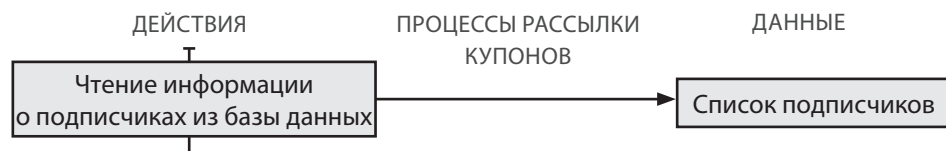
Мы собираемся рассмотреть один из способов наглядного представления этого процесса, хотя, разумеется, существуют и другие варианты реализации того же плана. Основное внимание следует обращать на проведение различий «действия/вычисления/данные» в процессе построения.

email	rec_count
john@coldmail.com	2
sam@pmail.co	16
linda1989@oal.com	1
jan1940@ahoy.com	0
mrbig@pmail.co	25
lol@lol.lol	0

code	rank
MAYDISCOUNT	good
10PERCENT	bad
PROMOTION45	best
IHEARTYOU	bad
GETADEAL	best
ILIKEDISCOUNTS	good

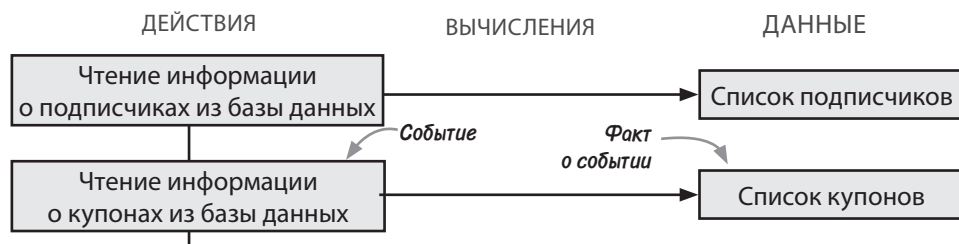
1. Начнем с выборки подписчиков из базы данных

Информацию о подписчиках необходимо загрузить из базы данных — это действие. При выполнении выборки сегодня мы получим не такой набор подписчиков, как тот, который получили бы завтра, так что результат зависит от того, когда выполняется операция. При выборке подписчиков из базы данных мы получаем список записей клиентов. Эти записи являются данными.



2. Загрузка купонов из базы данных

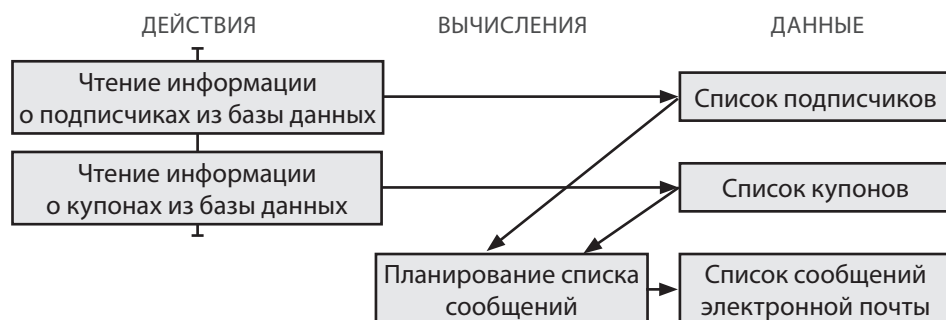
Загрузка купонов также является действием. База данных купонов постоянно изменяется, так что момент выполнения операции важен. Но после того, как купоны будут прочитаны, у вас появляется «снимок» содержимого базы данных на тот момент времени. Эти данные представляют собой факт, полученный в результате события — запроса к базе данных.



Пока все вполне прямолинейно. Имея два основных фрагмента данных из базы данных, можно переходить к принятию решений. Эти данные будут использованы на следующем шаге для принятия решения о том, кто должен получить те или иные сообщения.

3. Генерирование сообщений для отправки

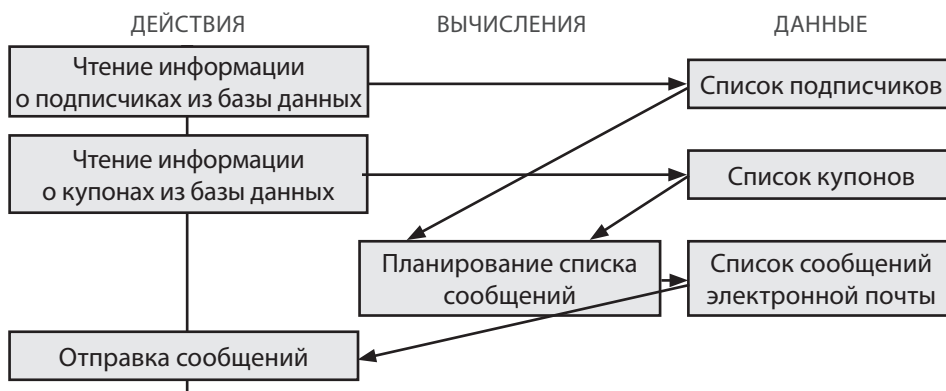
Возможно, вы привыкли к другому, но функциональные программисты часто генерируют необходимые им данные отдельно от их использования. Такой подход можно сравнить с составлением списка покупок до похода в магазин (вместо того, чтобы вспоминать, что же нужно купить, пока вы ходите по магазину).



Результатом генерирования списка сообщений становятся данные, которые могут использоваться на следующем шаге. Список сообщений представляет собой план отправки электронной почты.

4. Отправка электронной почты

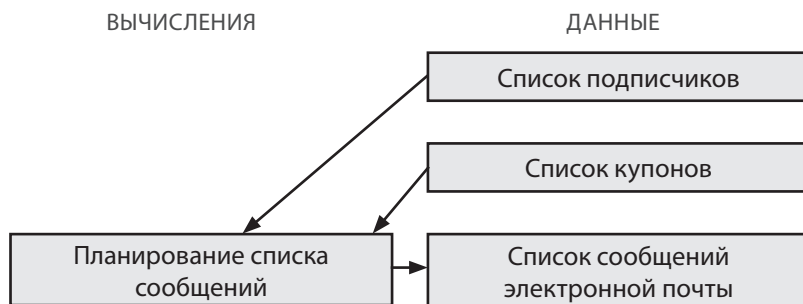
Когда у нас есть план отправки, можно переходить к его исполнению. Процесс сводится к перебору большого списка сообщений и отправке каждого из них. К этому моменту все решения уже были приняты.



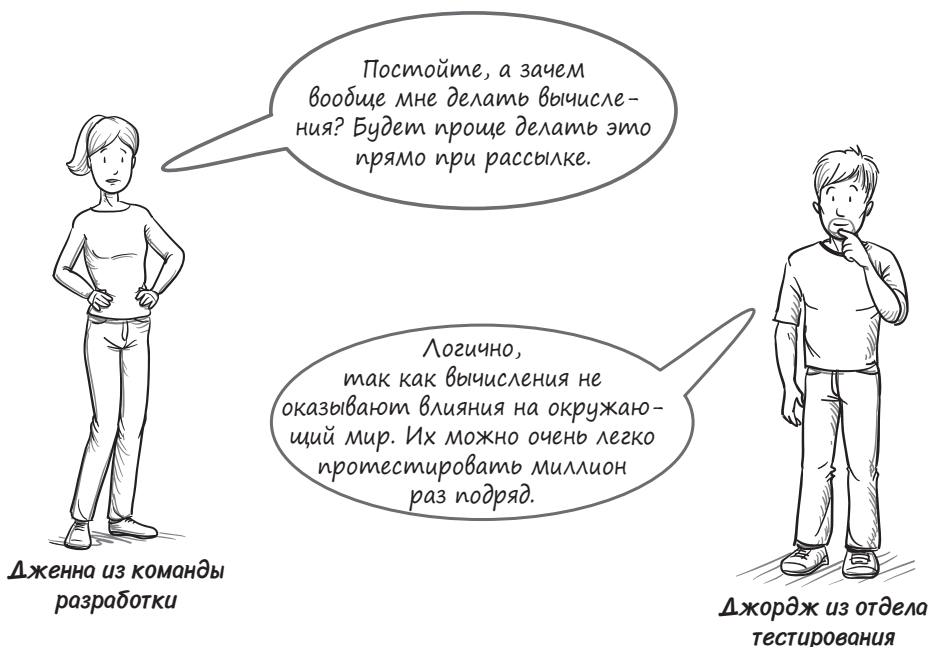
К этому моменту мы разобрались с общей структурой процесса, но как планировать список всех отправляемых сообщений?

Подробнее о генерировании электронной почты

Планирование всех сообщений, которые вы собираетесь отправить, перед их непосредственной отправкой может показаться противоестественным, но в функциональном программировании такая практика считается вполне обычной. Разберем это вычисление подробнее и посмотрим, как оно строится из меньших вычислений. Схема, которую вы уже видели ранее, выглядит так:



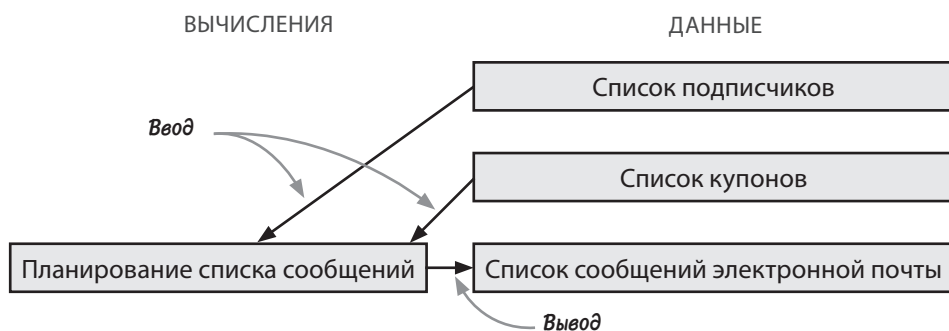
В процессе вычисления получаем два списка: список подписчиков и список купонов. Оно возвращает список сообщений.



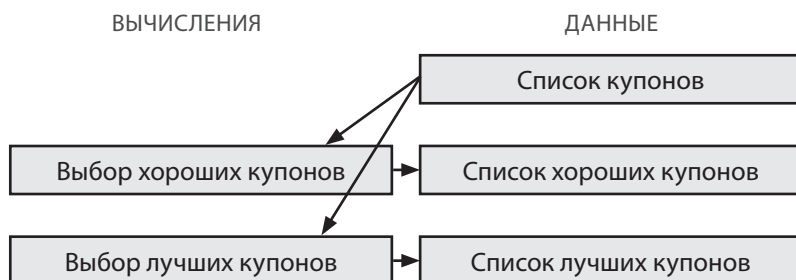
Хороший вопрос. Как правило, функциональные программисты стараются избегать действий, если это возможно, и заменять их вычислениями.

Одна из причин заключается в том, что это упрощает тестирование. Очень трудно тестировать систему, результатом работы которой является отправка электронной почты. Намного проще тестировать систему, выводящую список данных. Эта тема будет более подробно рассмотрена в нескольких ближайших главах.

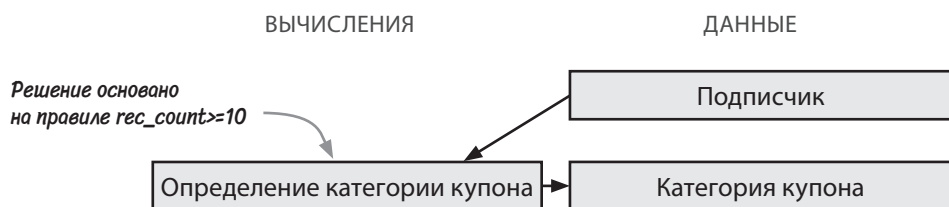
Так о чем я? Ах да. Давайте посмотрим, как реализовать наше вычисление из меньших вычислений.



Начать можно с вычисления списков «хороших» и «лучших» купонов.



Тогда мы можем создать вычисление, которое решит, какой купон получит подписчик: хороший или лучший.



Теперь можно объединить их и создать вычисление, которое планирует отправку одного сообщения для заданного подписчика.

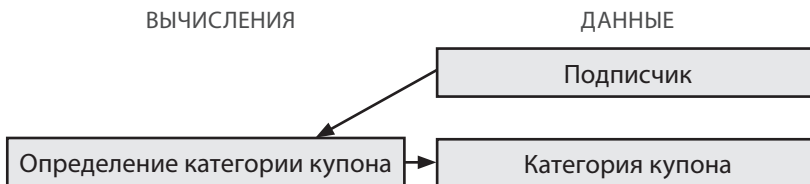


Чтобы запланировать список сообщений, достаточно перебрать список подписчиков и запланировать по одному сообщению для каждого. Запланированные сообщения сохраняются в списке, после чего полученный список возвращается.

Вычисления можно разбивать и дальше, насколько потребуется. Чем дальше проводится разбиение, тем проще реализуются части. В какой-то момент они настолько упрощаются, что их реализация становится очевидной. Давайте реализуем одну из таких частей.

Реализация процесса отправки купонов

Начнем с реализации трех блоков: одного вычисления и двух фрагментов данных, представленных в этой части диаграммы.



Информация о подписке из базы данных

Мы знаем, что информация о подписке читается из таблицы (вроде той, которая изображена справа). В JavaScript такие данные могут быть представлены простым объектом JavaScript. Он будет выглядеть примерно так:

```
var subscriber = {
  email: "sam@pmail.com",
  rec_count: 16
};
```

*Каждая строка
данных преобразуется в объект*

ТАБЛИЦА АДРЕСОВ
ЭЛЕКТРОННОЙ ПОЧТЫ

email	rec_count
john@coldmail.com	2
sam@pmail.co	16
linda1989@oal.com	1
jan1940@ahoy.com	0
mrbig@pmail.co	25
lol@lol.lol	0

В функциональном программировании данные представляются простыми типами данных. Такое представление понятно и подойдет для наших целей.

Категория купона представлена строкой

Ранее мы решили, что категория купона должна описываться строкой. На самом деле это может быть любой тип, но строковое представление удобно. Оно соответствует значениям в столбце rank таблицы купонов.

```
var rank1 = "best";
var rank2 = "good";
```

*Категории представлены
строками*

ТАБЛИЦА КУПОНОВ

code	rank
MAYDISCOUNT	good
10PERCENT	bad
PROMOTION45	best
IHEARTYOU	bad
GETADEAL	best
ILIKEDISCOUNTS	good

Определение категории купона реализуется функцией

В JavaScript вычисления обычно представляются функциями. Вводом для функций являются аргументы, а выводом — возвращаемое значение. Вычисление представляется кодом, содержащимся в теле функции.

```
function subCouponRank(subscriber) {
  if(subscriber.rec_count >= 10)
    return "best";
  else
    return "good";
}
```

Ввод

Вычисление

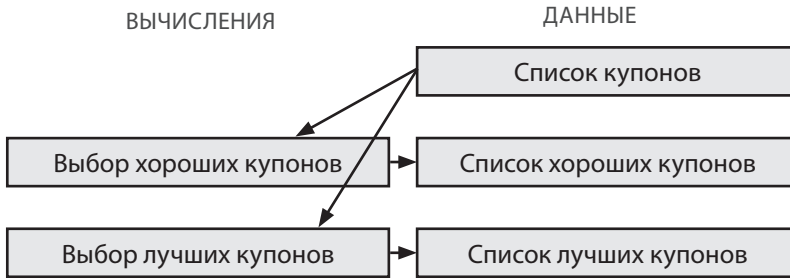
Вывод

Решение относительно присвоения категории сообщению, которое должен получить подписчик, реализуется аккуратным пакетом, упрощающим тестирование и повторное использование, то есть функцией.

Помните

Вычисление преобразует ввод в вывод. Оно не зависит от того, когда или сколько раз оно было выполнено. Для одного ввода всегда будет генерироваться одинаковый вывод.

Реализуем еще одну часть диаграммы: выбор купонов указанной категории из большого списка купонов.



Информация о купонах из базы данных

Для представления купона можно воспользоваться объектом JavaScript (по аналогии с тем, как это делалось с подписчиками):

```
var coupon = {
  code: "10PERCENT",
  rank: "bad"
};
```

*Каждая строка
данных преобразу-
ется в объект*

ТАБЛИЦА КУПОНОВ

code	rank
MAYDISCOUNT	good
10PERCENT	bad
PROMOTION45	best
IHEARTYOU	bad
GETADEAL	best
ILIKEDISCOUNTS	good

Таблица преобразуется в массив JavaScript, содержащий аналогичные объекты.

Вычисление для выбора купонов по категории реализуется функцией

Как и в предыдущем случае, для реализации вычисления будет использоваться функция. Ввод представляет собой список купонов разных категорий, а вывод — список купонов одной категории.

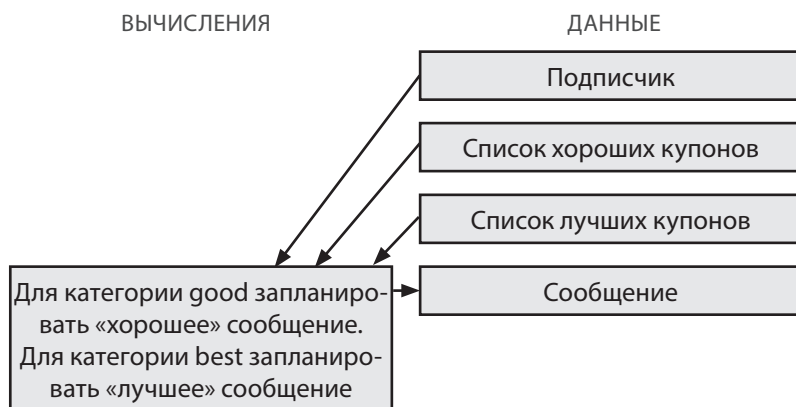
```
function selectCouponsByRank(coupons, rank) {
  var ret = []; // Инициализировать пустой массив
  for(var c = 0; c < coupons.length; c++) {
    var coupon = coupons[c]; // Перебрать все купоны
    if(coupon.rank === rank) // Если купон относится к нужной категории,
      ret.push(coupon.code); // добавьте строку в массив
  }
  return ret; // Вернуть массив
}
```

Ввод (на стрелке к параметру coupons)
Вывод (на стрелке от return ret)

Убедимся в том, что `selectCouponsByRank()` действительно является вычислением. Возвращает ли функция разные ответы, когда ей передаются одинаковые аргументы? Нет. Для одного набора купонов и одной категории каждый раз будет генерироваться одинаковый выходной список. Важно ли, сколько раз

она выполняется? Нет. Ее можно выполнить сколько угодно раз, и это никак не отразится на окружении. Значит, это вычисление.

Осталось реализовать еще одну важную часть диаграммы — ту, в которой планируется отправка отдельного сообщения.

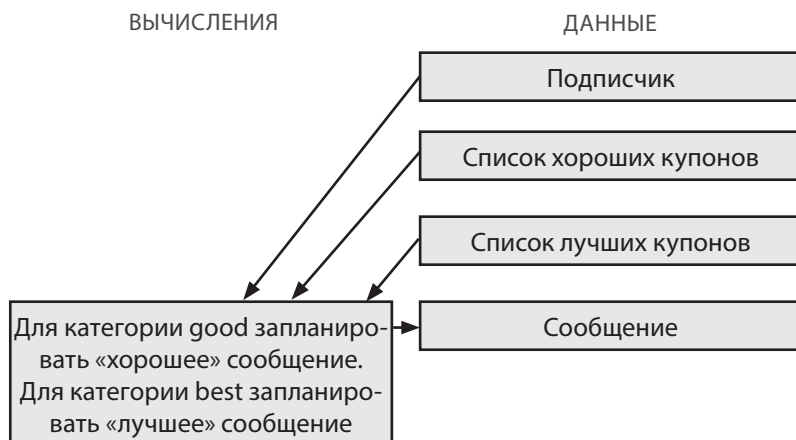


Сообщение — тоже данные

Перед отправкой сообщения как данных также необходимо предусмотреть соответствующее представление. Сообщение состоит из адреса *отправителя*, адреса *получателя*, *темы* и *тела*. Его можно реализовать в виде объекта JavaScript.

```
var message = {
  from: "newsletter@coupondog.co",
  to: "sam@mail.com",
  subject: "Your weekly coupons inside",
  body: "Here are your coupons ..."
};
```

Объект содержит все необходимое для отправки сообщения. Никаких решений принимать не нужно



Вычисление для планирования одного сообщения для подписчика

Как и прежде, для реализации вычисления будет использована функция. На вход должен поступить адрес подписчика для отправки, но менее очевидно то, что для отправки также необходимо иметь информацию о купонах. На данный момент мы еще не решили, какие купоны они получают, поэтому будут передаваться два разных списка: хорошие купоны и лучшие купоны. На выходе мы получаем одно сообщение, представленное в виде данных.

```

function emailForSubscriber(subscriber, goods, bests) {
  var rank = subCouponRank(subscriber);
  if(rank === "best")
    return {
      from: "newsletter@coupondog.co",
      to: subscriber.email,
      subject: "Your best weekly coupons inside",
      body: "Here are the best coupons: " + bests.join(", ")
    };
  else // rank === "good"
    return {
      from: "newsletter@coupondog.co",
      to: subscriber.email,
      subject: "Your good weekly coupons inside",
      body: "Here are the good coupons: " + goods.join(", ")
    };
}
  
```

Обратите внимание: это всего лишь вычисление. Оно только определяет, как должно выглядеть сообщение, и возвращает его в виде данных. Никаких побочных эффектов оно не создает.

Большинство необходимых частей у нас имеется. Посмотрим, как объединить их для организации рассылки.

Планирование всех сообщений

У нас имеется вычисление для планирования одного сообщения для одного подписчика. Теперь потребуется вычисление с целью планирования списка сообщений для списка подписчиков. Для этого можно воспользоваться циклом, как и прежде.

```
function emailsForSubscribers(subscribers, goods, bests) {
  var emails = [];
  for(var s = 0; s < subscribers.length; s++) {
    var subscriber = subscribers[s];
    var email = emailForSubscriber(subscriber, goods, bests);
    emails.push(email);
  }
  return emails;
}
```

В части II представлен более эффективный способ перебора с использованием `map()`

Генерирование всех сообщений сводится к генерированию одного сообщения в цикле

Отправка сообщений является действием

Это действие необходимо реализовать на JavaScript. Как правило, действия реализуются в виде функций, как и вычисления. Из-за этого может быть трудно с первого взгляда определить, что является вычислением, а что — действием. Действиям часто требуются ввод (аргументы) и вывод (возвращаемые значения), поэтому мы воспользуемся функцией для их реализации. Просто будьте внимательны и помните, что это действие.



Ваш ход

К какой категории относится эта функция — действие, вычисление или данные?

Ответ: вычисление. Оно не зависит от того, в каком моменте оно выполняется.

```
function sendIssue() {
  var coupons = fetchCouponsFromDB();
  var goodCoupons = selectCouponsByRank(coupons, "good");
  var bestCoupons = selectCouponsByRank(coupons, "best");
  var subscribers = fetchSubscribersFromDB();
  var emails = emailsForSubscribers(subscribers, goodCoupons, bestCoupons);
  for(var e = 0; e < emails.length; e++) {
    var email = emails[e];
    emailSystem.send(email);
  }
}
```

Действие, которое связывает все вместе

Теперь вся функциональность запрограммирована. Мы начали с самой ограниченной категории (данные), затем добавили вычисления для получения на их основе производных данных и, наконец, объединили все вместе действиями — наименее ограниченной категорией. Мы запрограммировали сначала данные, затем вычисления и, наконец, действия. Этот паттерн часто встречается в функциональном программировании.

Итак, мы написали немного кода, и вы увидели, как выглядит функциональное программирование в применении к чтению существующего кода. Но сначала нужно сказать несколько слов о вычислениях.

Типичный порядок реализации

1. Данные
2. Вычисления
3. Действия



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Почему мы генерируем все сообщения перед отправкой? Разве это эффективно? А если у вас миллионы клиентов?

О: Хороший вопрос. При очень большом количестве клиентов работоспособность системы может быть нарушена из-за нехватки памяти. А может, система будет работать просто отлично! Суть в том, что мы этого не знаем. Заниматься преждевременными оптимизациями неразумно.

Разумеется, нам хотелось бы увеличить количество подписчиков, поэтому следует по крайней мере рассмотреть возможность масштабируемости архитектуры. Если объем данных слишком велик, чтобы размещаться в памяти одновременно, вы все равно сможете использовать почти весь код. `emailsForSubscribers()` получает массив подписчиков. В коде нет никаких требований к тому, чтобы список содержал всех подписчиков. Это может быть небольшой массив подписчиков: допустим, первые 20. Конечно, 20 сообщений легко поместятся в памяти одновременно.

Затем можно перебрать в цикле и обработать всех подписчиков группами по 20. От вас потребуется лишь изменить функцию `fetchSubscribersFromDB()`, чтобы она возвращала группы подписчиков вместо полного набора. Обновленная версия `sendIssue()` выглядит так:

```
function sendIssue() {  
    var coupons = fetchCouponsFromDB();  
    var goodCoupons = selectCouponsByRank(coupons, "good");  
    var bestCoupons = selectCouponsByRank(coupons, "best");  
    var page = 0;  
    var subscribers = fetchSubscribersFromDB(page);  
    while(subscribers.length > 0) {  
        var emails = emailsForSubscribers(subscribers,  
            goodCoupons, bestCoupons);  
        for(var e = 0; e < emails.length; e++) {  
            var email = emails[e];  
            emailSystem.send(email);  
        }  
        page++;  
        subscribers = fetchSubscribersFromDB(page);  
    }  
}
```

Начать со страницы 0

Перебирать, пока не будет получена пустая страница

Перейти к следующей странице

Обратите внимание: сами вычисления не изменились. Мы оптимизировали систему внутри действия. В хорошо написанной системе вычисления определяют вневременные, часто абстрактные идеи вроде «сообщения для этого списка с нулем и более подписчиков». Чтение такой информации из базы данных в память является действием. Действие может просто прочитать меньшее количество записей.



Глубокое погружение: вычисления

Что такое вычисления?

Вычисления преобразуют ввод в вывод. Неважно, когда или сколько раз они выполняются, — для одного ввода всегда будет получен один и тот же вывод.

Как реализуются вычисления?

Обычно вычисления представляются в виде функций. Именно так мы поступаем в JavaScript. В языках без функций пришлось бы использовать что-то другое, например класс с методом.

Как в вычислениях кодируется смысловая нагрузка?

Вычисления представляют некоторые расчеты, преобразующие ввод в вывод. Когда и как их использовать, зависит от того, подходят ли эти вычисления для текущей ситуации.

Почему мы отдаем предпочтение вычислениям перед действиями?

Вычисления обладают рядом преимуществ по сравнению с действиями.

1. Они намного проще в тестировании. Вы можете выполнить их столько раз, сколько потребуется, или когда потребуется (локальная машина, сервер сборки, машина тестирования).
2. Они проще анализируются машинными средствами. В области так называемого статического анализа были проведены значительные научные исследования. В сущности, речь идет об автоматических проверках осмысленности вашего кода. В книге эта тема рассматриваться не будет.
3. Вычисления хорошо подходят для объединения, то есть их можно очень гибко объединять в более крупные вычисления. Также они могут использоваться в так называемых вычислениях более высокого порядка. Мы будем рассматривать их в главе 14.

.....

Важная часть функционального программирования подразумевает выполнение с помощью вычислений того, что в других видах программирования обычно делается с использованием действий.

.....

Примеры вычислений

- Сложение и умножение.
- Конкатенация строк.
- Планирование поездки за покупками.

Каких проблем позволяют избежать вычисления?

Функциональные программисты предпочитают использовать вычисления вместо действий там, где это возможно, потому что вычисления намного понятнее. Вы можете прочитать код и понять, что в нем будет происходить. При этом вам не нужно беспокоиться сразу о нескольких вещах.

1. Что еще выполняется в то же время.
2. Что выполнялось в прошлом и что будет выполняться в будущем.
3. Сколько раз вычисление уже было выполнено.

Недостатки

У вычислений также имеются свои недостатки, общие с действиями. Невозможно точно узнать, что должно произойти в результате вычислений или действий, без их выполнения.

Конечно, вы, программист, можете прочитать код и понять, что он будет делать. Но с точки зрения работающей программы функция представляет собой «черный ящик». Она получает некую входную информацию и выдает на выходе некий результат. С функцией почти ничего нельзя сделать, кроме как выполнить ее.

Если вас решительно не устраивает этот недостаток, придется использовать данные вместо вычислений или действий.

Другие типичные названия

В других публикациях вычисления обычно называются *чистыми функциями* или *математическими функциями*. В книге мы называем их *вычислениями*, чтобы избежать путаницы с конкретными языковыми средствами, такими как функции JavaScript.

Применение функционального мышления в существующем коде

Функциональные программисты также применяют функциональное мышление при чтении существующего кода. Они всегда обращают внимание на то, к какой категории относится та или иная часть кода: действиям, вычислениям или данным.

Рассмотрим часть кода Дженны для перевода оплаты работникам. Действие `sendPayout()` переводит деньги на банковский счет.

Вполне функционально, верно? Здесь только одно действие... не так ли?

```
function figurePayout(affiliate) {
  var owed = affiliate.sales * affiliate.commission;
  if(owed > 100) // Не переводить оплату менее $100
    sendPayout(affiliate.bank_code, owed);
}

function affiliatePayout(affiliates) {
  for(var a = 0; a < affiliates.length; a++)
    figurePayout(affiliates[a]);
}

function main(affiliates) {
  affiliatePayout(affiliates);
}
```

«Одно действие», о котором говорит Дженна



Дженна из команды разработки

Дженна ошибается. Этот код вряд ли можно назвать функциональным. И в нем выполняется не одно, а несколько действий.

Присмотримся к происходящему повнимательнее. Этот пример показывает, как трудно бывает работать с действиями. А вы наконец-то получите представление о тех приемах, которые будут продемонстрированы позднее.

Итак, за дело.

Начнем с единственной строки, которая, как мы знаем, является действием. Затем шаг за шагом мы увидим, как зависимость от времени распространяется по коду. Будьте внимательны!

```
function figurePayout(affiliate) {
  var owed = affiliate.sales * affiliate.commission;
  if(owed > 100) // Не переводить оплату менее $100
    sendPayout(affiliate.bank_code, owed);
}

function affiliatePayout(affiliates) {
  for(var a = 0; a < affiliates.length; a++)
    figurePayout(affiliates[a]);
}

function main(affiliates) {
  affiliatePayout(affiliates);
}
```

1. Начнем с исходной строки, которая является действием. Мы знаем, что это действие, потому что операция перевода денег на счет зависит от того, когда или сколько раз она выполняется. Выделим ее цветом.

```
function figurePayout(affiliate) {
  var owed = affiliate.sales * affiliate.commission;
  if(owed > 100) // Не переводить оплату менее $100
    sendPayout(affiliate.bank_code, owed);
}

function affiliatePayout(affiliates) {
  for(var a = 0; a < affiliates.length; a++)
    figurePayout(affiliates[a]);
}

function main(affiliates) {
  affiliatePayout(affiliates);
}
```

Действие выделено цветом

Вся функция является действием, потому что в ней вызывается действие

Цветом выделена строка, в которой вызывается функция figurePayout(), которая, как уже известно, является действием

2. Действие по определению зависит от того, когда или сколько раз оно выполняется. Но это означает, что функция figurePayout(), которая вызывает sendPayout(), также зависит от того, когда она выполняется. Следовательно, она тоже является действием. Цветом выделяется вся функция и место ее вызова.

```
function figurePayout(affiliate) {
  var owed = affiliate.sales * affiliate.commission;
  if(owed > 100) // Не переводить оплату менее $100
    sendPayout(affiliate.bank_code, owed);
}

function affiliatePayout(affiliates) {
  for(var a = 0; a < affiliates.length; a++)
    figurePayout(affiliates[a]);
}

function main(affiliates) {
  affiliatePayout(affiliates);
}
```

Выделяем всю функцию, потому что она вызывает действие

Вызывается здесь

3. По той же логике также придется выделить полное определение функции affiliatePayout() и все места, в которых она вызывается.

```
function figurePayout(affiliate) {
  var owed = affiliate.sales * affiliate.commission;
  if(owed > 100) // Не переводить оплату менее $100
    sendPayout(affiliate.bank_code, owed);
}

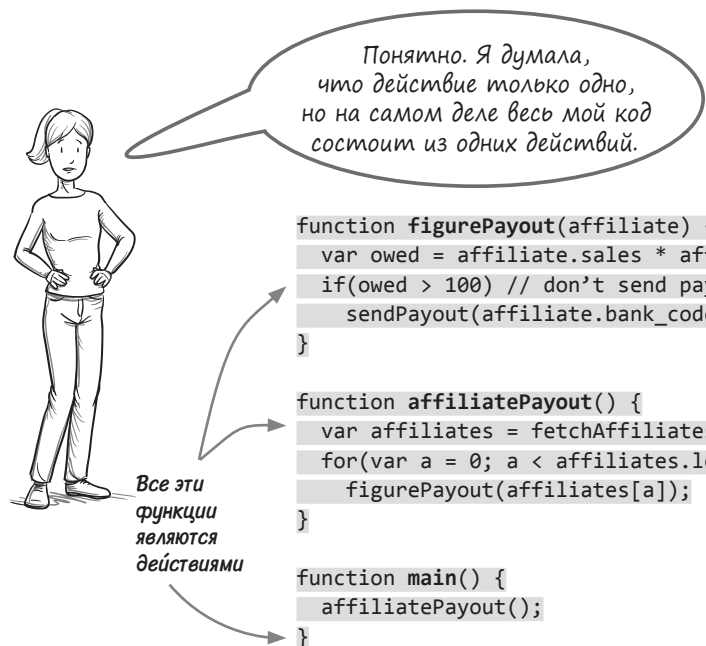
function affiliatePayout(affiliates) {
  for(var a = 0; a < affiliates.length; a++)
    figurePayout(affiliates[a]);
}

function main(affiliates) {
  affiliatePayout(affiliates);
}
```

4. Конечно, по той же логике следует, что `main()` также является действием. Вся программа является действием из-за одного малозаметного вызова действия где-то глубоко в коде.

← Все это действия

Распространение действий в коде



Понятно. Я думала, что действие только одно, но на самом деле весь мой код состоит из одних действий.

```
function figurePayout(affiliate) {
  var owed = affiliate.sales * affiliate.commission;
  if(owed > 100) // don't send payouts less than $100
    sendPayout(affiliate.bank_code, owed);
}

function affiliatePayout() {
  var affiliates = fetchAffiliates();
  for(var a = 0; a < affiliates.length; a++)
    figurePayout(affiliates[a]);
}

function main() {
  affiliatePayout();
}
```

Все эти функции являются действиями

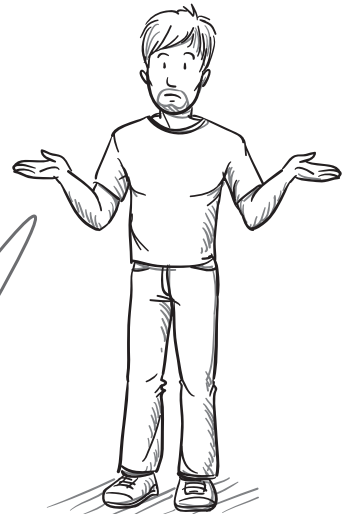
Мы вовсе не собирались придирааться к коду Дженны. Это был пример типичного кода, который не был написан с учетом функционального мышления.

Этот пример демонстрирует одно из свойств действий, из-за которых с ними так трудно работать: действия распространяются. Если вы вызываете действие в функции, эта функция становится действием. Если вызвать эту функцию из другой функции, она становится действием. Все начинается с одного маленького действия, которое постепенно распространяется в коде.

Это одна из причин, по которым функциональные программисты стараются избегать действий, если это возможно. Будьте осторожны, потому что как только вы начинаете пользоваться действиями, они начинают распространяться.

И как использовать действия, если они настолько опасны?

Хороший вопрос. Функциональные программисты используют действия, но обычно они делают это очень осторожно. Осторожность, проявляемая ими при работе с действиями, составляет значительную часть функционального мышления. Этой теме уделяется значительное внимание в нескольких ближайших главах книги.



Джордж из отдела тестирования

Действия могут принимать разные формы

Функциональные программисты различают действия, вычисления и данные, но в большинстве языков такие различия отсутствуют. В таких языках, как JavaScript, очень легко случайно вызвать действия. Как ни прискорбно, это усложняет нашу работу, но функциональные программисты учатся справляться с проблемами. Фокус в том, чтобы увидеть, что они представляют собой на самом деле.

Рассмотрим некоторые действия, которые встречаются в JavaScript. Вероятно, вы уже пользовались ими. Действия могут проявляться в самых разных местах.

Вызовы функций

```
alert("Hello world!");
```

Открытие этого маленького всплывающего окна является действием

**Вызовы методов**

```
console.log("hello");
```

Выводит текст на консоль

Конструкторы

```
new Date()
```

Создает разные значения в зависимости от того, когда вызывается. По умолчанию инициализируется текущей датой и временем

Выражения

Обращение к переменной
`y`

Если `y` является общей изменяемой переменной, чтение может давать разные результаты в разные моменты времени

Обращение к свойству
`user.first_name`

Если `user` является общим изменяемым объектом, чтение `first_name` может каждый раз давать новый результат

Обращение к массиву
`stack[0]`

Если `stack` является общим изменяемым массивом, его первый элемент может быть разным при каждом обращении

Команды

Присваивание
`z = 3;`

Запись в общую изменяемую переменную является действием, потому что она может влиять на другие части кода

Удаление свойства
`delete user.first_name;`

Удаление свойства может влиять на другие части кода, поэтому оно является действием

Все эти фрагменты кода являются действиями. Каждый из них приводит к разным результатам в зависимости от того, когда или сколько раз он выполнялся. А каждый раз, когда они используются, происходит их распространение.

К счастью, нам не придется составлять список всех действий, которых следует остерегаться. Достаточно спросить себя: «Зависит ли результат от того, когда или сколько раз выполняется код?»



Глубокое погружение: действия

Что такое действия?

К действиям относится все, что влияет на окружающий мир или находится под его влиянием. Как правило, действия зависят от того, когда или сколько раз они выполняются.

- Когда выполняются — *упорядочение*.
- Сколько раз выполняются — *повторение*.

Как реализуются действия?

В JavaScript для реализации действий используются функции. К сожалению, одна конструкция используется как для действий, так и для вычислений. Это может создать путаницу. Тем не менее к этому можно привыкнуть.

Как в действиях кодируется смысловая нагрузка?

Смыслом действия является эффект, который оно оказывает на окружение. Вы должны убедиться в том, что эффект будет именно таким, как вам нужно.

Примеры

- Отправка электронной почты.
- Снятие денег со счета.
- Изменение глобальной переменной.
- Отправка запроса AJAX.

Другие типичные названия

В других публикациях действия обычно называются *функциями с побочными эффектами*. В книге мы называем их *действиями*, чтобы избежать путаницы с конкретными языковыми средствами, такими как функции JavaScript.

.....

Действия играют исключительно важную роль в функциональном программировании. В нескольких ближайших главах мы будем учиться обходить ограничения, присущие действиям.

.....

У действий есть недостатки

А. При работе с ними возникает много проблем.

Б. Они являются главной причиной для выполнения программ.

И это довольно серьезные проблемы, если вы хотите знать мое мнение. Но с ними приходится справляться независимо от парадигмы, в которой вы работаете. У функциональных программистов есть свой арсенал приемов для эффективного использования действий. Приведу несколько примеров.

1. Используйте как можно меньше действий. Уменьшить количество действий до нуля никогда не удастся, но если без действия можно обойтись, используйте вместо него вычисление. Эта тема рассматривается в главе 15.
2. Сократите размер своих действий до минимума. Удалите из действий все, что не является абсолютно необходимым. Например, можно выделить из стадии выполнения, в которой выполняется необходимое действие, стадию планирования, реализованную в виде вычисления. Этот прием исследуется в следующей главе.
3. Ограничьте действия взаимодействиями с внешним миром. Ваши действия складываются из всего, что находится под влиянием окружающего мира или может влиять на окружающий мир. В идеале должны остаться только вычисления и данные. Эта тема будет более подробно рассмотрена в главе 18, когда речь пойдет о многослойной архитектуре.
4. Ограничьте зависимость действия от времени. Функциональные программисты разработали методы, несколько упрощающие работу с действиями. К их числу относится сокращение зависимости действий от момента их выполнения и количества их выполнений.

Итоги главы

В этой главе было показано, как три категории — действия, вычисления и данные — применяются на трех разных стадиях. Вы увидели, что вычисления можно рассматривать как планирование или принятие решений. В таком контексте данные представляют план или решение. После этого план выполняется с помощью действия.

Резюме

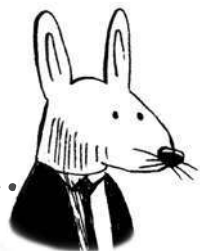
- Функциональные программисты различают три категории: действия, вычисления и данные. Понимание этой особенности станет вашей первой задачей в качестве функционального программиста.
- К действиям относятся операции, которые зависят от того, когда или сколько раз они выполняются. Обычно они влияют на окружение или находятся под его влиянием.
- Вычисления представляют собой расчеты, преобразующие ввод в вывод. Они не влияют ни на что за своими пределами, а следовательно, могут выполняться когда угодно или сколько угодно раз.
- К данным относятся факты, связанные с событиями. Факты регистрируются в неизменяемом виде, потому что они не изменяются.
- Функциональные программисты стараются по возможности вычисления заменять данными, а действия — вычислениями.
- Вычисления проще тестировать, чем действия, потому что вычисления всегда возвращают один и тот же результат для заданного ввода.

Что дальше?

Вы узнали, как распознавать эти три категории в вашем коде. Тем не менее этого недостаточно. Функциональные программисты стремятся преобразовать код из действий в вычисления, чтобы пользоваться преимуществами вычислений. В следующей главе вы узнаете, как это делается.

4

Извлечение вычислений из действий



В этой главе

- ✓ Пути ввода информации в функции и вывода из них.
- ✓ Функциональные методы, улучшающие удобство тестирования кода и его повторного использования.
- ✓ Извлечение вычислений из действий.

В этой главе мы основательно займемся рефакторингом. В ней мы возьмем существующую программу, расширим ее возможности, а затем выделим вычисления из действий посредством рефакторинга. Тем самым будет улучшено удобство тестирования кода и расширены возможности его повторного использования.

Добро пожаловать в MegaMart.com!

Когда покупательская корзина всегда полна

MegaMart — известный интернет-магазин. Одна из его ключевых особенностей, выделяющих его на фоне конкурентов, состоит в том, что в покупательской корзине всегда выводится общая стоимость ее содержимого — даже в процессе выбора покупок.



МegaMart показывает вам свой секретный код

Подписывать расписку о неразглашении не обязательно.

```
var shopping_cart = [];
var shopping_cart_total = 0;

function add_item_to_cart(name, price) {
    shopping_cart.push({
        name: name,
        price: price
    });
    calc_cart_total();
}

function calc_cart_total() {
    shopping_cart_total = 0;
    for(var i = 0; i < shopping_cart.length; i++) {
        var item = shopping_cart[i];
        shopping_cart_total += item.price;
    }
    set_cart_total_dom();
}
```

Глобальные переменные для корзины и ее общей стоимости

Добавить запись в массив cart, чтобы добавить элементы в корзину

Обновить total, потому что содержимое корзины изменилось

Сложить стоимость всех товаров

Обновить DOM для отображения новой суммы



Загляни в словарь

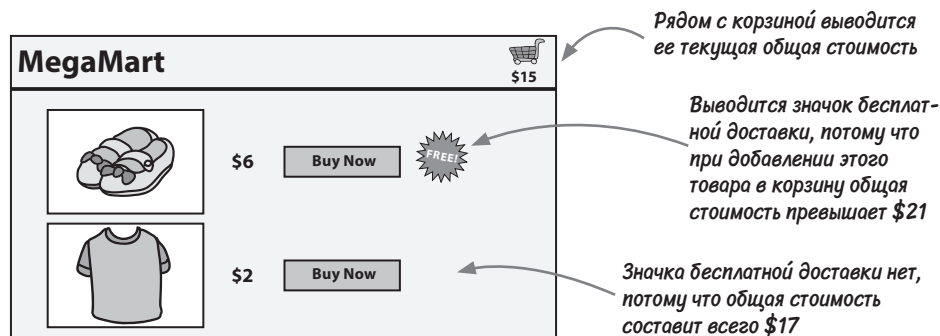
DOM (Document Object Model) — представление HTML-страницы в памяти.

В последней строке обновляется модель DOM (Document Object Model); так веб-программисты изменяют страницу в браузере.

Вычисление бесплатной доставки

Новое поручение

MegaMart хочет предложить бесплатную доставку, если общая стоимость заказа не менее \$20. Если при добавлении элемента в корзину ее общая стоимость превысит \$20, рядом с кнопкой покупки должен появиться специальный значок.



Императивное решение

Иногда императивный подход оказывается наиболее простым.

Можно просто написать функцию, которая добавляет значки ко всем кнопкам. Через несколько страниц мы проведем ее рефакторинг, чтобы сделать ее более функциональной.

```
function update_shipping_icons() {
  var buy_buttons = get_buy_buttons_dom();
  for(var i = 0; i < buy_buttons.length; i++) {
    var button = buy_buttons[i];
    var item = button.item;
    if(item.price + shopping_cart_total >= 20)
      button.show_free_shipping_icon();
    else
      button.hide_free_shipping_icon();
  }
}
```

Получить все кнопки покупки на странице, затем перебрать их

Определить, действует ли бесплатная доставка

Показать или скрыть кнопку в зависимости от результата

Затем новая функция вызывается в конце `calc_cart_total()`, поэтому при каждом изменении общей стоимости обновляются все значки.

```
function calc_cart_total() {
  shopping_cart_total = 0;
  for(var i = 0; i < shopping_cart.length; i++) {
    var item = shopping_cart[i];
    shopping_cart_total += item.price;
  }
  set_cart_total_dom();
  update_shipping_icons();
}
```

Функция, которая приводилась выше

Добавляется строка обновления значков

Девиз группы разработчиков MegaMart

Работает; выпускаем!

Вычисление налога

Следующее поручение

Теперь необходимо вычислить сумму налога и обновлять ее при каждом изменении общей стоимости корзины. И снова необходимый код можно легко добавить к существующей реализации, но функциональные программисты так не поступают.

Напишем новую функцию

```
function update_tax_dom() {
  set_tax_dom(shopping_cart_total * 0.10);
}
```

Вычислить 10 % от общей стоимости

Обновить DOM

Затем, как и прежде, функция вызывается после вычисления общей стоимости корзины в `calc_cart_total()`.

```
function calc_cart_total() {
  shopping_cart_total = 0;
  for(var i = 0; i < shopping_cart.length; i++) {
    var item = shopping_cart[i];
    shopping_cart_total += item.price;
  }
  set_cart_total_dom();
  update_shipping_icons();
  update_tax_dom();
}
```

Добавить строку для обновления суммы налога на странице

Работает;
выпускаем!



Дженна из команды разработки

Необходимо упростить тестирование

Код содержит бизнес-правила, которые нелегко тестировать

При каждом изменении кода Джорджу приходится писать тест, который делает следующее.

1. Настраивает браузер.
2. Загружает страницу.
3. Нажимает кнопки, чтобы добавить элементы в корзину.
4. Ожидает обновления DOM. ← *Должно быть проще!*
5. Извлекает значение из DOM.
6. Преобразует строку в число.
7. Сравнивает значение с ожидаемым.

Из заметок Джорджа по коду

```
function update_tax_dom() {
  set_tax_dom(shopping_cart_total * 0.10);
}
```

Ответ можно получить
только извлечением
данных из DOM

Глобальную переменную необходимо
создать перед тестированием

Бизнес-правило,
которое должен
протестировать
Джордж (сумма * 0,10)

Нельзя ли
как-то упро-
стить тестиро-
вание? Я детей уже
шесть дней
не видел!



Джордж из отдела
тестирования

Рекомендации Джорджа из отдела тестирования

Чтобы упростить тестирование, необходимо сделать следующее.

- Отделить бизнес-правила от обновлений DOM.
- Избавиться от глобальных переменных!

Девиз команды тестирования MegaMart

Стопроцентный
охват тестирования,
или домой не идем.

Эти предложения неплохо соответ-
ствуют канонам функционального
программирования (через несколько
страниц я объясню, чем именно)

Необходимо улучшить возможности повторного использования кода

Бухгалтерия и отдел доставки хотят использовать наш код

Бухгалтерия и отдел доставки хотят использовать наш код, но не могут по нескольким причинам.

- Код читает содержимое корзины из глобальной переменной, но им нужно обрабатывать заказы из базы данных, а не из переменной.
- Код осуществляет запись непосредственно в DOM, а им нужно печатать справки об уплате налогов и этикетки отгрузки.

Бухгалтерия и отдел доставки хотят использовать наш код, но не могут. Сможем ли мы им помочь?



Дженна из команды разработки

Заметки Дженны из команды разработки в коде

```
function update_shipping_icons() {
  var buy_buttons = get_buy_buttons_dom();
  for(var i = 0; i < buy_buttons.length; i++) {
    var button = buy_buttons[i];
    var item = button.item;
    if(item.price + shopping_cart_total >= 20)
      button.show_free_shipping_icon();
    else
      button.hide_free_shipping_icon();
  }
}
```

Бизнес-правило, которое нужно использовать повторно (≥ 20)

Эта функция может выполняться только после инициализации `shopping_cart_total`

Получить ответ на выходе невозможно, потому что отсутствует возвращаемое значение

Эти вызовы будут работать только в том случае, если модель DOM была инициализирована

Дженна о предложениях команды разработки

Чтобы улучшить возможности повторного использования кода, нужно сделать следующее.

- Устранить зависимости от глобальных переменных.
- Не предполагать, что ответ направляется в DOM.
- Возвращать ответ из функции.

Как вскоре будет показано, эти предложения также хорошо соответствуют канонам функционального программирования

Различия между действиями, вычислениями и данными

Первое, на что необходимо взглянуть, — к какой категории относится каждая функция. Это даст нам некоторое представление о коде и о том, как его усовершенствовать. Каждую функцию можно пометить соответствующим маркером: действия (A), вычисления (C) или данные (D) в зависимости от категории.

```

var shopping_cart = []; A
var shopping_cart_total = 0; A

function add_item_to_cart(name, price) { A
  shopping_cart.push({
    name: name,
    price: price
  });
  calc_cart_total();
}

function update_shipping_icons() { A
  var buy_buttons = get_buy_buttons_dom();
  for(var i = 0; i < buy_buttons.length; i++) {
    var button = buy_buttons[i];
    var item = button.item;
    if(item.price + shopping_cart_total >= 20)
      button.show_free_shipping_icon();
    else
      button.hide_free_shipping_icon();
  }
}

function update_tax_dom() { A
  set_tax_dom(shopping_cart_total * 0.10);
}

function calc_cart_total() { A
  shopping_cart_total = 0;
  for(var i = 0; i < shopping_cart.length; i++) {
    var item = shopping_cart[i];
    shopping_cart_total += item.price;
  }
  set_cart_total_dom();
  update_shipping_icons();
  update_tax_dom();
}

```

Эти глобальные переменные являются изменяемыми: действие

Изменяет глобальную переменную: действие

Читает из DOM: действие

Изменяет DOM: действие

Изменяет DOM: действие

Изменяет глобальную переменную: действие

Условные обозначения

A Действие

C Вычисление

D Данные

Помните:

Действия распространяются. Достаточно обнаружить в функции одно действие, чтобы вся функция стала действием.

Весь код состоит из действий. В нем нет ни вычислений, ни данных. Посмотрим, как функциональное программирование может помочь Дженне и Джорджу.

У функций есть ввод и вывод

У каждой функции есть ввод и вывод. К *вводу* относится внешняя информация, которая используется функцией в процессе работы. К *выводу* относится информация или действия, которые являются результатом функции. Функция вызывается ради получения вывода. На вход подается то, что необходимо функции для получения желаемого вывода.

Функция с пометкой ввода и вывода:

```
var total = 0;
function add_to_total(amount) {
  console.log("Old total: " + total);
  total += amount;
  return total;
}
```

Аргументы относятся к *вводу*

Чтение глобальной переменной относится к *вводу*

Вывод на консоль относится к *выводу*

Изменение глобальной переменной относится к *выводу*

Возвращаемое значение относится к *выводу*

Вся суть заключается в отслеживании информации на входе и информации/результатов на выходе.

Ввод и вывод бывают явными и неявными

К *явному* (explicit) вводу относятся аргументы. К явному выводу относится возвращаемое значение. Все остальные возможности для входа или выхода информации из функции являются *неявными*.

```
var total = 0;
function add_to_total(amount) {
  console.log("Old total: " + total);
  total += amount;
  return total;
}
```

Аргументы относятся к *явному вводу*

Чтение глобальной переменной относится к *неявному вводу*

Вывод на консоль относится к *неявному выводу*

Изменение глобальной переменной относится к *неявному выводу*

Возвращаемое значение относится к *явному выводу*

Ввод и вывод

Информация поступает в функцию через входные данные.

Информация и результаты покидают функцию через вывод.

Неявный ввод и вывод превращают функцию в действие

Если исключить из действия весь неявный ввод и вывод, оно становится вычислением. Фокус в том, чтобы заменить неявный ввод аргументами, а неявный вывод — возвращаемыми значениями.



Загляни в словарь

В функциональном программировании неявный ввод и вывод называются *побочными эффектами*. Они не относятся к основному результату функции (получение возвращаемого значения).

Явный ввод

- Аргументы

Неявный ввод

- Весь остальной ввод

Явный вывод

- Возвращаемое значение

Неявный вывод

- Весь остальной вывод

Тестирование и повторное использование связаны с вводом и выводом

Помните Джорджа и Дженну? Они были обеспокоены удобством тестирования и повторного использования кода. Тогда они предоставили рекомендации для улучшения:

Джордж из отдела тестирования

- Отделите бизнес-правила от обновлений DOM.
- Избавьтесь от глобальных переменных!



Дженна из команды разработки

- Устраните зависимость от глобальных переменных.
- Не надейтесь, что ответ попадет в DOM.
- Возвращайте ответ из функции.



Все эти рекомендации относятся к исключению неявного ввода и вывода. Рассмотрим эти рекомендации подробнее.

Джордж 1: Отделите бизнес-правила от обновлений DOM

Обновления DOM относятся к выводу, потому что они создают информацию, которая выходит из функции. А так как они не являются частью возвращаемого значения, они считаются неявными. Обновление DOM все равно должно где-то происходить, чтобы пользователь увидел информацию, но Джордж просит нас отделять бизнес-правило от неявного вывода обновлений DOM.

Джордж 2: Избавьтесь от глобальных переменных

Чтение из глобальных переменных относится к неявному вводу, а запись в глобальные переменные — к неявному выводу. Джордж просит нас исключить весь неявный ввод и вывод. Их можно заменить аргументами и возвращаемым значением.

Дженна 1: Устраните зависимость от глобальных переменных

Эта рекомендация совпадает со второй рекомендацией Джорджа. Дженна просит исключить неявный ввод и вывод.

Дженна 2: Не надейтесь, что ответ попадет в DOM

Наша функция осуществляет запись непосредственно в DOM. Как уже говорилось ранее, эта операция является неявным выводом. Неявный вывод можно заменить возвращаемым значением.

Дженна 3: Возвращайте ответ из функции

Дженна предлагает напрямую использовать явный вывод вместо неявного.

Предложения Джорджа и Дженны, направленные на улучшение удобства тестирования и возможностей повторного использования, соответствуют функциональным концепциям явного и неявного ввода и вывода. Посмотрим, как их можно применить для выделения вычислений.

Извлечение вычислений из действий

Разберемся, как происходит извлечение вычислений из действий. Сначала мы изолируем код вычисления, а затем преобразуем его ввод и вывод в аргументы и возвращаемые значения.

Внутри исходной функции существует блок кода, который выполняет работу вычисления общей стоимости. Мы выделим этот код в отдельную функцию, прежде чем изменять его.

Оригинал

```
function calc_cart_total() {
  shopping_cart_total = 0;
  for(var i = 0; i < shopping_cart.length; i++) {
    var item = shopping_cart[i];
    shopping_cart_total += item.price;
  }

  set_cart_total_dom();
  update_shipping_icons();
  update_tax_dom();
}
```

Выделяем
в функцию

После извлечения

```
function calc_cart_total() {
  calc_total();
  set_cart_total_dom();
  update_shipping_icons();
  update_tax_dom();
}
```

Код заменяется вызовом
новой функции

```
function calc_total() {
  shopping_cart_total = 0;
  for(var i = 0; i < shopping_cart.length; i++) {
    var item = shopping_cart[i];
    shopping_cart_total += item.price;
  }
}
```

Мы создаем новую функцию, присваиваем ей имя и копируем код в ее тело. Там, где раньше находился код, стоит вызов новой функции. На данный момент новая функция является действием. Мы продолжим работать над новой функцией, чтобы преобразовать ее в вычисление.

Только что проделанный рефакторинг можно назвать *выделением подпрограммы*. В результате выполнения этой операции код работает точно так же, как и прежде.



Пища для ума

После операции, выполненной на шаге 2, код работает так же, как прежде. Такие операции называются *рефакторингом*. Мы добиваемся того, чтобы код изменялся без повреждения. Почему полезно менять код, сохраняя его работоспособность?

Новую функцию необходимо преобразовать в вычисление. Для этого необходимо идентифицировать ее ввод и вывод.

Функция имеет два выхода и один вход. Вывод в обоих случаях осуществляет запись в глобальную переменную `shopping_cart_total`. Ввод является чтением из глобальной переменной `shopping_cart`.

Ввод и вывод необходимо преобразовать из неявного в явный.

```
function calc_total() {
  shopping_cart_total = 0;
  for(var i = 0; i < shopping_cart.length; i++) {
    var item = shopping_cart[i];
    shopping_cart_total += item.price;
  }
}
```

Вывод (на `shopping_cart_total = 0;`)

Ввод (на `shopping_cart[i]`)

Вывод (на `shopping_cart_total += item.price;`)

Присваивание глобальной переменной относится к *выводу*, потому что данные покидают функцию.

Чтение глобальной переменной относится к *вводу*, потому что данные входят в функцию.

Оба вывода были записаны в одну и ту же глобальную переменную. Их можно заменить одним возвращаемым значением. Вместо записи в глобальную переменную будет выполняться запись в локальную переменную, которая затем возвращается функцией. Затем значение записывается в глобальную переменную в исходной функции, для чего используется возвращаемое значение.

Текущая версия

```
function calc_cart_total() {
  calc_total();
  set_cart_total_dom();
  update_shipping_icons();
  update_tax_dom();
}
```

```
function calc_total(){
  shopping_cart_total = 0;
  for(var i = 0; i < shopping_cart.length; i++) {
    var item = shopping_cart[i];
    shopping_cart_total += item.price;
  }
}
```

Присваивание перемещается на сторону вызова (на `calc_total();`)

Работаем с локальной переменной (на `shopping_cart_total`)

Версия с исключением вывода

```
function calc_cart_total() {
  shopping_cart_total = calc_total();
  set_cart_total_dom();
  update_shipping_icons();
  update_tax_dom();
}
```

Возвращаемое значение используется для присваивания глобальной переменной (на `shopping_cart_total = calc_total();`)

```
function calc_total() {
  var total = 0;
  for(var i = 0; i < shopping_cart.length; i++) {
    var item = shopping_cart[i];
    total += item.price;
  }
  return total;
}
```

Преобразуется в локальную переменную (на `var total = 0;`)

Возвращается локальная переменная (на `return total;`)



Пища для ума

Мы только что внесли значительное изменение. Будет ли код работать на этой стадии?

Мы избавились от двух эффектов неявного вывода. Перейдем к неявному вводу.

Неявный вывод уже исключен. Осталось преобразовать неявный ввод в аргумент. Мы добавим аргумент `cart` и используем его в функции.

Аргумент необходимо добавить в вызов функции.

Текущая версия

```
function calc_cart_total() {
  shopping_cart_total = calc_total();
  set_cart_total_dom();
  update_shipping_icons();
  update_tax_dom();
}
```

```
function calc_total() {
  var total = 0;
  for(var i = 0; i < shopping_cart.length; i++) {
    var item = shopping_cart[i];
    total += item.price;
  }
  return total;
}
```

Используется для чтения в двух местах

shopping_cart передается в аргументе

Версия с исключением вывода

```
function calc_cart_total() {
  shopping_cart_total = calc_total(shopping_cart);
  set_cart_total_dom();
  update_shipping_icons();
  update_tax_dom();
}
```

```
function calc_total(cart) {
  var total = 0;
  for(var i = 0; i < cart.length; i++) {
    var item = cart[i];
    total += item.price;
  }
  return total;
}
```

Добавляем аргумент и используем его вместо глобальной переменной

На данный момент `calc_total()` является вычислением. Ввод и вывод функции ограничиваются аргументами и возвращаемыми значениями.

Таким образом, извлечение вычисления прошло успешно.

Все пожелания Джорджа и Дженны были учтены



Джордж из отдела тестирования

- ✓ Отделите бизнес-правила от обновлений DOM.
- ✓ Избавьтесь от глобальных переменных!

Способ вычисления общей стоимости товаров определенно является бизнес-правилом

calc_total() перестает зависеть от глобальных переменных



Дженна из команды разработки

- ✓ Устраните зависимости от глобальных переменных.
- ✓ Не надейтесь, что ответ попадет в DOM.
- ✓ Возвращайте ответ из функции.

Да, код теперь не читает из глобальных переменных

Не обновляет DOM

Теперь имеет возвращаемое значение

Извлечение другого вычисления из действия

Повторим тот же процесс, на этот раз для `add_item_to_cart()`. Процедура остается прежней: мы выберем подходящий фрагмент, извлечем его, а затем преобразуем ввод и вывод.

Выберем фрагмент кода, изменяющий корзину. Он является хорошим кандидатом для преобразования в вычисление. Выделим его в новую функцию.

Оригинал

```
function add_item_to_cart(name, price) {
  shopping_cart.push({
    name: name,
    price: price
  });
  calc_cart_total();
}
```

Этот код
извлекается
в новую
функцию

После извлечения

```
function add_item_to_cart(name, price) {
  add_item(name, price);
  calc_cart_total();
}
```

Вызов новой функции
вместо старого
фрагмента кода

```
function add_item(name, price) {
  shopping_cart.push({
    name: name,
    price: price
  });
}
```

Мы создаем новую функцию с именем `add_item()` и помещаем в нее фрагмент кода. Функции понадобятся аргументы `name` и `price`. Затем старый код будет заменен вызовом новой функции.

Как говорилось ранее, этот прием рефакторинга также называется *извлечением подпрограммы*. Выделенная функция является действием, потому что она изменяет глобальный массив `shopping_cart`. Преобразуем ее в вычисление.



Пицца для ума

Мы только что извлекли функцию. Изменится ли при этом поведение системы?

Мы извлекли фрагмент кода из `add_item_to_cart()` в новую функцию `add_item()`. Теперь займемся преобразованием новой функции в вычисление. Для этого необходимо обнаружить ее неявные входные и выходные данные.

`add_item()` осуществляет чтение из глобальной переменной, что относится к вводу. Кроме того, функция изменяет массив в этом значении вызовом `push()`, что относится к выводу.

```
function add_item(name, price) {
  shopping_cart.push({
    name: name,
    price: price
  });
}
```

Массив изменяется вызовом .push()

Чтение глобальной переменной shopping_cart

Помните:

Чтение глобальной переменной относится к *вводу*, потому что данные входят в функцию.

Изменение глобального массива относится к *выводу*, потому что данные покидают функцию.

Итак, мы определили неявный ввод и вывод, а теперь преобразуем их в аргументы и возвращаемые значения. Начнем с ввода.

Изначально мы обращались напрямую к глобальной переменной `shopping_cart`. Вместо этого мы добавим аргумент в `add_item()`. Функция будет обращаться к этой переменной вместо глобальной переменной.

Текущая версия

```
function add_item_to_cart(name, price)
{
  add_item(name, price);
  calc_cart_total();
}

function add_item(name, price) {
  shopping_cart.push({
    name: name,
    price: price
  });
}
```

Версия с исключением ввода

```
function add_item_to_cart(name, price)
{
  add_item(shopping_cart, name, price);
  calc_cart_total();
}

function add_item(cart, name, price) {
  cart.push({
    name: name,
    price: price
  });
}
```

Передача глобального значения в аргументе

Обращается к аргументу вместо глобальной переменной

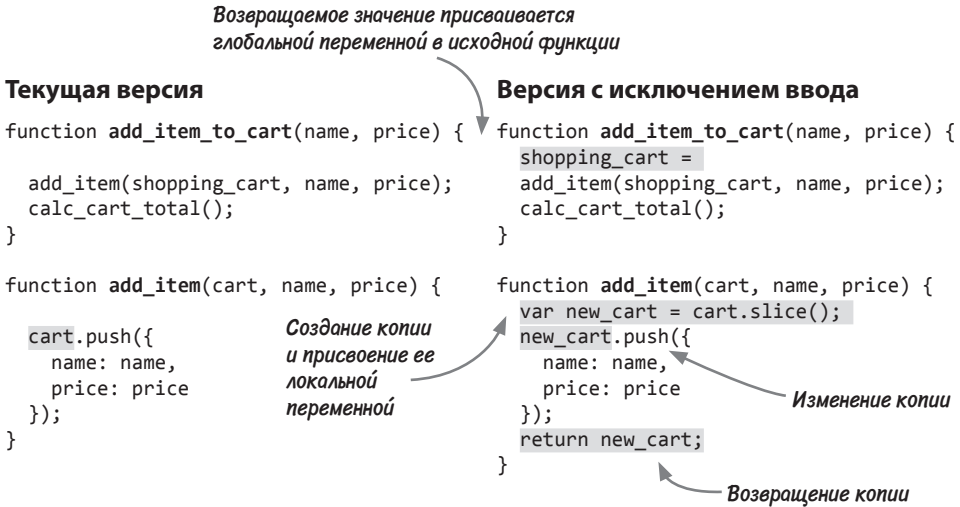
Новый аргумент

Затем необходимо передать глобальную переменную в аргументе. Неявный ввод был преобразован в явный (аргумент).

Но мы все еще изменяем глобальный массив вызовом `.push()`, что является неявным выводом. Рассмотрим эту проблему.

Код имеет один неявный ввод и один неявный вывод. Ввод уже был преобразован в аргумент. Остался только вывод, которым мы сейчас займемся.

Выводом, который мы обнаружили, было изменение массива, хранящегося в `shopping_cart`. Вместо изменения массива должна возвращаться его измененная копия. Посмотрим, как это делается.



Мы создаем копию, изменяем ее, а затем возвращаем копию. В исходной версии функции возвращаемое значение присваивается глобальной переменной. Неявный вывод преобразуется в возвращаемое значение.

На этом извлечение завершается. Функция `add_item()` не имеет неявного ввода или вывода и, следовательно, становится вычислением.



Загляни в словарь

Копирование изменяемого значения перед его изменением — один из способов реализации неизменяемости. Этот прием называется *копированием при записи*. За подробностями обращайтесь к главе 6.



Пицца для ума

Для предотвращения модификации корзины мы создали копию. Останется ли код вычислением, если изменить массив, переданный в аргументе? Почему?



Глубокое погружение

В JavaScript не существует средств прямого копирования массивов. В этой книге будет использоваться метод `.slice()`:

```
array.slice()
```

Подробности приводятся в главе 6.



Ваш ход

Ниже приведен написанный нами код. Мы выделили метод `add_item()`, чтобы упростить его тестирование и повторное использование. Соответствует ли `add_item()` всем рекомендациям Джорджа и Дженны?

```
function add_item_to_cart(name, price) {
  shopping_cart = add_item(shopping_cart, name, price);
  calc_cart_total();
}

function add_item(cart, name, price) {
  var new_cart = cart.slice();
  new_cart.push({
    name: name,
    price: price
  });
  return new_cart;
}
```

Проверьте, выполняются ли все рекомендации Джорджа и Дженны



Джордж из отдела тестирования

- ☐ Отделить бизнес-правила от обновлений DOM.
- ☐ Избавиться от глобальных переменных!



Дженна из команды разработки

- ☐ Устранить зависимости от глобальных переменных.
- ☐ Не рассчитывать, что ответ попадет в DOM.
- ☐ Возвращать ответ из функции.



Ответ

Да! Все рекомендации были выполнены.



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Похоже, объем кода увеличивается. Это нормально? Разве не лучше, когда кода меньше?

О: Как правило, чем меньше кода, тем лучше. Но количество строк в нашем коде постепенно растет.

Мы создаем новые функции, каждая из которых занимает как минимум две строки: для сигнатуры функции и для закрывающей фигурной скобки. Тем не менее разбиение кода на функции окупится со временем.

Причем польза от него начинает проявляться уже сейчас. Код становится более пригодным для повторного использования и тестирования. Два других отдела могут пользоваться готовыми функциями, а тесты становятся короче.

Уже хорошо, но работа еще не закончена! Просто подождите еще немного! :)

В: Удобство тестирования и повторного использования — единственные аспекты, с которыми помогает функциональное программирование?

О: Конечно нет! ФП помогает с ними, но есть и много других. К последним страницам этой книги мы рассмотрим параллелизм, архитектуру и моделирование данных. Кроме того, ФП — большая область, и изложить ее полностью в книге невозможно.

В: Я вижу, вы намеренно делаете некоторые вычисления полезными сами по себе, за пределами сценария использования, для которого они разрабатываются. Это важно?

О: Да, безусловно. Один из приемов, часто применяемых в функциональном программировании, — разделение на составляющие. Меньшие части проще понять, тестировать и повторно использовать.

В: В извлеченных нами вычислениях по-прежнему изменяются переменные. Я слышал, что в функциональном программировании все данные являются неизменяемыми. Как это понимать?

О: Отличный вопрос. Неизменяемость означает, что информация не должна изменяться после создания. Тем не менее в процессе создания она должна инициализироваться, и это требует ее изменения. Например, значения, хранящиеся в массиве, должны быть инициализированы. После этого вы уже не сможете их изменять, но в самом начале в массив могут добавляться элементы.

Мы изменяем локальные переменные или локальные значения только для вновь созданных значений, которые должны быть инициализированы. Они являются локальными, поэтому за пределами функции они не видны. После завершения инициализации мы возвращаем их. Предполагается, что в дальнейшем мы будем соблюдать правила и не станем изменять их. Тема неизменяемости более подробно рассматривается в главе 6.



Шаг за шагом: извлечение вычислений

Извлечение вычисления из действия является повторяемым процессом, который состоит из следующих шагов.

1. Выбор и извлечение кода вычисления.

Выберите подходящий фрагмент кода для извлечения. Проведите рефакторинг этого фрагмента и преобразуйте его в новую функцию. Возможно, придется добавить аргументы. Не забудьте вызвать новую функцию там, где в старой функции располагался выделенный фрагмент.

2. Идентификация неявного ввода и вывода функции.

Определите ввод и вывод новой функции. Ко вводу относится все, что может повлиять на результат между вызовами функции, а к выводу — все, на что может повлиять вызов функции.

Примеры ввода: аргументы, чтение из внешних переменных, запросы к базе данных и т. д.

Примеры вывода: возвращаемое значение, изменение глобальной переменной, изменение совместно используемого объекта, отправка веб-запроса и т. д.

3. Преобразование ввода в аргументы и вывода в возвращаемые значения.

Последовательно преобразуйте ввод в аргументы, а вывод — в возвращаемые значения. Если вы добавляете возвращаемые значения, возможно, это значение придется присвоить локальной переменной в исходной функции.

Важно заметить, что аргументы и возвращаемые значения должны быть неизменяемыми. Если вы вернете значение, а некоторая часть функции позднее изменит его, это станет разновидностью неявного вывода. Аналогичным образом, если что-то изменит значения аргументов после их получения функцией, это станет разновидностью неявного ввода. Неизменяемые данные более подробно рассматриваются в главе 6: в частности, вы узнаете, почему мы используем их и как обеспечивается неизменяемость. А пока будем считать, что мы просто не изменяем данные в коде.



Ваш ход

Бухгалтерия хочет использовать код для вычисления налога, но этот код жестко привязан к обновлению DOM. Выделите вычисление налога из `update_tax_dom()`. Запишите ответ ниже. Ответ приведен на следующей странице.

```
function update_tax_dom() {  
  set_tax_dom(shopping_cart_total * 0.10);  
}
```

*Бухгалтерия хочет
использовать этот код*

Извлечение вычислений

1. Выбор и извлечение кода вычисления.
2. Идентификация неявного ввода и вывода функции.
3. Преобразование ввода в аргументы и вывода в возвращаемые значения.

*На всякий случай напомним,
как это делается*

*Запишите здесь
код ответа*



Ответ

Наша задача — извлечь вычисление налога из `update_tax_dom()`. Начнем с извлечения кода в новую функцию с именем `calc_tax()`.

Оригинал

```
function update_tax_dom() {
  set_tax_dom(shopping_cart_total * 0.10);
}
```

После извлечения

```
function update_tax_dom() {
  set_tax_dom(calc_tax());
}

function calc_tax() {
  return shopping_cart_total * 0.10;
}
```

*Всего один неявный ввод,
неявные выводы отсутствуют*

Небольшое математическое выражение, которое является аргументом `set_tax_dom()`, извлекается в `calc_tax()`. Эта функция имеет всего один неявный ввод и не имеет неявного вывода. Единственный вывод — явное возвращаемое значение.

Заменяем неявный ввод явным аргументом.

После извлечения

```
function update_tax_dom() {
  set_tax_dom(calc_tax());
}

function calc_tax() {
  return shopping_cart_total * 0.10;
}
```

Итоговая версия

```
function update_tax_dom() {
  set_tax_dom(calc_tax(shopping_cart_total));
}

function calc_tax(amount) {
  return amount * 0.10;
}
```

*Удобная автоном-
ная функция для
вычисления налога,
которая может
использоваться
бухгалтерией*

На этом работа закончена. Мы извлекли функцию и заменили весь ее неявный ввод и вывод явным. Теперь она стала вычислением, притом вполне пригодным для повторного использования. Эта функция представляет бизнес-правило.



Ваш ход

Проверьте, выполняются ли все рекомендации Джорджа и Дженны для только что выделенного бизнес-правила `calc_tax()`.

```
function update_tax_dom() {  
  set_tax_dom(calc_tax(shopping_cart_total));  
}  
  
function calc_tax(amount) {  
  return amount * 0.10;  
}
```



Джордж из отдела тестирования

- ☐ Отделить бизнес-правила от обновлений DOM.
- ☐ Избавиться от глобальных переменных!



Дженна из команды разработки

- ☐ Устранить зависимости от глобальных переменных.
- ☐ Не рассчитывать, что ответ попадет в DOM.
- ☐ Возвращать ответ из функции.



Ответ

Да! Все рекомендации были выполнены.



Ваш ход

Отдел доставки хочет использовать ваш код для определения того, какие поставки являются бесплатными. Извлеките вычисление из `update_shipping_icons()`. Запишите ответ ниже. Ответ приведен на следующей странице.

```
function update_shipping_icons() {
  var buy_buttons = get_buy_buttons_dom();
  for(var i = 0; i < buy_buttons.length; i++) {
    var button = buy_buttons[i];
    var item = button.item;
    if(item.price + shopping_cart_total >= 20)
      button.show_free_shipping_icon();
    else
      button.hide_free_shipping_icon();
  }
}
```

Отдел доставки хочет
использовать это правило

Извлечение вычислений

1. Выбор и извлечение кода вычисления.
2. Идентификация неявного ввода и вывода функции.
3. Преобразование ввода в аргументы и вывода в возвращаемые значения.

На всякий случай напомним,
как это делается

Запишите здесь
код ответа



Ответ

Наша задача — выделить логику, которая определяет, действует ли предложение о бесплатной доставке. Начнем с извлечения кода, а затем преобразуем его в вычисление.

Оригинал

```
function update_shipping_icons() {
  var buy_buttons = get_buy_buttons_dom();
  for(var i = 0; i < buy_buttons.length; i++) {
    var button = buy_buttons[i];
    var item = button.item;
    if(item.price + shopping_cart_total >= 20)
      button.show_free_shipping_icon();
    else
      button.hide_free_shipping_icon();
  }
}
```

*Всего один неявный ввод — чтение
из глобальной переменной*

После извлечения

```
function update_shipping_icons() {
  var buy_buttons = get_buy_buttons_dom();
  for(var i = 0; i < buy_buttons.length; i++) {
    var button = buy_buttons[i];
    var item = button.item;
    if(gets_free_shipping(item.price))
      button.show_free_shipping_icon();
    else
      button.hide_free_shipping_icon();
  }
}
```

```
function gets_free_shipping(item_price) {
  return item_price + shopping_cart_total >= 20;
}
```

После извлечения функции `gets_free_shipping()` ее можно преобразовать в вычисление, исключив неявный ввод.

После извлечения

```
function update_shipping_icons() {
  var buy_buttons = get_buy_buttons_dom();
  for(var i = 0; i < buy_buttons.length; i++) {
    var button = buy_buttons[i];
    var item = button.item;
    if(gets_free_shipping(
      item.price))
      button.show_free_shipping_icon();
    else
      button.hide_free_shipping_icon();
  }
}
```

```
function gets_free_shipping(item_price) {
  return item_price + shopping_cart_total >= 20;
}
```

Итоговая версия

```
function update_shipping_icons() {
  var buy_buttons = get_buy_buttons_dom();
  for(var i = 0; i < buy_buttons.length; i++) {
    var button = buy_buttons[i];
    var item = button.item;
    if(gets_free_shipping(shopping_cart_total,
      item.price))
      button.show_free_shipping_icon();
    else
      button.hide_free_shipping_icon();
  }
}
```

```
function gets_free_shipping(total, item_price) {
  return item_price + total >= 20;
}
```



Ваш ход

Проверьте, выполняются ли все рекомендации Джорджа и Дженны для только что извлеченного бизнес-правила `get_free_shipping()`.

```
function update_shipping_icons() {
  var buy_buttons = get_buy_buttons_dom();
  for(var i = 0; i < buy_buttons.length; i++) {
    var button = buy_buttons[i];
    var item = button.item;
    if(gets_free_shipping(shopping_cart_total, item.price))
      button.show_free_shipping_icon();
    else
      button.hide_free_shipping_icon();
  }
}

function gets_free_shipping(total, item_price) {
  return item_price + total >= 20;
}
```



Джордж из отдела тестирования

- ☐ Отделить бизнес-правила от обновлений DOM.
- ☐ Избавиться от глобальных переменных!



Дженна из команды разработки

- ☐ Устранить зависимости от глобальных переменных.
- ☐ Не рассчитывать, что ответ попадет в DOM.
- ☐ Возвращать ответ из функции.



Ответ

Да! Все рекомендации были выполнены.

Весь код в одном месте

Ниже новый код приведен полностью. Каждая функция помечается соответствующим маркером категории действий (A), вычислений (C) или данных (D), чтобы вы получили представление о том, какая часть кода принадлежит той или иной категории.

Условные обозначения

- A** Действие
- C** Вычисление
- D** Данные

```
var shopping_cart = []; A
var shopping_cart_total = 0; A

function add_item_to_cart(name, price) { A
    shopping_cart = add_item(shopping_cart, name, price);
    calc_cart_total();
}

function calc_cart_total() { A
    shopping_cart_total = calc_total(shopping_cart);
    set_cart_total_dom();
    update_shipping_icons();
    update_tax_dom();
}

function update_shipping_icons() { A
    var buttons = get_buy_buttons_dom();
    for(var i = 0; i < buttons.length; i++) {
        var button = buttons[i];
        var item = button.item;
        if(gets_free_shipping(shopping_cart_total, item.price))
            button.show_free_shipping_icon();
        else
            button.hide_free_shipping_icon();
    }
}

function update_tax_dom() { A
    set_tax_dom(calc_tax(shopping_cart_total));
}

function add_item(cart, name, price) { C
    var new_cart = cart.slice();
    new_cart.push({
        name: name,
        price: price
    });
    return new_cart;
}

function calc_total(cart) { C
    var total = 0;
    for(var i = 0; i < cart.length; i++) {
        var item = cart[i];
        total += item.price;
    }
    return total;
}
```

Глобальные переменные: действие

Чтение глобальной переменной: действие

Чтение глобальной переменной: действие

Чтение глобальной переменной: действие

Чтение глобальной переменной: действие

Чтение глобальной переменной: действие

Напомню: это стандартный способ копирования массивов

Неявный ввод или вывод отсутствует

Неявный ввод или вывод отсутствует

Помните:

Достаточно обнаружить в функции одно действие, чтобы вся функция стала действием.

```
function gets_free_shipping(total, item_price) { C
    return item_price + total >= 20;
}
function calc_tax(amount) { C
    return amount * 0.10;
}
```

Неявный ввод или вывод отсутствует

Неявный ввод или вывод отсутствует

Итоги главы

После внесения изменений все счастливы. Джордж из отдела тестирования отправился домой — в эти выходные он наконец-то встретится со своими детьми. Они так быстро растут! А Дженна сообщает, что бухгалтерии и отделу доставки понравился новый код. Они немедленно запустили его в работу без малейших проблем.

И не забудьте про начальство. Оно счастливо, потому что акции MegaMart пошли вверх, и все благодаря новым значкам бесплатной доставки (впрочем, на премию все равно не надейтесь).

Однако у Ким из команды разработки есть кое-какие идеи относительно того, как можно улучшить структуру кода.

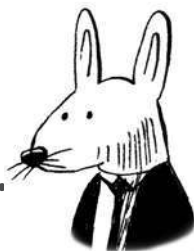


Резюме

- У функций, которые являются действиями, есть неявный ввод и вывод.
- Вычисления не имеют неявного ввода и вывода по определению.
- Общие переменные (например, глобальные) — типичный источник неявного ввода и вывода.
- Неявный ввод часто заменяется аргументами.
- Неявный вывод часто заменяется возвращаемыми значениями.
- При применении функциональных принципов соотношение доли кода в действиях к коду в вычислениях постепенно смещается в пользу вычислений.

Что дальше?

В этой главе вы увидели, что извлечение вычислений из действий способствует повышению качества кода. Однако иногда возможности для выделения вычислений оказываются исчерпанными: что бы вы ни делали, остаются действия. Можно ли улучшить те действия, от которых невозможно избавиться? Да. И этой теме посвящена следующая глава.



В этой главе

- ✓ Улучшение возможностей повторного использования посредством устранения неявного ввода и вывода.
- ✓ Улучшение структуры кода за счет разделения частей.

В предыдущей главе вы узнали, как полное устранение неявного ввода и вывода может преобразовать действия в вычисления. Тем не менее исключить действия полностью невозможно: они все равно должны присутствовать в программе. В этой главе вы увидите, что даже частичное исключение ввода и вывода может улучшить структуру действий.

Согласование структуры с бизнес-требованиями

Выбор уровня абстракции в соответствии с целями

Рефакторинг действий в вычисления, который мы выполняли, был в значительной степени механическим. Такая процедура не всегда приводит к лучшей возможной архитектуре: для этого потребуются участие человека. Давайте посмотрим, как это делается. Собственно, у Ким уже есть идеи по усовершенствованию.

Функция `gets_free_shipping()` оставляет желать лучшего. Идея заключалась в том, чтобы проверить, распространяется ли бесплатная доставка на заказ с текущим содержимым корзины и новым товаром. Но сама функция не проверяет заказ, она проверяет общую стоимость вместе с ценой нового товара. Она получает не те аргументы.

Это не те аргументы, которые нам нужны

```
function gets_free_shipping(total, item_price) {
  return item_price + total >= 20;
}
```

Также в программе присутствует неочевидное дублирование кода. Цена товара прибавляется к общей стоимости в двух разных местах. Дублирование не всегда является чем-то плохим, но это один из признаков «кода с душком», то есть признак потенциальной проблемы.

```
function calc_total(cart) {
  var total = 0;
  for(var i = 0; i < cart.length; i++) {
    var item = cart[i];
    total += item.price;
  }
  return total;
}
```

Мы должны изменить

```
gets_free_shipping(total, item_price)
```

на

```
gets_free_shipping(cart)
```

Это сигнатура функции отвечает на вопрос: «Действует ли для этой корзины бесплатная доставка?»

И также нам нужно избавиться от дублирования кода за счет повторного использования функции `calc_total()`.

Мы хотим знать, действует ли бесплатная доставка для заказа.



Ким из команды разработки

Дублирование вычисления общей стоимости корзины (товар + общая стоимость)



Загляни в словарь

«Код с душком» — характеристика части кода, которая может быть симптомом более глубоких проблем.

Приведение функции в соответствие с бизнес-требованиями

На самом деле такое изменение не является рефакторингом, потому что мы изменяем поведение

Сначала вернемся к нашей цели: функция `gets_free_shipping()` должна получить корзину и вернуть признак того, превышает ли общая стоимость этой корзины \$20.

Определить общую стоимость с помощью нашей удобной функции-вычисления

Оригинал

```
function gets_free_shipping(total,
                             item_price) {
    return item_price + total >= 20;
}
```

С новой сигнатурой

```
function gets_free_shipping(cart) {
    return calc_total(cart) >= 20;
}
```

Теперь наша функция работает со структурой данных корзины (вместо общей стоимости и цены нового товара). И это выглядит логично, потому что в интернет-магазине корзина покупателя является одним из основных объектов.

Из-за изменения сигнатуры необходимо изменить функции, в которых использовалась старая версия.

Оригинал

```
function update_shipping_icons() {
    var buttons = get_buy_buttons_dom();
    for(var i = 0; i < buttons.length; i++) {
        var button = buttons[i];
        var item = button.item;

        Создание новой корзины,
        содержащей товар
        if(gets_free_shipping(
            shopping_cart_total,
            item.price))
            button.show_free_shipping_icon();
        else
            button.hide_free_shipping_icon();
    }
}
```

С новой сигнатурой

```
function update_shipping_icons() {
    var buttons = get_buy_buttons_dom();
    for(var i = 0; i < buttons.length; i++) {
        var button = buttons[i];
        var item = button.item;
        var new_cart = add_item(shopping_cart,
                                item.name,
                                item.price);

        if(gets_free_shipping(
            new_cart))
            button.show_free_shipping_icon();
        else
            button.hide_free_shipping_icon();
    }
}
```

Вызов улучшенной функции

Теперь наша функция делает именно то, что нужно: она сообщает, действует ли бесплатная доставка для содержимого корзины.



Пища для ума

Каковы ваши первые впечатления от этого преобразования? Мы используем вычисление, которое создает измененную копию корзины, но не изменяем существующую корзину. Это самый функциональный код из написанных нами до настоящего момента. Как бы вы охарактеризовали его?



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Строк кода становится все больше! Разве это хороший признак?

О: Количество строк кода — хороший показатель для оценки сложности написания и сопровождения кодовой базы. Тем не менее его недостаточно. Для оценки сложности сопровождения также можно воспользоваться такой характеристикой, как размер каждой функции. Чем меньше функция, тем проще ее понять и написать правильно. Обратите внимание на то, что у нас не так много вычислений. Кроме того, они обладают хорошей связностью и пригодны для повторного использования. В конце концов, работа еще не закончена :)

В: Каждый раз при выполнении `add_item()` создается копия массива с корзиной. Не слишком ли это затратно?

О: И да и нет. Безусловно, такое решение требует больших затрат, чем изменение одного массива. Тем не менее современные исполнительные среды и сборщики мусора очень хорошо справляются с оптимизацией. Собственно, мы постоянно что-нибудь копируем, даже не замечая этого. В JavaScript строки являются неизменяемыми. Каждый раз, когда вы выполняете конкатенацию двух строк, вы создаете новую строку. Все символы исходных строк необходимо скопировать.

Кроме того, преимущества перевешивают затраты. Как будет неоднократно показано в книге, возможность создания измененных копий без изменения оригинала очень полезна. А если эта часть кода будет работать слишком медленно, ее можно будет оптимизировать позднее. Избегайте преждевременной оптимизации. Тема копирования более подробно рассматривается в главах 6 и 7.

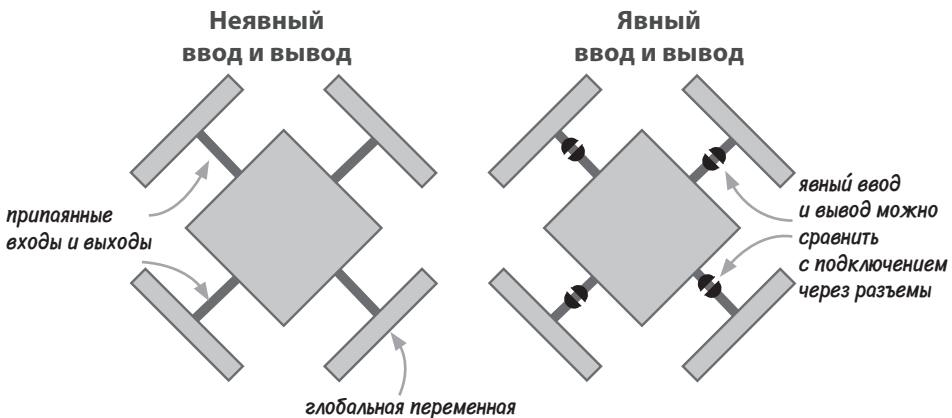


Принцип: минимизация неявного ввода и вывода

К неявному вводу относится весь ввод, кроме аргументов. А к неявному выводу относится весь вывод, кроме возвращаемого значения. Мы писали функции, не имеющие неявного ввода и вывода, и называли их вычислениями.

Тем не менее вычисления не единственное, к чему применяется этот принцип. Даже действия выиграют от исключения неявного ввода и вывода. Если исключить весь неявный ввод и вывод невозможно, чем больше вы исключите, тем лучше.

Функция с неявным вводом и выводом напоминает электронный компонент, припаянный к другим компонентам. Такой компонент не является модульным: его невозможно использовать в другом месте. Его поведение зависит от поведения компонентов, к которым он подключен. Преобразуя неявный ввод и вывод в явный, мы делаем компонент модульным. Вместо паяльника используется разъем, который при необходимости легко отсоединяется.



Неявный ввод ограничивает возможный момент вызова функции. Помните, что налог можно было вычислить только при инициализированной переменной `shopping_cart_total`? А если эту переменную будет использовать кто-то еще? Вам придется убедиться в том, что во время вычисления налога никакой другой код не выполняется.

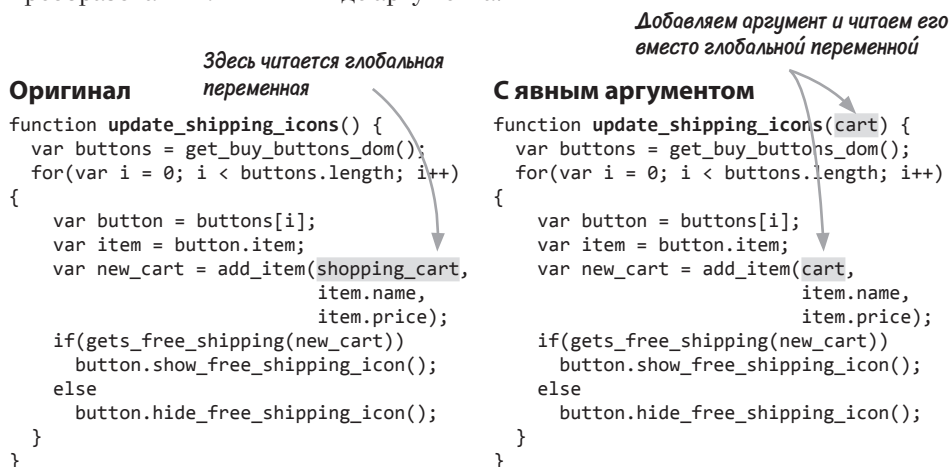
Неявный вывод также ограничивает возможность вызова функции. Функцию можно вызвать только в том случае, если этот вывод вам нужен. А если вы не хотите выполнять запись в DOM в этот момент времени? А если результат вам нужен, но вы хотите поместить его куда-то еще?

Из-за ограничений на возможность вызова функции с неявным вводом и выводом также сложнее в тестировании. Вам придется подготовить весь ввод, выполнить тест, а затем проверить вывод. Чем больше входных и выходных данных в функции, тем сложнее она в тестировании.

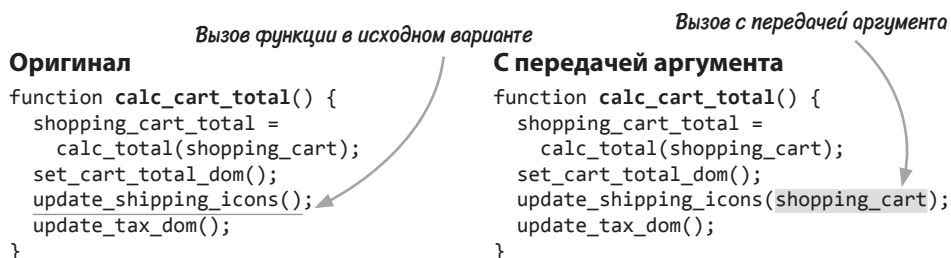
Вычисления тестируются проще всего, потому что у них нет неявного ввода и вывода. Но любая возможность исключения неявного ввода и вывода улучшит удобство тестирования и повторного использования ваших действий, даже если они не переходят в категорию вычислений.

Сокращение неявного ввода и вывода

Каждый принцип должен быть универсальным. Применим принцип минимизации неявного ввода и вывода в `update_shipping_icons()`. Неявный ввод можно преобразовать в явный в виде аргумента.



Сигнатура функции изменилась, нужно обновить сторону вызова. Функция, которая вызывает `update_shipping_icons()`:



Пища для ума

Мы только что применили принцип к функции и исключили неявный ввод. Ранее она была действием, после изменения она также осталась действием. Стала ли функция лучше после применения принципа? Можно ли ее повторно использовать в других обстоятельствах? Упростилось ли ее тестирование?



Ваш ход

Ниже приведен код всех действий, имеющихся на данный момент. Сколько операций чтения из глобальных переменных вы сможете преобразовать в аргументы? Найдите их и преобразуйте в аргументы. Ответ приведен на следующей странице.

```
function add_item_to_cart(name, price) {
    shopping_cart = add_item(shopping_cart,
                             name, price);
    calc_cart_total();
}

function calc_cart_total() {
    shopping_cart_total = calc_total(shopping_cart);
    set_cart_total_dom();
    update_shipping_icons(shopping_cart);
    update_tax_dom();
}

function set_cart_total_dom() {
    ...
    shopping_cart_total
    ...
}

function update_shipping_icons(cart) {
    var buy_buttons = get_buy_buttons_dom();
    for(var i = 0; i < buy_buttons.length; i++) {
        var button = buy_buttons[i];
        var item = button.item;
        var new_cart = add_item(cart, item.name, item.price);
        if(gets_free_shipping(new_cart))
            button.show_free_shipping_icon();
        else
            button.hide_free_shipping_icon();
    }
}

function update_tax_dom() {
    set_tax_dom(calc_tax(shopping_cart_total));
}
```

Код этой функции мы еще не видели, но команда разработки интерфейсной части говорит, что мы можем добавить аргумент



Ответ

В аргументы можно преобразовать многие операции чтения из глобальных переменных:

Оригинал

*Переменная `shopping_cart`
читается только в этих местах*

```
function add_item_to_cart(name, price) {
    shopping_cart = add_item(shopping_cart,
                             name, price);
    calc_cart_total();
}

function calc_cart_total() {
    shopping_cart_total =
        calc_total(shopping_cart);
    set_cart_total_dom();
    update_shipping_icons(shopping_cart);
    update_tax_dom();
} Здесь происходит запись в shopping_cart_total,
но переменная нигде не читается

function set_cart_total_dom() {
    ...
    shopping_cart_total
    ...
}

function update_shipping_icons(cart) {
    var buttons = get_buy_buttons_dom();
    for(var i = 0; i < buttons.length; i++)
    {
        var button = buttons[i];
        var item = button.item;
        var new_cart = add_item(cart,
                                item.name,
                                item.price);
        if(gets_free_shipping(new_cart))
            button.show_free_shipping_icon();
        else
            button.hide_free_shipping_icon();
    }
}

function update_tax_dom() {
    set_tax_dom(calc_tax(shopping_cart
total));
}
```

С исключением чтения глобальных переменных

```
function add_item_to_cart(name, price) {
    shopping_cart = add_item(shopping_cart,
                             name, price);
    calc_cart_total(shopping_cart);
}

function calc_cart_total(cart) {
    var total =
        calc_total(cart);
    set_cart_total_dom(total);
    update_shipping_icons(cart);
    update_tax_dom(total);
    shopping_cart_total = total;
}

function set_cart_total_dom(total) {
    ...
    total
    ...
}

function update_shipping_icons(cart) {
    var buttons = get_buy_buttons_dom();
    for(var i = 0; i < buttons.length; i++)
    {
        var button = buttons[i];
        var item = button.item;
        var new_cart = add_item(cart,
                                item.name,
                                item.price);
        if(gets_free_shipping(new_cart))
            button.show_free_shipping_icon();
        else
            button.hide_free_shipping_icon();
    }
}

function update_tax_dom(total) {
    set_tax_dom(calc_tax(total));
}
```

Проверим код еще раз

Вернемся к коду и посмотрим, что еще можно улучшить. Внимание следует обращать не только на возможности для применения функциональных принципов, но и на такие вещи, как дублирование и избыточные функции.

Все действия:

```
function add_item_to_cart(name, price) {
    shopping_cart = add_item(shopping_cart, name, price);
    calc_cart_total(shopping_cart);
}

function calc_cart_total(cart) {
    var total = calc_total(cart);
    set_cart_total_dom(total);
    update_shipping_icons(cart);
    update_tax_dom(total);
    shopping_cart_total = total;
}

function set_cart_total_dom(total) {
    ...
}

function update_shipping_icons(cart) {
    var buy_buttons = get_buy_buttons_dom();
    for(var i = 0; i < buy_buttons.length; i++) {
        var button = buy_buttons[i];
        var item = button.item;
        var new_cart = add_item(cart, item.name, item.price);
        if(gets_free_shipping(new_cart))
            button.show_free_shipping_icon();
        else
            button.hide_free_shipping_icon();
    }
}

function update_tax_dom(total) {
    set_tax_dom(calc_tax(total));
}
```

Функция, которая будет вызываться при нажатии кнопки "Buy Now"

Эта функция выглядит немного избыточной. Почему бы не проделать все это в add_item_to_cart()?

Эта глобальная переменная записывается, но нигде не читается. От нее можно избавиться

Остальное вроде пока выглядит нормально

Мы выявили два перспективных улучшения: во-первых, глобальная переменная `shopping_cart_total` нигде не читается. Во-вторых, функция `calc_cart_total()` является избыточной. Эти улучшения будут реализованы на следующей странице.

На предыдущей странице были выявлены два улучшения: переменная `shopping_cart_total` не использовалась, а функция `calc_cart_total()` была избыточной. Встроим ее код в функцию `add_item_to_cart()`, в которой она вызывалась.

Оригинал

```
function add_item_to_cart(name, price) {
    shopping_cart = add_item(shopping_cart,
                             name, price);
    calc_cart_total(shopping_cart);
}

function calc_cart_total(cart) {
    var total = calc_total(cart);
    set_cart_total_dom(total);
    update_shipping_icons(cart);
    update_tax_dom(total);
    shopping_cart_total = total;
}
```

После улучшения

```
function add_item_to_cart(name, price) {
    shopping_cart = add_item(shopping_cart,
                             name, price);

    var total = calc_total(shopping_cart);
    set_cart_total_dom(total);
    update_shipping_icons(shopping_cart);
    update_tax_dom(total);
}
```

Избавимся от `calc_cart_total()` и глобальной переменной `shopping_cart_total` и переместим все в `add_item_to_cart()`

Остаток кода не приводится, потому что других изменений не было. Теперь, когда действия были улучшены, вернемся к ним и сгруппируем их по смысловым уровням.



Пища для ума

Похоже, мы наконец-то значительно сократили количество строк в действиях. Почему это заняло столько времени? Можно ли было действовать более прямолинейно?

Классификация наших расчетов

Группировка вычислений дает информацию о смысловых уровнях

Мне хотелось бы исследовать вычисления чуть подробнее. Пометим маркером К (Корзина) каждую функцию, которая знает структуру корзины (то есть она знает, что это массив с данными товаров). Каждая функция, которой известна структура товара, помечается маркером Т. Наконец, функции, относящиеся к бизнес-правилам, будут помечены маркером Б (Бизнес-правила).

Условные обозначения

- К** Операции с корзиной
- Т** Операции с товаром
- Б** Бизнес-правила

```

      К Т
function add_item(cart, name, price) {
  var new_cart = cart.slice();
  new_cart.push({
    name: name,
    price: price
  });
  return new_cart;
}

```

Не забудьте: для копирования массива в JavaScript используется `.slice()`

```

      К Т Б
function calc_total(cart) {
  var total = 0;
  for(var i = 0; i < cart.length; i++) {
    var item = cart[i];
    total += item.price;
  }
  return total;
}

```

Эта функция определенно знает структуру корзины, но она также может считаться бизнес-правилом, так как определяет, как MegaMart вычисляет общую стоимость

```

      Б
function gets_free_shipping(cart) {
  return calc_total(cart) >= 20;
}

```

```

      Б
function calc_tax(amount) {
  return amount * 0.10;
}

```

Со временем группы выделяются более отчетливо. Обращайте внимание на них: они станут смысловыми уровнями в нашем коде. Я решил упомянуть об этом сейчас, раз уж мы занимаемся работой с кодом. В главах 8 и 9 эта тема будет исследована более подробно. А пока просто смотрите. Эти уровни проявляются тогда, когда вы начинаете разбивать код на составляющие. И раз уж речь зашла о разбиении, рассмотрим принцип, посвященный именно этому процессу.



Принцип: суть проектирования в разделении

Функции предоставляют чрезвычайно естественный механизм разделения обязанностей. Функции отделяют значение, передаваемое в аргументе, от его использования. Очень часто у нас возникает искушение объединять: большие, более сложные сущности кажутся более основательными. Однако то, что вы разделили, всегда можно собрать воедино. Трудности связаны скорее с поиском практических способов разъединения.

Простота повторного использования

Меньшие, более простые функции проще использовать заново. Они выполняют меньше работы и делают меньше предположений.

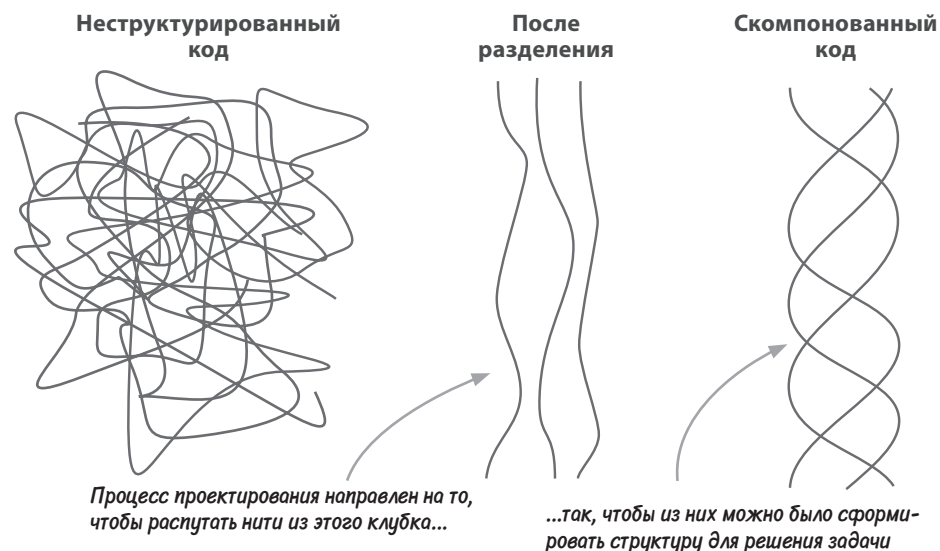
Простота сопровождения

Меньшие функции проще понять, и они создают меньше проблем с сопровождением. Они содержат меньше кода. Часто их правильность (или ошибочность) очевидна.

Простота тестирования

Меньшие функции проще в тестировании. Они делают что-то одно, и вы тестируете эту их единственную обязанность.

Даже когда в функции нет легко идентифицируемых проблем, если вы видите что-то, что можно извлечь, то по крайней мере стоит попробовать это сделать. Возможно, такое разделение приведет к улучшению структуры кода.



Улучшение структуры за счет разделения add_item()

Перед вами наш верный метод `add_item()`. Он делает вроде бы не так много: только добавляет товар в корзину. Но так ли это? Давайте посмотрим повнимательнее. Как выясняется, эта функция решает четыре разные задачи:

```
function add_item(cart, name, price) {
  var new_cart = cart.slice();
  new_cart.push({
    name: name,
    price: price
  });
  return new_cart;
}
```

1. Создание копии массива
2. Построение объекта для товара
3. Добавление товара в копию
4. Возвращение копии

Функция знает как структуру корзины, так и структуру товара. Товар можно выделить в отдельную функцию.

Оригинал

Создание функции-конструктора

```
function add_item(cart, name, price) {
  var new_cart = cart.slice();
  new_cart.push({
    name: name,
    price: price
  });
  return new_cart;
}

add_item(shopping_cart,
  "shoes", 3.45);
```

Изменение кода вызова

После разделения

```
function make_cart_item(name, price) {
  return {
    name: name,
    price: price
  };
}

function add_item(cart, item) {
  var new_cart = cart.slice();
  new_cart.push(item);
  return new_cart;
}

add_item(shopping_cart,
  make_cart_item("shoes", 3.45));
```

2. Создание объекта для товара
1. Создание копии массива
3. Добавление товара в копию
4. Возвращение копии

Выделяя этот код, мы создаем функцию, которая знает структуру товара, но не структуру корзины, а также функцию, которая знает структуру корзины, но не структуру товара. Это означает, что структуры корзины и товара могут эволюционировать независимо. Например, если вам понадобится переключиться с корзины, реализованной в виде массива, на реализацию в виде хеш-таблицы, это можно будет сделать с минимальным количеством изменений.

Что касается пунктов 1, 3 и 4, их желательно держать поблизости. Создание копии перед изменением значения — стратегия реализации неизменяемости, называемая *копированием при записи*, поэтому мы будем держать их вблизи друг от друга. Об этом будет рассказано в главе 6.

Эта функция не ограничивается корзинами и товарами. Изменим имена функции и аргументов на следующей странице.

Выделение паттерна копирования при записи

Мы только что выделили удобную маленькую функцию. Глядя на код, мы видим, что он добавляет элемент в массив с применением подхода копирования при записи. Это обобщенная операция, но ее имя обобщенным не является. Оно связано с покупательскими корзинами.

Конкретные имена



```
function add_item(cart, item) {
  var new_cart = cart.slice();
  new_cart.push(item);
  return new_cart;
}
```

Обобщенная реализация



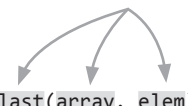
Представьте, что мы заменили имена функции и двух аргументов более общими:

Обобщенные имена, работающие с любым массивом и элементом

**Оригинал
(конкретные имена)**

```
function add_item(cart, item) {
  var new_cart = cart.slice();
  new_cart.push(item);
  return new_cart;
}
```

Обобщенная реализация



```
function add_element_last(array, elem) {
  var new_array = array.slice();
  new_array.push(elem);
  return new_array;
}
```

Тогда `add_item()` можно будет определить очень просто:

```
function add_item(cart, item) {
  return add_element_last(cart, item);
}
```

Мы выделили вспомогательную функцию, очень хорошо подходящую для повторного использования: она может работать с любыми массивами и элементами, а не только с корзинами и товарами. Вероятно, корзина будет не последним массивом, в который вы будете добавлять элементы. Также в какой-то момент возникнет необходимость в использовании неизменяемых массивов. Тема неизменяемости более глубоко рассматривается в главах 6 и 7.

Использование функции add_item()

Функция `add_item()` получала три аргумента: корзину, название и цену товара:

```
function add_item(cart, name, price) {
  var new_cart = cart.slice();
  new_cart.push({
    name: name,
    price: price
  });
  return new_cart;
}
```

Теперь она получает только два аргумента: корзину и товар:

```
function add_item(cart, item) {
  return add_element_last(cart, item);
}
```

Мы выделили отдельную функцию для конструирования товара:

```
function make_cart_item(name, price) {
  return {
    name: name,
    price: price
  };
}
```

Это означает, что функции, вызывающие `add_item()`, необходимо изменить, чтобы они передавали правильное количество аргументов:

Оригинал

```
function add_item_to_cart(name, price) {
  shopping_cart = add_item(shopping_cart,
    name, price);
  var total = calc_total(shopping_cart);
  set_cart_total_dom(total);
  update_shipping_icons(shopping_cart);
  update_tax_dom(total);
}
```

Использование новой версии

```
function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart = add_item(shopping_cart,
    item);
  var total = calc_total(shopping_cart);
  set_cart_total_dom(total);
  update_shipping_icons(shopping_cart);
  update_tax_dom(total);
}
```

Сначала мы строим товар, а затем передаем его `add_item()`. Собственно, это все! Давайте взглянем на вычисления под новым, более широким углом.

Классификация вычислений

Итак, код был изменен, теперь взглянем на вычисления еще раз. Пометим маркером К (Корзина) каждую функцию, которой известна структура корзины (то есть что корзина представляет собой массив с товарами). Пометим маркером Т (Товар) каждую функцию, которой известна структура товара. Функции, относящиеся к бизнес-правилам, будут помечены маркером Б (Бизнес-правила). Наконец, маркером М (Массив) будут помечены вспомогательные функции массивов.

Условные обозначения

К	Операции с корзиной
Т	Операции с товаром
Б	Бизнес-правила
М	Вспомогательные функции массивов

```
function add_element_last(array, elem) {
  var new_array = array.slice();
  new_array.push(elem);
  return new_array;
}
```

М

```
function add_item(cart, item) {
  return add_element_last(cart, item);
}
```

К

```
function make_cart_item(name, price) {
  return {
    name: name,
    price: price
  };
}
```

Т

Мы разделили эти три категории, которые когда-то составляли одну функцию

```
function calc_total(cart) {
  var total = 0;
  for(var i = 0; i < cart.length; i++) {
    var item = cart[i];
    total += item.price;
  }
  return total;
}
```

К

Т

Б

Эта функция очень интересна, потому что связывает воедино три концепции

```
function gets_free_shipping(cart) {
  return calc_total(cart) >= 20;
}
```

Б

```
function calc_tax(amount) {
  return amount * 0.10;
}
```

Б

Эти три функции не изменились



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Напомните, почему мы делим функции на вспомогательные, операции корзины и бизнес-правила?

О: Хороший вопрос. Это всего лишь подготовка для некоторых навыков проектирования, до которых мы доберемся позднее. Со временем эти группы будут разделены на изолированные уровни. Тем не менее эту классификацию стоит несколько раз показать заранее, чтобы она закрепилась у вас в памяти.

В: Чем отличается бизнес-правило от операций с корзиной? Разве это не интернет-магазин? Разве покупательская корзина не занимает в нем центральное место?

О: Взгляните на это так: в большинстве интернет-магазинов имеется корзина. Можно представить, что операции с корзиной характерны для многих магазинов. Таким образом, они являются общими для сайтов электронной торговли. С другой стороны, бизнес-правила являются специфическими для деятельности этого конкретного предприятия, MegaMart. Если перейти к другому магазину, можно ожидать, что в нем будет корзина, но аналогичного правила бесплатной доставки не будет.

В: Может ли функция быть одновременно бизнес-правилом и операцией с корзиной?

О: Замечательный вопрос! Да, на данный момент это так. Но это признак «кода с душком», с которым нужно будет разобраться, когда мы начнем говорить об уровнях. Если бизнес-правила должны будут учитывать, что корзина представляет собой массив, это может стать проблемой. Бизнес-правила изменяются чаще, чем такие низкоуровневые конструкции, как покупательские корзины. Если мы собираемся двигаться в будущее с этой архитектурой, их придется каким-то образом отделить. Но пока оставим их так, как есть.



Ваш ход

Функция `update_shipping_icons()` имеет наибольший объем и, вероятно, делает больше остальных функций. Далее приведен список решаемых ею задач с разбиением на категории:

```
function update_shipping_icons(cart) {
  var buy_buttons = get_buy_buttons_dom();
  for(var i = 0; i < buy_buttons.length; i++) {
    var button = buy_buttons[i];
    var item = button.item;
    var new_cart = add_item(cart, item);
    if(gets_free_shipping(new_cart))
      button.show_free_shipping_icon();
    else
      button.hide_free_shipping_icon();
  }
}
```

Задачи функции

1. Получение всех кнопок.
2. Перебор кнопок.
3. Получение товара для кнопки.
4. Создание новой корзины с этим товаром.
5. Проверка того, распространяется ли на корзину бесплатная доставка.
6. Отображение или скрытие значков.

*Операции с кнопками
покупки*

*Операции с корзиной
и товарами*

Операции с DOM

Ваша задача — разобрать эти задачи на функции, относящиеся только к одной категории. Попробуйте, существует много способов сделать это правильно.

*Запишите здесь
свое решение*



Ответ

Существует много способов разделения этой функции. Ниже приведен один из способов, в котором ясно показано, что что делает. Возможно, вы разобрали ее совершенно иным способом, и это нормально.

Операция с кнопками покупки

```
function update_shipping_icons(cart) {
  var buy_buttons = get_buy_buttons_dom();
  for(var i = 0; i < buy_buttons.length; i++) {
    var button = buy_buttons[i];
    var item = button.item;
    var hasFreeShipping =
      gets_free_shipping_with_item(cart, item);
    set_free_shipping_icon(button, hasFreeShipping);
  }
}
```

1. Получение всех кнопок

2. Перебор кнопок

3. Получение товара для кнопки

Операции с корзиной и товарами

```
function gets_free_shipping_with_item(cart, item) {
  var new_cart = add_item(cart, item);
  return gets_free_shipping(new_cart);
}
```

4. Создание новой корзины с этим товаром

5. Проверка того, распространяется ли на корзину бесплатная доставка

Операции с DOM

```
function set_free_shipping_icon(button, isShown) {
  if(isShown)
    button.show_free_shipping_icon();
  else
    button.hide_free_shipping_icon();
}
```

6. Отображение или сокрытие значков

Конечно, это не все, что можно сделать. Преимущество такого решения в разделении операций с кнопками и операций с товарами и корзиной. Это было полезное упражнение. Однако код и так был неплох, поэтому оставим его и пойдем дальше.

Уменьшение функций и новые вычисления

Ниже приведен новый код. Пометим эти функции соответствующим маркером категории действия (A), вычисления (C) или данных (D), чтобы вы получили представление о том, какая часть кода принадлежит той или иной категории.

Условные обозначения

- A** Действие
- C** Вычисление
- D** Данные

```

A ← Глобальная переменная: действие
var shopping_cart = [];

A
function add_item_to_cart(name, price) {
    var item = make_cart_item(name, price);
    shopping_cart = add_item(shopping_cart, item);
    var total = calc_total(shopping_cart);
    set_cart_total_dom(total); ← Чтение глобальной переменной: действие
    update_shipping_icons(shopping_cart);
    update_tax_dom(total);
}

A
function update_shipping_icons(cart) {
    var buttons = get_buy_buttons_dom();
    for(var i = 0; i < buttons.length; i++) {
        var button = buttons[i];
        var item = button.item;
        var new_cart = add_item(cart, item);
        if(gets_free_shipping(new_cart))
            button.show_free_shipping_icon(); ← Изменение DOM: действие
        else
            button.hide_free_shipping_icon();
    }
}

A ← Изменение DOM: действие
function update_tax_dom(total) {
    set_tax_dom(calc_tax(total));
}

C
function add_element_last(array, elem) {
    var new_array = array.slice();
    new_array.push(elem); ← Неявный ввод или вывод отсутствует
    return new_array;
}

C
function add_item(cart, item) {
    return add_element_last(cart, item); ← Неявный ввод или вывод отсутствует
}

C
function make_cart_item(name, price) {
    return {
        name: name, ← Неявный ввод или вывод отсутствует
        price: price
    };
}

```


С

```
function calc_total(cart) {
  var total = 0;
  for(var i = 0; i < cart.length; i++) {
    var item = cart[i];
    total += item.price;
  }
  return total;
}
```

Неявный ввод или вывод отсутствует

С

```
function gets_free_shipping(cart) {
  return calc_total(cart) >= 20;
}
```

Неявный ввод или вывод отсутствует

С

```
function calc_tax(amount) {
  return amount * 0.10;
}
```

Неявный ввод или вывод отсутствует

Помните:

Достаточно обнаружить в функции одно действие, чтобы вся функция стала действием.

Итоги главы

Похоже, рекомендации Ким помогли улучшить организацию кода. Теперь нашим действиям не нужно знать структуру данных. И мы видим, что в коде понемногу начинают проявляться полезные интерфейсные функции, пригодные для повторного использования.

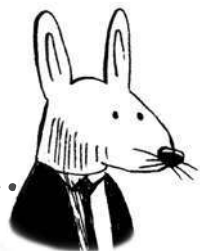
И хорошо, что мы занялись этим сейчас, потому что в корзине скрываются другие ошибки, о которых MegaMart пока ничего не знает. Что это за ошибки? Скоро мы ими займемся! Но сначала необходимо поближе познакомиться с концепцией неизменяемости.

Резюме

- В общем случае желательно устранить неявный ввод и вывод, заменяя их аргументами и возвращаемыми значениями.
- Суть проектирования в разделении. То, что вы разделили, всегда можно собрать воедино.
- При разделении кода, когда функции имеют только одну обязанность, они легко группируются на базе концепций.

Что дальше?

Мы вернемся к теме проектирования в главе 8. В следующих двух главах более подробно рассматривается неизменяемость. Как реализовать ее в новом коде, не теряя возможности взаимодействия с существующим кодом?



В этой главе

- ✓ Применение подхода копирования при записи для предотвращения изменения данных.
- ✓ Разработка операций копирования при записи для массивов и объектов.
- ✓ Копирование при записи для данных с большим уровнем вложенности.

Мы уже говорили о неизменяемости и даже реализовали ее в некоторых местах. В этой главе тема неизменяемости будет рассмотрена более подробно. Вы узнаете, как создать неизменяемые версии всех стандартных операций массивов и объектов JavaScript, которыми вы привыкли пользоваться.

Может ли неизменяемость применяться повсеместно

Мы уже реализовали некоторые операции с корзиной с использованием подхода копирования при записи. Помните: это означает, что мы создаем копию, изменяем копию, а затем возвращаем копию. Однако у корзины есть ряд операций, которые еще не были реализованы. Ниже перечислены операции с корзиной и товарами в корзине, которые нам понадобятся (или могут понадобиться).

Операции с корзиной

1. Получение количества товаров.
2. Получение товара по названию.
3. Добавление товара.
4. Удаление товара по названию.
5. Обновление количества единиц товара.

Уже реализована

Операции с товарами

1. Назначение цены.
2. Получение цены.
3. Получение названия.

Операция с вложенными данными



Дженна из команды разработки

Дженна скептически относится к идее о том, что все эти операции можно реализовать как неизменяемые. С пятой операцией дело обстоит сложнее, потому что она изменяет товар внутри корзины. Такие данные называются *вложенными*. Как реализовать неизменяемость для вложенных данных? Давайте разберемся.



Загляни в словарь

Данные называются *вложенными*, если структуры данных содержатся внутри других структур данных: например, массив, элементами которого являются объекты. Объекты являются *вложенными* по отношению к массиву. *Вложенные* данные можно представить себе как набор матрешек: одна внутри другой, другая внутри третьей и т. д.

Данные называются *глубоко вложенными* при достаточно высоком уровне вложения. Конечно, такое определение относительно, но в качестве примера можно привести объекты внутри объектов внутри массивов внутри объектов внутри объектов... Такое вложение может идти еще дальше.

Классификация операций чтения и/или записи

Мы можем классифицировать операцию как чтение или запись

Рассмотрим каждую из наших операций с новой точки зрения. Некоторые из наших операций являются операциями *чтения*. Мы извлекаем некоторую информацию из данных без их модификации. Это относительно простой случай, потому что данные не изменяются. Никакая другая работа не нужна. Операции чтения, получающие информацию из своих аргументов, являются вычислениями.

Остальные наши операции являются операциями *записи*. Они тем или иным способом изменяют данные. В их реализации необходима осторожность, потому что они не должны изменять какие-либо значения, которые могут использоваться в других местах.

Операции с корзиной

1. Получение количества товаров.
2. Получение товара по названию.
3. Добавление товара.
4. Удаление товара по названию.
5. Обновление количества единиц товара.

Чтение

Запись

Три из наших операций с корзиной относятся к категории операций записи. Мы хотим реализовать их с применением практики неизменяемости. Как было показано ранее, выбранный подход называется «копированием при записи». Это та же практика, которая используется в таких языках, как Haskell и Clojure. Различие в том, что эти языки реализуют данную практику за вас.

Так как мы работаем на JavaScript, по умолчанию данные изменяемые, поэтому программистам придется самостоятельно в явном виде применять практику в коде.

Но что насчет операций, совмещающих чтение с записью?

Иногда требуется изменить данные (запись) и одновременно извлечь из них некоторую ин-

Операции чтения

- Получают информацию из данных.
- Не изменяют данные.

Операции записи

- Изменяют данные.



Языковое сафари

Неизменяемые данные — это одна из типичных особенностей языков функционального программирования, хотя и не во всех языках она действует по умолчанию. Несколько функциональных языков, неизменяемых по умолчанию:

- Haskell;
- Clojure;
- Elm;
- Purescript;
- Erlang;
- Elixir.

В других языках неизменяемые данные поддерживаются в дополнение к изменяемым, используемым по умолчанию. А некоторые языки доверяют программисту применить любой порядок на его выбор.

формацию (чтение.) Если вас интересует эта тема, то подождите немного: мы доберемся до нее через несколько страниц. Короткий ответ: «Да, это возможно». Длинный ответ приведен на с. 154.

Операции с товарами

1. Назначение цены.

2. Получение цены.

3. Получение названия.

Запись

Чтение

Три этапа выполнения копирования при записи

Копирование при записи реализуется в вашем коде всего за три этапа. Выполняя эти шаги, вы правильно реализуете копирование при записи. А если вы замените копированием при записи каждую часть вашего кода, изменяющую глобальную корзину, корзина не будет изменяться. Она станет неизменяемой.

Ниже перечислены три этапа. Их необходимо реализовать при любых попытках изменения данных, которые должны быть неизменяемыми.

1. Создание копии.
2. Изменение копии (как хотите!).
3. Возвращение копии.

Обратимся к функции `add_element_last()`, реализующей копирование при записи (из предыдущей главы):

```
function add_element_last(array, elem) {
  var new_array = array.slice();
  new_array.push(elem);
  return new_array;
}
```

Требуется изменить массив

1. Создание копии

2. Изменение копии (как хотите!)

3. Возвращение копии

Почему эта схема работает? Как она предотвращает изменение массива?

1. Мы создаем копию массива, но нигде не изменяем оригинал.
2. Копия существует в локальной области видимости функции. Это означает, что другой код не сможет обратиться к ней, пока мы ее изменяем.
3. После того как изменение копии будет завершено, мы передаем ее за пределы области видимости (возвращаем ее). На этом изменение завершается.

Операции чтения

- Получают информацию из данных.
- Не изменяют данные.

Операции записи

- Изменяют данные.

.....

Копирование при записи преобразует операции записи в операции чтения.

.....

Вопрос: является ли `add_element_last()` операцией чтения или записи?

Она не изменяет данные, а теперь возвращает информацию, поэтому она должна быть операцией чтения! По сути, мы преобразовали операцию записи в операцию чтения. Вскоре мы поговорим об этом более подробно.

Преобразование записи в чтение с использованием копирования при записи

Рассмотрим еще одну операцию, изменяющую корзину. Эта операция удаляет из корзины товар с заданным названием.

```
function remove_item_by_name(cart, name) {
  var idx = null;
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      idx = i;
  }
  if(idx !== null)
    cart.splice(idx, 1);
}
```

cart.splice() изменяет массив cart

Является ли массив лучшей структурой данных для представления корзины? Возможно, нет. Но в системе, реализованной MegaMart, дело обстоит именно так. А это значит, что нам (по крайней мере пока) придется работать с тем, что есть.

Что делает `cart.splice()`?

`.splice()` — метод массивов, предназначенный для удаления элементов из массива.

Удаляет один элемент

С индексом idx

`cart.splice(idx, 1)`

`.splice()` также может выполнять другие операции с разными комбинациями аргументов, но здесь мы их рассматривать не будем.

Приведенная функция изменяет корзину (вызовом `cart.splice()`). Если передать `remove_item_by_name()` глобальную переменную `shopping_cart`, функция изменит глобальную корзину.

Но мы не хотим, чтобы корзина изменялась. Корзина должна рассматриваться как неизменяемая. Применим подход копирования при записи к функции `remove_item_by_name()`.

Итак, имеется функция, изменяющая корзину, и мы хотим использовать в ней подход копирования при записи. Начнем с создания копии.

*Создаем копию корзины
и сохраняем ее в локальной
переменной*

Текущая версия

```
function remove_item_by_name(cart, name) {  
    var idx = null;  
    for(var i = 0; i < cart.length; i++) {  
        if(cart[i].name === name)  
            idx = i;  
    }  
    if(idx !== null)  
        cart.splice(idx, 1);  
}
```

С копированием аргумента

```
function remove_item_by_name(cart, name) {  
    var new_cart = cart.slice();  
    var idx = null;  
    for(var i = 0; i < cart.length; i++) {  
        if(cart[i].name === name)  
            idx = i;  
    }  
    if(idx !== null)  
        cart.splice(idx, 1);  
}
```

Мы создаем копию, но пока ничего с ней не делаем. На следующей странице код, изменяющий аргумент `cart`, будет изменен для модификации копии.

Правила копирования при записи

1. Создание копии.
2. Изменение копии.
3. Возвращение копии.

Правила копирования при записи

- 
1. Создание копии.
 2. Изменение копии.
 3. Возвращение копии.

Мы только что создали копию, и теперь ее необходимо использовать. Заменяем все случаи использования исходной корзины нашей новой копией.

Правила копирования при записи

1. Создание копии.
2. Изменение копии.
3. Возвращение копии.

Текущая версия

```
function remove_item_by_name(cart, name) {
  var new_cart = cart.slice();
  var idx = null;
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      idx = i;
  }
  if(idx !== null)
    cart.splice(idx, 1);
}
```

С изменением копии

```
function remove_item_by_name(cart, name) {
  var new_cart = cart.slice();
  var idx = null;
  for(var i = 0; i < new_cart.length; i++) {
    if(new_cart[i].name === name)
      idx = i;
  }
  if(idx !== null)
    new_cart.splice(idx, 1);
}
```

Теперь оригинал вообще не изменяется. Тем не менее копия остается внутри функции. Чтобы вывести ее наружу, вернем ее из функции.

Правила копирования при записи

1. Создание копии.
2. Изменение копии.
3. Возвращение копии.

На предыдущей странице мы избавились от всех изменений массива корзины. Вместо этого изменялась копия. Пора сделать последний шаг копирования при записи и вернуть копию из функции.

Правила копирования при записи

- 
1. Создание копии.
 2. Изменение копии.
 3. Возвращение копии.

Текущая версия

```
function remove_item_by_name(cart, name) {  
  var new_cart = cart.slice();  
  var idx = null;  
  for(var i = 0; i < new_cart.length; i++)  
  {  
    if(new_cart[i].name === name)  
      idx = i;  
  }  
  if(idx !== null)  
    new_cart.splice(idx, 1);  
}
```

С возвращением копии

```
function remove_item_by_name(cart, name) {  
  var new_cart = cart.slice();  
  var idx = null;  
  for(var i = 0; i < new_cart.length; i++)  
  {  
    if(new_cart[i].name === name)  
      idx = i;  
  }  
  if(idx !== null)  
    new_cart.splice(idx, 1);  
  return new_cart;  
}
```

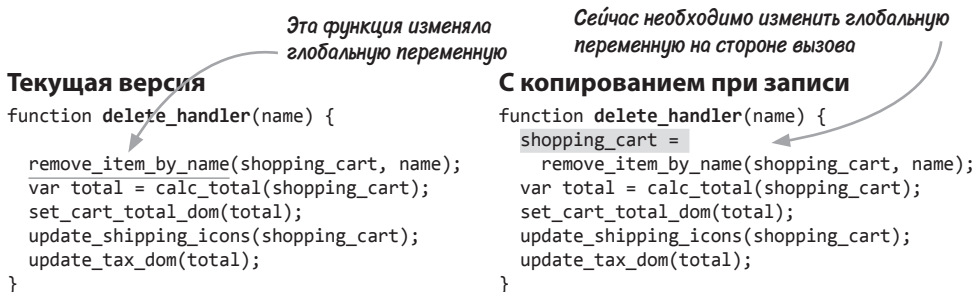
Возвращаем копию

Теперь у нас имеется полностью работоспособная версия функции с копированием при записи. Единственное, что осталось сделать, — изменить ее использование.

Правила копирования при записи

- 
1. Создание копии.
 2. Изменение копии.
 3. Возвращение копии.

На предыдущей странице мы избавились от всех модификаций массива корзины. Вместо этого мы изменяли копию. Теперь можно сделать последний шаг реализации копирования при записи и вернуть копию.



Далее необходимо перебрать все точки вызова `remove_item_by_name()` и присвоить возвращаемое значение глобальной переменной `shopping_cart`. Мы этого делать не будем: это было бы слишком скучно.

Сравнение двух версий

Мы внесли ряд изменений на нескольких страницах. Теперь я приведу весь код в одном месте:

Исходная версия с изменением

```
var idx = null;
for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
        idx = i;
}
if(idx !== null)
    cart.splice(idx, 1);
}

function delete_handler(name) {
    remove_item_by_name(shopping_cart, name);
    var total = calc_total(shopping_cart);
    set_cart_total_dom(total);
    update_shipping_icons(shopping_cart);
    update_tax_dom(total);
}
```

Версия с копированием при записи

```
function remove_item_by_name(cart, name) {
    var new_cart = cart.slice();
    var idx = null;
    for(var i = 0; i < new_cart.length; i++) {
        if(new_cart[i].name === name)
            idx = i;
    }
    if(idx !== null)
        new_cart.splice(idx, 1);
    return new_cart;
}

function delete_handler(name) {
    shopping_cart =
        remove_item_by_name(shopping_cart, name);
    var total = calc_total(shopping_cart);
    set_cart_total_dom(total);
    update_shipping_icons(shopping_cart);
    update_tax_dom(total);
}
```

Операции копирования при записи обобщаются

Очень похожие операции копирования при записи будут выполняться снова и снова. Мы можем обобщить уже написанный код, чтобы он лучше подходил для повторного использования, как было сделано ранее с `add_element_last()`.

Начнем с метода `.splice()` массива. Метод `.splice()` используется в функции `remove_item_by_name()`.

Исходная версия с изменением

```
function removeItems(array, idx, count) {
    array.splice(idx, count);
}
```

Версия с копированием при записи

```
function removeItems(array, idx, count) {
    var copy = array.slice();
    copy.splice(idx, count);
    return copy;
}
```

Теперь можно использовать новую реализацию в `remove_item_by_name()`.

Предыдущая версия с копированием при записи

```
function remove_item_by_name(cart, name) {
    var new_cart = cart.slice();
    var idx = null;
    for(var i = 0; i < new_cart.length; i++) {
        if(new_cart[i].name === name)
            idx = i;
    }
    if(idx !== null)
        new_cart.removeItems(idx, 1);
    return new_cart;
}
```

Версия с копированием при записи, использующая `splice()`

```
function remove_item_by_name(cart, name) {
    var idx = null;
    for(var i = 0; i < cart.length; i++) {
        if(cart[i].name === name)
            idx = i;
    }
    if(idx !== null)
        return removeItems(cart, idx, 1);
    return cart;
}
```

removeItems() копирует массив, чтобы нам не приходилось это делать

Дополнительный плюс: если массив не изменялся, создавать его копию не нужно

Поскольку, скорее всего, эти операции будут использоваться достаточно часто, их реализация, рассчитанная на повторное использование, сэкономит немало усилий. Нам не придется раз за разом повторять шаблонный код копирования массива или объекта.

Знакомство с массивами в JavaScript

Одним из основных типов коллекций в JavaScript является *массив*. Массивы в JavaScript представляют упорядоченные коллекции значений. Они являются гетерогенными, то есть могут одновременно содержать значения разных типов. К элементам можно обращаться по индексу. Массивы JavaScript отличаются от структур данных, которые называются *массивами* в других языках. Их можно расширять и сокращать — в отличие от массивов Java или C.

Чтение элемента с заданным индексом [idx]

Получает элемент с индексом `idx`. Нумерация индексов начинается с 0.

```
> var array = [1, 2, 3, 4];
> array[2]
3
```

Присваивание элемента с заданным индексом [] =

Оператор присваивания изменяет массив.

```
> var array = [1, 2, 3, 4];
> array[2] = "abc"
"abc"
> array
[1, 2, "abc", 4]
```

Длина массива .length

Содержит количество элементов в массиве. Не является методом, поэтому круглые скобки не нужны.

```
> var array = [1, 2, 3, 4];
> array.length
4
```

Добавление в конец массива .push(el)

Изменяет массив, присоединяя к нему `el`, и возвращает новую длину массива.

```
> var array = [1, 2, 3, 4];
> array.push(10);
5
> array
[1, 2, 3, 4, 10]
```

Удаление в конце массива .pop()

Изменяет массив, удаляя из него последний элемент, и возвращает удаленное значение.

```
> var array = [1, 2, 3, 4];  
> array.pop();  
4  
> array  
[1, 2, 3]
```

Добавление в начало массива .unshift(el)

Изменяет массив, добавляя **el** в начало, и возвращает новую длину массива.

```
> var array = [1, 2, 3, 4];  
> array.unshift(10);  
5  
> array  
[10, 1, 2, 3, 4]
```

Удаление в начале массива .shift()

Изменяет массив, удаляя из него первый элемент (с индексом 0), и возвращает удаленное значение.

```
> var array = [1, 2, 3, 4];  
> array.shift()  
1  
> array  
[2, 3, 4]
```

Копирование массива .slice()

Создает и возвращает поверхностную копию (shallow copy) массива.

```
> var array = [1, 2, 3, 4];  
> array.slice()  
[1, 2, 3, 4]
```

Удаление элементов .splice(idx, num)

Изменяет массив, удаляя **num** элементов, начиная с **idx**, и возвращает удаленные элементы.

```
> var array = [1, 2, 3, 4, 5, 6];  
> array.splice(2, 3); // Удаляет три элемента  
[3, 4, 5]  
> array  
[1, 2, 6]
```



Ваш ход

Ниже приведена операция для включения контакта в список рассылки. Эта операция добавляет адреса электронной почты в конец списка, хранящегося в глобальной переменной. Она вызывается обработчиком отправки данных формы.

```
var mailing_list = [];  
  
function add_contact(email) {  
    mailing_list.push(email);  
}  
  
function submit_form_handler(event) {  
    var form = event.target;  
    var email = form.elements["email"].value;  
    add_contact(email);  
}
```

Ваша задача заключается в том, чтобы преобразовать этот код к форме копирования при записи. Пара рекомендаций.

1. Функция `add_contact()` не должна обращаться к глобальной переменной. Она должна получать `mailing_list` в аргументе, создавать копию, изменять ее и возвращать копию.
2. При каждом вызове `add_contact()` необходимо присвоить возвращаемое значение глобальной переменной `mailing_list`.

Измените предоставленный код, чтобы он следовал подходу копирования при записи. Ответ приведен на следующей странице.

Запишите здесь свой ответ





Ответ

В процессе изменения необходимо было решить две задачи.

1. Функция `add_contact()` не должна обращаться к глобальной переменной. Она должна получать `mailing_list` в аргументе, создавать копию, изменять ее и возвращать копию.
2. При каждом вызове `add_contact()` необходимо присвоить возвращаемое значение глобальной переменной `mailing_list`.

Измененная версия кода может выглядеть так:

Оригинал

```
var mailing_list = [];

function add_contact(email) {

    mailing_list.push(email);

}

function submit_form_handler(event) {
    var form = event.target;
    var email =
        form.elements["email"].value;
    add_contact(email);
}
```

Версия с копированием при записи

```
var mailing_list = [];

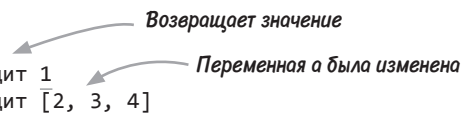
function add_contact(mailing_list,
    email) {
    var list_copy = mailing_list.slice();
    list_copy.push(email);
    return list_copy;
}

function submit_form_handler(event) {
    var form = event.target;
    var email =
        form.elements["email"].value;
    mailing_list =
        add_contact(mailing_list, email);
}
```

Что делать с операциями чтения/записи

Иногда функция играет сразу две роли: она изменяет значение, а также возвращает значение. Хорошим примером служит метод `.shift()`. Пример:

```
var a = [1, 2, 3, 4];
var b = a.shift();
console.log(b); // выводит 1
console.log(a); // выводит [2, 3, 4]
```



The diagram consists of two curved arrows. The first arrow starts from the `a.shift()` expression in the second line of code and points to the value `1` in the output of `console.log(b)`. The second arrow starts from the `a.shift()` expression and points to the new array `[2, 3, 4]` in the output of `console.log(a)`.

`.shift()` возвращает первый элемент одновременно с изменением массива. Метод выполняет как операцию чтения, так и операцию записи.

Как преобразовать его для использования подхода копирования при записи?

В реализации копирования при записи операция записи фактически преобразуется в операцию чтения, и это означает, что мы должны вернуть значение. Но `.shift()` уже выполняет операцию чтения, поэтому у него уже имеется возвращаемое значение. Как добиться желаемой цели? Существуют два возможных подхода.

1. Разбиение функции на чтение и запись.
2. Возвращение двух значений из функции.

Мы рассмотрим оба варианта. Если у вас есть выбор, предпочтение следует отдавать первому, так как он более четко разделяет обязанности. Как было показано в главе 5, суть проектирования в разделении.

Начнем с первого подхода.

Два подхода

1. Разбиение функции.
2. Возвращение двух значений.

Разделение функции, выполняющей чтение и запись

Это решение состоит из двух шагов. Сначала операция разбивается, чтобы чтение было отделено от записи. На втором шаге запись преобразуется в операцию копирования при записи. Это делается так же, как с любой другой операцией записи.

Разделение операции на чтение и запись

Операция чтения метода `.shift()` — это просто его возвращаемое значение, которым является первый элемент массива. Таким образом, мы просто пишем вычисление, которое возвращает первый элемент массива. Операция является операцией чтения, поэтому ничего изменять она не должна. Так как операция не имеет скрытого ввода и вывода, она является вычислением.


```
function first_element(array) {
  return array[0];
}
```

Просто функция, которая возвращает первый элемент (или undefined, если список пуст). Функция является вычислением

Нам не нужно преобразовывать `first_element()`, потому что это операция чтения, которая не изменяет массив.

Запись метода `.shift()` не нужна, но поведение `.shift()` должно быть заключено в отдельную функцию. Мы отбросим возвращаемое значение `.shift()`, просто чтобы подчеркнуть, что не собираемся использовать результат.

```
function drop_first(array) {
  array.shift();
}
```

Выполнить shift(), но отбросить возвращаемое значение

Преобразование записи в копирование при записи

Мы успешно отделили операцию чтения от операции записи. Тем не менее запись (`drop_first()`) изменяет ее аргумент. Ее следует преобразовать в копирование при записи.

Изменяющая версия

```
function drop_first(array) {
  array.shift();
}
```

Версия с копированием при записи

```
function drop_first(array) {
  var array_copy = array.slice();
  array_copy.shift();
  return array_copy;
}
```

Классическая реализация копирования при записи

Подход с разделением чтения и записи считается предпочтительным, потому что он предоставляет все необходимые составляющие. Их можно использовать как по отдельности, так и совместно. Ранее мы вынуждены были использовать их вместе, а теперь появился выбор.

Возвращение двух значений одной функцией

Второй подход, как и первый, тоже состоит из двух шагов. На первом шаге метод `.shift()` упаковывается в функцию, которую мы можем изменять. Эта функция выполняет как операцию чтения, так и операцию записи. На втором шаге она преобразуется, чтобы осталось только чтение.

Упаковка операции

Первый шаг — упаковка метода `.shift()` в функцию, которая находится под нашим контролем и которую можно изменять. Но в данном случае отбрасывать возвращаемое значение не нужно.

```
function shift(array) {
  return array.shift();
}
```

Преобразование чтения и записи в чтение

В данном случае только что написанную функцию `shift()` необходимо преобразовать так, чтобы она создавала копию, изменяла копию и возвращала как первый элемент, так и измененную копию. Посмотрим, как это можно сделать.

Изменяющая версия

```
function shift(array) {
  return array.shift();
}
```

Версия с копированием при записи

```
function shift(array) {
  var array_copy = array.slice();
  var first = array_copy.shift();
  return {
    first : first,
    array : array_copy
  };
}
```

Объект используется для возвращения двух разных значений

Другой вариант

Другое возможное решение — использовать подход, описанный на предыдущей странице, и объединить два возвращаемых значения в объекте:

```
function shift(array) {
  return {
    first : first_element(array),
    array : drop_first(array)
  };
}
```

Поскольку обе функции являются вычислениями, беспокоиться об объединении не нужно: результат также будет вычислением.



Ваш ход

Мы только что написали версии метода `.shift()` для массивов, использующие подход копирования при записи. У массивов также имеется метод `.pop()`, который удаляет последний элемент массива и возвращает его. Как и `.shift()`, метод `.pop()` выполняет как чтение, так и запись.

Ваша задача — преобразовать этот метод, выполняющий чтение и запись, в две разные версии. Пример использования `.pop()`:

```
var a = [1, 2, 3, 4];  
var b = a.pop();  
console.log(b); // выводит 4  
console.log(a); // выводит [1, 2, 3]
```

Преобразуем `.pop()` в версии с копированием при записи.

1. Разделение чтения и записи на две функции

Запишите здесь
свои ответы



2. Возвращение двух значений из одной функции



Ответ

Наша задача — переписать `.pop()` с использованием копирования при записи. Мы напишем две разные реализации.

1. Разделение чтения и записи на две операции

Первое, что необходимо сделать, — создать две функции-обертки для реализации частей чтения и записи по отдельности.

```
function last_element(array) {
  return array[array.length - 1];
}

function drop_last(array) {
  array.pop();
}
```

Чтение

Запись

Чтение завершено. Дальнейшие изменения не нужны. Однако операцию записи необходимо преобразовать в операцию копирования при записи.

Оригинал

```
function drop_last(array) {
  array.pop();
}
```

Версия с копированием при записи

```
function drop_last(array) {
  var array_copy = array.slice();
  array_copy.pop();
  return array_copy;
}
```

2. Возвращение двух значений

Начнем с создания функции-обертки для операции. Она не добавляет никакой новой функциональности, но дает нам материал для изменения.

```
function pop(array) {
  return array.pop();
}
```

Изменим функцию, чтобы она соответствовала подходу копирования при записи.

Оригинал

```
function pop(array) {
  return array.pop();
}
```

Версия с копированием при записи

```
function pop(array) {
  var array_copy = array.slice();
  var first = array_copy.pop();
  return {
    first: first,
    array: array_copy
  };
}
```



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Как получается, что версия `add_element_to_cart()` с копированием при записи становится операцией чтения?

О: Функция `add_element_to_cart()`, реализующая подход копирования при записи, является операцией чтения, потому что она не изменяет корзину. Можно рассматривать ее как вопрос, например: «Как бы выглядела эта корзина, если бы она также содержала этот элемент?»

Конечно, это гипотетический вопрос. Очень большая часть планирования и рассуждений связана с ответами на гипотетические вопросы. Помните: вычисления часто используются для планирования. Мы увидим еще немало примеров такого рода в следующих главах.

В: Корзина использует массив, и нам приходится проводить поиск по массиву, чтобы найти элемент с заданным именем. Является ли массив наиболее подходящей структурой данных для этого? Не лучше ли выбрать ассоциативную структуру данных, такую как объект?

О: Да, возможно, лучше было бы воспользоваться объектом. Часто оказывается, что в существующем коде выбор структуры данных уже зафиксирован и легко изменить его не удастся. Здесь как раз такой случай. Нам придется и дальше работать с реализацией корзины в виде массива.

В: Похоже, реализация неизменяемости требует значительной работы. Стоит ли оно того? Нет ли чего-то попроще?

О: В JavaScript стандартной библиотеки как таковой нет. Может показаться, что вам постоянно приходится писать процедуры, которые должны быть частью языка. Кроме того, JavaScript не помогает с копированием при записи. Во многих языках вам приходится писать собственную реализацию копирования при записи. Спросите себя, стоит ли оно того.

Прежде всего, писать новые функции не нужно: можно использовать встроенную реализацию. Тем не менее часто это требует дополнительной работы. При этом вам приходится писать много повторяющегося кода, и каждый раз, когда вы его пишете, нужно действовать очень внимательно, чтобы сделать все правильно. Гораздо лучше написать операции один раз и использовать их повторно.

К счастью, вам понадобится не так уж много операций. Поначалу кажется, что писать их довольно утомительно, но вскоре вам уже не придется писать новые операции с нуля. Вы будете повторно использовать существующие операции и комбинировать их для создания новых, более мощных.

Поскольку это потребует большого объема работы на начальной стадии, я рекомендую писать функции только тогда, когда они вам необходимы.



Ваш ход

Напишите версию метода массива `.push` с копированием при записи. Напомню, что `.push()` добавляет один элемент в конец массива.

```
function push(array, elem) {
```

Запишите здесь свою реализацию

```
}
```



Ответ

```
function push(array, elem) {  
  var copy = array.slice();  
  copy.push(elem);  
  return copy;  
}
```



Ваш ход

Проведите рефакторинг функции `add_contact()`, чтобы в ней использовалась новая функция `push()` из предыдущего упражнения. Существующий код:

```
function add_contact(mailing_list, email) {
  var list_copy = mailing_list.slice();
  list_copy.push(email);
  return list_copy;
}
```

Запишите здесь свою реализацию

```
function add_contact(mailing_list, email) {
```

```
}
```



Ответ

```
function add_contact(mailing_list,
                     email) {
  var list_copy = mailing_list.slice();
  list_copy.push(email);
  return list_copy;
}
```

```
function add_contact(mailing_list,
                     email) {
  return push(mailing_list, email);
}
```



Ваш ход

Напишите функцию `arraySet()` — версию оператора присваивания массива с копированием при записи.

Пример:

`a[15] = 2;`

Реализуйте версию этой операции с копированием при записи

`function arraySet(array, idx, value) {`

Запишите здесь свою реализацию

`}`



Ответ

```
function arraySet(array, idx, value) {
  var copy = array.slice();
  copy[idx] = value;
  return copy;
}
```


Операции чтения неизменяемых структур данных являются вычислениями



Операции чтения изменяемых данных являются действиями

Если вы читаете изменяемое значение, то при многократном чтении можно получить разные ответы, так что чтение изменяемых данных является действием.

Операция записи делает данные изменяемыми

Запись изменяет данные, поэтому именно из-за операций записи данные становятся изменяемыми.

Если данные не используются для записи, они являются неизменяемыми

Если избавиться от всех операций записи, преобразовав их в операции чтения, данные не будут изменяться после их создания. Это означает, что они будут неизменяемыми.

Операции чтения неизменяемых структур данных являются вычислениями

Когда мы делаем данные неизменяемыми, все операции чтения становятся вычислениями.

Преобразование операций записи в операции чтения увеличивает долю кода в вычислениях

Чем больше структур данных рассматривается как неизменяемые, тем больше кода содержится в вычислениях и тем меньше — в действиях.

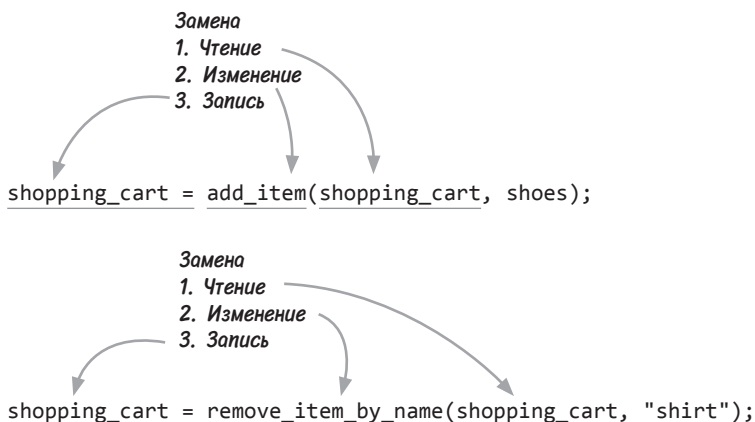
Приложения обладают состоянием, которое изменяется во времени

Теперь у нас имеются инструменты, которые позволяют перебрать весь код и преобразовать все для того, чтобы везде использовались неизменяемые данные.

Все операции записи преобразуются в чтение. Но тут возникает большая проблема, с которой мы еще не сталкивались: если все данные неизменяемы, то как приложению отслеживать изменения во времени? Как пользователю добавить товар в корзину, если ничто никогда не изменяется?

Ким абсолютно права. Мы реализовали неизменяемость повсюду, но хотя бы в одном месте данные должны остаться изменяемыми, чтобы мы могли отслеживать изменения во времени. И такое место существует: это глобальная переменная `shopping_cart`.

Мы присваиваем новые значения глобальной переменной `shopping_cart`. Она всегда указывает на текущее значение корзины. Собственно, можно сказать, что мы *заменяем* текущие значения в корзине новыми.



Глобальная переменная `shopping_cart` всегда будет указывать на текущее значение, и каждый раз, когда его потребуется изменить, мы будем использовать паттерн замены. Это чрезвычайно распространенный и мощный паттерн в функциональном программировании. Замена сильно упрощает реализацию команды отмены. Мы вернемся к этой теме и займемся повышением надежности замены в части II.

Без неизменяемых данных никак не обойтись. Как будет работать приложение, если корзина не изменяется?

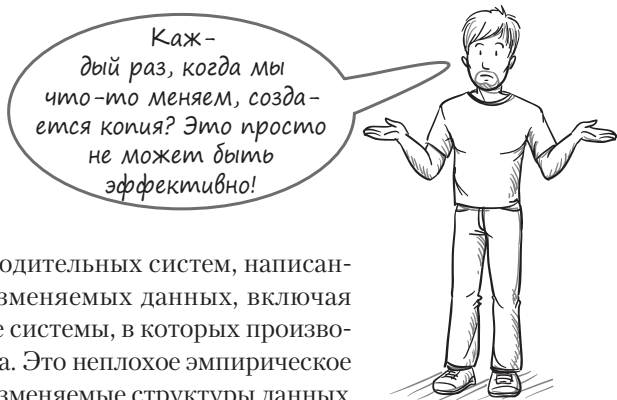


Ким из команды разработки

Неизменяемые структуры данных достаточно быстры

В общем случае неизменяемые структуры данных расходуют больше памяти и работают медленнее своих изменяемых эквивалентов.

Тем не менее существует много высокопроизводительных систем, написанных с использованием неизменяемых данных, включая высокоскоростные торговые системы, в которых производительность жизненно важна. Это неплохое эмпирическое доказательство того, что неизменяемые структуры данных достаточно быстры для типичных приложений. Впрочем, есть и другие аргументы.



Каждый раз, когда мы что-то меняем, создается копия? Это просто не может быть эффективно!

Джордж из отдела тестирования

Оптимизацией всегда можно заняться позднее

В быстрой разработке каждого приложения существуют узкие места, которые трудно предсказать в процессе разработки. Здравый смысл подсказывает, что не стоит заниматься оптимизацией, пока вы не уверены, что оптимизируемая часть действительно повлияет на производительность.

Функциональные программисты предпочитают по умолчанию использовать неизменяемые данные. Только если они обнаруживают, что какая-то часть работает слишком медленно, они переходят к оптимизации быстрого действия с изменяемыми данными.

Сборщики мусора работают очень быстро

Во многих языках (но, конечно, не во всех) в результате многолетней упорной работы сборщик мусора работает очень быстро. Некоторые сборщики мусора были оптимизированы настолько, что освобождение памяти требует всего одной-двух команд. Мы можем воспользоваться плодами этой упорной работы.

Копирования не так много, как может показаться на первый взгляд

Если вы взглянете на код копирования при записи, написанный нами к настоящему моменту, то окажется, что самого копирования в нем не так уж много. Например, если корзина содержит 100 товаров, то копироваться будет только массив из 100 ссылок. Сами элементы при этом не копируются. Копирование, при котором копируется только верхний уровень структуры данных, называется *поверхностным*. При поверхностном копировании две копии совместно

используют ссылки на одни и те же объекты в памяти. Такой вид совместного использования называется *структурным*.

В языках функционального программирования используются быстрые реализации

Мы пишем собственные функции с неизменяемыми данными на базе встроенных структур данных JavaScript, используя крайне прямолинейный код. Для нашего приложения это нормально. Однако в языках, поддерживающих функциональное программирование, часто реализованы неизменяемые структуры данных. Эти структуры данных работают намного эффективнее наших реализаций. Например, встроенные структуры данных Clojure оказались очень эффективными и даже послужили источником вдохновения для реализаций в других языках.

Насколько они эффективнее? Копии совместно используют существенно больше структурной информации, что означает снижение затрат памяти и сокращение нагрузки на сборщик мусора. При этом они также базируются на механизме копирования при записи.

Операции с копированием при записи для объектов

До сих пор операции с копированием при записи выполнялись только с массивами JavaScript. В текущей задаче также необходимо как-то задать цену для товара в корзине, который представлен объектом. Последовательность действий остается такой же.

1. Создание копии.
2. Изменение копии.
3. Возвращение копии.

Копию массива можно создать методом `.slice()`. Тем не менее эквивалентного способа копирования объектов в JavaScript не существует. С другой стороны, в JavaScript существует механизм копирования всех ключей и значений из одного объекта в другой. Если скопировать все ключи и значения в пустой объект, то фактически будет создана копия. Этот метод называется `Object.assign()`. Пример его использования для создания копий:

```
var object = {a: 1, b: 2};  
var object_copy = Object.assign({}, object);
```

Как копируются объекты
в JavaScript



Воспользуемся этим методом для копирования объектов. Мы сможем использовать его для реализации функции `set_price()`, которая задает цену объекта товара:

Оригинал

```
function setPrice(item, new_price) {
    item.price = new_price;
}
```

Версия с копированием при записи

```
function setPrice(item, new_price) {
    var item_copy = Object.assign({}, item);
    item_copy.price = new_price;
    return item_copy;
}
```

Основная идея остается той же, что и для массивов. Ее можно применить к абсолютно любой структуре данных: просто выполните уже знакомые три шага.



Загляни в словарь

Поверхностные копии дублируют только структуру вложенных данных верхнего уровня. Например, если у вас имеется массив объектов, поверхностное копирование продублирует только массив. Объекты в нем будут совместно использоваться оригиналом и копией массива. Сравнение поверхностных и глубоких копий будет приведено позднее.

Совместное использование некоторых ссылок двумя блоками вложенных данных называется *структурным совместным использованием*. Когда все данные являются неизменными, структурное совместное использование безопасно. Оно расходует меньше памяти и работает быстрее, чем полное копирование.

Кратко об объектах JavaScript

Объекты JavaScript имеют очень много общего с хеш-картами и ассоциативными массивами из других языков. Объекты JavaScript представляют собой коллекции пар «ключ/значение» с уникальными ключами. Ключи всегда являются строками, но их значения могут относиться к любому типу.

Операции, которые будут использоваться в наших примерах:

Поиск по ключу [key]

Ищет значение, соответствующее заданному ключу. Если ключ не существует, возвращается `undefined`.

```
> var object = {a: 1, b: 2};
> object["a"]
1
```

Поиск по ключу .key

Кроме того, для обращения к значениям может использоваться точечная запись. Это удобно, если ключ соответствует правилам синтаксиса лексем JavaScript.

```
> var object = {a: 1, b: 2};
> object.a
1
```

Присваивание значения для ключа .key или [key] =

Для присваивания значения по ключу может использоваться любой синтаксис, что приводит к изменению объекта. Если ключ существует, то значение заменяется. Если ключ не существует, то он добавляется в объект.

```
> var object = {a: 1, b: 2};
> object["a"] = 7;
7
> object
{a: 7, b: 2}
> object.c = 10;
10
> object
{a: 7, b: 2, c: 10}
```

Удаление пары «ключ/значение» delete

Метод изменяет объект, удаляя пару «ключ/значение» для заданного ключа. Вы можете использовать любую из двух форм синтаксиса поиска.

```
> var object = {a: 1, b: 2};
> delete object["a"];
true
> object
{b: 2}
```

Копирование объекта Object.assign(a, b)

С этой операцией дело обстоит сложнее. `Object.assign()` копирует все пары «ключ/значение» из объекта `b` в объект `a` (что приводит к его изменению). С ее помощью можно создать копию `b` посредством копирования всех пар «ключ/значение» в пустой объект.

```
> var object = {x: 1, y: 2};
> Object.assign({}, object);
{x: 1, y: 2}
```

Список ключей Object.keys()

Если вы хотите перебрать пары «ключ/значение» в объекте, это можно сделать косвенно, запросив у объекта все его ключи с помощью функции `Object.keys()`. Функция возвращает массив ключей объекта, который затем используется для перебора.

```
> var object = {a: 1, b: 2};
> Object.keys(object)
["a", "b"]
```



Ваш ход

Напишите функцию `objectSet()`, которая представляет собой версию оператора присваивания объекта с копированием при записи.

Пример:

```
o["price"] = 37;
```

*Напишите версию этой операции
с копированием при записи*

```
function objectSet(object, key, value) {
```

*Запишите здесь
свою реализацию*

```
}
```



Ответ

```
function objectSet(object, key, value) {
  var copy = Object.assign({}, object);
  copy[key] = value;
  return copy;
}
```



Ваш ход

Проведите рефакторинг `setPrice()` для использования функции `objectSet()`, написанной в последнем упражнении.

Существующий код:

```
function setPrice(item, new_price) {  
  var item_copy = Object.assign({}, item);  
  item_copy.price = new_price;  
  return item_copy;  
}
```

Напишите функцию `setQuantity()` с использованием функции `objectSet()`, которая задает количество единиц товара. Функция должна реализовать копирование при записи.

```
function setQuantity(item, new_quantity) {
```

*Запишите здесь
свою реализацию*

```
}
```



Ответ

```
function setQuantity(item, new_quantity) {  
  return objectSet(item, "quantity", new_quantity);  
}
```




Ваш ход

Напишите версию оператора `delete`, удаляющего ключ из объекта, с копированием при записи.

Пример:

```
> var a = {x : 1};
> delete a["x"];
> a
{}

```

*Напишите версию этой операции
с копированием при записи*

```
function objectDelete(object, key) {
```

*Запишите здесь
свою реализацию*

```
}
```



Ответ

```
function objectDelete(object, key) {
  var copy = Object.assign({}, object);
  delete copy[key];
  return copy;
}
```

Преобразование вложенных операций записи в чтение

У нашей корзины все еще остается одна операция записи, которую необходимо преобразовать в чтение. Операция, обновляющая цену товара по названию, все еще остается операцией записи. Тем не менее эта операция интересна тем, что она изменяет вложенную структуру данных: объект товара внутри массива корзины.

Обычно бывает проще преобразовать запись для более глубокой операции. Мы реализовали `setPrice()` в упражнении на с. 170. Функция `setPrice()`, работающая с товарами, может использоваться внутри функции `setPriceByName()`, работающей с корзинами.

Оригинал

```
function setPriceByName(cart, name, price) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      cart[i].price =
        price;
  }
}
```

Типичный паттерн «копирование + изменение копии» в копировании при записи

Версия с копированием при записи

```
function setPriceByName(cart, name, price) {
  var cartCopy = cart.slice();
  for(var i = 0; i < cartCopy.length; i++) {
    if(cartCopy[i].name === name)
      cartCopy[i] =
        setPrice(cartCopy[i], price);
  }
  return cartCopy;
}
```

Операция с копированием при записи вызывается для изменения вложенного товара

Вложенные операции записи следуют той же схеме, что и обычные: мы создаем копию, изменяем ее, а затем возвращаем копию. Единственное отличие вложенных операций заключается в том, что мы выполняем еще одну операцию с копированием при записи для изменения вложенных данных.

Если бы товар изменялся напрямую, как это делалось в исходном коде, то данные не были бы неизменяемыми. Возможно, ссылки в массиве корзины остаются в исходном состоянии, но изменяются значения, на которые они ссылаются. Такая ситуация неприемлема. Чтобы вложенная структура данных могла считаться неизменной, она вся должна оставаться без изменения.

Эта концепция очень важна. Все во вложенной структуре данных, сверху донизу, должно остаться неизменным. При обновлении вложенных данных необходимо скопировать внутреннее значение и все значения сверху донизу. Это настолько важно, что на следующих страницах мы будем досконально разбираться в том, что же именно копируется.

Что же копируется?

Предположим, в корзине лежат три товара: футболка, туфли и носки. Рассмотрим содержимое массивов и объектов: у нас имеется один массив `Array` (корзина) и три объекта (футболка, туфли и носки в корзине).

Требуется задать цену футболки \$13. Для этого воспользуемся вложенной операцией `setPriceByName()`:

```
shopping_cart = setPriceByName(shopping_cart, "t-shirt", 13);
```

Разберем код шаг за шагом и подсчитаем, что же копируется:

```
function setPriceByName(cart, name, price) {
  var cartCopy = cart.slice();
  for(var i = 0; i < cartCopy.length; i++) {
    if(cartCopy[i].name === name)
      cartCopy[i] = setPrice(cartCopy[i], price);
  }
  return cartCopy;
}
```

Копирование массива

setPrice() вызывается только один раз, когда цикл находит футболку

```
function setPrice(item, new_price) {
  var item_copy = Object.assign({}, item);
  item_copy.price = new_price;
  return item_copy;
}
```

Копирование объекта

Все начинается с одного массива и трех объектов. Что же копируется? Только один массив (корзина) и один объект (футболка). Оба объекта не копируются. Что происходит?

Мы создаем поверхностные копии вложенных данных, что приводит к структурному совместному использованию. В предложении слишком много заумных терминов, поэтому схема будет изображена на следующей странице.



Загляни в словарь

Освежим в памяти некоторые термины, которые вы уже видели.

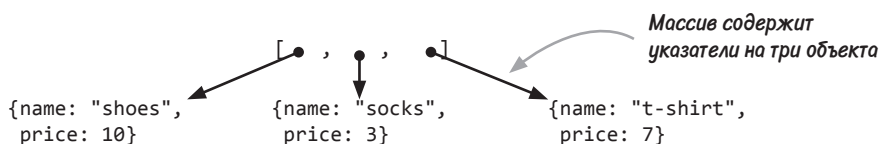
Вложенные данные — структуры данных внутри структур данных; мы будем различать *внутреннюю* структуру данных и структуру данных *верхнего уровня*.

Поверхностное копирование — копирование только структуры данных верхнего уровня во вложенных данных.

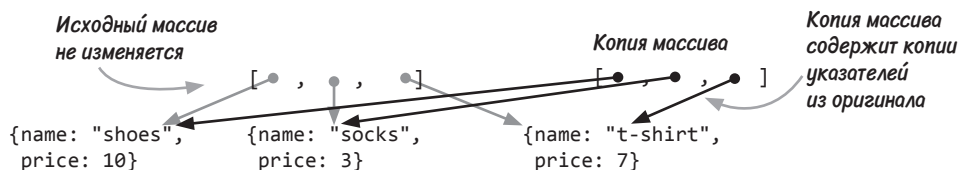
Структурное совместное использование — ситуация, при которой вложенные структуры данных ссылаются на одну и ту же внутреннюю структуру данных.

Наглядное представление поверхностного копирования и структурного совместного использования

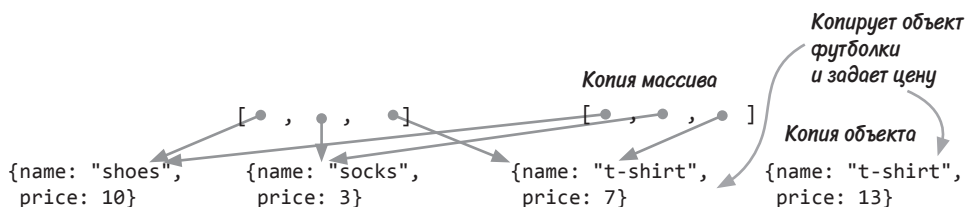
Мы начинаем с корзины (один массив) с тремя товарами (три объекта): всего четыре элемента данных. Требуется назначить футболке цену \$13.



Затем создается поверхностная копия корзины. Сначала копия содержит указатели на те же объекты в памяти.



Во время работы цикла вскоре находится футболка и для нее вызывается `setPrice()`. Функция создает поверхностную копию объекта футболки и изменяет цену на 13.

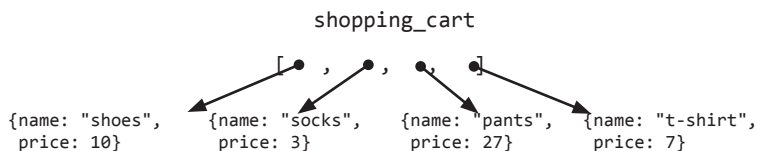


Функция `setPrice()` вернула копию, а функция `setPriceByName()` закрепила ее в массиве на месте исходного объекта футболки.



Ваш ход

Допустим, имеется корзина с четырьмя товарами:



Что потребуется скопировать при выполнении следующей строки кода:

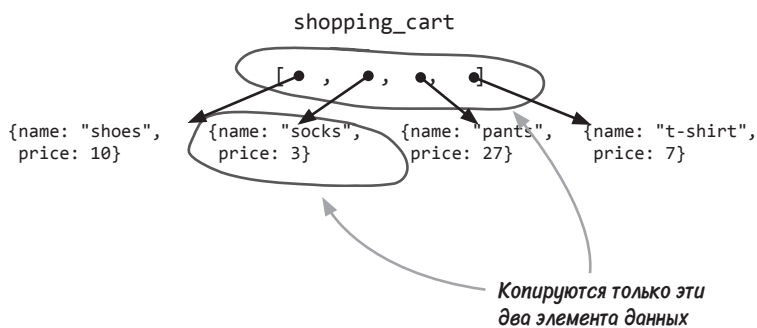
```
setPriceByName(shopping_cart, "socks", 2);
```

Обведите все копируемые элементы данных.



Ответ

Копироваться должен только изменяющийся элемент и все, что находится на пути выше него. В данном случае изменился элемент `socks`, а содержащий его массив должен измениться, чтобы в нем хранилась новая копия; следовательно, он тоже должен быть скопирован.





Ваш ход

Напишите версию вложенной операции с копированием при записи:

```
function setQuantityByName(cart, name, quantity) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      cart[i].quantity = quantity;
  }
}
```

*Запишите здесь
свою реализацию*

```
function setQuantityByName(cart, name, quantity) {
```

```
}
```



Ответ

```
function setQuantityByName(cart, name, quantity) {
  var cartCopy = cart.slice();
  for(var i = 0; i < cartCopy.length; i++) {
    if(cartCopy[i].name === name)
      cartCopy[i] =
        objectSet(cartCopy[i], 'quantity', quantity);
  }
  return cartCopy;
}
```

Итоги главы

В этой главе подробно рассматривался подход копирования при записи. И хотя этот подход также встречается в таких языках, как Clojure и Haskell, в JavaScript всю работу приходится выполнять самостоятельно. Именно поэтому так удобно упаковать ее во вспомогательные функции, которые берут на себя все рутинные операции. Если вы будете дисциплинированно использовать эти функции-обертки, все будет хорошо.

Резюме

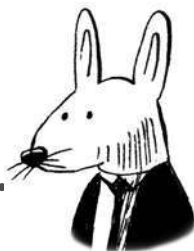
- В функциональном программировании желательно использовать неизменяемые данные. Невозможно писать вычисление, в котором используются неизменяемые данные.
- Копирование при записи — подход, обеспечивающий неизменяемость наших данных. Этот термин обозначает, что мы создаем копию и изменяем ее вместо оригинала.
- Копирование при записи требует создания поверхностной копии: в копию вносятся изменения, после чего она возвращается. Эта схема пригодится для реализации неизменяемости в коде, находящемся под вашим контролем.
- Мы можем реализовать версии основных операций с массивами и объектами для сокращения объема шаблонного кода, который нам придется писать.

Что дальше?

Практика копирования при записи — дело хорошее. Тем не менее не весь ваш код будет использовать написанные вами обертки. У многих из нас имеется значительный объем готового кода, написанного без подхода копирования при записи. Нам понадобится способ обмена данными с этим кодом без изменения наших данных. В следующей главе будет представлен другой подход, называемый *защитным копированием*.

Сохранение неизменяемости при взаимодействии с ненадежным кодом

7



В этой главе

- ✓ Создание защитных копий, позволяющих уберечь ваш код от унаследованного кода и другого кода, которому вы не доверяете.
- ✓ Сравнение глубокого копирования с поверхностным.
- ✓ Выбор между защитным копированием и копированием при записи.

Вы узнали, как поддерживать неизменяемость в вашем коде с применением копирования при записи. Тем не менее нам часто приходится взаимодействовать с кодом, не использующим эту процедуру. Мы имеем дело с библиотеками и существующим кодом, который рассматривает данные как изменяемые. Как же передать ему неизменяемые данные? В этой главе вы освоите практику обеспечения неизменяемости при взаимодействии с кодом, который может изменять ваши данные.

Неизменяемость при работе с унаследованным кодом

На сайте MegaMart снова начинается ежемесячная распродажа из-за «Черной пятницы» (да, они устраивают ее каждый месяц). Отдел маркетинга хочет провести акцию по продвижению старых запасов, чтобы очистить склад. Имеющийся у них код был написан уже давно и неоднократно дополнялся со временем. Он работает и важен для сохранения прибыльности бизнеса.



**Директор
по маркетингу**

Эй! А вы можете проследить за тем, чтобы при добавлении товара в корзину действовала скидка «Черной пятницы»? Кстати, в этом месяце «Черная пятница» уже в ближайшую пятницу.

Ой, я совсем забыла! В этом коде не используется копирование при записи. Не представляю, как мы будем безопасно обмениваться с ним данными.



**Дженна из команды
разработки**

Весь код корзины, которым мы управляли, рассматривал корзину как неизменяемую, для чего использовалось копирование при записи. Однако код распродаж «Черной пятницы» этого не делает — он достаточно часто изменяет корзину. Он был написан несколько лет назад и надежно работает, к тому же у вас просто нет времени возвращаться и переписывать его. Необходимо организовать его безопасное взаимодействие с существующим кодом.

Чтобы запустить распродажу, необходимо добавить в `add_item_to_cart()` следующую строку кода:

```
function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart = add_item(shopping_cart, item);
  var total = calc_total(shopping_cart);
  set_cart_total_dom(total);
  update_shipping_icons(shopping_cart);
  update_tax_dom(total);
  black_friday_promotion(shopping_cart);
}
```

Необходимо добавить
в код эту строку, но она
изменит корзину



Вызов этой функции нарушит схему копирования при записи, а возможности модифицировать код `black_friday_promotion()` у нас нет. К счастью, существует другой механизм, который позволяет безопасно вызвать функцию без нарушения принципа копирования при записи. Он называется *защитным копированием* и используется для обмена данными с кодом, изменяющим данные.



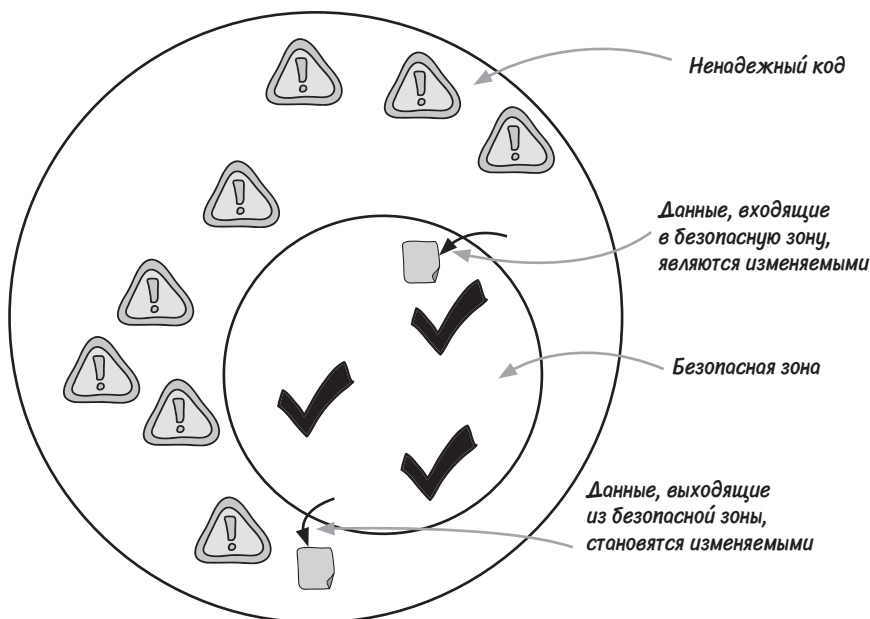
Загляни в словарь

В этой книге термином «*унаследованный код*» обозначается существующий код (возможно, с применением более старых практик), который не может быть заменен в данный момент. Приходится работать с ним в том виде, в котором он доступен.

Наш код копирования при записи должен взаимодействовать с ненадежным кодом

Код распродаж, используемый отделом маркетинга, ненадежен. Мы не доверяем ему, потому что он не реализует принцип неизменяемости на основе копирования при записи, соблюдаемый в нашем коде.

Наш код формирует *безопасную зону*, в которой мы можем быть уверены в том, что все функции поддерживают неизменяемость. При использовании кода, находящегося «внутри круга», можно не напрягаться.



Код распродажи выходит за пределы безопасной зоны, но наш код все равно должен его выполнять. А чтобы выполнить его, необходимо обмениваться с ним данными через операции ввода и вывода.

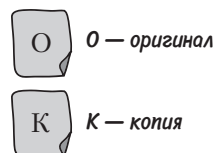
Уточню для ясности: любые данные, выходящие из безопасной зоны, являются потенциально изменяемыми. Они могут быть изменены ненадежным кодом. Аналогичным образом любые данные, входящие в безопасную зону из ненадежного кода, являются потенциально изменяемыми. Ненадежный код может сохранить ссылки на них и изменить после передачи. Вопрос в том, как организовать обмен данными без потери неизменяемости.

Паттерн копирования при записи вам уже знаком, но здесь он не поможет. В паттерне копирования при записи данные копируются перед изменением. Мы точно знаем, какие модификации будут происходить. Мы можем провести анализ того, какие данные необходимо скопировать. С другой стороны, в данном случае код распродажи настолько большой и устрашающий, что мы не знаем, что именно произойдет. Нам понадобится механизм с большим защитным потенциалом, который полностью оградит наши данные от изменения. Он называется *защитным копированием*. Посмотрим, как он работает.

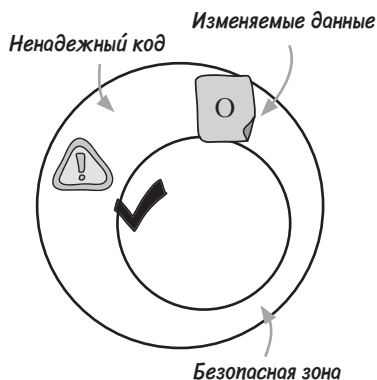
Защитное копирование позволяет сохранить неизменяемый оригинал

Проблема обмена данными с ненадежным кодом решается созданием копий — двух по факту. Посмотрим, как работает эта схема.

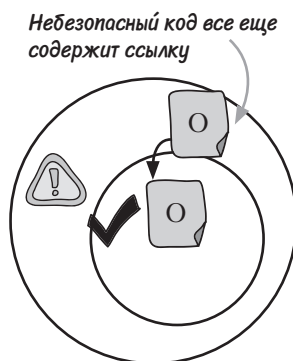
При входе данных в безопасную зону из ненадежного кода нельзя полагать, что данные являются неизменяемыми. Мы немедленно создаем глубокую копию и отбрасываем изменяемый оригинал. Так как только доверенный код располагает ссылкой на эту копию, она является неизменяемой. Таким образом организуется защита при входе данных в безопасную зону.



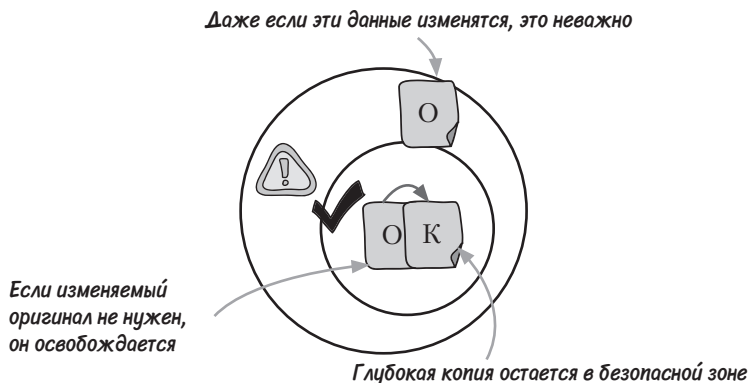
1. Данные в ненадежном коде



2. Данные входят в безопасную зону

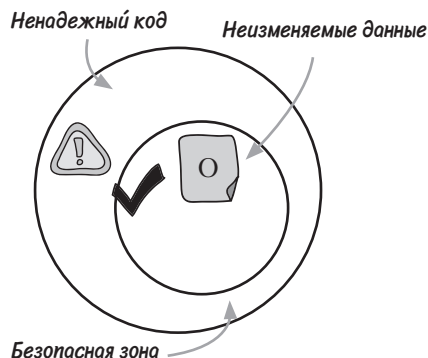


3. Создание глубокой копии

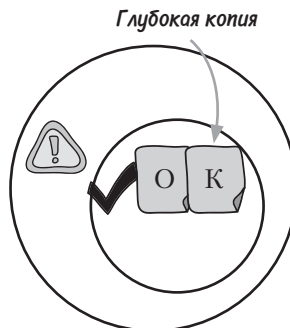


Защита при выходе остается необходимой. Как говорилось ранее, любые данные, покидающие безопасную зону, должны считаться изменяемыми, потому что ненадежный код может изменить их. Проблема решается созданием глубокой копии, передаваемой ненадежному коду.

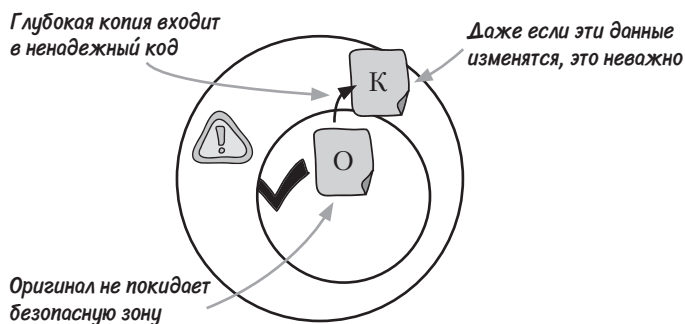
1. Данные в безопасной зоне



2. Создание глубокой копии



3. Глубокая копия выходит из безопасной зоны



Такова суть защитного копирования: вы создаете копии при входе и выходе данных. Цель в том, чтобы неизменяемые оригиналы оставались в безопасной зоне, а изменяемые данные не проникли в нее. Применим этот подход к распродаже «Черной пятницы».

Реализация защитного копирования

Вы хотите вызвать функцию, которая изменяет свой аргумент, но так, чтобы не нарушать так нелегко доставшуюся политику неизменяемости. Для защиты данных и сохранения неизменяемости можно создать защитные копии. Мы используем термин *защитные*, потому что эти копии защищают оригинал от изменений.

`black_friday_promotion()` изменяет свой аргумент, то есть корзину. Мы можем создать глубокую копию корзины и передать копию функции. Тем самым будет предотвращено изменение оригинала.

Оригинал

```
function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart = add_item(shopping_cart,
                           item);
  var total = calc_total(shopping_cart);
  set_cart_total_dom(total);
  update_shipping_icons(shopping_cart);
  update_tax_dom(total);

  black_friday_promotion(shopping_cart);
}
```

Копирование перед передачей данных

```
function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart = add_item(shopping_cart,
                           item);
  var total = calc_total(shopping_cart);
  set_cart_total_dom(total);
  update_shipping_icons(shopping_cart);
  update_tax_dom(total);
  var cart_copy = deepCopy(shopping_cart);
  black_friday_promotion(cart_copy);
}
```

*Данные копируются
при выходе*

Это здорово, но нам нужен результат вызова `black_friday_promotion()`. А результатом являются изменения, внесенные в корзину. К счастью, функция изменяет копию `cart_copy`. Но можем ли мы безопасно использовать `cart_copy`? Является ли она неизменяемой? Что, если `black_friday_promotion()` хранит ссылку на корзину для ее последующего изменения? Такие ошибки обнаруживаются спустя недели, месяцы или даже годы. Проблема решается созданием другой защитной копии при входе данных в наш код.

Копирование перед вызовом небезопасной функции

```
function add_item_to_cart(name, price) {
    var item = make_cart_item(name, price);
    shopping_cart = add_item(shopping_cart,
                             item);

    var total = calc_total(shopping_cart);
    set_cart_total_dom(total);
    update_shipping_icons(shopping_cart);
    update_tax_dom(total);
    var cart_copy = deepCopy(shopping_cart);
    black_friday_promotion(cart_copy);
}
```

Копирование до и после вызова небезопасной функции

```
function add_item_to_cart(name, price) {
    var item = make_cart_item(name, price);
    shopping_cart = add_item(shopping_cart,
                             item);

    var total = calc_total(shopping_cart);
    set_cart_total_dom(total);
    update_shipping_icons(shopping_cart);
    update_tax_dom(total);
    var cart_copy = deepCopy(shopping_cart);
    black_friday_promotion(cart_copy);
    shopping_cart = deepCopy(cart_copy);
}
```

 Копирование данных при входе

Собственно, так работает паттерн защитного копирования. Как было показано выше, безопасность достигается посредством копирования. Вы копируете данные при выходе из вашей системы, а потом еще и тогда, когда они возвращаются обратно.

Создаваемые копии должны быть *глубокими*. Вскоре вы увидите, как реализуется эта разновидность копирования.

Правила защитного копирования

Защитное копирование — механизм, обеспечивающий неизменяемость при обмене данными с кодом, не поддерживающим неизменяемость. Будем называть этот код ненадежным. Действуют два правила:

Правило 1. Копируйте данные, выходящие из вашего кода

Если у вас имеются неизменяемые данные, которые выходят из вашего кода и входят в ненадежный код, выполните следующие действия для защиты оригинала.

1. Создайте глубокую копию неизменяемых данных.
2. Передайте копию ненадежному коду.

Правило 2. Копируйте данные, входящие в ваш код

Данные, получаемые из ненадежного кода, могут оказаться изменяемыми. Выполните следующие действия.

1. Немедленно создайте глубокую копию изменяемых данных, переданных вашему коду.
2. Используйте копию в своем коде.

Выполняя эти два правила, можно взаимодействовать с любым кодом, которому вы не доверяете, без нарушения принципа неизменяемости.

Следует помнить, что эти правила могут применяться в любом порядке. Иногда вы передаете данные наружу, после чего они возвращаются обратно. Это происходит при вызове из вашего кода функции, входящей в ненадежную библиотеку.

С другой стороны, иногда вы получаете данные еще до их отправки. Например, это может произойти при вызове функции вашего кода из ненадежного кода, словно ваш код является частью общей библиотеки. При этом необходимо помнить, что два правила могут применяться в любом порядке.

Мы реализуем защитное копирование несколько раз. Но прежде чем переходить к другой реализации, продолжим работу над кодом, приведенным для распродажи «Черной пятницы». Его еще можно улучшить.

Также обратите внимание на то, что в некоторых ситуациях нет входных или выходных данных для копирования.



Загляни в словарь

Глубокое копирование дублирует все уровни вложенных структур данных, от верхнего уровня до самого нижнего.

Упаковка ненадежного кода

Мы успешно реализовали защитное копирование, но код остается несколько непонятным из-за всего происходящего копирования. Кроме того, нам придется много раз вызывать `black_friday_promotion()` в будущем. Не хотелось бы допустить ошибку при реализации защитного копирования. Упакуем вызов функции в новую функцию, которая также реализует защитное копирование.

А еще нам снова придется вызывать эту функцию через месяц. Давайте упакуем ее для безопасности вызова.



Ким из команды разработки

Оригинал

```
function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart = add_item(shopping_cart,
                           item);

  var total = calc_total(shopping_cart);
  set_cart_total_dom(total);
  update_shipping_icons(shopping_cart);
  update_tax_dom(total);
  var cart_copy = deepCopy(shopping_cart);
  black_friday_promotion(cart_copy);
  shopping_cart =
    deepCopy(cart_copy);
}
```

Этот код выделяется
в новую функцию

Выделенная безопасная версия

```
function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart = add_item(shopping_cart,
                           item);

  var total = calc_total(shopping_cart);
  set_cart_total_dom(total);
  update_shipping_icons(shopping_cart);
  update_tax_dom(total);
  shopping_cart =
    black_friday_promotion_safe
      (shopping_cart);
}

function black_friday_promotion_safe
  (cart) {
  var cart_copy = deepCopy(cart);
  black_friday_promotion(cart_copy);
  return deepCopy(cart_copy);
}
```

Теперь мы можем вызывать `black_friday_promotion_safe()` без малейших опасений. Функция защищает наши данные от изменения. И теперь становится намного удобнее и проще разобраться в том, что же происходит в коде. Рассмотрим еще один пример.



Ваш ход

MegaMart использует стороннюю библиотеку для расчета платежных ведомостей. Функции `payrollCalc()` передается массив записей о работниках, и она возвращает массив чеков на получение заработной платы. Код определенно является ненадежным. Вероятно, массив работников будет изменяться, и кто знает, что произойдет с чеками?

Ваша задача — упаковать код в функцию, которая обеспечивает безопасность передачи данных за счет использования защитных копий. Сигнатура `payrollCalc()` выглядит так:

```
function payrollCalc(employees) {
  ...
  return payrollChecks;
}
```

Создание версии этой функции с защитным копированием

Напишите функцию-обертку `payrollCalcSafe()`.

```
function payrollCalcSafe(employees) {
```

Запишите здесь свою реализацию

```
}
```



Ответ

```
function payrollCalcSafe(employees) {
  var copy = deepCopy(employees);
  var payrollChecks = payrollCalc(copy);
  return deepCopy(payrollChecks);
}
```



Ваш ход

MegaMart использует другую унаследованную систему, которая предоставляет данные о пользователях программной системы. Вы подписываетесь на обновления информации о пользователях, изменяющих свои настройки.

Но тут есть одна загвоздка: все части кода, которые подписываются на обновления, получают одни и те же данные пользователей. Все ссылки относятся к одним и тем же объектам в памяти. Очевидно, что информация о пользователях поступает из ненадежного кода. Ваша задача — защитить безопасную зону посредством защитного копирования.

Обратите внимание: никакие данные в небезопасную зону не возвращаются — есть только входная изменяемая информация о пользователях.

Вызов функции выглядит так:

```

    userChanges.subscribe(function(user) {
        processUser(user);
    });
  
```

Передается функция обратного вызова

При каждом изменении информации пользователей эта функция будет вызываться с обновленной информацией

Всем обратным вызовам будет передаваться ссылка на один и тот же изменяемый объект

Запишите здесь свою реализацию защитного копирования

Представьте, что это важная функция в вашей безопасной зоне. Защитите ее!

Правила защитного копирования

1. Копируйте данные, выходящие из надежного кода.
2. Копируйте данные, входящие в надежный код.



Ответ

```

userChanges.subscribe(function(user) {
    var userCopy = deepCopy(user);
    procssUser(userCopy);
});
  
```

Снова копировать не нужно, потому что из безопасной зоны никакие данные не выходят

Защитное копирование, которое вам может быть знакомо

Защитное копирование — стандартный паттерн, который можно встретить и за пределами традиционных мест его применения. Впрочем, чтобы разглядеть его, иногда приходится основательно напрячь зрение.

Защитное копирование в программных интерфейсах (API) веб-приложений

Многие API веб-приложений выполняют неявное защитное копирование. Возможный сценарий может выглядеть так, как описано ниже.

Веб-запрос поступает API в формате JSON. Разметка JSON определяет *глубокую копию* данных от клиента, сериализованную для передачи по интернету. Написанный вами сервис выполняет свою работу, после чего отправляет ответ обратно в виде сериализованной глубокой копии, также в формате JSON. В этой ситуации осуществляется копирование данных на входе и на выходе.

По сути здесь выполняется защитное копирование. Одно из преимуществ систем, основанных на сервисах или микросервисах, заключается в том, что сервисы выполняют защитное копирование при взаимодействии друг с другом. Сервисы, использующие разные практики и механизмы, могут взаимодействовать без малейших проблем.



Загляни в словарь

Когда модули реализуют защитное копирование при взаимодействии друг с другом, это часто называется *архитектурой без совместно используемых ресурсов* (shared nothing architecture), потому что модули не используют общие ссылки на какие-либо данные. Вы же не хотите, чтобы ваш код копирования при записи использовал ссылки совместно с ненадежным кодом.

Защитное копирование в Erlang и Elixir

Erlang и Elixir (два языка функционального программирования) также реализуют защитное копирование. Когда два процесса в Erlang передают сообщения друг другу, сообщение (данные) копируется в приемник получателя. Данные копируются на входе в процесс и на выходе из него. Защитное копирование играет ключевую роль в высокой надежности систем Erlang.

За дополнительной информацией о Erlang и Elixir обращайтесь по адресам <https://www.erlang.org> и <https://elixir-lang.org>.

Мы можем пользоваться преимуществами, предоставляемыми микросервисами и Erlang, в своих модулях.



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Погодите! А одновременное существование двух копий пользовательских данных — это нормально? Какая из двух копий «настоящая», представляющая пользователя?

О: Отличный вопрос. Это одно из концептуальных изменений, через которые проходят люди при изучении функционального программирования. Многие люди привыкли к тому, что у них есть пользовательский объект, представляющий пользователя, и существование двух копий одного объекта выглядит странно. Какая же из них представляет пользователя?

В функциональном программировании мы не представляем пользователя, а сохраняем и обрабатываем данные, относящиеся к пользователю. Вспомните определение данных: факты, относящиеся к событиям. Мы сохраняем такие факты, как имя пользователя или инициализированные им события (например, отправка формы). Эти факты можно копировать столько раз, сколько вы сочтете нужным.

В: Копирование при записи и защитное копирование выглядят очень похоже. Они чем-то отличаются? Нам действительно нужно и то и другое?

О: Оба подхода — копирование при записи и защитное копирование — используются для обеспечения неизменяемости, и может показаться, что нам достаточно чего-то одного. На самом деле действительно можно обойтись только защитным копированием, даже внутри безопасной зоны. Оно прекрасно обеспечит неизменяемость.

Однако защитное копирование создает глубокие копии. Глубокое копирование обходится намного дороже поверхностного, потому что оно копирует всю вложенную структуру данных от верха до низа. Если вы доверяете коду, которому передаются данные, такой избыток копий не обязателен. Следовательно, чтобы сэкономить на обработке и затратах памяти для всех этих копий, стоит использовать копирование при записи там, где это возможно, то есть везде в безопасной зоне. Два подхода работают совместно.

Важно сравнить два подхода, чтобы вы лучше понимали, в какой ситуации стоит применять каждый из них. Давайте займемся этим.

Сравнение копирования при записи с защитным копированием

Копирование при записи

Когда используется

Используйте копирование при записи, если вам требуется изменить данные, находящиеся под вашим контролем.

Где используется

В безопасной зоне копирование при записи следует использовать везде. Собственно, копирование при записи определяет вашу безопасную зону неизменяемости.

Тип копирования

Поверхностное копирование обходится относительно невысокими затратами.

Правила

1. Создайте поверхностную копию изменяемых данных.
2. Измените копию.
3. Верните копию.

Защитное копирование

Когда используется

Используйте защитное копирование при обмене данными с ненадежным кодом.

Где используется

Используйте защитное копирование на границах вашей безопасной зоны для данных, которые входят или выходят за эту границу.

Тип копирования

Глубокое копирование относительно затратное.

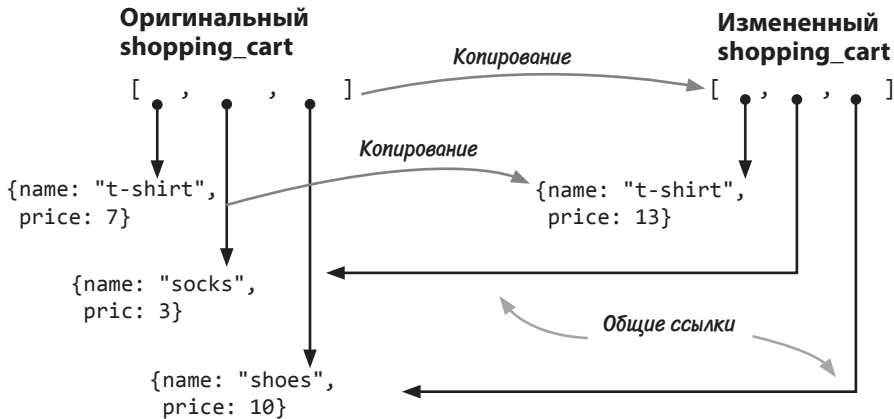
Правила

1. Создайте глубокую копию данных, входящих в безопасную зону.
2. Создайте глубокую копию данных, выходящих из безопасной зоны.

Глубокое копирование затратнее поверхностного

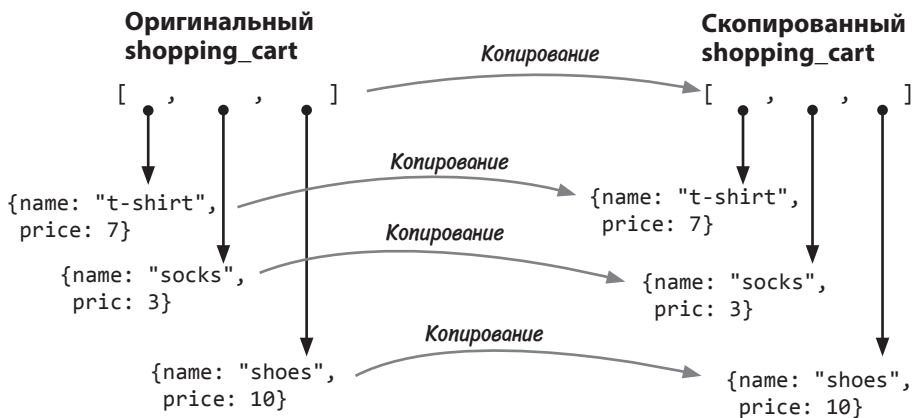
Поверхностное копирование отличается от глубокого тем, что у глубокого копирования отсутствует совместное использование данных с оригиналом. Копируются все вложенные объекты и массивы. При поверхностном копировании все, что не изменилось, может использоваться совместно.

Поверхностное копирование



При глубоком копировании создаются копии всех элементов данных. Мы используем глубокое копирование, потому что не уверены, что какие-либо элементы данных будут рассматриваться как неизменяемые в ненадежном коде.

Глубокое копирование



Очевидно, что глубокое копирование обходится дороже. Именно поэтому мы не применяем его везде, а только там, где не можем гарантировать соблюдение подхода копирования при записи.

Трудности реализации глубокого копирования в JavaScript

Глубокое копирование — простая идея, которая вроде бы должна иметь простую реализацию. Тем не менее в JavaScript ее сложно реализовать из-за отсутствия хорошей стандартной библиотеки. Надежная реализация глубокого копирования выходит за рамки темы книги.

Я рекомендую использовать реализацию из библиотеки **Lodash**. Если говорить конкретнее, функция `_.cloneDeep()` выполняет глубокое копирование вложенных структур данных. Этой библиотеке доверяют тысячи, если не миллионы разработчиков JavaScript.

Тем не менее для полноты ниже приведена простая реализация, которая может удовлетворить ваше любопытство. Она должна работать для всех функций и типов, допустимых для JSON.

```
function deepCopy(thing) {
  if(Array.isArray(thing)) {
    var copy = [];
    for(var i = 0; i < thing.length; i++)
      copy.push(deepCopy(thing[i]));
    return copy;
  } else if (thing === null) {
    return null;
  } else if (typeof thing === "object") {
    var copy = {};
    var keys = Object.keys(thing);
    for(var i = 0; i < keys.length; i++) {
      var key = keys[i];
      copy[key] = deepCopy(thing[key]);
    }
    return copy;
  } else {
    return thing;
  }
}
```

Рекурсивное создание копий всех элементов

Строки, числа, логические значения и функции являются неизменяемыми, так что копировать их не нужно

Эта функция не учитывает всех особенностей JavaScript. Существует много других типов, для которых эта схема работать не будет. Однако как общая схема того, что необходимо сделать, она работает неплохо. Данная реализация показывает, что массивы и объекты должны копироваться, но функция также рекурсивно обрабатывает все элементы этих коллекций.

Я настоятельно рекомендую использовать надежную реализацию глубокого копирования из популярной библиотеки JavaScript, такую как Lodash. Функция глубокого копирования приведена здесь только для учебных целей, в условиях реальной эксплуатации она работать не будет.



Ваш ход

Ниже приведены утверждения, относящиеся к двум типам копирования: поверхностному и глубокому. Одни утверждения истинны для глубокого копирования, другие — для поверхностного. А некоторые истинны для обоих типов! Поставьте пометку ГК рядом с утверждениями, относящимися к глубокому копированию, и пометку ПК — к поверхностному копированию.

1. Копирует все уровни вложенной структуры.
2. Намного эффективнее другого, потому что две копии могут совместно использовать структуру.
3. Копирует только изменяющиеся части.
4. Поскольку копии не используют структуру совместно, хорошо подходит для защиты оригинала из ненадежного кода.

Пригодится для реализации архитектуры без совместно используемых ресурсов.

Условные обозначения

ГК Глубокое копирование

ПК Поверхностное копирование



Ответ

1. ГК. 2. ПК. 3. ПК. 4. ГК. 5. ГК.

Диалог между копированием при записи и защитным копированием

Тема: какой подход важнее?

Копирование при записи:

Разумеется, я важнее. Я помогаю людям обеспечить неизменяемость своих данных.

Мои поверхностные копии намного эффективнее твоих глубоких.

Я это и хотело сказать! Без меня безопасной зоны вообще не было бы.

Конечно, меня бы стоило использовать и в этих унаследованных кодовых базах и библиотеках. От такого подхода, как я, люди могли бы узнать много полезного. Они научатся преобразовывать свои операции записи в чтение, а чтение естественным образом превращается в вычисления.

Ты право! *(Всхлипывая)* Я должно взглянуть в лицо фактам. Без тебя от меня не будет никакой пользы!

Защитное копирование:

Но это не значит, что ты важнее. Я тоже обеспечиваю неизменяемость данных.

Для тебя это важно, потому что тебе приходится создавать копию КАЖДЫЙ РАЗ при изменении данных. А мне достаточно создавать копии только при входе или выходе данных из безопасной зоны.

Что ж, с этим трудно не согласиться. Но от твоей безопасной зоны не было бы никакой пользы, если бы она не могла передавать данные наружу, туда, где находится весь существующий код и библиотеки.

Послушай, этого никогда не будет. Просто прими это. В мире слишком много кода. В нем никогда не найдется достаточно программистов, чтобы переписать его.

Ты меня прямо растрогало. *(Непрошенная слеза)* Я без тебя тоже не проживу.

(Дружеские объятия)

Двигаемся дальше...



Ваш ход

Ниже приведены утверждения, относящиеся к принципу неизменяемости. Одни утверждения истинны для защитного копирования, другие — для копирования при записи. А некоторые истинны для обоих типов! Поставьте пометку ЗК рядом с утверждениями, относящимися к защитному копированию, и пометку КЗ — к копированию при записи.

1. Создает глубокие копии.
2. Требуется меньших затрат, чем другая.
3. Является важным способом обеспечения неизменяемости.
4. Копирует данные перед изменением копии.
5. Используется внутри безопасной зоны для обеспечения неизменяемости.
6. Используется в ситуациях, когда вы хотите провести обмен данными с ненадежным кодом.
7. Полноценное решение для обеспечения неизменяемости. Может использоваться без второго.
8. Использует глубокое копирование.
9. Копирует данные перед их передачей ненадежному коду.
10. Копирует данные, полученные из ненадежного кода.

Условные обозначения

ЗК Защитное копирование

КЗ Копирование при записи



Ответ

1. ЗК. 2. КЗ. 3. КЗ и ЗК. 4. КЗ. 5. КЗ. 6. ЗК. 7. ЗК. 8. КЗ.
9. ЗК. 10. ЗК.



Ваш ход

Ваша команда недавно начала использовать подход копирования при записи для создания безопасной зоны. Каждый раз, когда ваша команда пишет новый код, она принимает меры для обеспечения его неизменяемости. Новая задача требует от вас написания кода, взаимодействующего с существующим кодом, который не обеспечивает неизменяемости. Какой из следующих порядков действий обеспечит сохранение неизменяемости? Пометьте все утверждения, действующие в вашем случае. Обоснуйте свои ответы.

1. Защитное копирование используется при обмене данными с существующим кодом.
2. Копирование при записи используется при обмене данными с существующим кодом.
3. Следует прочитать исходники существующего кода, чтобы узнать, изменяет ли он данные. Если данные не изменяются, то никакой специальный подход не нужен.
4. Существующий код переписывается с использованием копирования при записи, и переписанный код вызывается без защитного копирования.
5. Код принадлежит вашей команде, поэтому он уже является частью безопасной зоны.



Ответ

1. Да. Защитное копирование защищает вашу безопасную зону за счет памяти и работы, необходимой для создания копий.
2. Нет. Копирование при записи работает только в том случае, если вы вызываете другие функции, реализующие копирование при записи. Если вы не уверены, то скорее всего, оно не реализовано в существующем коде.
3. Возможно. При анализе исходного кода может выясниться, что он не изменяет передаваемые ему данные. Однако также необходимо обращать внимание на другие потенциально опасные операции, которые он может выполнять: например, передачу данных третьей стороне.
4. Да. Если вы можете себе это позволить, переписанный код с использованием копирования при записи решит проблему.
5. Нет. Принадлежность кода сама по себе не означает, что в нем соблюдается принцип неизменяемости.

Итоги главы

В этой главе был представлен более мощный, хотя и более затратный подход обеспечения неизменяемости — *защитное копирование*. Оно более мощное, потому что полностью обеспечивает неизменяемость само по себе. Оно более затратное, потому что вам придется копировать больше данных. Тем не менее использование защитного копирования в сочетании с копированием при записи открывает доступ к преимуществам обоих подходов: производительности там, где она необходима, и поверхностному копированию ради эффективности.

Резюме

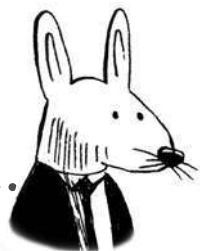
- Защитное копирование — механизм реализации неизменяемости, основанный на создании копий при входе или выходе данных из вашего кода.
- Защитное копирование создает глубокие копии, поэтому оно более затратно по сравнению с копированием при записи.
- В отличие от копирования при записи, защитное копирование может защитить ваши данные от кода, не реализующего принцип неизменяемости.
- Копирование при записи часто является предпочтительным, потому что не требует создания такого количества копий. Защитное копирование применяется только тогда, когда появляется необходимость взаимодействия с ненадежным кодом.
- При глубоком копировании копируется вся вложенная структура сверху донизу. Поверхностное копирование ограничивается минимальным копированием.

Что дальше?

В следующей главе мы соберем воедино все, что вы узнали к настоящему моменту, и определим метод организации кода для улучшения архитектуры системы.

8

Многоуровневое проектирование: часть 1



В этой главе

- ✓ Рабочее определение проектирования программного обеспечения.
- ✓ Многоуровневое (стратифицированное) проектирование и его потенциал для упрощения работы вашей команды.
- ✓ Выделение функций для того, чтобы код стал более понятным.
- ✓ Почему построение программной системы по уровням делает мышление более эффективным?

Мы подошли к последним главам части I, и теперь пришло время взглянуть на общую картину. Мы применим практику проектирования, которая называется многоуровневым проектированием. В многоуровневом проектировании функции пишутся в выражениях функций, определенных на нижних уровнях. Что такое уровни? Для чего они нужны? Ответы на эти вопросы будут даны в этой и следующей главах. А концепции, которые вы узнаете, станут залогом нашего успеха в части II.

Что такое проектирование программной системы

Программисты в компании MegaMart понимают, насколько важна корзина для их программного продукта. Но они считают, что ее реализация оставляет желать лучшего.



Разочарование Дженны — это важный сигнал того, что что-то пошло не так. Хорошая архитектура должна оставлять хорошее впечатление. Она должна упрощать усилия ваших разработчиков на всем цикле разработки: от замысла до программирования, тестирования и сопровождения.

Собственно, это выглядит как хорошее рабочее определение проектирования программной системы в том контексте, в котором оно будет рассматриваться в книге.

проектирование программной системы, суц.

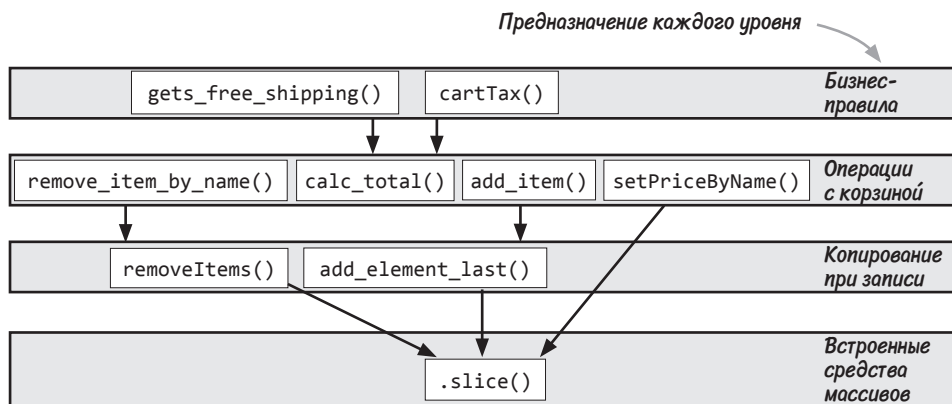
Применение эстетических представлений для принятия решений из области программирования, упрощающих программирование, тестирование и сопровождение продукта.

Мне не хочется вступать в ожесточенные дебаты относительно того, что такое проектирование. И вы не обязаны соглашаться с этим определением. Тем не менее полезно знать определение, которое будет использоваться в книге.

В этой главе мы будем оттачивать свои эстетические представления, используя практику *многоуровневого проектирования*. Итак, за дело!

Что такое многоуровневое проектирование

Многоуровневое проектирование — метод построения программного кода по уровням. Каждый уровень определяет новые функции в контексте функций уровней, расположенных ниже него. Тренируя свои навыки проектирования, можно найти конфигурацию уровней, которая обеспечит гибкость вашей системы перед изменениями, сделает ее удобочитаемой, простой в тестировании и намного более приспособленной для повторного использования. Вот что мы имеем в виду под *уровнями*:



Заявляю: определить эти уровни непросто. Необходимо знать, на что смотреть и что делать с тем, что вы обнаружите. В процессе задействовано слишком много факторов, чтобы можно было предложить абсолютную формулу «оптимального проектирования». Тем не менее мы можем развить у себя чувство качественного проектирования и следовать по тому пути, по которому это чувство нас поведет.

Как развить чувство хорошего проектирования? Это трудно объяснить, но в этой главе мы будем использовать следующий подход: мы будем учиться читать свой код и искать в нем сигналы, которые показывают, где можно улучшить архитектуру системы. И мы также внесем улучшения, чтобы вы видели, как это делается. После усердной работы (и небольшого везения) к концу главы вы начнете понимать принципы хорошего проектирования, и это понимание можно будет совершенствовать в последующие годы.



Загляни в словарь

Многоуровневое проектирование — метод проектирования, при котором программная система строится по уровням. Эта практика имеет долгие исторические корни, в ее развитие внесли вклад очень многие люди. Впрочем, я хочу особо поблагодарить Гарольда Абельсона (Harold Abelson) и Джеральда Зюссмана (Gerald Sussman) за документирование наблюдений и их практическое воплощение.

Развитие чувства проектирования

Проклятие эксперта

Всем известно, что эксперты плохо умеют объяснять то, что они делают, хотя и очевидно, что они очень хорошо знают свою работу. У них есть прекрасно проработанная модель, но нет средств для описания этой модели для других. Это иногда называют «проклятием эксперта»: даже если вы хорошо что-то умеете, вы не можете объяснить, что именно вы делаете. Решение становится своего рода «черным ящиком».

Одна из причин, по которым эксперты не могут объяснить суть своего «черного ящика», связана со сложностью их навыков. «Черный ящик» получает множество разнообразных входных данных и выдает столь же разнообразные выходные данные.

Ввод в контексте многоуровневого проектирования

Входные данные для многоуровневого проектирования можно рассматривать как *ориентиры*. Мы читаем код, ищем ориентиры и используем их для определения своих действий.

Некоторые источники ориентиров:

*Не беспокойтесь, если вы еще не понимаете эти термины.
Мы доберемся до них в этой и следующей главе*

Тела функций	Структура уровня	Сигнатуры функций
● Длина. ● Сложность. ● Уровень детализации. ● Вызываемые функции. ● Используемые языковые средства.	● Длина стрелки. ● Связность. ● Уровень детализации.	● Имя функции. ● Имена аргументов. ● Значения аргументов. ● Возвращаемое значение.

Вывод в контексте многоуровневого проектирования

После выявления всех этих входных данных мы как-то объединяем их в голове. Помните: даже эксперты не могут точно объяснить, что они делают. Но люди каким-то образом учатся объединять их в сложные решения и действия в коде. Эти решения и действия могут существовать во многих формах:

Организация	Реализация	Изменения
● Решение о том, где должна располагаться новая функция. ● Перемещение функций.	● Изменение реализации. ● Выделение функции. ● Изменение структуры данных.	● Выбор места, в котором пишется новый код. ● Выбор подходящего уровня детализации.

Мы смотрим на свой код под разными углами и применяем к нему паттерны многоуровневого проектирования. Если повезет, ваш мозг сделает то, с чем он справляется лучше всего (поиск закономерностей), и начнет видеть паттерны так, как их видит эксперт.

Паттерны многоуровневого проектирования

Мы будем рассматривать многоуровневое проектирование со многих позиций. Тем не менее в этой и следующей главе мы будем выделять в нем четыре паттерна. Паттерн 1 рассматривается в этой главе, а остальные — в следующей.

Паттерн 1. Прямолинейная реализация

Структура уровней при многоуровневом проектировании должна помочь нам в построении прямолинейных реализаций. Когда мы читаем функцию с прямолинейной реализацией, задача, представленная сигнатурой функции, должна решаться на правильном уровне детализации в теле. Слишком много подробностей — признак «кода с душком».

Паттерн 2. Абстрактный барьер

Некоторые уровни предоставляют интерфейс, который позволяет скрыть важные подробности реализации. Эти уровни помогают нам писать код на более высоком уровне и освобождают наши ограниченные умственные возможности, чтобы они могли работать на этом уровне.

Паттерн 3. Минимальный интерфейс

По мере развития системы интерфейсы, относящиеся к важным бизнес-концепциям, должны сходиться к небольшому мощному набору операций. Все остальные операции должны определяться в понятиях этих операций (прямо или косвенно).

Паттерн 4. Удобные уровни

Паттерны и практики многоуровневого проектирования должны служить нашим потребностям как программистов, которые, в свою очередь, должны обслуживать бизнес. Необходимо выделить время на уровни, которые помогут нам выдавать программный продукт быстрее и с более высоким качеством. Уровни не должны добавляться для забавы. С кодом и его уровнями абстракции должно быть удобно работать.

Эти паттерны абстрактны; сделаем их конкретными. Сценарии, диаграммы, объяснения и упражнения помогут вам лучше понять особенности многоуровневого проектирования. Начнем с паттерна 1.

Паттерн 1. Прямолинейная реализация

В этом разделе вы увидите, как написать код реализации, который легко читается. Благодаря структуре уровней вашей архитектуры даже самые мощные функции должны выражать то, что они делают, без лишней сложности.

Вернемся к тому, что говорила Дженна несколько страниц назад.

Паттерны

- ☒ Прямолинейная реализация
- ☐ Абстрактный барьер
- ☐ Минимальный интерфейс
- ☐ Удобные уровни

Вы находитесь
здесь

Да, я согласна с Дженной. Приведу пример из части кода, где при покупке галстука выдается бесплатный зажим для галстука.

Я считаю, что корзина плохо реализована! Каждый раз, когда мне приходится пользоваться корзиной, я боюсь что-нибудь сломать.



Сара
из команды
разработки

```
function freeTieClip(cart) {
  var hasTie = false;
  var hasTieClip = false;
  for(var i = 0; i < cart.length; i++) {
    var item = cart[i];
    if(item.name === "tie")
      hasTie = true;
    if(item.name === "tie clip")
      hasTieClip = true;
  }
  if(hasTie && !hasTieClip) {
    var tieClip = make_item("tie clip", 0);
    return add_item(cart, tieClip);
  }
  return cart;
}
```

Проверяем, содержит
ли корзина галстук
или зажим для
галстука

Добавляем
зажим



Дженна из команды
разработки

Разобраться в логике этого кода нетрудно, но в нем много функций, которые перебирают содержимое корзины, проверяют товары и принимают некое решение. Эти функции проектировались для конкретной ситуации. Программист не следует никаким принципам проектирования: он просто решает имеющуюся задачу (добавление зажима), пользуясь своими знаниями корзины (реализованной в виде массива). Многократно писать подобный код утомительно, к тому же он создает проблемы с сопровождением.

Код не следует паттерну 1 «Прямолинейная реализация». Он полон подробностей, неактуальных на этом уровне мышления. Почему в процессе маркетинговой кампании нужно знать, что корзина реализована в виде массива? Будет ли распродажа обречена на неудачу, если при переборе корзины произойдет классическая ошибка «смещения на 1» (off-by-one error)?

Операции с корзиной

Команда решает провести мозговой штурм, чтобы привести корзину в нормальное состояние. Руководствуясь своим знанием кода, команда сначала составляет список операций, которые должны поддерживаться корзиной. По этому списку можно получить некоторое представление о том, как должен выглядеть ваш код. Со временем эти операции заменят уже написанный ситуативный код.

Слева перечислены операции, которые команда хотела бы иметь в своем распоряжении; уже реализованные операции помечены галочкой. Справа приведен код уже реализованных операций.

Паттерны

- ☐ Прямой реализация
- ☐ Абстрактный барьер
- ☐ Минимальный интерфейс
- ☐ Удобные уровни



Остались еще две нереализованные операции. Вскоре мы ими займемся.

Проверка наличия товара в корзине может быть полезной

Список всех требуемых операций позволяет Ким взглянуть на ситуацию под новым углом, и она видит возможность упростить реализацию `freeTieClip()`.

```
function freeTieClip(cart) {
  var hasTie = false
  var hasTieClip = false;
  for(var i = 0; i < cart.length; i++) {
    var item = cart[i];
    if(item.name === "tie")
      hasTie = true;
    if(item.name === "tie clip")
      hasTieClip = true;
  }
  if(hasTie && !hasTieClip) {
    var tieClip = make_item("tie clip", 0);
    return add_item(cart, tieClip);
  }
  return cart;
}
```

Постойте! Одна из отсутствующих функций, которая проверяет наличие товара в корзине, сделает реализацию `freeTieClip()` более понятной.

Этот цикл `for` просто проверяет, присутствуют ли два товара в корзине



Ким
из команды
разработки

Если бы у вас была функция для проверки наличия товара в корзине, то этот низкоуровневый цикл `for` можно было заменить ею. Низкоуровневый код всегда оказывается хорошим кандидатом для рефакторинга. Проблема в том, что в данном случае проверяются два разных товара, поэтому функцию придется вызвать дважды.

```
function freeTieClip(cart) {
  var hasTie = false;
  var hasTieClip = false;
  for(var i = 0; i < cart.length; i++) {
    var item = cart[i];
    if(item.name === "tie")
      hasTie = true;
    if(item.name === "tie clip")
      hasTieClip = true;
  }
  if(hasTie && !hasTieClip) {
    var tieClip = make_item("tie clip", 0);
    return add_item(cart, tieClip);
  }
  return cart;
}
```

Цикл `for` выделяется в функцию

```
function freeTieClip(cart) {
  var hasTie = isInCart(cart, "tie");
  var hasTieClip = isInCart(cart, "tie clip");
```

```
  if(hasTie && !hasTieClip) {
    var tieClip = make_item("tie clip", 0);
    return add_item(cart, tieClip);
  }
  return cart;
}
```

```
function isInCart(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      return true;
  }
  return false;
}
```

Новая реализация стала короче, что сделало ее более понятной. Кроме того, новая реализация лучше читается, потому что все происходит на примерно одном уровне детализации.

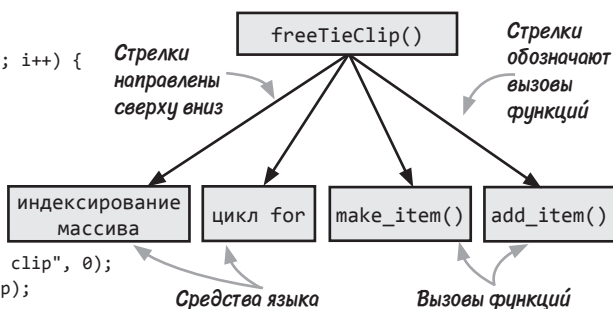
Визуализируем нашу функцию с помощью графа вызовов

Взглянем на исходную реализацию функции `freeTieClip()` под другим углом. Мы можем прочесть, какие функции она вызывает или какие средства языка использует, и нарисовать схему, которая называется *графом вызовов*. В данном случае нужно особо выделить цикл `for` и индексирование массива, поэтому мы включим их.

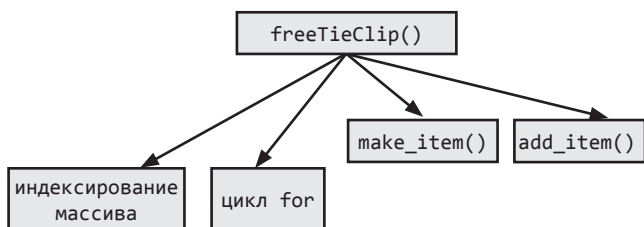
Код

```
function freeTieClip(cart) {
  var hasTie = false;
  var hasTieClip = false;
  for(var i = 0; i < cart.length; i++) {
    var item = cart[i];
    if(item.name === "tie")
      hasTie = true;
    if(item.name === "tie clip")
      hasTieClip = true;
  }
  if(hasTie && !hasTieClip) {
    var tieClip = make_item("tie clip", 0);
    return add_item(cart, tieClip);
  }
  return cart;
}
```

Диаграмма



Относятся ли все прямоугольники на нижнем уровне к одному уровню абстракции? Нет. Трудно представить, чтобы написанные нами функции (`make_item()` и `add_item()`) находились на одном уровне со встроенными средствами языка (цикл `for` и индексирование массива). Индексирование массива и циклы `for` должны находиться на более низком уровне. Выразим этот факт на диаграмме:



Паттерны

- ☐ Прямолинейная реализация
- ☐ Абстрактный барьер
- ☐ Минимальный интерфейс
- ☐ Удобные уровни

Теперь на диаграмме показано то, что мы почувствовали перед чтением кода: `freeTieClip()` реализуется частями, охватывающими разные уровни абстракции. Это хорошо заметно, так как стрелки указывают на разные уровни. Различия в уровнях делают реализацию менее очевидной и сложной для восприятия. Как наша улучшенная реализация будет смотреться в виде графа вызовов?

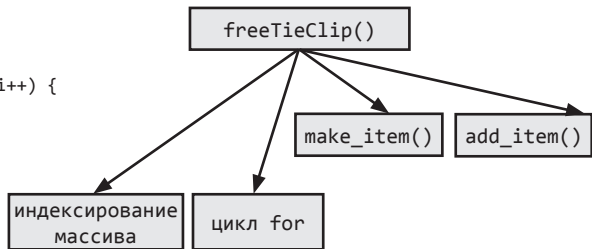
Прямой реализации вызывают функции примерно одного уровня абстракции

Из графа вызовов `freeTieClip()` было видно, что данная реализация не является прямой, так как она вызывает функции с сильно различающихся уровней абстракции. То же самое ощущение возникает при виде кода.

Код

```
function freeTieClip(cart) {
  var hasTie = false;
  var hasTieClip = false;
  for(var i = 0; i < cart.length; i++) {
    var item = cart[i];
    if(item.name === "tie")
      hasTie = true;
    if(item.name === "tie clip")
      hasTieClip = true;
  }
  if(hasTie && !hasTieClip) {
    var tieClip = make_item("tie clip", 0);
    return add_item(cart, tieClip);
  }
  return cart;
}
```

Диаграмма

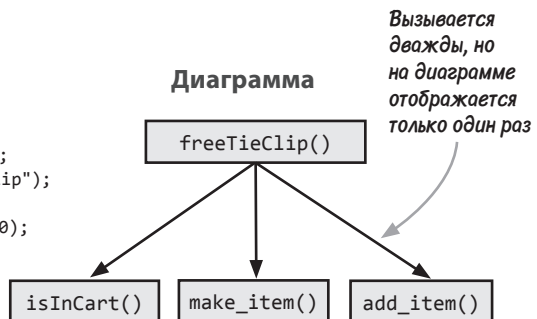


Теперь посмотрим, как выглядит новая реализация. Пожалуй, этот код более прямой. Вызываемые им функции находятся на близких уровнях абстракции.

Код

```
function freeTieClip(cart) {
  var hasTie = is_in_cart(cart, "tie");
  var hasTieClip = is_in_cart(cart, "tie clip");
  if(hasTie && !hasTieClip) {
    var tieClip = make_item("tie clip", 0);
    return add_item(cart, tieClip);
  }
  return cart;
}
```

Диаграмма



В самом деле, функции, вызываемые `freeTieClip()`, находятся на одинаковом (или почти одинаковом) уровне абстракции. Если сказанное пока не очень ясно, не огорчайтесь. Мы рассмотрим другие точки зрения, с которыми этот момент будет более понятен. А пока просто спросите себя: что нужно знать о корзине, чтобы вызывать эти функции? Нужно ли знать, что она реализована в виде массива?

Не нужно. Если вы пользуетесь этими функциями, можно полностью забыть о том, что это массив. Возможность игнорировать одни и те же подробности — один из признаков того, что они находятся на одном уровне абстракции. А раз они находятся на одном уровне, значит, реализация прямолинейна.



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: А зачем смотреть на граф вызовов? Я вижу проблемы прямо в коде.

О: Хороший вопрос. В данном случае код очевидно не был прямолинейным. Граф вызовов только подтвердил это. С этой точки зрения граф не был нужен.

Но в данный момент на графе всего два уровня. С добавлением новых функций появятся новые уровни, а глобальное представление вашего кода будет очень полезным — оно покажет структуру уровней в контексте системы. Трудно получить эту информацию из маленького фрагмента кода, который можно просмотреть за один раз. Эта структура уровней станет бесценным сигналом для формирования вашего чувства проектирования.

В: Вы действительно рисуете все эти диаграммы?

О: Очень хороший вопрос. Как правило, мы их не рисуем, а представляем. Когда вы хорошо освоите работу с диаграммами, вы сможете «рисовать» их в своем воображении.

Тем не менее общая диаграмма (например, нарисованная маркером на доске) будет отличным средством коммуникации. Обсуждения процесса проектирования нередко становятся излишне абстрактными. Изображение может дать конкретные ориентиры, на которые можно ссылаться, чтобы избежать этой проблемы.

В: Все эти уровни реальны? Это объективные составляющие структуры, с определением которых все согласны?



Отдых для мозга

О: Вот это серьезный философский вопрос.

Многоуровневое проектирование представляет собой практику (особую точку зрения на процесс), которую освоили многие люди. Считайте, что это своего рода очки, позволяющие взглянуть на структуру вашего кода в более высоком разрешении. Они помогают найти новые пути для улучшения удобства тестирования и сопровождения, а также выявить возможности повторного использования кода. Если они не помогают вам в данный момент, снимите их. А если кто-то видит что-то другое, поменяйтесь с ним очками.

В: Я вижу больше уровней, чем вы приводите, даже на этих простых диаграммах. Я делаю что-то не так?

О: Вовсе нет. Возможно, вы концентрируетесь на уровне детализации, который важен для ваших целей, но не важен для изучаемой темы. Ведите исследования с таким высоким (или низким) уровнем детализации, который вам нужен. Свободно меняйте масштаб восприятия. Пользуйтесь очками!

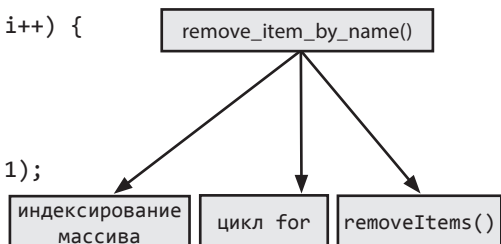
Добавление функции `remove_item_by_name()`

Граф, нарисованный для `freeTieClip()`, предоставляет хорошую точку зрения на структуру системы. Ее можно расширить на все операции для работы с корзиной. Добавим на граф функцию `remove_item_by_name()`. Подчеркнем все функции, которые она вызывает, а также средства языка, которые мы хотим выделить.

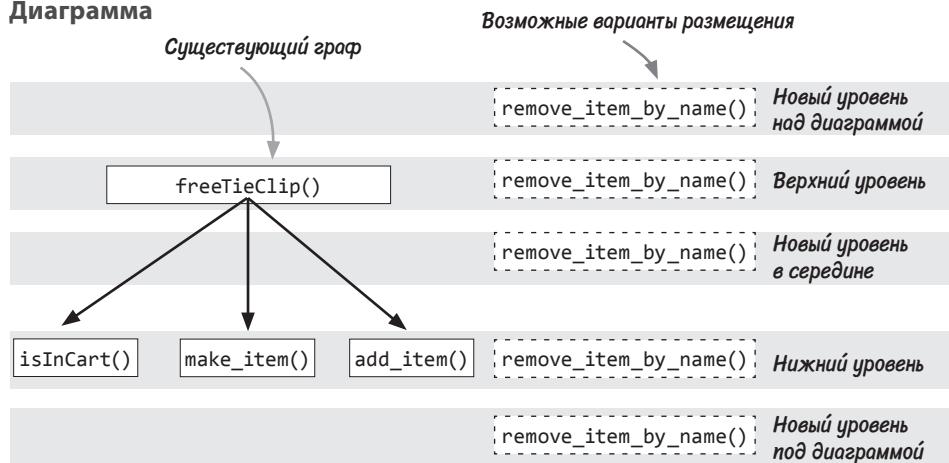
Код

```
function remove_item_by_name(cart, name) {
    var idx = null;
    for(var i = 0; i < cart.length; i++) {
        if(cart[i].name === name)
            idx = i;
    }
    if(idx !== null)
        return removeItems(cart, idx, 1);
    return cart;
}
```

Диаграмма



Требуется расширить уже имеющийся граф прямоугольниками и стрелками новой диаграммы. Ниже изображен граф, нарисованный для `freeTieClip()`. Где изобразить `remove_item_by_name()`? Есть пять вариантов, которые также обозначены на диаграмме:

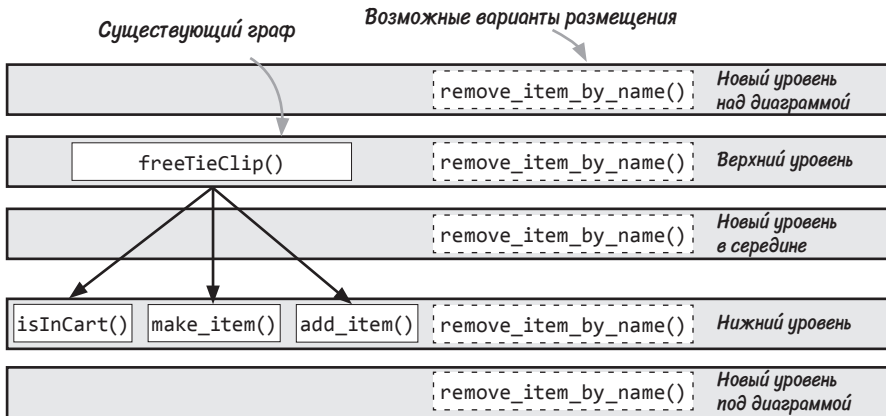
Диаграмма**Ваш ход**

На диаграмме изображены пять разных уровней, на которых можно было бы разместить `remove_item_by_name()`. Одни создают новые уровни; другие расширяют существующие уровни. Где же добавить новую функцию? Какая информация поможет принять это решение? Мы разберемся с этим на ближайших страницах.



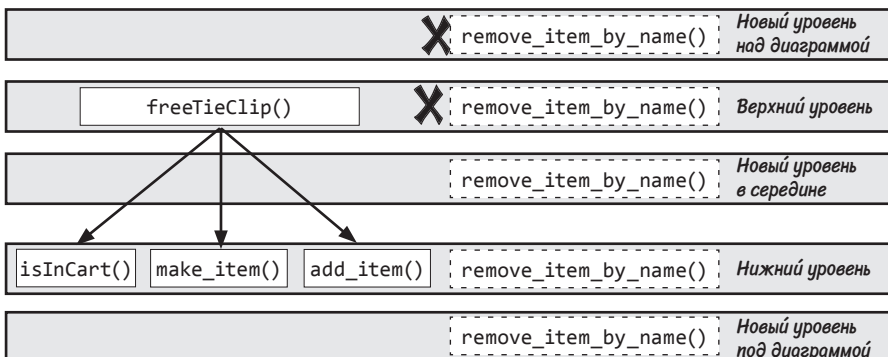
Ответ

Существует много входных данных, которые помогут выбрать уровень для размещения `remove_item_by_name()`. Воспользуемся процессом исключения:



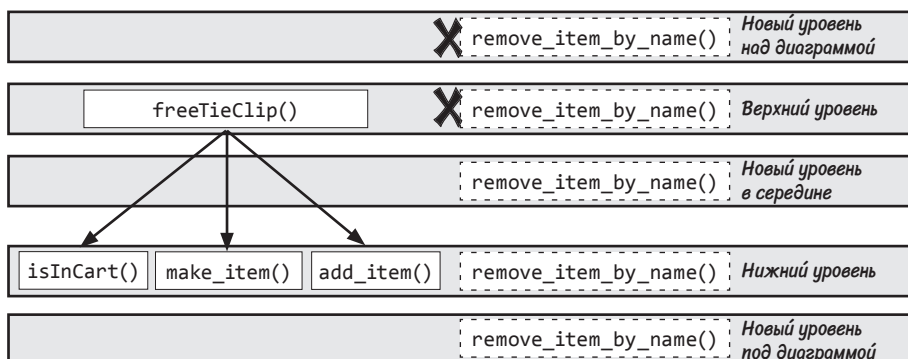
Принадлежит ли `remove_item_by_name()` верхнему уровню? Что сейчас находится на верхнем уровне? Взглянув на имя функции `freeTieClip()`, мы видим, что эта функция предназначена для реализации рекламной кампании. Функция `remove_item_by_name()` к маркетингу определенно не относится. Это операция более общего назначения. Она может использоваться для маркетинговых кампаний, операций пользовательского интерфейса (UI) или для множества других целей. И нетрудно представить, как функции маркетинговой кампании на верхнем уровне вызывают `remove_item_by_name()`. Следовательно, функция `remove_item_by_name()` должна располагаться ниже этого уровня, чтобы стрелки были направлены сверху вниз. Таким образом, два верхних уровня можно исключить.

По именам функций можно оценить, к какому уровню принадлежит функция.



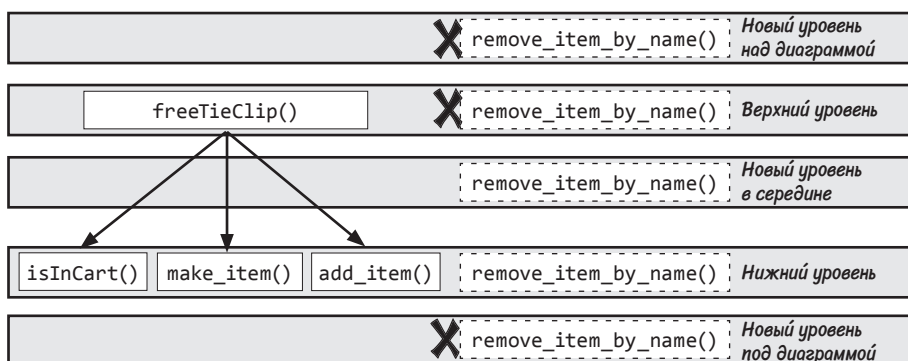


Ответ



Мы исключили два верхних варианта, потому что `remove_item_by_name()` может вызываться средствами маркетинговой кампании (верхний уровень). Как насчет нижнего уровня? Все имена на этом уровне обозначают операции с корзинами и товарами. Функция `remove_item_by_name()` также относится к корзинам. Пока это лучший из проверенных кандидатов.

Можно ли исключить два новых уровня для уверенности? Новый уровень под диаграммой можно исключить: ни одной функции на нижнем уровне не требуется вызывать `remove_item_by_name()`. Это позволяет нам исключить новый уровень под диаграммой.

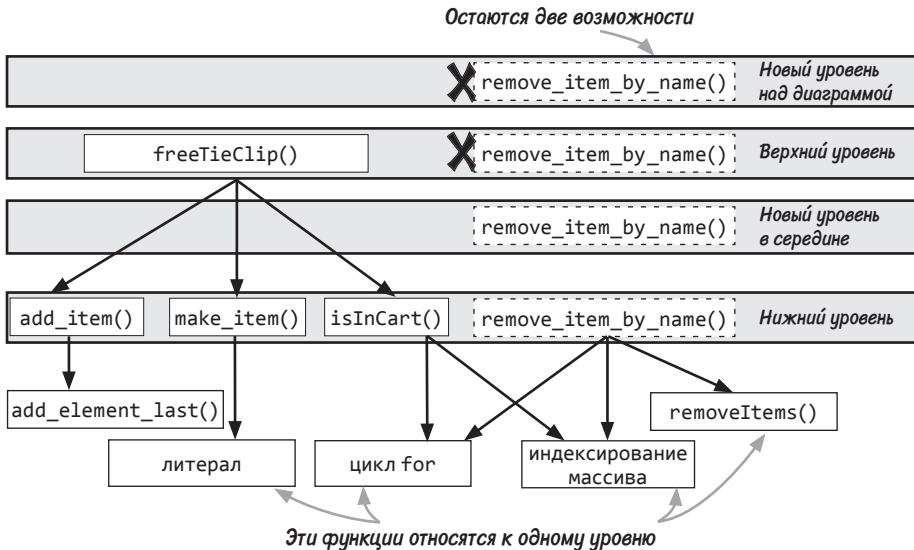


Поскольку `remove_item_by_name()` является общей операцией с корзинами, как и две другие операции на нижнем уровне, нижний уровень становится лучшим кандидатом. Тем не менее остался еще один новый уровень, который мы не исключили. Можно ли исключить новый промежуточный уровень?



Ответ

Не на 100 %. Но мы можем посмотреть на то, какие функции и средства языка вызываются функциями на нижнем уровне. Если между ними существует заметное перекрытие, это хороший признак того, что они принадлежат одному уровню.



```
function isInCart(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      return true;
  }
  return false;
}
```

```
function make_item(name, price) {
  return {
    name: name,
    price: price
  };
}
```

```
function add_item(cart, item) {
  return add_element_last(cart, item);
}
```

```
function remove_item_by_name(cart, name) {
  var idx = null;
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      idx = i;
  }
  if(idx !== null)
    return removeItems(cart, idx, 1);
  return cart;
}
```

`isInCart()` и `remove_item_by_name()` указывают на одни и те же блоки, что является надежным признаком того, что они принадлежат одному уровню. Позднее я приведу более убедительные аргументы. А пока мы выбираем нижний уровень как наиболее подходящее место для размещения функции, рядом с `isInCart()`, `make_item()` и `add_item()`.



Ваш ход

Внизу приведены все реализованные нами операции с корзиной. Некоторые из них уже добавлены на граф: эти операции выделены. Текущая диаграмма находится внизу. Многие функции еще не были добавлены.

Ваша задача — добавить остальные функции на граф и разместить функции по соответствующим уровням (при необходимости можете перемещать существующие блоки). Ответ приведен на следующей странице.

```
function freeTieClip(cart) {
  var hasTie = isInCart(cart, "tie");
  var hasTieClip = isInCart(cart, "tie clip");
  if(hasTie && !hasTieClip) {
    var tieClip = make_item("tie clip", 0);
    return add_item(cart, tieClip);
  }
  return cart;
}
```

```
function add_item(cart, item) {
  return add_element_last(cart, item);
}
```

```
function isInCart(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      return true;
  }
  return false;
}
```

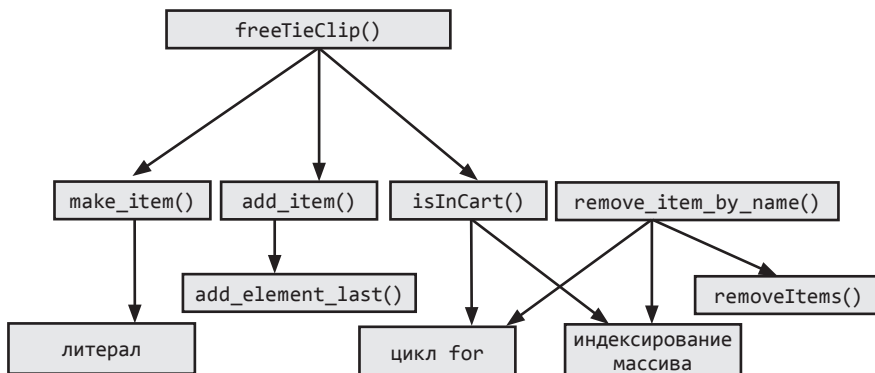
```
function remove_item_by_name(cart, name) {
  var idx = null;
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      idx = i;
  }
  if(idx !== null)
    return removeItems(cart, idx, 1);
  return cart;
}
```

```
function calc_total(cart) {
  var total = 0;
  for(var i = 0; i < cart.length; i++) {
    var item = cart[i];
    total += item.price;
  }
  return total;
}
```

```
function gets_free_shipping(cart) {
  return calc_total(cart) >= 20;
}
```

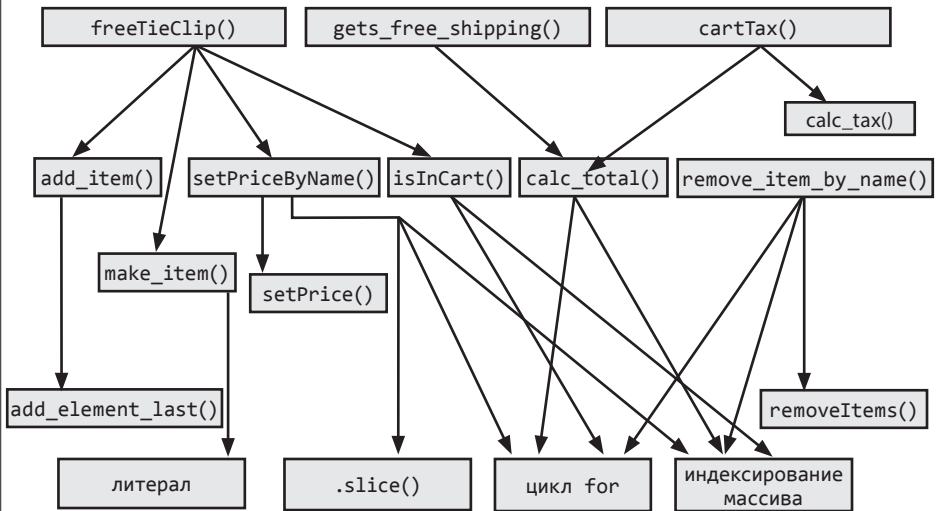
```
function setPriceByName(cart, name, price)
{
  var cartCopy = cart.slice();
  for(var i = 0; i < cartCopy.length; i++)
  {
    if(cartCopy[i].name === name)
      cartCopy[i] =
        setPrice(cartCopy[i], price);
  }
  return cartCopy;
}
```

```
function cartTax(cart) {
  return calc_tax(calc_total(cart));
}
```





Ответ



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Мой граф похож на ваш, но все же отличается от него. Я что-то сделал не так?

О: Наверное, нет. Если ваш граф выполняет следующие условия, то с ним все хорошо.

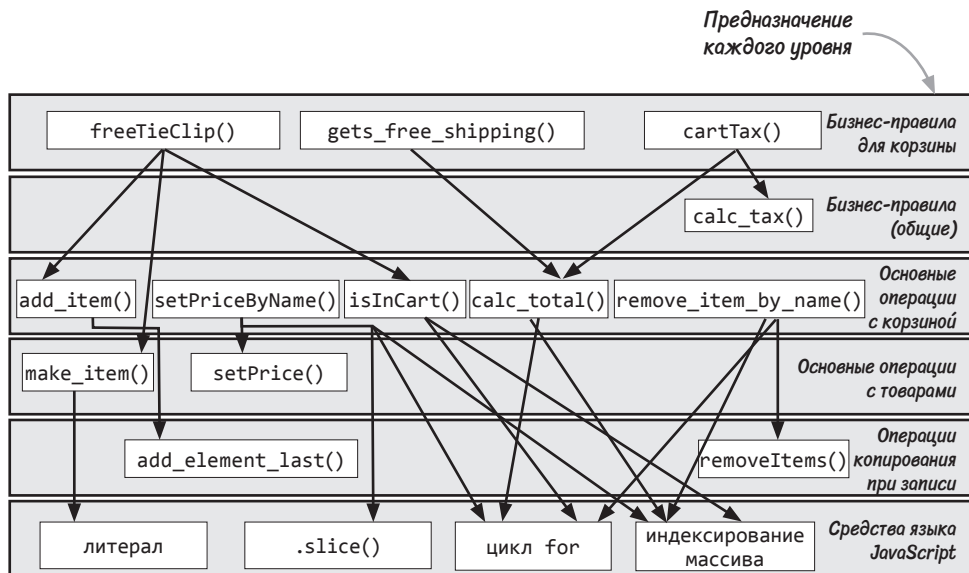
1. Все функции включены в граф.
2. Каждая функция указывает на все функции, которые она вызывает.
3. Все стрелки указывают вниз (не в сторону и не вверх).

В: Но почему вы выбрали именно эти конкретные уровни?

О: Очень хороший вопрос. Уровни на этой диаграмме были выбраны так, чтобы они соответствовали уровням абстракции. Каждый из этих уровней кратко рассматривается на следующей странице.

Все функции уровня должны иметь некое предназначение

Диаграмма состоит из шести уровней. Каждый уровень был выбран сознательно, ничто на этой диаграмме не является случайным. И хотя процесс принятия решения о том, к какому уровню относится тот или иной блок, может быть непростым, есть хороший признак того, что уровни были выбраны хорошо: для каждого уровня можно определить конкретное предназначение. Давайте сделаем это для нашего графа.



Каждый из этих уровней находится на своем уровне абстракции. Другими словами, когда вы работаете с функциями одного уровня, существуют некоторые общие подробности, на которые можно не обращать внимания. Например, когда вы работаете на уровне «Бизнес-правила для корзины», вам не нужно беспокоиться о таких подробностях, как реализация корзины в виде массива.

Имеющаяся диаграмма сочетает факты (что вызывают те или иные функции) с интуитивными умозаключениями (как они должны быть выстроены по уровням). Это хорошее представление нашего кода. Но не забывайте: мы хотим, чтобы многоуровневое проектирование помогало нам в построении прямолинейных реализаций (первый паттерн). Как эта диаграмма может помочь с реализациями? На нескольких следующих страницах мы рассмотрим разные уровни детализации (разные уровни масштабирования), чтобы сосредоточиться на той информации, которая может потребоваться в любой конкретный момент времени.

Паттерны

- ☐ Прямолинейная реализация
- ☐ Абстрактный барьер
- ☐ Минимальный интерфейс
- ☐ Удобные уровни

Три уровня детализации

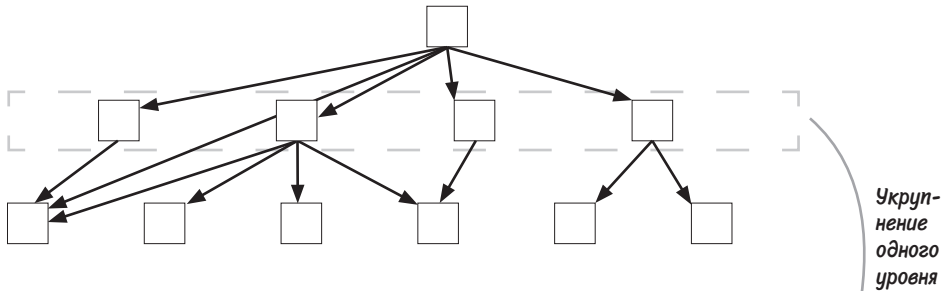
На диаграмме можно увидеть возможные проблемы. Однако информации так много, что мы не знаем, где эти проблемы следует искать. Проблемы многоуровневого проектирования могут относиться к трем областям:

1. Взаимодействие между уровнями.
2. Реализация одного уровня.
3. Реализация одной функции.

Чтобы сосредоточиться на одной проблемной области, следует выбрать правильный масштаб.

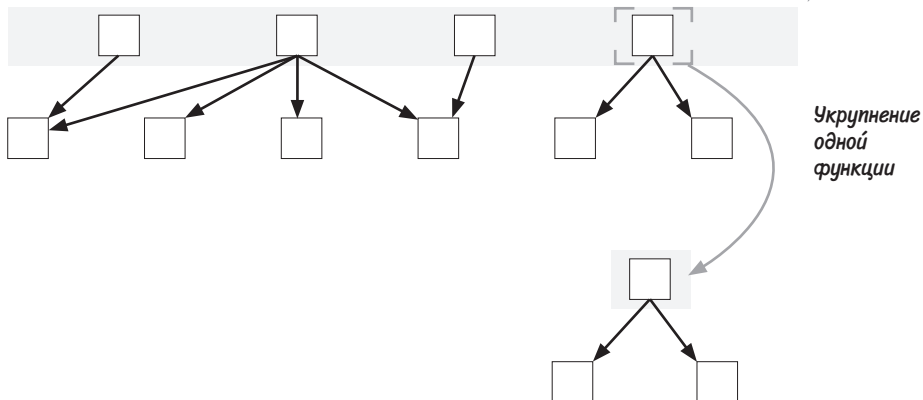
1. Глобальный масштаб

В глобальном масштабе рассматривается весь интересующий нас граф вызовов. Именно это представление используется по умолчанию. Оно позволяет видеть все, включая взаимодействия между уровнями.



2. Масштаб уровня

В масштабе уровня мы начинаем с уровня, который представляет для нас интерес, и рисуем все находящееся ниже, куда ведут стрелки с этого уровня. Мы видим, как строится уровень.



3. Масштаб функции

В масштабе функции мы начинаем с одной функции, представляющей для нас интерес, и рисуем все находящееся ниже, на что ведут стрелки из этой функции. Так можно диагностировать проблемы с реализацией.

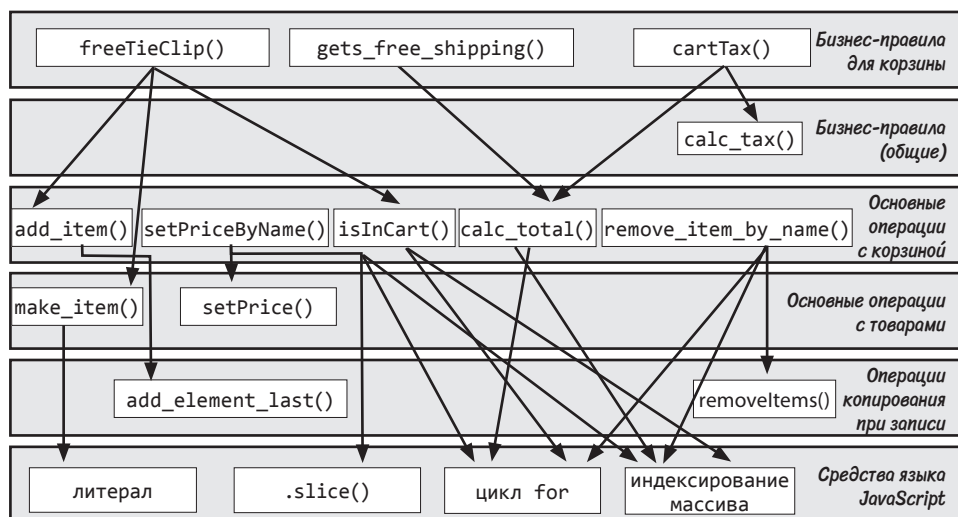
Концепция масштаба пригодится тогда, когда мы пытаемся найти и исправить проблемы проектирования. Рассмотрим диаграмму графа вызовов в масштабе уровня.

Масштаб

1. Глобальный (по умолчанию).
2. Уровень.
3. Функция.

В масштабе уровня сравниваются стрелки между функциями

Ниже приведена полная диаграмма (в глобальном масштабе):



Сосредоточившись на уровне, мы рассматриваем функции этого уровня вместе с теми функциями, которые вызываются ими напрямую. Давайте сосредоточимся на уровне основных операций с корзиной.

Масштаб

1. Глобальный (по умолчанию).
2. Уровень.
3. Функция.

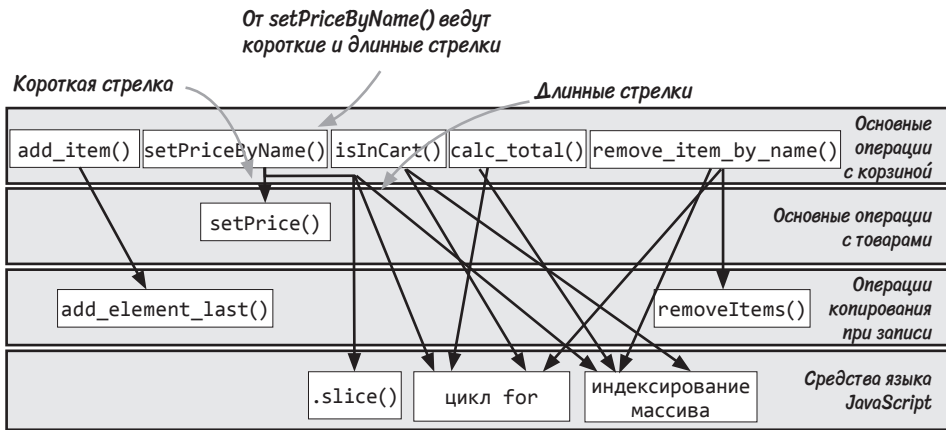


Диаграмма кажется довольно запутанной. Однако запутанные части — стрелки — всего лишь представляют факты о нашем коде. Путаница на диаграмме отражает путаницу в коде. Мы пытаемся собрать информацию, которая поможет исправить ситуацию.

В прямолинейной реализации все стрелки должны иметь приблизительно одинаковую длину. Но в данном случае мы видим, что у одних стрелок длина составляет один уровень, а у других — три уровня. Это доказывает, что мы не работаем на одном уровне детализации в пределах целого уровня.

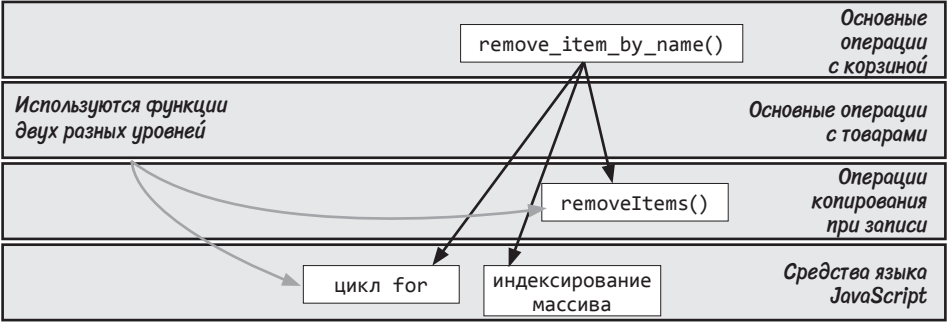
Прежде чем переходить к решению, взглянем на диаграмму в масштабе одной функции. Как правило, начинать решение проблем проще с этого масштаба.

В масштабе функции сравниваются стрелки, ведущие от одной функции

В масштабе функции рассматриваются стрелки, выходящие только из одной функции. Ниже изображен тот же граф, масштабированный на функции `remove_item_by_name()`. На графе изображена эта функция, а также функции и средства языка, которые она использует.

Паттерны

- ☐ Прямолинейная реализация
- ☐ Абстрактный барьер
- ☐ Минимальный интерфейс
- ☐ Удобные уровни



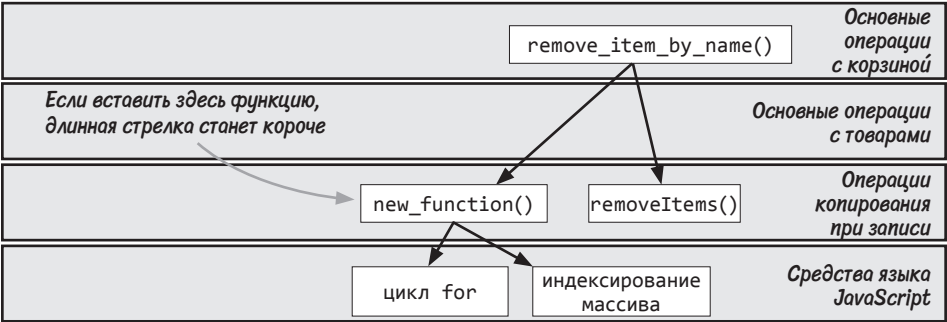
Даже в этой одной функции мы видим, что она использует блоки с двух разных уровней. Такая реализация не является прямолинейной.

В идеальной прямолинейной реализации все стрелки из `remove_item_by_name()` будут иметь одинаковую длину. Как этого добиться?

Чаще всего для этого используются промежуточные функции. Мы хотим укоротить две стрелки, уходящие вниз до языковых средств. Если вставить функцию, которая делает то же, что цикл `for` и индексирование массива на одном уровне с операцией `removeItems()`, все стрелки будут иметь одинаковую длину. Это будет выглядеть примерно так:

Паттерны

- ☐ Прямолинейная реализация
- ☐ Абстрактный барьер
- ☐ Минимальный интерфейс
- ☐ Удобные уровни



К счастью, эта операция эквивалентна извлечению цикла `for` в новую функцию, которое вы уже видели ранее! Диаграмма просто позволяет взглянуть на происходящее под новым углом.

Масштаб

- Глобальный (по умолчанию).
- Уровень.
- Функция.

Выделение цикла for

Из функции `remove_item_by_name()` можно выделить цикл `for`. В нем выполняется линейный поиск по массиву. Результатом поиска является индекс элемента, в котором был найден заданный товар. Назовем новую функцию `indexOfItem()`.

До

```
function remove_item_by_name(cart, name) {
  var idx = null;
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      idx = i;
  }
  if(idx !== null)
    return removeItems(cart, idx, 1);
  return cart;
}
```

*Цикл for выделяется
в новую функцию*

После

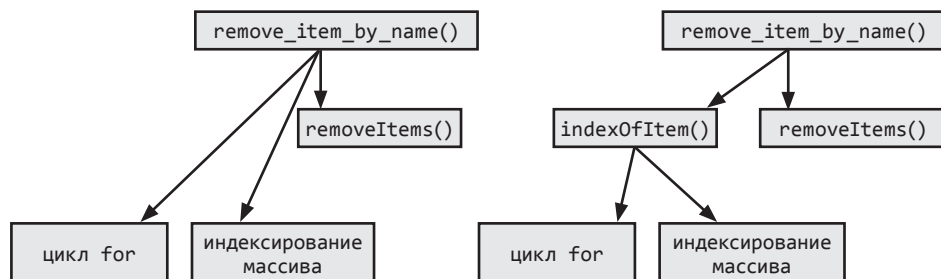
```
function remove_item_by_name(cart, name) {
  var idx = indexOfItem(cart, name);

  if(idx !== null)
    return removeItems(cart, idx, 1);
  return cart;
}

function indexOfItem(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      return i;
  }
  return null;
}
```

Реализация `remove_item_by_name()` проще читается. И этот факт отражен на диаграмме: функция `indexOfItem()` находится на чуть более высоком уровне, чем `removeItems()`, потому что функция `removeItems()` является более общей. Функция `indexOfItem()` знает структуру элементов массива, то есть знает, что они являются объектами со свойством `'name'`. В главе 10 будет показано, как создать более общую версию того же цикла `for`.

А пока эта функция открыла нам новую возможность повторного использования кода. Как покажет следующее упражнение, повторное использование обычно возможно только тогда, когда вы создали хорошую структуру уровней.





Ваш ход

Функции `isInCart()` и `indexOfItem()` содержат очень похожий код. Не кроется ли здесь возможность для повторного использования? Можно ли переписать одну функцию с использованием другой?

```
function isInCart(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      return true;
  }
  return false;
}

function indexOfItem(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      return i;
  }
  return null;
}
```

Реализуйте одну функцию с вызовом другой, а затем нарисуйте диаграмму для этих функций до уровня циклов `for` и индексирования массива.



Ответ

`indexOfItem()` и `isInCart()` содержат очень похожий код. `indexOfItem()` принадлежит более низкому уровню, чем `isInCart()`. `indexOfItem()` возвращает индекс элемента в массиве, который может быть полезен только в том случае, если вызывающий код знает, что корзина реализована в виде массива. `isInCart()` возвращает логическое значение. Вызывающему коду не нужно знать структуру данных.

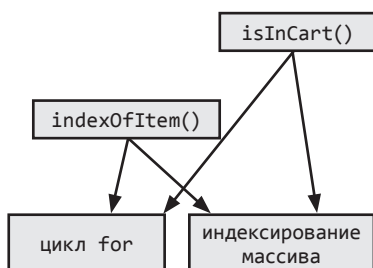
Поскольку функция `isInCart()` находится на более высоком уровне, она может быть реализована с использованием низкоуровневой функции `indexOfItem()`.

Оригинал

```
function isInCart(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      return true;
  }
  return false;
}

function indexOfItem(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      return i;
  }
  return null;
}
```

indexOfItem() содержит похожий цикл for

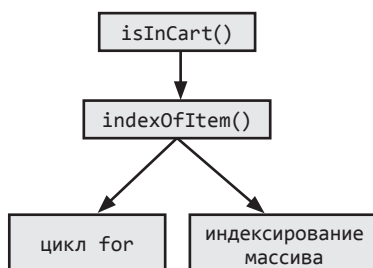


Запись с использованием другой функции

```
function isInCart(cart, name) {
  return indexOfItem(cart, name) !== null;
}

function indexOfItem(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      return i;
  }
  return null;
}
```

Цикл заменяется вызовом функции



Код получается более коротким, потому что мы достигаем определенной степени повторного использования. А еще наш код начинает расслаиваться на более понятные уровни. И то и другое безусловно хорошо.

Тем не менее выигрыш не всегда настолько очевиден. Рассмотрим другой пример повторного использования на следующей странице.



Ваш ход

Если присмотреться повнимательнее, можно заметить, что `setPriceByName()` также содержит цикл `for`, очень похожий на цикл из `indexOfItem()`.

```
function setPriceByName(cart, name, price) {  
    var cartCopy = cart.slice();  
    for(var i = 0; i < cartCopy.length; i++) {  
        if(cartCopy[i].name === name)  
            cartCopy[i] = setPrice(cartCopy[i], price);  
    }  
    return cartCopy;  
}  
  
function indexOfItem(cart, name) {  
    for(var i = 0; i < cart.length; i++) {  
        if(cart[i].name === name)  
            return i;  
    }  
    return null;  
}
```

Реализуйте одну функцию с вызовом другой, а затем нарисуйте диаграмму для этих функций до уровня циклов `for` и индексирования массива.



Ответ

`indexOfItem()` и `setPriceByName()` содержат очень похожий код. `indexOfItem()` относится к более низкому уровню, чем `setPriceByName()`.

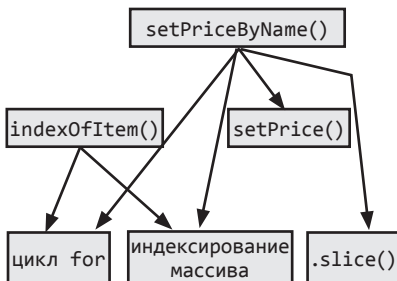
Оригинал

```
function setPriceByName(cart, name, price) {
  var cartCopy = cart.slice();
  for(var i = 0; i < cartCopy.length; i++) {

    if(cartCopy[i].name === name)

      cartCopy[i] =
        setPrice(cartCopy[i], price);
  }
  return cartCopy;
}

function indexOfItem(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      return i;
  }
  return null;
}
```



Запись с использованием другой функции

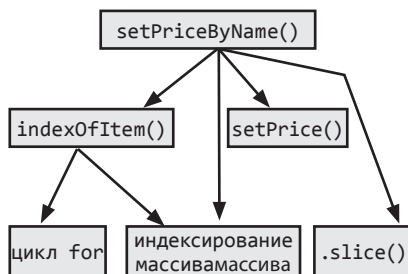
```
function setPriceByName(cart, name, price) {
  var cartCopy = cart.slice(b);

  var i = indexOfItem(cart, name);

  if(i !== null)
    cartCopy[i] =
      setPrice(cartCopy[i], price);

  return cartCopy;
}

function indexOfItem(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      return i;
  }
  return null;
}
```



Код выглядит лучше (потому что мы избавились от цикла `for`), но граф особо не улучшился. Ранее функция `setPriceByName()` указывала на два разных уровня. Сейчас она тоже указывает на два разных уровня. Разве деление кода на уровни не должно было помочь?

Количество уровней, на которые указывает функция, иногда является хорошим показателем сложности, но только не в данном случае. Вместо него следует обратить внимание на то, что одна из длинных стрелок была заменена: мы улучшили архитектуру, исключив одну из длинных стрелок. Теперь остались только две! И мы можем продолжить процесс и продолжить разделение уровней. И насколько полно используются уже имеющиеся уровни? Один из вариантов улучшения архитектуры рассматривается в следующем упражнении.



Ваш ход

В главе 6 мы разработали несколько функций для выполнения операций с копированием при записи для массивов и объектов. Одна из них, `arraySet()`, присваивала значение элементу с заданным индексом, используя подход копирования при записи. Похоже, она имеет много общего с `setPriceByName()`. Можно ли записать `setPriceByName()` с использованием `arraySet()`?

```
function setPriceByName(cart, name, price) {
  var cartCopy = cart.slice();
  var idx = indexOfItem(cart, name);
  if(idx !== null)
    cartCopy[idx] =
      setPrice(cartCopy[idx], price);
  return cartCopy;
}

function arraySet(array, idx, value) {
  var copy = array.slice();
  copy[idx] =
    value;
  return copy;
}
```

Реализуйте `setPriceByName()` с использованием `arraySet()` и нарисуйте диаграмму.



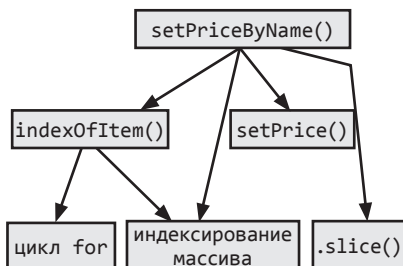
Ответ

`indexOfItem()` и `setPriceByName()` содержат очень похожий код. `indexOfItem()` находится на более низком уровне, чем `setPriceByName()`.

Оригинал

```
function setPriceByName(cart, name, price) {
  var cartCopy = cart.slice();
  var i = indexOfItem(cart, name);
  if(i !== null)
    cartCopy[i] =
      setPrice(cartCopy[i], price);
  return cartCopy;
}

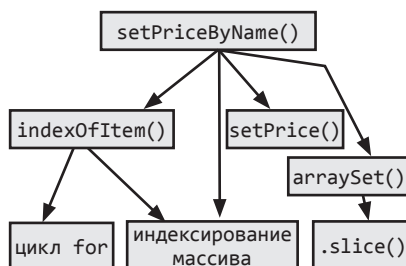
function arraySet(array, idx, value) {
  var copy = array.slice();
  copy[idx] = value;
  return copy;
}
```



Запись с использованием другой функции

```
function setPriceByName(cart, name, price) {
  var i = indexOfItem(cart, name);
  if(i !== null)
    return arraySet(cart, i,
      setPrice(cart[i], price));
  return cart;
}

function arraySet(array, idx, value) {
  var copy = array.slice();
  copy[idx] = value;
  return copy;
}
```



Код выглядит лучше. Мы избавились от длинной стрелки, ведущей к `.slice()`, заменив ее более короткой стрелкой к `arraySet()`. Но похоже, теперь стрелки ведут на три разных уровня!

И снова следует сосредоточиться на том, чего мы добились: нам удалось заменить более длинную стрелку, что соответствует другой игнорируемой подробности.

Тем не менее эту функцию все еще трудно назвать прямолинейной. Она так же указывает на самый низкий уровень из-за использования индекса. Этому чувству следует доверять. В прошлом оно возникало у многих функциональных программистов, пытавшихся выделить более общую функцию, которая сделает код более понятным. Не стесняйтесь, попробуйте. Возможно, вам удастся избавиться от последней стрелки, ведущей к индексированию.

Пока оставим этот код в покое. В следующей главе мы применим принцип абстрактного барьера, который прояснит его. А в главе 10 будет представлен метод, который сделает реализацию еще более прямолинейной.



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Действительно ли улучшается структура `setPriceByName()`? Похоже, граф становится не более прямолинейным, а более сложным.

О: Отличный вопрос. Не существует формулы для определения лучшей структуры. Это сложное сочетание многих факторов, включая специфику использования кода и квалификацию ваших разработчиков. Эти паттерны могут указывать на полезные признаки в вашем коде или графе вызовов. В конечном счете вам придется выработать конкретную структуру, руководствуясь итеративным анализом и интуицией.

Проектирование — сложная работа. Разные программисты часто расходятся во мнениях относительно того, какую архитектуру следует считать лучшей, а выбор больше зависит от ситуации. Важно иметь общую номенклатуру для обсуждения проектирования и не менее важно оценивать решения, связанные с проектированием, в контексте. Упражнения этой и следующей главы помогут усовершенствовать ваши навыки по оценке этих решений.



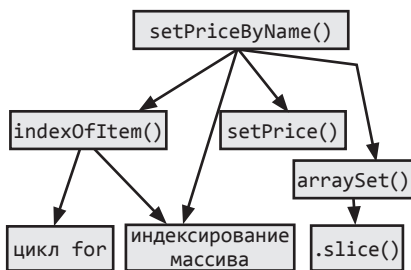
Пища для ума

Индексирование массива легко исключается добавлением новой функции. Улучшит ли это структуру кода?

С индексированием массива

```
function setPriceByName(cart, name, price) {
  var i = indexOfItem(cart, name);
  if(i !== null) {
    var item = cart[i];
    return arraySet(cart, i,
      setPrice(item, price));
  }
  return cart;
}

function indexOfItem(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      return i;
  }
  return null;
}
```

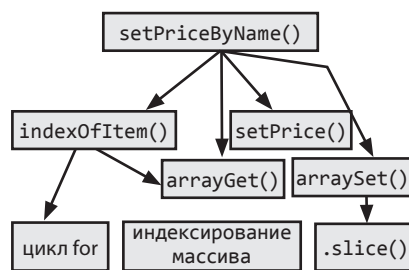


Без индексирования массива

```
function setPriceByName(cart, name, price) {
  var i = indexOfItem(cart, name);
  if(i !== null) {
    var item = arrayGet(cart, i);
    return arraySet(cart, i,
      setPrice(item, price));
  }
  return cart;
}

function indexOfItem(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(arrayGet(cart, i).name === name)
      return i;
  }
  return null;
}

function arrayGet(array, idx) {
  return array[idx];
}
```



Вместо того чтобы сразу отвечать «да» или «нет», проанализируйте несколько ситуаций, в которых тот или иной вариант будет лучше. Пара примеров вам поможет.

Индексирование массива лучше подходит, когда...

- команда привыкла работать с массивами;
-
-

Упаковка индексирования массива лучше подходит, когда...

- необходима более четкая структура уровней;
-
-

Обзор паттерна 1. Прямолинейная реализация

Прямолинейный код решает задачу на одном уровне детализации

Если программировать, не обращая внимания на структуру кода, в результате нередко появляется код, который трудно читать и изменять. Почему это трудно? Чаще всего код плохо читается, потому что его приходится понимать на разных уровнях детализации. Чтобы прочитать функцию, требуется осознать слишком много всего. Прямолинейные реализации стараются сократить набор уровней детализации, понимание которых необходимо для чтения кода.

Многоуровневое проектирование помогает направить внимание на конкретный уровень детализации

Хотя простой формулы не существует, нужно развивать свое чувство проектирования, чтобы распознавать эти уровни детализации по различным признакам в коде. После этого можно вносить соответствующие изменения.

Граф вызовов становится богатым источником информации об уровнях детализации

Сам код предоставляет множество ориентиров, но чтобы представить общую картину, иногда приходится читать слишком много кода. Граф вызовов может показать, сколько функций определяются в категориях вызова друг друга. В процессе построения графа функции можно размещать на уровнях, соответствующих их уровням детализации. Сигнатура функции, тело и граф вызовов предоставляют достаточно информации, чтобы помочь в написании прямолинейного кода.

Извлечение функций позволяет получать более общие функции

Один из способов получения более прямолинейного кода основан на извлечении более общей функции, которая позаботится о деталях, с которыми вам не хочется иметь дела на этом уровне. Более общую функцию обычно проще тестировать, так как она обрабатывает одну конкретную деталь. Кроме того, более понятный код и содержательные имена упрощают чтение кода.

Паттерны



Прямолинейная реализация



Абстрактный барьер



Минимальный интерфейс



Удобные уровни

Более общие функции лучше подходят для повторного использования

При извлечении функций нередко находятся другие места, в которых можно воспользоваться этими функциями. Ситуация отличается от поиска дублирующегося кода: общие функции выделяются для прояснения реализации. Но, как выясняется, общие функции обычно полезнее конкретных: они открывают непредвиденные возможности для повторного использования.

Сложность не скрывается

Очень легко заставить любой код *выглядеть* прямолинейно. Достаточно скрыть малопонятные части за «вспомогательными функциями». Тем не менее это нельзя назвать многоуровневым проектированием. При многоуровневом проектировании каждый уровень должен быть прямолинейным. Нельзя просто вынести сложный код с текущего уровня. Необходимо найти на более низком уровне общие функции, которые прямолинейны сами по себе, и построить из них программную систему по прямолинейным принципам.

Итоги главы

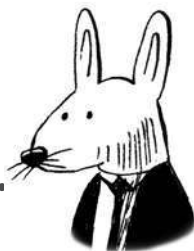
В этой главе вы узнали, как наглядно представить код в виде графа вызовов и как распознавать разные уровни абстракции. Мы рассмотрели первый и самый важный паттерн многоуровневого проектирования, который предлагает нам искать прямолинейные реализации. Структура уровней помогает организовать код так, чтобы простые функции строились в нем из еще более простых функций. Тем не менее многоуровневое проектирование этим не ограничивается. В следующей главе будут представлены еще три паттерна.

Резюме

- Многоуровневое проектирование разделяет код по уровням абстракции. Каждый уровень помогает игнорировать разные подробности реализации.
- При реализации новой функции необходимо определить, какие подробности важны для решения задачи. По этой информации можно судить о том, на каком уровне должна располагаться функция.
- Существует множество ориентиров, которые помогают найти правильный уровень для функций. В частности, стоит проверить имя, тело и граф вызовов.
- Имя сообщает предназначение функции. Его можно группировать с другими функциями, имеющими сходное предназначение.
- По телу функции можно определить, какие подробности важны для функции. По этой информации определяется ее место в структуре уровней.
- Если выходящие стрелки на графе вызовов имеют разную длину, это хороший признак того, что реализация не прямолинейна.
- Структуру уровня можно улучшить посредством выделения более общих функций. Более общие функции относятся к нижним уровням, и они лучше приспособлены для повторного использования.
- Паттерн прямолинейной реализации направляет наши усилия для формирования структуры уровней, с которой функции реализуются четко и элегантно.

Что дальше?

Паттерн прямолинейной реализации — всего лишь начало того, что можно узнать из структуры уровней. В следующей главе будут рассмотрены еще три паттерна, основанные на структуре уровней, которые упрощают тестирование, сопровождение и повторное использование кода.



В этой главе

- ✓ Построение абстрактных барьеров для разбиения кода на модули.
- ✓ Признаки хорошего интерфейса (и где их следует искать).
- ✓ Какая архитектура может считаться «достаточно хорошей»?
- ✓ Влияние многоуровневого проектирования на сопровождение, тестирование и повторное использование кода.

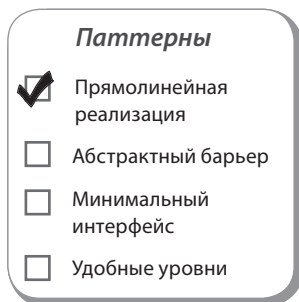
В предыдущей главе вы научились рисовать графы вызовов и находить уровни, которые помогают в организации кода. В этой главе мы продолжим углубленно изучать многоуровневое проектирование и совершенствовать свою интуицию проектирования на еще трех паттернах. Эти паттерны способствуют удобству сопровождения, тестирования и повторного использования.

Паттерны многоуровневого проектирования

На всякий случай напомним, что мы рассматриваем практику многоуровневого проектирования через призму четырех паттернов. Паттерн 1 уже был рассмотрен в предыдущей главе. Поскольку с азами мы уже разобрались, в этой главе будут рассматриваться три оставшихся паттерна. Снова перечислю их для удобства.

Паттерн 1. Прямолинейная реализация

Структура уровней при многоуровневом проектировании должна помочь нам в построении прямолинейных реализаций. Когда мы читаем функцию с прямолинейной реализацией, задача, представленная сигнатурой функции, должна решаться на правильном уровне детализации в теле. Слишком много подробностей — признак «кода с душком».



В этой главе рассматриваются три остальных паттерна

Паттерн 2. Абстрактный барьер

Некоторые уровни предоставляют интерфейс, который позволяет скрыть важные подробности реализации. Эти уровни помогают нам писать более абстрактный код и освобождают нас от рутины.

Паттерн 3. Минимальный интерфейс

По мере развития системы интерфейсы, относящиеся к важным бизнес-концепциям, должны сходиться к небольшому, но мощному набору операций. Все остальные операции должны определяться с точки зрения этих операций (прямо или косвенно).

Паттерн 4. Удобные уровни

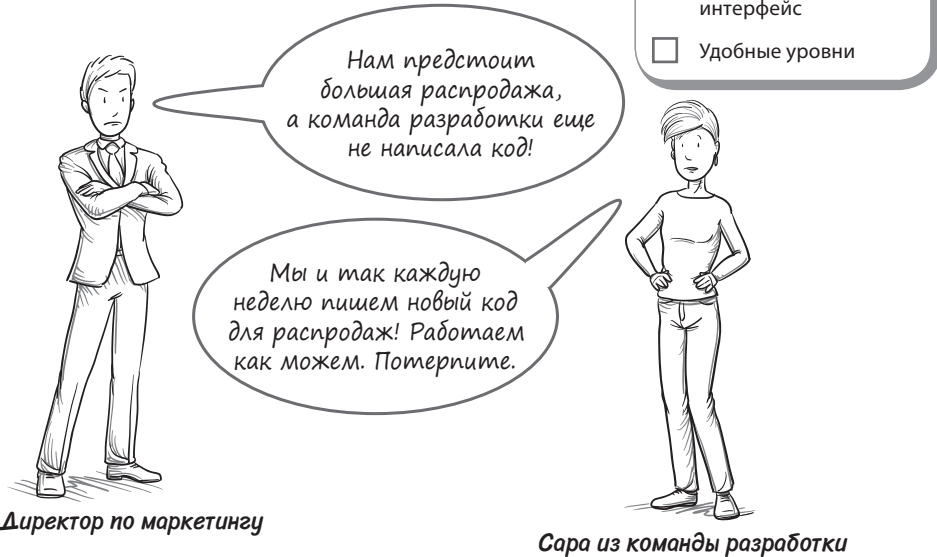
Паттерны и практики многоуровневого проектирования должны служить нашим потребностям как программистов, которые, в свою очередь, должны обслуживать бизнес. Необходимо выделить время на уровни, которые помогут нам выдавать программный продукт быстрее и с более высоким качеством. Уровни не должны добавляться для забавы. С кодом и его уровнями абстракции должно быть удобно работать.

Мы уже рассмотрели основы: построение графа вызовов и определение уровней. А теперь перейдем прямо к паттерну 2.

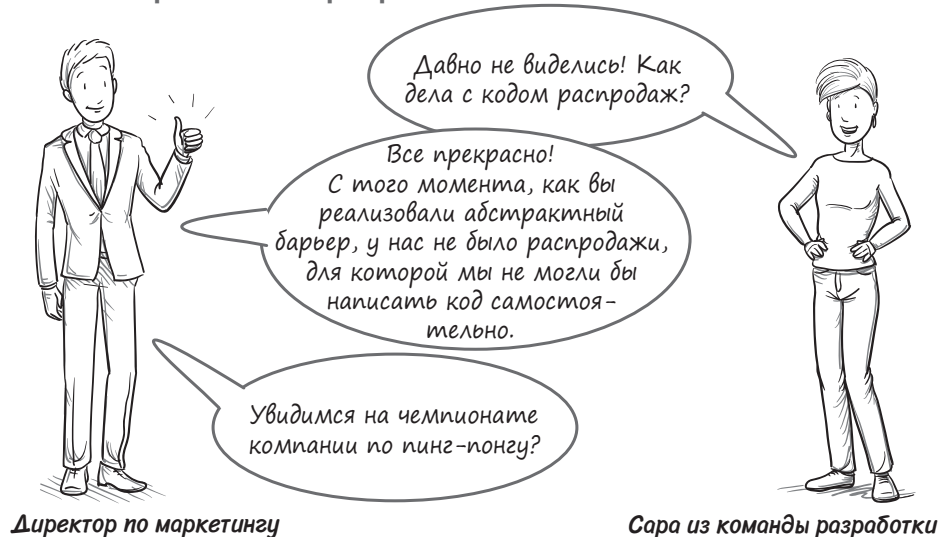
Паттерн 2. Абстрактный барьер

Второй паттерн, который мы рассмотрим, называется *абстрактный барьер*. Абстрактные барьеры решают множество задач, одна из которых — делегирование обязанностей между командами.

До абстрактного барьера

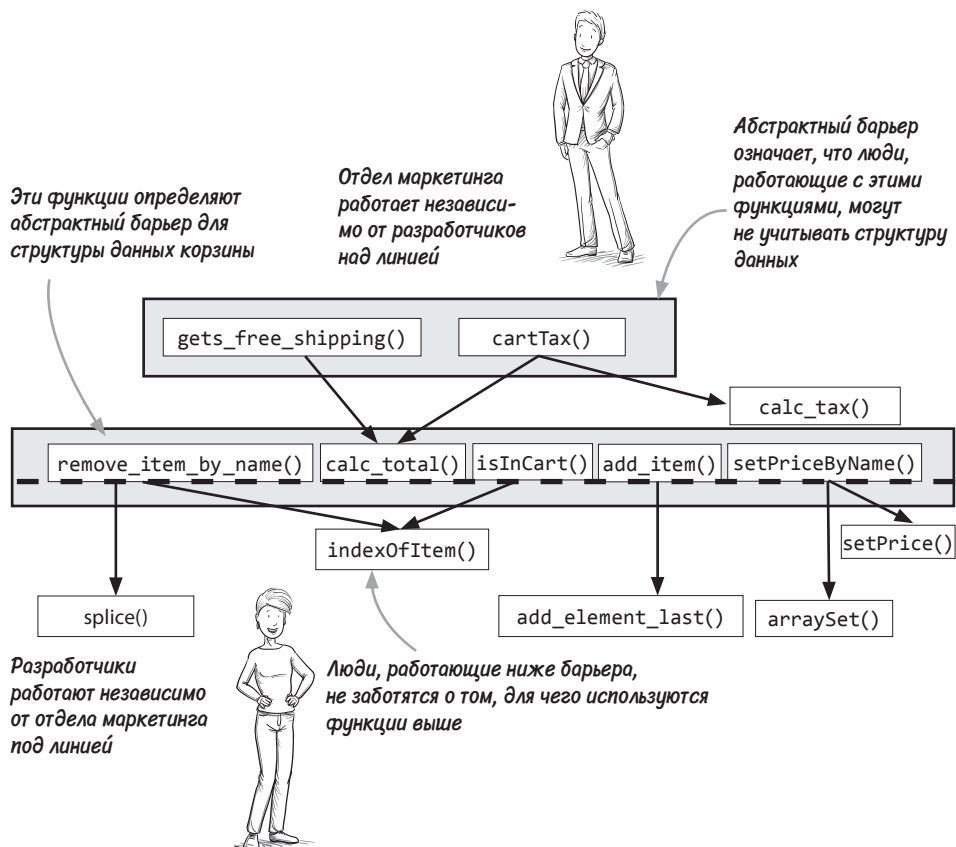


После абстрактного барьера



Абстрактные барьеры скрывают реализацию

Абстрактный барьер — уровень функций, скрывающий реализацию, чтобы при использовании этих функций вы могли полностью забыть о том, как они реализованы.

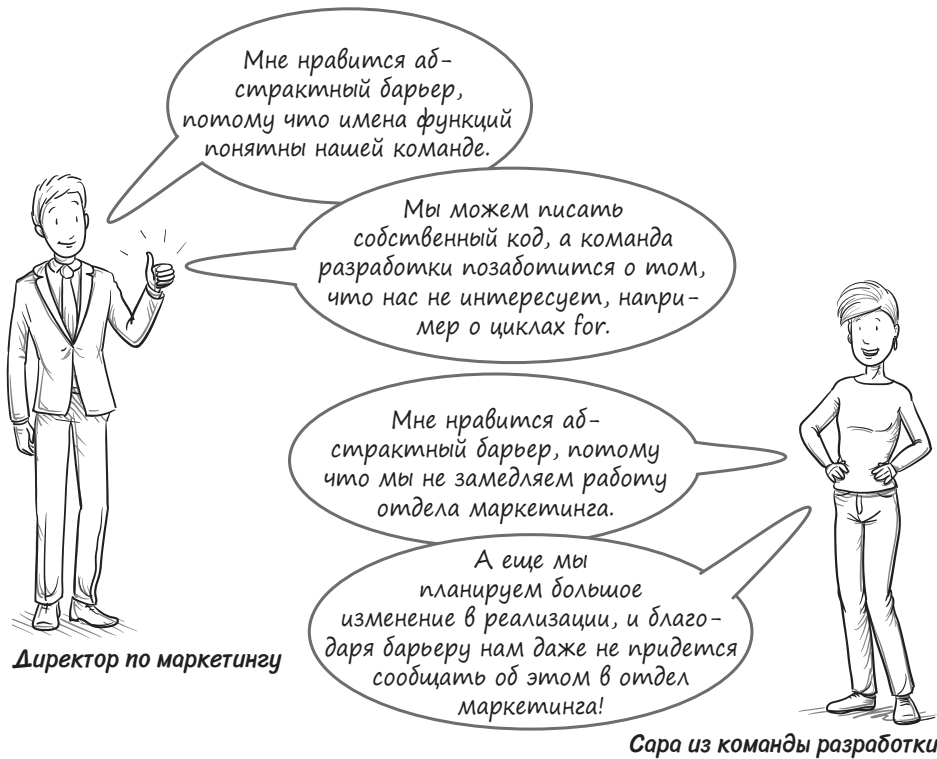


Функциональные программисты стратегически применяют абстрактные барьеры, потому что это позволяет им думать над задачей на более высоком уровне. Например, команда маркетинга может писать и читать функции, относящиеся к маркетинговой кампании, не отвлекаясь на «технические подробности» циклов `for` и массивов.

Паттерны

- ☒ Прямолинейная реализация
- ☐ **Абстрактный барьер**
- ☐ Минимальный интерфейс
- ☐ Удобные уровни

Игнорирование подробностей симметрично



Абстрактный барьер позволяет отделу маркетинга игнорировать подробности реализации. Однако это «безразличие» симметрично: команде разработки, реализующей барьер, не нужно беспокоиться о подробностях кода маркетинговой кампании, использующего функции абстрактного барьера. Благодаря мощи абстрактного барьера команды могут работать в значительной мере независимо.

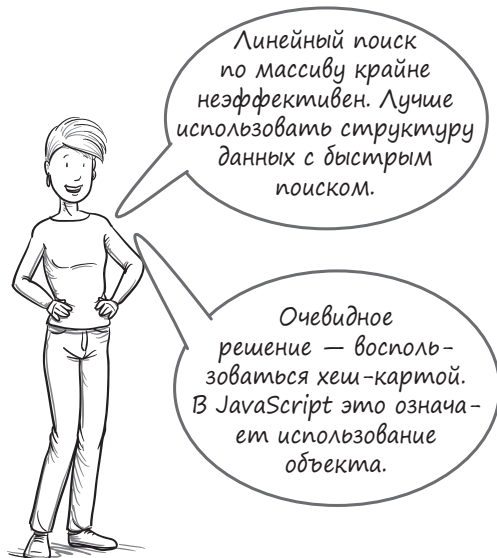
Вероятно, вы уже сталкивались с этим явлением в библиотеках или API. Предположим, вы используете API погодных данных от компании RainCo для создания метеорологического приложения. Ваша задача — задействовать API для вывода данных. Задача команды RainCo — реализация сервиса погодных данных. Ее вообще не интересует, что делает ваше приложение! API — абстрактный барьер, который явно разграничивает обязанности.

Команда разработки собирается проверить ограничения абстрактного барьера, изменяя нижележащую структуру данных корзины. Если абстрактный барьер построен правильно, команда маркетинга этого не заметит и их код вообще не изменится.

Паттерны

- ☒ Прямая реализация
- ☐ **Абстрактный барьер**
- ☐ Минимальный интерфейс
- ☐ Удобные уровни

Замена структуры данных корзины



```
function remove_item_by_name(cart, name) {
  var idx = indexOfItem(cart, name);
  if(idx !== null)
    return splice(cart, idx, 1);
  return cart;
}
```

```
function indexOfItem(cart, name) {
  for(var i = 0; i < cart.length; i++) {
    if(cart[i].name === name)
      return i;
  }
  return null;
}
```

Линейный поиск по массиву, который можно было бы заменить быстрым поиском в хеш-карте

Сара из команды разработки

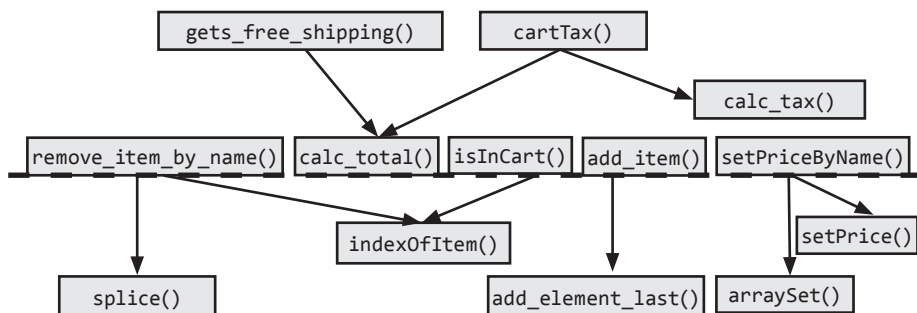
Сара обратилась с предложением: нужно разобраться с проблемой низкого быстродействия линейного поиска по массиву. Низкое быстродействие не должно скрываться за чистым интерфейсом.

Очевидное решение — попытаться использовать объект JavaScript (как хеш-карту) вместо массива. Все операции объектов JavaScript (добавление, удаление и проверка принадлежности) выполняются быстро.



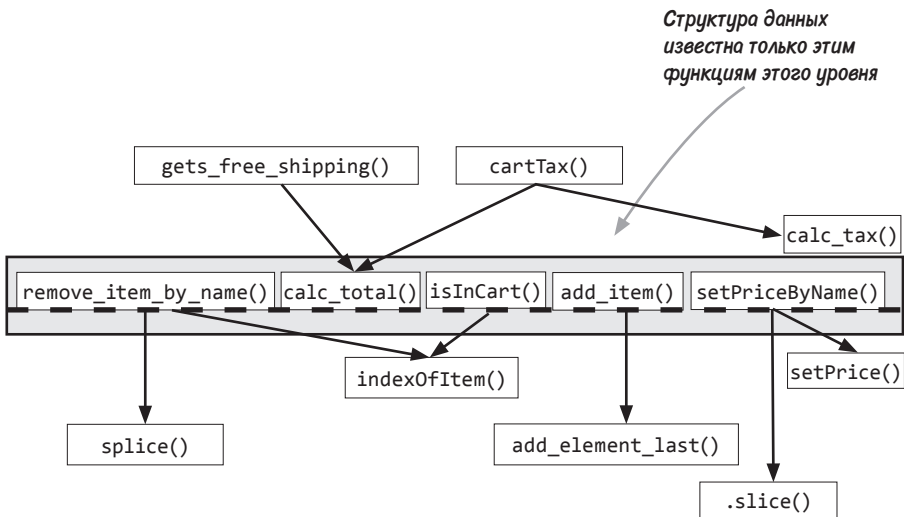
Ваш ход

Какие функции необходимо модифицировать для реализации этого изменения?



**Ответ**

Только функции на выделенном уровне. Никакие другие функции не предполагают, что корзина реализована в виде массива. Эти функции формируют абстрактный барьер.



Повторная реализация корзины в виде объекта

Повторная реализация корзины в виде объекта JavaScript сделает ее более эффективной, а также более прямолинейной (ура, паттерн 1!). Объект является более подходящей структурой данных для операций добавления и вставки в произвольной позиции.

Корзина в виде массива

```
function add_item(cart, item) {
    return add_element_last(cart, item);
}

function calc_total(cart) {
    var total = 0;

    for(var i = 0; i < cart.length; i++) {
        var item = cart[i];
        total += item.price;
    }
    return total;
}

function setPriceByName(cart, name, price)
{
    var cartCopy = cart.slice();
    for(var i = 0; i < cartCopy.length; i++)
    {
        if(cartCopy[i].name === name)
            cartCopy[i] =
                setPrice(cartCopy[i], price);
    }
    return cartCopy;
}

function remove_item_by_name(cart, name) {
    var idx = indexOfItem(cart, name);
    if(idx !== null)
        return splice(cart, idx, 1);
    return cart;
}

function indexOfItem(cart, name) {
    for(var i = 0; i < cart.length; i++) {
        if(cart[i].name === name)
            return i;
    }
    return null;
}

function isInCart(cart, name) {
    return indexOfItem(cart, name) !== null;
}
```

Корзина в виде объекта

```
function add_item(cart, item) {
    return objectSet(cart, item.name, item);
}

function calc_total(cart) {
    var total = 0;
    var names = Object.keys(cart);
    for(var i = 0; i < names.length; i++) {
        var item = cart[names[i]];
        total += item.price;
    }
    return total;
}

function setPriceByName(cart, name, price)
{
    if(isInCart(cart, name)) {
        var item = cart[name];
        var copy = setPrice(item, price);
        return objectSet(cart, name, copy);
    } else {
        var item = make_item(name, price);
        return objectSet(cart, name, item);
    }
}

function remove_item_by_name(cart, name) {
    return objectDelete(cart, name);
}
```

Эта функция становится лишней, удаляем ее

Встроенное средство для проверки наличия ключа в объекте

Иногда неясный код обусловлен использованием неподходящей структуры данных. Наш код стал более компактным и понятным — и более эффективным. При этом код отдела маркетинга работает без изменений!

Абстрактный барьер позволяет игнорировать подробности

Что позволяет изменить структуру данных без изменения всего кода, использующего корзину?

Изначально для хранения товаров в корзине использовался массив. Мы пришли к выводу, что этот выбор оказался неэффективным. Мы изменили группу функций, работающих с корзиной, и полностью заменили используемую структуру данных. При этом отделу маркетинга не пришлось изменять свой код.

Ему даже не обязательно знать о вносимых изменениях! Как нам это удалось?

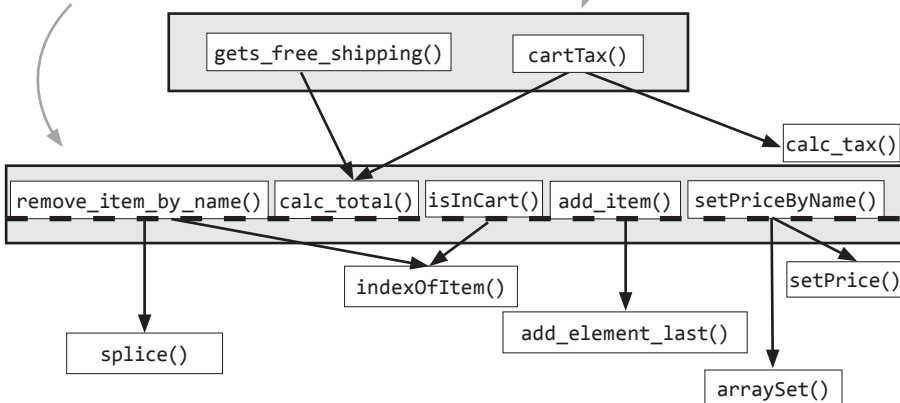
Причина, по которой нам удалось полностью заменить структуру данных с изменением всего пяти функций, заключается в том, что эти функции определяют абстрактный барьер. Абстракция — всего лишь необычное обозначение для вопроса: «Какие подробности я могу игнорировать?» Мы называем уровень абстрактным барьером, если все функции этого уровня позволяют нам игнорировать некоторое обстоятельство при работе над этим уровнем. Это промежуточный логический уровень, позволяющий игнорировать ненужные подробности.

Паттерны

- ☒ Прямолнейная реализация
- ☐ **Абстрактный барьер**
- ☐ Минимальный интерфейс
- ☐ Удобные уровни

Эти функции определяют абстрактный барьер для структуры данных корзины

Абстрактный барьер означает, что для этих функций структура данных не важна



Абстрактный барьер в данном случае означает, что функциям выше этого уровня не нужно знать, какая структура данных используется в реализации. Они могут пользоваться только этими функциями и рассматривать реализацию корзины как несущественную подробность. Это позволит переключиться с массива на объект так, что это изменение останется незамеченным для всех функций выше абстрактного барьера.

Обратите внимание: на диаграмме нет стрелок, пересекающих пунктирную линию. Например, если функция над линией вызовет `splice()` для корзины, она тем самым нарушит абстрактный барьер. Она будет использовать реализацию детали, которая должна быть несущественной для нее. Такая ситуация называется *неполным абстрактным барьером*. Проблема решается добавлением новой функции для завершения барьера.

Когда следует (или не следует!) использовать абстрактные барьеры

Абстрактные барьеры полезны для проектирования кода, но это не значит, что они должны применяться повсюду. В каких же ситуациях их рекомендуется использовать?

1. Для упрощения изменений реализации

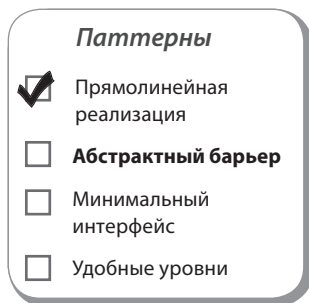
В ситуации высокой неопределенности с выбором реализации чего-либо абстрактный барьер может стать промежуточным уровнем, который позволяет изменить реализацию позднее. Это свойство может пригодиться, если вы строите прототип, но еще не знаете, какую реализацию лучше выбрать. А может, вы *знаете*, что что-то непременно изменится, просто еще не готовы заняться этим прямо сейчас (например, знаете, что данные позднее будут получены с сервера, а пока просто используете заглушку).

Впрочем, это преимущество часто превращается в ловушку, так что будьте осторожны. Мы часто пишем большой объем кода просто на случай, что что-то изменится в будущем. Почему? Чтобы не пришлось писать другой код! Глупо писать три строки сегодня, чтобы избежать написания трех строк завтра (которое может вообще не наступить): в 99 % случаев структура данных вообще не изменяется. В нашем примере она изменилась только по одной причине: команда разработки не переставала думать об эффективности до самых поздних стадий разработки.

2. Для упрощения чтения и написания кода

Абстрактные барьеры позволяют игнорировать подробности. Иногда эти подробности становятся настоящим рассадником ошибок. Правильно ли мы инициализировали переменные цикла? Нет ли ошибки смещения на 1 в условии выхода из цикла?

Абстрактный барьер, позволяющий игнорировать эти подробности, упростит написание кода. Если вы правильно выберете скрываемые подробности, менее опытные программисты смогут более эффективно работать при использовании вашего кода.



3. Для сокращения координации между командами

Команда разработки может изменять реализацию без предварительных обсуждений с отделом маркетинга. А отдел маркетинга сможет реализовать простые рекламные кампании без консультаций с разработчиками. Абстрактный барьер позволяет командам, работающим по разные стороны, забыть о подробностях, которыми занимается другая команда. Таким образом, каждая команда работает быстрее.

4. Для умственной концентрации на имеющейся задаче

А теперь мы подошли к настоящей ценности абстрактных барьеров: они упрощают анализ задачи, которую вы собираетесь решить. Давайте честно признаем: нам приходится беспокоиться о многочисленных подробностях. Абстрактный барьер делает некоторые подробности несущественными для той задачи, которую вы решаете в данный момент. А это значит, что сокращается риск того, что вы устанете и совершите ошибку.

Обзор паттерна 2. Абстрактный барьер

Абстрактный барьер — чрезвычайно мощный паттерн. Он надежно изолирует код, находящийся выше барьера, от кода, находящегося под барьером. Изоляция обеспечивается за счет определения подробностей, которые не обязательно учитывать по каждую сторону барьера.

Как правило, код над барьером может игнорировать подробности реализации: например, используемые структуры данных. В нашем примере для кода маркетинга (над барьером) несущественно, реализована ли корзина в виде массива или объекта.

Код на уровне барьера или под ним может игнорировать высокоуровневые подробности, например то, для чего используются функции. Функции на уровне барьера могут использоваться для чего угодно, им это знать не обязательно. В нашем примере для кода на уровне барьера совершенно не важна суть маркетинговой кампании.

Так работают все абстракции: они определяют, что может игнорировать код на более высоких и более низких уровнях. Любая конкретная функция может определять игнорируемые подробности: абстрактный барьер просто выражает это определение очень явно и сильно. Он заявляет, что никакому коду отдела маркетинга никогда не понадобится знать, как реализована корзина. Чтобы это стало возможно, все функции абстрактного барьера должны объединить усилия.

При этом следует избегать ловушки «упрощения будущих изменений». Абстрактные барьеры упрощают изменения, но это не основная причина для их использования. Их следует использовать стратегически, чтобы снизить уровень межгрупповых коммуникаций и прояснить запутанный код.

Паттерны

- ☒ Прямая реализация
- ☒ **Абстрактный барьер**
- ☐ Минимальный интерфейс
- ☐ Удобные уровни

Главное, что следует помнить об абстрактных барьерах, это то, что их суть заключается в игнорировании подробностей. Где будет полезно игнорировать подробности? Какие именно подробности поможет игнорировать ваш код? Удается ли вам найти группу функций, которые в совокупности помогают игнорировать одни и те же подробности?

Код становится более прямолинейным

Первый контакт с паттерном 1. Прямолинейная реализация

После изменения структуры данных многие функции становятся однострочными. Впрочем, количество строк не главное. Важно то, что решение выражается на правильном уровне общности и детализации. Как правило, в однострочных программах недостаточно места для смешения уровней, поэтому это хороший признак.

```
function add_item(cart, item) {
    return objectSet(cart, item.name, item);
}

function gets_free_shipping(cart) {
    return calc_total(cart) >= 20;
}

function cartTax(cart) {
    return calc_tax(calc_total(cart));
}

function remove_item_by_name(cart, name) {
    return objectDelete(cart, name);
}

function isInCart(cart, name) {
    return cart.hasOwnProperty(name);
}
```

Две функции все еще имеют сложные реализации:

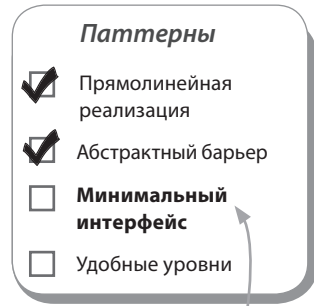
```
function calc_total(cart) {
    var total = 0;
    var names = Object.keys(cart);
    for(var i = 0; i < names.length; i++) {
        var item = cart[names[i]];
        total += item.price;
    }
    return total;
}

function setPriceByName(cart, name, price) {
    if(isInCart(cart, name)) {
        var itemCopy = objectSet(cart[name], 'price', price);
        return objectSet(cart, name, itemCopy);
    } else {
        return objectSet(cart, name, make_item(name, price));
    }
}
```

У нас пока еще отсутствуют некоторые средства, необходимые для того, чтобы сделать эти функции более прямыми. Эти средства будут описаны в главах 10 и 11. А пока осталось изучить еще два паттерна.

Паттерн 3. Минимальный интерфейс

Третий паттерн, который будет рассмотрен для выработки у вас чувства проектирования, — минимальный интерфейс. Этот паттерн предлагает подумать над тем, в каком месте должен находиться код для новой функциональности. Сохраняя минимальный интерфейс, мы избегаем нагромождения нижних уровней и их перенасыщения лишней функциональностью. Рассмотрим пример.



Отдел маркетинга хочет предоставить скидку на часы

Отдел маркетинга проводит новую кампанию: он хочет предоставить всем покупателям, корзина которых содержит много товаров, включая часы, 10%-ную скидку.

Кампания со скидкой на часы

Если общая стоимость корзины > \$100

и

корзина содержит часы,

то

предоставляется скидка 10 %.

Это условие необходимо реализовать в виде функции, возвращающей true или false

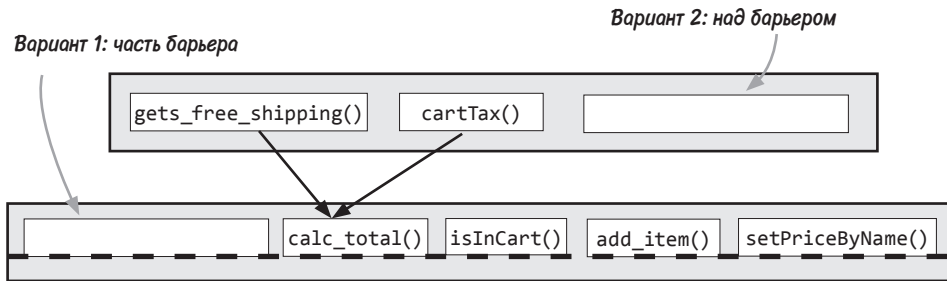


Вам поручено реализовать функцию, которая будет решать, кому полагается скидка. Ко вторнику успеете?

Директор по маркетингу

Два варианта реализации

Маркетинговую кампанию можно реализовать двумя способами. Во-первых, можно реализовать ее на одном уровне с абстрактным барьером. Во-вторых, ее можно реализовать над абстрактным барьером. Реализовать ее ниже абстрактного барьера невозможно, потому что тогда она не сможет вызываться кодом маркетинга. Какой вариант выбрать?



Вариант 1. Часть барьера

На уровне барьера можно обращаться к корзине как к хеш-карте. При этом мы не сможем вызывать какие-либо уровни того же уровня:

```
function getWatchDiscount(cart) {
    var total = 0;
    var names = Object.keys(cart);
    for(var i = 0; i < names.length; i++) {
        var item = cart[names[i]];
        total += item.price;
    }
    return total > 100 && cart.hasOwnProperty("watch");
}
```

Вариант 2. Над барьером

Над барьером работать с корзиной как с хеш-картой уже не получится. Придется проходить через функции, определяющие барьер:

```
function getWatchDiscount(cart) {
    var total = calcTotal(cart);
    var hasWatch = isInCart("watch");
    return total > 100 && hasWatch;
}
```



Пицца для ума

Как вы думаете, какой из вариантов лучше? Почему?

Реализация кампании над барьером лучше

Вариант с реализацией кампании над барьером (вариант 2) лучше по многим взаимосвязанным причинам. Во-первых, вариант 2 проще варианта 1, поэтому он предпочтительнее для паттерна 1. Вариант 1 увеличивает объем низкоуровневого кода в системе.

Вариант 1

```
function getWatchDiscount(cart) {
  var total = 0;
  var names = Object.keys(cart);
  for(var i = 0; i < names.length; i++) {
    var item = cart[names[i]];
    total += item.price;
  }
  return total > 100 &&
    cart.hasOwnProperty("watch");
}
```

Вариант 2

```
function getWatchDiscount(cart) {
  var total = calcTotal(cart);
  var hasWatch = isInCart("watch");
  return total > 100 && hasWatch;
}
```

Формально вариант 1 не нарушает абстрактный барьер. Стрелки не проходят через какие-либо барьеры. Тем не менее он противоречит цели, ради которой барьер создавался. Функция предназначена для маркетинговой кампании, а отдел маркетинга не хочет отвлекаться на такие подробности реализации, как циклы `for`. Поскольку в варианте 1 реализация размещается под барьером, заниматься ее сопровождением придется команде разработки. Чтобы изменить код, отделу маркетинга нужно будет согласовывать изменения с разработчиками. В варианте 2 такие проблемы отсутствуют.

Впрочем, существует и другая, менее очевидная проблема. Функции, образующие абстрактный барьер, являются частью контракта между отделом маркетинга и командой разработчиков. Добавление новой функции в абстрактный барьер увеличивает размер контракта. Если что-то потребуется изменить, изменения обойдутся дороже, потому что они требуют согласования условий контракта. Увеличивается объем кода, который необходимо понимать. Приходится держать в голове больше подробностей. Короче говоря, вариант 1 ослабляет преимущества абстрактного барьера.

Паттерн минимального интерфейса гласит, что новые уровни лучше записывать на более высоких уровнях (вместо расширения или изменения нижних уровней). К счастью, маркетинговая кампания достаточно тривиальна, и новая функция на уровне абстрактного барьера для нее не понадобится. Тем не менее есть много случаев, далеко не столь очевидных. Паттерн минимального интерфейса рекомендует

Прямолинейная реализация

- Вариант 1
- ✓ Вариант 2

Абстрактный барьер

- Вариант 1
- ✓ Вариант 2

Минимальный интерфейс

- Вариант 1
- ✓ Вариант 2

решать задачи на верхних уровнях, избегая модификации нижних уровней. И этот паттерн применим ко всем уровням, не только к абстрактным барьерам.

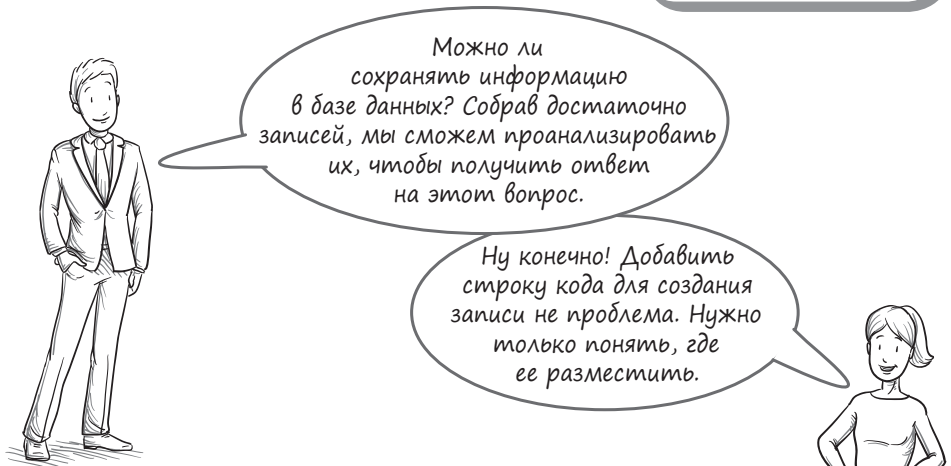
Рассмотрим более сложный пример, в котором даже у лучших проектировщиков появится соблазн принять неверное решение.

Отдел маркетинга хочет сохранять в журнале товары, добавленные в корзину

Отдел маркетинга хочет реализовать еще одну возможность. Люди добавляют товары в свои корзины, а потом уходят, не оформив заказ. Почему? Отдел маркетинга хочет получить больше информации, чтобы получить ответ на этот вопрос и повысить продажи. Они просят сохранять информацию каждый раз, когда кто-то добавляет товар в корзину.

Паттерны

- ☒ Прямолнейная реализация
- ☒ Абстрактный барьер
- ☐ Минимальный интерфейс
- ☐ Удобные уровни



Директор по маркетингу

Дженна создает таблицу базы данных и пишет действие для сохранения записи в базе данных. Вызов может выглядеть так:

```
logAddToCart(user_id, item)
```

Теперь необходимо его где-то разместить. Дженна предлагает включить его в функцию `add_item()`:

```
function add_item(cart, item) {
  logAddToCart(global_user_id, item);
  return objectSet(cart, item.name, item);
}
```



**Дженна
из команды
разработки**

Насколько это уместно? Взглянем на происходящее с точки зрения разработчика. Чего мы добиваемся, размещая вызов именно здесь? Что теряем? Рассмотрим возможные последствия.

Последствия выбора

Конечно, предложение Дженны выглядит разумно. Информация должна сохраняться каждый раз, когда пользователь добавляет товар в корзину, а это происходит в функции `add_item()`. К тому же это упрощает задачу, потому что функция будет сохранять информацию за нас. Нам не нужно помнить об этом. При работе над этим уровнем эту подробность (необходимость сохранения информации) можно игнорировать.

Тем не менее у сохранения информации внутри `add_item()` есть серьезные проблематичные последствия. Прежде всего `logAddToCart()` является действием. При вызове действия из `add_item()` сама функция также становится действием. По правилу распространения все, что вызывает `add_item()`, также становится действием. Это может иметь серьезные последствия для тестирования.

Поскольку функция `add_item()` является вычислением, ранее нам разрешалось использовать ее где угодно и когда удобно. Пример:

```
function update_shipping_icons(cart) {
  var buttons = get_buy_buttons_dom();
  for(var i = 0; i < buttons.length; i++) {
    var button = buttons[i];
    var item = button.item;
    var new_cart = add_item(cart, item);
    if(gets_free_shipping(new_cart))
      button.show_free_shipping_icon();
    else
      button.hide_free_shipping_icon();
  }
}
```

*Вызываем `add_item()` без
добавления товара в корзину*

*Этот вызов определенно
не должен регистрироваться!*

`update_shipping_icons()` использует `add_item()`, даже если пользователь не добавил товар в корзину. Эта функция вызывается каждый раз, когда товар отображается для пользователя! Мы не хотим регистрировать эти товары как уже добавленные в корзину.

Наконец (и это самое важное), у нас имеется удобный, логичный набор функций для работы с корзиной — интерфейс. Это можно только приветствовать. Такой интерфейс удовлетворяет нашим потребностям, он позволяет игнорировать соответствующие подробности. Предлагаемое изменение не улучшает интерфейс. Вызов `logAddToCart()` следовало бы разместить над абстрактным барьером. Попробуем сделать это на следующей странице.

Более правильное место для сохранения добавлений товаров в корзину

Мы совершенно точно знаем о `logAddToCart()` две вещи: это действие, и оно должно располагаться выше абстрактного барьера. Но где именно?

И снова речь идет о решении из области проектирования, поэтому не существует ответа, правильного в любом контексте. Тем не менее функция `add_item_to_cart()` может стать хорошим вариантом: это обработчик, привязанный нами к кнопке добавления товара. Здесь мы можем быть уверены, что правильно сохраняем намерение пользователя. Кроме того, эта функция уже является действием. Она определяет «все, что должно произойти, когда пользователь добавляет товар в корзину». Вызов `logAddToCart()` — это всего лишь еще одна задача.



```
function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart = add_item(shopping_cart, item);
  var total = calc_total(shopping_cart);
  set_cart_total_dom(total);
  update_shipping_icons(shopping_cart);
  update_tax_dom(total);
  logAddToCart();
}
```

Обработчик клика на кнопке добавления в корзину

Другие действия, которые должны вызываться при клике

Можно добавить вызов сюда вместе со всеми остальными операциями, которые должны выполняться при добавлении товаров в корзину

Это не лучшее из возможных решений, но в этой конкретной архитектуре отсюда хорошо вызывать функцию. Без полной переработки нашего приложения вызов должен выполняться именно здесь.

Мы почти разместили вызов функции в неподходящем месте, но нам повезло. К счастью, мы вовремя вспомнили правило распространения действий, а также вспомнили о вызове `add_item()` из `update_shipping_icons()`.

Тем не менее нельзя полагаться на везение при проектировании. Нужен принцип, который бы позволил избежать этого. Именно для этого и нужен паттерн минимального интерфейса.

Паттерн минимального интерфейса предлагает нам сосредоточиться на ощущении чистого, простого и надежного интерфейса и пользоваться им для разрешения непредвиденных последствий в остальном коде. Паттерн направляет наши усилия по защите интерфейса от ненужных изменений или расширений.

Обзор паттерна 3. Минимальный интерфейс

Функции, определяющие абстрактный барьер, могут рассматриваться как интерфейс. Они предоставляют операции, посредством которых мы можем обращаться к набору значений и оперировать им. В многоуровневом проектировании возникает динамическое равновесие между полнотой абстрактного барьера и паттерном, обеспечивающим его минимализм.

Существует много причин для поддержания минимального абстрактного барьера.

1. Если добавить в барьер дополнительный код, то нам придется вносить больше модификаций при изменении реализации.
2. Код на уровне барьера относится к нижнему уровню, поэтому он с большей вероятностью содержит ошибки.
3. Низкоуровневый код сложнее понять.
4. Увеличение числа функций в абстрактном барьере означает необходимость дополнительной координации между командами.
5. Более крупный интерфейс к абстрактному барьеру сложнее удержать в голове.

Здесь очень важно понимать, как функции уровня служат своей цели. Хорошо ли они справляются со своей задачей при небольшом количестве функций? Будут ли изменения соответствовать этой цели?

Паттерны

- ☒ Прямолнейная реализация
- ☒ Абстрактный барьер
- ☒ **Минимальный интерфейс**
- ☐ Удобные уровни

Паттерн 4. Удобные уровни

Первые три паттерна предполагали построение уровней. Объяснялось, как это лучше делать, и определялись идеалы, к которым нужно стремиться. Четвертый и последний паттерн — *удобные уровни* — обращается к практической стороне.

Часто кажется, что структура уровней должна быть очень высокой. Только посмотрите, сколько подробностей, — и я обо всех подумал! Тем не менее определить жизнеспособные уровни абстракции обычно бывает непросто. Часто со временем мы понимаем, что построенный нами абстрактный барьер был не таким уж полезным. Он оказался неполным. Или с ним архитектура была менее удобной, чем без него. Всем нам доводилось строить слишком высокие башни из абстракций. Исследования и последующие неудачи были частью процесса. Слишком большие архитектуры трудно строить.

В некоторых ситуациях абстракция также может превратить невозможное в возможное. Взгляните на язык JavaScript, предоставляющий удобный абстрактный барьер над машинным кодом. Кто думает о машинных командах, программируя на JavaScript? Да это невозможно! JavaScript делает слишком много, и реализации слишком сильно различаются.

Как был спроектирован и построен настолько полезный уровень? Тысячи человеко-лет работы за несколько десятилетий ушли на построение мощных парсеров, компиляторов и виртуальных машин.

Как работающие в отрасли программисты, получившие задачу, которая должна решаться программными средствами, мы не всегда располагаем такой роскошью, как поиск и построение хороших абстракций. На это уходит слишком много времени. Бизнес не может позволить себе ждать.

Паттерн удобных уровней предоставляет практический критерий для определения того, когда следует прекращать поиск других паттернов (и когда начать заново). Мы спрашиваем себя: «Нам удобно?» Если вам удобно работать с кодом, можно прекратить проработку архитектуры. Пусть циклы `for` так и останутся неупакованными. Пусть стрелки будут длинными, а уровни будут вращать друг в друга.

Тем не менее если вам неудобно хранить в голове большое количество подробностей или код начинает казаться недостаточно чистым, начните применять паттерны снова. Кодовая база не бывает идеальной. Существует постоянное взаимное противодействие между проектированием и потребностью в новой функциональности. Пусть удобство определит, когда вам лучше остановиться. Фактически вы и ваша команда живете в этом коде. Ваша задача — добиться того, чтобы он удовлетворял вашим потребностям как программистов, а также потребностям бизнеса.

На этом наше изучение четырех паттернов многоуровневой архитектуры завершается. Подведем итог, прежде чем бросить последний взгляд на граф вызовов и понять, сколько информации нам удастся из него извлечь.



Паттерны многоуровневой архитектуры

Мы подошли к концу, поэтому просто для удобства напомним четыре паттерна, которые изучались в этих двух главах.

Паттерн 1. Прямолинейная реализация

Структура уровней при многоуровневом проектировании должна помочь нам в построении прямолинейных реализаций. Когда мы читаем функцию с прямолинейной реализацией, задача, представленная сигнатурой функции, должна решаться на правильном уровне детализации. Слишком много подробностей — признак «кода с душком».

Паттерны

- ☒ Прямолинейная реализация
- ☒ Абстрактный барьер
- ☒ Минимальный интерфейс
- ☒ Удобные уровни

Паттерн 2. Абстрактный барьер

Некоторые уровни графа предоставляют интерфейс, который позволяет скрыть важные подробности реализации. Это позволяет нам писать код на более высоком уровне.

Паттерн 3. Минимальный интерфейс

По мере развития системы интерфейсы, относящиеся к важным бизнес-концепциям, должны сходиться к небольшому, но мощному набору операций. Все остальные операции должны определяться с точки зрения этих операций (прямо или косвенно).

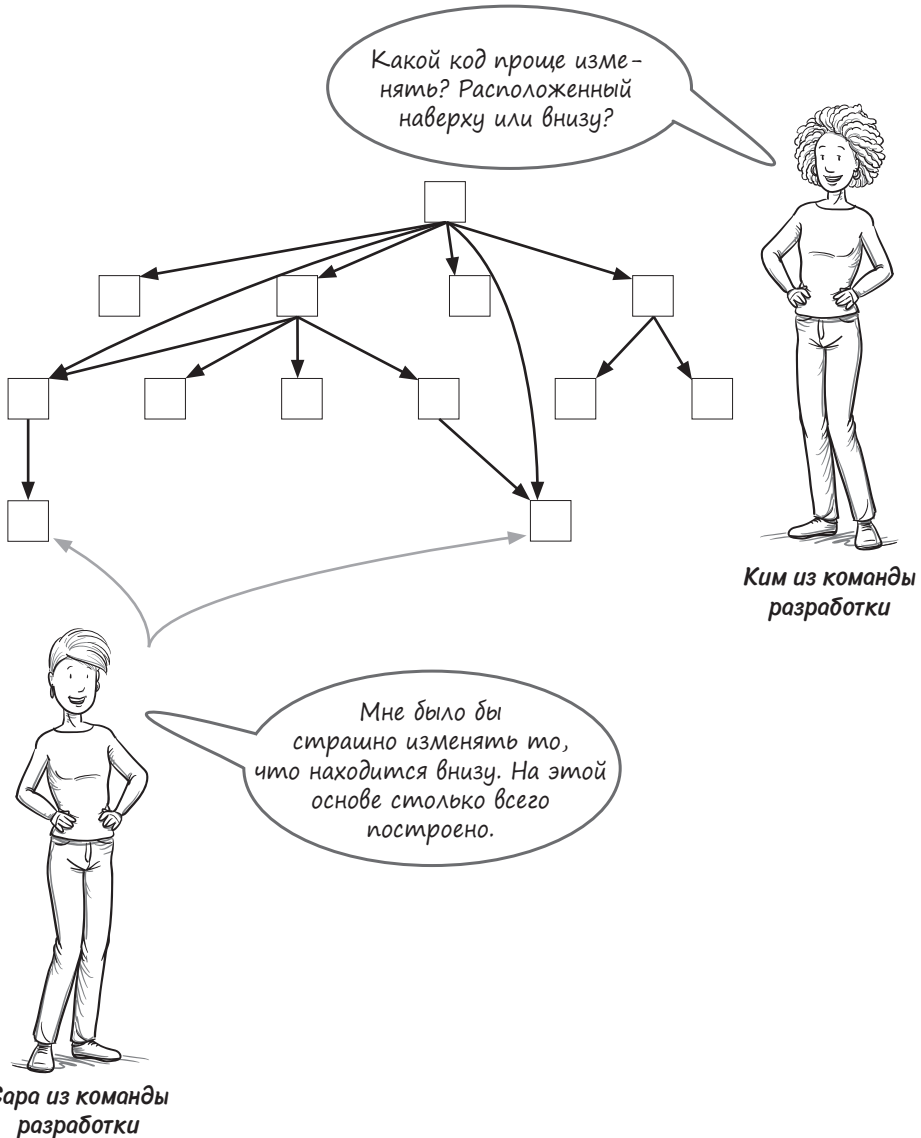
Паттерн 4. Удобные уровни

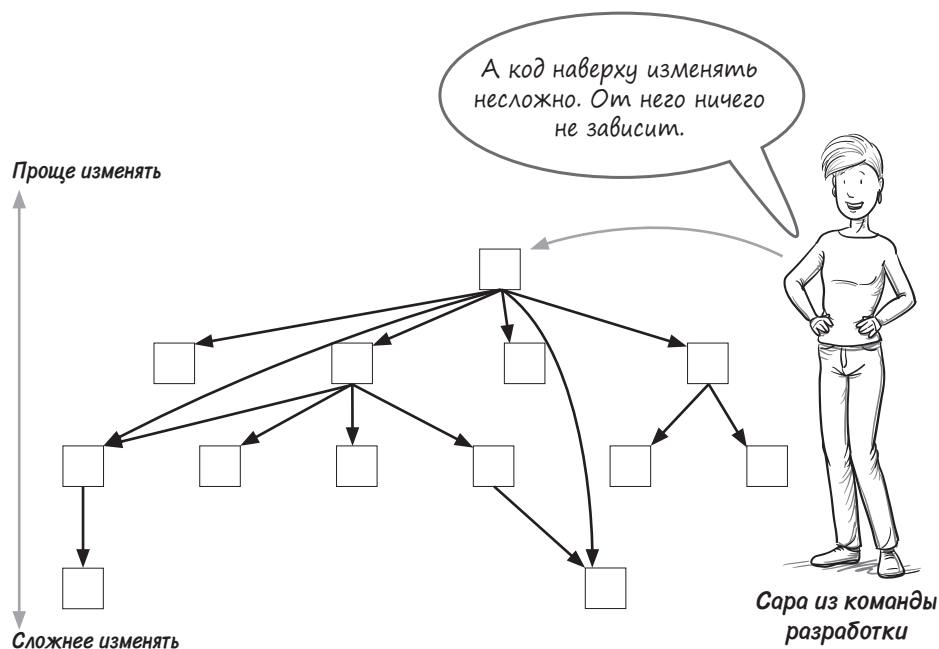
Паттерны и практики многоуровневого проектирования должны служить нашим потребностям как программистов, которые, в свою очередь, должны обслуживать бизнес. Необходимо выделить время на уровни, которые помогут нам выдавать программный продукт быстрее и с более высоким качеством. Уровни не должны добавляться для забавы. С кодом и его уровнями абстракции должно быть удобно работать. Если это условие выполняется, улучшать архитектуру просто ради самого процесса не стоит.

Теперь рассмотрим граф вызовов на абстрактном уровне, чтобы понять, что из него можно узнать об удобстве тестирования, внесения изменений и повторного использования. Мы будем учитывать эти факторы при добавлении кода на разных уровнях.

Код в верхней части графа проще изменять

Как по диаграмме абстрактного графа вызовов (без имен функций) определить, какой код будет проще изменить? Зная это, вы поймете, где разместить код, реализующий быстро изменяющиеся требования (например, бизнес-правила). Также вы будете знать, где разместить код, изменяющийся в наименьшей степени. Правильное размещение позволит кардинально сократить затраты на сопровождение.





Сара права. Код в верхней части графа проще изменить. Когда вы изменяете функцию на самом верхнем уровне, вам не нужно думать о том, что еще вызывает этот код, потому что его ничего не вызывает. Вы можете полностью изменить его поведение без каких-либо последствий для вызывающего кода.

Сравните с функциями нижнего уровня. От их поведения зависят три уровня функций. Изменяя внешнее поведение таких функций, вы изменяете поведение всех компонентов на пути до верхнего уровня. Именно это затрудняет безопасное внесение изменений.

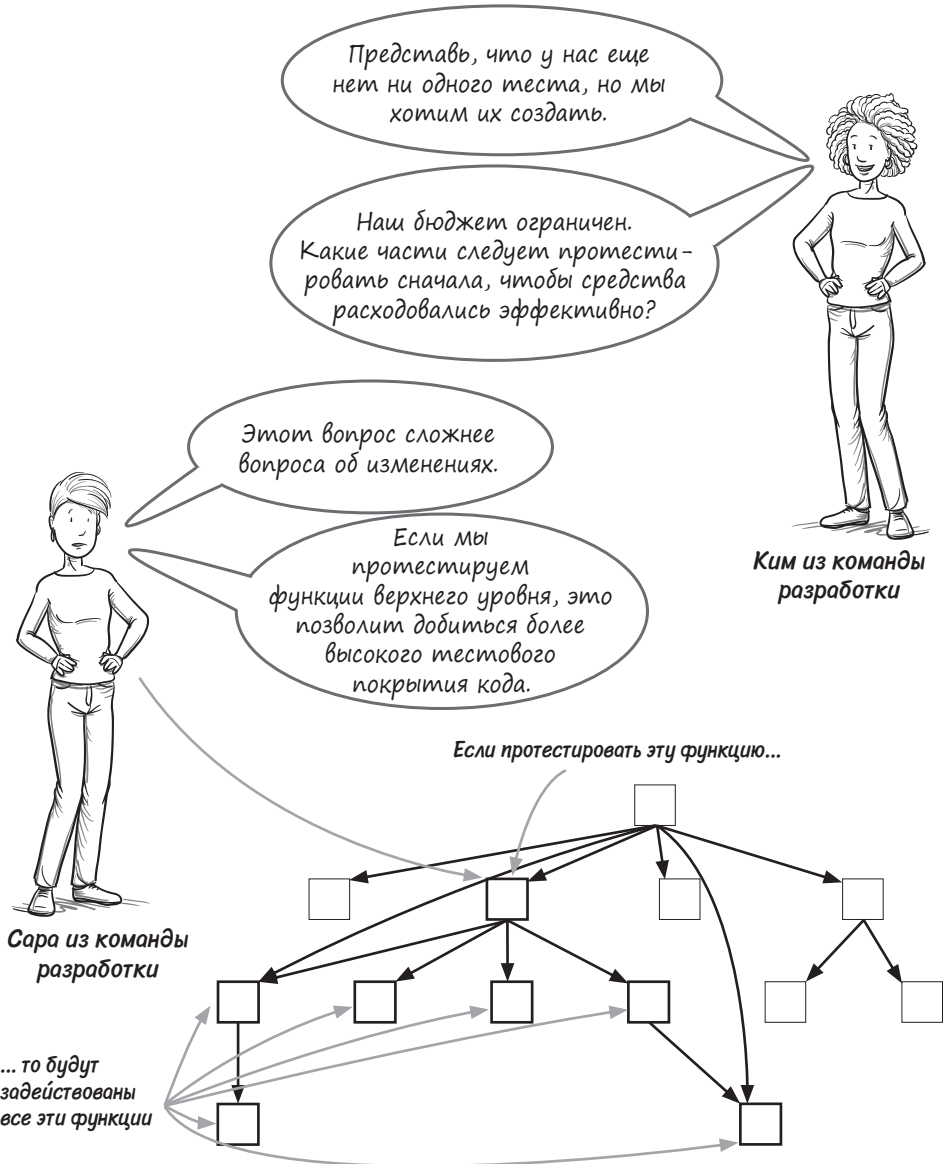
Мы хотим, чтобы известный нам код в нижней части реализовал функции, не изменяющиеся с течением времени. Собственно, этим и объясняется то, что копирования при записи размещаются в нижней части. Достаточно правильно реализовать их один раз и никогда не изменять в будущем. Когда мы выделяем функции на нижнем уровне (паттерн 1) или добавляем функции на более высоких уровнях (паттерн 3), мы разделяем код на уровни изменений.

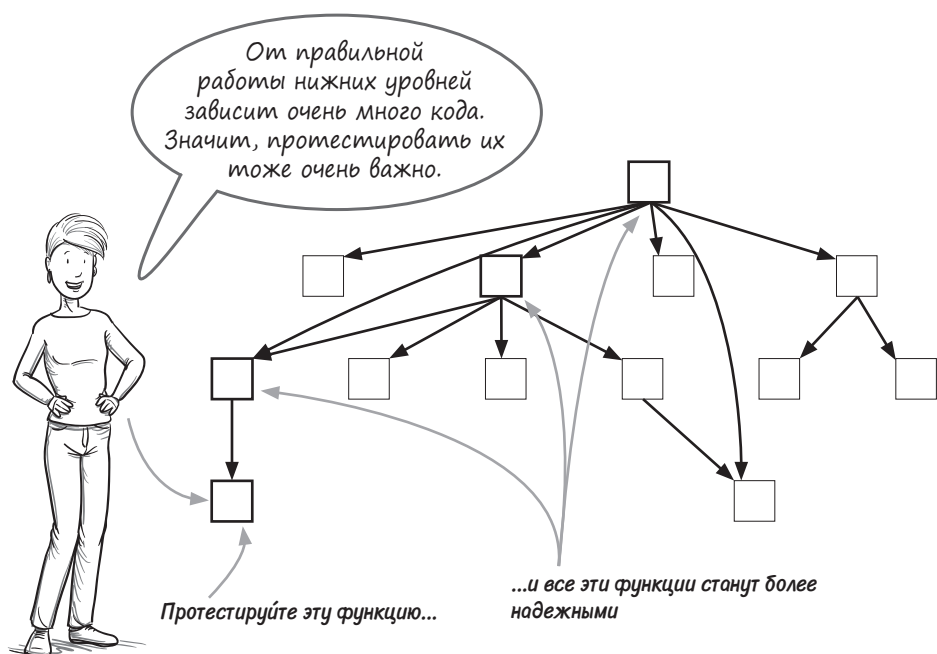
.....
**Чем длиннее путь сверху
 вниз до функции, тем
 дороже обойдется
 изменение функции.**

.....
**Если часто изменяющийся код будет располагаться повыше,
 это упростит нашу задачу. Старайтесь не строить
 на фундаменте, который может изменяться.**

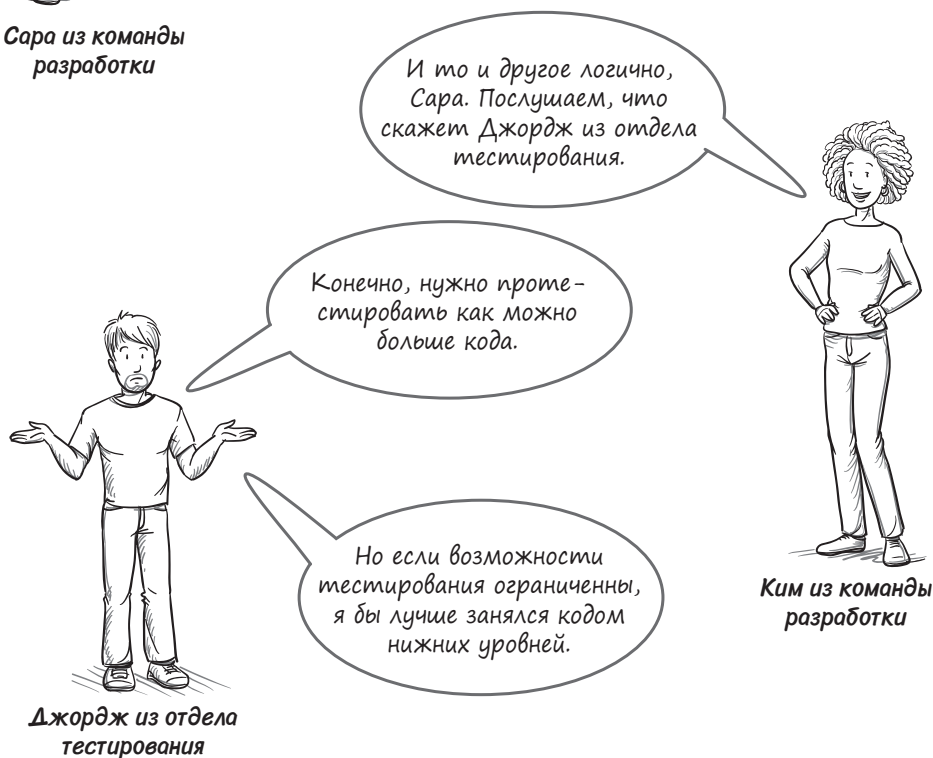
Важность тестирования кода нижних уровней

Посмотрим, как по этому графу определить, какой код важнее протестировать. Казалось бы, тестировать нужно весь код, однако на практике это не всегда возможно. Если протестировать все невозможно, то на что лучше направить свои усилия, чтобы потраченное время наиболее эффективно окупилось в отдаленной перспективе?

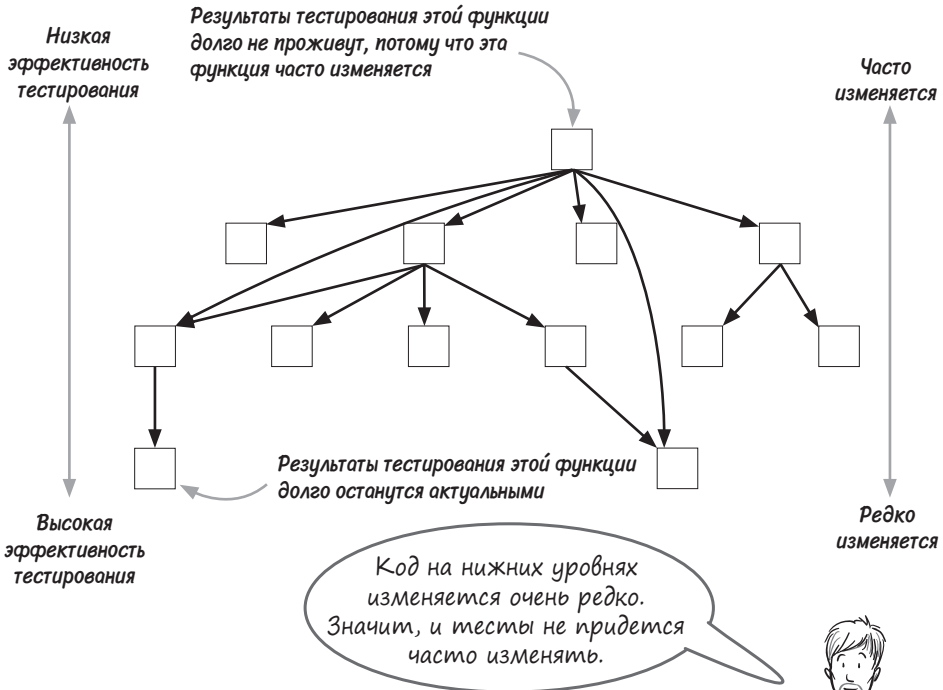




Сара из команды разработки



*Если все сделано правильно, код верхнего уровня
изменяется чаще кода нижних уровней.*



Тестирование требует времени, и мы хотим, чтобы это время было потрачено с максимальной эффективностью. Если все было сделано правильно, часто изменяющийся код будет располагаться наверху, а более стабильный код останется на нижних уровнях. Поскольку код верхних уровней часто изменяется, любые тесты, написанные для этого кода, также будут часто изменяться, чтобы соответствовать новому поведению. С другой стороны, код нижних уровней изменяется очень редко, поэтому и тесты тоже будут оставаться без изменений.

Наши паттерны помогают разделить код на уровни, соответствующие удобству тестирования. При извлечении функций на нижние уровни или построении функций на верхних уровнях мы выбираем, насколько ценными будут их тесты.

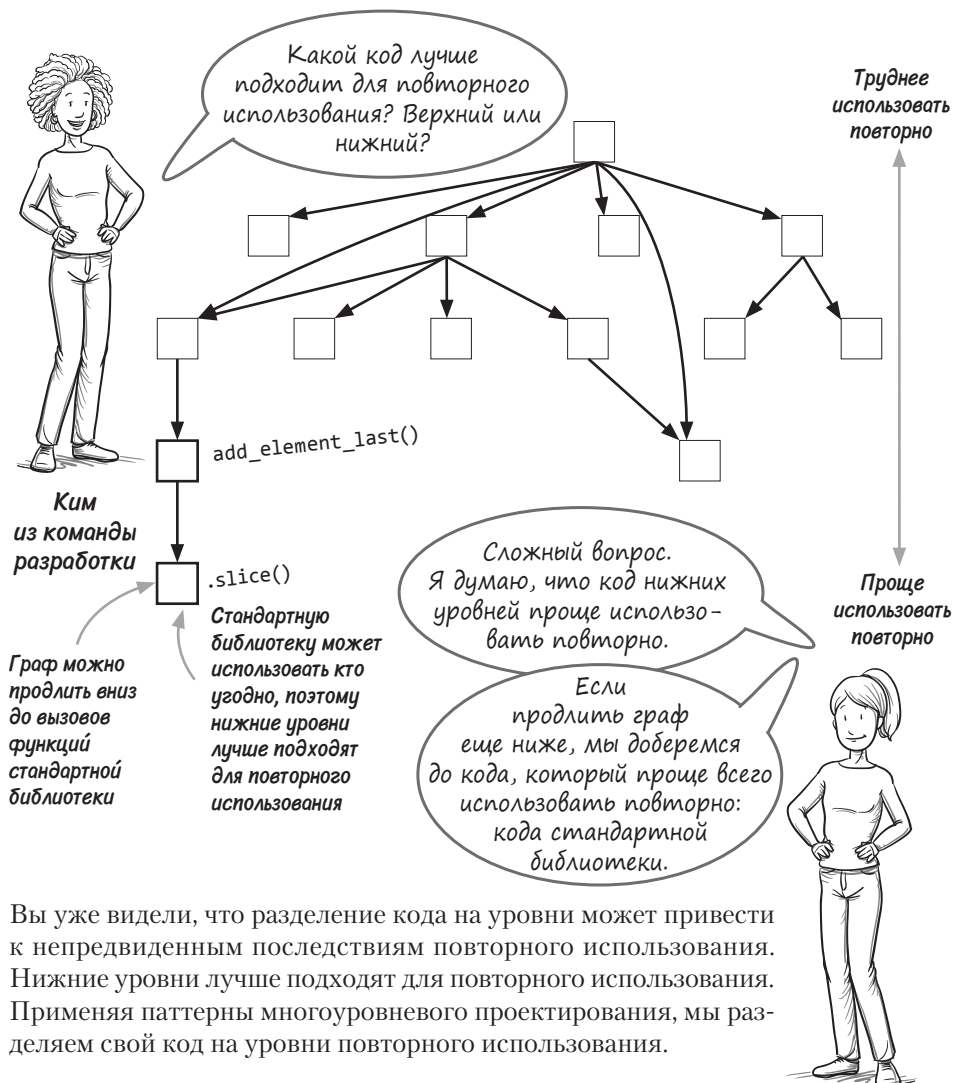
Верхний код
изменяется часто,
и тесты будут таки-
ми же эфемерными.



**Тестирование кода на нижних
уровнях оказывается более
результативным
в отдаленной перспективе.**

Код нижних уровней лучше подходит для повторного использования

Вы уже видели, что код верхнего уровня проще изменяется, а код нижних уровней важнее для тестирования. Какой код будет проще повторно использовать? Повторно используемый код не нужно писать, тестировать или изменять дважды. Повторное использование экономит время и деньги.



Вы уже видели, что разделение кода на уровни может привести к непредвиденным последствиям повторного использования. Нижние уровни лучше подходят для повторного использования. Применяя паттерны многоуровневого проектирования, мы разделяем свой код на уровни повторного использования.

Чем больше кода расположено под функцией, тем хуже она подходит для повторного использования.

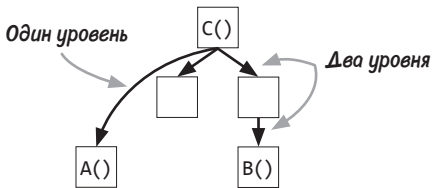
Дженна из команды разработки

Итоги: что можно узнать о коде по графу вызовов

Вы уже видели, что граф вызовов может сообщить много полезной информации о нефункциональных требованиях (НФТ) нашего кода. Ниже приводится краткая сводка НФТ, переформулированных в виде простых и надежных правил.

Удобство сопровождения

Правило: *чем меньше функций встречается на пути к вершине графа, тем проще изменять функцию.*

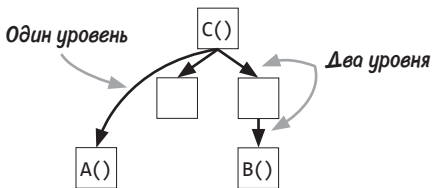


- A() проще изменить, чем B(). Над A() находится одна функция, а над B() — две.
- C() изменять проще всего, потому что над ней нет ни одной функции.

Вывод: *часто изменяемый код следует размещать на верхнем уровне.*

Удобство тестирования

Правило: *чем больше функций встречается на пути к вершине графа, тем выше ценность тестирования.*

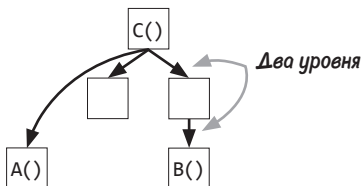


- При тестировании функция B() обладает более высокой ценностью, чем A(), потому что от нее зависит больше кода (две функции).

Вывод: *в первую очередь следует тестировать код нижних уровней.*

Удобство повторного использования

Правило: *чем меньше функций расположено ниже функции, тем лучше она подходит для повторного использования.*



- Функции A() и B() в одинаковой степени подходят для повторного использования. Под каждой из них нет ни одной функции.
- Функция C() хуже всего подойдет для повторного использования, потому что под ней располагаются два уровня функций.

Вывод: *выталкивайте функции на нижние уровни, чтобы расширить возможности их повторного использования.*

Эти свойства обусловлены структурой кода. Используйте их с целью определения оптимального набора уровней для изменения, тестирования и повторного использования кода. Практическое применение будет продемонстрировано в главе 16, когда мы займемся исследованием многослойной архитектуры.

Итоги главы

Многоуровневое проектирование — метод упорядочения функций по уровням абстракции, при котором каждая функция реализуется через вызовы функций более низкого уровня. Мы следуем своим интуитивным представлениям, чтобы преобразовать код в более удобную систему для удовлетворения потребностей бизнеса. По структуре уровней также можно определить, какой код лучше подходит для тестирования, модификации и повторного использования.

Резюме

- Паттерн абстрактного барьера позволяет мыслить на более высоком уровне. Абстрактные барьеры позволяют полностью скрывать те или иные детали.
- Паттерн минимального интерфейса направлен на построение уровней, которые сходятся к итоговой форме. Интерфейсы важных бизнес-концепций не должны расширяться или изменяться после того, как они стабилизировались.
- Паттерн удобства помогает применять другие паттерны, чтобы они соответствовали нашим целям. При применении паттернов легко увлечься чрезмерным абстрагированием. Паттерны следует применять сознательно.
- По структуре графа вызовов можно судить о том, где следует разместить некоторые свойства, которые сообщают нам, где следует разместить код для достижения максимального удобства тестирования, сопровождения и повторного использования.

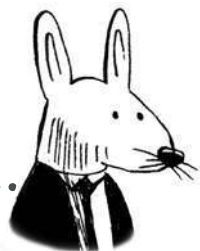
Что дальше?

Вместе с этой главой подходит к концу первый большой этап нашего пути. Вы узнали о действиях, вычислениях и данных и о том, как они проявляются в коде. Мы часто занимались рефакторингом, однако при этом нам встречались некоторые функции, не поддающиеся простому извлечению. В следующей главе вы узнаете, как правильно абстрагировать циклы `for`. А еще в ней начнется следующий этап нашего пути, в котором вы научитесь использовать код как данные.

Часть II

Первоклассные абстракции

Проведение различий между действиями, вычислениями и данными позволило нам освоить много новых полезных навыков. Конечно, понимание этих различий пригодится нам и далее. Но мы должны освоить новый навык, чтобы перейти на следующий этап путешествия. Речь идет о концепции первоклассных значений, прежде всего первоклассных функций. Вооружившись новыми знаниями, можно заняться изучением функционального подхода к перебору. Сложные вычисления можно строить из цепочек операций. Вы найдете новые возможности для работы с глубоко вложенными данными. А попутно научитесь управлять порядком и повторением действий для устранения ошибок синхронизации. Наше путешествие завершится знакомством с двумя архитектурами, которые позволяют определять структуру наших сервисов. Все это станет доступно вам после того, как вы узнаете о первоклассных значениях.



В этой главе

- ✓ Мощность первоклассных значений.
- ✓ Реализации элементов синтаксиса в виде первоклассных функций.
- ✓ Упаковка элементов синтаксиса с использованием функций высшего порядка.
- ✓ Применение двух методов рефакторинга, использующих первоклассные функции и функции высшего порядка.

Вы находитесь в зале ожидания перед второй частью книги. В нем есть дверь с надписью «Первоклассные функции». Эта глава откроет перед вами эту дверь и новый мир замечательных идей, относящихся к первоклассным функциям. Что такое первоклассные функции? Для чего они используются? Как они создаются? В этой главе вы найдете ответы на все эти вопросы. В остальных главах исследуется лишь малая часть широкого круга их практических применений.

В этой главе вы узнаете новый признак «кода с душком» и два метода рефакторинга, которые помогут устранить дублирование кода и найти более эффективные абстракции. Новые навыки будут применяться в этой главе и во всей части II. Не пытайтесь понять все прямо сейчас: это всего лишь краткая сводка. Каждая тема будет более подробно рассмотрена тогда, когда она понадобится нам в этой главе.

Признак «кода с душком»: неявный аргумент в имени функции

Этот признак указывает на аспекты кода, которые лучше было бы выразить в виде первоклассных значений. Если вы ссылаетесь на значение в теле функции и имя этого значения присутствует в имени функции, то, вероятно, перед вами проблема, которая должна решаться посредством описанного ниже рефакторинга.



Загляни в словарь

«Код с душком» — характеристика фрагмента кода, которая может оказаться симптомом более глубоких проблем.

Характеристики

1. Очень похожие реализации функции.
2. Имя функции указывает на различия в реализации.

Рефакторинг: явное выражение неявного аргумента

Если в имени функции присутствует неявный аргумент, как преобразовать его в реальный аргумент функции? Этот рефакторинг добавляет новый аргумент в функцию, чтобы значение стало первоклассным. Он поможет вам лучше выразить ваши намерения в коде, а иногда также будет противодействовать дублированию кода.

Последовательность действий

1. Выявление неявного аргумента в имени функции.
2. Добавление явного аргумента.
3. Использование нового аргумента в теле вместо жестко фиксированного значения.
4. Обновление кода вызова.

Рефакторинг: замена тела обратным вызовом

Этот метод рефакторинга позволяет вам заменить тело (изменяющуюся часть) блоком кода с обратным вызовом. В дальнейшем нужное поведение передается первоклассной функции. Это мощный механизм создания функций высшего порядка на базе существующего кода.

Последовательность действий

1. Определение частей: предшествующей, тела и завершающей.
2. Выделение всего кода в функцию.
3. Извлечение тела в функцию, которая передается в аргументе этой функции.

Эти три идеи дают неплохое представление о структуре главы. Мы будем использовать их в этой главе, а также в следующих восьми главах.

Отдел маркетинга все еще должен согласовывать действия с разработчиками

Абстрактный барьер, который позволил предоставить удобный API для отдела маркетинга, работал, но не так, как было задумано. Да, большая часть работы могла выполняться без координации, но часто отдел маркетинга предлагал команде разработчиков добавить в API новые функции, которые нельзя было выполнить средствами существующего API. Примеры таких запросов:

Результаты поиска: 2343 запроса на изменения от отдела маркетинга

Запрос на изменение: возможность задать цену товара в корзине			Назначить цену
Приоритет: СРОЧНО!!! Необходимо для распродажи с купонами на следующей неделе.		Запрос поступил: Директор по маркетингу	
		Ответственный: Дженна из команды разработки	
Запрос на изменение: возможность задать количество единиц товара в корзине			Назначить количество
Приоритет: СРОЧНО!!! Необходимо для воскресной акции на этой неделе.		Запрос поступил: Директор по маркетингу	
		Ответственный: Дженна из команды разработки	
Запрос на изменение: возможность задать способ доставки для товара в корзине			Назначить способ доставки
Приоритет: КРАЙНЕ СРОЧНО!!! Необходимо для рекламной акции с половинной стоимостью доставки, которая начинается ЗАВТРА!!!		Запрос поступил: Директор по маркетингу	Очень похожие запросы, которые отличаются только присваиваемым полем
		Ответственный: Дженна из команды разработки	

Подобных запросов очень много, и они продолжают поступать. Все запросы очень похожи — даже код их реализации был похожим. Разве абстрактный барьер не должен был это предотвратить? До этого отдел маркетинга мог просто обратиться к структуре данных. Теперь он снова вынужден ждать команду разработки. Очевидно, что абстрактный барьер не работает.

Признак «кода с душком»: неявный аргумент в имени функции

Команда маркетинга должна иметь возможность изменять товары в корзине для реализации своих рекламных акций, например назначить некоторым товарам бесплатную доставку или обнулить их цену. Команда разработки потрудились и написала функции, удовлетворяющие потребностям маркетинга. Тем не менее выглядят эти функции очень похоже. Ниже приведены четыре функции, у которых действительно очень много общего:

```
function setPriceByName(cart, name, price) {
  var item = cart[name];
  var newItem = objectSet(item, 'price', price);
  var newCart = objectSet(cart, name, newItem);
  return newCart;
}
```

```
function setQuantityByName(cart, name, quant) {
  var item = cart[name];
  var newItem = objectSet(item, 'quantity', quant);
  var newCart = objectSet(cart, name, newItem);
  return newCart;
}
```

Функции отличаются только этими строками

```
function setShippingByName(cart, name, ship) {
  var item = cart[name];
  var newItem = objectSet(item, 'shipping', ship);
  var newCart = objectSet(cart, name, newItem);
  return newCart;
}
```

```
function setTaxByName(cart, name, tax) {
  var item = cart[name];
  var newItem = objectSet(item, 'tax', tax);
  var newCart = objectSet(cart, name, newItem);
  return newCart;
}
```

Имя функции повторяет строку

Имя функции повторяет строку

Напомню, что функция `objectSet()` была определена в главе 6; снова приведу определение, чтобы напомнить его

```
function objectSet(object, key, value) {
  var copy = Object.assign({}, object);
  copy[key] = value;
  return copy;
}
```

В этом коде присутствует серьезный признак «кода с душком». Впрочем, если честно, от этих строк вообще разит. Первый и самый заметный признак — дублирование кода. Эти четыре функции почти идентичны.

Но есть и другой, более тонкий признак: главное различие между функциями — строки, определяющие поле, — также содержится в имени функции. Все выглядит так, словно имя функции (или его часть) передается в аргументе. Вот почему этот признак называется *неявным аргументом в имени функции*. Вместо явной передачи в аргументе значение «передается» как часть имени.



Чем-то пахнет...

Неявный аргумент в имени функции обладает двумя характеристиками:

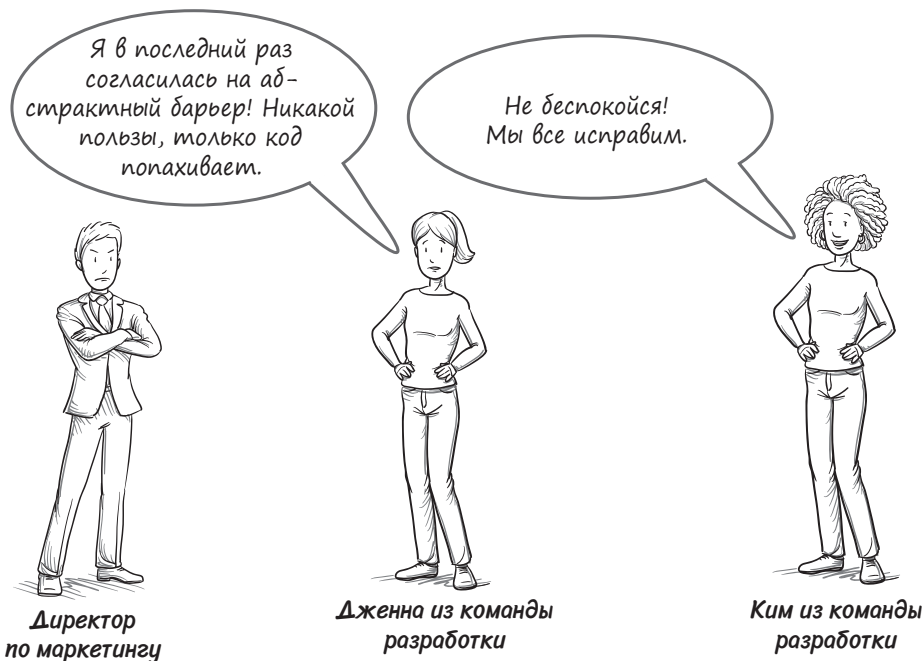
1. Очень похожие реализации.
2. Имя функции указывает на различия в реализации.

Различающиеся части имени функции становятся неявным аргументом.



Загляни в словарь

«Код с душком» — характеристика части кода, которая может быть симптомом более глубоких проблем.



Директор по маркетингу: Код может пахнуть?

Дженна: Да, в каком-то смысле. Выражение просто означает, что в коде на что-то стоит обратить внимание. Это не значит, что код плох, но это может быть признаком существующей проблемы.

Ким: Да! Этот код определенно пахнет. Только посмотри на все это дублирование.

Дженна: Да, код действительно очень похож. Но я не вижу, как избавиться от дубликатов. Нам нужны средства для назначения цены и количества единиц товара. Разве это не разные функции?

Ким: Дублирование означает, что эти функции *почти* одинаковы. Единственное различие — строка с именем поля ('price', 'quantity' или 'tax').

Дженна: Да, понимаю! И эта строка также присутствует в имени функции.

Ким: Точно. И это признак «кода с душком»: вместо передачи в аргументе имя поля становится частью имени функции.

Директор по маркетингу: И вы говорите, что его можно исправить?

Ким: Да. Я знаю прием рефакторинга, который позволит заменить все четыре функции одной. Для этого нужно сделать имя поля первоклассным значением.

Директор по маркетингу: Первоклассным? В смысле первого класса — как в поезде или самолете?

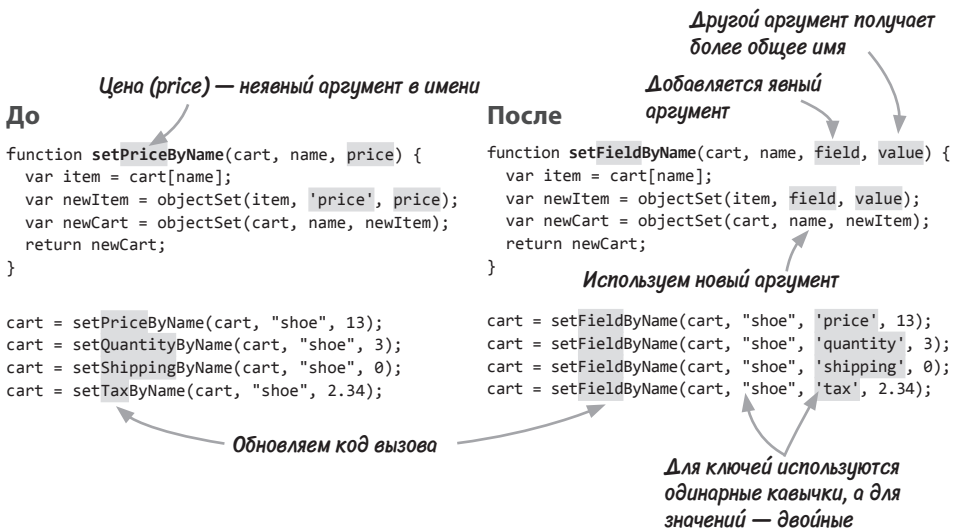
Ким: Хм. Да, пожалуй. Это просто означает, что имя поля становится аргументом. Мы определим его позднее.

Рефакторинг: явное выражение неявного аргумента

Метод рефакторинга, называемый *явным выражением неявного аргумента*, может применяться в любой ситуации, в которой неявный аргумент является частью функции. Основная идея заключается в том, чтобы превратить неявный аргумент в явный. Последовательность действий выглядит так:

1. Выявление неявного аргумента в имени функции.
2. Добавление явного аргумента.
3. Использование нового аргумента в теле вместо жестко фиксированного значения.
4. Обновление кода вызова.

Посмотрим, как провести рефакторинг функции `setPriceByName()`, которая может задать только цену, в функцию `setFieldByName()`, способную задать значение любого поля товара.



Применение этого рефакторинга к коду позволяет заменить четыре существующие функции одной обобщенной, — и кто знает, сколько еще функций нам не придется писать благодаря обобщенной функции `setFieldByName()`.

Что же здесь произошло? Имя поля было преобразовано в первоклассное значение. Ранее имя поля не раскрывалось перед клиентами API, кроме как в случае неявного раскрытия как части имен функций. Теперь оно стало значением (в данном случае стро-



Загляни в словарь

Первоклассное значение может использоваться так же, как и любые другие значения в языке.

кой), которое может передаваться в аргументе, но также может храниться в переменной или массиве. Именно это имеется в виду под *первоклассным значением*: для работы с ним может использоваться полный набор языковых средств. Преобразование к первоклассному статусу является темой этой главы.

На это можно возразить, что такое использование строк небезопасно. Эта тема будет рассмотрена на ближайших страницах, а пока просто продолжайте читать!



Директор по маркетингу: И нам не придется подавать заявку на изменение каждого поля?

Дженна: Вот именно. Теперь вы можете обратиться к любому нужному полю — просто укажите его имя в строковом формате и передайте его при вызове.

Директор по маркетингу: Как мы узнаем, как называется то или иное поле?

Ким: Очень просто. Мы сделаем имена частью спецификации API. Они будут частью абстрактного барьера.

Директор по маркетингу: Хмм... Идея мне начинает нравиться. Но тогда другой вопрос: а если вы добавите новое поле в спецификацию корзины или товара? Что тогда?

Дженна: Новая функция должна работать как с существующими, так и с новыми полями. Если мы добавляем новое поле, то должны будем сообщить вам

его имя, и тогда вы сможете пользоваться всеми функциями, которые вам известны.

Директор по маркетингу: Звучит неплохо. Кажется, такой подход сильно облегчит нашу задачу.

Ким: Так и должно быть! В старом варианте вы должны были знать набор функций (и подавать запросы на новые функции!), а теперь достаточно знать одну функцию и набор имен полей.

Старое представление API

```
function setPriceByName(cart, name, price)

function setQuantityByName(cart, name, quant)

function setShippingByName(cart, name, ship)

function setTaxByName(cart, name, tax)

...
```

Новое представление API

```
function setFieldByName(cart, name, field, value)
```

'price'
'quantity'
'shipping'
'tax'
...

Передается в этом аргументе

Определение того, что является и что не является первоклассным значением

В языке JavaScript полно непервоклассных сущностей. Впрочем, их полно в любом другом языке, каким бы вы ни пользовались

Подумайте, что можно сделать с числом в JavaScript. Его можно передать функции. Его можно вернуть из функции. Можно сохранить в переменной. Можно сделать элементом массива или значением в объекте. То же самое можно сделать со строками, логическими значениями, массивами и объектами. В JavaScript, как и во многих языках, все это можно делать и с функциями (как вы вскоре увидите). Эти значения называются первоклассными, потому что с ними можно делать все перечисленное.

Но в языке JavaScript много всего, что не относится к первоклассным значениям. Например, оператор + невозможно записать в виде значения, которое можно присвоить переменной. Также нельзя передать * функции. Арифметические операторы в JavaScript не являются первоклассными.

Постойте! Я не понял, что мы только что сделали.



Джордж из отдела тестирования

И это не все! Какое значение имеет ключевое слово `if`? Или ключевое слово `for`? У них нет значений в JavaScript. Именно это мы имеем в виду, говоря, что они *не являются первоклассными*. Это не является каким-то недостатком языка. Почти во всех языках есть сущности, не являющиеся первоклассными, поэтому важно узнавать их и знать, как сделать первоклассным то, что не является таковым по умолчанию.

На предыдущей странице мы сделали следующее:

*Невозможно сослаться на часть имени,
поэтому мы преобразуем ее в аргумент*

```
function setPriceByName(cart, name, price)
function setFieldName(cart, name, field, value)
```

В JavaScript невозможно сослаться на часть имени функции как на значение: она не является первоклассным значением. Из-за этого мы преобразовали ее в первоклассное значение, заменив ее строковым аргументом. В JavaScript строки могут использоваться для обращения к полям объектов. Именно это и было сделано! И это дало нам возможность решить проблему. Данный паттерн еще встретится вам в этой главе и других главах части II. Мы будем выявлять сущности, не являющиеся первоклассными, и преобразовывать их в первоклассные, и это откроет перед нами новые возможности для решения задач. Такие преобразования играют важную роль в функциональном программировании. Этот навык открывает путь ко многим нетривиальным паттернам функционального программирования.

Примеры непервоклассных сущностей в JavaScript

1. Арифметические операторы.
2. Циклы `for`.
3. Команды `if`.
4. Блоки `try/catch`.

Примеры операций, которые могут выполняться с первоклассными значениями

1. Присваивание переменной.
2. Передача в аргументе к функции.
3. Возвращение из функции.
4. Сохранение в массиве или объекте.

Не приведут ли строки с именами полей к новым ошибкам?

Джордж беспокоится о том, что со строками легко может возникнуть путаница. Что произойдет, если в одной из строк будет допущена опечатка?

Опасения вполне обоснованны, но мы учитываем такую возможность. Есть два варианта: проверки на стадии компиляции и проверки на стадии выполнения.

В проверках на стадии компиляции обычно задействована статическая система типов. В JavaScript статической системы типов нет, но ее можно добавить с помощью TypeScript. TypeScript позволяет проверить, что строки принадлежат к известному набору допустимых полей. Если при вводе будет допущена опе-

чатка, то система проверки типов сообщит об этом еще до запуска кода.

Во многих языках реализована статическая проверка типов, которую следует использовать для проверки правильности имен полей. Например, в JavaScript можно воспользоваться типом Enum, а в Haskell — дизъюнктивным объединением. В каждом языке используется своя система типов, поэтому вашей команде придется определить лучший подход к использованию такой системы.

Проверки времени выполнения не происходят на стадии компиляции. Они выполняются при каждом выполнении вашей функции. Также они проверяют, что передаваемые строки допустимы. Поскольку в JavaScript нет статической системы типов, мы можем выбрать этот вариант. Это будет выглядеть примерно так:

OMG! Передача имен полей в строках?? Да это будет настоящий рассадник ошибок.



Джордж из отдела тестирования

```
var validItemFields = ['price', 'quantity', 'shipping', 'tax'];

function setFieldByName(cart, name, field, value) {
  if(!validItemFields.includes(field))
    throw "Not a valid item field: " +
      "'" + field + "'.";
  var item = cart[name];
  var newItem = objectSet(item, field, value);
  var newCart = objectSet(cart, name, newItem);
  return newCart;
}
```

Здесь можно указывать любые действительные поля

Функция `objectSet()` была определена в главе 6

Проверки времени выполнения легко реализуются с первоклассными полями

```
function objectSet(object, key, value) {
  var copy = Object.assign({}, object);
  copy[key] = value;
  return copy;
}
```



Пицца для ума

JavaScript не проверяет имена полей или имена функций. Насколько обоснованны опасения Джорджа, касающиеся использования строковых имен полей вместо функций доступа? Использовать строки действительно плохо?

Усложнят ли первоклассные поля изменения API?

Дженна беспокоится о том, что использование первоклассных имен полей раскрывает подробности реализации наших сущностей. Сущности «корзина» и «товар» — объекты, у которых заданы некоторые поля, но они определяются под абстрактным барьером. Когда мы обращаемся к людям, работающим над абстрактным барьером, с требованием переименовывать поля, разве мы тем самым не нарушаем абстрактный барьер? Разве мы не раскрываем внутреннюю реализацию? Включение имен полей в спецификацию API фактически гарантирует, что они останутся там навсегда.

Идея мне нравится, потому что она решает проблемы маркетинга. Но разве легко будет изменять API?



Дженна из команды разработки

Верно, такая гарантия есть, но при этом мы не раскрываем реализацию. Изменяя имена во внутренней реализации, мы можем продолжить поддерживать гарантированные имена. Внутренние имена можно заменять.

Допустим, вам по какой-то причине потребовалось заменить 'quantity' на 'number'. Нарушать работу всего существующего кода не хочется, поэтому функция должна по-прежнему получать 'quantity'. Такая замена производится легко:

```
var validItemFields = ['price', 'quantity', 'shipping', 'tax', 'number'];
var translations = { 'quantity': 'number' };

function setFieldByName(cart, name, field, value) {
  if(!validItemFields.includes(field))
    throw "Not a valid item field: '" + field + "'.";
  if(translations.hasOwnProperty(field))
    field = translations[field];
  var item = cart[name];
  var newItem = objectSet(item, field, value);
  var newCart = objectSet(cart, name, newItem);
  return newCart;
}
```

Старое имя поля просто заменяется новым

Это возможно, потому что мы сделали поля первоклассными. Первоклассное имя поля означает, что его можно поместить в массив, сохранить в объекте и вообще применять в программной логике всю мощь языка.



Пища для ума

Все имена полей, передаваемые в настоящее время в строковом виде, ранее раскрывались в именах функций. Если имена полей изменятся, менять имена функций не обязательно. Можно ли использовать строки по-другому?



Ваш ход

Совсем простая задача: кто-то в команде написал следующие функции, в именах которых присутствуют неявные аргументы. Сделайте рефакторинг *явного выражения аргумента*, чтобы избавиться от дублирования кода.

```
function multiplyByFour(x) {
  return x * 4;
}
```

```
function multiplyBy12(x) {
  return x * 12;
}
```

```
function multiplyBySix(x) {
  return x * 6;
}
```

```
function multiplyByPi(x) {
  return x * 3.14159;
}
```

Запишите здесь свой ответ

Последовательность действий

1. Выявление неявного аргумента в имени функции.
2. Добавление явного аргумента.
3. Использование нового аргумента в теле вместо жестко фиксированного значения.
4. Обновление кода вызова.

В данном упражнении этот пункт не выполняется, не волнуйтесь



Ответ

```
function multiply(x, y) {
  return x * y;
}
```



Ваш ход

Задача от разработчиков UI: в представлении корзины имеются кнопки для инкрементирования (увеличения на 1) количества и размера.

```
function incrementQuantityByName(cart, name) {
  var item = cart[name];
  var quantity = item['quantity'];
  var newQuantity = quantity + 1;
  var newItem = objectSet(item, 'quantity', newQuantity);
  var newCart = objectSet(cart, name, newItem);
  return newCart;
}
```

Неявные аргументы

```
function incrementSizeByName(cart, name) {
  var item = cart[name];
  var size = item['size'];
  var newSize = size + 1;
  var newItem = objectSet(item, 'size', newSize);
  var newCart = objectSet(cart, name, newItem);
  return newCart;
}
```

Имена полей ('quantity' и 'size') являются частью имен функций. Примените рефакторинг явного выражения неявного аргумента, чтобы избавиться от дублирования.

Запишите здесь свой ответ

Последовательность действий

1. Выявление неявного аргумента в имени функции.
2. Добавление явного аргумента.
3. Использование нового аргумента в теле вместо жестко фиксированного значения.
4. Обновление кода вызова.

В данном упражнении этот пункт не выполняется, не волнуйтесь

**Ответ**

```
function incrementFieldByName(cart, name, field) {  
  var item = cart[name];  
  var value = item[field];  
  var newValue = value + 1;  
  var newItem = objectSet(item, field, newValue);  
  var newCart = objectSet(cart, name, newItem);  
  return newCart;  
}
```



Ваш ход

Команда разработки начинает беспокоиться, что люди, использующие API, попытаются изменить поля, к которым эта операция неприменима, например цену или название товара! Добавьте проверки времени выполнения. Выдавайте ошибку, если имя поля отлично от 'size' или 'quantity'. Проверки времени выполнения принято размещать в начале функции.

```
function incrementFieldByName(cart, name, field) {
```

← Запишите здесь свой ответ

```
    var item = cart[name];
    var value = item[field];
    var newValue = value + 1;
    var newItem = objectSet(item, field, newValue);
    var newCart = objectSet(cart, name, newItem);
    return newCart;
}
```



Ответ

```
function incrementFieldByName(cart, name, field) {
    if(field !== 'size' && field !== 'quantity')
        throw "This item field cannot be incremented: " +
            "\"" + field + "\".";
    var item = cart[name];
    var value = item[field];
    var newValue = value + 1;
    var newItem = objectSet(item, field, newValue);
    var newCart = objectSet(cart, name, newItem);
    return newCart;
}
```

Мы будем использовать множество объектов и массивов

Для представления таких сущностей, как товары в корзине, удобно использовать хеш-карты, потому что они позволяют легко представлять свойства и значения, а объекты JavaScript пре-

доставляют удобные средства для работы с ними. Конечно, в своем языке вы будете использовать что-то другое. Если вы работаете на Haskell, алгебраический тип данных может подойти лучше всего. Если вы работаете на Java, то при необходимости использования первоклассных ключей стоит выбрать хеш-карту. В других ОО-языках (например, в Ruby) предусмотрены простые возможности передачи первоклассных методов доступа. Каждый язык делает это по-своему, и вы должны руководствоваться здравым смыслом. Но, скорее всего, в JavaScript окажется, что вы пользуетесь объектами намного чаще, чем прежде.

Здесь важно то, что мы стараемся интерпретировать данные как данные, вместо того чтобы упаковывать их в специализированный интерфейс. Такие интерфейсы обеспечивают одну интерпретацию данных, но при этом блокируют другие. Они определяют крайне специализированные сценарии использования. При этом сами сущности (корзины и товары) имеют общую природу. Они находятся относительно низко на графе вызовов. Они слишком малы, чтобы иметь специализированный API. В таких ситуациях логично применять структуры общего назначения, такие как объекты и массивы.

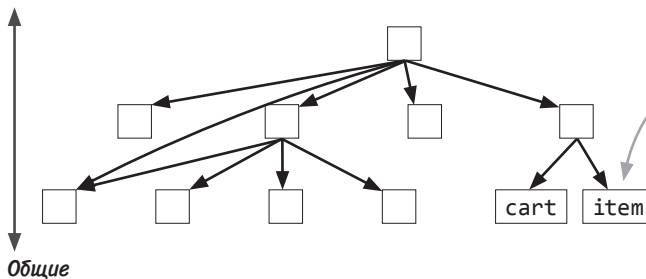
Похоже, с первоклассными именами полей в нашем коде часто используются объекты JavaScript.



Ким из команды разработки

Обобщенные сущности, такие как корзина или товар, должны храниться в обобщенных структурах (объектах и массивах).

Конкретные



Наши сущности являются общими и должны использоваться повторно, поэтому они должны быть в форматах общего назначения, таких как объекты и массивы

Важное преимущество данных — это возможность их интерпретации разными способами. Ограничение такой возможности за счет определения ограниченного API сокращает их потенциал. Да, не всегда получится предсказать, какие интер-

претации могут появиться в будущем, но они принесут пользу тогда, когда в них возникнет необходимость.

Это важный принцип стиля, который называется *программированием, ориентированным на данные*; этот стиль будет использоваться в коде книги. При этом ничто не мешает добавить необязательный интерфейс с абстрактными барьерами, как было показано в главе 9.



Загляни в словарь

Программирование, ориентированное на данные, — это стиль программирования, использующий обобщенные структуры данных для представления фактов о событиях и сущностях.

Статическая и динамическая типизация

В области программирования идет давний спор: проверять типы нужно во время компиляции или во время выполнения? Языки, проверяющие типы во время компиляции, называются языками *со статической типизацией*. А в языках, которые не проверяют типы во время компиляции, реализована проверка типов во время выполнения; такие языки называются языками *с динамической типизацией*. Споры не утихают даже спустя десятилетия. И нигде они не идут так ожесточенно, как в сообществе функционального программирования.

Истина в том, что единственно правильного ответа не существует. У обеих сторон есть веские доводы. Книга не может решить этот давний спор. Тем не менее важно понимать, что в споре две стороны и явного победителя еще нет. Даже после тщательного анализа неочевидно, что один способ лучше другого для производства качественных программных продуктов. Например, некоторые исследования показывают, что хороший ночной сон разработчика сильнее влияет на качество кода, чем различия между статической и динамической типизацией (<https://increment.com/teams/the-epistemology-of-software-quality/>).

Нужно обсудить еще один вопрос. В этой книге в коде примеров используется JavaScript, который является языком с динамической типизацией. Тем не менее это не стоит воспринимать как аргумент в пользу динамической типизации или в пользу JavaScript. Самая важная причина для использования JavaScript в книге заключается в том, что это популярный язык, понятный многим людям благодаря знакомому синтаксису. И я хотел использовать язык без статической системы типов, потому что системы типов намного проще изучать одновременно с изучением парадигмы функционального программирования.

Впрочем, есть и другие проблемы. В обсуждениях часто упускают из виду, что системы типов не одинаковы. Бывают хорошие статические системы типов, бывают и плохие. Аналогичным образом бывают хорошие динамические системы типов, бывают и не очень. Попытки сравнивать их группами не имеют смысла. Нельзя сказать, что одна группа однозначно лучше другой.

Что же делать? Выберите тот язык, с которым ваша команда чувствует себя уверенно. Затем поспите и перестаньте беспокоиться об этом.

Проблемы с передачей строк

Все верно! Строки могут содержать ошибки. Но прежде, чем отказываться от этой идеи, ознакомьтесь с некоторыми аргументами.

Многие языки программирования с динамической типизацией передают эквиваленты строк, представляющих поля структур данных. Самые известные примеры — JavaScript, Ruby, Clojure и Python.

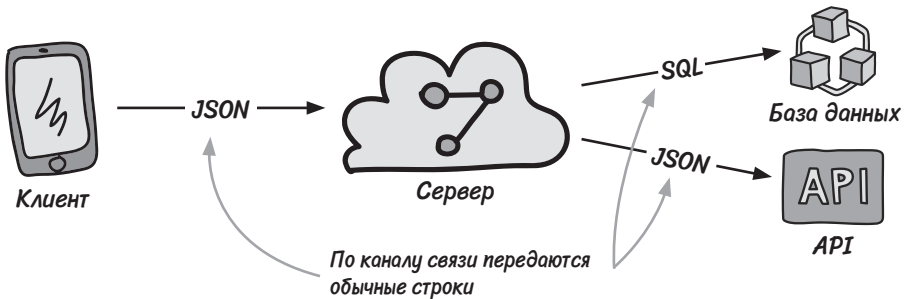
Да, в них часто встречаются ошибки из-за неправильных строк. Такое бывает. Однако на базе этих языков построены многие предприятия, а от правильного функционирования этих систем зависят миллиарды, если не триллионы долларов. В общем, со строками жить можно.

Но проблема строк заходит намного глубже, чем мы обычно представляем. Наши браузеры отправляют серверу разметку JSON. Разметка JSON представляет собой обычные строки. Сервер получает и разбирает эти строки. Сервер рассчитывает на то, что сообщение представляет собой правильно сформированную разметку JSON. Если разметка правильно сформирована, предполагается, что структура данных корректна.

*Секундочку!
Вы говорите, что мы
будем передавать обычные
строки? В них может быть
все что угодно. В том
числе и опечатки!*



Сара из команды разработки



То же самое можно сказать о веб-сервере, взаимодействующем с базой данных. Веб-сервер должен преобразовать команды к базе данных в строку, которая передается по каналу связи. База данных разбирает и интерпретирует полученные данные. В этом случае по каналу связи тоже передается строковая информация. И даже если в формат данных встроены типы, они остаются простыми байтами, и это открывает массу возможностей для искажения и злонамеренного вмешательства.

API должен проверять данные, поступающие на сервер от клиента во время выполнения, даже если программа написана на статическом языке. Все, что может сделать статическая система типов, — гарантировать, что код в одном месте системы соответствует предположениям, закодированным в виде типов.

Означает ли это, что статические типы не стоит использовать? Нет. Означает ли это, что их нужно использовать? Нет. Просто вы должны знать, что динамическая типизация не является источником проблемы, а статическая типизация не решит ее. По сути, мы видим обратную сторону данных: они требуют интерпретации.

Первокласные функции могут заменить любой синтаксис

Ранее уже упоминалось о том, что в JavaScript многие сущности не являются первокласными. Оператор `+` невозможно присвоить переменной. Однако вы можете создать эквивалент оператора `+` в виде функции.

```
function plus(a, b) {  
  return a + b;  
}
```

Функции являются первокласными значениями в JavaScript, поэтому фактически оператор `+` превращается в первокласное значение. Такой код выглядит странно, потому что мы могли просто написать `+`, но вскоре вы увидите потенциальные применения для первокласной функции сложения. Помните: преобразование сущностей в первокласную форму может расширить ваши возможности. Обретенная мощь может использоваться для решения различных задач.



Ваш ход

Напишите первоклассные версии других арифметических операторов: *, - и / (иначе говоря, «заверните» их в функции).

*

-

/



Ответ

```
function times(a, b) {  
  return a * b;  
}
```

```
function minus(a, b) {  
  return a - b;  
}
```

```
function dividedBy(a, b) {  
  return a / b;  
}
```



Директор по маркетингу: Сможете? Потому что мы все-таки не программисты. Мы делаем столько ошибок в циклах `for`, что это даже не смешно.

Дженна: Но как мы можем избавить вас от циклов `for`? Погодите, я угадаю: мы сделаем цикл `for` первоклассным.

Ким: И да и нет. Да, формально мы сделаем его первоклассным. Но нет, от делу маркетинга это не поможет. Мы поможем им, создав функцию, которая получает *первоклассную функцию* в аргументе. Иначе говоря, мы напишем *функцию высшего порядка*.

Директор по маркетингу: Вот это я не понял. Первоклассную? Высшего порядка?

Ким: Это связанные термины. «Первоклассная» означает, что функция может передаваться в аргументе. «Высшего порядка» — что функция получает другую функцию в аргументе. Функции высшего порядка не могут существовать без первоклассных функций.

Директор по маркетингу: Хорошо, примерно понятно. Я просто хочу, чтобы мои люди не мучались с синтаксисом циклов `for`. Вы можете сделать так, чтобы мне не приходилось снова писать циклы `for`?

Ким: Да! Я знаю метод рефакторинга, который фактически дает такую возможность. Он называется *заменой тела функции обратным вызовом*.

Дженна: Вот это да! Мне не терпится это увидеть!



Загляни в словарь

Функции высшего порядка получают другие функции в аргументах или возвращают их как возвращаемые значения.

Пример цикла for: еда и уборка

Рассмотрим два типичных цикла `for`, в которых перебираются элементы массива. В первом цикле мы готовим еду и обедаем. Во втором цикле приходится мыть грязную посуду.

Приготовление и еда

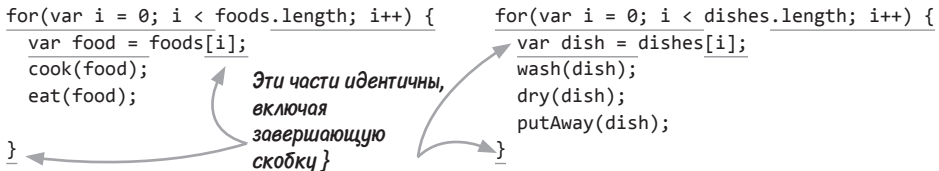
```
for(var i = 0; i < foods.length; i++) {
  var food = foods[i];
  cook(food);
  eat(food);
}
```

Мытье посуды

```
for(var i = 0; i < dishes.length; i++) {
  var dish = dishes[i];
  wash(dish);
  dry(dish);
  putAway(dish);
}
```

Циклы `for` различаются по своему предназначению, но код очень похож. Это нельзя назвать «дублированием», пока код не будет в точности одинаковым. Давайте систематично сделаем эти два цикла настолько похожими, насколько это возможно. Если сделать их совсем идентичными, один из них можно будет удалить. Мы не будем ничего пропускать, и если вам покажется, что материал излагается слишком медленно, бегло просмотрите объяснение.

Начнем с выделения всех совпадающих частей:

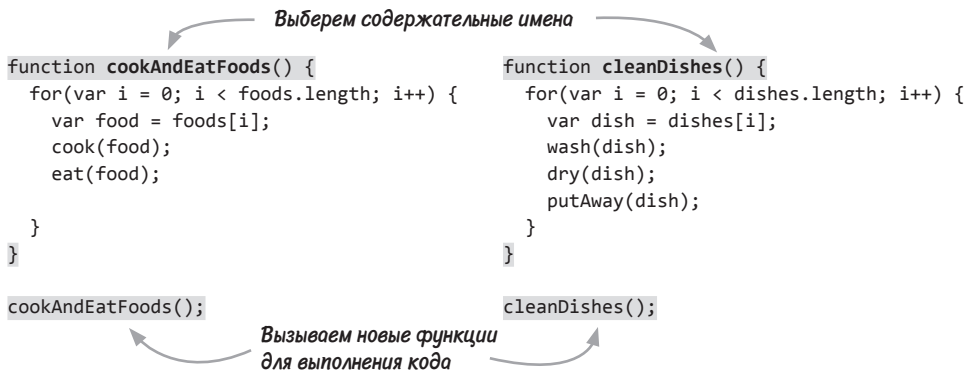


```
for(var i = 0; i < foods.length; i++) {
  var food = foods[i];
  cook(food);
  eat(food);
}

for(var i = 0; i < dishes.length; i++) {
  var dish = dishes[i];
  wash(dish);
  dry(dish);
  putAway(dish);
}
```

Эти части идентичны, включая завершающую скобку

Наша окончательная цель — убрать все несовпадения (части без подчеркивания). Для начала упакуем их в функции: это упростит дальнейшую работу.



```
function cookAndEatFoods() {
  for(var i = 0; i < foods.length; i++) {
    var food = foods[i];
    cook(food);
    eat(food);
  }
}

function cleanDishes() {
  for(var i = 0; i < dishes.length; i++) {
    var dish = dishes[i];
    wash(dish);
    dry(dish);
    putAway(dish);
  }
}

cookAndEatFoods();
cleanDishes();
```

Выберем содержательные имена

Вызываем новые функции для выполнения кода

Места не осталось, продолжим на следующей странице.

На предыдущей странице циклы `for` были упакованы в функции. Имена функций были выбраны в соответствии с их назначением. Вот как это выглядело:

```
function cookAndEatFoods() {
  for(var i = 0; i < foods.length; i++) {
    var food = foods[i];
    cook(food);
    eat(food);
  }
}
```

Эти переменные имеют одинаковое предназначение, но разные имена

```
function cleanDishes() {
  for(var i = 0; i < dishes.length; i++) {
    var dish = dishes[i];
    wash(dish);
    dry(dish);
    putAway(dish);
  }
}
```

```
cookAndEatFoods();
```

```
cleanDishes();
```

Признаки неявного аргумента в имени функции

1. Похожие реализации.
2. Упоминание различий в имени функции.

Бросаются в глаза слишком конкретные имена локальной переменной. В одной функции она называется `food`, в другой — `dish`. Имена выбираются произвольно, поэтому мы используем более универсальное имя:

```
function cookAndEatFoods() {
  for(var i = 0; i < foods.length; i++) {
    var item = foods[i];
    cook(item);
    eat(item);
  }
}
```

Обе переменные называются item

```
function cleanDishes() {
  for(var i = 0; i < dishes.length; i++) {
    var item = dishes[i];
    wash(item);
    dry(item);
    putAway(item);
  }
}
```

```
cookAndEatFoods();
```

```
cleanDishes();
```

Наверное, вы уже увидели здесь то, что узнали ранее: *неявный аргумент в имени функции*. Обратите внимание на присутствие `foods` в имени функции и в имени массива; с `dishes` ситуация аналогичная. Применим рефакторинг явного выражения неявного аргумента:

Последовательность действий в рефакторинге «явное выражение неявного аргумента»

1. Выявление неявного аргумента в имени функции.
2. Добавление явного аргумента.
3. Использование нового аргумента в теле вместо жестко фиксированного значения.
4. Обновление кода вызова.

```
function cookAndEatArray(array) {
  for(var i = 0; i < array.length; i++) {
    var item = array[i];
    cook(item);
    eat(item);
  }
}
```

Переименование указывает на общий характер переменных

Добавляется явный аргумент array

```
function cleanArray(array) {
  for(var i = 0; i < array.length; i++) {
    var item = array[i];
    wash(item);
    dry(item);
    putAway(item);
  }
}
```

```
cookAndEatArray(foods);
```

```
cleanArray(dishes);
```

Передаем массивы

Работа подходит к концу, но у нас снова кончилось место на странице. Перейдите на следующую страницу.

На предыдущей странице мы просто выразили неявные аргументы в именах функций. Теперь оба массива называются **array**. Вот что получилось:

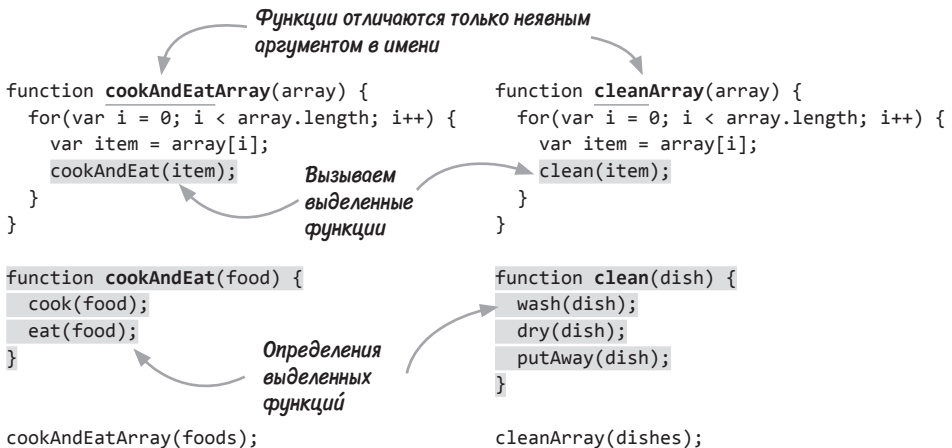
```
function cookAndEatArray(array) {
  for(var i = 0; i < array.length; i++) {
    var item = array[i];
    cook(item);
    eat(item);
  }
}
```

```
cookAndEatArray(foods);
```

```
function cleanArray(array) {
  for(var i = 0; i < array.length; i++) {
    var item = array[i];
    wash(item);
    dry(item);
    putAway(item);
  }
}
```

```
cleanArray(dishes);
```

Остается последнее — тело цикла **for**. Это единственное различие. Так как тело состоит из нескольких строк, выделим их в виде функций:



Теперь, когда тело стало одной именованной функцией, мы снова замечаем знакомый признак «кода с душком»: *неявный аргумент в имени функции!* `cookAndEatArray()` вызывает `cookAndEat()`, а `cleanArray()` вызывает `clean()`. Применим рефакторинг на следующей странице.

Признаки неявного аргумента в имени функции

1. Похожие реализации.
2. Упоминание различий в имени функции.

На предыдущей странице мы обнаружили признак «кода с душком»: неявный аргумент в имени функции. У нас были две функции с похожими реализациями, а различия между реализациями были отражены в именах функций:

Признаки неявного аргумента в имени функции

1. Похожие реализации.
2. Упоминание различий в имени функции.

Различия отражены в именах функций

```
function cookAndEatArray(array) {
  for(var i = 0; i < array.length; i++) {
    var item = array[i];
    cookAndEat(item);
  }
}

function cleanArray(array) {
  for(var i = 0; i < array.length; i++) {
    var item = array[i];
    clean(item);
  }
}

function cookAndEat(food) {
  cook(food);
  eat(food);
}

function clean(dish) {
  wash(dish);
  dry(dish);
  putAway(dish);
}

cookAndEatArray(foods);

cleanArray(dishes);
```

Похожие функции

Применим рефакторинг!

Присваиваем обобщенное имя *Явное выражение аргумента* *Явное выражение аргумента*

```
function operateOnArray(array, f) {
  for(var i = 0; i < array.length; i++) {
    var item = array[i];
    f(item);
  }
}

function cookAndEat(food) {
  cook(food);
  eat(food);
}

operateOnArray(foods, cookAndEat);

function clean(dish) {
  wash(dish);
  dry(dish);
  putAway(dish);
}

operateOnArray(dishes, clean);
```

Использование нового аргумента в теле *Аргумент добавляется в код вызова* *Аргумент добавляется в код вызова*

Аргумент добавляется в код вызова

Два фрагмента кода выглядят идентично. Различающиеся части были выделены в аргументы: массив и используемая функция.

Последовательность действий

1. Выявление неявного аргумента в имени функции.
2. Добавление явного аргумента.
3. Использование нового аргумента в теле вместо жестко фиксированного значения.
4. Обновление кода вызова.

На предыдущей странице мы завершили преобразование двух функций к идентичному виду. В конце были приведены следующие реализации:

<pre>function operateOnArray(array, f) { for(var i = 0; i < array.length; i++) { var item = array[i]; f(item); } }</pre>	 <i>Идентичные функции</i>	<pre>function operateOnArray(array, f) { for(var i = 0; i < array.length; i++) { var item = array[i]; f(item); } }</pre>
<pre>function cookAndEat(food) { cook(food); eat(food); }</pre>		<pre>function clean(dish) { wash(dish); dry(dish); putAway(dish); }</pre>
<pre>operateOnArray(foods, cookAndEat);</pre>		<pre>operateOnArray(dishes, clean);</pre>

Различия были исключены, поэтому одно из определений можно удалить. Кроме того, в JavaScript эта функция традиционно называется `forEach()`, поэтому мы переименуем ее:

<pre>function forEach(array, f) { for(var i = 0; i < array.length; i++) { var item = array[i]; f(item); } }</pre>	 <i>Мы видим, что <code>forEach()</code> получает функцию в аргументе. Это означает, что <code>forEach()</code> является функцией высшего порядка</i>	
<pre>function cookAndEat(food) { cook(food); eat(food); }</pre>		<pre>function clean(dish) { wash(dish); dry(dish); putAway(dish); }</pre>
<pre>forEach(foods, cookAndEat);</pre>		<pre>forEach(dishes, clean);</pre>

`forEach()` получает в аргументах массив и функцию. Так как она получает в аргументе функцию, она является *функцией высшего порядка*.

Готово! Рефакторинг состоял из нескольких этапов, поэтому на следующей странице приводится код до и после переработки.



Загляни в словарь

Функции высшего порядка получают другие функции в аргументах или возвращают их в возвращаемом значении.

На предыдущих страницах был проведен рефакторинг, состоящий из нескольких этапов. К концу легко забыть, с чего все начиналось: можно упустить из виду «общую картину». Сравним исходные версии с новыми. Рассмотрим различия с *анонимной функцией*:

Оригинал

```
for(var i = 0; i < foods.length; i++) {
  var food = foods[i];
  cook(food);
  eat(food);
}

for(var i = 0; i < dishes.length; i++) {
  var dish = dishes[i];
  wash(dish);
  dry(dish);
  putAway(dish);
}
```

Только посмотрите, сколько всего ненужного

Использование `forEach()`

```
forEach(foods, function(food) {
  cook(food);
  eat(food);
});

forEach(dishes, function(dish) {
  wash(dish);
  dry(dish);
  putAway(dish);
});
```

Анонимные функции

`forEach()` — последний цикл `for` для перебора массива, который вам придется написать. В нем инкапсулирован паттерн, который вы реализовали так много раз. А теперь для его использования достаточно вызвать `forEach()`.

`forEach()` является функцией высшего порядка. На это указывает то, что эта функция получает функцию в аргументе. Мощь функций высшего порядка проявляется в возможности абстрагирования кода. Ранее вам приходилось каждый раз писать код цикла `for`, потому что изменяющаяся часть находилась в теле цикла `for`. Но преобразовав его в функцию высшего порядка, мы можем передать в аргументе код, который различается в циклах `for`.

Функция `forEach()` играет важную роль для обучения. Вся глава 12 будет посвящена ей и другим похожим функциям. А сейчас речь идет о процессе создания функций высшего порядка. Одним из способов достижения этой цели является серия выполненных этапов рефакторинга. Процедура выглядит так:

1. Упаковка кода в функции.
2. Присваивание более общих имен функциям.
3. Явное выражение неявных аргументов.
4. Выделение функций.
5. Явное выражение неявных аргументов.

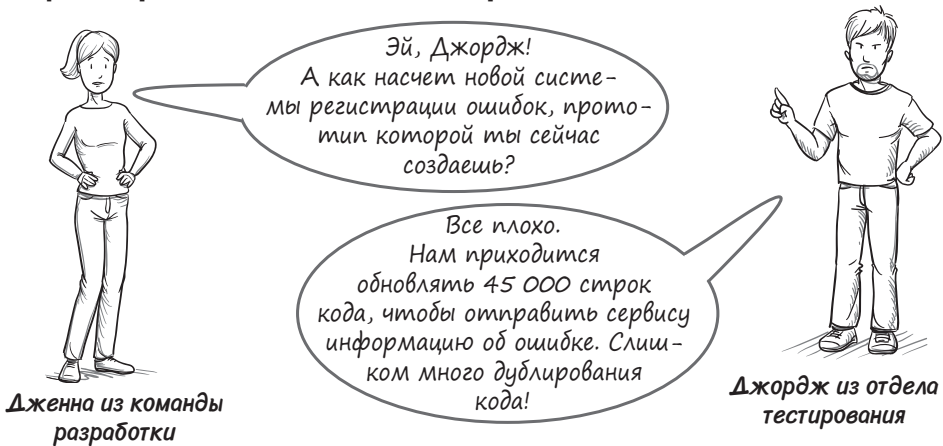
Последовательность получается длинной, но нам хотелось бы сделать все за один этап. По этой причине существует метод рефакторинга, называемый *заменой тела обратным вызовом*. Он позволяет быстрее и короче сделать то, что мы только что сделали. Мы применим его к новой системе регистрации ошибок, прототип которой сейчас строит Джордж.



Загляни в словарь

Анонимные функции — функции, которым не присвоено имя. Они могут определяться как *встроенные*, то есть непосредственно в месте их использования.

Рефакторинг: замена тела обратным вызовом



Дженна: Звучит пугающе.

Джордж: Ага. Нам придется упаковать тысячи строк в блоки `try/catch`, чтобы перехватывать и отправлять ошибки **Snap Errors®** — сервису регистрации ошибок. А самое худшее — дублирование кода: повсюду команды `try/catch`! Вот как будет выглядеть код с `try/catch`:

```
try {
    saveUserData(user);
} catch (error) {
    logToSnapErrors(error);
}
```

Мы пытались упаковать код в функцию, но не придумали, как это сделать. `catch` от `try` выделить невозможно. Это будет нарушением синтаксиса, поэтому они не могут находиться в разных функциях. Мы в тупике. Похоже, избавиться от дублирования невозможно.

Дженна: Надо же! Какое совпадение! Я как раз изучала метод рефакторинга, предназначенный именно для этого. Он называется заменой тела обратным вызовом.

Джордж: Не уверен, что это сработает, но все равно хочу попробовать.

Дженна: Буду рада помочь, но я только учусь. Для этого рефакторинга необходимо определить код предшествующий и завершающий, который останется постоянным, а также определить код тела, который изменяется. После этого тело заменяется функцией.

Джордж: Эй! Помедленнее! Все еще непонятно.

Дженна: Мы хотим передать в аргументе функцию. Она представляет другой код, который требуется выполнить.

Джордж: Да, уже понятнее, но мне хотелось бы увидеть код.

Snap Errors®

Человеку свойственно ошибаться, но Snap не ошибается.

Из документации Snap Errors API:

`logToSnapErrors(error)` — отправляет ошибку сервису Snap Errors®. Ошибка должна инициализироваться и перехватываться в вашем коде.

Джордж провел несколько последних недель за прототипизацией заключения важного кода в блоки `try/catch`. После всех уточнений результат выглядит так:

```
try {
  saveUserData(user);
} catch (error) {
  logToSnapErrors(error);
}
```

Одна и та же секция catch

```
try {
  fetchProduct(productId);
} catch (error) {
  logToSnapErrors(error);
}
```

Функция API Snap Errors

И насколько можно судить, ему предстоит писать похожие команды `try/catch` с тем же содержимым `catch` в течение следующего квартала. Однако Дженна знает, что замена тела обратным вызовом — это хорошее решение для предотвращения дублирования.

Весь фокус в том, чтобы выявить паттерн «до-тело-после». Судя по коду Джорджа, это выглядит так:

```
try {
  saveUserData(user);
} catch (error) {
  logToSnapErrors(error);
}
```

До

Тело

После

```
try {
  fetchProduct(productId);
} catch (error) {
  logToSnapErrors(error);
}
```

Разделы до и после не различаются от экземпляра к экземпляру. Между ними можно добавлять другой код

Предшествующая и завершающая части не изменяются в зависимости от экземпляра. Оба фрагмента кода содержат абсолютно одинаковый код предшествующей и завершающей части. Однако при этом между ними располагаются разные части (тело). Необходимо иметь возможность изменять его между предшествующей и завершающей частью. Это делается так:

1. Определите части: предшествующую, тело и завершающую.
2. Выделите весь код в функцию.
3. Выделите тело в функцию, которая передается в аргументе этой функции.

Шаг 1 уже выполнен, а вторым и третьим мы займемся на следующей странице.



Загляни в словарь

В мире JavaScript функции, передаваемые в аргументах, часто называются *обратными вызовами* (callbacks), но этот термин также часто встречается за пределами сообщества JavaScript. Предполагается, что функция, которой вы передаете обратный вызов, вызовет переданную функцию. В других сообществах также используется термин «обработчик». Опытные функциональные программисты настолько привыкли передавать функции в аргументах, что часто им не нужен для этого специальный термин.

На предыдущей странице мы уже определили предшествующую часть, тело функции и завершающую часть. Следующим шагом должно стать выделение кода в функцию. Назовем эту функцию `withLogging()`.

Оригинал

```
try {
  saveUserData(user);
} catch (error) {
  logToSnapErrors(error);
}
```

После выделения функции

```
function withLogging() {
  try {
    saveUserData(user);
  } catch (error) {
    logToSnapErrors(error);
  }
}

withLogging();
```

Вызываем функцию `withLogging()` после ее определения

Присваивая имя выделенной функции, мы получаем возможность обратиться к ней. Следующим шагом должно стать выделение тела (отличающейся части) в аргумент.

Текущая версия

```
function withLogging() {
  try {
    saveUserData(user);
  } catch (error) {
    logToSnapErrors(error);
  }
}

withLogging();
```

Эту часть можно выделить в обратный вызов

После выделения обратного вызова

```
function withLogging(f) {
  try {
    f();
  } catch (error) {
    logToSnapErrors(error);
  }
}

withLogging(function() {
  saveUserData(user);
});
```

f от слова *function*, то есть «функция»

Функция вызывается вместо старого тела

Теперь тело должно передаваться при вызове

Однострочная анонимная функция



Секундочку! Что это за синтаксис? Почему мы упаковали код в функцию?

Последовательность действий по замене тела обратным вызовом

1. Определение частей: предшествующей, тела и завершающей.
2. Выделение функции.
3. Выделение обратного вызова.

Сразу два хороших вопроса! Мы ответим на них на ближайших страницах.

Что это за синтаксис

Вот строка кода, которую мы только что написали и которая так смутила Джорджа:

```
withLogging(function() { saveUserData(user); });
```

Вскоре вы увидите, что это вполне нормальный способ определения и передачи функции. Существуют три способа определения функций, которые перечислены ниже.



Загляни в словарь

Встроенная функция определяется в месте ее использования. Например, функция может определяться в списке аргументов.

1. Глобальное определение

Мы можем определить и назвать функцию на глобальном уровне. Этот способ определения типичен для большинства функций. Он позволяет обратиться к функции по имени практически в любой точке программы.

```
function saveCurrentUserData() {  
    saveUserData(user);  
}
```

Функция определяется глобально

```
withLogging(saveCurrentUserData);
```

Функция передается по имени

2. Локальное определение

Мы можем определить и назвать функцию в локальной области видимости. У такой функции есть имя, но к ней нельзя будет обратиться по этому имени за пределами области видимости. Данная возможность будет полезна, если вам нужно обращаться к другим значениям в локальной области видимости, но вы хотите работать с функцией по имени.

```
function someFunction() {  
    var saveCurrentUserData = function() {  
        saveUserData(user);  
    };  
    withLogging(saveCurrentUserData);  
}
```

Функции присваивается имя только в локальной области видимости

Функция передается по имени



Загляни в словарь

Анонимная функция не имеет имени. Обычно анонимные функции встречаются при определении функций во встроенном виде.

3. Встроенное определение

Функция также может определяться непосредственно в месте ее использования. Иначе говоря, функция не присваивается переменной, поэтому у нее нет имени. Такие функции называются *анонимными*.

Обычно анонимными становятся короткие функции, которые имеют смысл в определенном контексте и используются только один раз.

```
withLogging(function() { saveUserData(user); });
```

У этой функции нет имени

Функция определяется в месте ее использования

Именно этот способ был использован на предыдущей странице. Мы записали анонимную функцию во встроенном виде. «Анонимность» означает, что у функции нет имени (потому что оно ей не нужно). «Встроенная» означает, что функция определяется непосредственно в месте ее использования.

Почему мы упаковали код в функцию

Перед вами код, который смутил Джорджа. Он не понимает, почему вызов `saveUserData(user)` нужно заключить в функцию. Разобьем его на части и посмотрим, как это помогает отложить выполнение кода:

```
function withLogging(f) {
  try {
    f();
  } catch (error) {
    logToSnapErrors(error);
  }
}
```

Почему эта строка заключена в определение функции?

```
withLogging(function() { saveUserData(user); });
```

У Джорджа имеется небольшой блок кода — `saveUserData(user)`, — который должен выполняться в определенном контексте, а именно внутри блока `try`. Строку кода можно было бы заключить в `try/catch`. А можно вместо этого заключить в определение функции. В таком случае код не будет выполняться немедленно. Он будет «сохранен на будущее», словно рыба, замороженная во льду. Функция предоставляет возможность отложить выполнение кода.

```
function() {
  saveUserData(user);
}
```

Эта строка не будет выполнена, пока не будет вызвана функция-обертка

Так как функции являются первоклассными значениями в JavaScript, это открывает ряд возможностей. Функции можно присвоить имя, сохранив ее в переменной. Функцию можно сохранить в коллекции (массиве или объекте),

а можно передать другой функции. Короче, можно делать все, что можно делать с первостепенными значениями.

Присваивание имени

```
var f = function() {
  saveUserData(user);
};
```

Сохранение в коллекции

```
array.push(function() {
  saveUserData(user);
});
```

Передача

```
withLogging(function() {
  saveUserData(user);
});
```

В нашем случае используется передача другой функции. Функция-получатель может вообще не вызвать полученную функцию, может вызвать ее позднее, а может подготовить контекст и выполнить в нем отложенный код.

Отказ от вызова

```
function callOnThursday(f)
{
  if(today === "Thursday")
    f();
}
```

f() вызывается только по вторникам

Отложенный вызов

```
function callTomorrow(f) {
  sleep(oneDay);
  f();
}
```

Ожидаем один день перед выполнением f()

Вызов в новом контексте

```
function withLogging(f) {
  try {
    f();
  } catch (error) {
    logToSnapErrors(error);
  }
}
```

Вызываем f() в try/catch

Мы заключаем код в функцию, чтобы отложить его выполнение, и выполняем его в контексте `try/catch`, который создается `withLogging()`. `withLogging()` закрепляет стандарт, необходимый команде Джорджа. Тем не менее позднее я покажу, как улучшить эту реализацию.



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Рефакторинг с заменой тела обратным вызовом хорошо подходит для устранения некоторых видов дублирования кода. Он нужен только для этого?

О: Хороший вопрос. В каком-то смысле да: вся его суть заключается в устранении дублирования. То же самое можно сказать о функциях, которые не являются функциями высшего порядка: они позволяют выполнять код по имени функции вместо дублирования тела. С функциями высшего порядка дело обстоит так же, но они позволяют менять поведение по выполняемому коду (обратный вызов), а не только по данным.

В: Почему мы передаем функции? Почему не передать обычные данные?

О: Снова хороший вопрос. Представьте, что в нашем примере `try/catch` передаются данные («обычный» аргумент) вместо аргумента-функции. Код выглядел бы так:

```
function withLogging(data) {
  try {
    data;
  } catch (error) {
    logToSnapErrors(error);
  }
}
```

Передается только результат вызова функции, а не сама функция

```
withLogging(saveUserData(user));
```

Обратите внимание: функция вызывается вне контекста блока try/catch

А теперь вопрос: что, если в `saveUserData()` произойдет ошибка? Будет ли она заключена в блок `try/catch`?

Нет, не будет. `saveUserData()` выполнится и выдаст ошибку до выполнения `withLogging()`. Конструкция `try/catch` в данном случае бесполезна.

Причина, по которой передается функция, заключается в том, что код внутри функции может выполняться в конкретном контексте. В данном случае контекст находится внутри `try/catch`. В случае `forEach()` контекстом является тело цикла `for`. Функции высшего порядка позволяют определять контексты для кода, определенного в другом месте. Появляется возможность повторного использования контекста, потому что он находится внутри функции.

Итоги главы

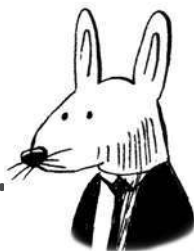
В этой главе были представлены концепции *первокласных значений*, *первокласных функций* и *функций высшего порядка*. В следующих главах мы проанализируем возможности этих концепций. При определенных различиях между действиями, вычислениями и данными идея функций высшего порядка поднимает возможности функционального программирования на новый уровень. Вторая часть книги (та, которую вы сейчас читаете) посвящена именно этим возможностям.

Резюме

- К первокласным значениям относится все, что можно сохранить в переменной, передать в аргументе или вернуть из функции. С первокласными значениями можно манипулировать в программном коде.
- Многие сущности языка не являются первокласными. Чтобы сделать их первокласными, можно упаковать их в функции, которые делают то же самое.
- В некоторых языках реализованы первокласные функции, то есть возможность интерпретации функций как первокласных значений. Первокласные функции необходимы для функционального программирования высокого уровня.
- Функциями высшего порядка называются функции, которые получают другие функции в аргументах (или возвращают функции). Функции высшего порядка позволяют абстрагировать изменяющееся поведение.
- *Неявный аргумент в имени функции* — признак «кода с душком»: различия между функциями отражаются в именах функций. Применение такого метода рефакторинга, как *явное выражение неявного аргумента*, позволяет сделать аргумент первокласным (вместо недоступной части имени функции).
- Метод рефакторинга, называемый *заменой тела функции обратным вызовом*, применяется для абстрагирования поведения. Он создает первокласный аргумент-функцию, представляющий различия в поведении между двумя функциями.

Что дальше?

Перед нами открылся путь к использованию потенциала функций высшего порядка. Мы рассмотрим много полезных приемов, которые нам помогут как в вычислениях, так и в действиях. В следующей главе мы продолжим применять методы рефакторинга, описанные в этой главе, для улучшения кода.



В этой главе

- ✓ Другие применения *замены тела функции обратным вызовом*.
- ✓ Возвращение функций другими функциями.
- ✓ Практика написания функций высшего порядка.

В предыдущей главе я представил навыки создания функций высшего порядка. В этой главе полученные знания будут применены в новых примерах. Мы начнем применять в коде подход копирования при записи, а затем улучшим систему регистрации ошибок, чтобы она не требовала столь значительной работы.

Одна проблема, два метода рефакторинга

В предыдущей главе вы узнали признак «кода с душком» и два метода рефакторинга, которые помогают устранить дублирование и найти более эффективные абстракции. С помощью этих методов создаются первоклассные значения и функции высшего порядка.

На всякий случай приведу их снова, так как они будут применяться во всех главах части II.

Признак «кода с душком»: неявный аргумент в имени функции

Этот признак указывает на аспекты кода, которые лучше было бы выразить в виде первоклассных значений. Если вы ссылаетесь на значение в теле функции и имя этого значения присутствует в имени функции, вероятно, перед вами проблема, которая должна решаться посредством описанного ниже рефакторинга.

Характеристики

1. Очень похожие реализации функции.
2. Имя функции указывает на различия в реализации.

Рефакторинг: явное выражение неявного аргумента

Если в имени функции присутствует неявный аргумент, как преобразовать его в реальный аргумент функции? Этот рефакторинг добавляет новый аргумент в функцию, чтобы значение стало первоклассным. Он поможет вам лучше выразить ваши намерения в коде, а иногда также противодействует дублированию кода.

Последовательность действий

1. Выявление неявного аргумента в имени функции.
2. Добавление явного аргумента.
3. Использование нового аргумента в теле вместо жестко фиксированного значения.
4. Обновление кода вызова.

Рефакторинг: замена тела обратным вызовом

Синтаксис языка часто не является первоклассным. Этот метод рефакторинга позволяет вам заменить тело (изменяющуюся часть) блоком кода с обратным вызовом. В дальнейшем нужное поведение передается первоклассной функцией. Это мощный механизм создания функций высшего порядка на базе существующего кода.

Последовательность действий

1. Определение частей: предшествующей, тела и завершающей.
2. Извлечение всего кода в функцию.
3. Извлечение тела в функцию, которая передается в аргументе этой функции.

Мы будем постепенно применять эти полезные навыки, чтобы довести их до автоматизма.

Рефакторинг копирования при записи



Дженна: Правда? Я думала, что замена тела обратным вызовом работает только для устранения дублирования в синтаксисе, например в циклах `for` и командах `try/catch`.

Ким: Да, как мы уже видели, в таких ситуациях она помогает. Но также может помочь и с другими видами дублирования.

Дженна: Ого! Хотела бы я это увидеть.

Ким: Что ж, ты уже знаешь первый шаг.

Дженна: Верно... Определить предшествующую часть, тело и завершающую часть.

Последовательность действий по замене тела обратным вызовом

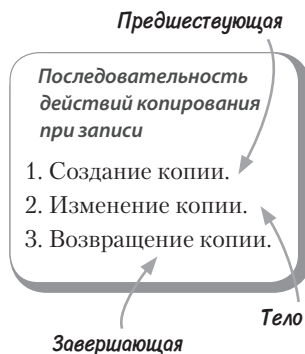
1. Определение частей: предшествующей, тела и завершающей.
2. Извлечение функции.
3. Извлечение обратного вызова.

Ким: Точно. А когда ты с ними определишься, дальше все пойдет как по маслу.

Дженна: Правила копирования при записи: создаем копию, изменяем копию, возвращаем копию. Переменная часть определяет то, как именно происходит изменение. Две другие части всегда одинаковы для заданной структуры данных.

Ким: Если что-то изменяется, это должно быть телом. И оно должно быть вложено между двумя постоянными частями: предшествующей и завершающей.

Дженна: И здесь можно воспользоваться рефакторингом!



Рефакторинг копирования при записи для массивов

В главе 6 мы разработали несколько функций копирования при записи для массивов. Все они строились по базовой схеме «создание копии — изменение копии — возвращение копии». Применим к ним рефакторинг замены тела обратным вызовом, чтобы стандартизировать паттерн.

Последовательность действий копирования при записи

1. Создание копии.
2. Изменение копии.
3. Возвращение копии.

1. Определение частей: предшествующей, тела и завершающей

Ниже приведены некоторые операции копирования при записи. Как видно, все они имеют очень похожие определения. Три фазы: «копирование/изменение/возврат» — имеют естественные аналоги: «предшествующая часть/тело/завершающая часть».

<pre>function arraySet(array, idx, value) { var copy = array.slice(); copy[idx] = value; return copy; }</pre>	← <i>Предшествующая</i> → ← <i>Тело</i> → ← <i>Завершающая</i> →	<pre>function push(array, elem) { var copy = array.slice(); copy.push(elem); return copy; }</pre>
<pre>function drop_last(array) { var array_copy = array.slice(); array_copy.pop(); return array_copy; }</pre>	← <i>Предшествующая</i> → ← <i>Тело</i> → ← <i>Завершающая</i> →	<pre>function drop_first(array) { var array_copy = array.slice(); array_copy.shift(); return array_copy; }</pre>

Поскольку копирование и возврат одинаковы, ограничимся первой функцией `arraySet()`. Впрочем, с таким же успехом можно выбрать любую.

2. Извлечение функции

Следующим шагом станет извлечение трех разделов в функцию. Функция будет содержать код предшествующей и завершающей части, поэтому ей стоит присвоить имя, относящееся к ее содержательной части: копированию массивов.

Последовательность действий по замене тела обратным вызовом

1. Определение частей: предшествующей, тела и завершающей.
2. Извлечение функции.
3. Извлечение обратного вызова.

Оригинал

```
function arraySet(array, idx, value) {
  var copy = array.slice();
  copy[idx] = value;
  return copy;
}
```

После выделения функции

```
function arraySet(array, idx, value) {
  return withArrayCopy(array);
}

function withArrayCopy(array) {
  var copy = array.slice();
  copy[idx] = value;
  return copy;
}
```

Выделение функции

Не определено в этой области видимости

Не определено в этой области видимости

Все сделано правильно, но запустить этот код пока не удастся. Дело в том, что `idx` и `value` не определены в области видимости `withArrayCopy()`. Перейдем к следующему шагу — извлечению тела в функцию.

Мы начали применять замену тела функции обратным вызовом к операциям копирования при записи для массивов. Только что был завершен шаг 2 замены тела обратным вызовом. При этом был получен следующий код:

Последовательность действий по замене тела обратным вызовом

1. Определение частей: предшествующей, тела и завершающей.
2. Извлечение функции.
3. Извлечение обратного вызова.

```
function arraySet(array, idx, value) {
  return withArrayCopy(array);
}

function withArrayCopy(array) {
  var copy = array.slice();
  copy[idx] = value;
  return copy;
}
```

Операция с копированием при записи

Предшествующая

Тело

Завершающая

Две переменные не определены в этой области видимости

Второй шаг был выполнен, но код запускать еще нельзя. Переменные `idx` и `value` не определены в области видимости `withArrayCopy()`. Продолжим на следующем шаге.

3. Извлечение обратного вызова

На следующем шаге тело выделяется в обратный вызов. Поскольку обратный вызов будет модифицировать массив, мы назовем его `modify`.

Текущая версия

```
function arraySet(array, idx, value) {
  return withArrayCopy(
    array
  );
}

function withArrayCopy(array) {
  var copy = array.slice();
  copy[idx] = value;
  return copy;
}
```

*Тело преобразуется в аргумент
и передается при вызове*

После извлечения обратного вызова

```
function arraySet(array, idx, value) {
  return withArrayCopy(
    array,
    function(copy) {
      copy[idx] = value;
    });
}

function withArrayCopy(array, modify) {
  var copy = array.slice();
  modify(copy);
  return copy;
}
```

*Обратный
вызов*

Готово!

Сравним код перед рефакторингом с кодом, полученным в результате, а затем обсудим, чего же мы добились применением рефакторинга.

До рефакторинга

```
function arraySet(array, idx, value) {
  var copy = array.slice();
  copy[idx] = value;
  return copy;
}
```

*Многоразовая функция, которая
стандартизирует механизм
копирования при записи*

После рефакторинга

```
function arraySet(array, idx, value) {
  return withArrayCopy(array, function(copy) {
    copy[idx] = value;
  });
}

function withArrayCopy(array, modify) {
  var copy = array.slice();
  modify(copy);
  return copy;
}
```

Иногда рефакторинг и избавление от дублирования сокращают объем кода. В данном случае это не так. Дублируемый код уже был достаточно коротким: всего две строки. Мы реализовали и стандартизировали механизм копирования при записи для массивов. Его уже не нужно одинаково записывать во всей кодовой базе. Код находится в одном месте.

Также появилась новая возможность: в главе 6, когда мы изучали механизм копирования при записи, были разработаны версии с копированием при записи практически для всех важных операций с массивами. А если мы что-то забыли? Новая функция `withArrayCopy()` — результат рефакторинга — справится с любой операцией, которая изменяет массив. Например, что случится, если вам попалась библиотека с более эффективной реализацией сортировки? Вы сможете легко использовать новую функцию сортировки с сохранением механизма копирования при записи.

```
var sortedArray = withArrayCopy(array, function(copy) {
    SuperSorter.sort(copy);
});
```

Улучшенная функция, которая выполняет сортировку «на месте»

Польза от применения рефакторинга огромна. Он даже открывает возможность оптимизации. Серия операций с копированием при записи создает новую копию для каждой операции. Процесс может быть медленным и требующим больших затрат памяти. `withArrayCopy()` предоставляет возможность оптимизировать операции за счет создания единственной копии.

Создаем промежуточные копии

```
var a1 = drop_first(array);
var a2 = push(a1, 10);
var a3 = push(a2, 11);
var a4 = arraySet(a3, 0, 42);
```

Код создает четыре копии массива

Создаем одну копию

```
var a4 = withArrayCopy(array, function(copy){
    copy.shift();
    copy.push(10);
    copy.push(11);
    copy[0] = 42;
});
```

Создается одна копия

Внесение четырех изменений в копию

Теперь все функции массивов с копированием при записи можно реализовать заново с использованием `withArrayCopy()`. Более того, это может быть неплохим упражнением.

Польза от рефакторинга

1. Стандартизация подхода.
2. Применение подхода к новым операциям.
3. Оптимизация серий изменений.



Ваш ход

Мы только что создали функцию `withArrayCopy()`, которая реализует практику копирования при записи, описанную в главе 6. Руководствуясь примером `arraySet()`, перепишите `push()`, `drop_last()` и `drop_first()`.

```
function withArrayCopy(array, modify) {
  var copy = array.slice();
  modify(copy);
  return copy;
}
```

Пример

```
function arraySet(array, idx, value) {
  var copy = array.slice();
  copy[idx] = value;
  return copy;
}
```

```
function arraySet(array, idx, value) {
  return withArrayCopy(array,
    function(copy) {
      copy[idx] = value;
    });
}
```

*Запишите здесь
свой ответ*

```
function push(array, elem) {
  var copy = array.slice();
  copy.push(elem);
  return copy;
}
```

```
function drop_last(array) {
  var array_copy = array.slice();
  array_copy.pop();
  return array_copy;
}
```

```
function drop_first(array) {
  var array_copy = array.slice();
  array_copy.shift();
  return array_copy;
}
```



Ответ

Оригинал

```
function push(array, elem) {  
  var copy = array.slice();  
  copy.push(elem);  
  return copy;  
}
```

```
function drop_last(array) {  
  var array_copy = array.slice();  
  array_copy.pop();  
  return array_copy;  
}
```

```
function drop_first(array) {  
  var array_copy = array.slice();  
  array_copy.shift();  
  return array_copy;  
}
```

С использованием withArrayCopy()

```
function push(array, elem) {  
  return withArrayCopy(array, function(copy) {  
    copy.push(elem);  
  });  
}
```

```
function drop_last(array) {  
  return withArrayCopy(array, function(copy) {  
    copy.pop();  
  });  
}
```

```
function drop_first(array) {  
  return withArrayCopy(array, function(copy) {  
    copy.shift();  
  });  
}
```



Ваш ход

Мы только что разработали функцию `withArrayCopy()`, которая реализует механизм копирования при записи для массивов. Сможете ли вы сделать то же самое для объектов?

Ниже приведен код для пары реализаций с копированием при записи:

```
function objectSet(object, key, value) {    function objectDelete(object, key) {
  var copy = Object.assign({}, object);      var copy = Object.assign({}, object);
  copy[key] = value;                        delete copy[key];
  return copy;                             return copy;
}
```

Напишите функцию `withObjectCopy()` и используйте ее для реализации следующих двух функций с копированием при записи для объектов.

Запишите здесь свой ответ



Ответ

```
function withObjectCopy(object, modify) {
  var copy = Object.assign({}, object);
  modify(copy);
  return copy;
}

function objectSet(object, key, value) {
  return withObjectCopy(object, function(copy) {
    copy[key] = value;
  });
}

function objectDelete(object, key) {
  return withObjectCopy(object, function(copy) {
    delete copy[key];
  });
}
```



Ваш ход

Джордж только что упаковал все необходимое в `withLogging()`. Это была серьезная работа, но к счастью, она завершена. Однако Джордж видит другую возможность создания более общей версии. `try/catch` содержит две изменяющиеся части: тело `try` и тело `catch`. Пока мы допускаем изменение только для тела `try`. Сможете ли вы адаптировать рефакторинг для случая с двумя изменяющимися телами? Фактически Джордж хотел бы написать

```
tryCatch(sendEmail, logToSnapErrors)
```

ВМЕСТО

```
try {
  sendEmail();
} catch(error) {
  logToSnapErrors(error);
}
```

Ваша задача — написать функцию `tryCatch()`.

Подсказка: функция очень похожа на `withLogging`, но получает два аргумента-функции.

Запишите здесь свой ответ



Ответ

```
function tryCatch(f, errorHandler) {
  try {
    return f();
  } catch(error) {
    return errorHandler(error);
  }
}
```



Ваш ход

Просто для тренировки упакуем другой элемент синтаксиса с использованием рефакторинга замены *тела функции обратным вызовом*. На этот раз рефакторинг будет применен к команде `if`. Чтобы упростить задачу, реализуем команду `if` без `else`. Ниже приведены две команды `if`:

<i>Проверяемое условие</i> <pre>if(array.length === 0) { console.log("Array is empty"); }</pre>		<i>Блок then</i> <pre>if(hasItem(cart, "shoes")) { return setPriceByName(cart, "shoes", 0); }</pre>
--	--	--

Возьмите эти два примера и сделайте рефакторинг, чтобы написать функцию с именем `when()`. Вы должны иметь возможность использовать ее следующим образом:

<i>Проверяемое условие</i> <pre>when(array.length === 0, function() { console.log("Array is empty"); });</pre>		<pre>when(hasItem(cart, "shoes"), function() { return setPriceByName(cart, "shoes", 0); });</pre> <i>Блок then</i>
---	--	---

Запишите здесь свой ответ



Ответ

```
function when(test, then) {
  if(test)
    return then();
}
```



Ваш ход

После того как вы написали функцию `when()` из последнего упражнения, люди стали пользоваться ею — и им понравилось! Теперь они хотят добавить команду `else`. Переименуем функцию `when()` в `IF()` и добавим новый обратный вызов для ветви `else`.

<i>Проверяемое условие</i> <pre>IF(array.length === 0, function() { console.log("Array is empty"); }, function() { console.log("Array has something in it."); });</pre>	<i>Выполняемый блок</i> <pre>IF(hasItem(cart, "shoes"), function() { return setPriceByName(cart, "shoes", 0); }, function() { return cart; // unchanged });</pre>
--	--

Блок else

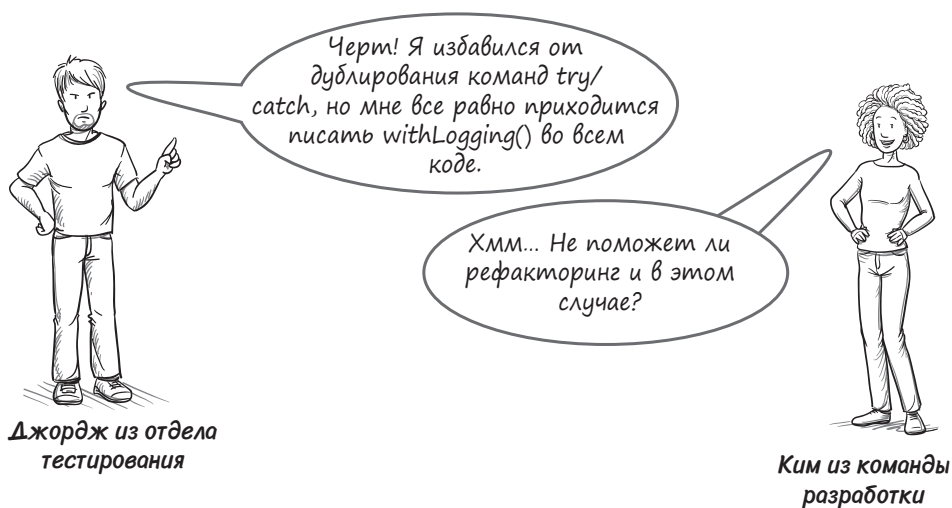
Запишите здесь свой ответ



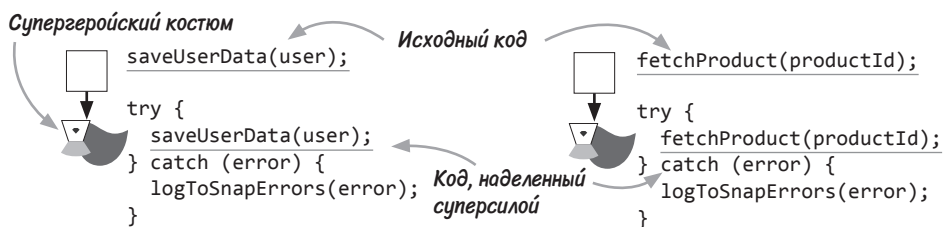
Ответ

```
function IF(test, then, ELSE) {
  if(test)
    return then();
  else
    return ELSE();
}
```

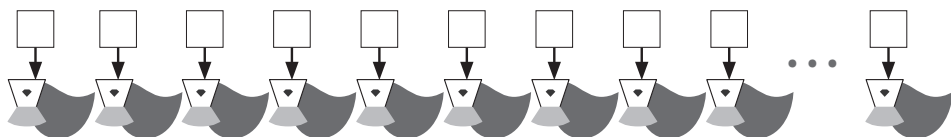
Возвращение функций функциями



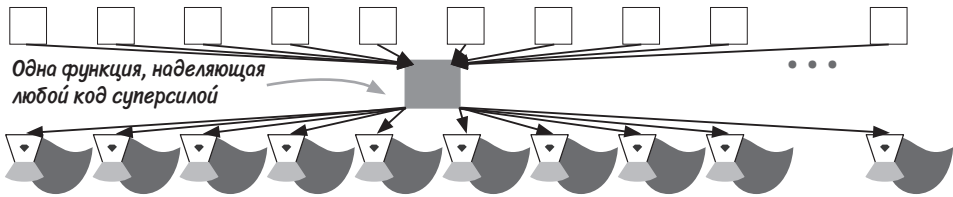
Джордж: Надеюсь, поможет. Требуется заключить код в try/catch и отправлять ошибки сервису Snap Errors®. Мы берем обычный код и наделяем его суперсилой. Эта суперсила выполняет код, перехватывает любые ошибки и отправляет их Snap Errors®.



Джордж: Все работает. Но так упаковываются тысячи строк кода. И даже после рефакторинга это приходится делать вручную, по одной строке.



Джордж: Было бы хорошо написать функцию, которая делает это за меня, чтобы мне не приходилось вручную проделывать это тысячу раз.



Ким: Так давай напишем ее! Это обычная функция высшего порядка.

Проанализируем проблему Джорджа и прототип решения.

Джордж хочет перехватывать ошибки и регистрировать их в сервисе Snap Errors®. Пара фрагментов кода:

```
try {
  saveUserData(user);
} catch (error) {
  logToSnapErrors(error);
}
```

```
try {
  fetchProduct(productId);
} catch (error) {
  logToSnapErrors(error);
}
```

Эти два фрагмента отличаются только одной строкой; слишком много дублирования!

Супергеройский костюм обозначает суперсилу

Джорджу придется писать очень похожие блоки `try/catch` по всему коду — везде, где вызываются эти функции. Ему хотелось бы решить эту проблему с дублированием заранее.

Вот какое решение предлагает он с Дженной:

```
function withLogging(f) {
  try {
    f();
  } catch (error) {
    logToSnapErrors(error);
  }
}
```

Эта функция инкапсулирует повторяющийся код

При использовании новой функции приведенные выше команды `try/catch` преобразуются к следующему виду:

```
withLogging(function() {
  saveUserData(user);
});
```

```
withLogging(function() {
  fetchProduct(productId);
});
```

При использовании кода все равно наблюдается значительное дублирование кода; различается только подчеркнутая часть

Теперь у нас появилась стандартная система. Тем не менее у нее есть две проблемы:

1. Вы можете забыть сохранить информацию в каком-то месте.
2. Вам все равно приходится писать свой код вручную.

И хотя дублируемого кода стало намного меньше, его все еще достаточно, чтобы вызывать раздражение. Мы хотим избавиться от всего дублирования.

По сути, нам хотелось бы иметь функцию, которая обладает всей функциональностью: исходной функциональностью кода в сочетании с суперсилой по перехвату и регистрации ошибок. Написать ее можно, но мы хотим, чтобы она была написана за нас автоматически.

Выше мы охарактеризовали текущее состояние прототипа Джорджа и пути его улучшения. Джордж может упаковать любой код стандартным образом, чтобы он последовательно регистрировал ошибки. Представим, как бы выглядело это решение, если бы мы переместили функциональность прямо в функцию. К счастью, это всего лишь прототип, и изменения вносятся достаточно просто. Вернемся к исходному коду:

Snap Errors®

Человеку свойственно ошибаться, но Snap не ошибается.

Из документации Snap Errors API:

`logToSnapErrors(error)` — отправляет ошибку сервису Snap Errors®. Ошибка должна инициализироваться и перехватываться в вашем коде.

Оригинал

```
try {
  saveUserData(user);
} catch (error) {
  logToSnapErrors(error);
}
```

```
try {
  fetchProduct(productId);
} catch (error) {
  logToSnapErrors(error);
}
```

Функциональность плюс суперсила (регистрация ошибок)

Чтобы ситуация стала предельно ясной, переименуем эти функции. Имена должны показывать, что они не выполняют регистрацию сами по себе:

Более содержательные имена

```
try {
  saveUserDataNoLogging(user);
} catch (error) {
  logToSnapErrors(error);
}
```

```
try {
  fetchProductNoLogging(productId);
} catch (error) {
  logToSnapErrors(error);
}
```

Изменяем имена, чтобы показать, что они не выполняют регистрацию ошибок

Эти фрагменты кода можно упаковать в функции с именами, которые показывают, что они *регистрируют* ошибки. Такое решение также получается более ясным.

Функции, регистрирующие ошибки

Можно вызвать эти функции и знать, что ошибки будут зарегистрированы

```
function saveUserDataWithLogging(user) {
  try {
    saveUserDataNoLogging(user);
  } catch (error) {
    logToSnapErrors(error);
  }
}

function fetchProductWithLogging(productId) {
  try {
    fetchProductNoLogging(productId);
  } catch (error) {
    logToSnapErrors(error);
  }
}
```

Но в телах еще наблюдается значительное дублирование кода

Две функции, не регистрирующие ошибки, упаковываются в обертку с функциональностью регистрации. При таком подходе при каждом вызове версий с поддержкой регистрации ошибок мы будем знать, что информация регистрируется. Имея эти функции, не нужно помнить о том, что код должен заключаться в блоки `try/catch`. И от нас потребуется только написать несколько функций с расширенными возможностями, вместо того чтобы упаковывать тысячи вызовов версий без регистрации. Но теперь появляется новое дублирование: эти две функции слишком похожи. Нужен способ автоматически создавать функции подобного вида.

Много дублирования кода

```
function saveUserDataWithLogging(user) {
  try {
    saveUserDataNoLogging(user);
  } catch (error) {
    logToSnapErrors(error);
  }
}

function fetchProductWithLogging(productId) {
  try {
    fetchProductNoLogging(productId);
  } catch (error) {
    logToSnapErrors(error);
  }
}
```

Представим на секунду, что у этих функций нет имен. Удалим имена и сделаем их анонимными. Кроме того, аргументу также будет присвоено более общее имя.

Предшествующая

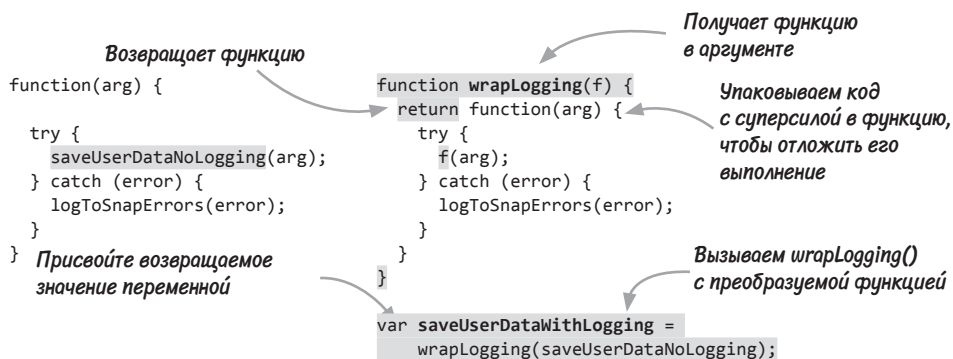
Тело

Завершающая

```
function(arg) {
  try {
    saveUserDataNoLogging(arg);
  } catch (error) {
    logToSnapErrors(error);
  }
}

function(arg) {
  try {
    fetchProductNoLogging(arg);
  } catch (error) {
    logToSnapErrors(error);
  }
}
```

Структура из предшествующей части, тела и завершающей части проявляется очень четко. Применим рефакторинг с заменой тела обратным вызовом. Но вместо того, чтобы добавлять к функции обратный вызов, мы применим исходный прием и упакуем ее в новую функцию. Начнем с функции слева:



Теперь `wrapLogging()` получает функцию `f` и возвращает функцию, которая упаковывает `f` в стандартную конструкцию `try/catch`. Таким образом, можно взять версию без регистрации ошибок и легко преобразовать ее в версию с регистрацией. Любую функцию можно наделять суперсилой регистрации ошибок!

```
var saveUserDataWithLogging = wrapLogging(saveUserDataNoLogging);
var fetchProductWithLogging = wrapLogging(fetchProductNoLogging);
```

Дублирование было устранено. Теперь у нас появился простой способ добавления стандартного поведения регистрации ошибок к любой функции. Посмотрим, как это делается.

Для этого разберем предшествующую и завершающую части:

Суперсила с ручной реализацией

```
try {
  saveUserData(user);
} catch (error) {
  logToSnapErrors(error);
}
```

Суперсила с автоматической реализацией

```
saveUserDataWithLogging(user)
```

Представьте, что вам пришлось написать `try/catch` и строку регистрации ошибок тысячу раз

Конечно, при этом очень многое происходит «за кулисами». Мы определили функцию, которая наделяет любую функцию той же суперсилой:

```
function wrapLogging(f) {
  return function(arg) {
    try {
      f(arg);
    } catch (error) {
```

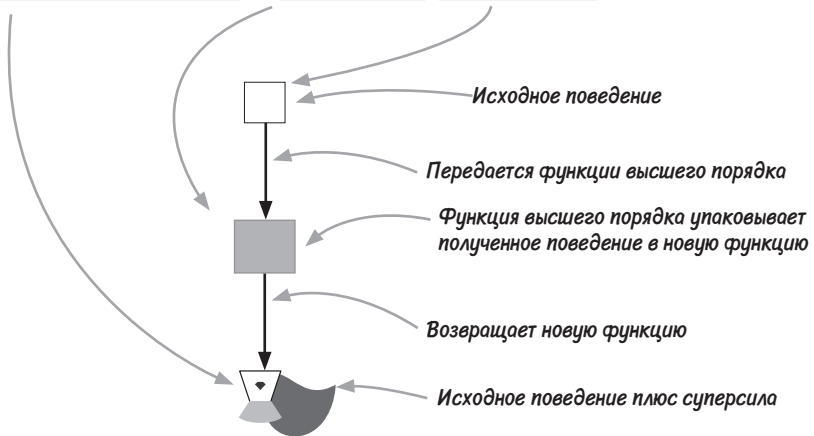
```

    logToSnapErrors(error);
  }
}

```

Мы воспользуемся этой функцией для определения функции `saveUserDataWithLogging()` на основе `saveUserData()`. Также изобразим ее наглядно:

```
var saveUserDataWithLogging = wrapLogging(saveUserData);
```



Возвращение функций функциями позволяет создавать фабрики функций. Таким образом мы автоматизируем создание функций и закрепляем стандарт.



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Вы присваиваете возвращаемое значение функции переменной, но я привык к тому, что все функции определяются с ключевым словом `function` на верхнем уровне. Разве это не вызовет путаницы?

О: Хороший вопрос. Возможно, вы будете привыкать к этому какое-то время. Впрочем, даже без применения паттерна вы, вероятно, уже используете другие признаки для определения того, какие переменные содержат данные, а какие содержат функции. Например, имена функций обычно представляются глаголами, а имена обычных переменных — существительными. Придется привыкать к тому, что существуют разные способы определения функций. Иногда они определяются прямо в коде, который вы пишете; иногда — как возвращаемые значения других функций.

В: Функция `wrapLogging()` получает функцию, которая работает с одним аргументом. Как заставить ее работать с несколькими аргументами? И как получить возвращаемое значение от функции?

О: С возвращаемым значением все просто: достаточно добавить ключевое слово `return`, чтобы вернуть значение из внутренней функции. Оно станет возвращаемым значением новой функции, которую вы создаете. Работа с переменным количеством аргументов в классическом JavaScript сопряжена с определенными трудностями. Задача существенно упростилась в ES6 — современном стиле JavaScript. Если вы используете ES6, поищите информацию об *операторе расширения* (spread operator) и *остаточных аргументах* (rest arguments). В других языках могут поддерживаться аналогичные средства.

Впрочем, на практике это не так уж трудно даже в классическом JavaScript, потому что JavaScript очень гибко обрабатывает недостающие или избыточные аргументы. А на практике обычно используются функции с небольшим количеством аргументов.

Если вы хотите, чтобы функция `wrapLogging()` могла работать с функциями, получающими до девяти аргументов, то это можно сделать так:

```
function wrapLogging(f) {  
  return function(a1, a2, a3, a4, a5, a6, a7, a8, a9) {  
    try {  
      return f(a1, a2, a3, a4, a5, a6, a7, a8, a9);  
    } catch (error) {  
      logToSnapErrors(error);  
    }  
  }  
}
```

При вызове JavaScript игнорирует неиспользуемые аргументы

Просто включите return во внутреннюю функцию

Существуют и другие методы, но этот легко объясняется и не требует глубокого понимания JavaScript. Посмотрите, как в вашем языке создаются функции с переменным количеством аргументов.



Ваш ход

Напишите функцию, преобразующую переданную ей функцию в функцию, которая перехватывает и игнорирует все ошибки. При возникновении ошибки просто верните `null`. Ваша функция должна работать с функциями, получающими до трех аргументов.

Подсказка

- Обычно для того, чтобы проигнорировать ошибку, достаточно заключить код в `try/catch` с пустой секцией `catch`.

```
try {
  codeThatMightThrow();
} catch(e) {
  // Ничего не делаем, чтобы игнорировать ошибки
}
```

Запишите здесь свой ответ




Ответ

```
function wrapIgnoreErrors(f) {
  return function(a1, a2, a3) {
    try {
      return f(a1, a2, a3);
    } catch(error) { // ошибки игнорируются
      return null;
    }
  };
}
```

**Ваш ход**

Напишите функцию `makeAdder()`, которая создает функцию для прибавления числа к другому числу. Например,

```
var increment = makeAdder(1);    var plus10 = makeAdder(10);  
  
> increment(10)                 > plus10(12)  
11                              22
```

Запишите здесь свой ответ

**Ответ**

```
function makeAdder(n) {  
  return function(x) {  
    return n + x;  
  };  
}
```




Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Похоже, возвращение функций из функций высшего порядка открывает много интересных возможностей. Можно ли записать так всю программу?

О: Хороший вопрос. Наверное, можно записать всю программу, не используя ничего, кроме функций высшего порядка. Но лучше спросить, стоит ли это делать? Очень легко увлечься таким интересным делом, как написание функций высшего порядка. Оно стимулирует центр нашего мозга, который заставляет нас почувствовать себя умными, как при решении сложной головоломки. Но хорошее программирование направлено не на решение головоломок, а на эффективное решение задач.

Истина такова: функции высшего порядка должны использоваться ради их сильной стороны, то есть сокращения дублирования в кодовых базах. В коде часто используются циклы, поэтому полезно иметь функцию высшего порядка для их правильной реализации (`forEach()`). В коде часто перехватываются ошибки, поэтому функция для стандартного выполнения этой операции тоже может пригодиться.

Многие функциональные программисты поддаются азарту. Написаны целые книги о том, как выполнять простейшие операции с использованием только функций высшего порядка. Но если посмотреть на код, можно ли сказать, что он действительно более понятен, чем прямолинейная реализация?

Исследуйте и экспериментируйте. Пробуйте применять функции высшего порядка в разных местах для разных целей. Ищите для них новые применения. Исследуйте их ограничения. Но не стоит делать это в коде, предназначенном для реальной эксплуатации. Помните, что исследования нужны только для обучения.

Когда вы предлагаете решение, в котором используется функция высшего порядка, сравните его с прямолинейным решением. Действительно ли оно лучше? Делает ли оно код более понятным? Сколько дублирующихся строк кода вы реально устраняете? Насколько легко будет постороннему читателю понять, что делает код? Не забывайте обо всех этих вопросах.

Резюме: все описанные методы обладают мощными возможностями, но за них приходится расплачиваться. Писать такой код приятно, и это заставляет нас забыть о проблемах с его чтением. Возьмите их на вооружение, но применяйте только тогда, когда они действительно улучшают код.

Итоги главы

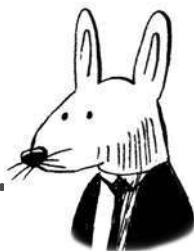
В этой главе углубленно рассматриваются концепции первоклассных значений, первоклассных функций и функций высшего порядка. Мы займемся изучением их потенциала в следующих главах. После проведения различий между действиями, вычислениями и данными идея функций высшего порядка открывает новый уровень мощи функционального программирования. Этой теме полностью посвящена часть II книги.

Резюме

- Функции высшего порядка могут закреплять паттерны и механизмы, которые обычно нам пришлось бы поддерживать вручную. Так как они определяются только один раз, их достаточно правильно реализовать однократно, чтобы потом использовать везде, где потребуется.
- Функции можно создавать, возвращая их из функций высшего порядка. Такие функции могут использоваться обычным образом: чтобы определить для них имя, присвойте их переменной.
- У функций высшего порядка есть как достоинства, так и недостатки. Они могут устранять дублирование кода, но иногда за это приходится расплачиваться удобочитаемостью. Хорошо изучите функции высшего порядка и применяйте разумно.

Что дальше?

В предыдущей главе была представлена функция `forEach()`, которая позволяет нам перебирать массивы. В следующей главе мы рассмотрим функциональный стиль перебора, основанный на расширении этой идеи. Мы изучим три важных инструмента функционального программирования, которые отражают стандартные паттерны перебора массивов.



В этой главе

- ✓ Три инструмента функционального программирования: `map()`, `filter()` и `reduce()`.
- ✓ Замена простых циклов `for` с перебором массива инструментами функционального программирования.
- ✓ Построение реализаций трех инструментов функционального программирования.

Многие функциональные языки включают различные мощные абстрактные функции для работы с коллекциями данных. В этой главе мы сосредоточимся на трех очень распространенных функциях, а именно `map()`, `filter()` и `reduce()`.

Эти инструменты закладывают основу многих функциональных программ. Они заменяют циклы `for` в арсенале функционального программиста. Так как перебор массивов — операция, которую мы выполняем очень часто, эти инструменты в высшей степени полезны.

Один признак «кода с душком» и два рефакторинга

В главе 10 вы узнали признак «кода с душком» и два метода рефакторинга, которые помогают избавиться от дублирования и найти более эффективные абстракции. С ними мы создаем первоклассные значения и функции высшего порядка. Эти новые навыки будут актуальны во всей части II книги. На всякий случай напомним их суть.

Признак «кода с душком»: неявный аргумент в имени функции

Этот признак указывает на аспекты кода, которые лучше было бы выразить в виде первоклассных значений. Если вы ссылаетесь на значение в теле функции и имя этого значения присутствует в имени функции, вероятно, перед вами проблема, которая должна решаться посредством описанного ниже рефакторинга.

Характеристики

1. Очень похожие реализации функции.
2. Имя функции указывает на различия в реализации.

Рефакторинг: явное выражение неявного аргумента

Если в имени функции присутствует неявный аргумент, как преобразовать его в реальный аргумент функции? Этот рефакторинг добавляет новый аргумент в функцию, чтобы значение стало первоклассным. Он поможет вам лучше выразить ваши намерения в коде, а иногда также противодействует дублированию кода.

Последовательность действий

1. Выявление неявного аргумента в имени функции.
2. Добавление явного аргумента.
3. Использование нового аргумента в теле вместо жестко фиксированного значения.
4. Обновление кода вызова.

Рефакторинг: замена тела обратным вызовом

Этот метод рефакторинга позволяет вам заменить тело (изменяющуюся часть) блоком кода с обратным вызовом. В дальнейшем нужное поведение передается первоклассной функцией. Это мощный механизм создания функций высшего порядка на базе существующего кода.

Последовательность действий

1. Определение частей: предшествующей, тела и завершающей.
2. Извлечение всего кода в функцию.
3. Извлечение тела в функцию, которая передается в аргументе этой функции.

Мы будем постепенно применять эти полезные навыки, чтобы они стали вашей привычкой.

MegaMart создает группу взаимодействия с клиентами

To: Персонал MegaMart

From: Руководство MegaMart

Добрый день!

У нас более 1 миллиона клиентов, которым нужно рассылать электронную почту.

- Отдел маркетинга отправляет информацию о рекламных акциях.
- Юридический отдел отправляет информацию, связанную с правовыми вопросами.
- Отдел развития бизнеса отправляет сообщения для клиентов.
- И это еще не все!

Все эти сообщения служат разным целям. Но у них есть одно общее свойство: они должны отправляться некоторым клиентам, а некоторые их получать не должны. И это создает серьезную проблему. По нашим оценкам, существуют сотни разных подгрупп клиентов, которым нужно отправлять сообщения. Чтобы должным образом отреагировать на эту потребность, мы создаем группу взаимодействия с клиентами. Участники группы будут отвечать за написание и сопровождение кода.

Состав группы:

- Ким из команды разработки.
- Джон из отдела маркетинга.
- Гарри из службы поддержки.

С этого момента, если вам понадобятся данные клиентов, отправляйте запросы непосредственно им.

Искренне ваше,
руководство

Эта служебная записка была разослана этим утром и стала полным сюрпризом для участников группы. Новая группа еще находится в стадии организации, а запросы уже начали поступать. Вот первый из них.

Запрос на получение данных: обработка купонов		
Мы реализовали эту процедуру еще в главе 3, но теперь это ваша работа.		Отправитель: директор по маркетингу
		Ответственный: Ким из группы разработки



Код из главы 3

```
function emailsForCustomers(customers, goods, bests) {
  var emails = [];
  for(var i = 0; i < customers.length; i++) {
    var customer = customers[i];
    var email = emailForCustomer(customer, goods, bests);
    emails.push(email);
  }
  return emails;
}
```

Это цикл `for`, но теперь у нас есть `forEach()`

Джон: Думаете, это решение можно улучшить?

Гарри: Не знаю. Выглядит довольно просто: это уже вычисление, а не действие.

Ким: Верно. Но представьте, сколько раз нам придется писать такой код. Вы же читали служебную записку: сотни разных подгрупп клиентов.

Джон: Да! От такого количества циклов `for` можно свихнуться.

Ким: Уверена, что в функциональном программировании найдется ответ, но я его еще не знаю. Просто для затравки: разве мы недавно не узнали «последний цикл `for` для перебора массива, который вам придется написать»? Здесь это должно сработать.

Гарри: Ты права! Преобразуем цикл `for` в `forEach()`.

```
function emailsForCustomers(customers, goods, bests) {
  var emails = [];
  forEach(customers, function(customer) {
    var email = emailForCustomer(customer, goods, bests);
    emails.push(email);
  });
  return emails;
}
```



После преобразования в `forEach()`

```
function emailsForCustomers(customers, goods, bests) {
  var emails = [];
  forEach(customers, function(customer) {
    var email = emailForCustomer(customer, goods, bests);
    emails.push(email);
  });
  return emails;
}
```

Гарри: Намного лучше! Мы избавились от большого количества однообразного мусора.

Джон: Да, но я все равно свихнусь, если мне придется записывать одну и ту же функцию миллион раз. Я не для того пошел в маркетинг, чтобы моя жизнь превратилась в какой-то День сурка.

Ким: Погодите! Это очень похоже на одну штуку, про которую я читала. Это называется `map()`.

Джон: `map()`? В смысле карта?

Ким: Нет! `map()` — это функция, преобразующая один массив в другой массив той же длины. Вы заметили, что мы берем массив клиентов и возвращаем массив сообщений электронной почты? Это идеально подходит для `map()`.

Гарри: Хмм... Кажется, я начинаю понимать. А можешь еще раз объяснить?

Ким: Хорошо. Подобно тому как `forEach()` — это функция высшего порядка, которая перебирает массив, `map()` — функция высшего порядка, которая перебирает массив. Различие в том, что `map()` возвращает новый массив.

Джон: Что находится в новом массиве?

Ким: В этом вся суть: функция, которую ты передаешь, указывает, что должно быть в новом массиве.

Джон: И это упрощает код?

Ким: Да! Код становится короче и проще. Я хотела бы объяснить больше, но для этого понадобится целая страница...

map() в примерах

Рассмотрим некоторые функции, построенные по той же схеме, — теперь за них отвечает группа взаимодействия с клиентами.



Если присмотреться, становится ясно, что отличается только код, генерирующий новые элементы массива. Можно провести стандартную *замену тела функции обратным вызовом*, как это делалось в двух последних главах.

Последовательность действий по замене тела функции обратным вызовом

1. Определение частей: предшествующей, тела и завершающей.
2. Извлечение функции.
3. Извлечение обратного вызова.

Извлечение тела в обратный вызов

Оригинал

```

function emailsForCustomers(customers, goods, bests) {
  var emails = [];
  forEach(customers, function(customer) {
    var email = emailForCustomer(customer, goods, bests);
    emails.push(email);
  });
  return emails;
}

```

Извлечение forEach() в map()

После замены обратным вызовом

```

function emailsForCustomers(customers, goods, bests) {
  return map(customers, function(customer) {
    return emailForCustomer(customer, goods, bests);
  });
}

function map(array, f) {
  var newArray = [];
  forEach(array, function(element) {
    newArray.push(f(element));
  });
  return newArray;
}

```

Тело передается в форме обратного вызова

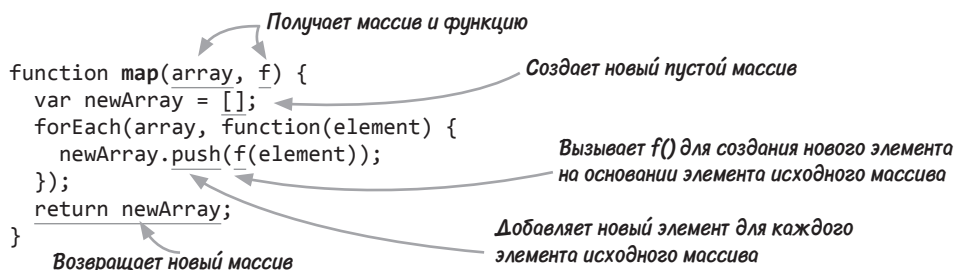
Аргумент с обратным вызовом

Здесь происходит обратный вызов

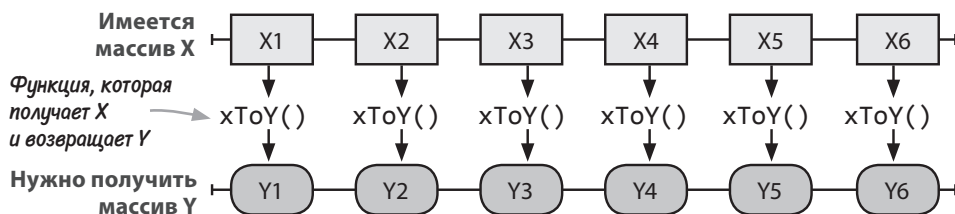
Мы извлекли новую функцию `map()`, которая выполняет типичный перебор. Эта функция является стандартным инструментом, находящимся на вооружении у функциональных программистов. Собственно, именно вследствие своей полезности она стала одним из трех главных инструментов функционального программирования. Посвятим еще немного времени изучению того, что она делает.

Инструмент функционального программирования: `map()`

`map()` — это один из трех инструментов функционального программирования, выполняющих огромную работу для функционального программиста. Вскоре мы рассмотрим два других инструмента: `filter()` и `reduce()`. А пока повнимательнее присмотримся к `map()`.



Можно сказать, что `map()` преобразует массив X (некоторую группу значений) в массив Y (другую группу значений). Для этого необходимо передать функцию, преобразующую X в Y , то есть функцию, которая получает X и возвращает Y . `map()` превращает функцию, которая оперирует с одним значением, в функцию, способную работать с целым массивом.



Проще всего использовать функцию `map()`, когда ей передается вычисление. В таком случае выражение, вызывающее `map()`, тоже будет вычислением. Если передать `map()` действие, то действие будет вызвано по одному разу для каждого элемента массива. В таком случае выражение будет действием. Посмотрим, как `map()` используется в нашем примере.

map() передается массив клиентов

map() передается функция, которая получает клиента и возвращает адрес электронной почты

```
function emailsForCustomers(customers, goods, bests) {
  return map(customers, function(customer) {
    return emailForCustomer(customer, goods, bests);
  });
}
```

Возвращает адрес электронной почты, вычисленный на основании клиента

Погодите! Что происходит с синтаксисом? Откуда здесь взялось имя *customer*?

*Откуда мы знаем, что здесь должно использоваться имя *customer*?*

```
function emailsForCustomers(customers, goods, bests) {
  return map(customers, function(customer) {
    return emailForCustomer(customer, goods, bests);
  });
}
```



Гарри из службы поддержки



Загляни в словарь

Встроенная функция определяется в месте ее использования (вместо назначения имени для вызова в будущем).

Хорошо, что мы об этом заговорили. Это очень частое препятствие для людей, недавно познако- мившихся с функциями высшего порядка.

Разберем происходящее поэтапно. `map()` полу- чает два аргумента: массив и функцию.

Здесь `map()` передается массив, в котором должны находиться данные клиентов. JavaScript не проверяет типы, поэтому может оказаться, что в массиве хранится что-то другое. Тем не менее мы ожидаем, что в нем хранятся только данные клиен- тов. Язык с проверкой типов смог бы гарантировать это ожидание. Но если вы доверяете своему коду, то в массиве будут именно данные клиентов.

```
function emailsForCustomers(customers, goods, bests) {
  return map(customers, function(customer) {
    return emailForCustomer(customer, goods, bests);
  });
}
```

Передаваемая функция является встроенной анонимной функцией, и это означает, что она определяется прямо в месте использования. Определяя функ-

цию, мы должны сообщить ей имена аргументов. Эти имена могут быть любыми: X, Y или даже **pumpkin**. Но для ясности мы использовали имя **customer**.

```
function(X) {
  return
  emailForCustomer(
    X, goods, bests
  );
}
```

```
function(Y) {
  return
  emailForCustomer(
    Y, goods, bests
  );
}
```

```
function(pumpkin) {
  return
  emailForCustomer(
    pumpkin, goods, bests
  );
}
```

Все эти функции
эквивалентны

Почему **customer**? Потому что **map()** будет вызывать эту функцию с элементами из переданного массива, по одному элементу за раз. Потому что мы ожидаем, что массив содержит только данные клиентов, и имя **customer** выглядит логично. Единственное число используется потому, что **map()** будет вызывать функцию для одного элемента.

Три способа передачи функций

В JavaScript существуют три способа передачи функций другим функциям. В других языках их может быть больше или меньше. Ситуация в JavaScript довольно типична для языков с первоклассными функциями.

Глобальное определение

Функцию можно определить и назвать на глобальном уровне. Этот способ определения типичен для большинства функций. Он позволяет обратиться к функции по имени практически в любой точке программы.

```
function greet(name) {
  return "Hello, " + name;
}
```

Функция определяется с именем в одной
точке программы

```
var friendGreetings = map(friendsNames, greet);
```

Программа обращается к функции по имени,
в данном случае функция передается **map()**



Загляни в словарь

Анонимная функция не имеет имени. Обычно анонимные функции встречаются при определении функций во встроенном виде.



Загляни в словарь

Встроенная функция определяется в месте ее использования. Например, функция может определяться в списке аргументов.

Локальное определение

Функцию можно определить и назвать в локальной области видимости. У такой функции есть имя, но к ней нельзя будет обратиться по этому имени за пределами области видимости. Данная возможность будет полезна, если вам нужно обращаться к другим значениям в локальной области видимости, но вы хотите работать с функцией по имени.



Загляни в словарь

Анонимная функция не имеет имени. Обычно анонимные функции встречаются при определении функций во встроенном виде.

```
function greetEverybody(friends) {
  var greeting;
  if(language === "English")
    greeting = "Hello, ";
  else
    greeting = "Salut, ";
  var greet = function(name) {
    return greeting + name;
  };
  return map(friends, greet);
}
```

Находимся в области видимости этой функции

Функция определяется с именем в одной точке области видимости

Обращаемся к функции по имени в той же области видимости

Встроенное определение

Функция также может определяться непосредственно в месте ее использования. Иначе говоря, функция не присваивается переменной, поэтому имя ей не присваивается. Такие функции называются *анонимными*. Обычно анонимными становятся короткие функции, которые имеют смысл в определенном контексте и используются только один раз.

```
var friendGreetings = map(friendsNames, function(name) {
  return "Hello, " + name;
});
```

Функция определяется в точке использования

Закрывающая фигурная скобка для определения функции, а также закрывающая скобка для (из вызова map())

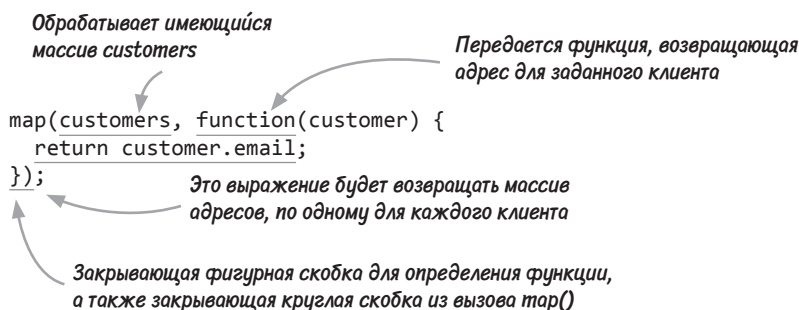
Пример: адреса всех клиентов

Рассмотрим простой, но довольно типичный пример использования `map()`. Требуется сгенерировать массив адресов электронной почты для всех клиентов. Массив всех клиентов передается на вход программы. Такая ситуация идеально подходит для использования `map()`.

Имеется: массив заказчиков.

Требуется получить: массив их адресов электронной почты.

Функция: берет одного заказчика и возвращает его адрес электронной почты.



```
map(customers, function(customer) {
  return customer.email;
});
```

Обработывает имеющийся массив customers (ссылка на `customers`)

Передается функция, возвращающая адрес для заданного клиента (ссылка на `function(customer) { return customer.email; }`)

Это выражение будет возвращать массив адресов, по одному для каждого клиента (ссылка на `return customer.email;`)

Закрывающая фигурная скобка для определения функции, а также закрывающая круглая скобка из вызова map() (ссылка на `});`)

`map()` помогает применить функцию к целому массиву значений.

Будьте внимательны!

`map()` — очень полезная функция. Функциональные программисты постоянно пользуются ею. Тем не менее это очень простая функция (собственно, этим она нам нравится). Учтите, что она вообще не проверяет, что будет добавляться в возвращаемый массив. А если у клиента нет адреса электронной почты, и `customer.email` содержит `null` или `undefined`? `null` попадет в массив.

Возникает уже знакомая проблема: если язык допускает `null` (как JavaScript), вы можете в отдельных случаях получить `null`. Однако `map()` усугубляет проблему, потому что функция будет применяться к целым массивам. Возможны два решения: либо использовать язык, в котором `null` не поддерживаются, либо действовать очень осторожно.

А если вы ожидаете появления значений `null`, но хотите избавиться от них, следующий инструмент — `filter()` — поможет вам с этим.



Ваш ход

Один из фрагментов кода, который нам предстоит написать, будет использоваться по праздникам. Всем клиентам нужно разослать поздравительные открытки. Нам потребуется объект с именем, фамилией и адресом каждого клиента. Используя `map()`, напишите код для генерирования этого массива объектов.

Исходные данные

- `customers` — массив всех клиентов.
- `customer.firstName`, `customer.lastName` и `customer.address` содержат все необходимые данные.

Запишите здесь свой ответ



Ответ

```
map(customers, function(customer) {  
  return {  
    firstName : customer.firstName,  
    lastName  : customer.lastName,  
    address   : customer.address  
  };  
});
```

Запрос на получение данных: список лучших клиентов

Нам хотелось бы отправить сообщения своим лучшим и самым надежным клиентам, чтобы они покупали еще больше! Я имею в виду клиентов, сделавших три и более покупки.



Отправитель:
директор по маркетингу



Ответственный:
Ким из группы разработки



*Джон из отдела
маркетинга*



*Гарри из службы
поддержки*



*Ким из команды
разработки*

Гарри: Нельзя ли воспользоваться для этой цели `map()`?

Джон: Сомневаюсь. `map()` всегда выдает массив с такой же длиной, как у полученного массива. В данном случае мы хотим получить подмножество только лучших клиентов.

Ким: Ты прав. Посмотрим, как бы это выглядело при использовании `forEach()`:

```
function selectBestCustomers(customers) {
  var newArray = [];
  forEach(customers, function(customer) {
    if(customer.purchases.length >= 3)
      newArray.push(customer);
  });
  return newArray;
}
```

Гарри: Похоже, но это не `map()`. У `map()` нет условных проверок.

Джон: Ким, ты всегда узнаешь ситуацию, в которой можно применить тот или иной паттерн. Есть идеи?

Ким: Да, есть. Нам поможет второй инструмент функционального программирования: `filter()`!

Джон: Второй инструмент функционального программирования? Надо же, какое странное совпадение.

Ким: А что ты хотел от нарисованного персонажа? Как бы то ни было, `filter()` — функция высшего порядка, которая позволяет создать новый массив на основе существующего массива. Она позволяет указать, какие элементы исходного массива должны остаться, а какие нужно пропустить.

Гарри: Понятно! Значит, мы можем вызвать функцию `filter()` и передать ей функцию, которая выбирает только лучших клиентов.

Ким: В точку! На следующей странице показано, как это делается.

filter() в примерах

Рассмотрим функции, за которые теперь отвечает группа взаимодействия с клиентами. Все эти функции строятся по одной схеме:

```
function selectBestCustomers(customers) {
  var newArray = [];
  forEach(customers, function(customer) {
    if(customer.purchases.length >= 3)
      newArray.push(customer);
  });
  return newArray;
}

function selectCustomersAfter(customers, date) {
  var newArray = [];
  forEach(customers, function(customer) {
    if(customer.signupDate > date)
      newArray.push(customer);
  });
  return newArray;
}

function selectCustomersBefore(customers, date) {
  var newArray = [];
  forEach(customers, function(customer) {
    if(customer.signupDate < date)
      newArray.push(customer);
  });
  return newArray;
}

function singlePurchaseCustomers(customers) {
  var newArray = [];
  forEach(customers, function(customer) {
    if(customer.purchases.length === 1)
      newArray.push(customer);
  });
  return newArray;
}
```

Отличается только проверяемое условие команды `if`: то есть код, отбирающий элементы для включения в новый массив. Он является телом, и мы можем применить стандартную замену тела обратным вызовом.

Выражение упаковывается в функцию и передается в аргументе

Оригинал

```
function selectBestCustomers(customers) {
  var newArray = [];
  forEach(customers, function(customer) {
    if(customer.purchases.length >= 3)
      newArray.push(customer);
  });
  return newArray;
}
```

forEach() выделяется в filter()

После замены обратным вызовом

```
function selectBestCustomers(customers) {
  return filter(customers, function(customer) {
    return customer.purchases.length >= 3;
  });
}

function filter(array, f) {
  var newArray = [];
  forEach(array, function(element) {
    if(f(element))
      newArray.push(element);
  });
  return newArray;
}
```

Условие теперь содержится в обратном вызове

Мы выделили новую функцию с именем `filter()`, которая выполняет обычный перебор. Эта функция является типичным инструментом функциональных программистов. Из-за своей полезности она заняла второе место среди трех главных инструментов функционального программирования.

Напомню, что обратный вызов может определяться глобально, локально или во встроенном виде. В данном случае функция короткая и понятная, поэтому мы определяем ее во встроенном виде.

Давайте чуть подробнее разберемся, что делает `filter()`.

Три способа определения функций

1. Глобальное определение.
2. Локальное определение.
3. Встроенное определение.

Инструмент функционального программирования: filter()

Функция `filter()` — это второй из трех инструментов, которыми пользуются функциональные программисты. Два других инструмента — `map()` и `reduce()`. Функция `reduce()` будет представлена ниже. А пока чуть подробнее рассмотрим `filter()`:

```
function filter(array, f) {
  var newArray = [];
  forEach(array, function(element) {
    if(f(element))
      newArray.push(element);
  });
  return newArray;
}
```

Получает массив и функцию

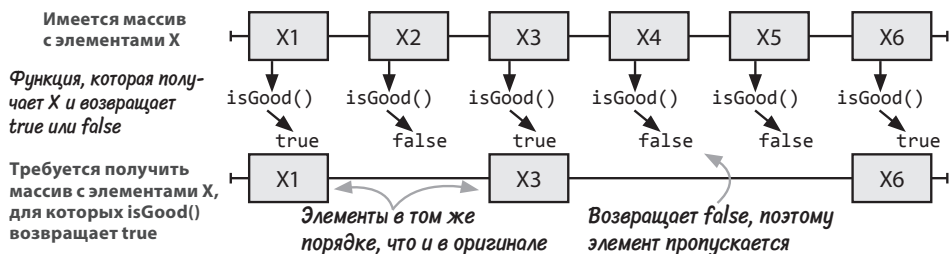
Создает новый пустой массив

Вызывает `f()` для проверки того, должен ли элемент попасть в новый массив

Добавляет исходный элемент, если он прошел проверку

Возвращает новый массив

Можно сказать, что `filter()` выбирает подмножество элементов массива. Для массива с элементами `X` результат все еще будет массивом с элементами `X`, но, возможно, с меньшим количеством элементов. Чтобы провести отбор, необходимо передать функцию для преобразования `X` в `Boolean`, то есть функцию, которая получает `X` и возвращает `true` или `false` (или эквивалент). Эта функция определяет, остается ли каждый элемент в результате (для `true`) или пропускается (для `false`). Функции, возвращающие `true` или `false`, часто называются *предикатами*. Элементы нового массива следуют в том же порядке, что и в оригинале, но некоторые могут пропускаться.



Как и в случае с `map()`, проще всего вызвать `filter()` с вычислением. `filter()` вызывает переданную функцию по одному разу для каждого элемента в массиве. Пример использования `filter()`:

```
function selectBestCustomers(customers) {
  return filter(customers, function(customer) {
    return customer.purchases.length >= 3;
  });
}
```

`filter()` передается массив клиентов

`filter()` передается функция, которая получает данные клиента и возвращает `true` или `false`

Возвращает `true` или `false`



Загляни в словарь

Предикаты — функции, возвращающие `true` или `false`. Часто используются для передачи `filter()` и другим функциям высшего порядка.

Пример: клиенты без покупок

Рассмотрим простой, но типичный пример использования `filter()`. Требуется сгенерировать массив всех клиентов, которые еще не оформили покупку. Задача идеально подходит для `filter()`.

Имеется: массив данных клиентов.

Требуется получить: массив данных клиентов с нулем покупок.

Функция: получает данные одного клиента и возвращает `true`, если у клиента не оформлено ни одной покупки.

```
filter(customers, function(customer) {
  return customer.purchases.length === 0;
});
```

`filter()` выбирает подмножество значений массива с сохранением их исходного порядка.

Осторожно!

Ранее в этой главе мы говорили о том, что при обработке `map()` в массиве могут оказаться значения `null`. Иногда это нормально! Но как избавиться от элементов `null`? Их можно просто отфильтровать.

```
var allEmails = map(customers, function(customer) {
  return customer.email;
});

var emailsWithoutNulls = filter(allEmails, function(email) {
  return email !== null;
});
```

`map()` и `filter()` хорошо работают в сочетании друг с другом. В следующей главе мы будем довольно долго изучать построение сложных запросов посредством объединения `map()`, `filter()` и `reduce()`.



Ваш ход

Отдел маркетинга хочет протестировать наш код. Они хотят случайным образом выбрать примерно треть клиентов и отправить им сообщение, отличное от того, которое отправляется другим. Для задач маркетинга можно взять идентификатор пользователя и проверить, делится ли он на 3 без остатка. Если делится, то клиент относится к тестовой группе. Ваша задача — написать код для генерирования тестовой группы.

Исходные данные

- `customers` — массив всех клиентов.
- `customer.id` — идентификатор пользователя.
- `%` — оператор вычисления остатка от деления; `x % 3 === 0` проверяет, делится ли `x` на 3 без остатка.

Запишите здесь свой ответ

```
var testGroup =
```

```
var nonTestGroup
```



Ответ

```
var testGroup = filter(customers, function(customer) {  
  return customer.id % 3 === 0;  
});  
  
var nonTestGroup = filter(customers, function(customer) {  
  return customer.id % 3 !== 0;  
});
```

Запрос на получение данных: общее количество покупок всех клиентов

Требуется узнать, сколько покупок было оформлено всеми клиентами. Покупки хранятся как часть данных клиента, нужно каким-то образом подсчитать их



Отправитель:
директор по маркетингу



Ответственный:
Ким из группы разработки



*Джон из отдела
маркетинга*



*Гарри из службы
поддержки*



*Ким из команды
разработки*

Гарри: Да, эта задача вряд ли подойдет для `map()` или `filter()`. Массив здесь вообще не возвращается.

Джон: Ты прав. В этом случае должно возвращаться число. Ким, у тебя найдется очередной инструмент функционального программирования?

Ким: Думаю, да. Посмотрим, как выглядит программа с `forEach()`.

```
function countAllPurchases(customers) {
  var total = 0;
  forEach(customers, function(customer) {
    total = total + customer.purchases.length;
  });
  return total;
}
```

Гарри: Похоже, но это не `map()` и не фильтр.

Джон: Но выглядит интересно, потому что предыдущий счетчик используется при вычислении следующего счетчика.

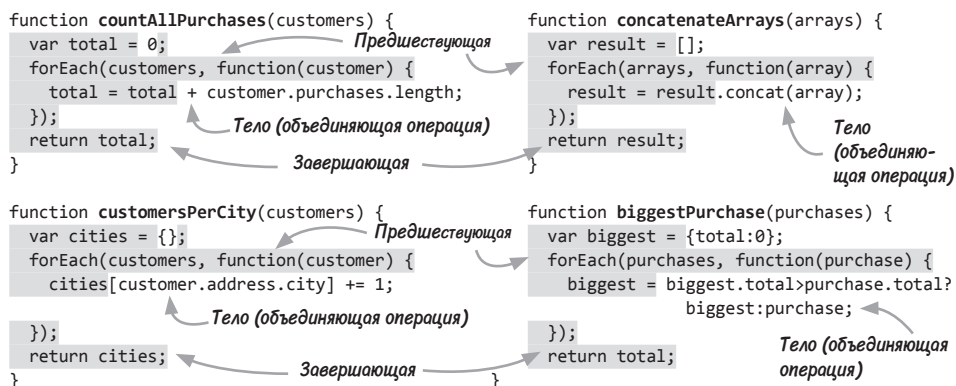
Ким: Да! Это `reduce()`, третий и последний инструмент функционального программирования. `reduce()` также является функцией высшего порядка, как и две другие. Но она используется для накопления значения в процессе перебора массива. В данном случае накапливается сумма, вычисляемая простым суммированием. Но в накоплении могут быть задействованы другие операции — собственно, любые.

Гарри: Давай я угадаю: функция, передаваемая `reduce()`, сообщает, как должно накапливаться значение.

Ким: Точно! На следующей странице показано, как она работает.

reduce() в примерах

Рассмотрим некоторые функции, за которые теперь отвечает группа взаимодействия с клиентами. Все эти функции строятся по одной схеме:



Эти примеры различаются только в двух аспектах. Первый — инициализация переменной. Второй — вычисление, определяющее следующее значение переменной. Следующее значение вычисляется на основании предыдущего значения переменной и текущего элемента обрабатываемого массива. Выполняется некая объединяющая операция, которая и является изменяемой частью.

Следующее значение вычисляется на основании

1. Текущего значения.
2. Текущего элемента массива

Оригинал

```

function countAllPurchases(customers) {
  var total = 0;
  forEach(customers, function(customer) {
    total = total + customer.purchases.length;
  });
  return total;
}

```

Цикл forEach() выделяется в reduce()

После замены обратным вызовом

```

function countAllPurchases(customers) {
  return reduce(
    customers, 0, function(total, customer) {
      return total + customer.purchases.length;
    }
  );
}

function reduce(array, init, f) {
  var accum = init;
  forEach(array, function(element) {
    accum = f(accum, element);
  });
  return accum;
}

```

Исходное значение

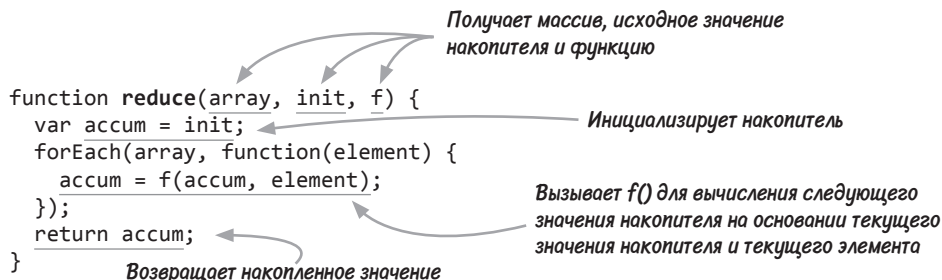
Функция обратного вызова

Два аргумента обратного вызова

Мы выделили новую функцию `reduce()`, которая выполняет обычный перебор. Это третий из трех основных инструментов, которыми пользуются функциональные программисты. Рассмотрим `reduce()` чуть подробнее.

Инструмент функционального программирования: `reduce()`

`reduce()` — третий инструмент, которым интенсивно пользуются функциональные программисты (два других — `map()` и `filter()`). Давайте разберемся в том, как работает `reduce()`.



```
function reduce(array, init, f) {
  var accum = init;
  forEach(array, function(element) {
    accum = f(accum, element);
  });
  return accum;
}
```

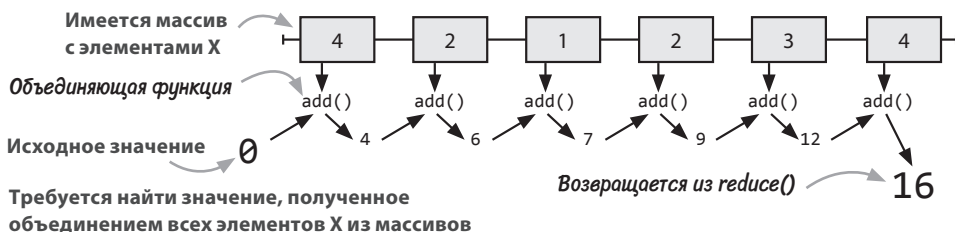
Получает массив, исходное значение накопителя и функцию

Инициализирует накопитель

Вызывает `f()` для вычисления следующего значения накопителя на основании текущего значения накопителя и текущего элемента

Возвращает накопленное значение

`reduce()` накапливает результат вычислений при переборе массива. Идея подобного накопления выглядит несколько абстрактно. Она может воплощаться во многих конкретных формах. Например, суммирование является разновидностью накопления, как и добавление значений в хеш-карту или конкатенация строк. Вы решаете, как следует понимать процесс накопления, передавая соответствующую функцию. Единственное ограничение заключается в том, что это вычисление, которое получает текущее значение накопителя и текущий элемент перебора. Возвращаемое значение функции станет новым значением накопителя.



Функция, передаваемая `reduce()`, должна получать два аргумента: текущее значение накопителя и текущий элемент массива. Функция должна вернуть значение, тип которого соответствует типу первого аргумента. Следующий пример демонстрирует использование этой функции.

`reduce()` передается массив клиентов

`reduce()` передается исходное значение

`reduce()` передается функция с двумя аргументами. Функция должна возвращать значение, тип которого соответствует типу первого аргумента

Возвращает сумму текущего накопленного значения и количества покупок текущего клиента

```
function countAllPurchases(customers) {
  return reduce(
    customers, 0,
    function(total, customer) {
      return total + customer.purchase.length;
    }
  );
}
```

Пример: конкатенация строк

Рассмотрим простой, но типичный пример использования `reduce()`. Имеется массив строк, которые необходимо объединить операцией конкатенации. Задача идеально подходит для `reduce()`.

Объединяющая операция

Имеется: массив строк.

Требуется получить: строку, которая представляет собой результат конкатенации исходных строк.

Функция: получает накопленную строку и текущую строку из массива для выполнения конкатенации.

Выполняет свертку массива строк посредством конкатенации

Исходное значение — пустая строка

Передается функция, которая выполняет конкатенацию

```
reduce(strings, "", function(accum, string) {
  return accum + string;
});
```

Это выражение возвращает строку, которая представляет собой результат конкатенации всех строк в массиве

`reduce()` помогает объединить элементы массива в одно значение, начиная с исходного значения.

В книге применяются эти правила, но в других языках они могут выглядеть иначе

Осторожно!

При использовании `reduce()` необходимо обращать внимание на два аспекта. Первый — порядок аргументов. Так как `reduce()` получает три аргумента, а передаваемая `reduce()` функция должна получать два аргумента, порядок легко перепутать. Кроме того, у эквивалентов `reduce()` в других языках аргументы следуют в другом порядке. Единого мнения нет и в помине! В этой книге для всех функций массивов используется схема «сначала массив, в конце функция». С таким правилом исходное значение может располагаться только в одном месте — в середине.

Второй аспект — способ определения исходного значения. Он зависит от операции и контекста. Но ответ можно получить по следующим вопросам.

- **С чего начинаются вычисления?** Например, суммирование начинается с нуля, поэтому нуль становится исходным значением для сложения. С другой стороны, умножение начинается с 1, и в этом случае исходное значение будет другим.
- **Какое значение должно возвращаться для пустого массива?** Для пустого массива строк конкатенация должна возвращать пустую строку.

Порядок аргументов для инструментов функционального программирования, используемый в книге

1. Сначала массив.
2. В конце обратный вызов.
3. Другие аргументы (если они есть) между ними.

Как определить исходное значение

1. С какого значения должно начинаться вычисление?
2. Что должно возвращаться для пустого массива?
3. Существует ли бизнес-правило?



Ваш ход

Бухгалтерия часто выполняет операции сложения и умножения. Напишите для нее функции суммирования и умножения списка чисел. Будьте внимательны с выбором исходного значения, передаваемого `reduce()`.

```
// Суммирование всех чисел в массиве
function sum(numbers) {
```

Запишите здесь свой ответ



```
}
```

```
// Умножение всех чисел в массиве
function product(numbers) {
```

```
}
```



Ответ

```
function sum(numbers) {
  return reduce(numbers, 0, function(total, num) {
    return total + num;
  });
}

function product(numbers) {
  return reduce(numbers, 1, function(total, num) {
    return total * num;
  });
}
```



Ваш ход

Функция `reduce()` очень полезна, если разумно подойти к выбору функции-накопителя. Напишите две функции, использующие `reduce()` для определения наименьшего и наибольшего числа в массиве, без использования `Math.min()` и `Math.max()`.

Исходные данные

- `Number.MAX_VALUE` — наибольшее число, поддерживаемое в JavaScript.
- `Number.MIN_VALUE` — наименьшее число, поддерживаемое в JavaScript.

```
// Вернуть наименьшее число в массиве
// (или Number.MAX_VALUE, если массив пуст)
function min(numbers) {
```

Запишите здесь свой ответ

```
}
```

```
// Вернуть наибольшее число в массиве
// (или Number.MIN_VALUE, если массив пуст)
function max(numbers) {
```

```
}
```



Ответ

```
function min(numbers) {
  return reduce(numbers, Number.MAX_VALUE, function(m, n) {
    if(m < n) return m;
    else      return n;
  });
}

function max(numbers) {
  return reduce(numbers, Number.MIN_VALUE, function(m, n) {
    if(m > n) return m;
    else      return n;
  });
}
```



Ваш ход

Разбираться в том, как работают инструменты функционального программирования, проще всего на крайних значениях. Ответьте на следующие вопросы, в каждом из которых задействовано то или иное крайнее значение.

1. Что возвращает функция `map()`, если передать ей пустой массив?
`> map([], xToY)`
2. Что возвращает функция `filter()`, если передать ей пустой массив?
`> filter([], isGood)`
3. Что возвращает функция `reduce()`, если передать ей пустой массив?
`> reduce([], init, combine)`
4. Что возвращает функция `map()`, если переданная ей функция просто возвращает свой аргумент?
`> map(array, function(x) { return x; })`
5. Что возвращает функция `filter()`, если переданная ей функция всегда возвращает `true`?
`> filter(array, function(_x) { return true; })`
6. Что возвращает функция `filter()`, если переданная ей функция всегда возвращает `false`?
`> filter(array, function(_x) { return false; })`

Символ подчеркивания в начале указывает на неиспользуемый аргумент



Ответ

1. `[]`
2. `[]`
3. `init`
4. Поверхностная копия массива.
5. Поверхностная копия массива.
6. `[]`

Функция `reduce()` эффективна, но она не кажется особо полезной. Не настолько полезной, как `map()` и `filter()`.

Интересно, что вы так считаете, потому что функция `reduce()` намного мощнее. Собственно, функции `map()` и `filter()` можно записать с использованием `reduce()`, но не наоборот.

С `reduce()` можно сделать очень много полезного. Этот инструмент открывает перед вами множество интересных возможностей. Мы не будем рассматривать эту тему подробно, но ниже описаны задачи, для решения которых можно воспользоваться `reduce()`.



Что можно сделать с `reduce()`

Отмена/повторение

Отмена и повторение — две операции, которые обычно невероятно сложно правильно реализовать, особенно если их поддержка не планировалась заранее. Если представить текущее состояние как результат применения `reduce()` к списку операций пользователя, отмена будет означать простое удаление последней операции из списка.

Воспроизведение операций пользователя для тестирования

Если исходное значение представляет исходное состояние системы, а массив является последовательностью операций пользователя, `reduce()` объединит их все в одно значение — текущее состояние.

Отладчик с перемещением во времени

Некоторые языки позволяют воспроизвести все изменения до определенного момента. Если что-то работает неправильно, вы отступаете и анализируете состояние в любой момент времени, решаете проблему, а затем воспроизводите с новым кодом. Звучит почти сверхъестественно, но `reduce()` предоставляет такую возможность.

Контрольный след

Иногда требуется узнать состояние системы в определенный момент времени: например, если юридический отдел обращается к вам с вопросом: «Что нам было известно на 31 декабря?» Функция `reduce()` позволяет сохранить историю, чтобы вы знали не только текущее состояние, но и путь, по которому вы к нему пришли.



Языковое сафари

В разных языках функция свертки, то есть `reduce()`, обозначается разными именами. Также часто встречается имя `fold()`. Иногда используются такие вариации, как `foldLeft()` и `foldRight()`, обозначающие направление обработки списка.



Ваш ход

Выше было сказано, что функции `map()` и `filter()` могут быть реализованы на базе `reduce()`. Попробуйте сделать это.



Ответ

`map()` и `filter()` можно реализовать по-разному. У каждой функции есть два варианта реализации; все они являются вычислениями. В одном варианте используются неизменяющие операции, а в другом возвращаемый массив изменяется на каждом шаге. Изменяющая версия намного более эффективна. Однако все они остаются вычислениями, потому что изменяется только локальное значение, которое не будет изменяться после его возвращения.

```
function map(array, f) {
  return reduce(array, [], function(ret, item) {
    return ret.concat(f([item]));
  });
}

function map(array, f) {
  return reduce(array, [], function(ret, item) {
    ret.push(f(item));
    return ret;
  });
}

function filter(array, f) {
  return reduce(array, [], function(ret, item) {
    if(f(item)) return ret.concat([item]);
    else return ret;
  });
}

function filter(array, f) {
  return reduce(array, [], function(ret, item) {
    if(f(item))
      ret.push(item);
    return ret;
  });
}
```

Использование только неизменяющих операций (неэффективно)

Использование изменяющих операций (более эффективно)

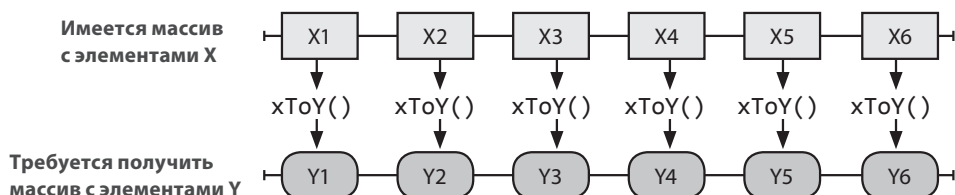
Использование только неизменяющих операций (неэффективно)

Использование изменяющих операций (более эффективно)

Важно рассмотреть обе эти реализации, потому что ранее говорилось, что передаваемая `reduce()` функция должна быть вычислением. В изменяющих операциях это правило нарушается. Тем не менее мы также ясно видим, что в этих функциях изменение происходит только в локальном контексте. `map()` и `filter()` так и остаются вычислениями. Эти примеры показывают, что правила больше похожи на рекомендации. По умолчанию они должны соблюдаться, а при их нарушении следует действовать осторожно и осознанно.

Сравнение трех инструментов функционального программирования

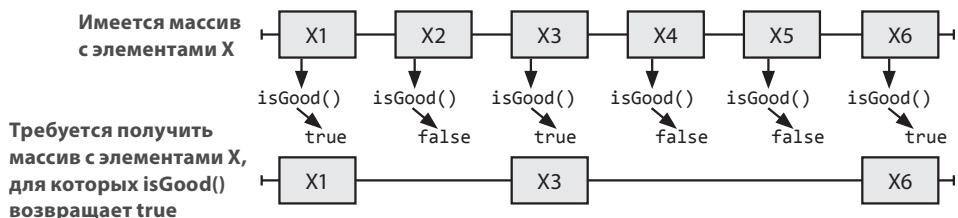
`map()` преобразует массив в новый массив, применяя функцию к каждому элементу.



```
map(array, function(element) {
  ...
  return newElement;
});
```

Получает X (arrow from element to function)
Возвращает Y (arrow from return to newElement)

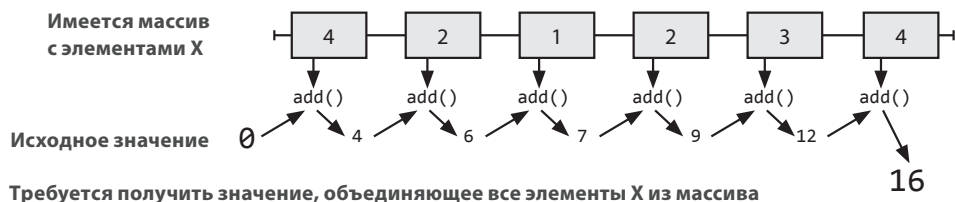
`filter()` выбирает подмножество элементов из массива в новый массив.



```
filter(array, function(element) {
  ...
  return true;
});
```

Должна возвращать либо true, либо false (arrow from return to true)

`reduce()` объединяет элементы массива в итоговое значение.



```
reduce(array, 0, function(accum, element) {
  ...
  return combine(accum, element);
});
```

Любая функция combine() на ваш выбор (arrow from combine to function)

Итоги главы

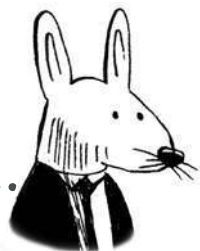
В функциональном программировании часто используются маленькие абстрактные функции, которые хорошо справляются с одной операцией. Из них чаще всего встречаются три инструмента функционального программирования, представленные в этой главе: `map()`, `filter()` и `reduce()`. Вы уже видели, что эти функции легко определяются, они чрезвычайно полезны, но при этом легко строятся на основе распространенных паттернов перебора.

Резюме

- Три самых популярных инструмента функционального программирования — `map()`, `filter()` и `reduce()`. Почти каждый функциональный программист часто пользуется ими.
- `map()`, `filter()` и `reduce()` фактически являются специализированными циклами `for` для массивов. Они могут заменять циклы `for` и лучше читаются из-за своей специализации.
- `map()` преобразует массив в новый массив. Для преобразования каждого элемента используется заданный вами обратный вызов.
- `filter()` выбирает подмножество элементов из одного массива в новый массив. Для выбора элементов передается предикат.
- `reduce()` объединяет исходное значение с элементами массива, в результате чего вычисляется одно значение. Обычно функция используется для обобщения данных или для вычисления сводного значения для серии.

Что дальше?

В этой главе был представлен ряд мощных инструментов для работы с последовательностями данных. Тем не менее остаются некоторые сложные вопросы о клиентах, на которые пока мы ответить не можем. В следующей главе вы узнаете, как объединить инструменты функционального программирования в этапы последовательного процесса. Это позволит нам выполнять еще более мощные преобразования данных.



В этой главе

- ✓ Объединение инструментов функционального программирования для построения сложных запросов к данным.
- ✓ Замена сложных циклов `for`.
- ✓ Построение конвейеров преобразования данных.

Вы узнали, как с помощью инструментов функционального программирования выполнять работу, которая обычно выполняется в циклах `for` с перебором элементов массива. Однако с усложнением вычислений один функциональный инструмент уже не может справиться с задачей. В этой главе вы узнаете, как сложные вычисления выражаются в виде последовательности шагов, на каждом из которых применяется какой-либо функциональный инструмент. Объединяя несколько инструментов, можно строить очень сложные вычисления, причем каждый шаг остается простым и понятным. Этим навыком владеет практически каждый функциональный программист. Он доказывает, насколько мощными могут стать функциональные инструменты.

Группа взаимодействия с клиентами продолжает работу

Запрос на получение данных: самые дорогие покупки лучших клиентов

Мы подозреваем, что самые верные клиенты также совершают самые дорогие покупки. Нам хотелось бы знать размер самой большой покупки для каждого из наших лучших клиентов (с тремя и более покупками).



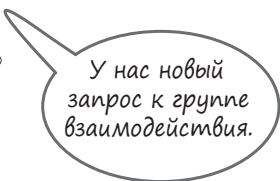
Отправитель:
директор по маркетингу



Ответственный:
Ким из команды разработки



Джон из отдела
маркетинга



Гарри из службы
поддержки



Ким из команды
разработки

Гарри: Похоже, новый запрос сложнее предыдущих.

Джон: Да. Требуется определить самую дорогую покупку, но только для лучших клиентов. Непросто будет.

Ким: Да, сложнее. Но мы знаем, как выполнить каждый шаг по отдельности. Теперь эти шаги можно объединить, чтобы построить один запрос. Такое объединение нескольких шагов называется *сцеплением* (chaining).

Джон: Понятно! Как будем действовать?

Гарри: Думаю, сначала нужно выбрать лучших клиентов, а затем определить самую дорогую покупку для каждого клиента.

Ким: Ясно: сначала `filter()`, потом `map()`. Но как выбрать самую дорогую покупку?

Джон: Мы уже выбирали наибольшее число. Нельзя ли сделать что-то похожее?

Ким: Разумно. Еще не знаю, как это будет выглядеть, но, думаю, справимся, когда привыкнем. Начнем строить запрос. Результат одного шага может стать входными данными для следующего.

Требуется найти самую дорогую покупку для каждого из лучших клиентов. Разобьем задачу на серию шагов, которые будут выполняться по порядку.

1. Провести фильтрацию (**filter()**) для отбора только лучших клиентов (с тремя и более покупками).
2. Применить отображение (**map()**) для определения самой дорогой покупки каждого клиента.

Вероятно, для выбора самых дорогих покупок будет использоваться функция **reduce()**, по аналогии с **max()** на с. 348. Начнем с определения функции.

```
function biggestPurchasesBestCustomers(customers) {
```

Начинаем с сигнатуры

Затем проведем фильтрацию для отбора только лучших клиентов. Это уже было сделано на с. 339. Операция станет первым звеном цепочки:

```
function biggestPurchasesBestCustomers(customers) {
  var bestCustomers = filter(customers, function(customer) {
    return customer.purchases.length >= 3;
  });
```

Шаг 1

Теперь необходимо получить самую дорогую покупку для каждого из этих клиентов и поместить ее в массив. Функции для этого еще нет, но мы знаем, что она будет реализована на базе **map()**. Добавим этот шаг в цепочку:

```
function biggestPurchasesBestCustomers(customers) {
  var bestCustomers = filter(customers, function(customer) {
    return customer.purchases.length >= 3;
  });
  var biggestPurchases = map(bestCustomers, function(customer) {
    return ...;
  });
}
```

Шаг 1

Шаг 2

Мы знаем, что здесь нужно использовать map(), но что должно возвращаться?

Вы уже знаете, как определить наибольшее число (с. 348). Решение легко адаптируется для определения самой дорогой покупки. В коде использовалась функция **reduce()**, поэтому на шаге 2 будет происходить нечто похожее. Реализация приведена на следующей странице.

На предыдущей странице мы сделали заготовку для шага 2. Тем не менее мы все еще не определили, как должен выглядеть обратный вызов для шага `map()`. Мы остановились в следующей точке:

```
function biggestPurchasesBestCustomers(customers) {
  var bestCustomers = filter(customers, function(customer) {
    return customer.purchases.length >= 3;
  });

  var biggestPurchases = map(bestCustomers, function(customer) {
    return ...;
  });
}
```

Шаг 1

Шаг 2

Мы знаем, что здесь нужно использовать `map()`, но что должно возвращаться?

Вы уже знаете, как определить наибольшее число (с. 348). Решение легко адаптируется для определения самой дорогой покупки. В коде использовалась функция `reduce()`, поэтому на шаге 2 будет происходить нечто похожее.

```
function biggestPurchasesBestCustomers(customers) {
  var bestCustomers = filter(customers, function(customer) {
    return customer.purchases.length >= 3;
  });

  var biggestPurchases = map(bestCustomers, function(customer) {
    return reduce(customer.purchases, {total: 0}, function(biggestSoFar,
    purchase) {
      if(biggestSoFar.total > purchase.total)
        return biggestSoFar;
      else
        return purchase;
    });
  });
  return biggestPurchases;
}
```

Шаг 1

В качестве исходного значения для `reduce()` используется пустая покупка

Шаг 2

`reduce()` используется для определения самой дорогой покупки

`reduce()` содержится в обратном вызове для `map()`, потому что мы определяем самую дорогую покупку для каждого клиента

Такое решение работает, но функция получается весьма громоздкой, с несколькими вложенными обратными вызовами. Понять ее будет сложно. Именно из-за таких функций функциональное программирование имеет дурную репутацию. Не будем останавливаться на этом: остается много возможностей для упрощения кода. Итак, за дело!

Перед вами код с предыдущей страницы. Он работает, но понять его сложно:

```
function biggestPurchasesBestCustomers(customers) {
  var bestCustomers = filter(customers, function(customer) {
    return customer.purchases.length >= 3;
  });

  var biggestPurchases = map(bestCustomers, function(customer) {
    return reduce(customer.purchases, {total: 0}, function(biggestSoFar,
purchase) {
      if(biggestSoFar.total > purchase.total)
        return biggestSoFar;
      else
        return purchase;
    });
  });
  return biggestPurchases;
}
```

*Вложенные обратные вызовы
плохо читаются*

Возьмем шаг `reduce()` и сравним его с функцией `max()`, написанной на с. 348:

Определение самой дорогой покупки

```
reduce(customer.purchases,
{total: 0},
function(biggestSoFar, purchase) {
  if(biggestSoFar.total > purchase.total)
    return biggestSoFar;
  else
    return purchase;
});
```

*Инициализируется наименьшим
возможным значением*

Определение наибольшего числа

```
reduce(numbers,
Number.MIN_VALUE,
function(m, n) {
  if(m > n)
    return m;
  else
    return n;
});
```

Сравнение

Возвращается наибольшее значение

Две функции отличаются тем, что код самой дорогой покупки должен сравнивать суммы, а обычная реализация `max()` может сравнивать числа напрямую. Выделим операцию вычисления суммы в обратный вызов.

Оригинал

```
reduce(customer.purchases,
{total: 0},
function(biggestSoFar, purchase) {
  if(biggestSoFar.total > purchase.total)
    return biggestSoFar;
  else
    return purchase;
});
```

Выделяем reduce() в maxKey()

После выделения обратного вызова

```
maxKey(customer.purchases, {total: 0},
function(purchase) { return purchase.total; }
);

function maxKey(array, init, f) {
  return reduce(array,
    init,
    function(biggestSoFar, element) {
      if(f(biggestSoFar) > f(element)) {
        return biggestSoFar;
      }
      else
        return element;
    });
}
```

*Передаем обратный
вызов, определяющий
способ сравнения
двух значений*

Мы только что создали функцию `maxKey()`, которая находит наибольшее значение в массиве. Она использует функцию, определяющую, какая часть значения должна использоваться для сравнения. Подключим ее к исходной функции.

На предыдущей странице мы дописали функцию `maxKey()` для определения наибольшего значения в массиве. Подключим ее к коду, который используется вместо `reduce()`:

```
function biggestPurchasesBestCustomers(customers) {
  var bestCustomers = filter(customers, function(customer) {
    return customer.purchases.length >= 3;
  });
  var biggestPurchases = map(bestCustomers, function(customer) {
    return maxKey(customer.purchases, {total: 0}, function(purchase) {
      return purchase.total;
    });
  });
  return biggestPurchases;
}
```

Шаг 1

Шаг 2

Вложенные return плохо читаются

Вызываем maxKey() вместо reduce()

Код получился весьма компактным. Мы выделили еще одну функцию (`maxKey()`), которая намного лучше выражает наши намерения. `reduce()` является низкоуровневой функцией, а это означает, что она универсальна. Сама по себе она не выражает ничего, кроме объединения значений в массиве. Функция `maxKey()` более конкретна. Она предназначена для выбора наибольшего значения из массива.



Ваш ход

Функции `max()` и `maxKey()` очень похожи, поэтому теоретически они должны содержать очень похожий код. Если вы планируете записать одну функцию с использованием другой, сделайте следующее.

1. Ответьте на вопросы: какая функция будет выражаться через другую функцию? Почему?
2. Напишите код обеих функций.
3. Нарисуйте граф вызовов между двумя функциями.
4. Ответьте на вопросы: какие выводы можно сделать относительно того, какая из функций более универсальна?

(Ответы на следующей странице)

И хотя код получился очень компактным, его можно сделать более понятным. А во время обучения также можно продолжать улучшение кода просто для того, чтобы увидеть, как выглядит качественное сцепление в функциональном программировании. Мы видим вложенные обратные вызовы с вложенными командами `return`. Код недостаточно хорошо объясняет, что он делает. Возможны два пути улучшения. Мы исследуем оба, а затем сравним их.



Ответ

1. Функция `max()` должна быть записана с использованием `maxKey()`, потому что `maxKey()` является более общей. `maxKey()` может найти наибольшее значение на основании произвольного сравнения, тогда как `max()` сравнивает только напрямую.
2. Чтобы записать `max()` через `maxKey()`, можно воспользоваться тождественной функцией (то есть функцией, которая возвращает свой аргумент в неизменном виде).

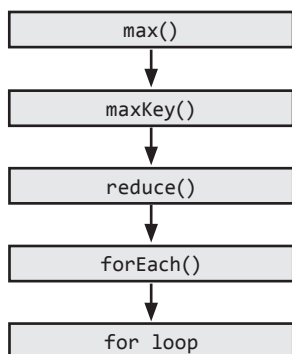
```
function maxKey(array, init, f) {
    return reduce(array,
        init,
        function(biggestSoFar, element) {
            if(f(biggestSoFar) > f(element))
                return biggestSoFar;
            else
                return element;
        });
}
```

*Приказываем `maxKey()`
сравнивать все значение*

```
function max(array, init) {
    return maxKey(array, init, function(x) {
        return x;
    });
}
```

*Функция, которая возвращает
свой аргумент в неизменном виде,
называется "тождественной"*

3. Граф вызовов выглядит так:



Загляни в словарь

Тождественная функция возвращает свой аргумент в неизменном виде. Казалось бы, она ничего не делает, однако с ее помощью можно обозначить именно этот факт: ничего делать не нужно.

4. Поскольку `maxKey()` находится ниже `max()`, функция `maxKey()` неизбежно оказывается более общей, чем `max()`. И это логично, потому что `max()` представляет собой специализированную версию `maxKey()`.

Улучшение цепочек, способ 1: присваивание имен шагам

Чтобы прояснить смысл шагов в нашей цепочке, можно присвоить каждому шагу имя. Вот как выглядел код, приведенный ранее:

```
function biggestPurchasesBestCustomers(customers) {
  var bestCustomers = filter(customers, function(customer) {
    return customer.purchases.length >= 3;
  });

  var biggestPurchases = map(bestCustomers, function(customer) {
    return maxKey(customer.purchases, {total: 0}, function(purchase) {
      return purchase.total;
    });
  });

  return biggestPurchases;
}
```

Шаг 1

Шаг 2

Если выделить каждую функцию высшего порядка и присвоить ей имя, она будет выглядеть так:

```
function biggestPurchasesBestCustomers(customers) {
  var bestCustomers = selectBestCustomers(customers);
  var biggestPurchases = getBiggestPurchases(bestCustomers);
  return biggestPurchases;
}

function selectBestCustomers(customers) {
  return filter(customers, function(customer) {
    return customer.purchases.length >= 3;
  });
}

function getBiggestPurchases(customers) {
  return map(customers, getBiggestPurchase);
}

function getBiggestPurchase(customer) {
  return maxKey(customer.purchases, {total: 0}, function(purchase) {
    return purchase.total;
  });
}
```

Шаг 1

Шаг 2

Шаги становятся более короткими и насыщенными смыслом

Функции высшего порядка вызываются из именованных функций для добавления контекста

Здесь также можно выделить функцию высшего порядка

В результате преобразования шаги функции определенно становятся более понятными. Кроме того, две функции, реализующие шаги, тоже достаточно ясны. Впрочем, мы всего лишь убираем с глаз непонятные части. Обратные вызовы все равно определяются как встроенные функции и не могут использоваться повторно. Вы уже знаете, что ключом к повторному использованию является создание небольших функций, находящихся на нижних уровнях графа вызовов. Есть ли меньшие функции, на которые можно разбить наши функции?

Ответ: да, второй способ решает эти проблемы. Он рассматривается на следующей странице.

Улучшение цепочек, способ 2: присваивание имен обратным вызовам

Другой способ прояснения цепочек основан на присваивании имен обратным вызовам. Вернемся к коду до присваивания имен шагам:

```
function biggestPurchasesBestCustomers(customers) {
  var bestCustomers = filter(customers, function(customer) {
    return customer.purchases.length >= 3;
  });

  var biggestPurchases = map(bestCustomers, function(customer) {
    return maxKey(customer.purchases, {total: 0}, function(purchase) {
      return purchase.total;
    });
  });

  return biggestPurchases;
}
```

Шаг 1

Шаг 2

На этот раз вместо выделения и присваивания имен шагам мы будем выделять и присваивать имена *обратным вызовам*:

```
function biggestPurchasesBestCustomers(customers) {
  var bestCustomers = filter(customers, isGoodCustomer);
  var biggestPurchases = map(bestCustomers, getBiggestPurchase);
  return biggestPurchases;
}

function isGoodCustomer(customer) {
  return customer.purchases.length >= 3;
}

function getBiggestPurchase(customer) {
  return maxKey(customer.purchases, {total: 0}, getPurchaseTotal);
}

function getPurchaseTotal(purchase) {
  return purchase.total;
}
```

Шаг 1

Шаг 2

Шаги остаются короткими и содержательными

Имена присваиваются обратным вызовам

Выделяя и присваивая имена обратным вызовам, мы улучшаем возможности повторного использования функции. Мы знаем, что они могут использоваться повторно, потому что под ними на графе вызовов расположено меньше блоков. Интуитивно понятно, что они должны лучше подходить для повторного использования. Например, `isGoodCustomer()` работает с одним клиентом, а `selectBestCustomers()` работает только с массивом клиентов. Функция `isGoodCustomer()` всегда применяется к массиву с помощью `filter()`.

На следующей странице приведено сравнение этих двух функций.

Улучшение цепочек: сравнение двух способов

Мы рассмотрели два способа улучшения цепочек инструментов функционального программирования. Сравним полученный код и обсудим их достоинства и недостатки:

Способ 1. Присваивание имен шагам

```
unction biggestPurchasesBestCustomers(customers) {
  var bestCustomers    = selectBestCustomers(customers);
  var biggestPurchases = getBiggestPurchases(bestCustomers);
  return biggestPurchases;
}

function selectBestCustomers(customers) {
  return filter(customers, function(customer) {
    return customer.purchases.length >= 3;
  });
}

function getBiggestPurchases(customers) {
  return map(customers, getBiggestPurchase);
}

function getBiggestPurchase(customer) {
  return maxKey(customer.purchases, {total: 0}, function(purchase) {
    return purchase.total;
  });
}
```

Способ 2. Присваивание имен обратным вызовам

```
function biggestPurchasesBestCustomers(customers) {
  var bestCustomers    = filter(customers, isGoodCustomer);
  var biggestPurchases = map(bestCustomers, getBiggestPurchase);
  return biggestPurchases;
}

function isGoodCustomer(customer) {
  return customer.purchases.length >= 3;
}

function getBiggestPurchase(customer) {
  return maxKey(customer.purchases, {total: 0}, getPurchaseTotal);
}

function getPurchaseTotal(purchase) {
  return purchase.total;
}
```

В общем случае способ 2 дает более понятный код, более пригодный для повторного использования, потому что обратные вызовы лучше подходят для этого, чем вызовы функций высшего порядка. Также устраняется лишний уровень вложенности, потому что обратные вызовы теперь обладают именами, а не определяются во встроенном виде.

Конечно, все зависит от синтаксиса и семантики используемого языка. Функциональные программисты пробуют оба способа, сравнивают результаты и принимают решение о том, какой код следует использовать.

Пример: адреса клиентов с одной покупкой

Рассмотрим простой, но типичный пример сцепления функциональных инструментов. Отдел маркетинга хочет отправить сообщение со специальным предложением для разовых клиентов.

Имеется: массив данных клиентов.

Требуется получить: адреса клиентов, оформивших только одну покупку.

План:

1. Отфильтровать клиентов с одной покупкой.
2. Получить адреса электронной почты для этих клиентов.

```
var firstTimers = filter(customers, function(customer) {
  return customer.purchases.length === 1;
});

var firstTimerEmails = map(firstTimers, function(customer) {
  return customer.email;
});
```

Определяется новая переменная, содержащая результат фильтрации

Эта переменная используется как аргумент для следующего шага

Последняя переменная содержит искомый ответ

Если вы захотите сделать функцию более компактной за счет извлечения и присваивания имен функциям обратного вызова, это будет выглядеть так:

```
var firstTimers = filter(customers, isFirstTimer);
var firstTimerEmails = map(firstTimers, getCustomerEmail);

function isFirstTimer(customer) {
  return customer.purchases.length === 1;
}

function getCustomerEmail(customer) {
  return customer.email;
}
```

Вероятно, эти функции определяются в других местах и предназначаются для повторного использования



Ваш ход

Отдел маркетинга хочет знать, кто из клиентов сделал хотя бы одну покупку на сумму свыше \$100 и более двух покупок. Ваша задача — написать функцию в виде цепочки функциональных инструментов. Проследите за тем, чтобы код был чистым и хорошо читаемым.

```
function bigSpenders(customers) {
```

Запишите здесь свой ответ



```
}
```



Ответ

```
function bigSpenders(customers) {
  var withBigPurchases = filter(customers, hasBigPurchase);
  var with2OrMorePurchases = filter(withBigPurchases, has2OrMorePurchases);
  return with2OrMorePurchases;
}

function hasBigPurchase(customer) {
  return filter(customer.purchases, isBigPurchase).length > 0;
}

function isBigPurchase(purchase) {
  return purchase.total > 100;
}

function has2OrMorePurchases(customer) {
  return customer.purchases.length >= 2;
}
```



Ваш ход

Практически любой отдел рано или поздно хочет вычислить среднее значение для массива чисел. Напишите функцию для вычисления среднего.

Подсказка: Среднее значение равно сумме чисел, разделенной на их количество.

Подсказка: Для суммирования можно воспользоваться функцией `reduce()`.

```
function average(numbers) {
```

Запишите здесь свой ответ



```
}
```



Ответ

```
function average(numbers) {  
  return reduce(numbers, 0, plus) / numbers.length;  
}
```

```
function plus(a, b) {  
  return a + b;  
}
```



Ваш ход

Требуется вычислить среднюю величину покупки для каждого клиента. Предполагается, что у вас уже имеется функция `average()`, написанная на предыдущей странице.

```
function averagePurchaseTotals(customers) {
```

Запишите здесь свой ответ



```
}
```



Ответ

```
function averagePurchaseTotals(customers) {  
  return map(customers, function(customer) {  
    var purchaseTotals = map(customer.purchases, function(purchase) {  
      return purchase.total;  
    });  
    return average(purchaseTotals);  
  });  
}
```

Да, и `filter()`, и `map()` создают новые массивы и теоретически могут добавлять в них множество элементов при каждом вызове. Это может быть неэффективно, но в большинстве случаев проблемой не является. Массивы создаются и уничтожаются сборщиком мусора очень быстро. Вы не поверите, насколько быстро работают современные сборщики мусора.

И все же иногда это действительно недостаточно эффективно. К счастью, `map()`, `filter()` и `reduce()` очень легко оптимизируются без возврата к циклам `for`. Процесс оптимизации цепочки вызовов `map()`, `filter()` и `reduce()` называется *потокowym слиянием*. Посмотрим, как он работает.

Если цепочка содержит два последовательных вызова `map()`, их можно объединить в один шаг. Пример:

Два шага `map()` подряд

```
var names      = map(customers, getFullName);
var nameLengths = map(names, stringLength);
```

Эквивалентный шаг `map()`

```
var nameLengths = map(customers, function(customer) {
  return stringLength(getFullName(customer));
});
```

Две операции
объединяются в одну

Эти два фрагмента кода выдают одинаковый ответ, но правый фрагмент делает все за один шаг `map()` без создания мусорного массива.

Нечто похожее можно сделать и с `filter()`. Два шага `filter()` подряд эквивалентны выполнению логической операции И с двумя логическими значениями.

Два шага `filter()` подряд

```
var goodCustomers = filter(customers,
                           isGoodCustomer);
var withAddresses = filter(goodCustomers,
                           hasAddress);
```

Эквивалентный шаг `filter()`

```
var withAddresses = filter(customers,
                           function(customer) {
  return isGoodCustomer(customer) && hasAddress
                                     (customer);
});
```

Используйте `&&` для объединения
двух предикатов

И снова вы получаете тот же результат с меньшим количеством мусора.

Наконец, функция `reduce()` может выполнить значительную работу сама по себе. Например, если имеется вызов `map()`, за которым следует вызов `reduce()`, можно поступить так:

Шаг `map()` с последующим шагом `reduce()`

```
var purchaseTotals = map(purchases,
                        getPurchaseTotal);
var purchaseSum    = reduce(purchaseTotals, 0, plus);
```

Эквивалентный шаг `reduce()`

```
var purchaseSum = reduce(purchases, 0,
                        function(total, purchase) {
  return total + getPurchaseTotal(purchase);
});
```

Операции выполняются внутри
обратного вызова `reduce()`

Секундочку,
но разве такое решение
не будет очень неэффективным?
Ведь при каждом вызове
`map()` или `filter()` создается
новый массив.



Дженна из команды
разработки

Исключая вызов `map()`, мы тем самым избегаем создания промежуточных массивов, которые должны уничтожаться сборщиком мусора. Еще раз подчеркну, что это оптимизация. Она повлияет на быстродействие только в том случае, если она является узким местом. В большинстве случаев лучше выполнять операции за несколько шагов, потому что тогда каждый шаг будет понятным и удобочитаемым.

Преобразование существующих циклов for в инструменты функционального программирования

До настоящего момента мы рассмотрели множество примеров написания новых цепочек вызовов функциональных инструментов по описаниям требований. Но иногда в коде встречаются уже существующие циклы `for`, которые требуется переработать. Как это сделать?

Стратегия 1. Понимание и переписывание

В первой стратегии вы просто читаете цикл `for`, разбираетесь в том, что он делает, после чего забываете его реализацию. Теперь остается сделать то, что уже делалось в примерах этой главы: вы просто заново записываете цикл в виде последовательности шагов.

Стратегия 2. Рефакторинг по признакам

Иногда смысл кода удастся понять, но в некоторых случаях это невозможно. В таком случае можно разобрать существующий цикл `for` и преобразовать его в цепочку функциональных инструментов.

Пример кода с вложенным циклом `for`:

```
var answer = [];
var window = 5;
for(var i = 0; i < array.length; i++) {
  var sum = 0;
  var count = 0;
  for(var w = 0; w < window; w++) {
    var idx = i + w;
    if(idx < array.length) {
      sum += array[idx];
      count += 1;
    }
  }
  answer.push(sum/count);
}
```

Массив (как предполагается, построенный в цикле)

Внешний цикл перебирает все элементы массива

Внутренний цикл перебирает небольшой диапазон 0–4

Вычисляем новый индекс

Накапливаем значения

Добавляем значение в массив `answer`

Даже без полного понимания того, что делает этот код, можно начать разбивать его на части. В коде встречаются многочисленные подсказки, которыми мы можем воспользоваться.

Самая сильная подсказка заключается в том, что мы добавляем в массив `answer` один элемент для каждого элемента исходного массива. Это сильный признак того, что нам понадобится `map()`. Это будет внешний цикл. Внутренний цикл напоминает `reduce()`: он что-то перебирает и объединяет элементы в один ответ.

Внутренний цикл — нормальная отправная точка, ничуть не хуже любой другой. Но что он перебирает? Далее мы ответим на этот вопрос.

Совет 1. Создавайте данные

Мы знаем, что при последовательном выполнении нескольких шагов `map()` и `filter()` часто создаются промежуточные массивы, которые немедленно уничтожаются после использования. При написании цикла перебираемые данные часто не реализуются в виде массива. Например, цикл `for` может использоваться для отсчета до 10. При каждом проходе цикла `i` будет новым числом, однако эти числа не хранятся в массиве. Эта подсказка предполагает, что данные следует поместить в массив, чтобы использовать с ними инструменты функционального программирования.

Приведу код с предыдущей страницы:

```
var answer = [];

var window = 5;

for(var i = 0; i < array.length; i++) {
  var sum = 0;
  var count = 0;
  for(var w = 0; w < window; w++) {
    var idx = i + w;
    if(idx < array.length) {
      sum += array[idx];
      count += 1;
    }
  }
  answer.push(sum/count);
}
```

*w изменяется в диапазоне от 0 до window - 1;
эти числа не хранятся в массиве*

*idx изменяется в диапазоне от i до i + window - 1;
эти числа не хранятся в массиве*

*Небольшой диапазон значений из массива
не сохраняется в отдельном массиве*

Внутренний цикл перебирает небольшой диапазон элементов массива. Спросите себя: а если бы эти данные находились в отдельном массиве и нам пришлось перебирать этот массив?

Рекомендации по рефакторингу

1. Создавайте данные.
2. Работайте с целыми массивами.
3. Используйте много мелких шагов.

Задача легко решается методом `.slice()` того же массива. `.slice()` создает массив элементов для подпоследовательности массива. Проведем соответствующую замену:

```
var answer = [];
```

```
var window = 5;
```

```
for(var i = 0; i < array.length; i++) {
  var sum = 0;
  var count = 0;
  var subarray = array.slice(i, i + window);
  for(var w = 0; w < subarray.length; w++) {
    sum += subarray[w];
    count += 1;
  }
  answer.push(sum/count);
}
```

Поместить подмножество в отдельный массив

После этого можно перебрать его в стандартном цикле for

Совет 2. Работайте с целыми массивами

После построения подмассива мы перебираем весь массив. Это означает, что мы можем использовать с ним `map()`, `filter()` или `reduce()`, потому что эти инструменты работают с целыми массивами. Рассмотрим код, приведенный выше, и попробуем заменить этот цикл.

```
var answer = [];
```

```
var window = 5;
```

```
for(var i = 0; i < array.length; i++) {
  var sum = 0;
  var count = 0;
  var subarray = array.slice(i, i + window);
  for(var w = 0; w < subarray.length; w++) {
    sum += subarray[w];
    count += 1;
  }
  answer.push(sum/count);
}
```

Стандартный цикл for по подмассиву

Объединение значений из подмассива в sum и count

Среднее вычисляется разделением двух значений

Рекомендации по рефакторингу

1. Создавайте данные.
2. Работайте с целыми массивами.
3. Используйте много мелких шагов.

В данном случае код объединяет элементы из подмассива в переменные `sum` и `count`, после чего одна переменная делится на другую. Мы знаем, как это делается, так как задача уже решалась на с. 366. По сути, это просто `reduce()`. Можно либо записать код здесь, либо вызвать уже готовую функцию `average()`.

Выберем второй вариант: в конце концов, мы писали функцию для того, чтобы использовать ее повторно.

```
var answer = [];
var window = 5;
for(var i = 0; i < array.length; i++) {
  var subarray = array.slice(i, i + window);
  answer.push(average(subarray));
}
```

Этот цикл for не использует элемент массива; вместо этого в теле используется индекс цикла

Мы полностью заменили внутренний цикл .slice() и вызовом average()

Выглядит неплохо! Остался один цикл `for`. Он перебирает весь массив, что наводит на мысль об использовании `map()`. Однако этот цикл `for` не использует текущий элемент массива, так что заменить его напрямую не удастся. Напомним, что обратный вызов `map()` получает только текущий элемент. Вместо этого для выделения подмассива используется индекс `i` цикла `for`. Прямая замена `map()` невозможна. Решение существует, и далее мы его рассмотрим.



Пища для ума

Мы использовали цикл `for`, теперь его нет. Куда он делся?

Совет 3. Используйте много мелких шагов

В исходном варианте кода происходило слишком много всего. Мы сократили его до чего-то более управляемого. Тем не менее мы подозреваем, что здесь присутствует скрытый вызов `map()`. Мы перебираем весь массив и генерируем новый массив с тем же количеством элементов. Снова приведу код:

```
var answer = [];
var window = 5;
for(var i = 0; i < array.length; i++) {
  var subarray = array.slice(i, i + window);
  answer.push(average(subarray));
}
```

Рекомендации по рефакторингу

1. Создавайте данные.
2. Работайте с целыми массивами.
3. **Используйте много мелких шагов.**

Индекс цикла используется для создания подмассива

Проблема в том, что мы хотим внести перебор по индексам, а не по значениям массива. Индексы позволяют сегментировать исходный массив с созданием подмассивов или «окон». Перебрать все индексы за один уже имеющийся

шаг может быть трудно (или невозможно), поэтому мы сделаем это за много мелких шагов. Прежде всего, так как нам понадобятся индексы, почему бы не сгенерировать их в виде массива (совет 1)? Тогда мы сможем оперировать со всем массивом индексов с помощью функциональных инструментов (шаг 2):

```
var indices = [];
```

```
for(var i = 0; i < array.length; i++)  
  indices.push(i);
```

Мелкий шаг с созданием индексов

Обратите внимание: мы добавили новый шаг! Теперь цикл `for` можно преобразовать в `map()` по этим индексам:

```
var indices = [];
```

```
for(var i = 0; i < array.length; i++)  
  indices.push(i);
```

map с массивом индексов

```
var window = 5;
```

```
var answer = map(indices, function(i) {  
  var subarray = array.slice(i, i + window);  
  return average(subarray);  
});
```

*Обратному вызову
будет поочередно
передаваться каждый
индекс*

В новом шаге (генерирование массива чисел) заменим цикл `for` вызовом `map()`. Теперь внутри обратного вызова для `map()` выполняются две операции. Нельзя ли его разделить?

Совет 3. Делайте много мелких шагов

Еще раз посмотрим на код:

```
var indices = [];  
for(var i = 0; i < array.length; i++)  
  indices.push(i);
```

```
var window = 5;
```

```
var answer = map(indices, function(i) {  
  var subarray = array.slice(i, i + window);  
  return average(subarray);  
});
```

Рекомендации по рефакторингу

1. Создавайте данные.
2. Работайте с целыми массивами.
3. **Используйте много мелких шагов.**

*Выполняются две операции:
создание подмассива
и вычисление среднего*

В обратном вызове `map()` выполняются две операции. Мы создаем подмассив и вычисляем среднее значение. Очевидно, его можно было бы разделить на два шага:

```
var indices = [];
for(var i = 0; i < array.length; i++)
    indices.push(i);

var window = 5;

var windows = map(indices, function(i) {
    return array.slice(i, i + window);
});

var answer = map(windows, average);
```

Шаг 1, создание подмассива

Шаг 2, вычисление среднего

Последнее, что осталось сделать, — выделить цикл, который генерирует индексы, во вспомогательную функцию. Она наверняка пригодится нам позднее.

```
function range(start, end) {
    var ret = [];
    for(var i = start; i < end; i++)
        ret.push(i);
    return ret;
}

var window = 5;

var indices = range(0, array.length);
var windows = map(indices, function(i) {
    return array.slice(i, i + window);
});

var answer = map(windows, average);
```

Функция range() еще пригодится

Сгенерировать индексы с использованием range()

Шаг 1, создание индексов

Шаг 2, создание подмассива

Шаг 3, вычисление среднего

Все циклы `for` были заменены цепочкой инструментов функционального программирования.

Сравнение функционального кода с императивным

Готово! Посмотрим, чего мы добились:

Оригинал: императивный код

```
var answer = [];

var window = 5;

for(var i = 0; i < array.length; i++) {
  var sum = 0;
  var count = 0;
  for(var w = 0; w < window; w++) {
    var idx = i + w;
    if(idx < array.length) {
      sum += array[idx];
      count += 1;
    }
  }
  answer.push(sum/count);
}
```

Код, использующий функциональные инструменты

```
var window = 5;

var indices = range(0, array.length);
var windows = map(indices, function(i) {
  return array.slice(i, i + window);
});
var answer = map(windows, average);
```

Плюс функция для повторного использования

```
function range(start, end) {
  var ret = [];
  for(var i = start; i < end; i++)
    ret.push(i);
  return ret;
}
```

Мы начали с кода с вложенными циклами, вычислениями с индексами и локальными изменяемыми переменными. В итоге получился трехшаговый процесс, в котором каждый шаг был простым и понятным. На самом деле шаги можно записать на естественном языке.

Скольльзящее среднее

1. Для заданного списка чисел генерируется «окно» вокруг каждого числа.
2. Вычисляется среднее значение для каждого окна.

Шаги в коде близко соответствуют шагам в описании алгоритма. Кроме того, функциональная версия породила вспомогательную функцию `range()`. Функциональные программисты постоянно используют эту функцию.



Пицца для ума

В каком месте графа вызовов должна находиться функция `range()`? Указывает ли эта позиция на ее простоту:

- 1) повторного использования;
- 2) тестирования;
- 3) сопровождения?

Советы по сцеплению

На нескольких последних страницах были приведены три совета по рефакторингу циклов `for` в цепочки трех инструментов функционального программирования. Ниже снова приведены эти советы, а для полноты картины к ним добавлено еще несколько советов.

Создавайте данные

Функциональные инструменты лучше всего работают с целыми массивами данных. Если вам попадется цикл `for`, работающий с подмножеством данных, попробуйте выделить данные в отдельный массив. Тогда `map()`, `filter()` и `reduce()` легко справятся с ним.

Работайте с целыми массивами

Спросите себя: «Как мне обработать весь массив как единое целое вместо итеративной обработки в цикле `for`?» `map()` преобразует каждый элемент. `filter()` оставляет или удаляет каждый элемент. `reduce()` выполняет свертку, то есть объединяет все элементы в одно значение. Мыслите масштабно и обработайте весь массив.

Используйте множество мелких шагов

Когда возникает ощущение, что алгоритм делает слишком много всего, попробуйте разбить его на два или более шага. Если количество шагов увеличится, будет ли проще обдумывать происходящее? Да! Потому что каждый шаг заметен и упрощается. Спросите себя, какой мелкий шаг приблизит вас к цели.

Дополнительно: заменяйте условия вызовом `filter()`

Условия, встроенные в циклы `for`, часто пропускают элементы массива. Почему бы не отфильтровать их заранее вызовом `filter()`?

Дополнительно: извлекайте вспомогательные функции

`map()`, `filter()` и `reduce()` — не все, а только самые распространенные инструменты функционального программирования. Кроме них существуют и другие; вполне возможно, что вы найдете многие самостоятельно. Определите их, присвойте подходящее имя и пользуйтесь!

Дополнительно: экспериментируйте, чтобы совершенствоваться

Некоторые люди быстро находят элегантные и понятные способы решения задач с использованием функциональных инструментов. Как у них это получается? Они делали попытку за попыткой. Они практиковались. Они старались найти новые комбинации этих инструментов.



Ваш ход

Ниже приведен пример кода из кодовой базы MegaMart. Ваша задача — преобразовать его в цепочку функциональных инструментов. Учтите, что это можно сделать несколькими способами.

```
function shoesAndSocksInventory(products) {
  var inventory = 0;
  for(var p = 0; p < products.length; p++) {
    var product = products[p];
    if(product.type === "shoes" || product.type === "socks") {
      inventory += product.numberInInventory;
    }
  }
  return inventory;
}
```

Запишите здесь
свой ответ




Ответ

```
function shoesAndSocksInventory(products) {
  var shoesAndSocks = filter(products, function(product) {
    return product.type === "shoes" || product.type === "socks";
  });
  var inventories = map(shoesAndSocks, function(product) {
    return product.numberInInventory;
  });
  return reduce(inventories, 0, plus);
}
```

Советы по отладке

Работа с функциями высшего порядка может быть очень абстрактной, и вам может быть трудно понять, когда что-то идет не так. Ниже приведены некоторые рекомендации.

Действуйте конкретно

Легко забыть, как выглядят ваши данные, особенно на промежуточных этапах конвейера. Обязательно включайте в код понятные имена: они помогут вам следить за тем, что есть что. Имена переменных вроде `x` или `a` уменьшают размер кода, но они несодержательны. Используйте имена с пользой.

Включайте команды вывода

Даже опытные функциональные программисты могут запутаться в том, какие данные используются в той или иной точке программы. Как они поступают? Они вставляют команду вывода между двумя шагами, а потом выполняют код. Такие проверки помогут убедиться в том, что каждый шаг работает так, как ожидалось. Профессиональный совет: в очень сложных цепочках добавляйте по одному шагу и проверяйте результат перед тем, как переходить к следующему шагу.

Следите за типами

У каждого функционального инструмента есть вполне конкретный тип. Да, даже если вы работаете на нетипизованном языке (таком, как JavaScript), функциональные инструменты все равно обладают типом. Просто этот тип не проверяется компилятором. Обратите этот факт себе на пользу и мысленно следите за типами значений, переходящих по цепочке.

Например, вы знаете, что `map()` возвращает новый массив. Что он содержит? Тип, возвращаемый обратным вызовом.

А как насчет `filter()`? Результирующий массив относится к тому же типу, что и аргумент.

А `reduce()`? Тип результата совпадает с типом возвращаемого значения обратного вызова, который совпадает с типом исходного значения.

С учетом сказанного вы можете мысленно пройти каждый шаг и определить тип, который генерируется на каждом шаге. Это поможет вам понять код и устранить возможные проблемы.

Другие функциональные инструменты

Существует много других функциональных инструментов, которые часто применяются программистами. `map()`, `filter()` и `reduce()` просто используются чаще других. Вы узнаете, что в стандартных библиотеках функциональных языков полно таких инструментов. Не жалейте времени и просмотрите документацию: вы найдете в ней немало полезного. Далее рассмотрим несколько примеров.

pluck()

Вам надоело писать обратные вызовы для `map()`, которые просто извлекают одно поле? Тогда вам поможет функция `pluck()`:

```
function pluck(array, field) {
  return map(array, function(object) {
    return object[field];
  });
}
```

Использование

```
var prices = pluck(products, 'price');
```

Вариация

```
function invokeMap(array, method) {
  return map(array, function(object) {
    return object[method]();
  });
}
```

concat()

`concat()` извлекает элементы массивов, вложенных в массив. Таким образом устраняется лишний уровень вложенности:

```
function concat(arrays) {
  var ret = [];
  forEach(arrays, function(array) {
    forEach(array, function(element) {
      ret.push(element);
    });
  });
  return ret;
}
```

Использование

```
var purchaseArrays = pluck(customers,
  "purchases");
var allPurchases = concat(purchaseArrays);
```

Вариация

```
function concatMap(array, f) {
  return concat(map(array, f));
}
```

В других языках может называться `mapcat()` или `flatMap()`

frequenciesBy() и groupBy()

Подсчет и группировка принадлежат к числу самых полезных операций. Эти функции возвращают объекты (хеш-карты):

```
function frequenciesBy(array, f) {
  var ret = {};
  forEach(array, function(element) {
    var key = f(element);
    if(ret[key]) ret[key] += 1;
    else ret[key] = 1;
  });
  return ret;
}
```

```
function groupBy(array, f) {
  var ret = {};
  forEach(array, function(element) {
    var key = f(element);
    if(ret[key]) ret[key].
      push(element);
    else ret[key] = [element];
  });
  return ret;
}
```

Использование

```
var howMany = frequenciesBy(products, function(p) {
  return p.type;
});
> console.log(howMany['ties'])
4
```

```
var groups = groupBy(range(0, 10), isEven);
> console.log(groups)
{
  true: [0, 2, 4, 6, 8],
  false: [1, 3, 5, 7, 9]
}
```

Где искать функциональные инструменты

Функциональные программисты знают, что расширение инструментария умножает их эффективность. Например, Clojure-программист знает функции, которые помогут ему в решении задачи на JavaScript.

Поскольку эти функции легко пишутся, программист сначала пишет их на JavaScript, а затем реализует решение на их основе. Просмотрите следующие сборники исходного кода и заимствуйте лучшие инструменты из других языков.

Lodash: функциональные инструменты для JavaScript

Lodash — библиотека JavaScript, которую нередко называют «пропавшей стандартной библиотекой JavaScript». Она полна абстрактных операций с данными. Каждая из них достаточно проста для реализации в нескольких строках. Вы найдете здесь немало источников вдохновения!

- Документация Lodash (<https://lodash.com/docs>)

Коллекции Laravel • Функциональные инструменты для PHP

Laravel содержит замечательные функциональные инструменты для работы со встроенными массивами PHP. Многие разработчики безгранично доверяют им. Если вы хотите увидеть пример функциональных инструментов, в которых встроенные коллекции реализованы лучше оригинала, присмотритесь к Laravel.

- Документация коллекций Laravel (<https://laravel.com/docs/collections#available-methods>).

Стандартная библиотека Clojure

Стандартная библиотека Clojure содержит множество функциональных инструментов. Собственно, изобилие становится главной проблемой. Официальная документация представляет собой неструктурированный алфавитный список, но я рекомендую ClojureDocs со страницей, содержащей организованный список.

- Краткая документация ClojureDocs (<https://clojuredocs.org/quickref#sequences>).
- Официальная документация (<https://clojure.github.io/clojure/clojure.core-api.html>).

Haskell Prelude

Чтобы получить представление о том, какими короткими и компактными могут быть функциональные инструменты, обратитесь к Haskell Prelude. Хотя этот модуль не настолько полон, как другие языки, он содержит множество замечательных решений. И если вы умеете читать сигнатуры типов, то получите четкое представление о том, как они работают. Модуль включает сигнатуры типов, реализации, качественное объяснение и пару примеров для каждой функции.

- Haskell Prelude (<https://hackage.haskell.org/package/base-4.16.0.0/docs/Prelude.html>).

Вспомогательные средства JavaScript

Стоит еще раз подчеркнуть: несмотря на то что в примерах книги используется JavaScript, эта книга не посвящена функциональному программированию на JavaScript. Примеры, которые вы читали, несколько сложнее тех, которые встречаются в типичной кодовой базе JavaScript.

Почему они сложнее? Потому что в JavaScript работать с `map()`, `filter()` и `reduce()` намного удобнее, чем в нашем тексте. Во-первых, функции встроены в язык — вам не придется писать их самостоятельно. Во-вторых, они являются методами массивов, что упрощает их вызов.

Реализация в книге

```
var customerNames = map(customers, function(c) {
  return c.firstName + " " + c.lastName;
});
```

Встроенные средства JavaScript

```
var customerNames = customers.map(function(c) {
  return c.firstName + " " + c.lastName;
});
```

В JavaScript класс массива содержит метод `.map()`

Так как они являются методами массивов, это означает, что вы можете использовать сцепление методов вместо присваивания промежуточным переменным. Таким образом, код скользящего среднего на с. 375 можно записать иначе. Некоторые предпочитают эту форму записи:

Реализация в книге

```
var window = 5;

var indices = range(0, array.length);
var windows = map(indices, function(i) {
  return array.slice(i, i + window);
});
var answer = map(windows, average);
```

Со сцеплением методов

```
var window = 5;
var answer =
  range(0, array.length)
    .map(function(i) {
      return array.slice(i, i + window);
    })
    .map(average);
```

Точки можно выровнять по вертикали

В JavaScript реализован модный синтаксис для определения встроенных функций. С ним использование `map()`, `filter()` и `reduce()` становится более коротким и удобным. Например, приведенный выше код становится еще более компактным:

```
var window = 5;

var answer =
  range(0, array.length)
    .map(i => array.slice(i, i + window))
    .map(average);
```

Со стрелочным синтаксисом => обратные вызовы становятся короче и понятнее

Наконец, в JavaScript `map()` и `filter()` также передается индекс элемента вместо одного искомого элемента. Поверите ли вы, что скользящее среднее вычисляется в одну строку? Мы даже можем добавить `average()` как еще одно однострочное определение:

```
var window = 5;
var average = array => array.reduce((sum, e) => sum + e, 0) / array.length;
var answer = array.map((e, i) => array.slice(i, i + window)).map(average);
```

Текущий индекс передается во втором аргументе после текущего элемента

Если вы работаете на JavaScript, более подходящего момента для функционального программирования еще не было.

Потоки данных Java

В Java 8 появились новые средства, упрощающие функциональное программирование. Нововведений было достаточно много, и мы не можем описать их все. Остановимся на трех новых возможностях, относящихся к инструментарию функционального программирования.

Лямбда-выражения

Лямбда-выражения предназначены для записи того, что выглядит как встроенные функции (на самом деле они преобразуются компилятором в анонимные классы). Но независимо от того, как они реализуются, у них есть много достоинств: они являются *замыканиями* (то есть ссылаются на переменные в области видимости) и с ними возможно многое из того, что мы делали в этой главе.

Функциональные интерфейсы

Интерфейсы Java с одним методом называются *функциональными интерфейсами*. Экземпляр любого функционального интерфейса может быть создан лямбда-выражением. Что еще важнее, в Java 8 появился набор заранее определенных функциональных интерфейсов, полностью обобщенных, которые фактически реализуют типизованный функциональный язык. Существуют четыре группы функциональных интерфейсов, о которых стоит упомянуть, потому что они соответствуют типам обратных вызовов трех функциональных инструментов из последней главы и `forEach()`:

- **Функция:** функция с одним аргументом, которая возвращает значение, — идеально подходит для передачи `map()`.
- **Предикат:** функция с одним аргументом, которая возвращает `true` или `false`, — идеально подходит для передачи `filter()`.
- **Бифункция:** функция с двумя аргументами, которая возвращает значение, — идеально подходит для передачи `reduce()`, если тип первого аргумента соответствует возвращаемому типу.
- **Потребитель:** функция с одним аргументом, которая не возвращает значение — идеально подходит для передачи `forEach()`.

Stream API

Stream API — реакция Java на потребность в функциональных инструментах. Потоки данных (streams) строятся на основе источников данных (таких, как массивы или коллекции) и содержат многочисленные методы для их обработки функциональными инструментами, включая `map()`, `filter()`, `reduce()` и многие другие. У потоков данных много достоинств: они не изменяют свои источники данных, хорошо подходят для сцепления и эффективно работают.

reduce() для построения значений

До настоящего момента вы видели много примеров использования `reduce()` для сводных вычислений с данными. Берется коллекция данных, а все ее элементы объединяются в одно значение. Например, вы видели, как реализовать суммирование или вычисление среднего в одно значение. И хотя это важное применение, возможности `reduce()` не ограничиваются простым обобщением.

Функция `reduce()` также может использоваться для построения значений. Один из возможных сценариев: допустим, корзина пользователя была потеряна. К счастью, мы сохранили все товары, добавленные пользователем в корзину, в массиве. Массив выглядит так:

```
var itemsAdded = ["shirt", "shoes", "shirt", "socks", "hat", ....];
```

*Массив всех товаров, добавленных
пользователем в корзину*

Если вам известна эта информация, можно ли построить текущее состояние корзины? И помните о необходимости отслеживать дубликаты увеличением поля количества.

Ситуация идеально подходит для `reduce()`. Функция перебирает массив и объединяет все элементы в одно значение. В данном случае таким значением является объект корзины.

Построим этот код шаг за шагом. Начнем с вызова `reduce()`. Мы знаем, что первый аргумент содержит список товаров. Во втором аргументе передается исходное значение. Корзина изначально пуста; этот объект используется для представления корзины, поэтому мы передаем пустой объект.

Также известна сигнатура передаваемой функции. Функция должна возвращать корзину, это тип первого аргумента. А массив содержит названия товаров, это второй аргумент:

```
var shoppingCart = reduce(itemsAdded, {}, function(cart, item) {
```

Остается заполнить код функции. Что она должна делать?

Рассмотрим два случая. Простой случай — когда такой товар в корзине отсутствует:

```
var shoppingCart = reduce(itemsAdded, {}, function(cart, item) {
  if(!cart[item])
    return add_item(cart, {name: item, quantity: 1, price: priceLookup(item)});
```

*Предполагается, что
цену можно опреде-
лить по названию*

Второй, более сложный случай — когда товар уже находится в корзине. Обрабатываем его, и задачу можно считать решенной!

```
var shoppingCart = reduce(itemsAdded, {}, function(cart, item) {
  if(!cart[item])
    return add_item(cart, {name: item, quantity: 1, price: priceLookup(item)});
  else {
    var quantity = cart[item].quantity;
    return setFieldByName(cart, item, 'quantity', quantity + 1);
  }
});
```

*Увеличиваем количество
единиц товара*

Готово. Обсудим получившийся код на следующей странице.

Мы только что воспользовались `reduce()` для построения корзины по списку товаров, добавленных пользователем. На предыдущей странице мы остановились на следующем коде:

```
var shoppingCart = reduce(itemsAdded, {}, function(cart, item) {
  if(!cart[item])
    return add_item(cart, {name: item, quantity: 1, price: priceLookup(item)});
  else {
    var quantity = cart[item].quantity;
    return setFieldByName(cart, item, 'quantity', quantity + 1);
  }
});
```

Функция, передаваемая `reduce()`, очень полезна. Возможно, ее стоило бы поднять к абстрактному барьеру корзины, сделав ее частью API. Функция получает корзину и товар и возвращает новую корзину с добавлением этого товара, включая обработку случая, когда такой товар уже присутствует в корзине.

```
var shoppingCart = reduce(itemsAdded, {}, addOne);

function addOne(cart, item) {
  if(!cart[item])
    return add_item(cart, {name: item, quantity: 1, price: priceLookup(item)});
  else {
    var quantity = cart[item].quantity;
    return setFieldByName(cart, item, 'quantity', quantity + 1);
  }
}
```

Просто выделяем обратный вызов и присваиваем ему имя

Чрезвычайно полезная функция

Давайте подробнее проанализируем этот код. Он означает, что мы можем построить корзину в любой момент: для этого достаточно просто сохранять информацию о товарах, добавляемых пользователем. Постоянно поддерживать объект корзины не обязательно. Его можно в любой момент восстановить по сохраненным данным.

Этот прием играет важную роль в функциональном программировании. Представьте, что вы сохраняете товары, добавленные пользователем в корзину, в массиве. Как реализовать отмену? Достаточно извлечь последний товар из массива. Впрочем, в книге эта тема подробно рассматриваться не будет.

А пока проанализируем пример более подробно, чтобы подчеркнуть некоторые полезные моменты.

Одной из операций, которая не поддерживалась этим примером, было удаление товаров. Как реализовать добавление и удаление? На следующей странице вы это узнаете.



Подсказка

Подготовьте вызов функционального инструмента, прежде чем заполнять тело функции обратного вызова.

Творческий подход к представлению данных

На предыдущей странице мы использовали `reduce()` для построения корзины по списку добавленных товаров. Код выглядит так:

```
var itemsAdded = ["shirt", "shoes", "shirt", "socks", "hat", ....];
var shoppingCart = reduce(itemsAdded, {}, addOne);

function addOne(cart, item) {
  if(!cart[item])
    return add_item(cart, {name: item, quantity: 1, price: priceLookup(item)});
  else {
    var quantity = cart[item].quantity;
    return setFieldByName(cart, item, 'quantity', quantity + 1);
  }
}
```

Отлично, но мы также хотим предоставить пользователю возможность удаления товаров. А если вместо простого сохранения названий товаров мы также будем сохранять информацию о том, были ли они добавлены или удалены?

```
var itemOps = [['add', "shirt"], ['add', "shoes"], ['remove', "shirt"],
               ['add', "socks"], ['remove', "hat"], ....];
```

Обратите внимание на 'remove'

Каждая пара состоит из операции и названия товара

Уже можно обработать два случая — добавление ('add') и удаление ('remove'):

```
var shoppingCart = reduce(itemOps, {}, function(cart, itemOp) {
  var op = itemOp[0];
  var item = itemOp[1];
  if(op === 'add') return addOne(cart, item);
  if(op === 'remove') return removeOne(cart, item);
});

function removeOne(cart, item) {
  if(!cart[item])
    return cart;
  else {
    var quantity = cart[item].quantity;
    if(quantity === 1)
      return remove_item_by_name(cart, item);
    else
      return setFieldByName(cart, item, 'quantity', quantity - 1);
  }
}
```

Выбираем ветвь в зависимости от операции, после чего вызываем соответствующую функцию

Если товар отсутствует в корзине, ничего не делать

Если количество равно 1, товар удаляется

В противном случае количество уменьшается на 1

Корзину можно восстановить по списку операций добавления и удаления, сохраненных на основании действий пользователя. Мы только что использовали важный прием: *дополнение данных*. Операция представляется в виде блока данных: в данном случае массива с именем операции и ее «аргумента». Это распространенный прием функционального программирования, который помогает строить более качественные цепочки вызовов функциональных инструментов. При использовании сцепления стоит подумать, не пригодится ли дополнение возвращаемых данных на более позднем шаге цепочки.



Ваш ход

Приближается ежегодный турнир по софтболу среди персонала интернет-магазинов, и компания MegaMart намерена отправить команду для защиты титула. Необходимо построить реестр игроков (то есть выбрать, кто будет играть и в какой позиции). Каждого работника оценивает профессиональный тренер, который рекомендует позицию на поле и выдает его оценку как игрока.

У вас имеется список таких рекомендаций. Они уже отсортированы по оценкам, начиная с лучших оценок:

```
var evaluations = [{name: "Jane", position: "catcher", score: 25},
                  {name: "John", position: "pitcher", score: 10},
                  {name: "Harry", position: "pitcher", score: 3},
                  ...];
```

Реестр должен выглядеть так:

```
var roster = {"pitcher": "John",
              "catcher": "Jane",
              "first base": "Ellen",
              ...};
```

Ваша задача — написать код построения реестра для заданного набора оценок.

*Запишите здесь
свой ответ*



Ответ

```
var roster = reduce(evaluations, {}, function(roster, eval) {
  var position = eval.position;
  if(roster[position]) // Позиция уже заполнена
    return roster;     // Не делать ничего
  return objectSet(roster, position, eval.name);
});
```




Ваш ход

Приближается ежегодный турнир по софтболу среди персонала интернет-магазинов, и компания MegaMart намерена отправить команду для защиты титула. Необходимо оценить работников и понять, для какой позиции они лучше подходят. Хорошо бросает? Подающий! Хорошо ловит? Принимающий! У вас имеется функция `recommendPosition()`, предоставленная профессиональным тренером; она берет работника, проводит набор тестов и возвращает рекомендацию.

Пример:

```
> recommendPosition("Jane")
"catcher"
```

Вы получаете список имен работников. Ваша задача — расширить список в запись, содержащую имя работника и рекомендуемую позицию. Это выглядит примерно так:

```
{
  name: "Jane",
  position: "catcher"
}
```

Возьмите список имен работников и преобразуйте его в список записей с рекомендациями, используя функцию `recommendPosition()`.

```
var employeeNames = ["John", "Harry", "Jane", ...];
```

```
var recommendations
```

*Запишите здесь
свой ответ*



Ответ

```
var recommendations = map(employeeNames, function(name) {
  return {
    name: name,
    position: recommendPosition(name)
  };
});
```



Ваш ход

Приближается ежегодный турнир по софтболу среди персонала интернет-магазинов, и компания MegaMart намерена отправить команду для защиты титула. Необходимо обеспечить максимальную вероятность выигрыша, поэтому мы должны определить, какие игроки лучше подходят для рекомендуемых позиций. К счастью, профессиональный тренер, нанятый фирмой, предоставил функцию `scorePlayer()`. Эта функция получает имя работника и рекомендуемую позицию и возвращает числовую оценку. Чем выше оценка, тем лучше игрок.

```
> scorePlayer("Jane", "catcher")
25
```

Вы получаете список записей с рекомендациями. Ваша задача — дополнить записи оценкой, полученной от `scorePlayer()`. Результат должен выглядеть примерно так:

```
{
  name: "Jane",
  position: "catcher",
  score: 25
}
```

Возьмите список записей и дополните его:

```
var recommendations = [{name: "Jane", position: "catcher"},
                        {name: "John", position: "pitcher"},
                        ...];
```

```
var evaluations =
```

*Запишите здесь
свой ответ*




Ответ

```
var evaluations = map(recommendations, function(rec) {
  return objectSet(rec, 'score', scorePlayer(rec.name, rec.position));
});v
```



Ваш ход

Приближается ежегодный турнир по софтболу среди персонала интернет-магазинов, и компания MegaMart намерена отправить команду для защиты титула. В трех предыдущих упражнениях мы решили три задачи. Пора объединить их! Ваша задача — пройти весь путь от списка имен работников до реестра в одной цепочке.

Кроме ответов от трех предыдущих упражнений, вам также понадобятся следующие функции:

- `sortBy(array, f)` — возвращает копию массива с элементами, отсортированными в соответствии с возвращаемыми значениями `f` (используйте для сортировки по оценке).
- `reverse(array)` — возвращает копию массива с элементами, следующими в обратном порядке.

Поторопитесь! Турнир состоится на этих выходных!

```
var employeeNames = ["John", "Harry", "Jane", ...];
```

*Запишите здесь
свой ответ*





Ответ

```
var recommendations = map(employeeNames, function(name) {
  return {
    name: name,
    position: recommendPosition(name)
  };
});

var evaluations = map(recommendations, function(rec) {
  return objectSet(rec, 'score', scorePlayer(rec.name, rec.position));
});

var evaluationsAscending = sortBy(evaluations, function(eval) {
  return eval.score;
});

var evaluationsDescending = reverse(evaluationsAscending);

var roster = reduce(evaluations, {}, function(roster, eval) {
  var position = eval.position;
  if(roster[position]) // Позиция уже заполнена
    return roster;    // Не делать ничего
  return objectSet(roster, position, eval.name);
});
```

О выравнивании точек

Сцепление функциональных инструментов удовлетворяет страсть программиста к четкому форматированию. В аккуратном столбце точек есть нечто радующее глаз. Тем не менее это делается не для красоты.

Длинная линия точек означает, что вы хорошо используете функциональные инструменты. Чем длиннее линия, тем больше шагов вы используете и тем больше ваш код напоминает конвейер, на который сверху поступают входные данные, а снизу выходят результаты.

Просто для развлечения приведу несколько примеров того, как могут выглядеть такие линии. Мы воспользуемся примером со скользящим средним из этой главы.

ES6

```
function movingAverage(numbers) {
  return numbers
    .map((_e, i) => numbers.slice(i, i + window))
    .map(average);
}
```

Классический JavaScript с Lodash

```
function movingAverage(numbers) {
    return _.chain(numbers)
        .map(function(_e, i) { return numbers.slice(i, i + window); })
        .map(average)
        .value();
}
```

Потоки данных в Java 8

```
public static double average(List<Double> numbers) {
    return numbers
        .stream()
        .reduce(0.0, Double::sum) / numbers.size();
}

public static List<Double> movingAverage(List<Double> numbers) {
    return IntStream
        .range(0, numbers.size())
        .mapToObj(i -> numbers.subList(i, Math.min(i + 3, numbers.size())))
        .map(Utils::average)
        .collect(Collectors.toList());
}
```

C#

```
public static IEnumerable<Double> movingAverage(IEnumerable<Double> numbers) {
    return Enumerable
        .Range(0, numbers.Count())
        .Select(i => numbers.ToList().GetRange(i, Math.Min(3,
            numbers.Count() - i)))
        .Select(l => l.Average());
}
```

Итоги главы

В этой главе вы научились комбинировать функциональные инструменты, представленные в предыдущей главе. Для объединения используются многошаговые процессы, называемые *цепочками*. Каждый шаг цепочки представляет собой простую операцию, которая преобразует данные для приближения к желаемому результату. Вы также узнали, как провести рефакторинг существующих циклов `for` в цепочки функциональных инструментов. Наконец, вы увидели, насколько мощно может работать функция `reduce()`. Функциональные программисты очень часто применяют все эти приемы. Они закладывают основу менталитета, воспринимающего вычисления как преобразование данных.

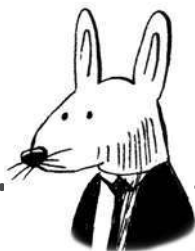
Резюме

- Функциональные инструменты можно объединять в цепочки. Комбинирование позволяет выражать очень сложные вычисления с данными в виде маленьких понятных шагов.
- Согласно одной из точек зрения на сцепление, функциональные инструменты образуют язык запросов, который имеет много общего с SQL. Сцепление функциональных инструментов позволяет выражать сложные запросы к массивам данных.
- Нам часто приходится создавать новые или расширять существующие данные, чтобы следующие шаги стали возможными. Ищите возможность представления неявной информации в виде явных данных.
- Существует много функциональных инструментов. Вы обнаружите их в процессе переработки своего кода. Также образцом для подражания могут стать средства других языков.
- Функциональные инструменты прокладывают путь в языки, которые традиционно не относились к функциональным, например Java. Используйте их там, где это уместно.

Что дальше?

В этой главе были рассмотрены некоторые мощные паттерны для работы с последовательностями данных. Тем не менее работа с вложенными данными все еще создает определенные проблемы. Чем больше глубина вложения, тем труднее работать с данными.

Мы разработаем другие функциональные инструменты с использованием функций высшего порядка. Они помогут нам работать с данными на больших уровнях вложенности.



В этой главе

- ✓ Построение функций высшего порядка для работы со значениями, хранящимися в хеш-картах.
- ✓ Простая работа с вложенными данными с использованием функций высшего порядка.
- ✓ Рекурсия и ее безопасное применение.
- ✓ Применение абстрактных барьеров для глубоко вложенных сущностей.

Мы изучили несколько полезных функциональных инструментов для работы над массивами. В этой главе мы разработаем и используем функциональные инструменты, работающие с объектами как с хеш-картами. Эти инструменты позволяют выполнять операции с картами глубокой вложенности, которые часто встречаются при построении более сложных структур данных. Без таких инструментов неизменяемая работа с вложенными данными становится очень неудобной. Но с такими инструментами вы можете создавать структуры сколь угодно большой вложенности, что предоставляет возможность структурировать данные так, как вы считаете нужным. Функции высшего порядка очень часто используются в функциональных языках.

Функции высшего порядка для значений в объектах



Директор по маркетингу: Мы пользовались вашими функциями высшего порядка, и они очень сильно помогли в улучшении нашего кода.

Дженна: Здорово!

Директор по маркетингу: Да! Действительно здорово. Мы проделали большую работу с методами рефакторинга, которые вы показали пару глав назад. Но сейчас мы занимаемся операциями, которые должны быть функциями высшего порядка, и не можем понять, как это сделать.

Дженна: Понятно. Можете рассказать подробнее?

Директор по маркетингу: Конечно. Мы пытаемся изменить значения, вложенные в объект товара. У нас множество операций, которые увеличивают или уменьшают размер или количество единиц товара. Но иногда обойтись без дублирования никак не удастся. Я покажу, как мы пришли к такому положению дел.

Дженна: Похоже, вам понадобилась функция высшего порядка, работающая с данными в объектах. Мы пока работали только с функциями высшего порядка, которые работают с данными в массивах. Будет очень полезно также иметь возможность работать со значениями, хранящимися в объектах.

Ким: Ясно, тогда за дело. Здесь места не осталось, продолжим на следующей странице.

Явное выражение имени поля

Отдел маркетинга начал применять методы рефакторинга из главы 10 самостоятельно. Посмотрим, что у них получилось. Изначально у них были похожие функции следующего вида:

```
function incrementQuantity(item) {
  var quantity = item['quantity'];
  var newQuantity = quantity + 1;
  var newItem = objectSet(item, 'quantity', newQuantity);
  return newItem;
}

function incrementSize(item) {
  var size = item['size'];
  var newSize = size + 1;
  var newItem = objectSet(item, 'size', newSize);
  return newItem;
}
```

Сначала они заметили, что имя поля указывается в имени функции. Мы называли этот признак «кода с душком» *неявным аргументом в имени функции*. В именах всех этих операций упоминалось имя поля. Недостаток был устранен посредством рефакторинга явного выражения неявного аргумента, который неоднократно использовался в нескольких последних главах.

```
function incrementQuantity(item) {
  var quantity = item['quantity'];
  var newQuantity = quantity + 1;
  var newItem = objectSet(item, 'quantity', newQuantity);
  return newItem;
}

function incrementField(item, field) {
  var value = item[field];
  var newValue = value + 1;
  var newItem = objectSet(item, field, newValue);
  return newItem;
}
```

Замечательно, это позволило устранить большое количество повторяющегося кода. Но после того как это было сделано для разных операций (инкремент, декремент, удвоение и т. д.), все стало выглядеть так, словно в коде снова появились дубликаты. Несколько примеров:

```
function incrementField(item, field) {
  var value = item[field];
  var newValue = value + 1;
  var newItem = objectSet(item, field, newValue);
  return newItem;
}

function decrementField(item, field) {
  var value = item[field];
  var newValue = value - 1;
  var newItem = objectSet(item, field, newValue);
  return newItem;
}

function doubleField(item, field) {
  var value = item[field];
  var newValue = value * 2;
  var newItem = objectSet(item, field, newValue);
  return newItem;
}

function halveField(item, field) {
  var value = item[field];
  var newValue = value / 2;
  var newItem = objectSet(item, field, newValue);
  return newItem;
}
```

Эти функции очень похожи. Они отличаются только выполняемой операцией. Но если присмотреться повнимательнее, можно заметить тот же признак «кода с душком» — *неявный аргумент в имени функции*. В имени каждой функции указывается операция. К счастью, мы можем просто повторно применить рефакторинг *явного выражения неявного аргумента*.

Построение update()

Ниже приведен код с предыдущей страницы. Обратите внимание: перед вами четыре очень похожие функции. Они отличаются только операцией, выполняемой со значением. И эта операция упоминается в имени функции. Нужно устранить это дублирование и создать функциональный инструмент, который будет обновлять объект за нас.

```
function incrementField(item, field) {
  var value = item[field];
  var newValue = value + 1;
  var newItem = objectSet(item, field, newValue);
  return newItem;
}

function doubleField(item, field) {
  var value = item[field];
  var newValue = value * 2;
  var newItem = objectSet(item, field, newValue);
  return newItem;
}

function decrementField(item, field) {
  var value = item[field];
  var newValue = value - 1;
  var newItem = objectSet(item, field, newValue);
  return newItem;
}

function halveField(item, field) {
  var value = item[field];
  var newValue = value / 2;
  var newItem = objectSet(item, field, newValue);
  return newItem;
}
```

Предшествующая часть

Тело

Завершающая часть

Можно провести два рефакторинга одновременно. Мы можем *явно выразить этот неявный аргумент*. Но аргументом будет функция, выполняющая операцию, поэтому происходящее будет напоминать *замену тела обратным вызовом*. Посмотрим, как это делается, на примере первой функции из приведенного кода:

```
function incrementField(item, field) {
  var value = item[field];
  var newValue = value + 1;
  var newItem = objectSet(item, field, newValue);
  return newItem;
}

function incrementField(item, field) {
  return updateField(item, field, function(value) {
    return value + 1;
  });
}

function updateField(item, field, modify) {
  var value = item[field];
  var newValue = modify(value);
  var newItem = objectSet(item, field, newValue);
  return newItem;
}
```

Извлечение в отдельную функцию

Передается функция изменения

Теперь все эти операции были сжаты до одной функции высшего порядка. Различия в поведении (операция, выполняемая с полем) передаются в форме обратного вызова. Как правило, специально оговаривать тот факт, что мы указываем поле, не нужно, поэтому функция обычно называется просто `update()`.

```
function update(object, key, modify) {
  var value = object[key];
  var newValue = modify(value);
  var newObject = objectSet(object, key, newValue);
  return newObject;
}
```

Чтение

Изменение

Запись

Функция `update()` позволяет изменить значение, содержащееся в объекте. Ей передается объект, ключ, под которым хранится значение, и функция для его изменения. В ней используется подход копирования при записи, потому что строится на основе функции `objectSet()`, также поддерживающей его. На следующей странице приведен пример ее использования.

Использование `update()` для изменения значений

Предположим, вы хотите повысить зарплату работника на 10 %. Для этого необходимо обновить его запись:

```
var employee = {  
  name: "Kim",  
  salary: 120000  
};
```

Имеется функция `raise10Percent()`, которая получает значение `salary` и возвращает его увеличенным на 10 %:

```
function raise10Percent(salary) {  
  return salary * 1.1;  
}
```

Применим ее к записи работника с помощью `update()`. Известно, что зарплата хранится с ключом `salary`:

```
> update(employee, 'salary', raise10Percent)
```

```
{  
  name: "Kim",  
  salary: 132000  
}
```

`update()` позволяет использовать `raise10Percent()` в контексте объекта работника (хеш-карта). `update()` получает операцию, которая применяется к конкретной разновидности значения (в данном случае зарплате), и применяет ее прямо к хеш-карте, содержащей это значение под определенным ключом. Можно сказать, что `update()` применяет функцию к значению во вложенном контексте.



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Изменяет ли `update()` исходную хеш-карту?

О: Нет, функция `update()` не изменяет исходную хеш-карту. Она использует подход копирования при записи, о котором вы узнали в главе 6. `update()` возвращает измененную копию переданной хеш-карты.

В: Если она не изменяет оригинал, то какая от нее польза?

О: Хороший вопрос. Как было показано в главе 6, функции могут использоваться для представления изменений во времени, для чего возвращаемое значение функции `update()` заменяет прежнее значение. Пример:

```
var employee = {
  name: "Kim",
  salary: 120000
};

employee = update(employee, salary, raise10Percent);
```

*Старое значение заменяется
новым*

Вычисление (определение зарплаты после повышения) отделяется от действия (изменение состояния).

Рефакторинг: замена схемы «чтение, изменение, запись» функцией `update()`

Мы применили два метода рефакторинга одновременно: 1) *явное выражение неявного аргумента* и 2) *замена тела обратным вызовом*. Однако процесс можно сделать и более прямолинейным. Код предшествующей и завершающей части:

До рефакторинга

```
function incrementField(item, field) {
  var value = item[field];
  var newValue = value + 1;
  var newItem = objectSet(item, field, newValue);
  return newItem;
}
```

Чтение
Изменение
Запись

После рефакторинга

```
function incrementField(item, field) {
  return update(item, field, function(value) {
    return value + 1;
  });
}
```

Обратите внимание: слева выполняются три шага.

1. Чтение значения из объекта.
2. Изменение значения.
3. Присваивание нового значения в объекте (с использованием копирования при записи).

Если все три операции выполняются с одним ключом, их можно заменить одним вызовом `update()`. Мы передаем объект, ключ, значение которого требуется изменить, и вычисление, которое его изменяет.

Последовательность действий для замены схемы «чтение, изменение, запись» вызовом `update()`

Данный метод рефакторинга состоит из двух шагов.

1. Определение фаз получения, изменения и присваивания.
2. Замена кода вызовом `update()` с передачей операции изменения в форме обратного вызова.

Шаг 1. Определение фаз получения, изменения и присваивания

```
function halveField(item, field) {
  var value = item[field];
  var newValue = value / 2;
  var newItem = objectSet(item, field, newValue);
  return newItem;
}
```

Чтение
Изменение
Запись

Шаг 2. Замена вызовом `update()`

```
function halveField(item, field) {
  return update(item, field, function(value) {
    return value / 2;
  });
}
```

Операция изменения передается в форме обратного вызова

Этот метод рефакторинга чрезвычайно полезен, потому что разработчику часто приходится работать с вложенными объектами.

Функциональный инструмент: `update()`

`update()` — другой важный функциональный инструмент. Инструменты, рассмотренные в предыдущих главах, работали с массивами, но этот работает с объектами (которые он рассматривает как хеш-карты). Рассмотрим его более подробно:

```
function update(object, key, modify) {
  var value = object[key];
  var newValue = modify(value);
  var newObject = objectSet(object, key, newValue);
  return newObject;
}
```

Получает объект, позицию значения (ключ) и операцию изменения
Чтение
Изменение
Запись
Возвращает измененный объект (с копированием при записи)

Функция `update()` позволяет взять функцию, работающую с отдельным значением, и применить ее к позиции внутри объекта. Она изменяет одно значение с одним конкретным ключом, поэтому речь идет о высокоточной операции:

Имеется объект:

```
{
  key1: X1,
  key2: Y1,
  key3: Z1
}
```

Требуется изменить одно значение:

```
{
  key1: X1,
  key2: Y2,
  key3: Z1
}
```

→ modifyY() →
Заменяет значение Y другим

`update()` необходимо получить: (1) изменяемый объект, (2) ключ для нахождения изменяемого значения и (3) функцию, вызываемую для изменения значения. Проследите за тем, чтобы функция, передаваемая `update()`, являлась вычислением. Она должна получать один аргумент (текущее значение) и возвращать новое значение. Посмотрим, как использовать ее в нашем примере:

```
function incrementField(item, field) {
  return update(item, field, function(value) {
    return value + 1;
  });
}
```

Передать update() объект (товар) — указывает на `item`
Передать update() поле для изменяемого значения — указывает на `field`
Передать update() функцию, изменяющую значение — указывает на `function(value) { return value + 1; }`
Возвращает значение, увеличенное на 1 — указывает на `return`

Наглядное представление значений в объектах

Рассмотрим операцию `update()` более наглядно. Допустим, у вас имеется объект для представления товара в корзине, который выглядит так:

Код

```
var shoes = {
  name: "shoes",
  quantity: 3,
  price: 7
};
```

Диаграмма с наглядным представлением этого объекта

Эквивалентная диаграмма

```
shoes
  name: "shoes"
  quantity: 3
  price: 7
```

Мы собираемся выполнить следующий фрагмент, который должен удвоить количество единиц:

```
> update(shoes, 'quantity', function(value) {
  return value * 2; // double the number
});
```

Разберем код `update()` шаг за шагом:

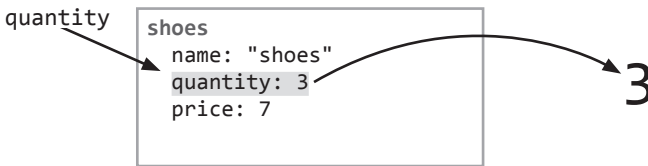
Шаг	Код
1.	<code>function update(object, key, modify) {</code>
2.	<code> var value = object[key];</code>
3.	<code> var newValue = modify(value);</code>
	<code> var newObject = objectSet(object, key, newValue);</code>
	<code> return newObject;</code>
	<code>}</code>

Чтение

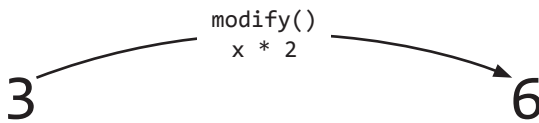
Изменение

Запись

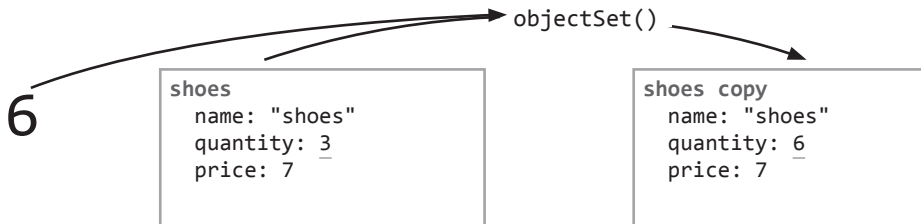
Шаг 1. Чтение из объекта значения с заданным ключом



Шаг 2. Вызов `modify()` для получения нового значения



Шаг 3. Создание измененной копии





Ваш ход

Имеется функция с именем `lowercase()`, которая преобразует строку к нижнему регистру. Адрес электронной почты пользователя хранится с ключом `'email'`. Измените запись пользователя с помощью `update()`, применяя `lowercase()` к адресу электронной почты.

```
var user = {
  firstName: "Joe",
  lastName: "Nash",
  email: "JOE@EXAMPLE.COM",
  ...
};
```

Эта строка преобразуется к нижнему регистру

Запишите здесь свой ответ



Ответ

```
> update(user, 'email', lowercase)

{
  firstName: "Joe",
  lastName: "Nash",
  email: "joe@example.com",
  ...
}
```




Ваш ход

Команда пользовательского интерфейса предлагает стимулировать крупные покупки. Они считают, что кнопка 10x, умножающая количество единиц товара на 10, сыграет положительную роль. Напишите функцию с использованием `update()`, которая умножает текущее количество единиц товара на 10. Пример:

```
var item = {
  name: "shoes",
  price: 7,
  quantity: 2,
  ...
};
```

Умножить на 10

```
function tenXQuantity(item) {
```

Запишите здесь
свой ответ



Ответ

```
function tenXQuantity(item) {
  return update(item, 'quantity', function(quantity) {
    return quantity * 10;
  });
}
```



Ваш ход

В следующих вопросах будет использоваться такая структура данных:

```
var user = {
  firstName: "Cindy",
  lastName: "Sullivan",
  email: "cindy@randomemail.com",
  score: 15,
  logins: 3
};
```

Исходные данные

- `increment()` прибавляет 1.
- `decrement()` вычитает 1.
- `uppercase()` преобразует строку к верхнему регистру.

1. Какое значение вернет следующий код?

> `update(user, 'score', increment).score`

Запишите здесь свой ответ

2. Какое значение вернет следующий код?

> `update(user, 'logins', decrement).score`

3. Какое значение вернет следующий код?

> `update(user, 'firstName', uppercase).firstName`



Ответ

1.

```
> update(user, 'score', increment).score
16
```

2.

```
> update(user, 'logins', decrement).score
15
```

Так как мы обновляем logins, score не изменяется

3.

```
> update(user, 'firstName', uppercase).firstName
"CINDY"
```



Директор по маркетингу: Да. Похоже, `update()` хорошо работает с данными внутри объекта. Но теперь внутри объектов содержатся другие объекты. До трех уровней вложенности!

Дженна: Ого! Можно посмотреть, как это выглядит?

Директор по маркетингу: Да. Вот наш код для увеличения размера рубашки:

```
var shirt = {
  name: "shirt",
  price: 13,
  options: {
    color: "blue",
    size: 3
  }
};
```

Вложенный объект внутри объекта (points to `options`)

Необходимо извлечь объект options (points to `options`)

```
function incrementSize(item) {
  var options = item.options;
  var size = options.size;
  var newSize = size + 1;
  var newOptions = objectSet(options, 'size', newSize);
  var newItem = objectSet(item, 'options', newOptions);
  return newItem;
}
```

Чтение (points to `item.options`)

Чтение (points to `options.size`)

Изменение (points to `size + 1`)

Запись (points to `objectSet(options, 'size', newSize)`)

Запись (points to `objectSet(item, 'options', newOptions)`)

objectSet() в обоих случаях (points to both `objectSet` calls)

Ким: О, теперь я поняла! У вас схема «чтение, чтение, изменение, запись, запись». Не совсем соответствует нашему методу рефакторинга.

Директор по маркетингу: Здесь что-нибудь можно сделать?

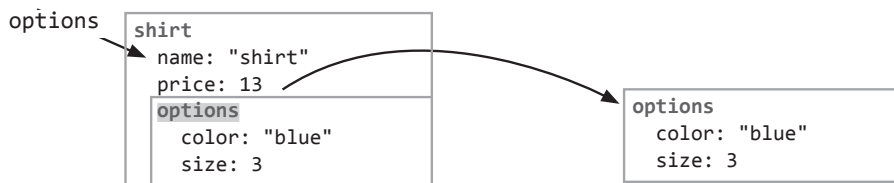
Дженна: Пока рано терять надежду. Думаю, здесь присутствует скрытое обновление. Мы все же сможем применить рефакторинг.

Наглядное представление обновлений вложенных данных

Ниже приведено определение функции, работающей с вложенным объектом `options`, который был представлен на предыдущей странице. Необходимо хорошо понимать, как она работает. Разберем происходящее строку за строкой:

Шаг	Код	
	<code>function incrementSize(item) {</code>	
1.	<code>var options = item.options;</code>	Чтение
2.	<code>var size = options.size;</code>	Чтение
3.	<code>var newSize = size + 1;</code>	Изменение
4.	<code>var newOptions = objectSet(options, 'size', newSize);</code>	Запись
5.	<code>var newItem = objectSet(item, 'options', newOptions);</code>	Запись
	<code>return newItem;</code>	
	<code>}</code>	

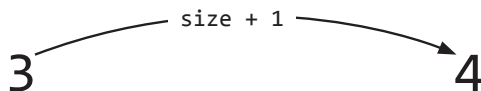
Шаг 1. Чтение из объекта значения с заданным ключом



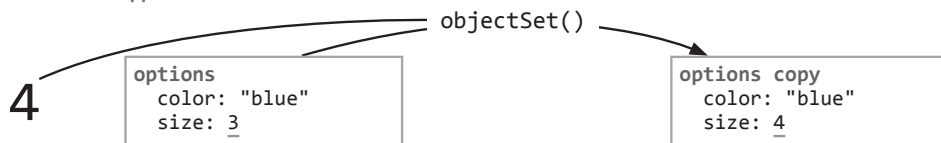
Шаг 2. Чтение из объекта значения с заданным ключом



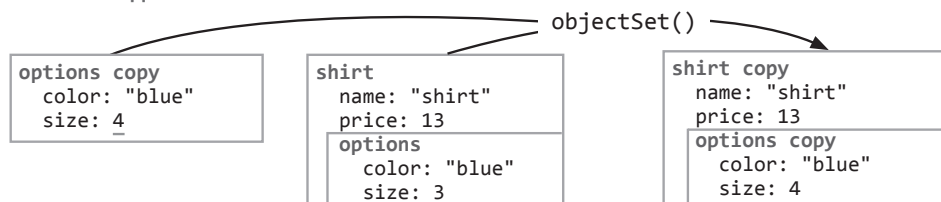
Шаг 3. Вычисление нового значения



Шаг 4. Создание измененной копии



Шаг 5. Создание измененной копии



Применение update() к вложенным данным

После того как вы наглядно увидели суть происходящего, попробуем применить рефакторинг для использования update(). На данный момент наш код выглядит так:

```
function incrementSize(item) {
  var options = item.options;
  var size = options.size;
  var newSize = size + 1;
  var newOptions = objectSet(options, 'size', newSize);
  var newItem = objectSet(item, 'options', newOptions);
  return newItem;
}
```

Вложенная серия «чтение, изменение, запись»

Чтение

Чтение

Изменение

Запись

Запись

Нам хотелось бы воспользоваться методом рефакторинга, заменяющим схему «чтение, изменение, запись» функцией update(), которую нам хотелось бы использовать. Необходимо выявить серии операций «чтение, изменение, запись». Можно ли применить здесь этот рефакторинг?

Оказывается, можно! Прямо в середине серия «чтение, изменение, запись» заключена между чтением наверху и записью внизу. Применим рефакторинг к этим трем командам в середине:

Шаги для замены серии операций «чтение, изменение, запись» с помощью update()

1. Выявление серии операций «чтение, изменение, запись»
2. Замена с помощью update() и передача изменения в качестве обратного вызова.

Оригинал

Серия «чтение, изменение, запись», заменяемая update()

```
function incrementSize(item) {
  var options = item.options;
  var size = options.size;
  var newSize = size + 1;
  var newOptions = objectSet(options, 'size', newSize);
  var newItem = objectSet(item, 'options', newOptions);
  return newItem;
}
```

После рефакторинга

```
function incrementSize(item) {
  var options = item.options;
  var newOptions = update(options, 'size', increment);
  var newItem = objectSet(item, 'options', newOptions);
  return newItem;
}
```

Чтение

Изменение

Запись

Снова появляется серия «чтение, изменение, запись»

Все вполне понятно. А теперь смотрите! Мы преобразовали среднюю серию «чтение, изменение, запись» в изменение. Вызов update() изменяет options. Следовательно, появляется новая серия «чтение, изменение, запись», к которой можно применить рефакторинг.

После одного рефакторинга

```
function incrementSize(item) {
  var options = item.options;
  var newOptions = update(options, 'size', increment);
  var newItem = objectSet(item, 'options', newOptions);
  return newItem;
}
```

Чтение

Изменение

Запись

После двух рефакторингов

```
function incrementSize(item) {
  return update(item, 'options', function(options) {
    return update(options, 'size', increment);
  });
}
```

Заменяется update()

Внутренняя серия update()

Следует осознать важный момент: для работы с вложенными объектами можно использовать вложенные обновления. При вложении вызовов update() мы работаем на более глубоком уровне вложения объектов. Мы продолжим развивать эту идею на следующей странице.

Построение updateOption()

Мы только что написали код, который выполнял обновление внутри обновления. Операцию можно обобщить в функцию вызовом `update2()`:

```
function incrementSize(item) {
  return update(item, 'options', function(options) {
    return update(options, 'size', increment);
  });
}
```

```
var shirt = {
  name: "shirt",
  price: 13,
  options: {
    color: "blue",
    size: 3
  }
};
```

Обратите внимание: мы вызываем `date()` дважды, и данные `size` вложены дважды (чтобы добраться до них, необходимо пройти через два объекта). Глубина вложения вызовов `update()` должна соответствовать глубине вложения данных.

Это важное обстоятельство, к которому мы вскоре вернемся. Но для начала поработаем с только что написанной функцией. Можно заметить два признака «кода с душком», которые уже встречались нам ранее. На самом деле это один признак, повторенный дважды:

Последовательность действий при явном выражении неявного аргумента

1. Выявление неявного аргумента в имени функции.
2. Добавление явного аргумента.
3. Использование нового аргумента в теле.
4. Обновление кода вызова.

```
function incrementSize(item) {
  return update(item, 'options', function(options) {
    return update(options, 'size', increment);
  });
}
```

Неявный аргумент в имени функции! Дважды!

С неявным аргументом характеристики

```
function incrementSize(item) {
  return update(item, 'options', function(options) {
    return update(options, 'size', increment);
  });
}
```

С неявным аргументом операции

```
function incrementOption(item, option) {
  return update(item, 'options', function(options) {
    return update(options, option, increment);
  });
}
```

С неявным аргументом характеристики

```
function incrementOption(item, option) {
  return update(item, 'options', function(options) {
    return update(options, option, increment);
  });
}
```

С неявным аргументом операции

```
function updateOption(item, option, modify) {
  return update(item, 'options', function(options) {
    return update(options, option, modify);
  });
}
```

В коде встречаются два неявных аргумента, их нужно сделать явными. Сделаем это последовательно. Начнем с 'size':

Отлично! Функция получает товар (объект), имя характеристики и функцию, которая изменяет эту характеристику.

```
function updateOption(item, option, modify) {
  return update(item, 'options', function(options) {
    return update(options, option, modify);
  });
}
```

Проблема осталась! Имя неявного аргумента все еще присутствует в имени функции

Снова та же проблема! На этот раз неявным аргументом оказывается 'options', и его имя присутствует в имени функции.

Построение update2()

На предыдущей странице мы выполнили явное выражение двух неявных аргументов. Рефакторинг выявил третий неявный аргумент. Если провести его еще один раз, мы получим универсальный инструмент, который назовем update2(). Еще раз приведу код:

Имя поля отражено в имени функции, хотя могло бы быть аргументом

```
function updateOption(item, option, modify) {
  return update(item, 'options', function(options) {
    return update(options, option, modify);
  });
}
```

Применим рефакторинг в третий раз. Функция станет более универсальной, поэтому мы изменим имена, чтобы они отражали ее новую роль:

Так как функция становится более универсальной, мы присваиваем более общие имена ее аргументам

С неявным аргументом

```
function updateOption(item, option, modify) {
  return update(item, 'options', function(options) {
    return update(options, option, modify);
  });
}
```

2 означает два уровня вложенности

С явным аргументом

```
function update2(object, key1, key2, modify) {
  return update(object, key1, function(value1) {
    return update(value1, key2, modify);
  });
}
```

Становится явным аргументом

Функция стала предельно общей. Ее можно назвать update2(), потому что она работает с любыми объектами, вложенными в объекты (второй уровень глубины). Это объясняет то, что ей нужны два ключа.

Последовательность действий при явном выражении неявного аргумента

1. Выявление неявного аргумента в имени функции.
2. Добавление явного аргумента.
3. Использование нового аргумента в теле.
4. Обновление кода вызова.

Просто для уверенности попробуем заново реализовать `incrementSize()` с новой функцией. Структура данных, над которой мы работали:

```
var shirt = {
  name: "shirt",
  price: 13,
  options: {
    color: "blue",
    size: 3
  }
};
```

Требуется увеличить это значение по пути 'options', 'size'

А ниже исходная реализация сравнивается с реализацией, использующей `update2()`:

Оригинал

```
function incrementSize(item) {
  var options = item.options;
  var size = options.size;
  var newSize = size + 1;
  var newOptions = objectSet(options, 'size', newSize);
  var newItem = objectSet(item, 'options', newOptions);
  return newItem;
}
```

С использованием update2()

```
function incrementSize(item) {
  return update2(item, 'options', 'size', function(size) {
    return size + 1;
  });
}
```

`update2()` берет на себя все серии «чтение, чтение, изменение, запись, запись», которые вам пришлось бы писать самостоятельно. Впрочем, описание становится немного абстрактным, поэтому далее оно будет представлено в наглядном виде.

Наглядное представление update2() с вложенными объектами

Итак, мы разработали функцию `update2()`, изменяющую значение на втором уровне вложенности в объектах. Посмотрим, как это выглядит на диаграмме.

Требуется увеличить значение `size` в `options`. Для этого следует пройти по вложенным объектам. Начиная с объекта товара, мы входим в объект с ключом `'options'`, а затем находим значение с ключом `'size'`.

Код, который должен быть выполнен:

```
> return update2(shirt, 'options', 'size', function(size) {
  return size + 1;
});
```

Увеличение значения (подчеркнуто в строке `return size + 1;`)

Путь к нужному значению (подчеркнуты в строке `'options', 'size'`)

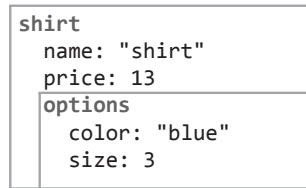


Загляни в словарь

Последовательный список ключей называется *путем*. Он используется для поиска желаемого значения во вложенном объекте.

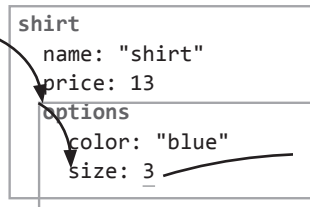
А на этой диаграмме представлен объект товара. Обратите внимание на вложенный объект **options**:

```
var shirt = {
  name: "shirt",
  price: 13,
  options: {
    color: "blue",
    size: 3
  }
};
```



На пути к значению (два чтения)

Чтобы добраться до вложенного значения, выполняем операции чтения по заданному пути



В конце пути находится искомое значение, увеличиваем его

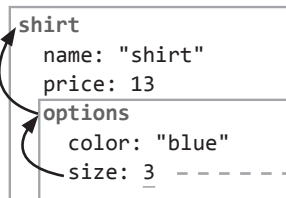
size+1

4

Приводит к созданию измененной копии всего, что находилось на пути

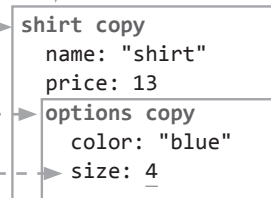
На пути от значения (два чтения)

На обратном пути выполняется копирование при записи для вложенных объектов



objectSet() →

objectSet() →



size+1 - - - - -



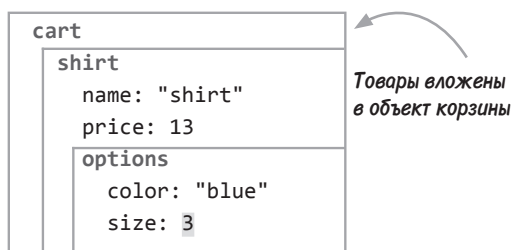
Директор по маркетингу: Нет, ничего ужасного. Просто мы обычно работаем с товарами, находящимися в корзине. Не всегда, но часто.

Дженна: И?

Директор по маркетингу: В этом примере изменяется характеристика в объекте `options`, который находится внутри объекта товара. Но объект товара находится внутри объекта корзины.

```
var cart = {
  shirt: {
    name: "shirt",
    price: 13,
    options: {
      color: "blue",
      size: 3
    }
  }
}
```

Три уровня вложенности



Дженна: А, понимаю. Еще один уровень вложенности.

Директор по маркетингу: Вот именно. Функция называется `incrementSizeByName()`. Означает ли это, что нам понадобится функция `update3()`?

Ким: Нет, но она может пригодиться. Посмотрим, что можно сделать с тем, что у нас уже есть. Думаю, мы можем просто добавить один уровень к уже имеющейся структуре.

Четыре варианта реализации `incrementSizeByName()`

Директор по маркетингу говорит об изменении характеристик товаров, находящихся в корзине. Получаем три уровня вложенности. Рассмотрим четыре разных варианта решения этой проблемы. Итак, директор по маркетингу хочет увеличить характеристику размера для товара в корзине по имени, для чего должна использоваться функция с именем `incrementSizeByName()`. Функция получает объект корзины и название товара и увеличивает характеристику размера для этого товара. Как же написать такую функцию?

Вариант 1: использование `update()` и `incrementSize()`

Требуется выполнить операцию с товаром, вложенным в объект корзины. Функция `update()` изменяет значение, вложенное в корзину, и к товару можно применить функцию `incrementSize()`:

```
function incrementSizeByName(cart, name) {
  return update(cart, name, incrementSize);
}
```

Самый прямолинейный способ использования уже имеющегося кода

Вариант 2: использование `update()` и `update2()`

Также можно встроить реализацию `incrementSize()` и использовать `update2()`:

```
function incrementSizeByName(cart, name) {
  return update(cart, name, function(item) {
    return update2(item, 'options', 'size', function(size) {
      return size + 1;
    });
  });
}
```

При встроенном вызове `incrementSize()` функция `update2()` вкладывается в `update()`

Вариант 3: использование `update()`

Можно использовать встроенную реализацию `update2()`, что дает два вложенных вызова `update()`.

```
function incrementSizeByName(cart, name) {
  return update(cart, name, function(item) {
    return update(item, 'options', function(options) {
      return update(options, 'size', function(size) {
        return size + 1;
      });
    });
  });
}
```

Используем, пока не останутся только вызовы `update()`



Пицца для ума

Какой из четырех вариантов кажется вам предпочтительным? Почему? Считаете ли вы, что какие-то варианты полностью исключаются? Почему? Вскоре мы обсудим эту тему.

Вариант 4: ручная реализация операций чтения, изменения и присваивания

Также возможно использование встроенной реализации `update()` с операциями чтения, изменения и присваивания:

```
function incrementSizeByName(cart, name) {
  var item      = cart[name];
  var options   = item.options;
  var size      = options.size;
  var newSize   = size + 1;
  var newOptions = objectSet(options, 'size', newSize);
  var newItem   = objectSet(item, 'options', newOptions);
  var newCart   = objectSet(cart, name, newItem);
  return newCart;
}
```

Чтение, чтение, чтение
Изменение
Запись, запись, запись

Построение update3()

Займемся разработкой функции `update3()`. Мы уже несколько раз проделывали нечто подобное, поэтому вы довольно быстро разберетесь в происходящем. Начнем с варианта 2 на предыдущей странице, применим явное выражение неявного аргумента и получим определение `update3()`. Все будет сделано за один заход:

Вариант 2

```
function incrementSizeByName(cart, name) {
  return update(cart, name, function(item) {
    return update2(item, 'options', 'size',
      function(size) { return size + 1; });
  });
}
```

Неявные аргументы
Выделяется в update3()

После рефакторинга

```
function incrementSizeByName(cart, name) {
  return update3(cart,
    name, 'options', 'size',
    function(size) { return size + 1; });
}

function update3(object, key1, key2, key3, modify) {
  return update(object, key1, function(object2) {
    return update2(object2, key2, key3, modify);
  });
}
```

Путь из трех частей
Функция update3() — всего лишь функция update2(), вложенная в update()

`update3()` вкладывает `update2()` в `update()`. Функция `update()` позволяет опуститься на один уровень глубже функции `update2()`, всего на четыре уровня.

Последовательность действий при явном выражении неявного аргумента

1. Выявление неявного аргумента в имени функции.
2. Добавление явного аргумента.
3. Использование нового аргумента в теле.
4. Обновление кода вызова.



Ваш ход

Отделу маркетинга понадобились функции `update4()` и `update5()`. Напишите их.

А еще нам пригодятся функции `update4()` и `update5()`!



**Директор
по маркетингу**

Запишите здесь свой ответ



Ответ

```
function update4(object, k1, k2, k3, k4, modify) {
  return update(object, k1, function(object2) {
    return update3(object2, k2, k3, k4, modify);
  });
}

function update5(object, k1, k2, k3, k4, k5, modify) {
  return update(object, k1, function(object2) {
    return update4(object2, k2, k3, k4, k5, modify);
  });
}
```

Построение nestedUpdate()

Мы только что написали функцию `update3()` и выявили схему, которая позволяет быстро написать `date4()` и `update5()`. Но если закономерность настолько ясна, наверняка ее можно отразить в функции. Пока от нас еще не потребовали вывести функции `update6()` через `update21()`, начнем работать над функцией `nestedUpdate()`, которая работает с любым количеством уровней вложенности.

Для начала разберем схему на части:

```
function update3(object, key1, key2, key3, modify) {
  return update(object, key1, function(value1) {
    return update2(value1, key2, key3, modify);
  });
}

function update4(object, key1, key2, key3, key4, modify) {
  return update(object, key1, function(value1) {
    return update3(value1, key2, key3, key4, modify);
  });
}
```

В этих функциях `X` — это количество уровней вложенности, а `X-1` — количество уровней, которое уже было обработано.

Закономерность проста: мы определяем `updateX()` как функцию `updateX-1()`, вложенную в `update()`. `update()` использует первый ключ, после чего передает остальные ключи с сохранением порядка, а также `modify` функции `updateX-1()`. Как будет выглядеть эта схема для `update2()`? Функция `update2()` у нас уже есть, но пока забудем об этом:

```
function update2(object, key1, key2, modify) {
  return update(object, key1, function(value1) {
    return update1(value1, key2, modify);
  });
}
```

*2 в имени означает два ключа
и вызов update1()*

Как будет выглядеть `update1()`? `X-1` дает 0, поэтому

```
function update1(object, key1, modify) {
  return update(object, key1, function(value1) {
    return update0(value1, modify);
  });
}
```

*1 в имени означает один
ключ и вызов update0()*

`update0()` нарушает эту схему, потому что она отличается от других в двух отношениях. Во-первых, ключей нет, поэтому мы не можем вызвать `update()` с первым ключом (первого ключа нет). Во-вторых, `X-1` дает `-1`, что не имеет смысла для длины пути.

Интуитивно понятно, что `update()` означает вложение на нулевую глубину. В схеме 0 операций чтения и 0 операций присваивания: только изменение. Иначе говоря, мы ищем искомое значение, поэтому достаточно применить функцию `modify()`:

```
function update0(value, modify) {
  return modify(value);
}
```

*0 в имени означает
нуль ключей*

**Признаки неявного
аргумента в имени функции**

1. Похожие реализации.
2. Упоминание различий в имени функции.

До сих пор процедура была довольно сухой. Извините! Но здесь происходит кое-что интересное: признак «кода с душком» снова проявился! *Неявный аргумент* встречается в имени функции. Число в имени функции всегда соответствует количеству аргументов. Разберемся с этой проблемой на следующей странице.

На предыдущей странице мы выявили закономерность в функциях `updateX()`. Был также выявлен такой признак «кода с душком», как *неявный аргумент в имени функции*. К счастью, имеется метод рефакторинга, который может устранить его: *явное выражение неявного аргумента*.

Для примера возьмем `update3()`. Как превратить 3 в явный аргумент?

```
function update3(object, key1, key2, key3, modify) {
  return update(object, key1, function(value1) {
    return update2(value1, key2, key3, modify);
  });
}
```

X — X ключей
X-1 — Первый ключ опускается

Можно просто добавить аргумент с именем `depth`:

```
function updateX(object, depth, key1, key2, key3, modify) {
  return update(object, key1, function(value1) {
    return updateX(value1, depth-1, key2, key3, modify);
  });
}
```

Явный аргумент depth
depth соответствует количеству ключей
Учтите, что эта функция не работает
На один ключ меньше
Рекурсивный вызов
Передача depth-1

Теперь аргумент выражается явно, но при этом возникает новая проблема: как обеспечить совпадение глубины и количества ключей? Отдельный параметр глубины с большой вероятностью породит ошибки в цепочке, однако он дает подсказку: необходимо поддерживать *порядок* и *количество ключей*. Это четко указывает на конкретную структуру данных: массивы. Что произойдет, если передавать ключи в массиве? Параметр глубины станет длиной. Сигнатура выглядит так:

```
function updateX(object, keys, modify) {
```

Массив ключей

Мы будем действовать по той же схеме. Функция `update()` вызывается с первым ключом, а остальные ключи передаются `updateX()`. Набор оставшихся ключей будет иметь длину $X - 1$:

```
function updateX(object, keys, modify) {
  var key1 = keys[0];
  var restOfKeys = drop_first(keys);
  return update(object, key1, function(value1) {
    return updateX(value1, restOfKeys, modify);
  });
}
```

Первый ключ используется для вызова update()
Первый ключ в рекурсивном вызове опускается



Загляни в словарь

Рекурсивная функция — функция, которая определяется в контексте самой себя. Рекурсивная функция содержит *рекурсивный вызов*, то есть вызов этой же функции.

Рекурсия более подробно рассматривается через несколько страниц, а пока просто допишем функцию

Отлично! Это позволит нам заменить все функции `updateX()`, кроме функции `update0()`, с которой необходимо поступить иначе. Код функции `update0()` приведен ниже.

Вот что у нас получилось с `updateX()` на данный момент:

```
function updateX(object, keys, modify) {
  var key1 = keys[0];
  var restOfKeys = drop_first(keys);
  return update(object, key1, function(value1) {
    return updateX(value1, restOfKeys, modify);
  });
}
```

Что произойдет, если ключей нет?

Функция может заменить `update1()`, `update2()` и `update3()` (и предположительно `update4`, `5`, `6`, ...()), потому что все эти функции строятся по одной схеме. Но с `update0()` дело обстоит иначе. Она вообще не вызывает `update()`, а только `modify()`. Как реализовать это ограничение?

```
function update0(value, modify) {
  return modify(value);
}
```

Определение `update0()` отличается от других

Эта функция не рекурсивна

Для нуля придется предусмотреть особый случай. Мы знаем, что при нулевой длине массива `keys` количество ключей равно нулю. В таком случае достаточно вызвать `modify()`. В противном случае выполняется код `updateX()`.

Давайте сделаем это:

Без обработки нуля

```
function updateX(object, keys, modify) {

  var key1 = keys[0];
  var restOfKeys = drop_first(keys);
  return update(object, key1, function(value1) {
    return updateX(value1, restOfKeys, modify);
  });
}
```

С обработкой нуля

```
function updateX(object, keys, modify) {
  if(keys.length === 0)
    return modify(object);
  var key1 = keys[0];
  var restOfKeys = drop_first(keys);
  return update(object, key1, function(value1) {
    return updateX(value1, restOfKeys, modify);
  });
}
```

Обработка нуля

Без рекурсии

Рекурсивный вызов

Теперь у нас имеется версия `updateX()`, которая работает для любого количества ключей. Она может использоваться для применения функции `modify()` к значению на любой глубине вложенности объектов. Чтобы узнать значение, достаточно знать последовательность ключей каждого объекта.

`updateX()` обычно присваивается имя `nestedUpdate()`. Переименуем функцию на следующей странице.



Загляни в словарь

Базовым случаем в рекурсии называется случай без рекурсивного вызова, который останавливает рекурсию. Каждый последующий рекурсивный вызов должен постепенно продвигаться к базовому случаю.

На предыдущей странице мы завершили разработку функции `updateX()`, работающей с вложенными данными произвольной глубины, вплоть до нулевой. Данные, вложенные на 0 уровней, изменяются напрямую.

Обычно этой функции не присваивается имя `updateX()`. Лучше было бы назвать ее `nestedUpdate()`. Функция получает объект, последовательность ключей для перехода к вложенному значению и функцию, которая должна быть вызвана для значения после того, как оно будет обнаружено. Затем на пути наружу создаются измененные копии всех промежуточных уровней:

```
function nestedUpdate(object, keys, modify) {
  if(keys.length === 0)
    return modify(object);
  var key1 = keys[0];
  var restOfKeys = drop_first(keys);
  return update(object, key1, function(value1) {
    return nestedUpdate(value1, restOfKeys, modify);
  });
}
```

Базовый случай (путь нулевой длины)

Продвижение к базовому случаю (с исключением одного элемента из пути)

Рекурсивный случай

Функция `nestedUpdate()` работает с путями любой длины, включая нулевую. Она является рекурсивной, то есть определяется в контексте самой себя. Функциональные программисты используют рекурсию немного чаще, чем любые другие программисты. Это достаточно глубокая концепция, и нам понадобится по крайней мере пара страниц, чтобы вы действительно хорошо поняли, как она работает.



Отдых для мозга

Это еще не все, но сделаем небольшой перерыв для ответов на вопросы.

В: Как функция может вызывать саму себя?

О: Хороший вопрос. Функция может вызвать любую функцию, в том числе и себя. Если функция вызывает сама себя, она называется *рекурсивной*. *Рекурсия* — общая концепция функций, которые вызывают сами себя. Только что написанная нами функция `nestedUpdate()` является рекурсивной:

```
function nestedUpdate(object, keys, modify) {
  if(keys.length === 0)
    return modify(object);
  var key1 = keys[0];
  var restOfKeys = drop_first(keys);
  return update(object, key1, function(value1) {
    return nestedUpdate(value1, restOfKeys, modify);
  });
}
```

Вызывает саму себя

В: Для чего нужна рекурсия? Честно говоря, это трудно понять.

О: Понять смысл рекурсии бывает нелегко, даже если у вас появится практический опыт. Рекурсия очень хорошо подходит для работы с вложенными данными. Вспомните, что мы определяли функцию `deepCopy()` рекурсивно (см. с. 194) по той же причине. При работе с вложенными данными все уровни часто обрабатываются по единым принципам. Каждый вызов рекурсивной функции устраняет один уровень вложения, после чего снова применяет ту же операцию на следующем уровне.

В: Почему бы не использовать обычный перебор? Циклы `for` проще понять.

О: Циклы `for` очень часто проще понять, чем рекурсию. Пишите код в том виде, который наиболее ясно представляет ваши намерения. Хотя в данном случае рекурсия оказывается более понятной. Механизм рекурсивных вызовов использует стек вызовов, который хранит значения аргументов и адреса возврата для вызовов функций. Для цикла `for` нам пришлось бы поддерживать собственный стек. Стек JavaScript делает все необходимое, при этом нам не нужно беспокоиться об операциях занесения и извлечения значений из стека.

В: Разве рекурсия не опасна? Программа не может заиклиться или выйти за границу стека?

О: Да! Рекурсия может заиклиться, но это возможно и в традиционных циклах. Иногда в зависимости от рекурсивного вызова и языка при достижении определенной глубины рекурсии будет исчерпана память стека. Если программа написана правильно, стек не должен достигнуть такой глубины. Тем не менее, чтобы правильно организовать рекурсию, необходимо кое-что уяснить. Впрочем, когда вы узнаете основные тонкости, все не так уж сложно. Рассмотрим анатомию безопасной рекурсии.

Анатомия безопасной рекурсии

По аналогии с тем, как циклы `for` и `while` могут выполняться бесконечно, рекурсия также может стать бесконечной. Если соблюдать ряд полезных правил, никаких проблем не будет.

1. Базовый случай

Если вы хотите, чтобы рекурсия в какой-то момент остановилась, необходимо определить *базовый случай*, то есть ситуацию, в которой рекурсия должна остановиться. Базовый случай не включает рекурсивные вызовы, поэтому рекурсия на этом останавливается:

```
function nestedUpdate(object, keys, modify) {
  if(keys.length === 0) {
    return modify(object);
  }
  var key1 = keys[0];
  var restOfKeys = drop_first(keys);
  return update(object, key1, function(value1) {
    return nestedUpdate(value1, restOfKeys, modify);
  });
}
```

Базовый случай

Без рекурсии

Обычно базовый случай проверяется достаточно легко. Часто это пустой массив в аргументе, обнуление счетчика или обнаружение искомого значения. В таких ситуациях задача ясна. Как правило, базовый случай — самая простая часть рекурсии.

2. Рекурсивный случай

Рекурсивная функция должна содержать как минимум один рекурсивный случай, то есть случай, в котором происходит рекурсивный вызов.

```
function nestedUpdate(object, keys, modify) {
  if(keys.length === 0) {
    return modify(object);
  }
  var key1 = keys[0];
  var restOfKeys = drop_first(keys);
  return update(object, key1, function(value1) {
    return nestedUpdate(value1, restOfKeys, modify);
  });
}
```

Длина `restOfKeys` уменьшается на 1

Рекурсивный вызов

3. Продвижение к базовому случаю

При выполнении рекурсивного вызова необходимо проследить за тем, чтобы по крайней мере один аргумент «уменьшился», то есть хотя бы на один шаг приблизился к базовому случаю. Например, если базовым случаем является пустой массив, на каждом шаге из массива необходимо удалять по элементу.

Если каждый рекурсивный вызов приближается на один шаг, со временем вы достигнете базового случая, и рекурсия остановится. Худшее, что можно сделать, — выполнить рекурсивный вызов с теми же аргументами, которые он получил. Это наверняка создаст бесконечный цикл.

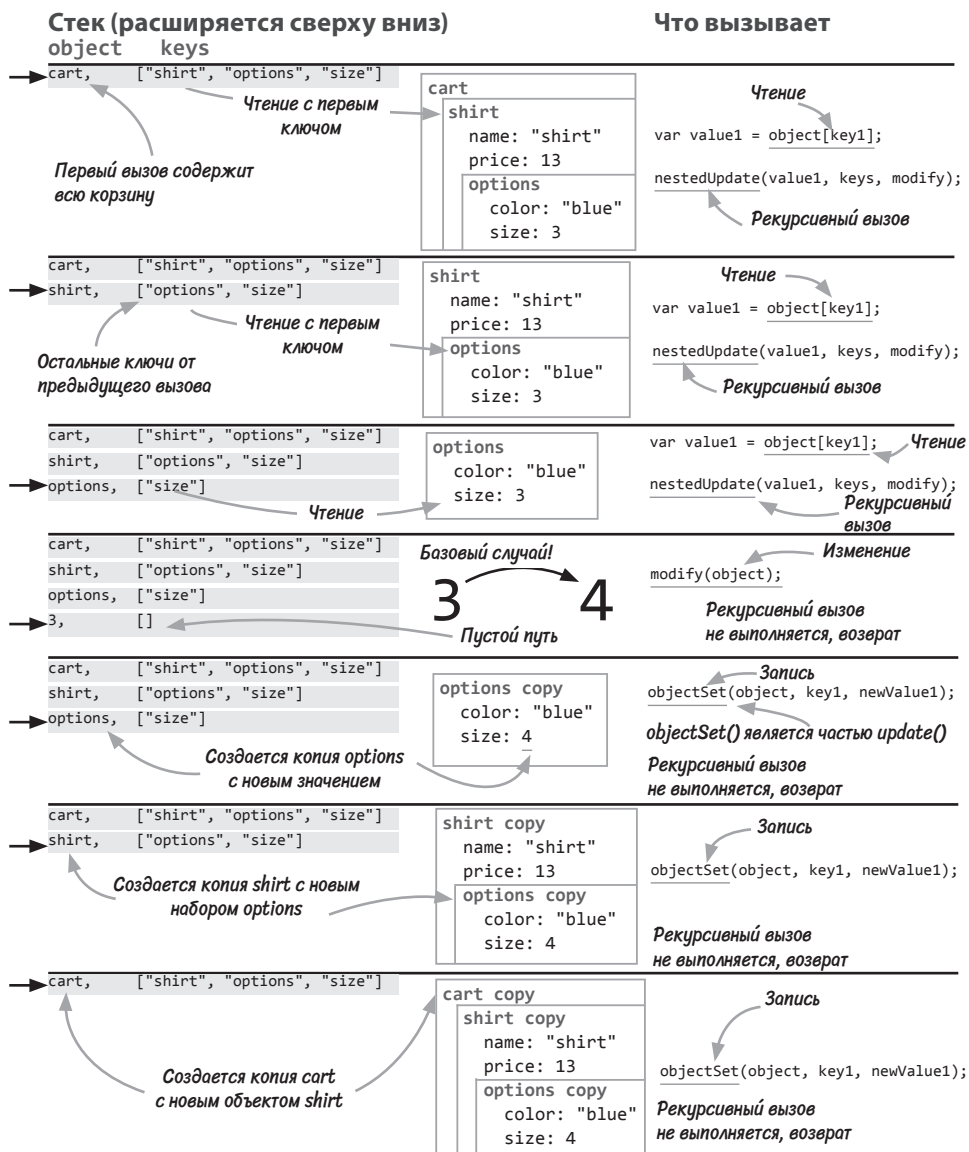
Наглядное представление поведения этой функции поможет вам лучше понять ее.

Наглядное представление nestedUpdate()

Все эти описания становятся слишком абстрактными, поэтому мы поэтапно разберем вызов `nestedUpdate()` в наглядном виде. Это будет сделано для глущины 3 (ранее эта функция называлась `update3()`).

Выполняться будет следующий код, который увеличивает размер товара в корзине:

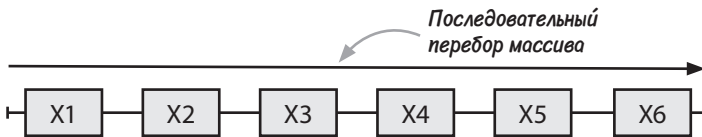
```
> nestedUpdate(cart, ["shirt", "options", "size"], increment)
```



Сила рекурсии

Циклы `for` очень полезны. До настоящего момента мы использовали их везде, где требовалось работать с массивами. Даже наши функциональные инструменты для массивов были реализованы с использованием циклов `for`. Тем не менее на этот раз ситуация изменилась, так как мы работаем с вложенными данными.

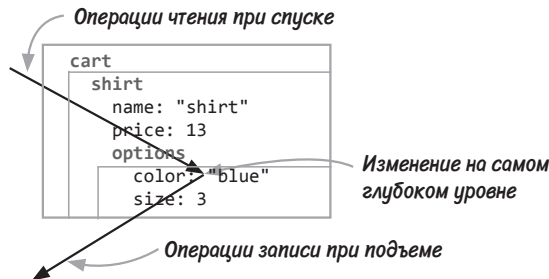
При переборе массива обработка велась по порядку, начиная с самого начала. При посещении каждого элемента мы добавляли элемент в конец полученного массива:



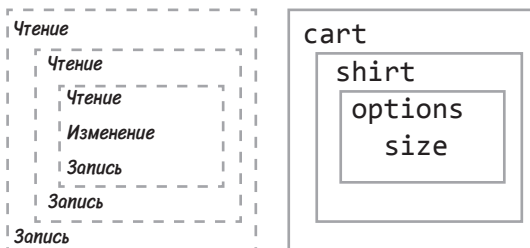
Дженна из команды разработки

Я все еще не понимаю, почему рекурсия здесь работает лучше цикла `for`.

С другой стороны, при работе с вложенными данными необходимо выполнять чтение на всем пути до нижнего уровня, затем изменить итоговое значение и после этого выполнять запись на обратном пути. Эти операции записи создают копии (потому что они выполняются с копированием при записи).



Вложенность операций чтения, изменения и записи отражает вложенность данных. Такое вложение не удастся реализовать без рекурсии и стека вызовов.



Пища для ума

Как бы вы записали `nestedUpdate()` с циклом (`for` или `while`)? Попробуйте сами!



Ваш ход

На с. 413 были представлены четыре варианта реализации `incrementSizeByName()`. Вам предлагается записать функцию пятым способом с использованием `nestedUpdate()`.

```
function incrementSizeByName(cart, name) {
```

*Запишите здесь
свой ответ*

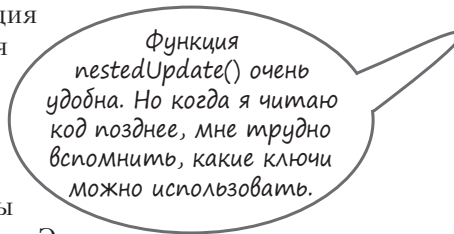


Ответ

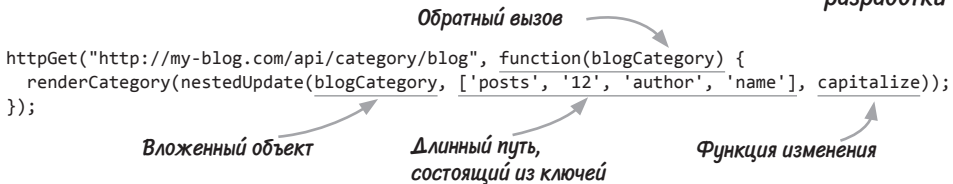
```
function incrementSizeByName(cart, name) {  
  return nestedUpdate(cart, [name, 'options', 'size'],  
    function(size) {  
      return size + 1;  
    });  
}
```

Конструктивные особенности при глубоком вложении

Часто приходится слышать очень распространенную претензию. Функция `nestedUpdate()` используется для обращения к глубоко вложенным данным по длинной цепочке ключей. Трудно запомнить все промежуточные объекты и понять, какие ключи должны присутствовать в каждом объекте. Это становится особенно очевидно тогда, когда вы используете сторонний API и не контролируете модель данных.



Ким из команды разработки



В этом коде приведен упрощенный пример. Мы производим выборку категории «blog» через API блога; запрос возвращает разметку JSON, которая обрабатывается в обратном вызове. В обратном вызове имя автора 12-го сообщения в категории «blog» преобразуется к верхнему регистру. Пример получился искусственным, но он наглядно демонстрирует проблему. Когда через три недели вы возвращаетесь к чтению этого кода, сколько всего нужно будет понимать? Вот краткий список.

1. Категория содержит сообщения, хранящиеся с ключом `'posts'`.
2. К записям отдельных сообщений можно обращаться по идентификатору.
3. Сообщение содержит запись пользователя, хранящуюся с ключом `'author'`.
4. Запись пользователя содержит имя пользователя, которое хранится с ключом `'name'`.



Напоминание

Абстрактный барьер — прослойка функций, которая так хорошо скрывает реализацию, что вы можете полностью забыть о том, как она реализована (даже при использовании этих функций).

Выясняется, что для каждого уровня вложенности имеется совершенно новая структура данных, которую необходимо помнить,

чтобы понять путь. Именно из-за этого сложно запомнить имеющиеся ключи. Промежуточные объекты имеют разные наборы ключей, и никакие из них не очевидны при просмотре пути `nestedUpdate()`.

Что же делать? К счастью, решение уже упоминалось при обсуждении многоуровневого проектирования (а именно в главе 9). Если вам приходится слишком много информации держать в памяти, одним из возможных решений может стать абстрактный барьер. Абстрактные барьеры позволяют игнорировать лишние подробности. Далее показано, как выглядит такое решение.

Абстрактные барьеры для глубоко вложенных данных

На предыдущей странице было показано, что глубоко вложенные данные часто создают высокую когнитивную нагрузку. Нам приходится держать в памяти одну структуру данных на каждый уровень вложенности. Нужно каким-то образом сократить количество структур данных, которые необходимо понимать для выполнения той же работы. Для этой задачи можно воспользоваться абстрактными барьерами. Другими словами, вы создаете функции, которые используют эти структуры данных, и назначаете функциям содержательные имена. Впрочем, вы в любом случае должны стремиться к этой цели по мере укрепления вашего понимания данных.

А если написать функцию, которая может изменить в заданной категории сообщение с заданным идентификатором?

```
function updatePostById(category, id, modifyPost) {
  return nestedUpdate(category, ['posts', id], modifyPost);
}
```

Понятное имя (стрелка на `updatePostById`)

Чтобы использовать эту функцию, не обязательно знать, как сообщения хранятся в категории (стрелка на `category`)

Структурные подробности категории скрыты от кода над барьером (стрелка на `['posts', id]`)

Чтобы использовать эту функцию, не обязательно знать, как информация автора хранится в сообщении (стрелка на `modifyPost`)

Теперь можно создать операцию для изменения автора сообщения:

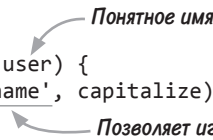
```
function updateAuthor(post, modifyUser) {
  return update(post, 'author', modifyUser);
}
```

Понятное имя (стрелка на `updateAuthor`)

Чтобы использовать эту функцию, не обязательно знать, как информация автора хранится в сообщении (стрелка на `post`)

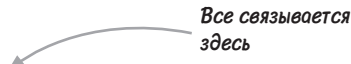
modifyUser умеет работать с пользователями (стрелка на `modifyUser`)

Если пойти еще дальше, можно создать операцию для преобразования имени любого пользователя к верхнему регистру:



```
function capitalizeName(user) {
  return update(user, 'name', capitalize);
}
```

Теперь соберем все вместе:



```
updatePostById(blogCategory, '12', function(post) {
  return updateAuthor(post, capitalizeUserName);
});
```

Стало лучше? Да, по двум причинам. Во-первых, в голове достаточно держать три структуры вместо четырех. Это сильно упрощает жизнь. Во-вторых, имена операций помогают запомнить отдельные компоненты. Мы уже знаем, что категории содержат сообщения. Теперь не нужно запоминать ключи, под которыми они хранятся. Это относится и к автору. Мы уже знаем, что у сообщения есть автор, а теперь не нужно знать, как хранится информация.

Сводка использования функций высшего порядка

В главе 10 мы сначала изучили концепцию функций высшего порядка, то есть функций, которые получают другие функции в аргументах и/или возвращают функции.

Некоторые применения этих функций были представлены выше. Сейчас стоит привести несколько примеров, которые показывают, насколько полезна эта концепция.

Замена циклов for при переборе массивов

`forEach()`, `map()`, `filter()` и `reduce()` — функции высшего порядка, эффективно работающие с массивами. Вы уже видели, как они объединяются для построения сложных вычислений.

См. с. 291, 331, 339 и 344.

Эффективная работа с вложенными данными

Изменение глубоко вложенного значения требует создания копий данных на пути к значению, которое требуется изменить. Мы разработали `update()` и `nestedUpdate()` — функции высшего порядка, которые позволяют с хирургической точностью применить операцию к конкретному значению независимо от глубины его вложенности.

См. с. 396 и 419.

Применение копирования при записи

Наш подход копирования при записи провоцирует появление большого количества дублирующегося кода. Мы копируем, изменяем и возвращаем. Функции `withArrayCopy()` и `withObjectCopy()` предоставляли возможность применения любой операции (обратного вызова) в контексте подхода копирования при записи. Это отличный пример закрепления данного подхода в коде.

См. с. 306 и 310.

Закрепление политики регистрации ошибок `try/catch`

Мы создали функцию с именем `wrapLogging()`, которая получает любую функцию и возвращает функцию, которая делает то же самое, но с перехватом и регистрацией ошибок. Это пример функции, преобразующей поведение другой функции.

См. с. 396 и 419.

Итоги главы

В нескольких последних главах мы применяли два метода рефакторинга, чтобы прийти к трем главным функциональным инструментам для работы с массивами данных. В этой главе вы узнали, как применять те же методы к операциям с вложенными данными. Для работы с данными произвольной глубины вложенности использовалась рекурсия. Мы также обсудили, как такие мощные операции с данными влияют на проектирование и как избежать проблем, которые приходят с этой мощностью.

Резюме

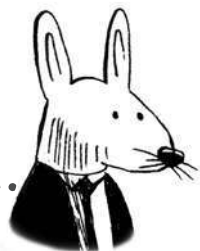
- `update()` — функциональный инструмент, реализующий стандартный паттерн. Он позволяет изменить значение внутри объекта без необходимости извлекать его вручную, а затем записывать обратно.
- `nestedUpdate()` — функциональный инструмент для работы с глубоко вложенными данными. Функция очень полезна для изменения значения, если вам известна ведущая к нему последовательность ключей.
- Циклы (перебор) часто бывают понятнее рекурсии. С другой стороны, рекурсия яснее и понятнее при работе с вложенными данными.
- Рекурсия с помощью стека вызовов сохраняет информацию о том, из какой точки было передано управление при вызове функции. Это позволяет структуре рекурсивной функции воспроизводить структуру вложенных данных.
- Глубокая вложенность затрудняет понимание кода. При работе с глубоко вложенными данными часто приходится помнить все структуры данных и их ключи на пути к нужному значению.

- Абстрактные барьеры могут использоваться для структур данных, чтобы вам не приходилось запоминать много лишней информации. Они могут упростить работу с глубокими структурами.

Что дальше?

Итак, теперь вы достаточно хорошо понимаете смысл первоклассных значений и функций высшего порядка, и мы начнем применять их к одной из самых сложных областей современного программирования: распределенным системам.

Нравится нам это или нет, но большинство современных программных продуктов содержит как минимум внешнюю (frontend) и внутреннюю (backend) подсистему. Совместное использование ресурсов (в том числе и данных) между внутренней и внешней подсистемой может быть нетривиальной задачей. Концепции первоклассных значений и функций высшего порядка помогут нам в ее решении.



В этой главе

- ✓ Рисование временных диаграмм на основании кода.
- ✓ Чтение временных диаграмм для поиска ошибок.
- ✓ Улучшение структуры кода за счет сокращения совместного использования ресурсов между временными линиями.

В этой главе мы начнем использовать временные диаграммы для представления последовательностей действий во времени. Они помогут вам понять, как работает программа. Такие диаграммы особенно полезны в распределенных системах, например при взаимодействии веб-клиента с веб-сервером. Временные диаграммы упрощают диагностику и прогнозирование ошибок. После этого можно переходить к разработке решения.



Осторожно, ошибка!

Служба поддержки MegaMart получает множество телефонных звонков о том, что в корзине выводится неправильная общая стоимость. Клиенты добавляют товары в корзину, приложение сообщает им, что покупка стоит \$X, но при оформлении заказа с них списывается сумма \$Y. Это неприятно, и покупатели недовольны. Удастся ли нам найти ошибку?

При медленных кликах ошибка не воспроизводится

MegaMart

\$0

\$6

Buy Now

\$2

Buy Now

Начинаем с пустой корзины

Кликаем...

Ждем...

MegaMart

\$8

\$6

Buy Now

\$2

Buy Now

\$6 + \$2 за доставку

Кликаем (снова)

Ждем...

MegaMart

\$14

\$6

Buy Now

\$2

Buy Now

Получается \$6 + \$6 (за две пары туфель) + \$2 за доставку (обе пары помещаются в одну коробку). \$14 — правильный результат

Медленные взаимодействия с приложением работают. С другой стороны, быстрые клики приводят к другому результату. Посмотрим, как это происходит.



Пробуем кликать вдвое чаще

Клиенты сообщают, что ошибка возникала тогда, когда они быстро кликали на кнопках приложения.

Чтобы воспроизвести ошибку, следует очень быстро дважды кликнуть на кнопке покупки. Убедимся в этом:

MegaMart \$0

Начинаем с пустой корзины

Быстро кликаем два раза

Ждем...

MegaMart \$16

Вот оно! Ошибка! Здесь должно быть \$14

Мы протестировали приложение еще несколько раз и получили разные результаты

Тот же сценарий (добавление туфель два раза) был повторен несколько раз. Мы получили следующие ответы:

- \$14 ← Правильный ответ
- \$16
- \$22

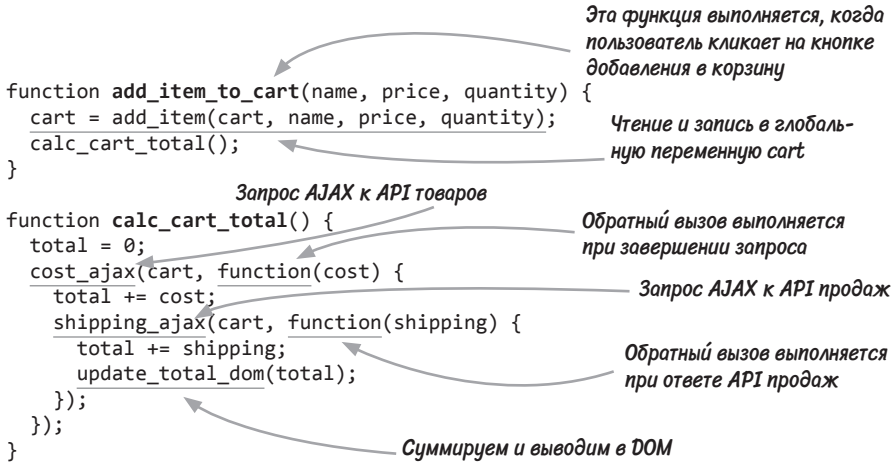


Пицца для ума

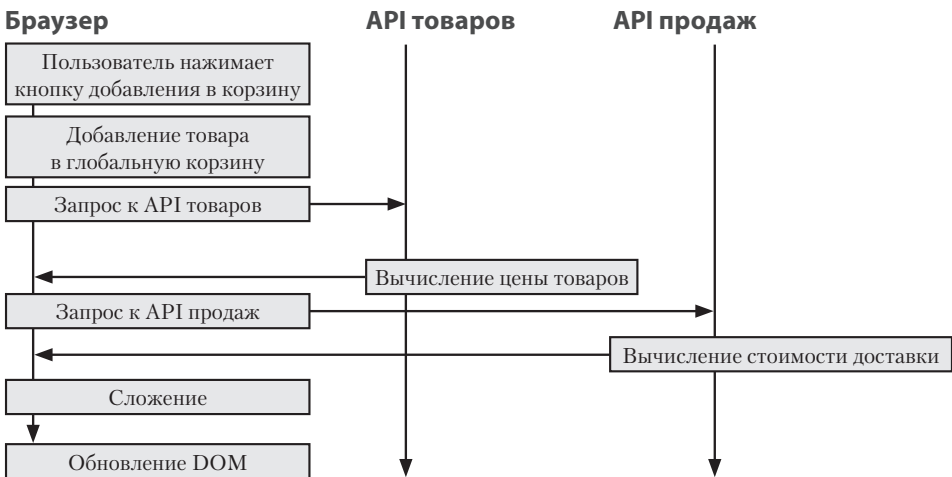
Похоже, проблема возникает из-за быстрых кликов. Как вы думаете, что происходит?

Внимательно прочитаем код, чтобы понять суть ошибки

Ниже приведен соответствующий код кнопок добавления в корзину: `add_item_to_cart()` — функция-обработчик, вызываемая при нажатии кнопки.



Также полезно взглянуть на традиционную диаграмму сценариев использования. Следует заметить, что код взаимодействует с двумя разными API последовательно:



К сожалению, и код, и диаграммы сценариев использования выглядят правильными. Собственно, система работает правильно, если вы добавите один товар в корзину, а потом немного подождете, прежде чем добавлять следующий. Необходимо понять, как работает система без ожидания, когда две операции выполняются одновременно. Временные диаграммы на следующей странице покажут нам это.

Временные диаграммы показывают, что происходит с течением времени

Ниже приведена временная диаграмма для двух кликов, выполняемых очень быстро. *Временная линия* представляет последовательность действий. *Временная диаграмма* представляет последовательность действий во времени в графическом виде. На ней видно, как действия взаимодействуют и вмешиваются в работу друг друга.

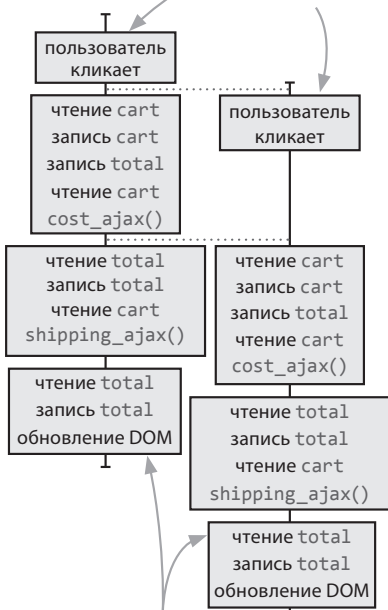


Загляни в словарь

Временная линия — последовательность действий во времени. В вашей системе могут одновременно выполняться несколько таких последовательностей.

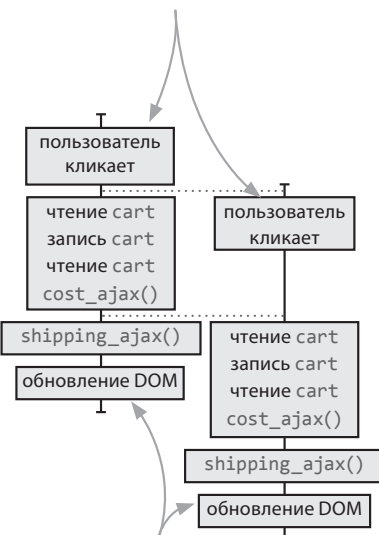
Время

*Временная диаграмма для двух быстрых кликов.
В этой главе вы научитесь строить такие диаграммы*



Проблема кроется здесь! В этой главе вы узнаете, как это понять

*Улучшенная временная диаграмма,
к которой мы придем к концу главы*



*Уже лучше, но и здесь остается ошибка,
которую мы исправим в следующей главе*

Хотите верье, хотите нет, но на диаграмме слева четко проявляется проблема, вызывающая неправильное поведение. В этой главе вы узнаете, как строить временные диаграммы на основании кода. Вы научитесь читать диаграммы, чтобы видеть проблемы с синхронизацией. И мы исправим эту ошибку (большой частью!), используя некоторые принципы работы временных диаграмм.

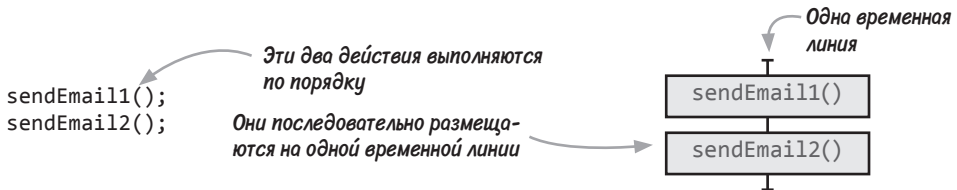
Вам предстоит многое узнать, а разгневанные клиенты ждать не хотят! Давайте научимся строить такие диаграммы.

Два фундаментальных принципа временных диаграмм

Временные диаграммы показывают, какие действия выполняются последовательно, а какие — параллельно. Благодаря этому можно получить хорошее представление о том, как работает ваш код: правильно или неправильно. Существуют два фундаментальных правила, которые помогают преобразовать код во временные диаграммы. Эти правила представлены ниже.

1. Если два действия выполняются по порядку, разместите их на одной временной линии

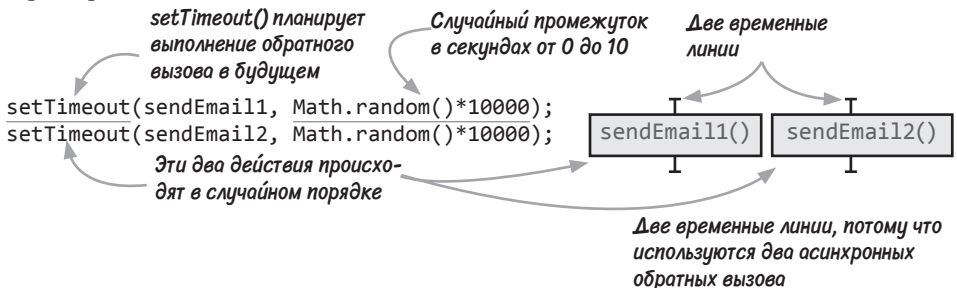
Пример:



Только действия должны размещаться на временных линиях. Вычисления можно опустить, потому что они не зависят от времени их выполнения.

2. Если два действия могут выполняться одновременно или без определенного порядка, они размещаются на разных временных линиях

Пример:



Разные временные линии используются при выполнении действий в разных программных потоках, процессах, машинах или асинхронных обратных вызовах. В данном случае имеем два асинхронных обратных вызова. Так как величина тайм-аута случайна, мы не знаем, какой из них будет выполнен первым.

Краткая сводка

1. Действия могут выполняться последовательно или параллельно.
2. Последовательные действия размещаются на одной временной линии, одно за другим.
3. Параллельные действия размещаются на разных временных линиях рядом друг с другом.

Если вы умеете применять эти правила, то для преобразования кода в диаграмму достаточно понимать, как происходит выполнение этого кода во времени.



Ваш ход

Приведенный ниже код моделирует обед. Нарисуйте соответствующую ему временную диаграмму. Каждая функция, вызываемая `dinner()`, является действием.

```
function dinner(food) {
  cook(food);
  serve(food);
  eat(food);
}
```

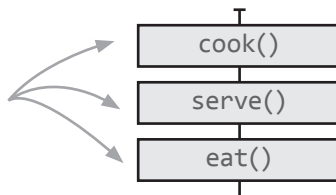
Нарисуйте здесь диаграмму



Ответ

```
function dinner(food) {
  cook(food);
  serve(food);
  eat(food);
}
```

Каждый вызов является отдельным действием. Все действия выполняются по порядку, поэтому они размещаются на одной временной линии





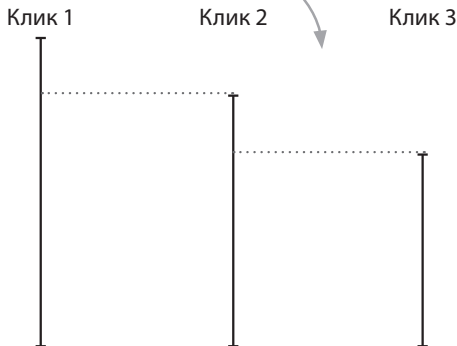
Ваш ход

Приведенный ниже код моделирует трех людей, обедающих по отдельности. Каждый обед реализуется асинхронным вызовом: `dinner()` выполняется при нажатии кнопки. Завершите временную диаграмму, соответствующую трем быстрым кликам на кнопке.

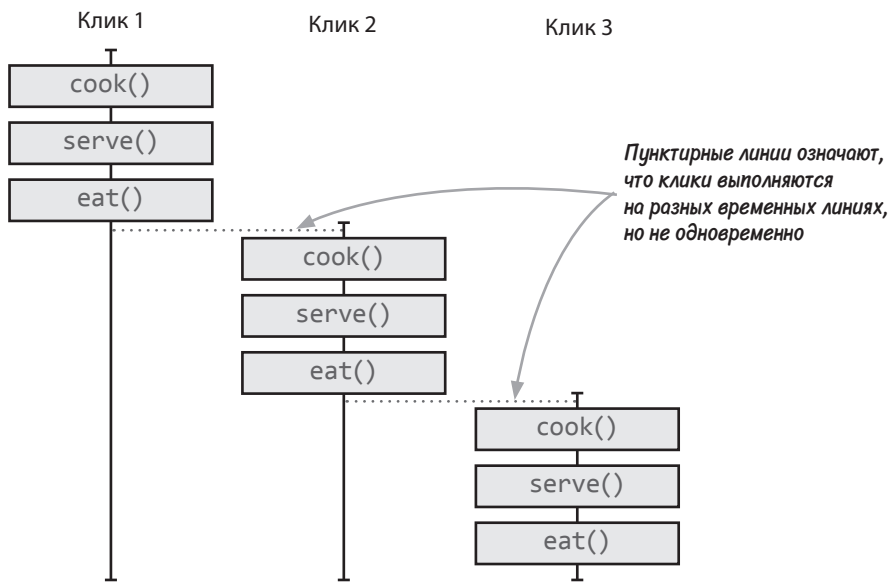
```
function dinner(food) {
  cook(food);
  serve(food);
  eat(food);
}

button.addEventListener('click', dinner);
```

Завершите временную диаграмму для трех кликов



Пунктирные линии означают, что клики выполняются на разных временных линиях, но не одновременно

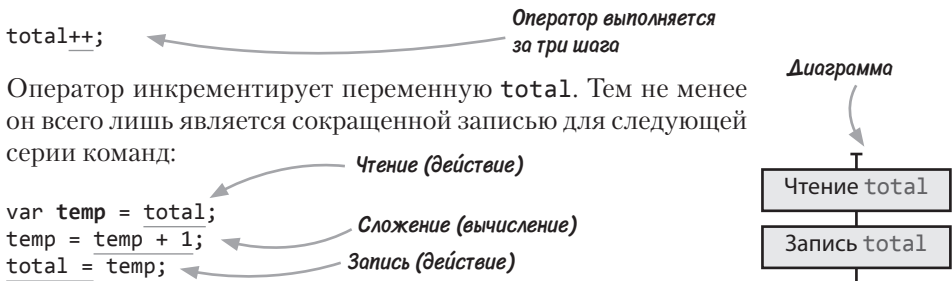
**Ответ**

Две неочевидные детали порядка действий

Важно идентифицировать каждое действие и понять, в каком порядке они выполняются. В разных языках подробности выполнения реализованы по-разному, и JavaScript не является исключением. Об этом уже упоминалось в части I, но стоит сказать снова, потому что эти подробности сыграют важную роль на временных диаграммах.

1. ++ и += в действительности состоят из трех шагов

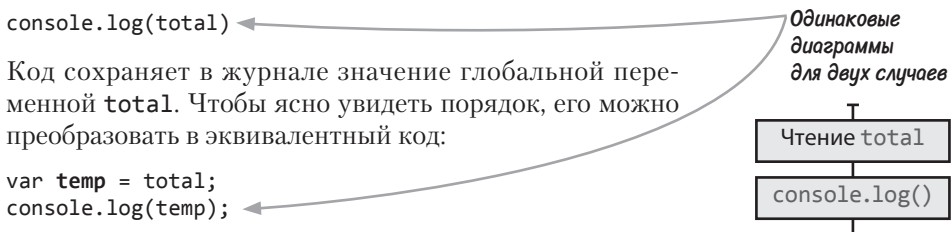
Два оператора JavaScript (и аналогичных языков, таких как Java, C, C++ и C#, среди прочих) записываются очень кратко. Но за краткостью скрывается тот факт, что операция состоит из трех шагов. В следующем примере оператор инкремента применяется к глобальной переменной:



Сначала программа читает переменную `total`, затем прибавляет к ней 1, после чего записывает `total` обратно. Если `total` является глобальной переменной, шаги 1 и 3 являются действиями. Второй шаг (увеличение на 1) является вычислением, поэтому на диаграмму он не наносится. Это означает, что для `total++` или `total+=3` на диаграмму необходимо нанести два разных действия, чтение и запись.

2. Аргументы выполняются перед вызовом функции

Если вызвать функцию с аргументом, аргумент выполняется раньше той функции, которой он передается. Таким образом определяется порядок выполнения, который должен отображаться на временной диаграмме. В следующем примере сохраняется (действие) значение глобальной переменной (действие):



Этот фрагмент наглядно показывает, что чтение глобальной переменной `total` выполняется в первую очередь. Очень важно разместить все действия на диаграмме в правильном порядке.

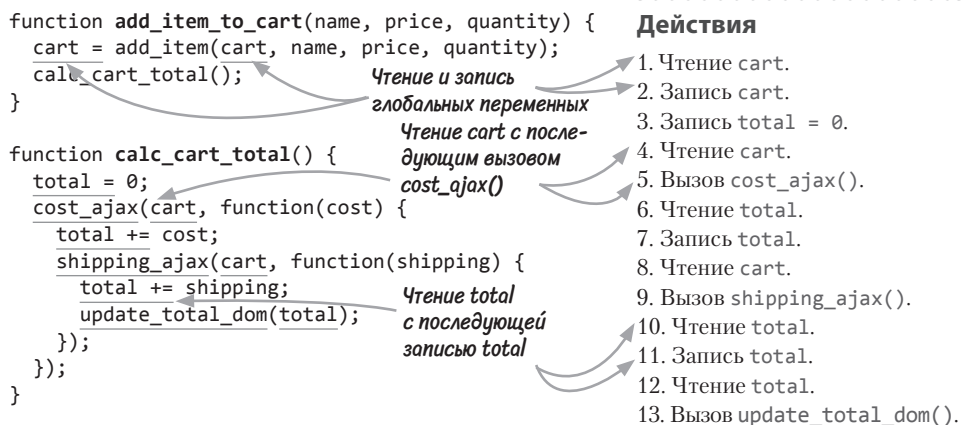
Построение временной линии добавления товара в корзину: шаг 1

Вы только что узнали, что по временной диаграмме можно определить два важных показателя: что выполняется последовательно, а что — параллельно. Теперь нарисуем диаграмму для кода добавления в корзину. Построение временной диаграммы состоит из трех шагов.

1. Идентификация действий.
2. Рисование всех действий на диаграмме (как последовательных, так и параллельных).
3. Упрощение диаграммы на основе платформенно-зависимых знаний.

1. Идентификация действий

В следующем коде подчеркнуты все действия (вычисления игнорируются):



Этот короткий фрагмент содержит 13 действий. Следует также понимать, что в нем содержатся два асинхронных обратных вызова. Один обратный вызов передается `cost_ajax()`, а другой — `shipping_ajax()`. Как представлять на диаграммах обратные вызовы, мы пока не разобрались.

Отложим этот код (помните, мы всего лишь завершили шаг 1) и вернемся к нему после того, как вы научитесь представлять обратные вызовы на диаграммах.

Асинхронным вызовам необходимы новые временные линии

Мы только что видели, что асинхронные вызовы работают на новых временных линиях. Важно хорошо понимать, как это происходит, поэтому несколько ближайших страниц будут посвящены тонкостям асинхронного ядра JavaScript. Прочитайте эти страницы, если вас интересует эта тема. А пока мы поговорим о том, почему используем пунктирные линии.

Ниже приведен пояснительный код, который сохраняет пользователя и документ и управляет выводом круговых индикаторов ожидания:

```

saveUserAjax(user, function() {
  setUserLoadingDOM(false);
});
setUserLoadingDOM(true);
saveDocumentAjax(document, function() {
  setDocLoadingDOM(false);
});
setDocLoadingDOM(true);

```

Сохранение пользователя на сервере (ajax)
 Индикатор загрузки для пользователя скрывается
 Индикатор загрузки для пользователя отображается
 Сохранение документа на сервере (ajax)
 Индикатор загрузки для документа скрывается
 Индикатор загрузки для документа отображается

Этот код интересен тем, что отдельные строки кода выполняются не в том порядке, в котором они написаны. Разберем первые два шага построения временной диаграммы для этого кода.

Начнем с подчеркивания всех действий. Предполагается, что переменные `user` и `document` являются локальными, поэтому их чтение не является действием:

```

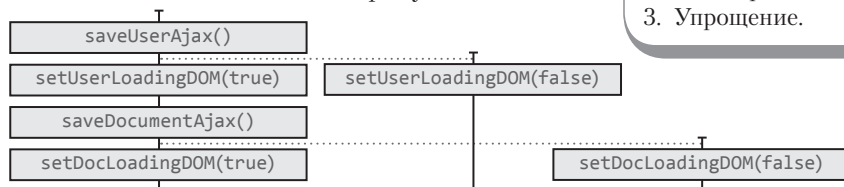
saveUserAjax(user, function() {
  setUserLoadingDOM(false);
});
setUserLoadingDOM(true);
saveDocumentAjax(document, function() {
  setDocLoadingDOM(false);
});
setDocLoadingDOM(true);

```

Действия

1. `saveUserAjax()`
2. `setUserLoadingDOM(false)`
3. `setUserLoadingDOM(true)`
4. `saveDocumentAjax()`
5. `setDocLoadingDOM(false)`
6. `setDocLoadingDOM(true)`

На шаге 2 происходит непосредственное рисование диаграммы. На нескольких следующих страницах мы поэтапно рассмотрим процесс ее создания. А пока посмотрите, как она будет выглядеть после завершения. Если диаграмма вам понятна, объяснение можно пропустить.



Три шага построения диаграммы

1. Идентификация действий.
2. Отображение действий на диаграмме.
3. Упрощение.

Разные языки, разные потоковые модели

JavaScript использует однопоточную асинхронную модель. Для каждого нового асинхронного обратного вызова создается новая временная линия. Впрочем, многие платформы не используют эту потоковую модель. Рассмотрим потоковую модель и некоторые другие распространенные механизмы работы потоков в языках.

*В JavaScript
есть асинхронные
обратные вызовы. А если
я работаю на языке,
в котором их нет?*



Дженна из команды разработки

Однопоточная синхронная модель

Некоторые языки или платформы по умолчанию не поддерживают многопоточное выполнение. Например, так работает PHP, если не импортировать библиотеку потоков. Все происходит по порядку. Когда вы выполняете любую разновидность ввода/вывода, вся программа блокируется в ожидании его завершения. И хотя такая модель ограничивает то, что вы можете сделать, при таких ограничениях становится очень легко рассуждать о системе. Один поток означает одну временную линию, но могут появиться и другие временные линии при взаимодействии с другим компьютером (как при использовании API). Такие временные линии не могут совместно использовать память, поэтому вы исключаете обширный класс общих ресурсов.

Однопоточная асинхронная модель

JavaScript использует один поток. Если вы хотите реагировать на ввод пользователя, читать файлы или выдавать сетевые вызовы (любая разновидность ввода/вывода), используйте асинхронную модель. Как правило, это означает, что вы передаете обратный вызов, который будет активизироваться с результатом операции ввода/вывода. Так как продолжительность операции ввода/вывода неизвестна заранее, обратный вызов будет активизирован в непредсказуемый момент в будущем. Вот почему асинхронный вызов создает новую временную линию.

Многопоточная модель

Java, Python, Ruby, C и C# (среди многих других) поддерживают многопоточное выполнение. Многопоточность создает больше всего трудностей в программировании, потому что она не устанавливает почти никаких ограничений на порядок

выполнения. Каждый новый поток создает новую временную линию. Языки этих категорий допускают неограниченное чередование потоков. Для преодоления проблем синхронизации приходится использовать такие конструкции, как блокировки (locks), которые не позволяют двум потокам одновременно выполнять код, защищенный блокировкой. Блокировки позволяют в определенной степени управлять порядком выполнения.

Процессы с передачей сообщений

Erlang и Elixir используют потоковую модель, которая обеспечивает возможность одновременного выполнения многих разных процессов. Каждый процесс представлен отдельной временной линией. Процессы не используют память совместно. Вместо этого они взаимодействуют с помощью сообщений. Уникальность этой модели в том, что процессы выбирают, какое сообщение они будут обрабатывать следующим. В этом они отличаются от вызовов методов в Java или других ОО-языках. Действия отдельных временных линий чередуются, но из-за того, что они не используют память совместно, обычно они не работают с общими ресурсами, а это значит, что вам не придется беспокоиться о большом количестве возможных вариантов упорядочения.

Поэтапное построение временной линии

Окончательный результат построения временной линии вы уже видели, но будет полезно проанализировать процесс его создания по одной строке кода. Еще раз приведу код и содержащиеся в нем действия:

```
1 saveUserAjax(user, function() {
2   setUserLoadingDOM(false);
3 });
4 setUserLoadingDOM(true);
5 saveDocumentAjax(document, function() {
6   setDocLoadingDOM(false);
7 });
8 setDocLoadingDOM(true);
```

Действия

1. saveUserAjax()
2. setUserLoadingDOM(false)
3. setUserLoadingDOM(true)
4. saveDocumentAjax()
5. setDocLoadingDOM(false)
6. setDocLoadingDOM(true)

Код JavaScript в общем случае выполняется сверху вниз, поэтому начнем с верхней строки с номером 1. Здесь все просто: нужна новая временная линия, потому что на диаграмме еще нет ни одной.

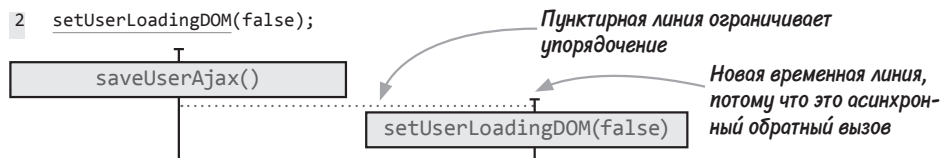
```
1 saveUserAjax(user, function() {
```



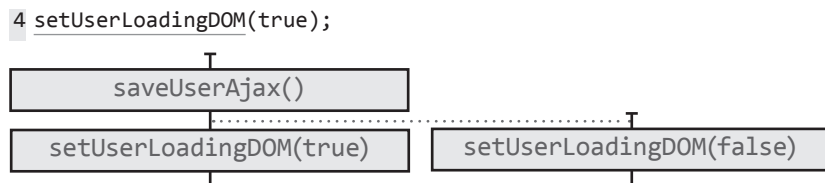
Три шага построения диаграммы

1. Идентификация действий.
2. Отображение действий на диаграмме.
3. Упрощение.

Строка 2 является частью обратного вызова. Этот обратный вызов является асинхронным, и это означает, что он будет активизирован когда-то в будущем, когда завершится запрос. Ему нужна новая временная линия. Мы также рисуем пунктирную линию, которая показывает, что обратный вызов будет активизирован после функции ajax. Это логично, потому что ответ не может поступить до отправки запроса.



В строке 3 никаких действий нет, переходим к строке 4. В ней выполняется `setUserLoadingDOM(true)`. Но где его разместить? Так как это не обратный вызов, он происходит в исходной временной линии. Разместим его здесь, сразу же после пунктирной линии:

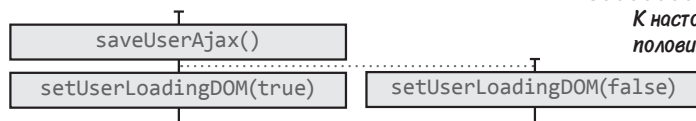


Мы уже нанесли половину действий на диаграмму. Для удобства приведу код, действия и диаграмму:

```

1 saveUserAjax(user, function() {
2   setUserLoadingDOM(false);
3 });
4 setUserLoadingDOM(true);
5 saveDocumentAjax(document, function() {
6   setDocLoadingDOM(false);
7 });
8 setDocLoadingDOM(true);

```



Действия

1. saveUserAjax()
2. setUserLoadingDOM(false)
3. setUserLoadingDOM(true)
4. saveDocumentAjax()
5. setDocLoadingDOM(false)
6. setDocLoadingDOM(true)

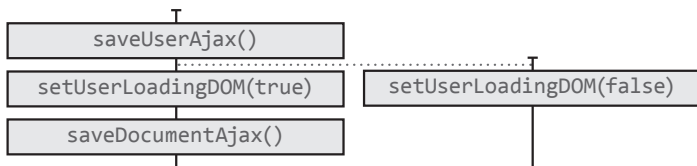
К настоящему моменту нарисована половина

Строка 4 завершена. Перейдем к главе 5, в которой выполняется еще один вызов ajax. Вызов ajax не является обратным вызовом, и, следовательно, он должен быть частью исходной временной линии. Разместим его под последним изображенным действием:

Три шага построения диаграммы

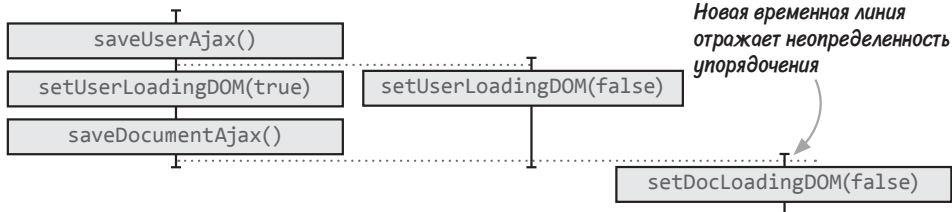
1. Идентификация действий.
2. Отображение действий на диаграмме.
3. Упрощение.

```
5 saveDocumentAjax(document, function() {
```



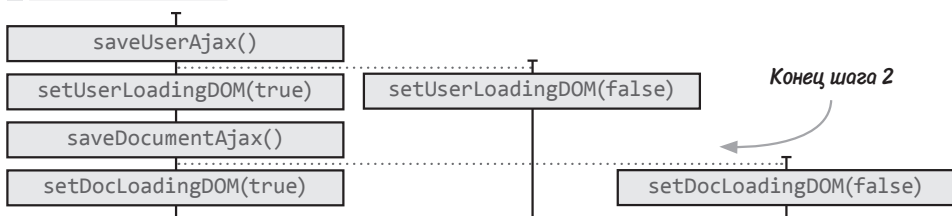
Он является частью асинхронного обратного вызова, который создает новую временную линию. Эта временная линия начинается где-то в будущем, когда вернется ответ. Мы не знаем, когда это будет, потому что работа сети непредсказуема. На диаграмме эта неопределенность отражается новой временной линией:

```
6 setDocLoadingDOM(false);
```



В строке 8 содержится последнее действие. Оно принадлежит исходной временной линии:

```
8 setDocLoadingDOM(true);
```



Мы завершили шаг 2 для этого кода. Шагом 3 займемся позднее, а пока вернемся к коду добавления товара в корзину и завершим шаг 2.

Изображение временной линии добавления товара в корзину: шаг 2

Несколько страниц назад мы идентифицировали все действия в коде. Также были отмечены два асинхронных обратных вызова. Настало время сделать шаг 2: рисование действий на диаграмме. Вот что мы имеем после идентификации действий:

Три шага построения диаграммы

1. Идентификация действий.
2. Отображение действий на диаграмме.
3. Упрощение.

```
function add_item_to_cart(name, price, quantity) {
    cart = add_item(cart, name, price, quantity);
    calc_cart_total();
}

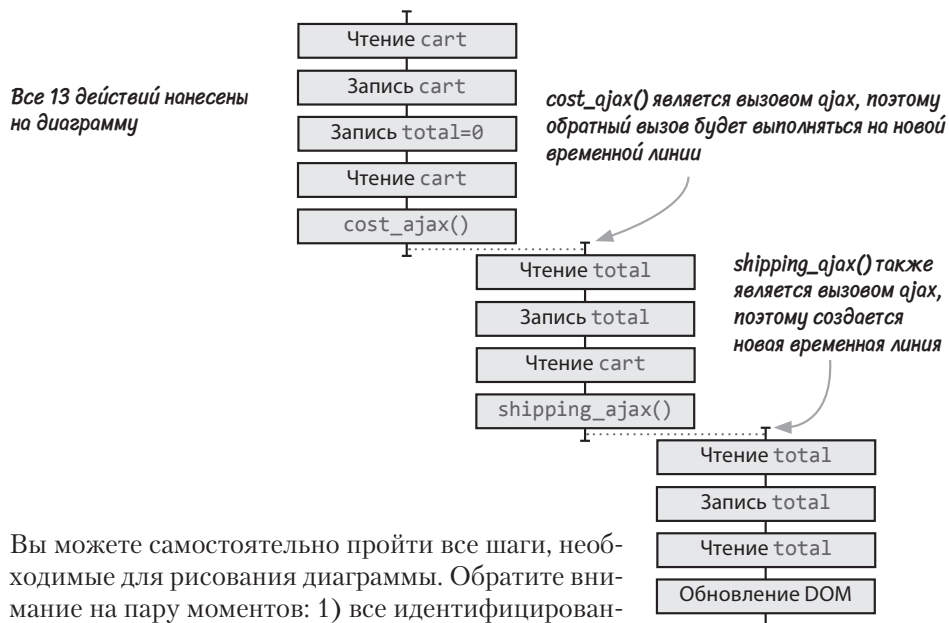
function calc_cart_total() {
    total = 0;
    cost_ajax(cart, function(cost) {
        total += cost;
        shipping_ajax(cart, function(shipping) {
            total += shipping;
            update_total_dom(total);
        });
    });
}
```

Действия

1. Чтение cart.
2. Запись cart.
3. Запись total = 0.
4. Чтение cart.
5. Вызов cost_ajax().
6. Чтение total.
7. Запись total.
8. Чтение cart.
9. Вызов shipping_ajax().
10. Чтение total.
11. Запись total.
12. Чтение total.
13. Вызов update_total_dom().

2. Рисование всех действий (как последовательных, так и параллельных)

Все действия определены. На следующем шаге мы нанесем их на диаграмму по порядку. Помните: для обратных вызовов ajax, которых у нас два, необходимы новые временные линии.



Вы можете самостоятельно пройти все шаги, необходимые для рисования диаграммы. Обратите внимание на пару моментов: 1) все идентифицированные действия (их было 13) нанесены на диаграмму; 2) каждый асинхронный обратный вызов (их было два) привел к появлению новой временной линии.

Прежде чем переходить к шагу 3, сосредоточимся на том, что можно узнать из этой диаграммы.

Временные диаграммы отражают две разновидности последовательного кода

Существуют два варианта последовательного выполнения кода. Обычно любое действие может чередоваться между любыми другими двумя действиями с другой временной линией. Тем не менее в некоторых обстоятельствах такое чередование можно заблокировать. Например, в потоковой модели JavaScript синхронные действия не чередуются. Другие способы предотвращения чередования будут описаны позднее.

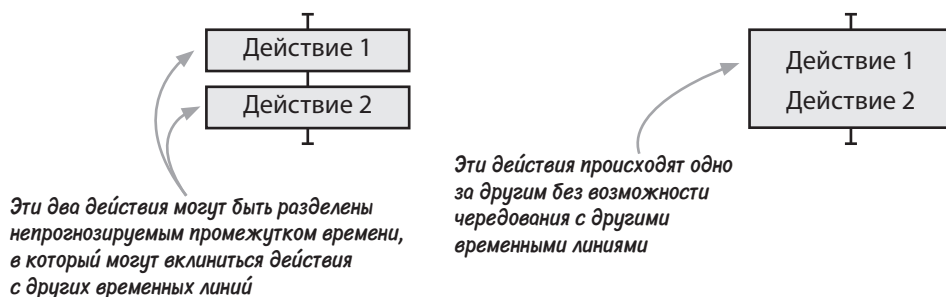
Таким образом, последовательное выполнение возможно в двух вариантах, и на временной диаграмме можно представить оба варианта.

Выполнение с чередованием

Между двумя действиями может пройти произвольный промежуток времени. Каждое действие представляется прямоугольником, а время между действиями — линией. Линию можно нарисовать короткой или длинной, но в любом случае она означает одно: между действием 1 и действием 2 может пройти неизвестный промежуток времени.

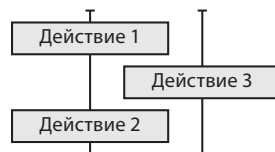
Выполнение без чередования

Два действия выполняются одно за другим, и какая-то причина гарантирует, что между ними ничего выполняться не может. Что это за причина? Это может быть исполнительная среда или какие-то хитроумные средства программирования (о них будет рассказано позднее). Действия рисуются в одном прямоугольнике.



Загляни в словарь

Действия на разных временных линиях могут чередоваться, если они могут выполняться между другими действиями. Такая ситуация возникает при одновременном выполнении нескольких потоков.



Эти две временные линии выполняются по-разному. Линия слева допускает чередование, то есть между действием 1 и действием 2 может быть выполнено действие 3 (здесь не показано). С временной линией справа это невозможно.

Левая временная линия (действия с чередованием) содержит два прямоугольника, тогда как на правой временной линии прямоугольник только один. Короткими временными линиями удобнее управлять. Нам хотелось бы, чтобы прямоугольников было мало, а не много.

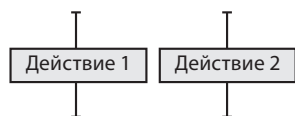
Мы еще не размещали несколько действий в одном прямоугольнике. Обычно это делается на шаге 3, до которого мы еще не добрались (но вскоре доберемся!). Просто нужно еще немного узнать о том, какую информацию можно получить из диаграммы.

Временные диаграммы отражают неопределенность в упорядочении параллельного кода

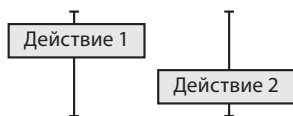
Кроме представления последовательного кода, временные диаграммы также выражают неопределенность упорядочения параллельного кода.

Параллельный код представляется временными линиями, которые рисуются рядом друг с другом. Тем не менее такое расположение вовсе не означает, что действие 1 и действие 2 будут выполняться одновременно. Два действия на параллельных временных линиях могут выполняться в трех вариантах порядка. В общем случае возможны все три варианта.

Одновременное выполнение



Сначала левое

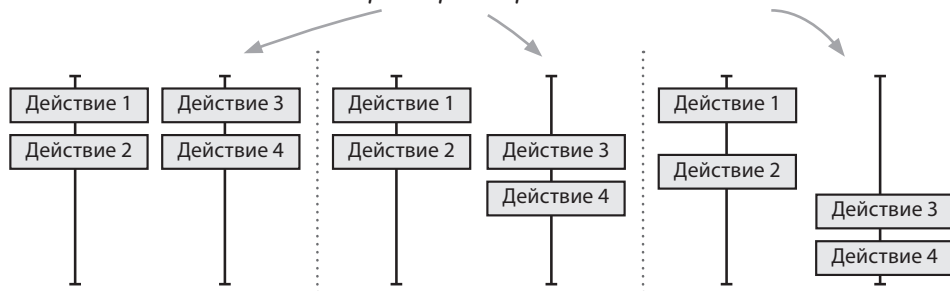


Сначала правое



Во время чтения временной диаграммы вы должны видеть все три порядка, независимо от длины линий и выравнивания действий. Все следующие диаграммы означают одно и то же, хотя и выглядят по-разному:

Эти три диаграммы представляют одно и то же



Умение видеть эти варианты и воспринимать их как эквивалентные — важный навык для чтения временных диаграмм. Вы должны уметь мысленно представлять *возможные варианты упорядочения*, особенно те, которые могут оказаться проблематичными. Диаграмму можно нарисовать по-другому, так, чтобы подчеркнуть один возможный порядок для наглядности.

Две временные линии, с одним прямоугольником каждая, могут иметь три варианта упорядочения. С увеличением длины и количества временных линий количество вариантов упорядочения стремительно растёт.

Возможные варианты упорядочения также зависят от потоковой модели вашей платформы. Этот факт тоже важно отразить на временной диаграмме, и мы займемся этим на шаге 3.



Загляни в словарь

Несколько временных линий могут выполняться в разных сочетаниях в зависимости от выбора времени. Эти сочетания называются *вариантами упорядочения*. Одна временная линия имеет только один вариант упорядочения.

Принципы работы с временными линиями

При работе с временными линиями действуют определенные принципы, которые помогают совершенствовать код, чтобы он стал более понятным, а работать с ним стало проще. Не забывайте, что один из факторов сложности систем — количество возможных вариантов упорядочения, которые необходимо учитывать. И хотя эти пять принципов действуют всегда, в этой главе мы сосредоточимся на первых трех. Остальные рассматриваются в главах 16 и 17.

1. Чем меньше временных линий, тем проще

Самая простая система состоит из одной временной линии. Каждое действие выполняется сразу же после предшествующего. Тем не менее в современных системах приходится иметь дело с несколькими временными линиями. Многопоточные приложения, асинхронные обратные вызовы, клиент-серверные взаимодействия — во всех этих ситуациях используются множественные временные линии.

Каждая новая временная линия радикально усложняет понимание системы. Если вам удастся сократить количество временных линий (t в формуле справа), это

Формула для определения количества возможных вариантов упорядочения

$$O = \frac{(ta)!}{(a!)^t}$$

Количество временных линий

Количество действий на одну временную линию

Возможные варианты упорядочения

! — факториал

очень сильно упростит вашу задачу. К сожалению, часто мы не можем управлять количеством временных линий.

2. Чем короче временные линии, тем проще

Другая возможная мера — сокращение количества шагов на каждой временной линии. Если вам удастся устранить шаги (уменьшить a в формуле справа), это также позволит радикально сократить количество возможных вариантов упорядочения.

3. Чем меньше совместного использования ресурсов, тем проще

Если два шага на разных временных линиях не используют ресурсы совместно, порядок их выполнения роли не играет. При этом сокращается не количество возможных вариантов упорядочения, а количество вариантов, которые вам приходится учитывать. При рассмотрении двух временных линий достаточно учитывать только те шаги, в которых ресурсы используются совместно.

4. Координируйте совместное использование ресурсов

Даже после исключения всех возможных совместно используемых ресурсов у вас останутся ресурсы, от которых избавиться не удастся. Необходимо позаботиться о том, чтобы совместное использование ресурсов разными временными линиями было безопасным. А это означает, что вы должны позаботиться о том, чтобы они получали управление в правильном порядке. Координация между временными линиями означает исключение возможных вариантов упорядочения, которые не дают правильного результата.

5. Рассматривайте время как первоклассную концепцию

Упорядочить действия и правильно выбрать момент для их выполнения непросто. Для упрощения задачи можно создать объекты, манипулирующие временной линией. Примеры такого рода встретятся вам в следующих двух главах.

В этой и нескольких ближайших главах применение этих принципов позволит избавиться от ошибок и упростить написание кода.

Однопоточная модель в JavaScript

Потоковая модель JavaScript уменьшает масштаб проблем, связанных с совместным использованием ресурсов между временными линиями. Так как JavaScript использует только один главный поток, большинству действий не нужны отдельные прямоугольники на временной линии.

Рассмотрим пример. Представьте следующий код Java:

```
int x = 0;

public void addToX(int y) {
    x += y;
}
```


В языке Java, если переменная совместно используется двумя потоками, выполнение операции `+=` в действительности состоит из трех шагов.

1. Чтение текущего значения.
2. Прибавление к нему числа.
3. Сохранение нового значения на прежнем месте.

Операция `+` является вычислением, поэтому размещать ее на временной линии не нужно. Это означает, что два потока, одновременно выполняющие метод `addToX()`, могут чередоваться разными способами, что приведет к разным возможным ответам. Поточковая модель Java работает по этому принципу.

Тем не менее в JavaScript существует только один поток. Следовательно, в нем этой конкретной проблемы нет. Вместо этого в JavaScript поток остается в вашем распоряжении на то время, пока вы продолжаете пользоваться им. Следовательно, вы можете сколько угодно выполнять чтение и запись без чередования. Кроме того, два действия не могут выполняться одновременно.

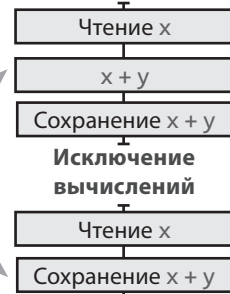
В стандартном императивном программировании (например, при чтении и записи в общие переменные) нет временных линий, о которых вам пришлось бы беспокоиться.

Но стоит добавить асинхронный вызов, как проблема немедленно проявится снова. Асинхронные вызовы активизируются исполнительской средой в неизвестный момент в будущем. Это означает, что линии между прямоугольниками могут растягиваться и сокращаться. В JavaScript важно знать, какие операции вы выполняете: синхронные или асинхронные.

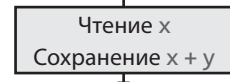
- В JavaScript используется один поток.
- Синхронные действия, такие как изменение глобальной переменной, не могут чередоваться на временных линиях.
- Асинхронные вызовы активизируются исполнительской средой в неизвестный момент в будущем.
- Два синхронных действия не могут выполняться одновременно.

Операция `+` является вычислением (неважно, когда она будет выполнена), поэтому на временной линии ее размещать не нужно

`x += y` состоит из трех шагов



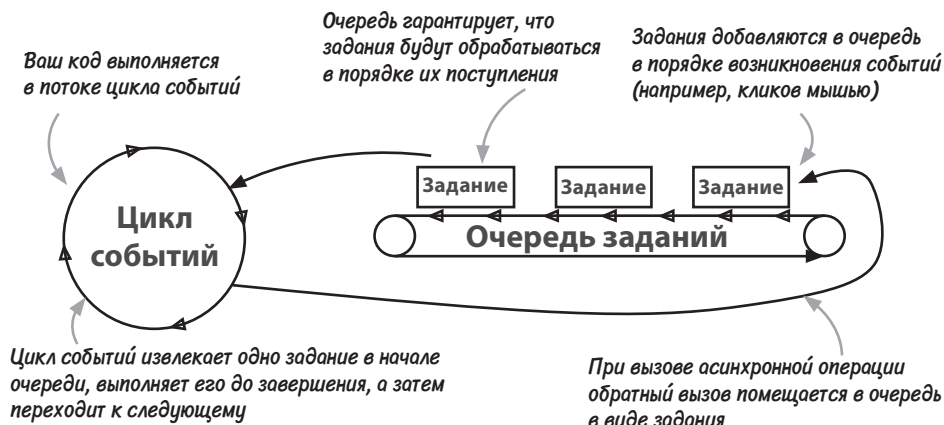
В JavaScript



Так как в JavaScript эти операции не могут чередоваться, они размещаются в одном прямоугольнике

Асинхронная очередь JavaScript

Браузерное ядро JavaScript поддерживает очередь, которая называется *очередью заданий* и обрабатывается *циклом событий*. Цикл событий извлекает одно задание из очереди и выполняет его до завершения, после чего берет следующее задание и выполняет его до завершения... и т. д. Цикл событий выполняется в одном потоке, что исключает одновременное выполнение двух заданий.



Что такое задание

Задания, находящиеся в очереди, состоят из двух частей: данных события и обратного вызова, который обрабатывает это событие. Цикл событий активизирует обратный вызов с данными события как единственным аргументом. Обратные вызовы — это всего лишь функции, которые определяют, что должно выполняться циклом событий. Цикл событий просто выполняет их с передачей данных события в первом аргументе.

Кто ставит задания в очередь

Задания добавляются в очередь в ответ на события. К событиям относятся такие операции, как клики мышью, ввод с клавиатуры или события AJAX. Если назначить кнопке функцию обратного вызова «click», функция обратного вызова и данные события (данные о клике) добавляются в очередь. Так как клики мышью и другие события прогнозировать невозможно, мы говорим, что они поступают *непредсказуемо*. Очередь заданий создает некоторое подобие порядка.

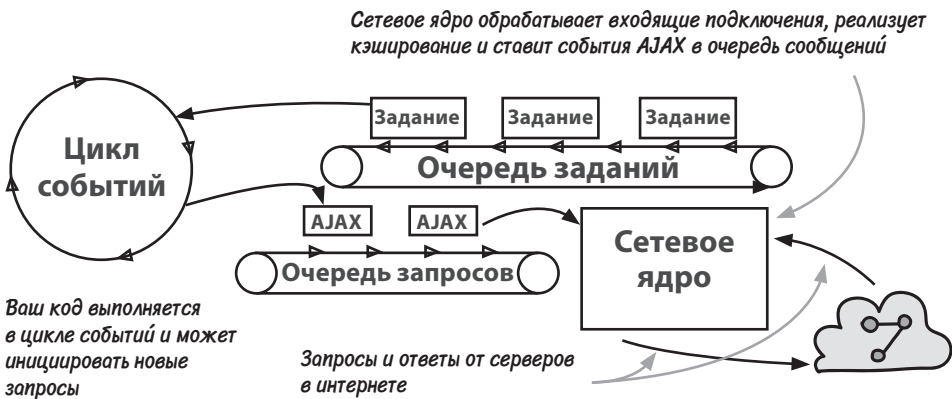
Что делает ядро при отсутствии заданий

Иногда заданий для обработки нет. Цикл событий может простаивать и экономить энергию, а может использовать время для таких служебных операций, как сборка мусора. Это решают разработчики браузера.

AJAX и очередь событий

AJAX — термин для обозначения браузерных веб-запросов. Название происходит от слов «*Asynchronous JavaScript And XML*». Да, это глупое сокращение. И мы не всегда используем XML. Тем не менее термин прижился. В браузере AJAX часто применяется для взаимодействия с сервером.

В этой книге функции, выдающие запросы AJAX, помечаются суффиксом `_ajax`. С такой пометкой вы с первого взгляда можете определить, какие функции являются асинхронными.



Когда вы инициируете запрос AJAX в JavaScript, где-то за кулисами запрос AJAX ставится в очередь для обработки сетевым ядром.

После добавления в очередь ваш код продолжает выполняться. Он совершенно не ожидает запросов — отсюда и «асинхронность» в сокращении AJAX. Во многих языках поддерживаются синхронные запросы, с которыми программа ожидает завершения запроса, прежде чем продолжить работу. Так как сеть работает хаотично, ответы могут приходить с нарушением порядка, поэтому обратные вызовы AJAX будут добавляться в очередь заданий с нарушением порядка.

Если не ожидать завершения запроса, то как получить ответ?

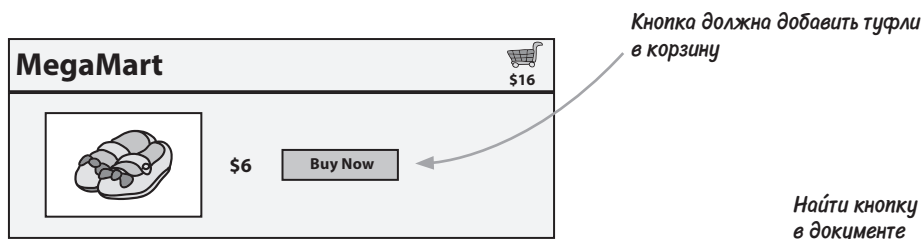
Вы можете регистрировать обратные вызовы для различных событий запросов AJAX. Напоминаю: обратный вызов — всего лишь функция, которая будет вызвана при срабатывании события.

- AJAX означает Asynchronous JavaScript And XML, то есть асинхронный JavaScript и XML.
- AJAX используется для выдачи веб-запросов из JavaScript в браузере.
- Ответы обрабатываются асинхронно обратными вызовами.
- Ответы могут поступать с нарушением исходного порядка.

На протяжении жизненного цикла запроса сетевое ядро выдает множество событий. Особенно часто используются два события: **load** и **error**. Событие **load** вызывается при завершении загрузки ответа. Событие **error** вызывается, когда что-то идет не так. Зарегистрировав обратные вызовы для этих двух событий, вы сможете выполнять код при завершении запроса.

Полный пример с асинхронными вызовами

Ниже приведена простая страница с сайта MegaMart. Рассмотрим последовательность действий при добавлении товаров в корзину кнопкой.



При загрузке страницы HTML необходимо запросить кнопку у страницы:

```
var buy_button = document.getElementById('buy-now-shoes');
```

А затем назначается обратный вызов для кликов на этой кнопке:

```
Иницилируем запрос ajax
buy_button.addEventListener('click', function() {
    add_to_cart_ajax({item: 'shoes'}, function() {
        shopping_cart.add({item: 'shoes'});
        render_cart_icon();
        buy_button.innerHTML = "Buy Now";
    });
    buy_button.innerHTML = "loading";
});
```

Определяем обратный вызов для событий click кнопки

Этот обратный вызов будет выполнен при завершении запроса ajax

Сразу же после инициализации запроса изменить кнопку, чтобы на ней выводилась надпись «loading»

Позднее, после завершения запроса ajax, мы снова обновим пользовательский интерфейс

Затем пользователь нажимает кнопку, что приводит к постановке задания в очередь. Цикл событий обрабатывает задания в очереди, пока не доберется до задания события клика. И тогда он вызывает зарегистрированный ранее обратный вызов.

Обратный вызов добавляет запрос AJAX в очередь запросов, который будет обработан сетевым ядром в будущем. Затем обратный вызов изменит текст кнопки. На этом обратный вызов завершится, а цикл событий извлечет из очереди следующее задание.

Затем запрос AJAX завершится, а сетевое ядро добавит задание в очередь с зарегистрированным нами обратным вызовом. Обратный вызов добирается до начала очереди, после чего выполняется. Он обновляет корзину, отображает значок корзины и возвращает текст кнопки к исходному состоянию.

Временная линия этого примера



Упрощение временной линии

Мы завершили шаг 2 представления временных линий на диаграмме. Теперь, когда вы понимаете, как работает наша платформа, ее можно упростить на шаге 3. Вот что мы имеем на данный момент:

```

1 saveUserAjax(user, function() {
2   setUserLoadingDOM(false);
3 });
4 setUserLoadingDOM(true);
5 saveDocumentAjax(document, function() {
6   setDocLoadingDOM(false);
7 });
8 setDocLoadingDOM(true);
  
```

Действия

1. saveUserAjax()
2. setUserLoadingDOM(false)
3. setUserLoadingDOM(true)
4. saveDocumentAjax()
5. setDocLoadingDOM(false)
6. setDocLoadingDOM(true)

Переходим к шагу 3, на котором мы займемся упрощением диаграммы на основе знания потоковой модели нашей платформы. Так как все три временные линии выполняются в JavaScript в браузере, к диаграмме можно применить знание исполнительской среды браузера. В JavaScript это сводится к двум шагам:

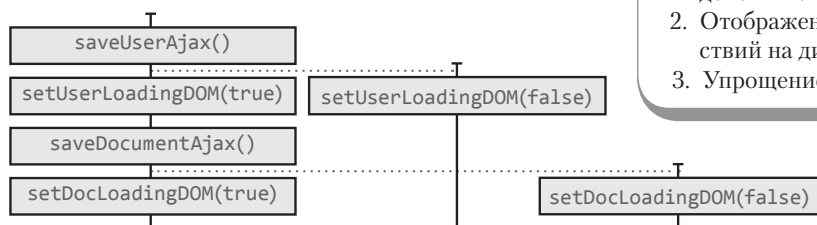
1. Объединение всех действий на одной временной линии.
2. Объединение завершаемых временных линий с созданием одной новой временной линии.

Давайте выполним эти шаги по порядку.

Три шага построения диаграммы

1. Идентификация действий.
2. Отображение действий на диаграмме.
3. Упрощение.

На предыдущей странице мы построили такую диаграмму. Напомню, что сейчас мы находимся на шаге 2. У нас имеется полная диаграмма, а на шаге 3 мы займемся ее упрощением:



Три шага построения диаграммы

1. Идентификация действий.
2. Отображение действий на диаграмме.
3. Упрощение.

В JavaScript существуют два метода упрощения, которые могут применяться в однопоточных исполнительных средах.

1. Объединение всех действий на одной временной линии.
2. Объединение завершаемых временных линий с созданием одной новой временной линии.

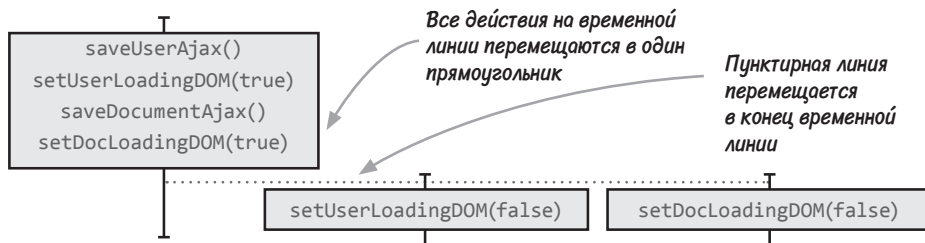
Два упрощения в JavaScript

1. Объединение действий.
2. Объединение временных линий.

Разберем эти два метода.

1. Объединение всех действий на одной временной линии

Так как код JavaScript выполняется в одном потоке, действия на одной временной линии чередоваться не могут. Временная линия отрабатывает до завершения, и только после этого может быть запущена другая временная линия. Если на диаграмме присутствуют пунктирные линии, они перемещаются в конец временной линии.



Как видите, специфика исполнительных сред JavaScript упрощает выполнение кода за счет устранения многих возможных вариантов упорядочения.

2. Объединение завершаемых временных линий с созданием одной новой временной линии

Так как первая временная линия завершается созданием двух новых временных линий, это правило не действует. Пример его применения будет продемонстрирован в коде добавления товара в корзину. А это означает, что работа над текущим примером завершена!



Ваш ход

Перед вами код и диаграмма, использованные в предыдущем упражнении. Поскольку выполняется код JavaScript, можно применить два шага упрощения. Выполните первый шаг с объединением действий. Считайте, что `cook()`, `serve()` и `eat()` являются синхронными действиями.

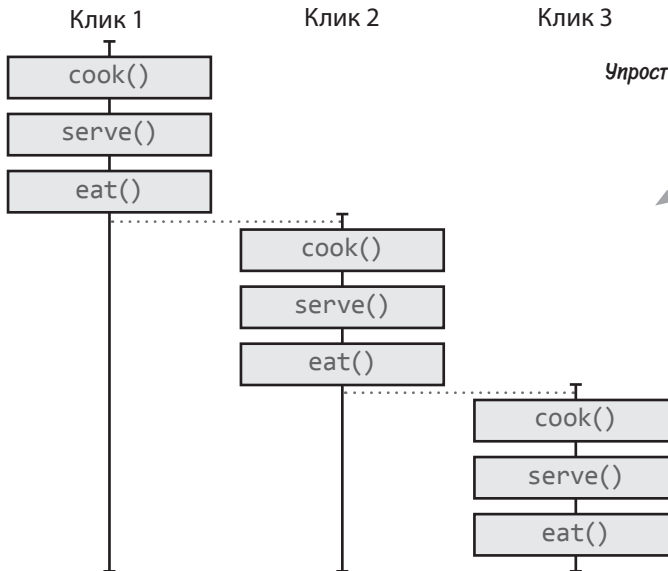
```
function dinner(food) {
  cook(food);
  serve(food);
  eat(food);
}
```

Выполните
этот шаг

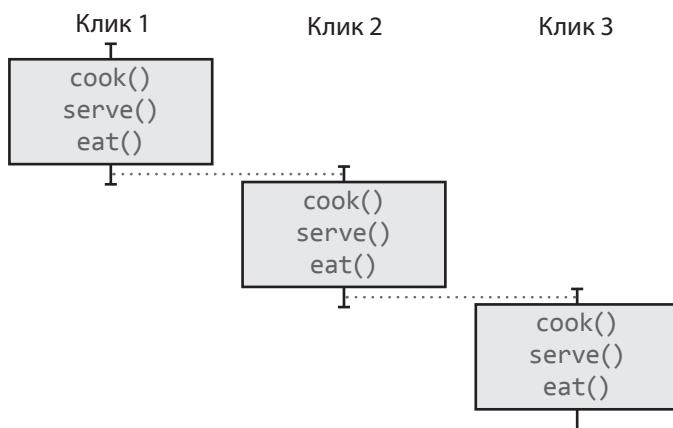
Два упрощения в JavaScript

1. Объединение действий.
2. Объединение временных линий.

```
button.addEventListener('click', dinner);
```



Упростите эту диаграмму

**Ответ**



Ваш ход

Перед вами код и диаграмма, использованные в предыдущем упражнении. Так как выполняется код JavaScript, можно применить два шага упрощения. Выполните второй шаг с объединением временных линий. Считайте, что `cook()`, `serve()` и `eat()` являются синхронными действиями.

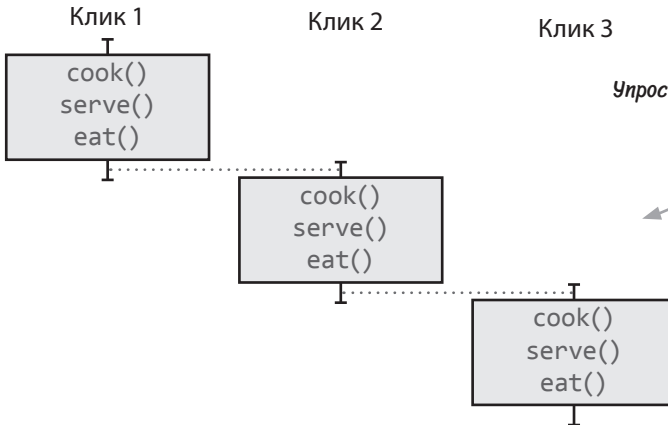
```
function dinner(food) {
  cook(food);
  serve(food);
  eat(food);
}
```

Выполните
этот шаг

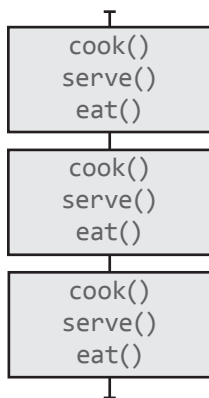
Два упрощения в JavaScript

1. Объединение действий.
2. Объединение временных линий.

```
button.addEventListener('click', dinner);
```

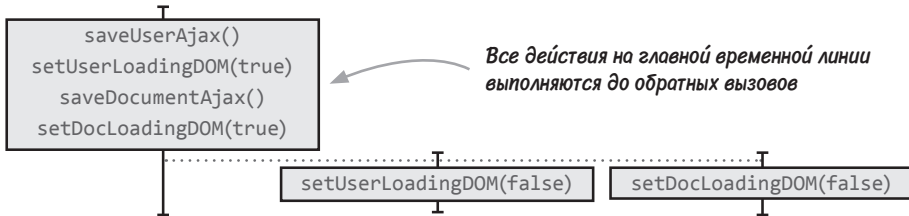


Упростите эту диаграмму

**Ответ**

Чтение завершенной временной линии

Прежде чем двигаться дальше, посмотрим, что можно узнать по только что завершенной временной диаграмме:



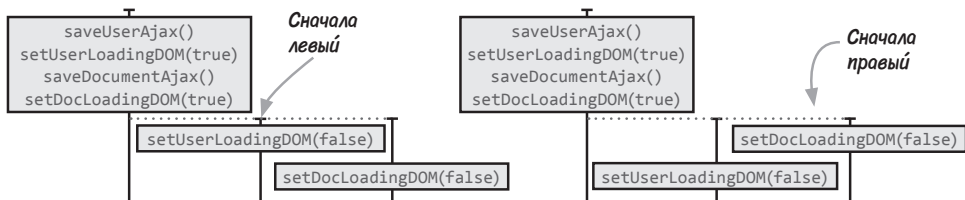
Помните, что временные диаграммы показывают возможные варианты упорядочения действий. Понимая эти варианты, можно определить, правильно ли работает наш код. Если удастся найти упорядочение, которое приводит к неправильному результату, значит, обнаружена ошибка. А если вы можете показать, что все упорядочения дают правильный результат, значит, код работает хорошо.

Существуют два вида упорядочения: определенные и неопределенные. Начнем с определенных. Поскольку все действия на главной временной линии (слева) расположены на одной временной линии, мы знаем, что они будут выполняться в указанном порядке. Кроме того, из-за пунктирной линии мы знаем, что главная временная линия завершится до выполнения остальных.

Теперь займемся неопределенным упорядочением. Заметим, что две временные линии обратных вызовов имеют разное упорядочение. Как было показано ранее, существуют три возможных варианта упорядочения одного действия на двух временных линиях. Приведу их снова:



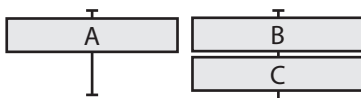
В JavaScript одновременные действия невозможны, потому что поток только один. Следовательно, остаются два возможных варианта упорядочения в зависимости от того, какой ответ аjax пришел первым:



Мы всегда отображаем индикатор загрузки, а затем скрываем его — именно в таком порядке. Все нормально, в этом коде нет проблем синхронизации.

**Ваш ход**

Ниже приведена временная диаграмма с тремя действиями в JavaScript. Перечислите возможные варианты упорядочения для этой диаграммы. Нарисуйте их, если хотите.



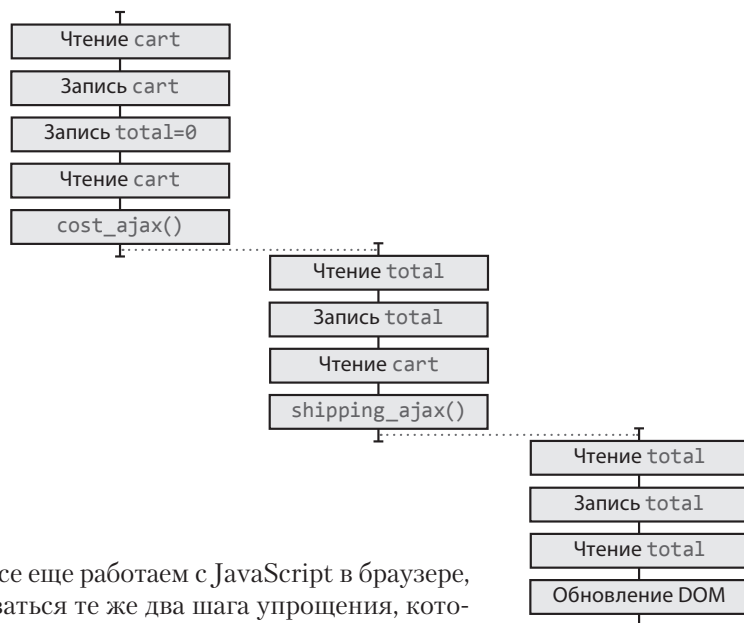
*Запишите здесь
свой ответ*

**Ответ**

1. A B C
2. B A C
3. B C A

Упрощение временной диаграммы добавления товара в корзину: шаг 3

Что ж, мы уже давно ждем шага 3. Теперь можно применить его к временной линии добавления в корзину. Результат шага 2:



Поскольку мы все еще работаем с JavaScript в браузере, будут использоваться те же два шага упрощения, которые упоминались ранее.

1. Объединение всех действий на одной временной линии.
2. Объединение завершаемых временных линий с созданием одной новой временной линии.

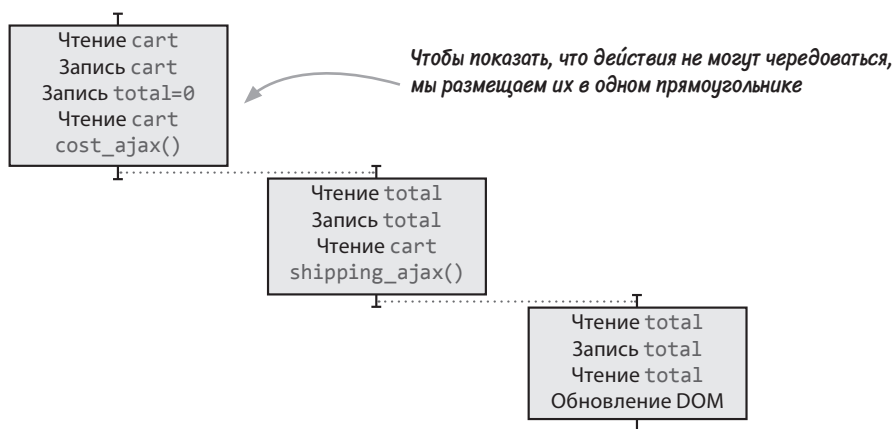
Эти шаги должны выполняться в указанном порядке, или процедура не сработает.

1. Объединение всех действий на одной временной линии

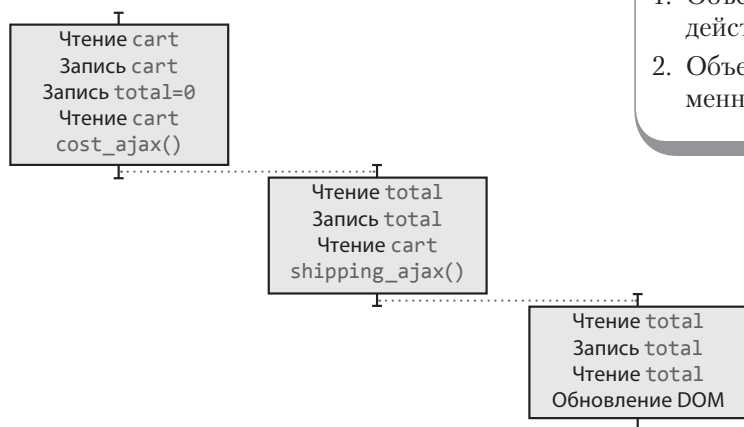
И снова JavaScript выполняется в одном потоке. Другой поток не сможет прервать текущую временную линию, и следовательно, возможность чередования между временными линиями отсутствует. Все действия можно разместить на одной временной линии, по одному прямоугольнику на каждую исходную временную линию.

Два упрощения в JavaScript

1. Объединение действий.
2. Объединение временных линий.



Так выглядит диаграмма после объединения всех действий с временной линии в один прямоугольник:



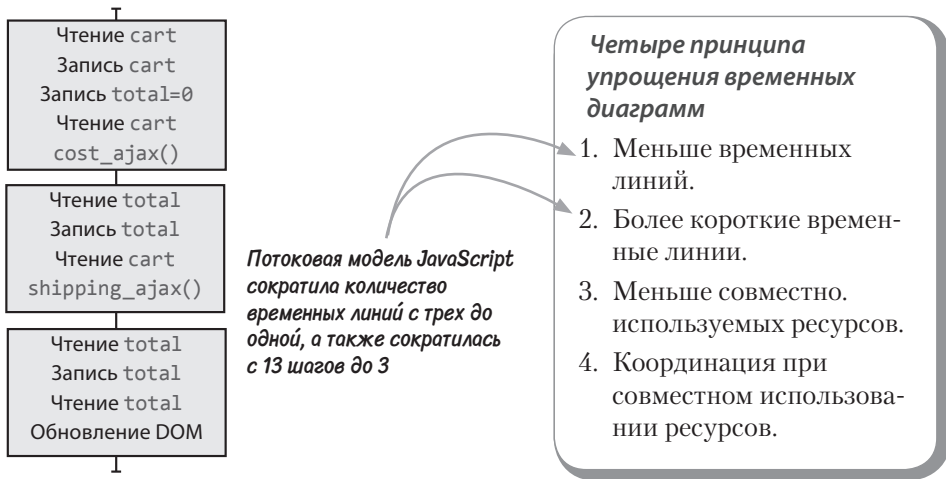
Два упрощения в JavaScript

1. Объединение действий.
2. Объединение временных линий.

Теперь можно воспользоваться вторым упрощением.

2. Объединение завершаемых временных линий с созданием одной новой временной линии

Каждая временная линия на нашей диаграмме завершается запуском новой временной линии. Каждая временная линия завершается вызовом ajax, а работу продолжает обратный вызов. Эти три временные линии можно объединить в одну.



Обратите внимание: в этой точке мы не можем вернуться к шагу 1 и разместить все действия в одном прямоугольнике. Их необходимо оставить в таком виде. Почему? Потому что разные прямоугольники отражают возможные варианты чередования, которые существовали при их отображении на отдельных временных линиях.

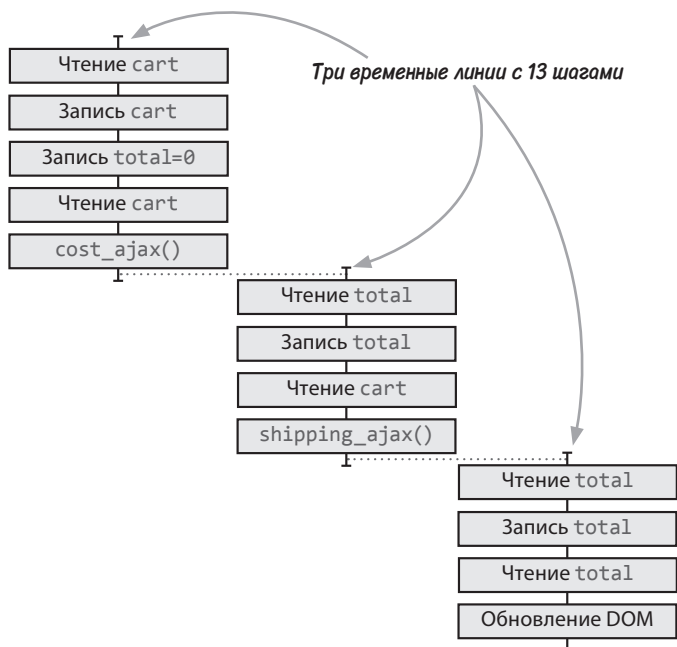
Такое представление отражает наше интуитивное восприятие цепочек обратных вызовов, особенно то, что они воспринимаются как одна временная линия. Кроме того, его проще рисовать. И на этом наши три шага завершаются!

Рисование временной линии (шаги 1–3)

Посмотрим, чего же мы добились. Первым шагом была идентификация действий в коде. Их было 13:

<pre>function add_item_to_cart(name, price, quantity) { cart = add_item(cart, name, price, quantity); calc_cart_total(); } function calc_cart_total() { total = 0; cost_ajax(cart, function(cost) { total += cost; shipping_ajax(cart, function(shipping) { total += shipping; update_total_dom(total); }); }); }</pre>	Действия 1. Чтение cart. 2. Запись cart. 3. Запись total = 0. 4. Чтение cart. 5. Вызов cost_ajax(). 6. Чтение total. 7. Запись total. 8. Чтение cart. 9. Вызов shipping_ajax(). 10. Чтение total. 11. Запись total. 12. Чтение total. 13. Вызов update_total_dom().
--	---

Второй шаг — рисование исходной диаграммы. Временная линия показывает, будет ли следующее представляемое действие последовательным или параллельным. Последовательные действия попадают на одну временную линию. Параллельные действия размещаются на новой временной линии:



Три шага построения диаграммы

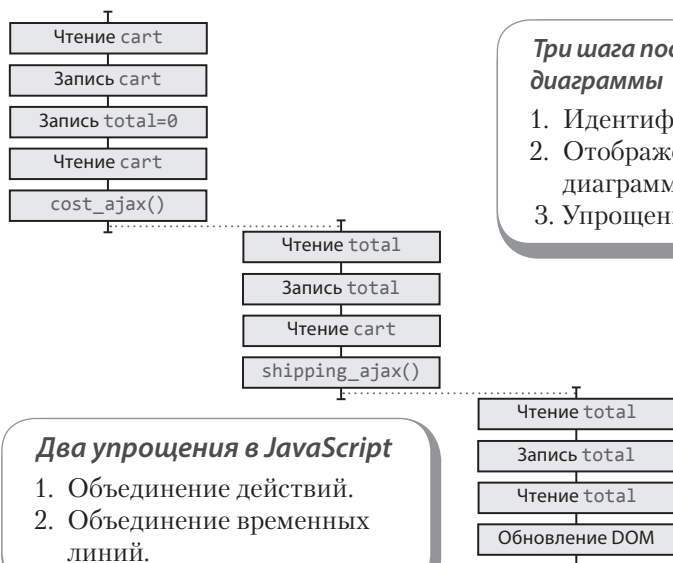
1. Идентификация действий.
2. Отображение действий на диаграмме.
3. Упрощение.

Четыре принципа упрощения временных диаграмм

1. Меньше временных линий.
2. Более короткие временные линии.
3. Меньше совместно используемых ресурсов.
4. Координация при совместном использовании ресурсов.

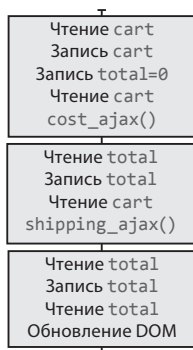
Третий шаг — упрощение — рассматривается на следующей странице.

На предыдущей странице был приведен обзор шага 2:



Третьим и последним шагом станет упрощение временной линии на основании знания платформы. Так как код выполняется в браузере в JavaScript, мы применили два шага. Однопоточная модель JavaScript позволила поместить все действия на одной временной линии в один прямоугольник. Затем мы могли преобразовать обратные вызовы, которые продолжают вычисление после асинхронного действия, в одну временную линию. Неопределенность временных характеристик и возможность чередования отражены на диаграмме несколькими прямоугольниками.

Одна временная линия с тремя шагами



Тот факт, что мы смогли упростить три временные линии с 13 действиями в одну временную линию из трех шагов, показывает, что потоковая модель JavaScript способна упростить диаграмму. Однако диаграмма также показывает, что проблема не устранена полностью. Асинхронные действия все еще требуют отдельных прямоугольников. На следующей странице вы увидите, как по этой диаграмме найти ошибку, проявившуюся в программе.

Четыре принципа упрощения временных диаграмм

1. Меньше временных линий.
2. Более короткие временные линии.
3. Меньше совместно используемых ресурсов.
4. Координация при совместном использовании ресурсов.

Резюме: построение временных диаграмм

Ниже приведена краткая сводка процесса рисования временных диаграмм.

Идентификация действий

Каждое действие отмечается на временной диаграмме. Анализируйте составные действия, пока не идентифицируете атомарные действия, такие как чтение и запись в переменные. Будьте внимательны с операциями, которые выглядят как одно действие, но в действительности представляют несколько действий (например, ++ и +=).

Рисование действий

Действия могут выполняться либо последовательно, либо параллельно.

Действия, выполняемые последовательно — одно за другим

Если действия происходят по порядку, разместите их на одной временной линии. Обычно это происходит тогда, когда два действия происходят на линиях, расположенных друг за другом. Также последовательные действия возможны при другой семантике выполнения, например при вычислении аргументов слева направо.

Действия, выполняемые параллельно — одновременно, сначала левое или сначала правое

Если действия могут происходить одновременно или без определенного порядка, разместите их на разных временных линиях. Это может происходить по разным причинам, в том числе таким, как:

- асинхронные обратные вызовы;
- множественные потоки;
- множественные процессы;
- выполнение на разных машинах.

Нарисуйте каждое действие и используйте пунктирные линии для обозначения ограничений порядка. Например, обратный вызов ajax не может выполняться раньше запроса ajax. Этот факт можно обозначить пунктирной линией.

Упрощение временной линии

Семантика конкретного языка, который вы используете, может устанавливать дополнительные ограничения на порядок выполнения. Эти ограничения можно применить к временной линии, чтобы ее было проще понять. Несколько рекомендаций, применимых к любому языку.

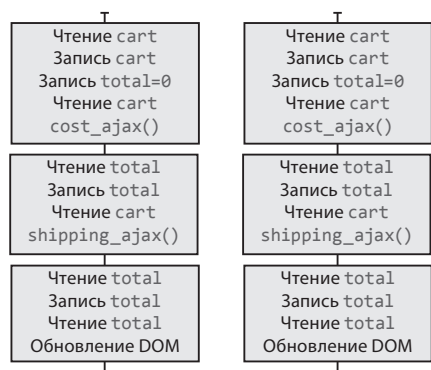
- Если два действия не могут чередоваться, объедините их в один прямоугольник.
- Если одна временная линия завершается и за ней начинается другая, объедините их в одну временную линию.
- Добавьте пунктирные линии, если порядок ограничен.

Чтение временных линий

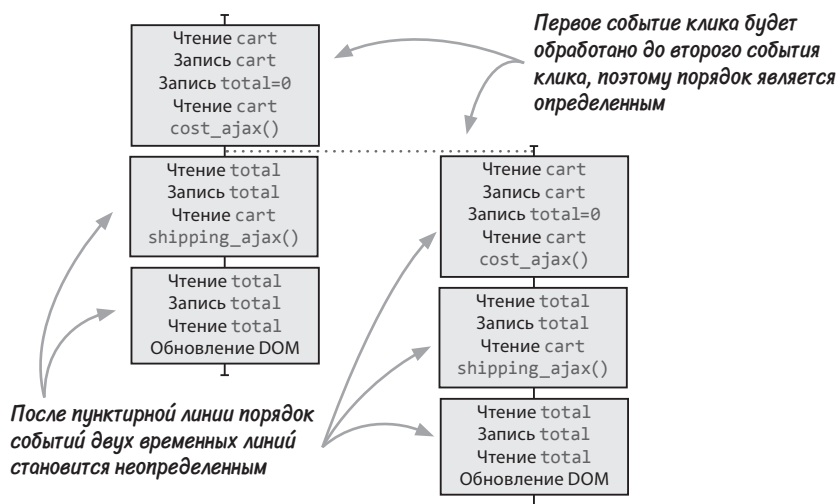
В общем случае действия на разных временных линиях могут происходить в трех разных вариантах упорядочения: одновременно, сначала левое и сначала правое. Оцените возможные варианты как невозможные, желательные или нежелательные.

Сопоставление временных диаграмм помогает выявить проблемы

Как было показано ранее, шаги, выполняемые кодом для обновления общей стоимости корзины, выглядят правильно для одиночных кликов на кнопку. Ошибка проявляется только при быстрых повторных кликах. Чтобы увидеть эту ситуацию, полезно разместить временную линию рядом с ней самой.



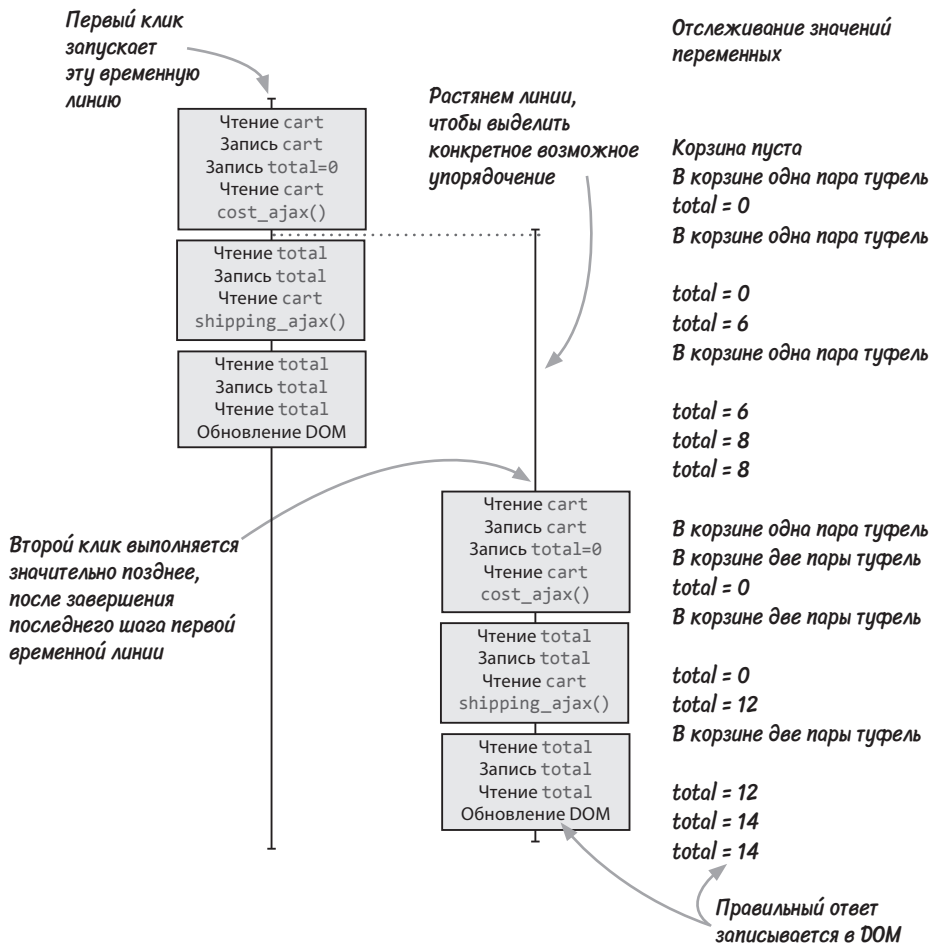
Такое размещение показывает, что две временные линии, по одной для каждого клика, могут чередоваться друг с другом. Осталось сделать еще один завершающий штрих. Так как исходный шаг на временных линиях будет обрабатываться по порядку (очередь событий гарантирует это), диаграмму можно слегка видоизменить. Мы добавим пунктирную линию, чтобы показать, что вторая временная линия не может начаться до завершения первого шага первой временной линии:



Возможно, это и неочевидно, но эта диаграмма прямо-таки кишит проблемами. К концу главы вы научитесь распознавать их самостоятельно.

Два медленных клика приводят к правильному результату

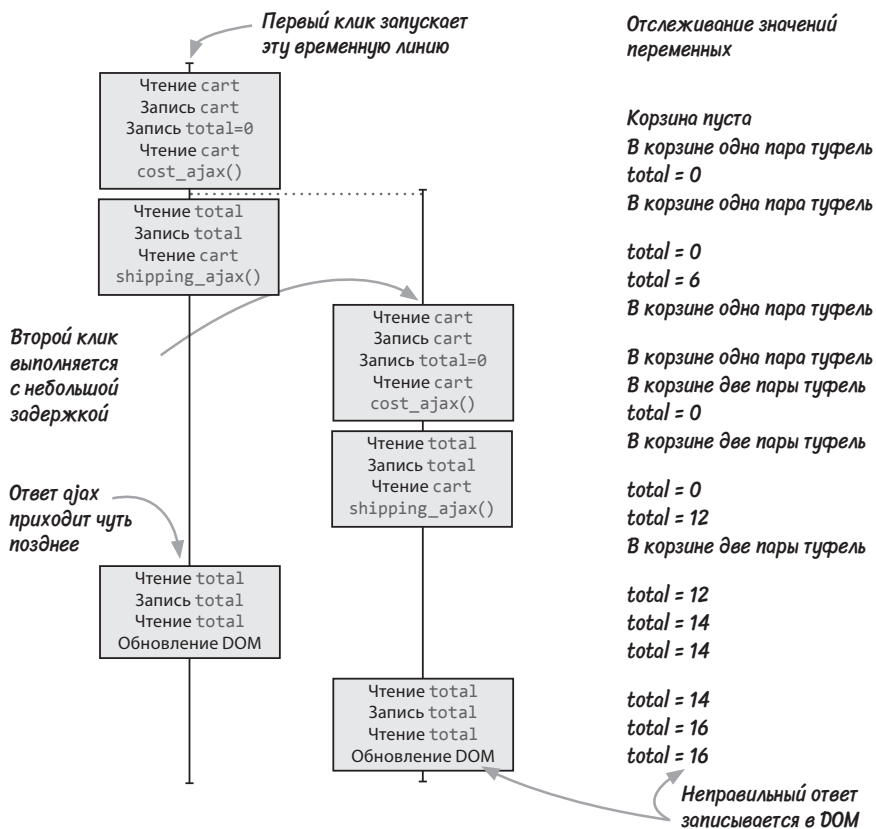
Итак, мы подготовили диаграмму для двух кликов. Проведем линии между шагами, чтобы наглядно выделить разные варианты чередования. Сначала рассмотрим простой вариант, который всегда приводит к правильному результату, — два медленных клика.



Отслеживание шагов на диаграмме конкретного варианта упорядочения показывает, что будет происходить. В данном случае все работает так, как задумано. А теперь посмотрим, удастся ли нам найти возможный вариант упорядочения, который дает неправильный результат \$16 как в реальной системе.

Два быстрых клика приводят к неправильному результату

Мы только что рассмотрели простой случай: второй клик выполняется сразу же после завершения первой временной линии. Посмотрим, удастся ли нам найти вариант упорядочения, при котором будет получен неправильный ответ. Значения переменных выводятся справа:



Мы нашли ошибку! Она связана с порядком, в котором действия выполняются на временных линиях обработчика кликов. Так как мы не контролируем чередование шагов, иногда они выполняются в одном варианте, а иногда — в другом.

Эти две относительно короткие временные линии могут сгенерировать 10 возможных вариантов упорядочения. Какие из них правильные? Какие неправильные? Можно потрудиться и отследить их, но на практике временные линии обычно намного длиннее. Они могут генерировать сотни, тысячи и даже миллионы возможных вариантов упорядочения. Рассмотреть каждый из них просто невозможно. Необходимо каким-то другим способом гарантировать работоспособность нашего кода. Давайте исправим ошибку и попытаемся понять, как предотвратить подобные ошибки в будущем.

Временные линии с совместным использованием ресурсов создают проблемы

Мы неплохо понимаем временные линии и их код. Что именно стало источником проблем? В данном случае это совместное использование ресурсов. Обе временные линии используют одни глобальные переменные. При этом они мешают друг другу при чередовании.

Подчеркнем все глобальные переменные в коде.

```
function add_item_to_cart(name, price, quantity) {
  cart = add_item(cart, name, price, quantity);
  calc_cart_total();
}
function calc_cart_total() {
  total = 0;
  cost_ajax(cart, function(cost) {
    total += cost;
    shipping_ajax(cart, function(shipping) {
      total += shipping;
      update_total_dom(total);
    });
  });
}
```

Глобальные
переменные

Действия, совместно
использующие глобаль-
ную переменную *total*



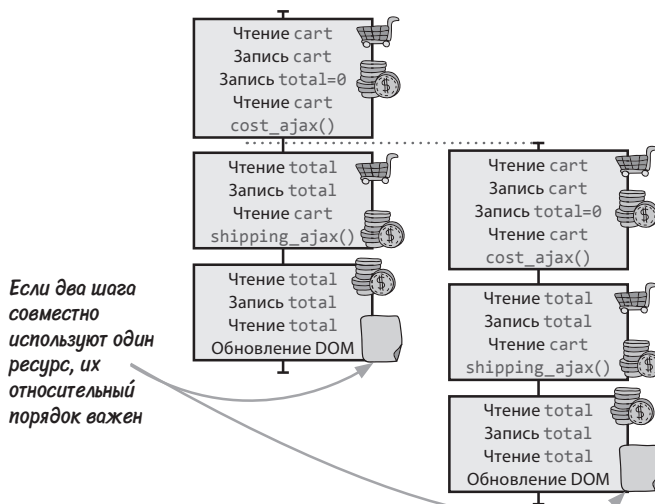
Действия, совместно
использующие глобаль-
ную переменную *cart*



Действия, совместно
использующие DOM



Затем для ясности пометим шаги временной диаграммы информацией о том, какие шаги используют те или иные глобальные переменные.



На каждом шаге выполняется чтение и запись **total**, что может привести к ошибкам. Если операции выполняются в неправильном порядке, они определенно смогут помешать друг другу. Начнем с глобальной переменной **total** и преобразуем ее в локальную.

Преобразование глобальной переменной в локальную

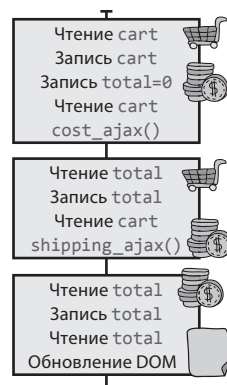
Совместного использования глобальной переменной `total` можно избежать

Использовать для общей стоимости глобальную переменную не обязательно. Самое простое улучшение — переход на локальную переменную.

1. Определение глобальной переменной, которая заменяется локальной

```
function calc_cart_total() {
  total = 0;
  cost_ajax(cart, function(cost) {
    total += cost;
    shipping_ajax(cart, function(shipping) {
      total += shipping;
      update_total_dom(total);
    });
  });
}
```

Здесь значение `total` уже может быть отлично от нуля. Другая временная линия может выполнить запись до активизации обратного вызова



2. Замена глобальной переменной на локальную

```
function calc_cart_total() {
  var total = 0;
  cost_ajax(cart, function(cost) {
    total += cost;
    shipping_ajax(cart, function(shipping) {
      total += shipping;
      update_total_dom(total);
    });
  });
}
```

Заменяется локальной переменной

Теперь чтение и запись `total` не будут иметь последствий за пределами функции, поэтому они не являются действиями. На временных линиях должны размещаться только действия



Временная линия действительно сокращается и упрощается

Что ж, это было несложно! Преобразование `total` в локальную переменную сработало весьма эффективно. Временная линия по-прежнему состоит из трех шагов, поэтому возможны 10 вариантов упорядочения. Правильным будет большее количество вариантов, потому что они не используют одну глобальную переменную `total`. Тем не менее они по-прежнему используют глобальную переменную `cart`. Разберемся и с этой проблемой.

Преобразование глобальной переменной в аргумент

Помните принцип, который гласил, что количество неявных входных данных для действия следует уменьшать? Он относится и к временным линиям. Эта временная линия использует глобальную переменную `cart` как неявный ввод. Мы можем устранить этот неявный ввод и одновременно сократить совместное использование ресурсов временными линиями! Процесс остается таким же, как при устранении неявного ввода для действий: чтение глобальных переменных заменяется аргументом.

1. Идентификация неявного ввода

```
function add_item_to_cart(name, price, quantity) {
  cart = add_item(cart, name, price, quantity);
  calc_cart_total();
}
function calc_cart_total() {
  var total = 0;
  cost_ajax(cart, function(cost) {
    total += cost;
    shipping_ajax(cart, function(shipping) {
      total += shipping;
      update_total_dom(total);
    });
  });
}
```

Эти две операции чтения могут прочитать разные значения, если корзина изменится между чтениями

Остается один шаг, использующий глобальную переменную cart

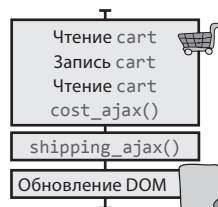


2. Замена неявного ввода аргументом

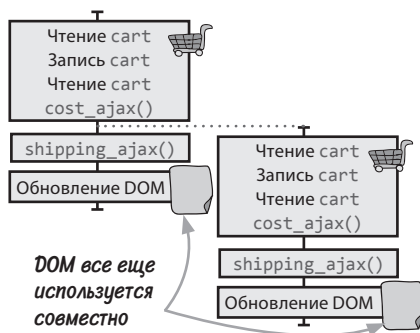
```
function add_item_to_cart(name, price, quantity) {
  cart = add_item(cart, name, price, quantity);
  calc_cart_total(cart);
}
function calc_cart_total(cart) {
  var total = 0;
  cost_ajax(cart, function(cost) {
    total += cost;
    shipping_ajax(cart, function(shipping) {
      total += shipping;
      update_total_dom(total);
    });
  });
}
```

Переменная cart передается в аргументе

Эти операции чтения уже не относятся к глобальной переменной



Остался еще один шаг, использующий глобальную переменную `cart`, но я напомню, что вторая временная линия ограничена выполнением после первого шага (отсюда пунктирная линия), так что эти первые шаги, использующие `cart`, всегда будут выполняться по порядку. Они не могут помешать работе друг друга. Мы часто будем пользоваться этим свойством



в книге: оно позволяет безопасно использовать глобальное изменяемое состояние даже при наличии нескольких временных линий.

Тем не менее в коде остается ошибка. В нем все еще совместно используется DOM как ресурс. Избавиться от этого невозможно, потому что код должен манипулировать с DOM. О том, как организовать совместное использование ресурсов, будет рассказано в следующей главе.



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Мы только что исключили все глобальные переменные из `calc_cart_total()`. Не означает ли это, что функция стала вычислением?

О: Хороший вопрос. Ответ: нет, не означает. Функция `calc_cart_total()` все еще выполняет несколько действий.

Во-первых, она дважды связывается с сервером, а это определенно действия. Во-вторых, она обновляет DOM, что также является действием.

Тем не менее после исключения чтения и записи в глобальные переменные она безусловно приближается к вычислению — и это хорошо. Вычисления вообще не зависят от того, когда они выполняются. Эта функция начинает в меньшей степени зависеть от того, когда она выполняется, чем прежде. Она еще не перешла границу, но приблизилась к ней.

На нескольких ближайших страницах мы переместим обновление DOM за пределы функции, благодаря чему она станет еще ближе к границе вычисления и будет более пригодной для повторного использования.

В: Нам пришлось обсуждать потоковую модель JavaScript, AJAX и цикл событий JavaScript. Вы уверены, что эта книга написана не о JavaScript?

О: Да, уверен. Эта книга посвящена функциональному программированию на любом языке. Но просто чтобы убедиться в том, что читатели хорошо понимают, что здесь происходит, мне пришлось объяснить некоторые подробности внутренней реализации JavaScript.

Я должен был выбрать какой-то язык, а JavaScript великолепно подходит для изучения функционального программирования по нескольким причинам. Одна из них заключается в том, что JavaScript очень популярен. Если бы я выбрал Java или Python, мне пришлось бы также описывать некоторые подробности их реализации. Я старался сделать так, чтобы код JavaScript не отвлекал от идей функционального программирования. Попробуйте отвлечься от языка и сосредоточиться на логическом обосновании происходящего.

**Ваш ход**

Пометьте каждое из следующих утверждений как истинное или ложное.

1. Две временные линии могут совместно использовать ресурсы.
2. Временные линии, совместно использующие ресурсы, безопаснее тех временных линий, которые этого не делают.
3. Два действия, находящиеся на одной временной линии, должны избегать совместного использования одного ресурса.
4. Изображать вычисления на временной линии не обязательно.
5. Два действия на одной временной линии могут происходить параллельно.
6. Однопоточная модель JavaScript означает, что временные линии можно не рассматривать.
7. Два действия на разных временных линиях могут происходить одновременно, сначала левое или сначала правое.
8. Для исключения совместного использования глобальной переменной используются аргументы и локальные переменные.
9. Временные диаграммы помогают понять возможные варианты упорядочения при выполнении вашей программы.
10. Временные линии с совместно используемыми ресурсами могут создать проблемы синхронизации.

**Ответ**

1. И. 2. Л. 3. Л. 4. И. 5. Л. 6. Л. 7. И. 8. И. 9. И. 10. И.

Расширение возможностей повторного использования кода

Бухгалтерия хочет использовать `calc_cart_total()` без изменения DOM. Общая стоимость заказа должна вычисляться как число, которое может использоваться в других вычислениях, а не для обновления DOM. Однако мы не можем передать сумму в возвращаемом значении `calc_cart_total()`. Оно недоступно до завершения двух асинхронных вызовов. Как получить значение? Другими словами, как преобразовать неявный вывод в возвращаемое значение при использовании асинхронных вызовов?

В главах 4 и 5 было показано, как неявный вывод выделяется в возвращаемые значения. Модификация DOM является неявным выводом, но она выполняется в асинхронном обратном вызове. Использовать возвращаемое значение нельзя. Что же делать? Помогут новые обратные вызовы!

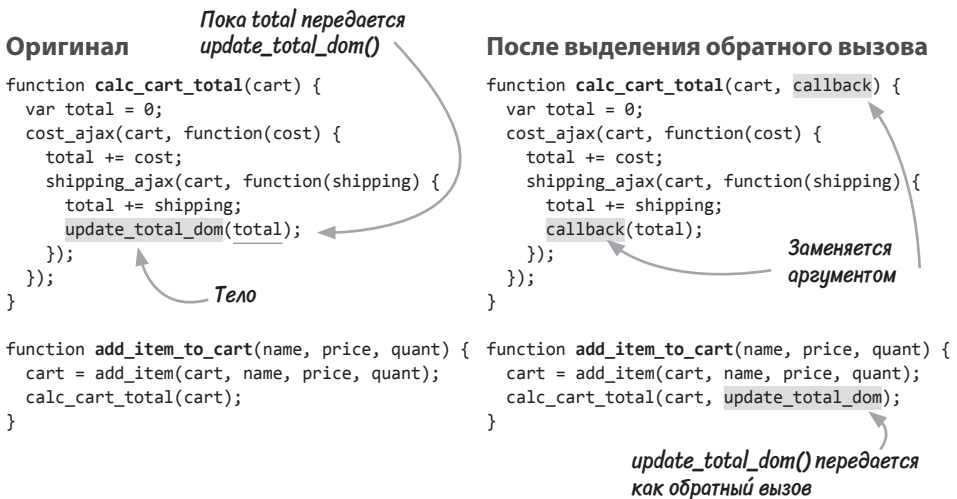
Так как мы не можем вернуть нужное значение, его необходимо передать функции обратного вызова. После завершения вычисления общей стоимости мы передаем сумму при вызове `update_total_dom()`. Для ее извлечения будет использоваться замена тела функции обратным вызовом:

Я знаю, что бухгалтерия хочет использовать эту функцию. Нельзя ли изменить ее, чтобы она лучше подходила для повторного использования?



Дженна из команды разработки

При использовании асинхронных вызовов неявный вывод преобразуется в обратные вызовы.



Теперь мы можем получить общую стоимость после того, как она будет полностью вычислена. С ней можно сделать все что угодно: например, записать в DOM или использовать для вычислений в бухгалтерии.

*Уже в функции,
этот пункт
необязателен*

Последовательность действий по замене тела обратным вызовом

1. Определение частей: предшествующей, тела и завершающей.
2. Извлечение функции.
3. Извлечение обратного вызова.



Принцип: в асинхронном контексте в качестве явного вывода вместо возвращаемого значения используется обратный вызов

Асинхронные вызовы не могут возвращать значения. Асинхронные вызовы возвращают управление немедленно, но значение будет сгенерировано только в будущем, когда будет активизирован обратный вызов. Получить значение обычным способом, как с синхронными функциями, невозможно.

Для получения значений из асинхронных вызовов используется обратный вызов. Обратный вызов передается в аргументе и вызывается с требуемым значением. Это стандартный прием асинхронного программирования JavaScript.

В функциональном программировании этот прием используется для извлечения действий из асинхронных функций. С синхронной функцией для выделения действия мы возвращали значение вместо вызова действия внутри функции. Затем значение используется для вызова действия, находящегося уровнем выше в стеке вызовов. С асинхронными функциями действие передается как обратный вызов.

Рассмотрим две функции, синхронную и асинхронную, которые делают одно и то же:

Синхронные функции

- Возвращают значение, которое может использоваться на стороне вызова.
- Возвращают значения, которые передаются в аргументах действий.

Асинхронные функции

- В какой-то момент будущего активизируют обратный вызов с результатом.
- Действия передаются как обратные вызовы.

Исходная синхронная функция

```
function sync(a) {  
  ...  
  action1(b);  
}
```

```
function caller() {  
  ...  
  sync(a);  
}
```

Исходная асинхронная функция

```
function async(a) {  
  ...  
  action1(b);  
}
```

```
function caller() {  
  ...  
  async(a);  
}
```

Синхронные и асинхронные функции на первый взгляд похожи

Их вызовы тоже выглядят одинаково

С выделенным действием

```
function sync(a) {  
  ...  
  return b;  
}
```

```
function caller() {  
  ...  
  action1(sync(a));  
}
```

С выделенным действием

```
function async(a, cb) {  
  ...  
  cb(b);  
}
```

```
function caller() {  
  ...  
  async(a, action1);  
}
```

Синхронная функция использует возвращаемое значение; асинхронная использует обратный вызов

Сторона вызова синхронной функции использует возвращаемое значение для вызова; сторона вызова асинхронной функции передает действие как обратный вызов



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Почему мы не можем вернуть значение из асинхронной функции? Я думал, что все функции могут иметь возвращаемые значения.

О: Формально это возможно, однако вы не сможете использовать его обычным образом. Пример:

```
function get_pets_ajax() {
  var pets = 0;
  dogs_ajax(function(dogs) {
    cats_ajax(function(cats) {
      pets = dogs + cats;
    });
  });
  return pets;
}
```

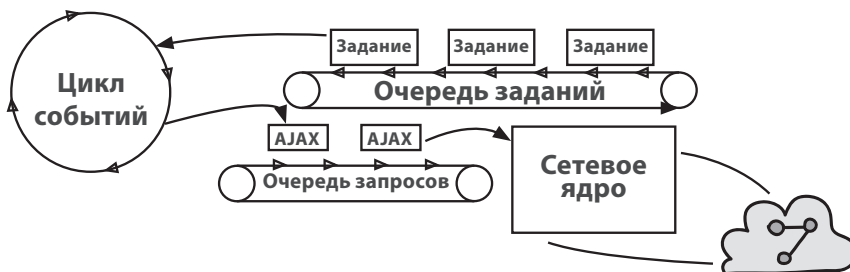
Обратный вызов, переданный dogs_ajax(), и обратный вызов, переданный cats_ajax(), не будет выполняться до поступления ответов по сети. До этого момента значение pets присвоено не будет

Возвращает управление немедленно, до завершения запросов ajax

Что вернет эта функция? Она возвращает текущее содержимое переменной `pets`, но оно всегда равно нулю. Да, она возвращает значение, но не то, что вы пытаетесь вычислить.

Здесь `get_pets_ajax()` вызывает функцию `dogs_ajax()`, которая отправляет запрос сетевому ядру, после чего немедленно возвращает управление. Далее следует команда `return`. В дальнейшем, когда завершится запрос `ajax`, событие завершения (с именем `load`) будет помещено в очередь заданий. Когда-нибудь в будущем цикл событий извлечет его из очереди и активизирует обратный вызов.

Формально значение можно вернуть, но это должно быть что-то вычисленное в синхронном коде. Все, что выполняется асинхронно, использовать `return` не может, потому что оно работает в другой итерации цикла событий. Стек вызовов к этому моменту будет пуст. В асинхронном коде результирующие значения должны передаваться обратному вызову.





Ваш ход

Ниже приведен код приготовления некоторых блюд. Он использует глобальные переменные и записывает данные в DOM для вывода количества блюд. Проведите рефакторинг, чтобы устранить неявный ввод и вывод. В нем должны использоваться аргументы и локальные переменные и вместо записи в DOM должен активизироваться обратный вызов.

```
var plates = ...;
var forks  = ...;
var cups   = ...;
var total  = ...;

function doDishes() {
    total = 0;
    wash_ajax(plates, function() {
        total += plates.length;
        wash_ajax(forks, function() {
            total += forks.length;
            wash_ajax(cups, function() {
                total += cups.length;
                update_dishes_dom(total);
            });
        });
    });
}

doDishes();
```

Запишите здесь
свой ответ



**Ответ**

```
var plates = ...;
var forks  = ...;
var cups   = ...;

function doDishes(plates, forks, cups, callback) {
  var total = 0;
  wash_ajax(plates, function() {
    total += plates.length;
    wash_ajax(forks, function() {
      total += forks.length;
      wash_ajax(cups, function() {
        total += cups.length;
        callback(total);
      });
    });
  });
}

doDishes(plates, forks, cups, update_dishes_dom);
```


Итоги главы

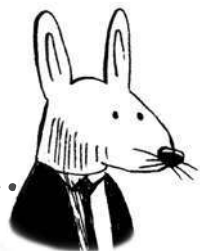
В этой главе вы научились рисовать временные диаграммы и читать их для выявления ошибок. Для упрощения временных линий мы применяли свое знание потоковой модели JavaScript, которая сокращает длину и количество временных линий. Принцип устранения совместно используемых ресурсов был применен для исправления ошибки.

Резюме

- Временные линии представляют последовательности действий, которые могут выполняться одновременно. Они показывают, какой код выполняется последовательно, а какой — параллельно.
- Современные программы часто работают с несколькими временными линиями. Каждый компьютер, поток, процесс или асинхронный обратный вызов добавляет новую временную линию.
- Так как действия на временных диаграммах могут чередоваться и это чередование нам неподконтрольно, при наличии нескольких временных линий возможны разные варианты упорядочения. Чем больше вариантов, тем сложнее будет понять, всегда ли ваш код приводит к правильному результату.
- Временные диаграммы показывают, как наш код выполняется последовательно и параллельно. Они помогают понять, где они могут мешать друг другу.
- Важно понимать потоковую модель вашего языка и платформы. Для распределенных систем очень важно понимать, как код выполняется: последовательно или параллельно.
- Совместное использование ресурсов является источником ошибок. Выявление и исключение ресурсов способствует улучшению кода.
- Временные линии, которые не используют ресурсы совместно, можно понять и выполнить в изоляции. Это избавит вас от лишней мыслительной нагрузки.

Что дальше?

У нас еще остается один совместно используемый ресурс, а именно DOM. Две временные линии, добавляющие товар в корзину, будут пытаться записать в DOM разные значения. Избавиться от этого ресурса не удастся, так как приложение должно вывести общую стоимость заказа для пользователя. Совместное использование DOM требует координации между временными линиями. Об этом речь пойдет в следующей главе.



В этой главе

- ✓ Диагностика ошибок, вызванных совместным использованием ресурсов.
- ✓ Создание примитива, обеспечивающего безопасность совместного использования ресурсов.

В предыдущей главе вы узнали о временных линиях и способах сокращения количества совместно используемых ими ресурсов. Временные линии, не использующие ресурсы совместно, не идеальные, но иногда без них не обойтись. В таких случаях необходимо проследить за тем, чтобы их совместное использование было безопасным. В этой главе вы увидите, как создавать *примитивы синхронизации* — повторно используемые фрагменты кода, которые позволяют использовать общие ресурсы между временными линиями.

Принципы работы с временными линиями

Еще раз перечислю эти принципы, чтобы вы освежили их в памяти. В предыдущей главе мы рассмотрели принципы 1–3, а также показали, как они помогают обеспечить правильность кода. В этой главе будет применяться принцип 4. Имеется ресурс, совместно используемый между временными линиями, и мы построим универсальный механизм координации временных линий, чтобы сделать возможным безопасное использование этого ресурса.

1. Чем меньше временных линий, тем проще

Каждая новая временная линия радикально усложняет понимание системы. Если вам удастся сократить количество временных линий (t в формуле справа), это очень сильно упростит вашу задачу. К сожалению, часто мы не можем управлять количеством временных линий.

2. Чем короче временные линии, тем проще

Если вам удастся устранить шаги (уменьшить a в формуле справа), это также позволит существенно сократить количество возможных вариантов упорядочения.

Формула для определения количества возможных вариантов упорядочения

3. Чем меньше совместного использования ресурсов, тем проще

При рассмотрении двух временных линий достаточно учитывать только те шаги, в которых ресурсы используются совместно. Фактически при этом сокращается количество шагов на диаграмме, а следовательно, количество возможных вариантов упорядочения.

$$O = \frac{(ta)!}{(a!)^t}$$

Количество временных линий

Количество действий на одну временную линию

Возможные варианты упорядочения

! — факториал

4. Координируйте совместное использование ресурсов

Даже после исключения всех возможных совместно используемых ресурсов у вас останутся ресурсы, от которых избавиться не удастся. Необходимо позаботиться о том, чтобы совместное использование ресурсов разными временными линиями было безопасным. Иными словами, что вы должны позаботиться о том, чтобы они получали управление в правильном порядке. Координация между временными линиями означает исключение возможных

вариантов упорядочения, которые не дают правильного результата. Вы можете взять за образец средства координации из реального программирования, чтобы разработать собственные способы координации, пригодные для повторного использования.

5. Рассматривайте время как первоклассную концепцию

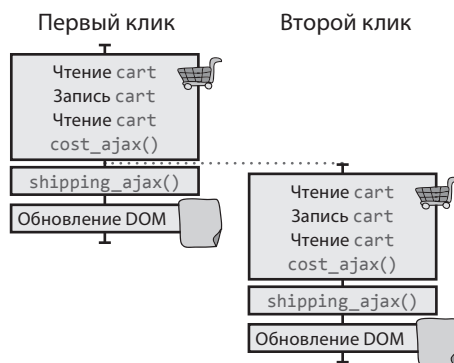
Упорядочить действия и правильно выбрать момент для их выполнения непросто. Для упрощения задачи можно создать объекты, манипулирующие временной линией. Примеры такого рода встретятся вам в следующей главе.

Начнем с применения принципа 4 к корзине, которая все еще содержит ошибку.

Корзина все еще содержит ошибку

В конце предыдущей главы мы пришли к следующей временной диаграмме. Эта временная диаграмма явно указывает на наличие ошибки. Возможно, вы ее не замечаете, но опытный функциональный программист ее непременно заметит. К концу этой главы вы тоже научитесь видеть подобные ошибки, а также узнаете, как их исправить.

Временная линия добавления товара в корзину



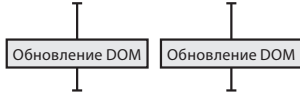
Ошибка связана с совместным использованием ресурса DOM. Если два действия не используют одни и те же ресурсы, можно не беспокоиться о том, в каком порядке они происходят. Любой из трех вариантов порядка приведет к одному ответу. Но при совместном использовании ресурсов нам приходится учитывать возможный порядок их выполнения. Обе временные линии совместно используют DOM, поэтому в данном случае возможны потенциальные проблемы.

Возможные варианты упорядочения

1. Одновременно.
2. Сначала левое.
3. Сначала правое.

Три возможных упорядочения двух обновлений DOM:

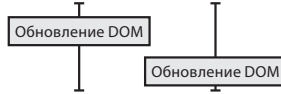
Одновременное



✓ **невозможно**

Потоковая модель JavaScript делает одновременное выполнение невозможным, поэтому его можно исключить. Тем не менее при других потоковых моделях придется учитывать такую возможность.

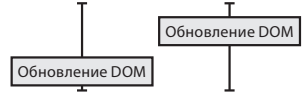
Сначала левое



✓ **желательно**

Желательное поведение. Мы хотим, чтобы второе обновление DOM перезаписывало первое, потому что второе обновление содержит более актуальную информацию.

Сначала правое



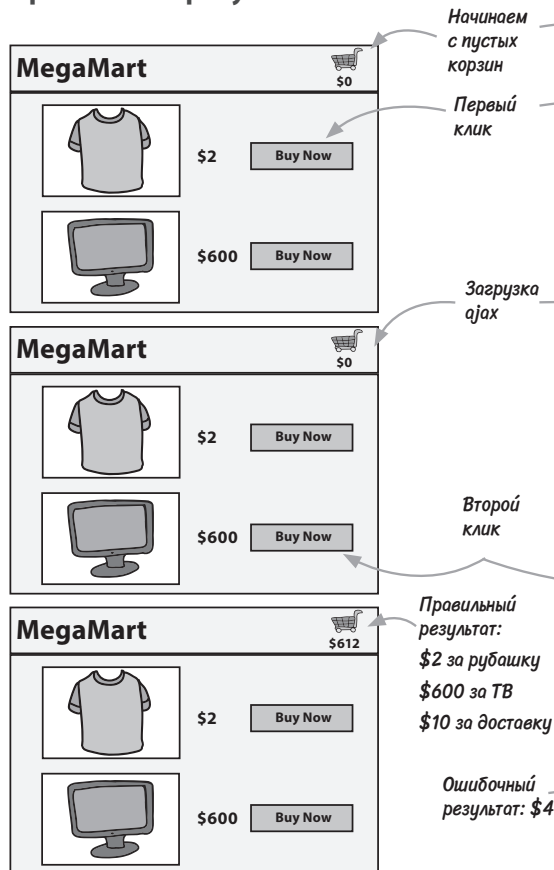
✗ **нежелательно**

Неправильное поведение. Общая стоимость для первого товара не должна перезаписывать общую стоимость для второго товара. Тем не менее пока ничто не предотвращает такую возможность.

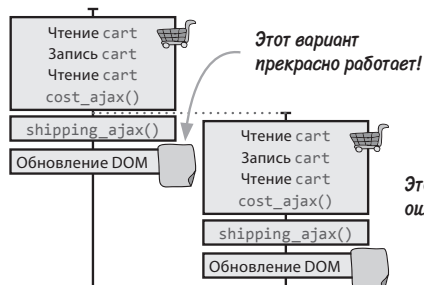
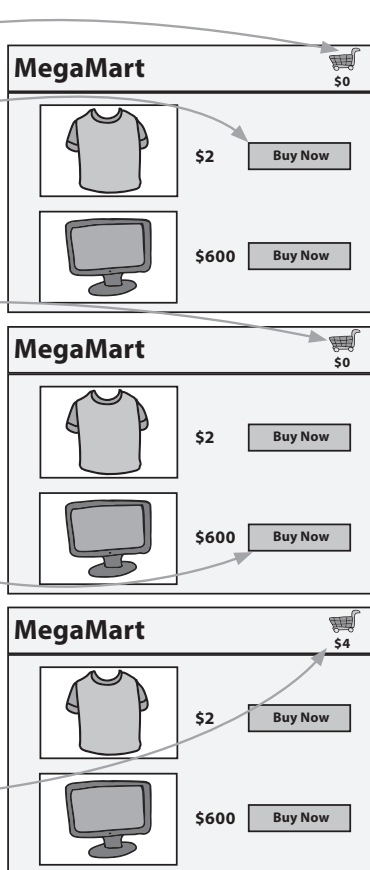
Если обновление DOM для второго клика произойдет раньше обновления DOM для первого клика, данные будут перезаписаны. На следующей странице показано, к чему это приведет.

С одним вводом от пользователя — добавлением в корзину одних и тех же товаров в одном порядке — возможны два разных результата.

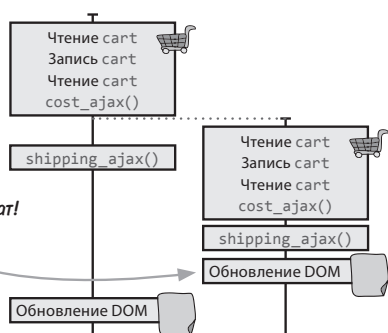
Правильный результат



Неправильный результат

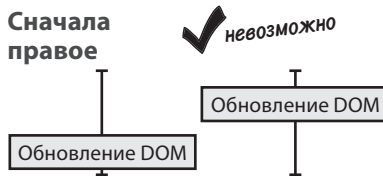


Этот вариант дает ошибочный результат!



Необходимо гарантировать порядок обновлений DOM

Нужно, чтобы обновления DOM происходили в определенном порядке, но природа временных линий такова, что порядок происходящего на двух разных временных линиях не гарантирован. Необходимо установить определенный порядок для этих обновлений. Фактически нужно сделать так, чтобы порядок «сначала правое» стал невозможным.



Как нам гарантировать, что обновление DOM будет происходить в нужном порядке? Два клика ничего не знают друг о друге.



Дженна из команды разработки

Необходимо гарантировать, что обновления DOM будут происходить в порядке поступления кликов. Но время, в течение которого происходят действия обновления DOM, нам неподконтрольно. Они происходят тогда, когда завершается сетевой запрос, а этот момент зависит от многих факторов, которыми вы не можете управлять. Требуется координировать использование DOM, чтобы обновления всегда происходили в том же порядке, что и клики.

В реальном программировании мы часто координируем совместное использование ресурсов, даже не подозревая об этом. Координация в реальном мире может послужить образцом для координации временных линий. Один из способов обеспечения нужной последовательности событий в реальном программировании основан на использовании очередей.

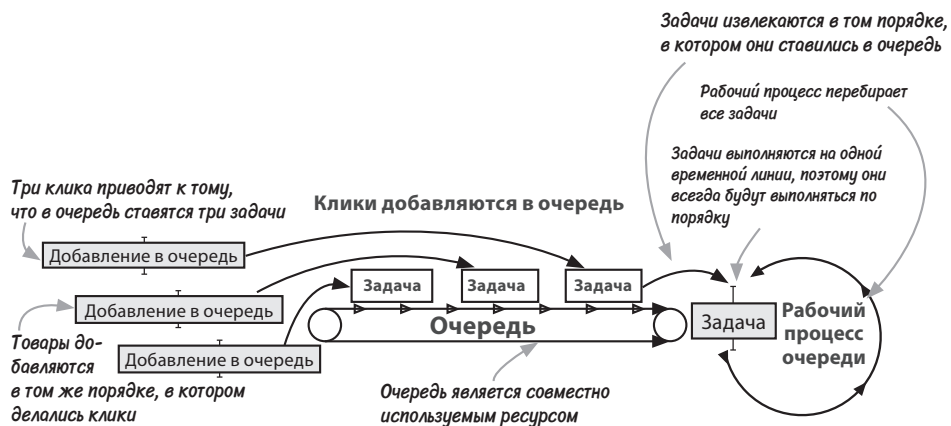
Очередь представляет собой структуру данных, в которой элементы извлекаются в том порядке, в котором они добавлялись. Это означает, что при добавлении элементов по кликам мы затем можем извлекать их в том же порядке. Очереди часто используются для координации нескольких действий между временными линиями с целью обеспечения определенного порядка.



Загляни в словарь

Очередь — структура данных, из которой элементы извлекаются в том порядке, в котором они помещались в очередь.

Затем очередь становится совместно используемым ресурсом, но так, чтобы это использование было безопасным. Далее элементы извлекаются из очереди и обрабатываются по порядку. Все задачи будут выполняться на одной временной линии, что дополнительно способствует обеспечению нужного порядка.



**Ваш ход**

В следующем списке обведите кружком все ресурсы, которые могут создать проблемы при совместном использовании между двумя временными линиями.

1. Глобальные переменные.
2. DOM.
3. Вызовы вычислений.
4. Общие локальные переменные.
5. Неизменяемые значения.
6. База данных.
7. Вызовы функций API.

**Ответ**

При совместном использовании между двумя временными линиями проблемы возможны со следующими пунктами: 1, 2, 4, 6, 7.

Реализация очереди в JavaScript

В JavaScript нет готовой структуры очереди, поэтому нам придется реализовать ее самостоятельно

Очередь является структурой данных, но при использовании ее для координации временных линий она называется *примитивом синхронизации*. Это небольшой блок функциональности, который помогает организовать совместное использование ресурсов.

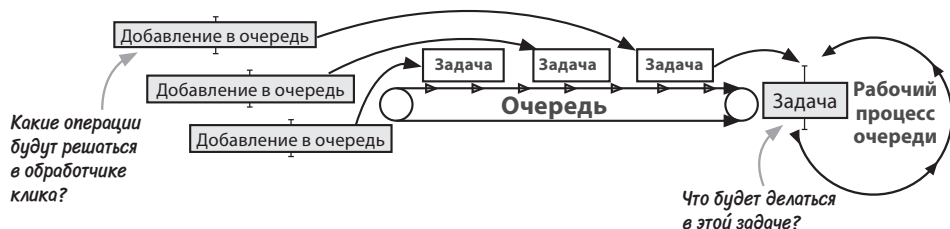
Возможно, в вашем языке уже существуют встроенные примитивы синхронизации. В JavaScript их нет, поэтому этот язык идеально подойдет для того, чтобы вы могли научиться строить их самостоятельно. Вскоре вы увидите, почему самостоятельная реализация может быть хорошим вариантом.

А пока необходимо определить, какую задачу наша очередь будет выполнять и какая работа будет выполняться непосредственно в обработчике клика.

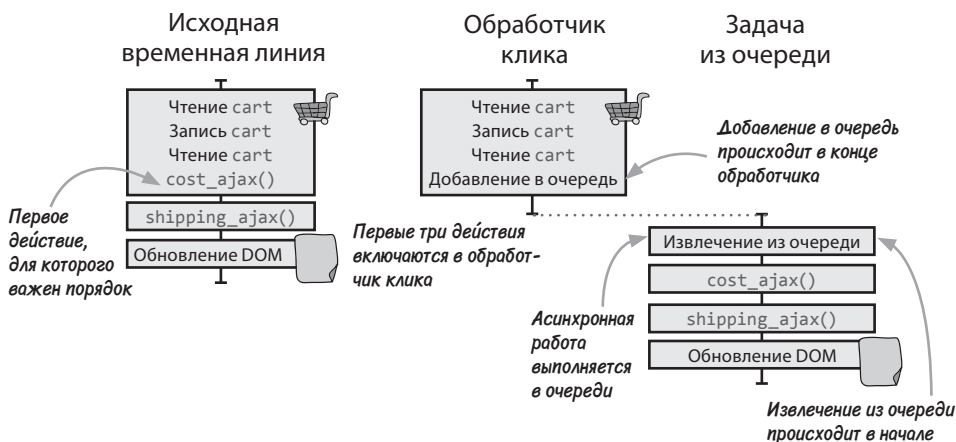


Загляни в словарь

Примитив синхронизации — небольшой блок функциональности, который помогает организовать совместное использование ресурсов между временными линиями.



Чтобы разобраться в происходящем, начнем с диаграммы текущего обработчика клика:



Все асинхронные операции
выполняются в `calc_cart_total()`

```
function add_item_to_cart(item) {
    cart = add_item(cart, item);
    calc_cart_total(cart, update_total_dom);
}

function calc_cart_total(cart, callback) {
    var total = 0;
    cost_ajax(cart, function(cost) {
        total += cost;
        shipping_ajax(cart, function(shipping) {
            total += shipping;
            callback(total);
        });
    });
}
```

Мы хотим включить в обработчик клика как можно больше действий, для которых порядок не важен. `cost_ajax()` — первое действие, который нарушает порядок (из-за асинхронности), поэтому мы включаем все, что ему предшествует.

К счастью, это соответствует функции `calc_cart_total()`. Если бы нам не повезло, пришлось бы перемещать код (без изменения порядка).

Замена работы добавлением в очередь

Текущая версия кода делает все на одной временной линии. Нам хотелось бы переместить работу на другую линию. Начнем с замены работы одним действием: добавлением товара в очередь.

Текущая диаграмма

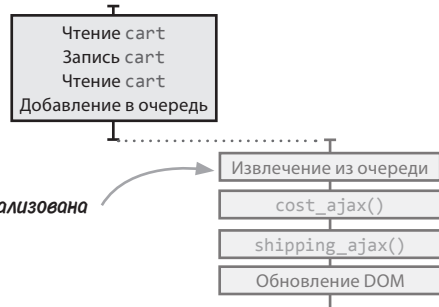
Временная линия
обработчика клика



Нужная диаграмма

Обработчик
клика

Рабочий процесс
очереди



Теперь обработчик клика
добавляет товар в очередь

Текущая версия

```
function add_item_to_cart(item) {
    cart = add_item(cart, item);
    calc_cart_total(cart, update_total_dom);
}

function calc_cart_total(cart, callback) {
    var total = 0;
    cost_ajax(cart, function(cost) {
        total += cost;
        shipping_ajax(cart, function(shipping) {
            total += shipping;
            callback(total);
        });
    });
}
```

Начало нового кода очереди (еще не реализовано). Вскоре `update_total_queue()` уже не будет ограничиваться простым добавлением в очередь

Новая версия

```
function add_item_to_cart(item) {
    cart = add_item(cart, item);
    update_total_queue(cart);
}

function calc_cart_total(cart, callback) {
    var total = 0;
    cost_ajax(cart, function(cost) {
        total += cost;
        shipping_ajax(cart, function(shipping) {
            total += shipping;
            callback(total);
        });
    });
}

var queue_items = [];

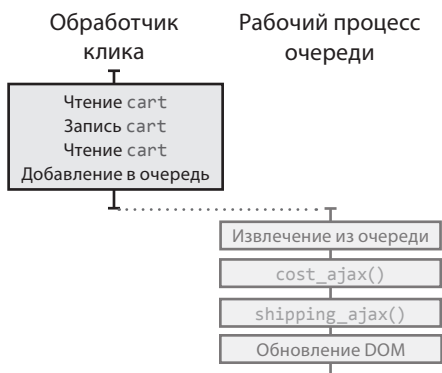
function update_total_queue(cart) {
    queue_items.push(cart);
}
```

Наша очередь устроена очень просто: в данный момент это обычный массив. Добавление элемента в очередь реализуется как простое добавление элемента в конец массива.

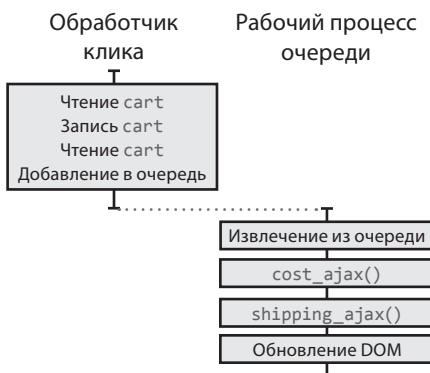
Обработка первого товара в очереди

Теперь наши товары добавляются в конец очереди, и мы можем приступить к работе. Для этого необходимо извлечь первый элемент в начале очереди (для сохранения порядка) и запустить обработку:

Текущая диаграмма



Нужная диаграмма



Текущая версия

```
function add_item_to_cart(item) {
    cart = add_item(cart, item);
    update_total_queue(cart);
}

function calc_cart_total(cart, callback) {
    var total = 0;
    cost_ajax(cart, function(cost) {
        total += cost;
        shipping_ajax(cart, function(shipping) {
            total += shipping;
            callback(total);
        });
    });
}

var queue_items = [];
```

Первый товар извлекается из массива и добавляется в корзину

```
function update_total_queue(cart) {
    queue_items.push(cart);
}
```

Запускаем рабочий процесс очереди после добавления товара

Новая версия

```
function add_item_to_cart(item) {
    cart = add_item(cart, item);
    update_total_queue(cart);
}

function calc_cart_total(cart, callback) {
    var total = 0;
    cost_ajax(cart, function(cost) {
        total += cost;
        shipping_ajax(cart, function(shipping) {
            total += shipping;
            callback(total);
        });
    });
}

var queue_items = [];
```

setTimeout() добавляет задание в цикл событий JavaScript

```
function runNext() {
    var cart = queue_items.shift();
    calc_cart_total(cart, update_total_dom);
}
```

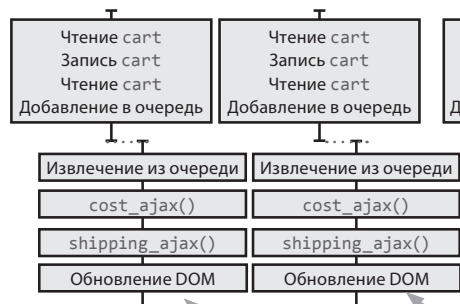
```
function update_total_queue(cart) {
    queue_items.push(cart);
    setTimeout(runNext, 0);
}
```

Товары обрабатываются по порядку, но пока ничто не мешает нам добавить два товара одновременно. Помните, что нам хотелось бы четко упорядочить все товары, а это означает, что в любой момент может добавляться только один товар. Эта проблема будет решена на следующей странице.

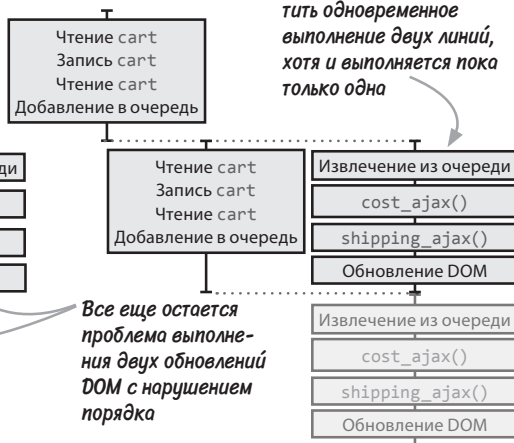
Предотвращение выполнения второй временной линии одновременно с первой

Наш код не предотвращает чередования двух временных линий. Чтобы решить эту задачу, мы будем проверять, выполняется ли что-нибудь в настоящее время:

Текущая диаграмма



Нужная диаграмма



Все еще остается проблема выполнения двух обновлений DOM с нарушением порядка

Текущая версия

```
function add_item_to_cart(item) {
    cart = add_item(cart, item);
    update_total_queue(cart);
}

function calc_cart_total(cart, callback) {
    var total = 0;
    cost_ajax(cart, function(cost) {
        total += cost;
    });
    shipping_ajax(cart, function(shipping) {
        total += shipping;
        callback(total);
    });
}

var queue_items = [];

function runNext() {
    var cart = queue_items.shift();
    calc_cart_total(cart, update_total_dom);
}

function update_total_queue(cart) {
    queue_items.push(cart);
    setTimeout(runNext, 0);
}
```

Переменная для отслеживания выполнения

Предотвращаем одновременное выполнение двух линий

Новая версия

```
function add_item_to_cart(item) {
    cart = add_item(cart, item);
    update_total_queue(cart);
}

function calc_cart_total(cart, callback) {
    var total = 0;
    cost_ajax(cart, function(cost) {
        total += cost;
    });
    shipping_ajax(cart, function(shipping) {
        total += shipping;
        callback(total);
    });
}

var queue_items = [];
var working = false;

function runNext() {
    if(working)
        return;
    working = true;
    var cart = queue_items.shift();
    calc_cart_total(cart, update_total_dom);
}

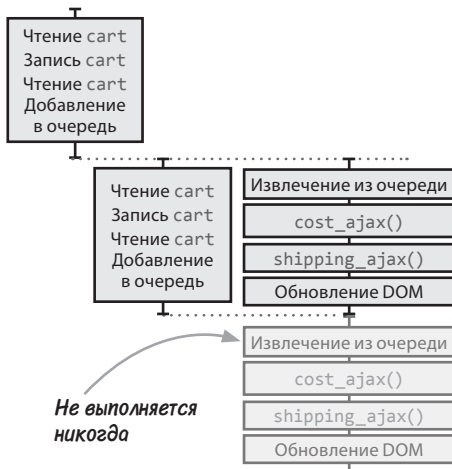
function update_total_queue(cart) {
    queue_items.push(cart);
    setTimeout(runNext, 0);
}
```

Мы предотвращаем одновременное выполнение двух временных линий, но в корзину будет добавляться только один товар. Проблема решается запуском обработки следующего товара при завершении обработки текущего.

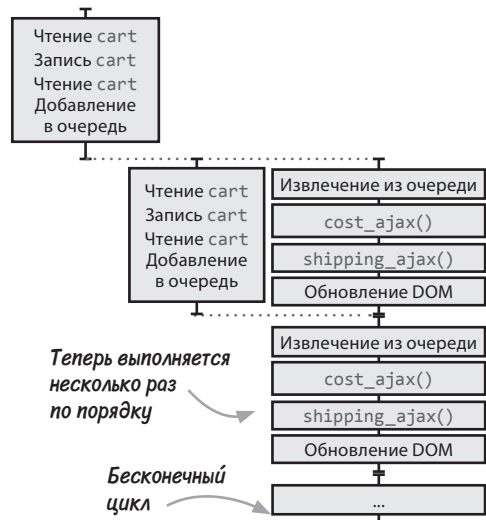
Изменение обратного вызова `calc_cart_total()` для запуска обработки следующего товара

Мы будем передавать `calc_cart_total()` в новый обратный вызов. Он будет сохранять информацию о том, что работа закончена (`working = false`), и запускать следующую задачу:

Текущая диаграмма



Нужная диаграмма



Текущая версия

```

var queue_items = [];
var working = false;

function runNext() {
  if(working)
    return;
  working = true;
  var cart = queue_items.shift();
  calc_cart_total(cart, update_total_dom);
}
  
```

Признак того, что
обработка завершена,
а мы переходим
к следующему товару

```

function update_total_queue(cart) {
  queue_items.push(cart);
  setTimeout(runNext, 0);
}
  
```

Новая версия

```

var queue_items = [];
var working = false;

function runNext() {
  if(working)
    return;
  working = true;
  var cart = queue_items.shift();
  calc_cart_total(cart, function(total) {
    update_total_dom(total);
    working = false;
    runNext();
  });
}
  
```

```

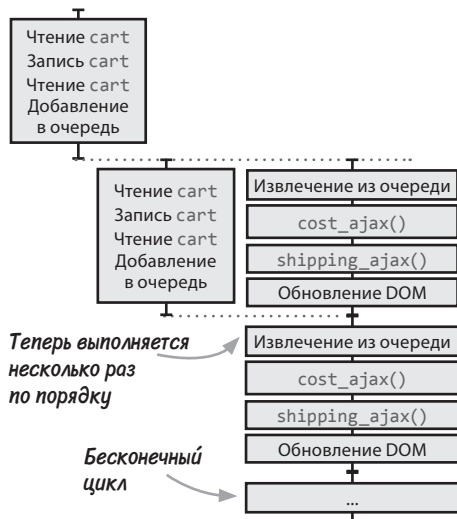
function update_total_queue(cart) {
  queue_items.push(cart);
  setTimeout(runNext, 0);
}
  
```

Фактически мы создали цикл (хотя и асинхронный). Он перебирает все элементы списка. Но тут возникает проблема: обработка не останавливается при пустом списке! Займемся ее решением.

Остановка перебора при отсутствии элементов

Цикл рабочего процесса очереди фактически игнорирует конец очереди. Вызов `queue_items.shift()` вернет `undefined`. Конечно, это значение не должно добавляться в корзину.

Текущая диаграмма



Текущая версия

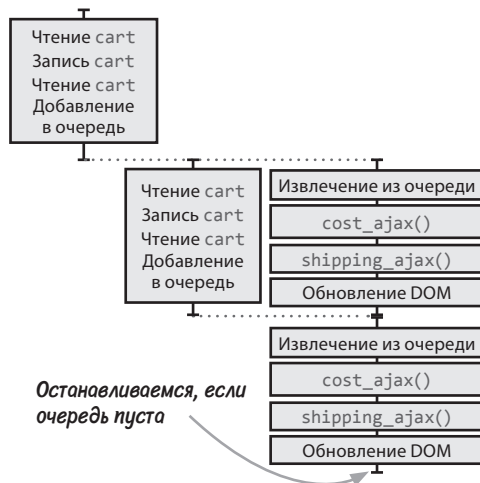
```
var queue_items = [];  
var working = false;
```

```
function runNext() {  
  if(working)  
    return;  
  
  working = true;  
  var cart = queue_items.shift();  
  calc_cart_total(cart, function(total) {  
    update_total_dom(total);  
    working = false;  
    runNext();  
  });  
}
```

```
function update_total_queue(cart) {  
  queue_items.push(cart);  
  setTimeout(runNext, 0);  
}
```

Останавливаемся, если не осталось предметов

Нужная диаграмма



Новая версия

```
var queue_items = [];  
var working = false;
```

```
function runNext() {  
  if(working)  
    return;  
  if(queue_items.length === 0)  
    return;  
  working = true;  
  var cart = queue_items.shift();  
  calc_cart_total(cart, function(total) {  
    update_total_dom(total);  
    working = false;  
    runNext();  
  });  
}
```

```
function update_total_queue(cart) {  
  queue_items.push(cart);  
  setTimeout(runNext, 0);  
}
```

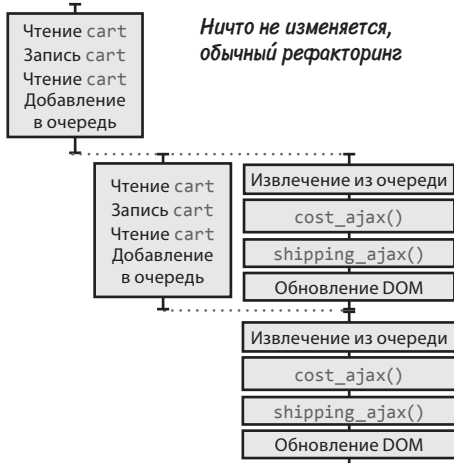
И теперь у нас имеется работоспособная очередь! Она позволяет пользователю кликнуть столько раз, сколько ему захочется, с любой скоростью: все проверки всегда обрабатываются поочередно.

И последнее, что осталось сделать перед перерывом: мы ввели две глобальные переменные. С глобальными переменными часто возникают проблемы, поэтому от них лучше избавиться.

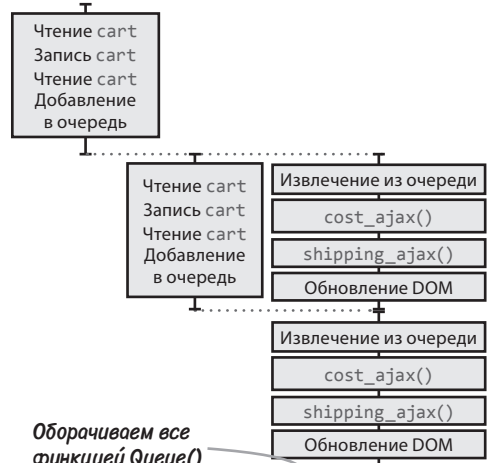
Упаковка переменных и функций в области видимости функции

Мы используем две глобальные изменяемые переменные. Упакуем их (а также функции, к ним обращающиеся) в новую функцию, которую мы назовем `Queue()`. Поскольку мы ожидаем, что `update_total_queue()` будет вызываться только клиентским кодом, мы вернем соответствующее значение из функции, сохраним в переменной и используем:

Текущая диаграмма



Нужная диаграмма



Текущая версия

```
var queue_items = [];
var working = false;

function runNext() {
  if(working)
    return;
  if(queue_items.length === 0)
    return;
  working = true;
  var cart = queue_items.shift();
  calc_cart_total(cart, function(total) {
    update_total_dom(total);
    working = false;
    runNext();
  });
}

function update_total_queue(cart) {
  queue_items.push(cart);
  setTimeout(runNext, 0);
}
```

Глобальные переменные становятся локальными для Queue()

Queue() возвращает функцию, которая добавляется в очередь

Новая версия

```
function Queue() {
  var queue_items = [];
  var working = false;

  function runNext() {
    if(working)
      return;
    if(queue_items.length === 0)
      return;
    working = true;
    var cart = queue_items.shift();
    calc_cart_total(cart, function(total) {
      update_total_dom(total);
      working = false;
      runNext();
    });
  }

  return function(cart) {
    queue_items.push(cart);
    setTimeout(runNext, 0);
  };
}

var update_total_queue = Queue();
```

Оборачиваем все функцией Queue()

Возвращенная функция выполняется, как и прежде

Упаковывая переменные в области видимости функции, мы гарантируем, что ничто не сможет изменить их за пределами небольшого фрагмента кода внутри функции. Такое решение также позволяет создать несколько очередей, хотя все они делают одно и то же (добавляют товары в корзину).



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Не многовато ли изменений для функционального программирования?

О: Очень хороший вопрос. Функциональное программирование не диктует никаких конкретных привычек программирования. Вместо этого оно предоставляет основу для обдумывания принимаемых решений. В данном случае `update_total_queue()` является действием. Это совместно используемый ресурс, связанный с порядком и количеством вызовов. ФП говорит, что мы должны уделять особенно значительное внимание действиям — безусловно большее, чем вычислениям.

Важно заметить, что очередь была тщательно построена с расчетом на совместное использование. Любая временная линия может вызвать `update_total_queue()`, и ее поведение будет прогнозируемым. ФП помогает взять действия под контроль. И если для этого вам понадобится пара изменяемых значений, это нормально.

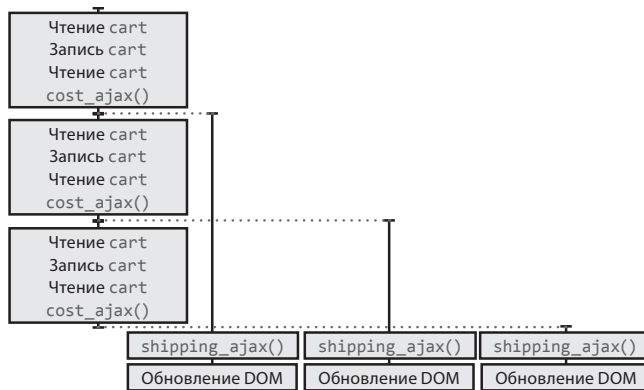
В: Для чего добавлять `runNext()` в обратный вызов? Если нужно вызвать `runNext()` после `calc_cart_total()`, почему не сделать это в следующей строке?

О: Вы спрашиваете, почему мы сделали это вместо вот этого:

<pre>calc_cart_total(cart, function(total) { update_total_dom(total); working = false; runNext(); });</pre>	<pre>calc_cart_total(cart, update_total_dom); working = false; runNext();</pre>
---	---

Дело в том, что функция `calc_cart_total()` асинхронна. Она содержит шаги, которые будут выполнены в какой-то момент в будущем: ответы двух вызовов `ajax` будут добавлены в очередь событий и обработаны циклом событий. А тем временем обрабатываются другие события.

Если вызвать `runNext()` немедленно, функция запустит следующий элемент, пока запросы `ajax` находятся в незавершенном состоянии, и мы не получим ожидаемого поведения. Таков путь JavaScript.



В: Что-то слишком много кода для того, чтобы две временные линии могли совместно использовать ресурс. Нет ли более простого способа?

О: Тщательная разработка очереди состояла из нескольких шагов. Однако заметим, что код получился довольно компактным. И большая часть этого кода будет пригодна для повторного использования.



Принцип: берите образец решения по совместному использованию из реального мира

Мы, люди, постоянно используем ресурсы совместно. У нас это получается вполне естественно. Проблема в том, что компьютеры не знают, как устроить совместное использование. Нам приходится писать программы специально с расчетом на совместное использование.

Мы решили построить очередь, потому что очереди очень часто используются для организации совместного использования ресурсов. Мы выстраиваемся в очередь в банк, в туалет, на остановке автобуса...

Очереди используются очень часто, но они не идеальны. У них есть свои недостатки, например они требуют ожидания. Существуют и другие способы совместного использования ресурсов, которые избавлены от этих недостатков.

- **Защелки на дверях туалета** обеспечивают соблюдение правила «не более одного посетителя в любой момент».
- **Публичные библиотеки** позволяют сообществу совместно использовать множество книг.
- **Доска на стене** позволяет одному преподавателю передавать информацию целому классу.

Все эти (и многие другие) схемы могут использоваться для программирования доступа к общим ресурсам. Более того, как вы сейчас увидите, написанные нами конструкции можно использовать повторно.



Пища для ума

А вы сможете припомнить другие способы совместного использования ресурсов в реальном мире? Составьте список. Поразмыслите над тем, как они работают.

Совместное использование очереди

Выделение функции done()

Нам хотелось бы, чтобы очередь была на 100 % пригодной для повторного использования. В данный момент она позволяет только добавлять товары в корзину. Но воспользовавшись методом замены тела обратным вызовом, мы можем отделить код цикла очереди (вызов `runNext()`) от работы, которая должна выполняться очередью (вызов `calc_cart_total()`).

Текущая версия

```
function Queue() {
  var queue_items = [];
  var working = false;

  function runNext() {
    if(working)
      return;
    if(queue_items.length === 0)
      return;
    working = true;
    var cart = queue_items.shift();

    calc_cart_total(cart, function(total) {
      update_total_dom(total);

      working = false;
      runNext();
    });
  }

  return function(cart) {
    queue_items.push(cart);
    setTimeout(runNext, 0);
  };
}

var update_total_queue = Queue();
```

Две строки выделяются в новую функцию

Тело

Новая версия

```
function Queue() {
  var queue_items = [];
  var working = false;

  function runNext() {
    if(working)
      return;
    if(queue_items.length === 0)
      return;
    working = true;
    var cart = queue_items.shift();
    function worker(cart, done) {
      calc_cart_total(cart, function(total) {
        update_total_dom(total);
        done(total);
      });
    }
    worker(cart, function() {
      working = false;
      runNext();
    });
  }

  return function(cart) {
    queue_items.push(cart);
    setTimeout(runNext, 0);
  };
}

var update_total_queue = Queue();
```

done — имя обратного вызова

Локальная переменная cart также выделяется в аргумент

`done()` — это обратный вызов, который продолжает работу временной линии очереди. Он присваивает `working` значение `false`, чтобы при следующей проверке не произошел преждевременный возврат. Затем вызов `runNext()` запускает следующую итерацию. Теперь функция `worker()` изолирована, и мы можем выделить ее в аргумент `Queue()`.

Извлечение специализированного поведения работника

Сейчас наша очередь специализируется на добавлении товаров в корзину. Возможно, в будущем вы захотите создать универсальную очередь, которая может использоваться для многих разных операций. Можно провести другой рефакторинг с выделением аргумента функции, чтобы исключить специализированный код и передавать его очереди при создании:

Текущая версия

```
function Queue() {
  var queue_items = [];
  var working = false;

  function runNext() {
    if(working)
      return;
    if(queue_items.length === 0)
      return;
    working = true;
    var cart = queue_items.shift();
    function worker(cart, done) {
      calc_cart_total(cart, function(total) {
        update_total_dom(total);
        done(total);
      });
    }
    worker(cart, function() {
      working = false;
      runNext();
    });
  }

  return function(cart) {
    queue_items.push(cart);
    setTimeout(runNext, 0);
  };
}
```

```
var update_total_queue = Queue();
```

Новая версия

```
function Queue(worker) {
  var queue_items = [];
  var working = false;

  function runNext() {
    if(working)
      return;
    if(queue_items.length === 0)
      return;
    working = true;
    var cart = queue_items.shift();

    worker(cart, function() {
      working = false;
      runNext();
    });
  }

  return function(cart) {
    queue_items.push(cart);
    setTimeout(runNext, 0);
  };
}
```

Добавляется новый аргумент — функция, которая выполняет специализированную работу

```
function calc_cart_worker(cart, done) {
  calc_cart_total(cart, function(total) {
    update_total_dom(total);
    done(total);
  });
}
```

```
var update_total_queue = Queue(calc_cart_worker);
```

Мы создали обобщенную очередь! Функция `Queue()` содержит только обобщенный код, а все специализированное передается в аргументе. Давайте поразмыслим над тем, что же мы только что сделали.

Получение обратного вызова при завершении задачи

Нашим программистам нужна еще одна возможность — передача обратного вызова, который будет активизироваться при завершении задачи. Данные задачи можно сохранить вместе с обратным вызовом в маленьком объекте. Вот что будет помещаться в очередь:

Текущая версия

```
function Queue(worker) {
  var queue_items = [];
  var working = false;

  function runNext() {
    if(working)
      return;
    if(queue_items.length === 0)
      return;
    working = true;
    var cart = queue_items.shift();
    worker(cart, function() {
      working = false;
      runNext();
    });
  }

  return function(cart) {
    queue_items.push(cart);

    setTimeout(runNext, 0);
  };
}

function calc_cart_worker(cart, done) {
  calc_cart_total(cart, function(total) {
    update_total_dom(total);
    done(total);
  });
}
```

*worker передаются
только данные*

*Данные вместе
с обратным вызовом
заносятся в массив*

```
var update_total_queue = Queue(calc_cart_worker); var update_total_queue = Queue(calc_cart_worker);
```

Новая версия

```
function Queue(worker) {
  var queue_items = [];
  var working = false;

  function runNext() {
    if(working)
      return;
    if(queue_items.length === 0)
      return;
    working = true;
    var item = queue_items.shift();
    worker(item.data, function() {
      working = false;
      runNext();
    });
  }

  return function(data, callback) {
    queue_items.push({
      data: data,
      callback: callback || function(){}
    });
    setTimeout(runNext, 0);
  };
}

function calc_cart_worker(cart, done) {
  calc_cart_total(cart, function(total) {
    update_total_dom(total);
    done(total);
  });
}
```



Дженна
из команды
разработки

*Очередь — это
хорошо, но мне нужен
обратный вызов, который будет
срабатывать при завершении
задачи.*

Идиома JavaScript используется для определения значения по умолчанию для `callback`. Значение `callback` может быть не определено — это может произойти, если второй аргумент не передается. Мы хотим иметь возможность запускать обратный вызов безусловно, поэтому мы используем эту идиому для замены неопределенного обратного вызова функцией, которая не делает ничего.

Теперь мы сохраняем обратный вызов, но он еще не вызывается. Это будет сделано на следующей странице.

`callback || function(){}`

*Если обратный вызов не определен,
вместо него используется функция,
которая ничего не делает*

Активизация обратного вызова при завершении задачи

На предыдущей странице мы занимались получением и сохранением обратного вызова вместе с данными задачи. Теперь необходимо организовать активизацию обратного вызова при завершении задачи:

Текущая версия

```
function Queue(worker) {
  var queue_items = [];
  var working = false;

  function runNext() {
    if(working)
      return;
    if(queue_items.length === 0)
      return;
    working = true;
    var item = queue_items.shift();
    worker(item.data, function() {
      working = false;
      runNext();
    });
  }

  return function(data, callback) {
    queue_items.push({
      data: data,
      callback: callback || function(){}
    });
    setTimeout(runNext, 0);
  };
}

function calc_cart_worker(cart, done) {
  calc_cart_total(cart, function(total) {
    update_total_dom(total);
    done(total);
  });
}
```

```
var update_total_queue = Queue(calc_cart_worker);
```

Новая версия

```
function Queue(worker) {
  var queue_items = [];
  var working = false;

  function runNext() {
    if(working)
      return;
    if(queue_items.length === 0)
      return;
    working = true;
    var item = queue_items.shift();
    worker(item.data, function(val) {
      working = false;
      setTimeout(item.callback, 0, val);
      runNext();
    });
  }

  return function(data, callback) {
    queue_items.push({
      data: data,
      callback: callback || function(){}
    });
    setTimeout(runNext, 0);
  };
}

function calc_cart_worker(cart, done) {
  calc_cart_total(cart, function(total) {
    update_total_dom(total);
    done(total);
  });
}
```

```
var update_total_queue = Queue(calc_cart_worker);
```

Функция Queue() универсальна, поэтому для переменных также выбраны обобщенные имена

done() получает аргумент

Организуем асинхронный вызов item.callback

val передается обратному вызову

Корзина получает данные товара; при завершении вызывается done()

Здесь уже известно, что конкретно происходит, поэтому мы используем конкретные имена переменных

Обратите внимание: в коде Queue() содержатся ссылки на item.data и val. Здесь используются обобщенные имена, потому что мы не знаем, для чего будет использоваться Queue(). Однако в коде calc_cart_worker() мы обращаемся к тем же значениям по именам cart и total (соответственно), потому что к этому моменту задача уже известна. Имена переменных должны отражать уровень детализации, на котором вы работаете.

Очередь стала пригодной для повторного использования. Она полностью упорядочивает все задачи, проходящие через нее, и позволяет временной линии продолжиться после завершения. На нескольких ближайших страницах мы более внимательно изучим результат нашей работы.

Функция высшего порядка расширяет возможности действия

Мы построили функцию с именем `Queue()`, которая получает функцию в аргументе и возвращает новую функцию.

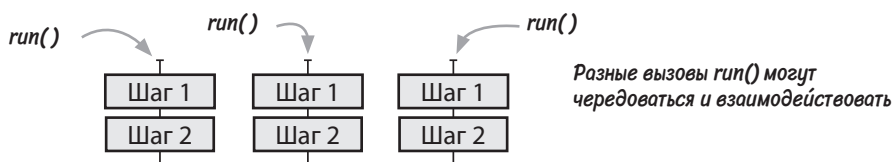
Функция Функция Функция

```
var update_total_queue = Queue(calc_cart_worker);
```

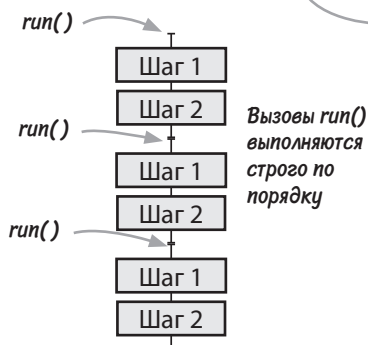
Мы создали функцию высшего порядка, которая получает функцию, создающую временную линию, и делает так, чтобы в любой момент времени могла выполняться только одна версия временной линии.

`Queue()` преобразует временную линию вида

Помните супергеройский костюм из главы 11?



к такому виду:



Другими словами, `Queue()` наделяет действия суперспособностью — гарантией определенного порядка.

Возможно, вместо `Queue()` стоило бы назвать функцию `linearize()`, потому что она обеспечивает линейный порядок вызовов действия. В ней используется очередь, но это всего лишь подробность внутренней реализации.

`Queue()` является примитивом синхронизации (небольшой блок повторно используемого кода, который помогает нескольким временным линиям выполняться правильно). Работа примитивов синхронизации обычно основана

на ограничении возможных вариантов упорядочивания. Если устранить нежелательные варианты, то код будет гарантированно выполняться в одном из желательных порядков.

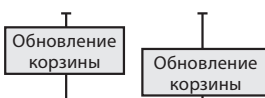


Загляни в словарь

Примитив синхронизации — небольшой блок функциональности, который помогает организовать совместное использование ресурсов между временными линиями.

Одновременное выполнение **невозможно**

Потоковая модель JavaScript делает одновременное выполнение невозможным, поэтому ее можно исключить. Тем не менее при других потоковых моделях придется учитывать такую возможность.

Сначала левое **желательно**

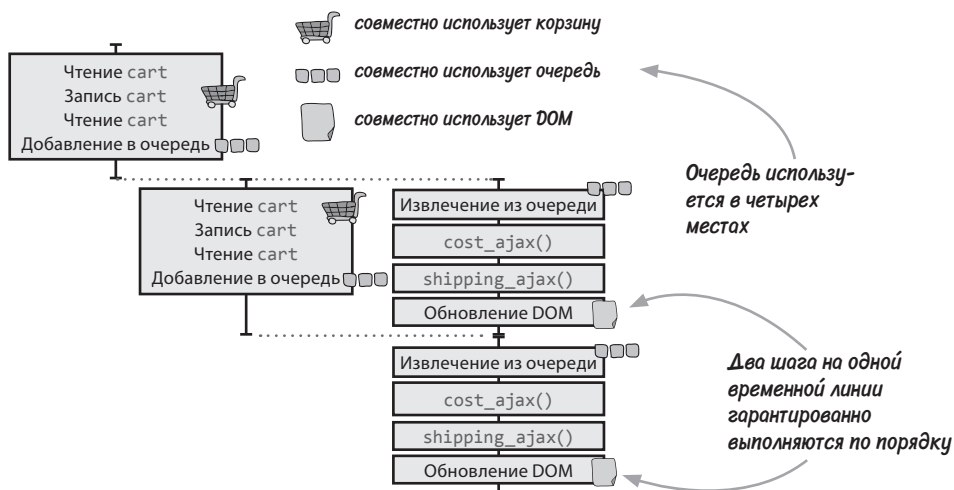
Желательное поведение. Два обработчика кликов выполняются в порядке, соответствующем порядку кликов.

Сначала правое **невозможно**

Такое поведение нежелательно, но оно возможно из-за пунктирной линии. Пунктир представляет порядок событий UI в очереди событий, которая сохраняет порядок. Обработчики выполняются в том же порядке, что и клики.

С корзиной все хорошо, переходим к DOM.

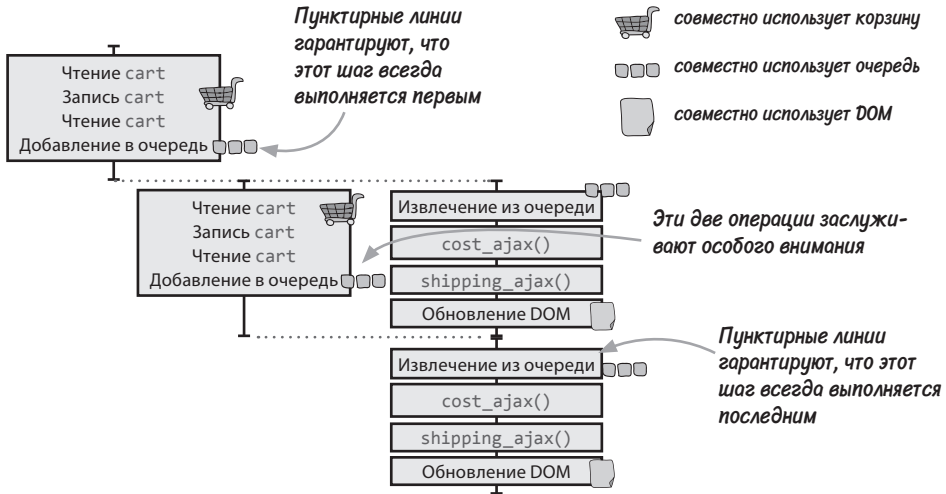
Мы только что видели, что ресурсы корзины используются правильно. Теперь перейдем к модели DOM, ради которой, собственно, и создавалась очередь:



Поскольку все обновления DOM были вынесены на одну временную линию благодаря использованию очереди, они не могут происходить с нарушением порядка. Они будут происходить в порядке, соответствующем порядку кликов на кнопке добавления товара в корзину. Нам даже не нужно проверять варианты упорядочения, потому что они находятся на одной временной линии. Действия на одной временной линии всегда выполняются по порядку.

Последний совместно используемый ресурс — очередь. Он используется на четырех разных шагах! Работа с очередью рассматривается на следующей странице.

На предыдущей странице вы видели, что обновления DOM всегда выполняются в правильном порядке. Однако сейчас мы сталкиваемся с тем, что кажется более серьезной проблемой. Очередь совместно используется в четырех разных шагах трех временных линий. Посмотрим, как ее можно проанализировать. Начнем с исключения простых случаев:



Согласно диаграмме, одно из добавлений в очередь будет предшествовать всем остальным операциям, связанным с очередью. Кроме того, одно извлечении из очереди будет предшествовать всем остальным операциям, связанным с очередью. Об этом нам сообщают пунктирные линии.

Остаются две средние операции. Необходимо проверить, что все варианты порядка, в котором они выполняются, являются либо желательными, либо невозможными:

Добавление в очередь

```
queue_items.push({
  data: data,
  callback: callback
});
```

Извлечение из очереди

```
queue_items.shift();
```

Одновременное выполнение невозможно



Потоковая модель JavaScript делает одновременное выполнение невозможным, поэтому ее можно исключить. Тем не менее при других потоковых моделях придется учитывать такую возможность.

Сначала левое желательно



Желательное поведение. Если одна временная линия добавляет в очередь до того, как другая извлекает из очереди, это нормально. Порядок элементов будет сохранен.

Сначала правое желательно



Это поведение также является желательным. Мы можем взять существующий элемент из очереди, а затем добавить еще один, и это не создаст никаких проблем. Порядок элементов сохраняется.

Мы не можем гарантировать, что одно из этих действий произойдет раньше другого. Но это нормально! Оба варианта упорядочения приводят к одному правильному результату. Очередь как примитив синхронизации гарантирует это.



Принцип: чтобы узнать о возможных проблемах, проанализируйте временную диаграмму

Самое большое достоинство временных диаграмм в том, что они наглядно демонстрируют проблемы синхронизации. Вы видите совместно используемые ресурсы и понимаете, не выполняются ли они в ошибочном порядке. Мы воспользуемся этим обстоятельством и нарисуем диаграмму.

Это необходимо, потому что ошибки синхронизации невероятно трудно воспроизвести. Они неочевидны при просмотре кода и могут пройти все тесты. Даже если тесты выполняются сотни раз, это не гарантирует воспроизведения всех возможных вариантов упорядочения. Но после того как код отправится в эксплуатацию и будет выполняться у тысяч и миллионов пользователей, маловероятное когда-нибудь неизбежно произойдет. Временные диаграммы наглядно показывают наличие ошибок без запуска кода в эксплуатацию.

Если вы программируете действия, стоит нарисовать временную диаграмму. Этот гибкий инструмент поможет понять, как может работать ваша программа, включая все возможные варианты упорядочения.

Пропуск задач в очереди

Да, Сара права. При такой реализации очереди рабочий процесс будет запускать каждую задачу до завершения, прежде чем переходить к следующей. Код будет работать очень медленно.

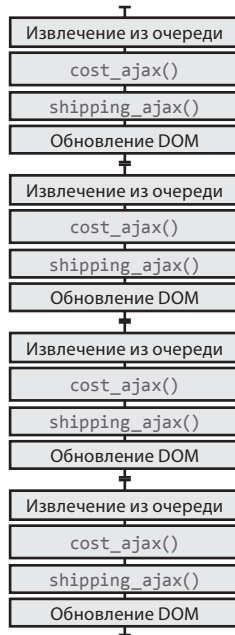
Представьте, что пользователь очень быстро нажал кнопку добавления товара четыре раза. В DOM должна отображаться только последняя общая стоимость, но наша очередь обработает все четыре обновления, одно за другим. В каждом случае задействованы два запроса AJAX, поэтому может пройти целая секунда, прежде чем вы увидите актуальную сумму.

Конечно, правильный порядок мы выдержали, но программа будет очень медленной!



Сара из команды разработки

Очередь после четырех быстрых нажатий кнопки



Эти элементы будут обрабатываться по порядку

Эти обновления DOM происходят, но они не выводят окончательный ответ

Нас интересует только последнее обновление DOM

Мириться с этим нельзя. Обратите внимание на то, что нам действительно необходимо только последний элемент в очереди. Другие будут немедленно перезаписаны сразу же после завершения следующего элемента. А если удалять элементы, которые все равно будут перезаписаны? Для этого достаточно внести одно небольшое изменение в текущий код очереди.

Текущая версия очереди выполняет каждую задачу до завершения, прежде чем запускать следующую. Нам хотелось бы, чтобы она пропускала старую работу при поступлении новой:

Нормальная очередь

```
function Queue(worker) {
  var queue_items = [];
  var working = false;

  function runNext() {
    if(working)
      return;
    if(queue_items.length === 0)
      return;
    working = true;
    var item = queue_items.shift();
    worker(item.data, function(val) {
      working = false;
      setTimeout(item.callback, 0, val);
      runNext();
    });
  }

  return function(data, callback) {
    queue_items.push({
      data: data,
      callback: callback || function(){}
    });
    setTimeout(runNext, 0);
  };
}

function calc_cart_worker(cart, done) {
  calc_cart_total(cart, function(total) {
    update_total_dom(total);
    done(total);
  });
}

var update_total_queue =
  Queue(calc_cart_worker);
```

*Продолжаем удалять
элементы от начала,
пока не останется так
или менее*

Очередь с пропуском

```
function DroppingQueue(max, worker) {
  var queue_items = [];
  var working = false;

  function runNext() {
    if(working)
      return;
    if(queue_items.length === 0)
      return;
    working = true;
    var item = queue_items.shift();
    worker(item.data, function(val) {
      working = false;
      setTimeout(item.callback, 0, val);
      runNext();
    });
  }

  return function(data, callback) {
    queue_items.push({
      data: data,
      callback: callback || function(){}
    });
    while(queue_items.length > max)
      queue_items.shift();
    setTimeout(runNext, 0);
  };
}

function calc_cart_worker(cart, done) {
  calc_cart_total(cart, function(total) {
    update_total_dom(total);
    done(total);
  });
}

var update_total_queue =
  DroppingQueue(1, calc_cart_worker);
```

*Переименовываем
в DroppingQueue*

*Передается
максимальное
количество остав-
ляемых задач*

Удаляем все, кроме одного

С таким изменением `update_total_queue` никогда не станет длиннее одного необработанного элемента, сколько бы элементов мы ни добавляли и как бы быстро это ни делали. Пользователю придется ждать максимум два циклических обращения к серверу (вместо всех).

Очень небольшое изменение в коде очереди приводит к улучшению поведения в нашем сценарии использования. Обе очереди выглядят достаточно обычно, чтобы мы решили оставить их. Важно то, что мы можем использовать очереди как примитив синхронизации, пригодный для повторного использования с другими ресурсами.



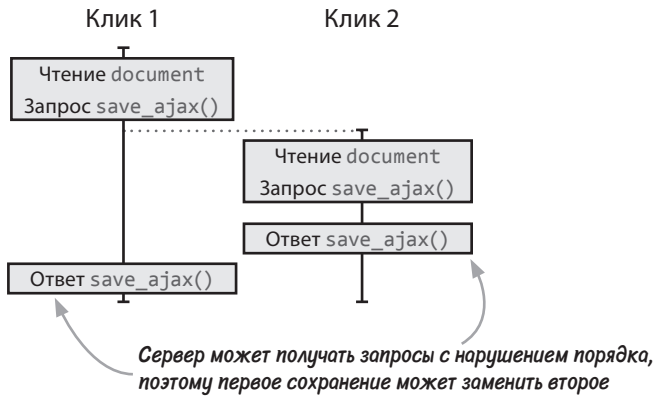
Ваш ход

Одна из проблем с реализацией кнопки сохранения документа заключается в том, что при медленной работе сети вызовы `save_ajax()` могут перезаписывать друг друга. Ниже приведена временная диаграмма, поясняющая суть проблемы. Используйте очередь с пропуском для решения этой проблемы.

```
var document = {...};
```

```
function save_ajax(document, callback) {...}
```

```
saveButton.addEventListener('click', function() {  
    save_ajax(document);  
});
```



Запишите здесь
свой ответ



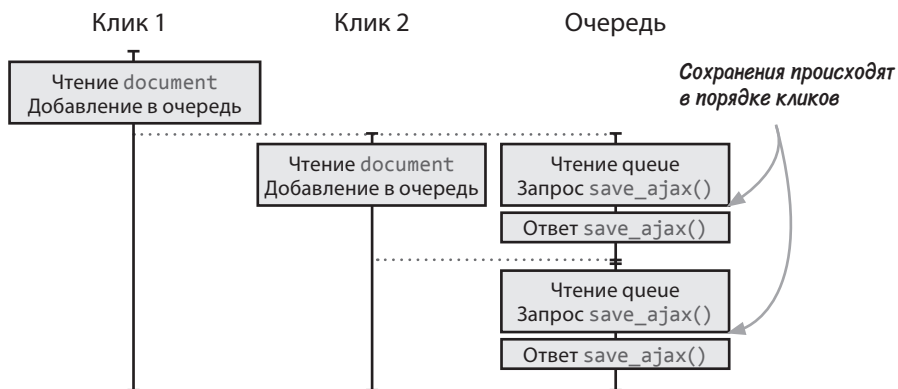
Ответ

```
var document = {...};

function save_ajax(document, callback) {...}

var save_ajax_queued = DroppingQueue(1, save_ajax);

saveButton.addEventListener('click', function() {
  save_ajax_queued(document);
});
```



Итоги главы

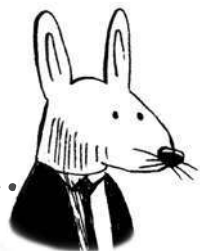
В этой главе мы занялись диагностикой проблем, связанных с совместным использованием ресурсов. Обновление DOM должно происходить в определенном порядке. После того как проблема была обнаружена, мы решили ее построением очереди. После доработки очередь превратилась в функцию высшего порядка с широкими возможностями повторного использования.

Резюме

- Проблемы синхронизации трудно воспроизвести, и они часто остаются незамеченными в ходе тестирования. Используйте временные диаграммы для анализа и диагностики проблем синхронизации.
- Если вы столкнулись с ошибкой, связанной с совместным использованием ресурсов, поищите в реальном мире образцы для ее решения. Люди постоянно делятся информацией и очень часто без малейших проблем. Учитесь у людей.
- Стройте универсальные средства, которые помогают совместно использовать ресурсы. Они называются *примитивами синхронизации*, а ваш код становится более понятным и простым.
- Примитивы синхронизации часто реализуются в форме функций высшего порядка для действий. Эти функции высшего порядка наделяют действия суперспособностями.
- Самостоятельная реализация примитивов синхронизации не обязана быть сложной. Не торопитесь, применяйте рефакторинг, и вы сможете построить собственные примитивы синхронизации.

Что дальше?

Вы научились диагностировать проблемы совместного использования ресурсов и решать их с помощью специализированных примитивов синхронизации. В следующей главе вы научитесь координировать две временные линии, чтобы они могли совместно работать над решением задачи.



В этой главе

- ✓ Создание примитивов для координации нескольких временных линий.
- ✓ Манипуляция двумя важными аспектами времени: упорядочением и повторением.

В предыдущей главе вы научились диагностировать ошибки, связанные с совместным использованием ресурсов, а также создали *примитив синхронизации* для безопасного совместного использования ресурсов. Иногда несколько временных линий должны работать вместе при отсутствии явного общего ресурса. В этой главе мы построим примитив синхронизации, который поможет координировать временные линии и исключить неправильные варианты упорядочения.

Принципы работы с временными линиями

Еще раз напомним вам основные принципы. В двух последних главах мы про-работали принципы 1–4 и показали, как они помогают обеспечить правильность выполнения. Эта глава посвящена принципу 5. Вы научитесь рассматривать само время как нечто такое, чем можно управлять.

1. Чем меньше временных линий, тем проще

Каждая новая временная линия ощутимо усложняет понимание системы. Если вам удастся сократить количество временных линий (t в формуле справа), это очень сильно упростит вашу задачу. К сожалению, часто мы не можем управлять количеством временных линий.

Формула для определения количества возможных вариантов упорядочения

$$O = \frac{(ta)!}{(a!)^t}$$

Количество временных линий t — количество действий на одну временную линию a — возможные варианты упорядочения O — факториал

2. Чем короче временные линии, тем проще

Если вам удастся устранить шаги на временной диаграмме (уменьшить a в формуле справа), это также позволит радикально сократить количество возможных вариантов упорядочения.

3. Чем меньше совместного использования ресурсов, тем проще

При рассмотрении двух временных линий достаточно учитывать только те шаги, в которых ресурсы используются совместно. При этом сокращается количество шагов на диаграмме, а следовательно, и количество вариантов, которые вам приходится учитывать.

4. Координируйте совместное использование ресурсов

Координация между временными линиями означает исключение возможных вариантов упорядочения, которые не дают правильного результата; при этом также упрощается анализ.

5. Рассматривайте время как первоклассную концепцию

Упорядочить действия и правильно выбрать момент для их выполнения непросто. Для упрощения задачи можно создать объекты, манипулирующие временной линией. Важнейшими временными аспектами — упорядочением вызовов и повторением вызовов — можно управлять напрямую.

В каждом языке существует неявная модель времени. Тем не менее эта модель времени часто не соответствует модели, необходимой для решения задачи. В функциональном программировании можно создать новую модель времени, которая лучше подходит для задачи.

Применим принцип 5 к корзине, которая теперь содержит новую ошибку!



Ошибка!

Прошло несколько недель с момента запуска в эксплуатацию кода очереди корзины, представленного в последней главе. С того момента появился запрос на скорость пользовательского интерфейса. Все, что может замедлить работу корзины или кнопок добавления в корзину, подвергалось жесткой оптимизации. И теперь мы столкнулись с ошибкой.

Ошибка заключается в том, что даже всего с одним товаром иногда выводится неправильная сумма. Попробуем воспроизвести ошибку:

Начинаем с пустой корзины

Один клик

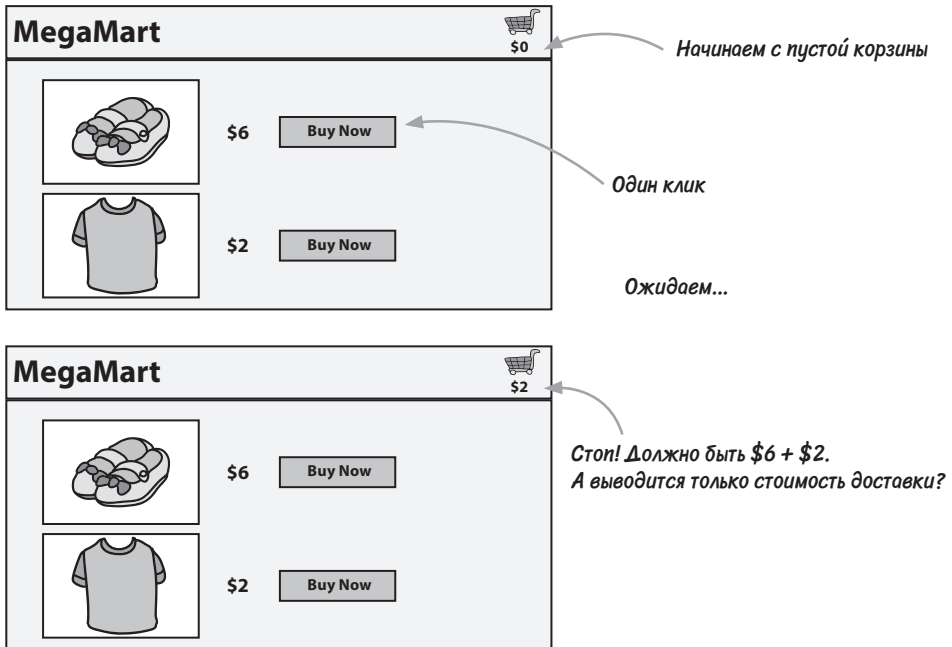
Ожидаем...

\$6 + \$2 за доставку — правильное значение

Ого! Быстро. Спецы по оптимизации действительно помогли.

Дженна из команды разработки

А вот как выглядит приложение при возникновении ошибки:



И это только для добавления одного товара в корзину! Код также не работает при быстром добавлении нескольких товаров, но я не буду демонстрировать этот факт. Начнем с исправления случая с одним товаром.

Как изменился код

До оптимизаций все работало прекрасно. Теперь ошибка иногда происходит даже при добавлении одного товара. Возьмем код из предыдущей главы и сравним его с тем, что имеем сейчас, после оптимизации быстрого действия.

До оптимизации (работает)

```
function add_item_to_cart(item) {
    cart = add_item(cart, item);
    update_total_queue(cart);
}
```

```
function calc_cart_total(cart, callback) {
    var total = 0;
    cost_ajax(cart, function(cost) {
        total += cost;

        shipping_ajax(cart, function(shipping) {
            total += shipping;
            callback(total);
        });
    });
}
```

```
function calc_cart_worker(cart, done) {
    calc_cart_total(cart, function(total) {
        update_total_dom(total);
        done(total);
    });
}
```

```
var update_total_queue =
    DroppingQueue(1, calc_cart_worker);
```

После оптимизации (не работает)

```
function add_item_to_cart(item) {
    cart = add_item(cart, item);
    update_total_queue(cart);
}
```

```
function calc_cart_total(cart, callback) {
    var total = 0;
    cost_ajax(cart, function(cost) {
        total += cost;
    });
    shipping_ajax(cart, function(shipping) {
        total += shipping;
        callback(total);
    });
}
```

```
function calc_cart_worker(cart, done) {
    calc_cart_total(cart, function(total) {
        update_total_dom(total);
        done(total);
    });
}
```

```
var update_total_queue =
    DroppingQueue(1, calc_cart_worker);
```

*Закрывающая фигурная и круглая
скобки переместились в другое
место*

Похоже, закрывающая фигурная и круглая скобки были перемещены в ходе оптимизации. В результате вызов `shipping_ajax()` может происходить немедленно (а не в обратном вызове `cost_ajax()`). Конечно, код будет работать быстрее, потому что два запроса ajax выполняются одновременно. С другой стороны, это очевидным образом приводит к ошибке.

Нарисуем временную диаграмму, чтобы вы лучше поняли, что здесь происходит.

Идентификация действий: шаг 1

На предыдущей странице были продемонстрированы различия в коде. Простое выведение кода из обратного вызова ускорило работу, но создало ошибку в программе. Начнем с идентификации действий:

```
function add_item_to_cart(item) {
  cart = add_item(cart, item);
  update_total_queue(cart);
}

function calc_cart_total(cart, callback) {
  var total = 0;
  cost_ajax(cart, function(cost) {
    total += cost;
  });
  shipping_ajax(cart, function(shipping) {
    total += shipping;
    callback(total);
  });
}

function calc_cart_worker(cart, done) {
  calc_cart_total(cart, function(total) {
    update_total_dom(total);
    done(total);
  });
}

var update_total_queue = DroppingQueue(1, calc_cart_worker);
```

Действия подчеркнуты в коде

Три шага построения диаграммы

1. Идентификация действий.
2. Отображение действий на диаграмме.
3. Упрощение.

При создании этой диаграммы необходимо действовать очень внимательно, поэтому мы будем учитывать переменную **total**, несмотря на то что она является локальной. Вспомните, о чем говорилось пару глав назад: локальную переменную **total** можно удалить с диаграмм, потому что все обращения к **total** происходят на одной временной линии. Однако теперь переменная стала доступной для нескольких временных линий. Мы не знаем, используется ли она безопасно (вскоре мы увидим, что нет). При рисовании диаграммы очень важно начать на пустом месте, без каких-либо допущений. Диаграмму всегда можно снова упростить на шаге 3.

Обратимся к шагу 2.

Представление каждого действия: шаг 2

На шаге 1 мы идентифицировали все действия в коде. Перейдем к шагу 2, на котором мы начнем рисовать действия на диаграмме. Помните, что мы начинаем с пустого места. Все предположения, которые могли быть сделаны ранее, полностью игнорируются. Мы сможем снова оптимизировать код на шаге 3.

```

1 function add_item_to_cart(item) {
2   cart = add_item(cart, item);
3   update_total_queue(cart);
4 }
5
6 function calc_cart_total(cart, callback) {
7   var total = 0;
8   cost_ajax(cart, function(cost) {
9     total += cost;
10  });
11  shipping_ajax(cart, function(shipping) {
12    total += shipping;
13    callback(total);
14  });
15 }
16
17 function calc_cart_worker(cart, done) {
18   calc_cart_total(cart, function(total) {
19     update_total_dom(total);
20     done(total);
21   });
22 }
23
24 var update_total_queue = DroppingQueue(1, calc_cart_worker);

```

Три шага построения диаграммы

1. Идентификация действий.
2. Отображение действий на диаграмме.
3. Упрощение.

Действия

1. Чтение cart.
2. Запись cart.
3. Чтение cart.
4. Вызов update_total_queue().
5. Инициализация total = 0.
6. Вызов cost_ajax().
7. Чтение total.
8. Запись total.
9. Вызов shipping_ajax().
10. Чтение total.
11. Запись total.
12. Чтение total.
13. Вызов update_total_dom().

Начнем рисовать диаграмму. Так как у вас уже есть некоторый опыт, действия будут обрабатываться небольшими блоками, а не по одному:

```

2   cart = add_item(cart, item);
3   update_total_queue(cart);

```



Эти четыре действия работают синхронно, поэтому они размещаются на одной временной линии

Это действие выполняет добавление в очередь

На предыдущей странице мы начали рисовать временную диаграмму. Продолжим. Итак, мы прошли строку 3:

```

1 function add_item_to_cart(item) {
2   cart = add_item(cart, item);
3   update_total_queue(cart);
4 }
5
6 function calc_cart_total(cart, callback) {
7   var total = 0;
8   cost_ajax(cart, function(cost) {
9     total += cost;
10  });
11  shipping_ajax(cart, function(shipping) {
12    total += shipping;
13    callback(total);
14  });
15 }
16
17 function calc_cart_worker(cart, done) {
18   calc_cart_total(cart, function(total) {
19     update_total_dom(total);
20     done(total);
21   });
22 }
23
24 var update_total_queue = DroppingQueue(1, calc_cart_worker);

```

Действия

1. Чтение cart.
2. Запись cart.
3. Чтение cart.
4. Вызов update_total_queue().
5. Инициализация total = 0.
6. Вызов cost_ajax().
7. Чтение total.
8. Запись total.
9. Вызов shipping_ajax().
10. Чтение total.
11. Запись total.
12. Чтение total.
13. Вызов update_total_dom().

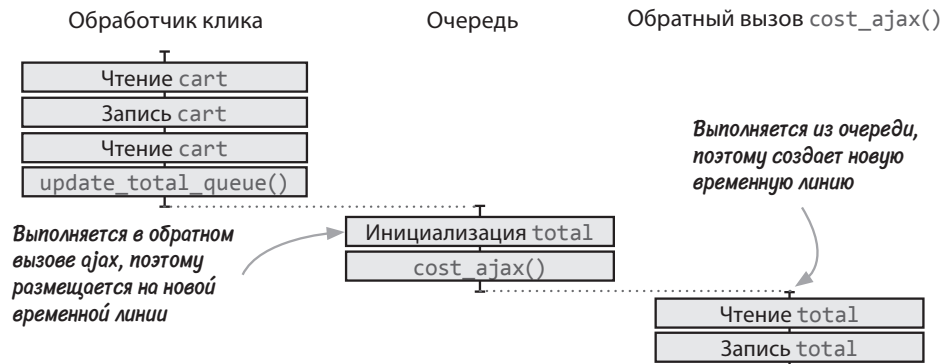
Далее идет следующий фрагмент кода:

```

7   var total = 0;
8   cost_ajax(cart, function(cost) {
9     total += cost;
10  });

```

Вспомните, += состоит из чтения и записи



Мы проработали два блока кода и нанесли на диаграмму восемь действий. Осталось еще три. Разделим их на два блока:

```

1 function add_item_to_cart(item) {
2   cart = add_item(cart, item);
3   update_total_queue(cart);
4 }
5
6 function calc_cart_total(cart, callback) {
7   var total = 0;
8   cost_ajax(cart, function(cost) {
9     total += cost;
10  });
11  shipping_ajax(cart, function(shipping) {
12    total += shipping;
13    callback(total);
14  });
15 }
16
17 function calc_cart_worker(cart, done) {
18   calc_cart_total(cart, function(total) {
19     update_total_dom(total);
20     done(total);
21   });
22 }
23
24 var update_total_queue = DroppingQueue(1, calc_cart_worker);

```

Действия

1. Чтение cart.
2. Запись cart.
3. Чтение cart.
4. Вызов update_total_queue().
5. Инициализация total = 0.
6. Вызов cost_ajax().
7. Чтение total.
8. Запись total.
9. Вызов shipping_ajax().
10. Чтение total.
11. Запись total.
12. Чтение total.
13. Вызов update_total_dom().

Теперь вызываем shipping_ajax():

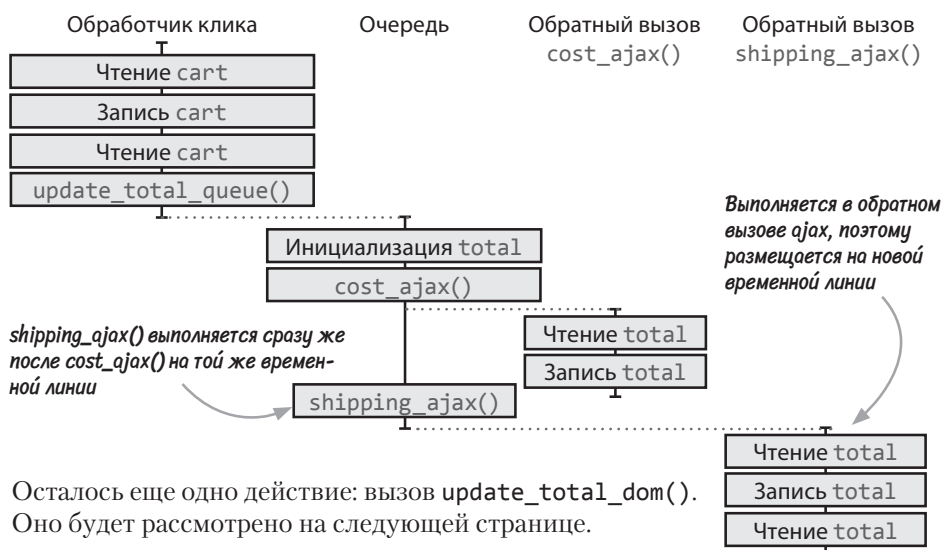
```

11 shipping_ajax(cart, function(shipping) {
12   total += shipping;
13   callback(total);
14 });

```

Три шага построения диаграммы

1. Идентификация действий.
2. Отображение действий на диаграмме.
3. Упрощение.



Осталось еще одно действие: вызов update_total_dom(). Оно будет рассмотрено на следующей странице.

Мы изобразили 12 из 13 действий из нашего списка. Осталось еще одно:

```

1 function add_item_to_cart(item) {
2   cart = add_item(cart, item);
3   update_total_queue(cart);
4 }
5
6 function calc_cart_total(cart, callback) {
7   var total = 0;
8   cost_ajax(cart, function(cost) {
9     total += cost;
10  });
11  shipping_ajax(cart, function(shipping) {
12    total += shipping;
13    callback(total);
14  });
15 }
16
17 function calc_cart_worker(cart, done) {
18   calc_cart_total(cart, function(total) {
19     update_total_dom(total);
20     done(total);
21   });
22 }
23
24 var update_total_queue = DroppingQueue(1, calc_cart_worker);

```

Действия

1. Чтение cart.
2. Запись cart.
3. Чтение cart.
4. Вызов update_total_queue().
5. Инициализация total = 0.
6. Вызов cost_ajax().
7. Чтение total.
8. Запись total.
9. Вызов shipping_ajax().
10. Чтение total.
11. Запись total.
12. Чтение total.
13. Вызов update_total_dom().

Этот обратный вызов активизируется через обратный вызов `shipping_ajax()`, поэтому он находится на этой временной линии:

```

18 calc_cart_total(cart, function(total) {
19   update_total_dom(total);
20   done(total);
21 });

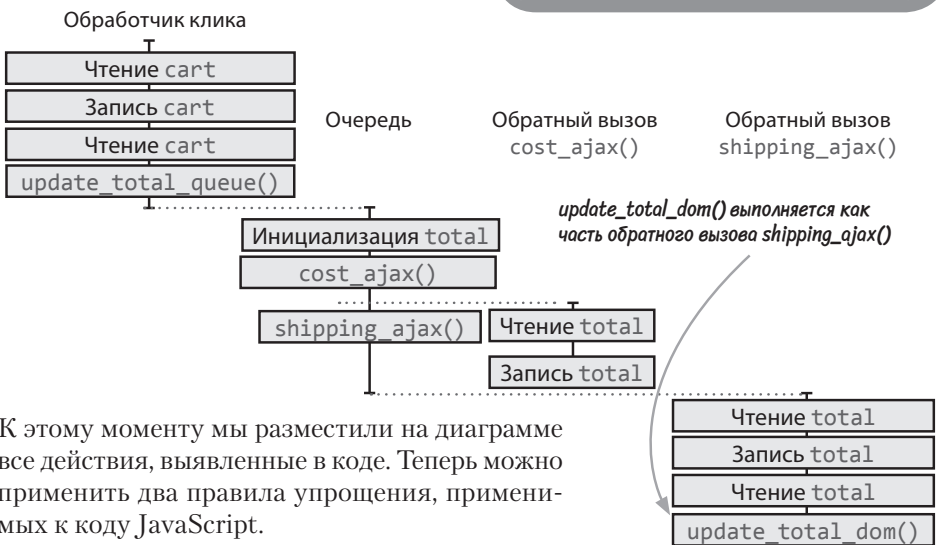
```

Три шага построения диаграммы

1. Идентификация действий.
2. Отображение действий на диаграмме.
3. Упрощение.

Два упрощения JavaScript

1. Объединение действий.
2. Объединение временных линий.



К этому моменту мы разместили на диаграмме все действия, выявленные в коде. Теперь можно применить два правила упрощения, применимых к коду JavaScript.

Упрощение диаграммы: шаг 3

Мы представили на диаграмме все 13 действий. Все готово к оптимизации. Вспомните, что говорилось ранее о двух приемах упрощения, которые могут применяться по порядку из-за особенностей потоковой модели JavaScript.

Действия по упрощению для потоковой модели JavaScript

1. Объединение всех действий на одной временной линии в один блок.
2. Объединение завершаемых временных линий с созданием одной новой временной линии.

Три шага построения диаграммы

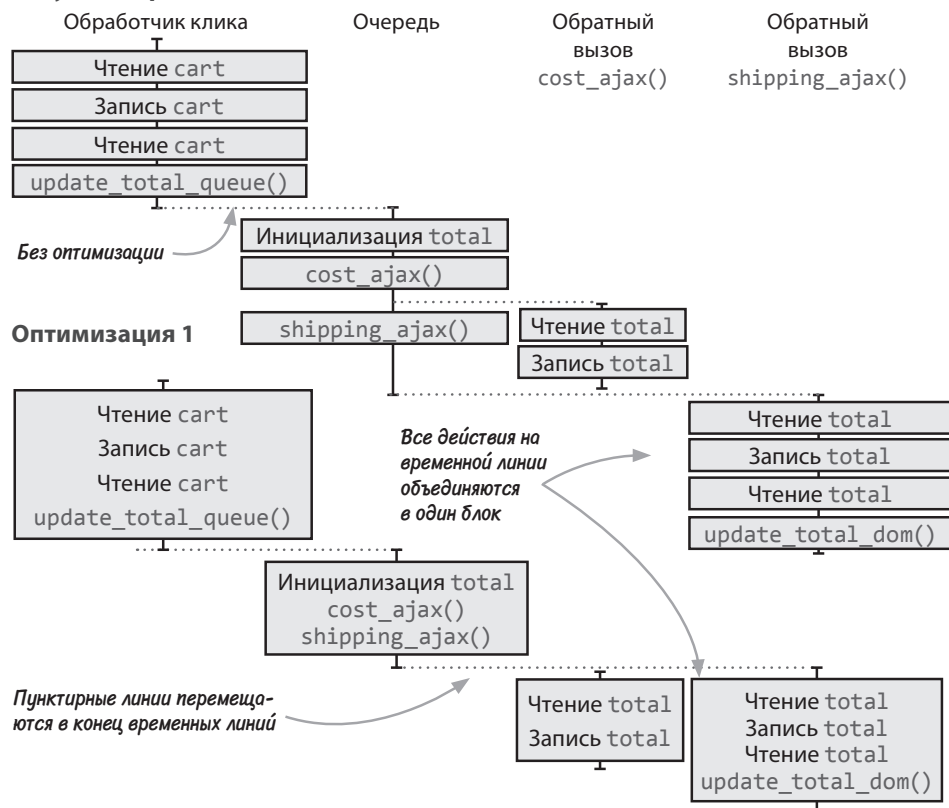
1. Идентификация действий.
2. Отображение действий на диаграмме.
3. Упрощение.

Два упрощения JavaScript

1. Объединение действий.
2. Объединение временных линий.

Хочется надеяться, что эти два приема сократят когнитивную нагрузку для понимания диаграммы. Диаграмма без упрощений выглядит так:

Текущая версия

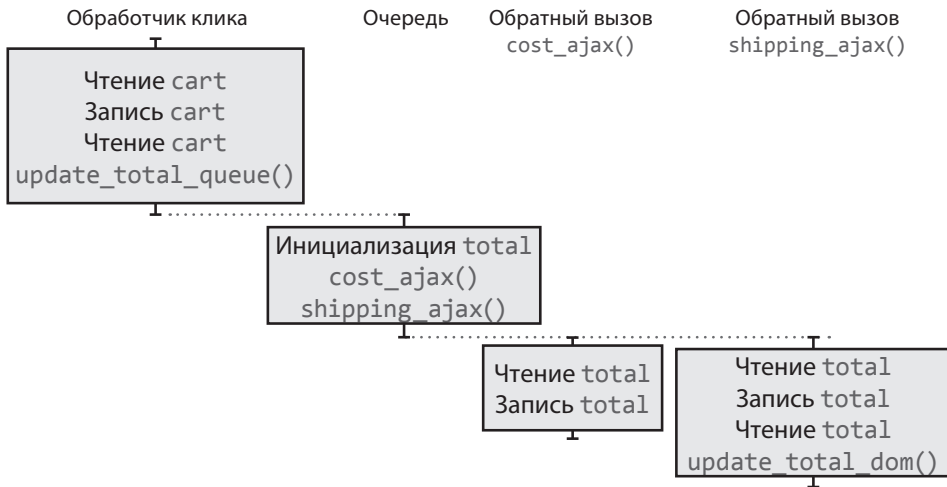


Пунктирные линии перемещаются в конец соответствующих шагов диаграммы, чтобы лучше представлять порядок происходящих событий.

Возможно, проблема к этому моменту уже стала очевидной, но мы пройдем процесс упрощения полностью.

Первый шаг упрощения выполнен, переходим ко второму. Диаграмма после первого шага выглядит так:

Оптимизация 1

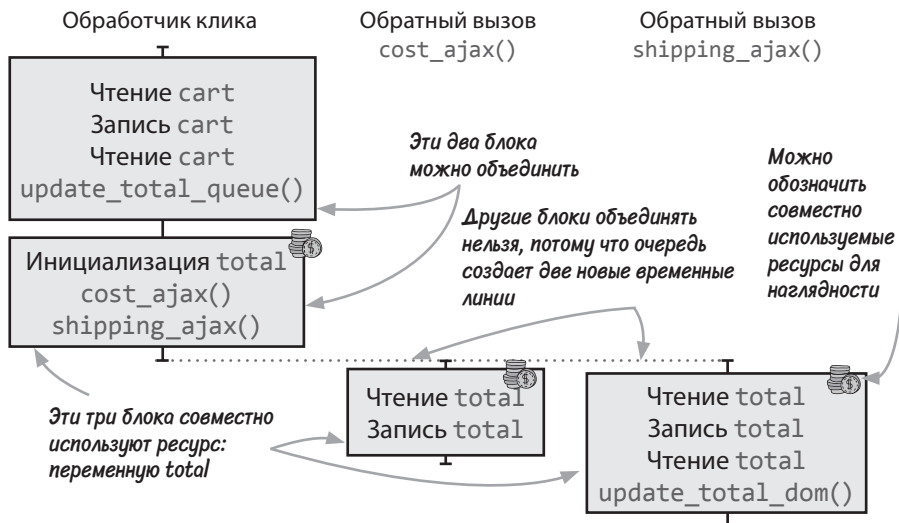


Второй шаг позволяет объединять временные линии, если первая временная линия завершается созданием второй. Это можно делать только в том случае, если она не создает других временных линий. В нашем примере такое упрощение может выполняться только в одном месте, между временными линиями обработчика клика и очереди. Очередь нельзя объединить с временными линиями обратных вызовов ajax, потому что временные линии очереди завершаются созданием двух новых временных линий.

Два упрощения JavaScript

1. Объединение действий.
2. Объединение временных линий.

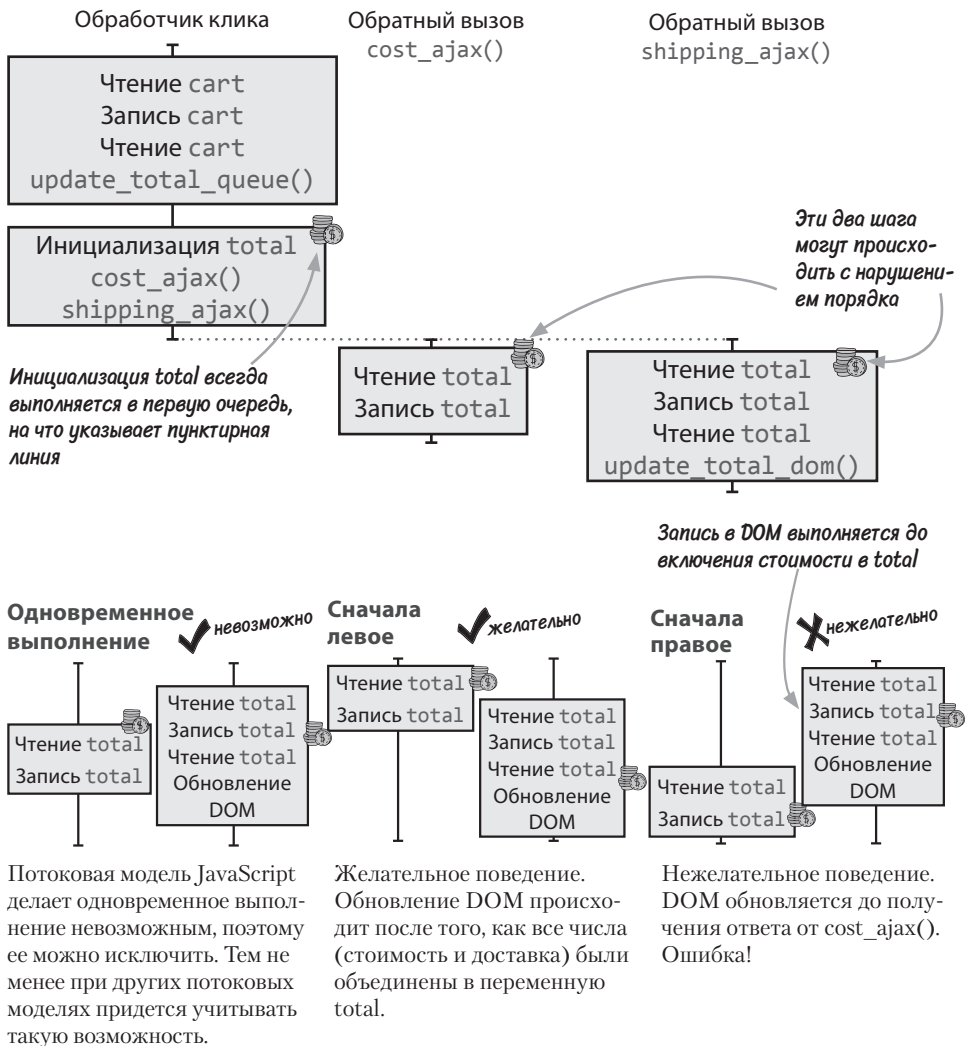
Оптимизация 2



После завершения упрощения мы видим, какие шаги временной диаграммы совместно используют ресурсы. Единственным общим ресурсом является переменная `total`. И хотя переменная является локальной, к ней обращаются три разные временные линии. Ситуация более подробно рассматривается на следующей странице.

Анализ возможных вариантов упорядочения

На предыдущей странице мы завершили временную диаграмму и идентифицировали `total` как единственный ресурс, совместно используемый между временными линиями.



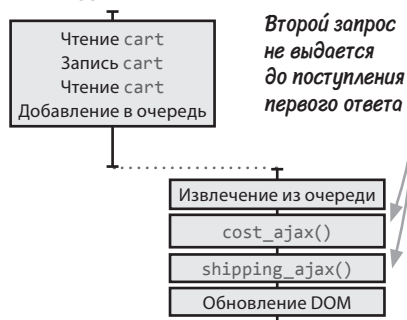
Мы видим, что два обратных вызова могут выполняться в нежелательном порядке. Может оказаться, что обратный вызов `shipping_ajax()` отработает после обратного вызова `cost_ajax()`, несмотря на то что запросы иницируются в правильном порядке. Ошибка обнаружена!

Прежде чем исправлять ошибку, посмотрим, почему неправильный код может работать быстрее.

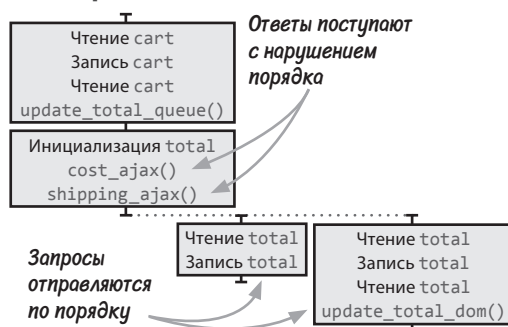
Почему эта временная линия выполняется быстрее

Активная оптимизация привела к появлению ошибки, описанной на предыдущей странице. Вы увидели, почему код иногда работает нормально, а в других случаях может происходить ошибка. Удастся ли вам понять, почему этот код работает быстрее старого кода, по соответствующим временным диаграммам?

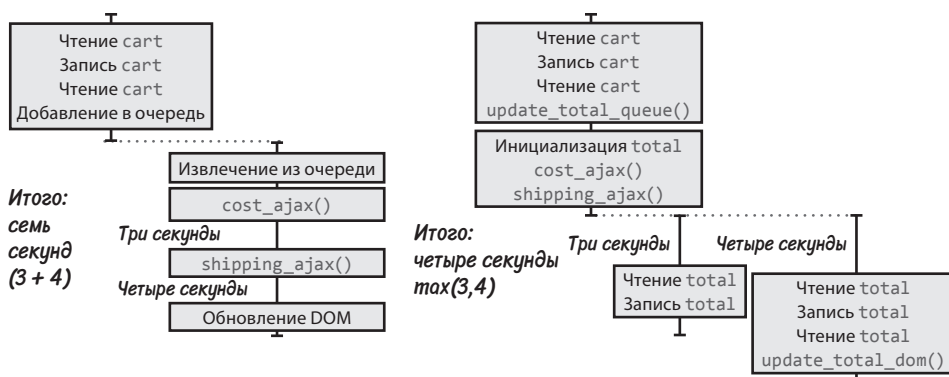
Старый код (правильный, но медленный)



Новый код (быстрый, но неправильный)



Представьте, что ответ для `cost_ajax()` приходит через три секунды, а ответ для `shipping_ajax()` — через четыре секунды. Вероятно, это слишком много для простых веб-запросов, но воспользуемся этими числами. Что говорят две временные линии о минимальном времени, которое пользователю придется ожидать до обновления DOM? Ответ можно получить в графическом виде:



Хотя эти числа и вымышлены, они весьма показательны. Временная линия слева ожидает поступления двух *последовательных* ответов. Это означает, что время складывается. Временная линия справа ожидает двух ответов от *параллельных* запросов. В этом случае время ожидания определяется большим из двух чисел. В этом случае код завершается быстрее. Но конечно, более быстрая временная линия работает неправильно. Можно ли обеспечить выигрыш по скорости, присущий параллельным ответам, без ошибочного поведения? Да! Можно воспользоваться другим примитивом синхронизации для координации временных линий, чтобы операции всегда выполнялись в правильном порядке.



Отдых для мозга

*Маловероятно, но при
тысячах пользователей
в день это возможно*

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: На временной линии `cost_ajax` определенно отправляется до `shipping_ajax`. Почему ответ `shipping_ajax` иногда возвращается первым?

О: Отличный вопрос. Существует множество причин, из-за которых запросы могут возвращаться с нарушением порядка. Их так много, что привести полный список невозможно. Впрочем, я укажу некоторые возможные варианты.

1. `cost_ajax` возвращает больший объем данных. Загрузка этих данных занимает больше времени.
2. Сервер, обрабатывающий `cost_ajax`, загружен сильнее, чем сервер API доставки.
3. Телефон в движущейся машине переключается на другую вышку связи при отправке `cost_ajax`, что приводит к задержке. `shipping_ajax` отправляется с одной вышки, поэтому он обрабатывается быстрее.

В сети на пути от компьютера к серверу (и обратно!) порой творится такой хаос, что произойти может все что угодно.

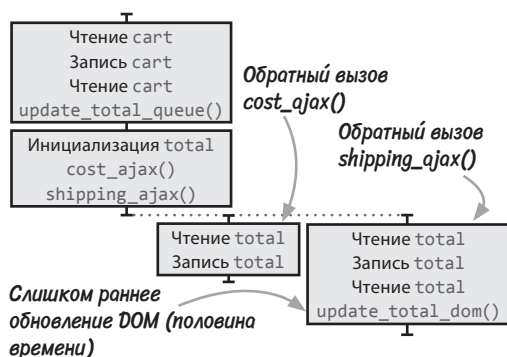
В: Весь этот анализ утомляет. Нам действительно необходимо все это проделывать и рисовать все временные линии?

О: Да, это серьезная проблема. Отвечаю: действительно необходимо, но работу можно заметно ускорить. Когда вы начнете более уверенно пользоваться диаграммами, большую часть анализа вы начнете проделывать в голове. Рисовать все вовсе не обязательно. Мы сейчас описываем каждый шаг только для того, чтобы показать, как это делается. Но научившись строить диаграммы, вы будете пропускать шаги, и это вполне нормально.

Ожидание двух параллельных обратных вызовов

Наша цель проста: мы хотим, чтобы ответы ajax возвращались параллельно, но при этом хотим дождаться двух ответов перед записью в DOM. Если запись в DOM будет выполнена после завершения одного запроса, но до завершения другого, вы получите неправильный ответ. Мы хотим получить то, что изображено справа:

Что имеем



Что хотим получить

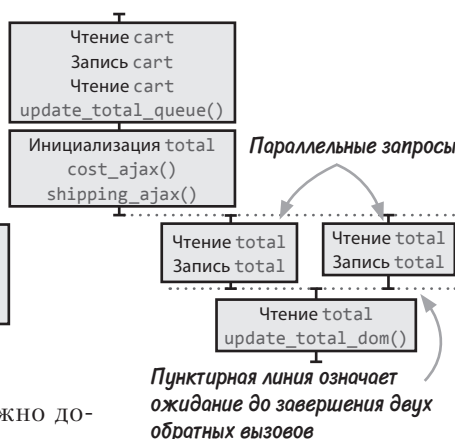
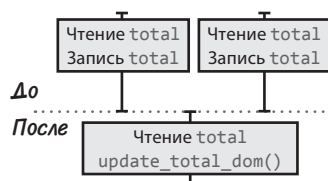
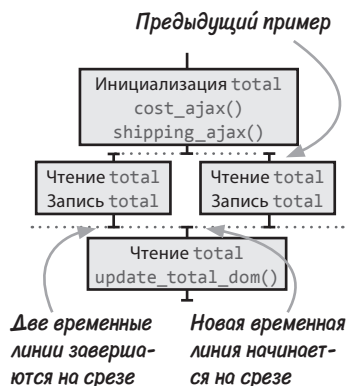


Диаграмма справа показывает, как можно достичь цели. Мы видим, что два ответа все еще обрабатываются параллельно. Они могут происходить в любом порядке. Только после того, как оба ответа будут обработаны, мы наконец-то можем обновить DOM. Каждый из двух обратных вызовов ожидает завершения другого. На диаграмме ожидание представляется пунктирной линией.

Пунктирная линия называется *срезом*. Как и пунктирные линии, которые использовались ранее, срез гарантирует определенное упорядочение. Но в отличие от тех линий, срезы проходят через концы нескольких временных линий. Срез на временной линии означает, что все, что находится выше среза, происходит ранее всего, что находится под ним.

Нарезка сильно упрощает анализ. Срез делит все задействованные временные линии на две части: «до» и «после». Временные линии до среза можно анализировать отдельно от линий после среза. Действия после среза ни при каких условиях не смогут пересекаться с действиями



до среза. Нарезка существенно уменьшает количество возможных вариантов упорядочения, а это, в свою очередь, снижает сложность приложения.

В данном случае мы имеем две временные линии обратных вызовов, которые необходимо координировать для вычисления итоговой суммы. У каждой временной линии есть число, которое объединяется с накапливаемой суммой. Две временные линии работают вместе с одним общим ресурсом (локальная переменная `total`). Временные линии необходимо координировать, чтобы после чтения `total` значение было записано в DOM.

Мы можем построить примитив синхронизации, который реализует срезы. За дело!

Примитив синхронизации для нарезки временных линий

Требуется написать простой, универсальный примитив, который позволяет нескольким временным линиям дожидаться друг друга, даже если временные линии завершаются в другом порядке. При наличии такого примитива можно не обращать внимания на то, что события происходят в другом порядке, и думать только о том, когда они все завершатся. Таким образом можно предотвратить ситуацию гонки.

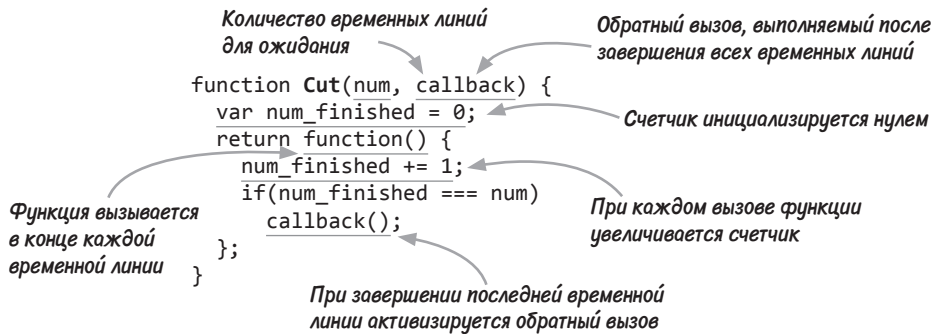
Метафора, заложенная в основу примитива, следует принципу поиска образцов в реальном мире. Если вы с другом работаете над разными задачами, вы можете договориться, чтобы каждый из вас дождался другого независимо от того, кто закончит работу первым. После этого вы вместе отправитесь на обед. Нам нужен примитив, который позволяет временным линиям завершаться в любом порядке и продолжает работу только после завершения всех временных линий. Мы сделаем это на следующей странице.



Загляни в словарь

В *ситуации гонки* поведение зависит от того, какая временная линия завершится первой.

В языках с несколькими потоками нам хотелось бы использовать некую разновидность атомарного обновления, чтобы потоки могли совместно использовать изменяемое состояние. Тем не менее мы можем воспользоваться однопоточностью JavaScript и реализовать примитив простой переменной при условии, что обращения к нему будут синхронными. Мы напишем функцию, которая будет вызываться каждой временной линией при завершении. Каждый раз, когда вызывается эта функция, она будет инкрементировать количество вызовов. Затем при последнем вызове будет активизироваться обратный вызов:



Простой пример

```
var done = Cut(3, function() {
  console.log("3 timelines are finished");
});
```

```
done();
done();
done();
```

```
console=> "3 timelines are finished"
```

Ожидаем три вызова done(), после чего выводим сообщение

num_finished = 0

num_finished = 1

num_finished = 2

num_finished = 3

После третьего вызова done() выводится сообщение

Примитив готов; включим его в код добавления товара в корзину.



Напоминание

В JavaScript используется один поток. Временная линия выполняется до завершения, после чего могут начаться другие временные линии. Cut() использует этот факт для безопасного совместного использования изменяемой переменной. В других языках для координации временных линий пришлось бы использовать блокировки или другие механизмы синхронизации.

Использование Cut() в коде

Мы реализовали примитив синхронизации Cut(), но теперь нужно использовать его в коде корзины. К счастью, необходимые изменения невелики. Остается прояснить два вопроса.

1. В какой области видимости хранить Cut()?
2. Какой обратный вызов передать Cut()?

1. В какой области видимости разместить Cut()?

Функция done() должна вызываться в конце каждого обратного вызова. Это наводит на мысль о том, что Cut() следует добавить в области видимости calc_cart_total(), где создаются оба обратных вызова.

2. Какой обратный вызов передать Cut()?

Внутри calc_cart_total() мы уже выделили обратный вызов, который должен происходить после вычисления общей стоимости. Обычно это update_total_dom(), но может быть и любая другая функция. Мы просто передаем этот обратный вызов Cut(). Результат выглядит так:

До

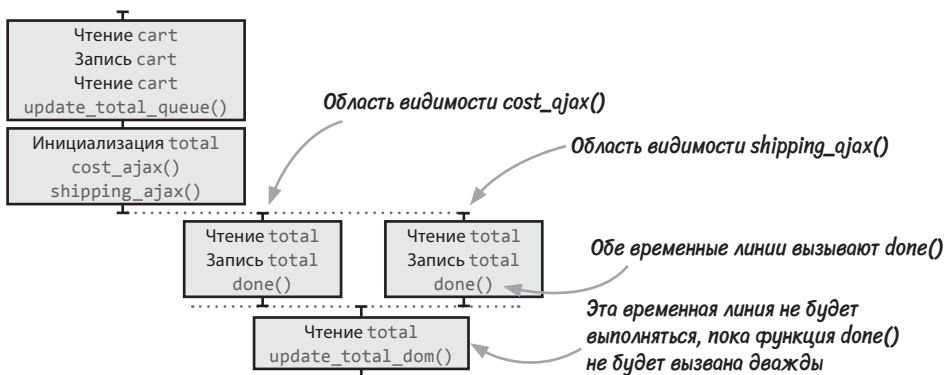
```
function calc_cart_total(cart, callback) {
    var total = 0;

    cost_ajax(cart, function(cost) {
        total += cost;
    });
    shipping_ajax(cart, function(shipping) {
        total += shipping;
        callback(total);
    });
}
```

С Cut()

```
function calc_cart_total(cart, callback) {
    var total = 0;
    var done = Cut(2, function() {
        callback(total);
    });
    cost_ajax(cart, function(cost) {
        total += cost;
        done();
    });
    shipping_ajax(cart, function(shipping) {
        total += shipping;
        done();
    });
}
```

Временная диаграмма





Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Функция `Cut()` действительно работает?

О: Хороший вопрос. `Cut()` работает, но очень важно, чтобы функция `done()` вызывалась в конце временной линии. Если вызвать ее до завершения, временная линия может продолжиться после вызова `done()`, чего быть не должно. Лучше избегать подобных ситуаций, потому что они создают путаницу. Соблюдайте правило и вызывайте `done()` в конце временных линий, в которых вы захотите создать срез.

В: Но `Cut()` содержит там мало кода. Может ли что-то реально полезное быть настолько простым?

О: Я немного переформулирую вопрос: может ли что-то непростое быть пригодным для повторного использования? Мы реализуем очень простую ситуацию. Если пятеро друзей хотят вместе пообедать, они ожидают в месте сбора, пока все пять не будут готовы. Потребуется совсем немного: посчитать до пяти. И функция `Cut()` делает именно это: она считает до заданного числа, а затем вызывает функцию.

В: Разве не лучше воспользоваться чем-то другим, например обещаниями (`promises`)?

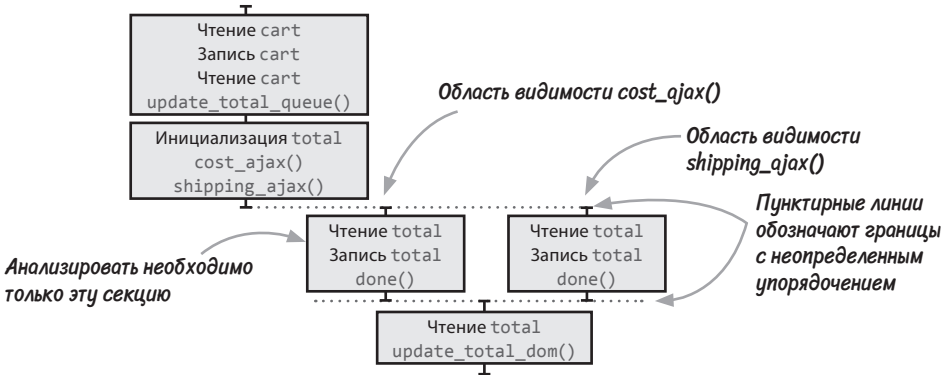
О: Тоже верно. Есть много готовых реализаций примитивов синхронизации. В каждом языке используется свой набор таких примитивов. Опытным программистам JavaScript знаком объект `Promise`, и особенно метод `Promise.all()`, который делает нечто очень похожее.

Если вам известен существующий примитив, который решит вашу проблему, — используйте его. Тем не менее эта книга была написана не для того, чтобы учить вас JavaScript, а чтобы научить вас функциональному программированию. Ничто не мешает вам применять эти принципы в любом языке, чтобы решать собственные задачи из области программирования. А если нужного примитива не существует, просто реализуйте его самостоятельно.

Анализ неопределенных упорядочений

На нескольких последних страницах мы использовали примитив синхронизации с функцией добавления товара в корзину. Похоже, он обеспечивает двойной выигрыш: параллельное выполнение (ускорение загрузки) с правильным поведением (все заказы обрабатываются правильно).

Во всяком случае, мы на это надеемся. Проанализируем временную линию и убедимся в том, что мы действительно получаем двойной выигрыш. Временная линия выглядит так:



Для начала разберемся с более важным вопросом: обеспечивается ли правильное поведение при всех возможных вариантах упорядочения?

Мы основательно разделили временную линию пунктирными линиями, что должно сильно упростить анализ. Линию можно анализировать по частям.

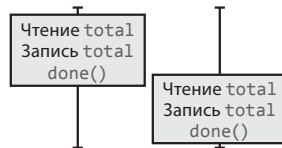
Над верхней пунктирной линией располагается только одна временная линия, поэтому возможное упорядочение только одно. Ниже второй временной линии (нижняя часть) временная линия тоже только одна. Одно возможное упорядочение — это нормально. Остается последняя часть, заключенная между двумя пунктирными линиями. Она состоит из двух временных линий, по одному шагу каждая. Рассмотрим варианты упорядочения и проверим их:

Одновременное выполнение ✓ *невозможно*



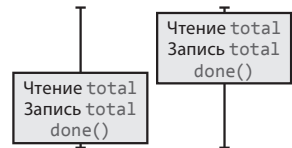
Потоковая модель JavaScript делает одновременное выполнение невозможным, поэтому ее можно исключить. Тем не менее при других потоковых моделях придется учитывать такую возможность.

Сначала левое ✓ *желательно*



Желательное поведение. Общая стоимость добавляется перед стоимостью доставки, после чего done() вызывается во второй раз.

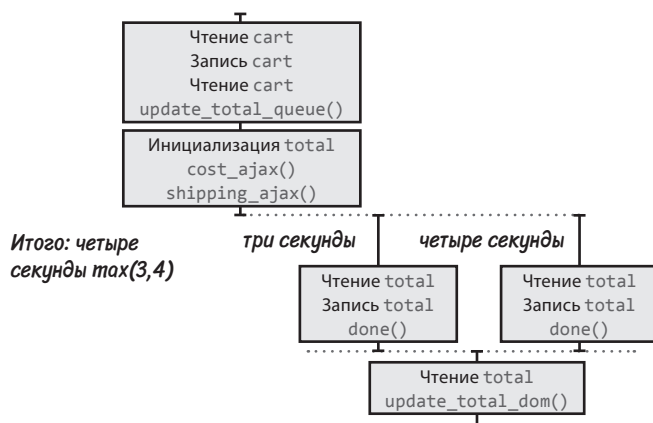
Сначала правое ✓ *желательно*



Тоже желательное поведение. Сначала добавляется стоимость доставки, затем общая стоимость, после чего done() вызывается во второй раз.

Анализ параллельного выполнения

Мы только что проанализировали временную линию на предмет правильного упорядочения. Посмотрим, действительно ли она работает быстрее:



И снова рассмотрим гипотетическую ситуацию, в которой ответ `cost_ajax()` занимает три секунды, а ответ `shipping_ajax()` занимает четыре секунды. Если воспроизвести этот сценарий на диаграмме, мы увидим, что общее время составляет больший из двух промежутков, то есть четыре секунды. Победа! Скорость параллельного выполнения сочетается с правильностью последовательного выполнения.

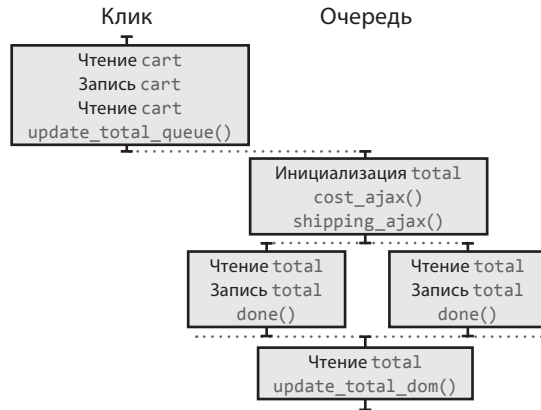
Все началось с временной ошибки, которая происходила даже при однократном клике. Мы разделили временную линию, чтобы две параллельные временные линии ожидали друг друга перед продолжением работы. И теперь все работает.

...Вернее, работает для одного клика. А будет ли работать для двух и более? Выясним это на следующей странице.

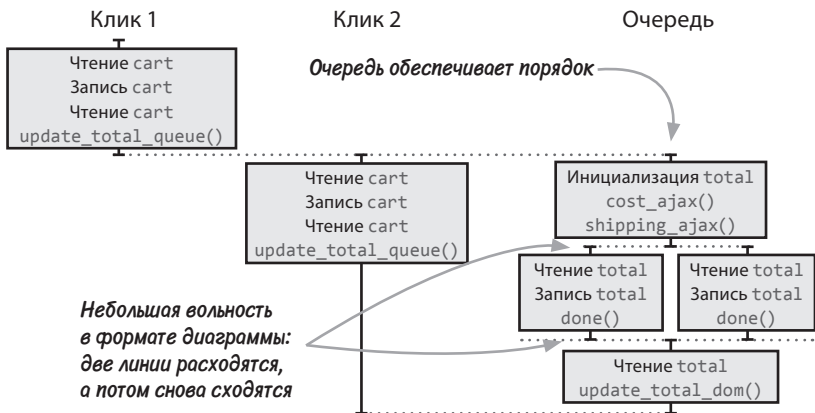
Анализ для нескольких кликов

Вы видели, что код работает правильно (и быстро) для одного клика. А что произойдет с двумя и более кликами? Будет ли очередь хорошо работать со срезом? Посмотрим!

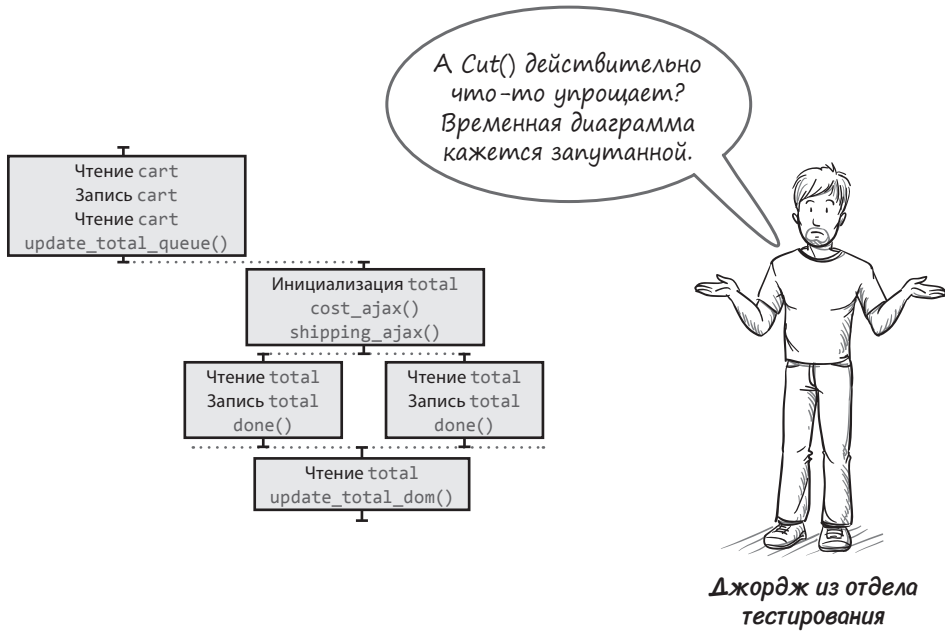
Сначала слегка изменим временную линию, чтобы все происходящее в очереди находилось на одной линии:



Теперь по этой диаграмме можно определить, что происходит при нескольких кликах.



Мы допустили небольшую вольность в представлении: два параллельных обратных вызова ajax происходят на временной линии очереди. На самом деле это две временные линии, но они снова сходятся в одну из-за использования `Cut()`. Эта вольность лишний раз показывает, что формат диаграммы чрезвычайно гибок и его следует использовать для представления сложности ситуации на том уровне детализации, который необходим для ее анализа.



Трудный вопрос! На диаграмме происходит много всего, но она только представляет сложность ситуации. Здесь параллельно выполняются два запроса ajax. Для вычисления общей стоимости корзины нужны оба ответа, и их необходимо дождаться перед обновлением DOM данными из ответа, поэтому мы и вызываем Cut(). Но программа также должна правильно работать при быстрых кликах на кнопку добавления товара, отсюда применение очереди. Одной кнопке приходится делать достаточно много, а нам приходится достаточно много анализировать.

Тем не менее Cut() упрощает анализ диаграммы. Помните, что более короткие временные линии проще анализировать. Пунктирная линия означает, что временную линию можно разделить надвое, отсюда и термин «срез». Часть над пунктирной линией анализируется отдельно от части под ней. На временной диаграмме параллельное выполнение происходит только в одной области, всего с двумя временными линиями ($t = 2$) из одного шага каждая ($a = 1$). Достаточно рассмотреть всего два варианта упорядочения (мы рассмотрели одновременное выполнение, хотя это и не обязательно: в JavaScript оно невозможно). Остальное происходит последовательно, что ясно следует из диаграммы. Функция Cut() не только обеспечивает правильное поведение, но и упрощает его проверку.

Формула для определения количества возможных вариантов упорядочения

Количество временных линий

Количество действий на одну временную линию

$$O = \frac{(ta)!}{(a!)^t}$$

Возможные варианты упорядочения

! — факториал

Превосходный вопрос. Чтобы ответить на него, полезно задуматься над тем, откуда берется эта сложность. В нашем случае сложность обусловлена тремя аспектами.

1. Асинхронные веб-запросы.
2. Два ответа API, которые необходимо объединить для формирования результата.
3. Неопределенность пользовательских взаимодействий.

Пункты 1 и 3 обусловлены архитектурными решениями. Если мы хотим выполнять код в браузере как приложение JavaScript, нам придется иметь дело с асинхронными веб-запросами. И еще мы хотим, чтобы корзина была интерактивной, поэтому придется иметь дело с пользователем. Эти обстоятельства неизбежно следуют из архитектурных решений. А сложности 1 и 3 действительно необходимы? Нет.

От пункта 3 можно избавиться, сделав приложение менее интерактивным. Можно отобразить форму для пользователя. Пользователи вводят все товары, которые они хотят купить, и отправляют форму. Конечно, такой процесс взаимодействия с пользователем неприемлем. Скорее всего, приложение должно быть еще более, а не менее интерактивным.

От пункта 1 можно избавиться отказом от использования запросов ajax. Можно создать стандартное веб-приложение без ajax, которое использует ссылки и отправку форм и перезагружает страницу при каждом мелком изменении. Но и это нас не устраивает.

Однако с пунктом 2 ситуация иная. Можно представить изменение API, в котором два запроса объединяются в один. Нам не придется беспокоиться о параллельном выполнении запросов и объединении ответа. Правда, мы не избавляемся от сложности, а только перемещаем ее на сервер.

Сервер может в большей или меньшей степени подходить для решения проблем со сложностью, чем браузер. Это зависит от архитектуры серверной части. Использует ли она потоки? Могут ли два вычисления (общая стоимость и доставка) выполняться из одной базы данных? Должна ли серверная часть использовать несколько API? Возникают тысячи вопросов, которые раскрывают решения, приводящие к этой сложности.

Итак, сложность необходима? Нет. Но она появляется из-за решений, которые мы, скорее всего, изменять не захотим. Если исходить из таких решений, сложность с большой вероятностью окажется неизбежной. Нам понадобятся эффективные практики программирования, которые помогут справиться с этой сложностью.

Так ли необходима вся эта сложность? Ведь мы просто строим графический интерфейс в браузере.



Дженна из команды разработки



Ваш ход

В следующем коде представлены несколько временных линий, которые осуществляют чтение и запись в глобальную переменную `sum` для подсчета денег от нескольких кассовых аппаратов. Нарисуйте временную диаграмму.

```
var sum = 0;

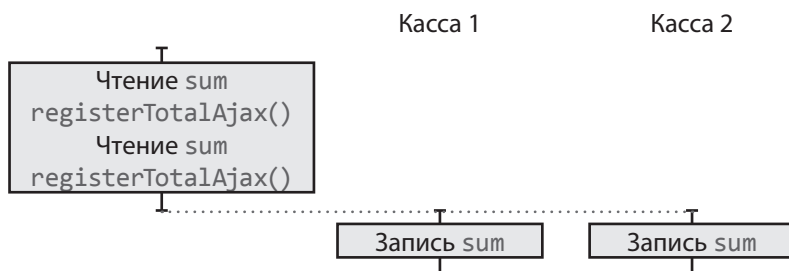
function countRegister(registerid) {
  var temp = sum;
  registerTotalAjax(registerid, function(money) {
    sum = temp + money;
  });
}

countRegister(1);
countRegister(2);
```

Нарисуйте здесь
свою диаграмму



Ответ





Ответ

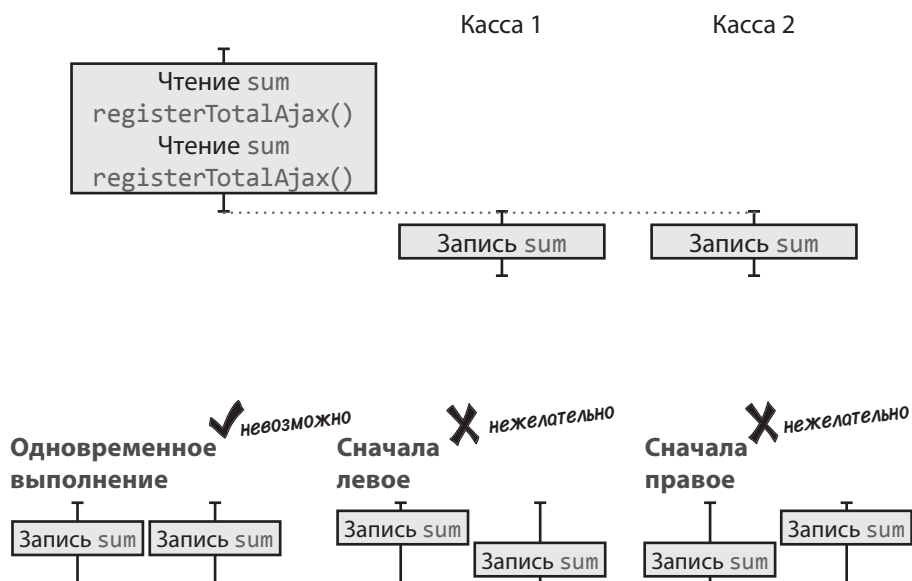
Ниже приведен код из последней пары упражнений. Не существует вариантов упорядочения, которые дают правильный ответ. Сможете ли вы найти ошибку? Исправьте код, нарисуйте диаграмму и проанализируйте временные линии.

Подсказка: а если переместить чтение `sum` ближе к точке, в которой происходит запись в эту переменную?

```
var sum = 0;

function countRegister(registerid) {
  var temp = sum;
  registerTotalAjax(registerid, function(money) {
    sum = temp + money;
  });
}

countRegister(1);
countRegister(2);
```



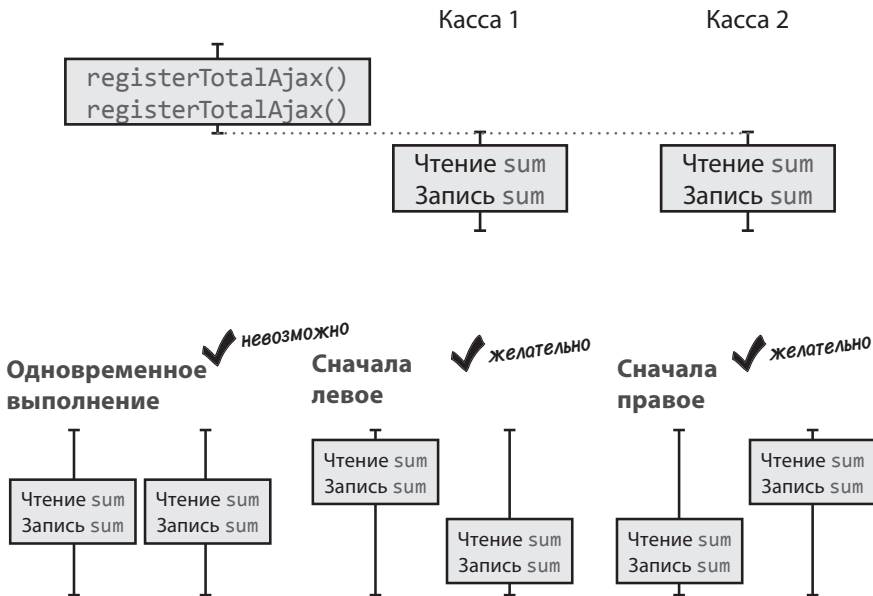


Ваш ход

```
var sum = 0;

function countRegister(registerid) {
    registerTotalAjax(registerid, function(money) {
        sum += money;
    });
}

countRegister(1);
countRegister(2);
```



Примитив для однократного вызова



Ким: Разве работа `Cut()` не основана на том, чтобы все временные линии завершались перед активизацией обратного вызова?

Дженна: Да. Но взгляни на это так: `Cut()` активизирует обратный вызов, когда последняя временная линия вызывает `done()`. Так осуществляется координация. А если создать примитив, который активизирует обратный вызов при первом обращении от первой временной линии?

Ким: О! Тогда обратный вызов работает только один раз!

Дженна: Верно! Мы назовем его `JustOnce()`! Посмотрим, удастся ли нам сделать это, на следующей странице.

Ким необходим инструмент, который позволит выполнить действие только один раз независимо от того, сколько раз это действие будет вызываться в коде. Похоже на задачу для примитива синхронизации. Им будет функция высшего порядка, которая наделяет обратный вызов суперспособностью выполняться только один раз.

Посмотрим, как это может выглядеть. У Ким есть функция, которая отправляет пользователю приветственное сообщение:

```
function sendAddToCartText(number) {
    sendTextAjax(number, "Thanks for adding something to your cart. " +
        "Reply if you have any questions!");
}
```

При каждом вызове отправляется текст

Код примитива синхронизации, который упаковывает эту функцию в новую функцию:

```
function JustOnce(action) {
    var alreadyCalled = false;
    return function(a, b, c) {
        if(alreadyCalled) return;
        alreadyCalled = true;
        return action(a, b, c);
    };
}
```

Для отслеживания того, вызывалась ли функция

Прямо перед вызовом запоминаем

Передается действие

Если функция вызывалась ранее, завершить преждевременно

Вызвать действие и передать аргументы



Напоминание

В JavaScript используется один поток. Временная линия выполняется до завершения, после чего могут начаться другие временные линии. `JustOnce()` использует этот факт для безопасного совместного использования изменяемой переменной. В других языках для координации временных линий пришлось бы использовать блокировки или другие механизмы синхронизации.

Как и `Cut()`, `JustOnce()` совместно использует переменную между временными линиями, но в JavaScript это безопасно, потому что код неасинхронен. В языках с многопоточным выполнением нам пришлось бы использовать некую разновидность атомарного обновления, чтобы координировать работу потоков. Мы можем воспользоваться этим обстоятельством и упаковать вызов функции `sendAddToCart()` в обертку, чтобы она выполнялась только один раз:

```
var sendAddToCartTextOnce = JustOnce(sendAddToCartText);

sendAddToCartTextOnce("555-555-5555-55");
sendAddToCartTextOnce("555-555-5555-55");
sendAddToCartTextOnce("555-555-5555-55");
sendAddToCartTextOnce("555-555-5555-55");
```

 Наделяет `sendAddToCartText()` суперспособностью
Текст передается только для первого вызова

Мы только что создали другой примитив синхронизации. Он позволяет совместно использовать действие на двух временных линиях и определяет, как они будут при этом взаимодействовать. Как и большинство примитивов, он чрезвычайно универсален.

К настоящему моменту мы рассмотрели три примитива синхронизации. Позднее в книге будут приведены и другие примеры. Здесь важно подчеркнуть, что примитивы необязательно сложно писать, — и становится еще проще, если вы будете совершенствовать их поэтапно и разделите на две части: обобщенную, пригодную для повторного использования, и часть, специфичную для приложения.



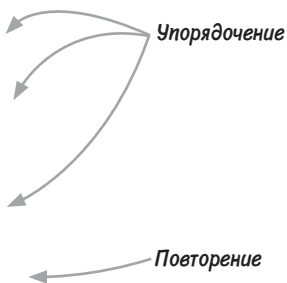
Загляни в словарь

Действие, эффект которого проявляется только при первом выполнении, называется *идемпотентным*. Функция `JustOnce()` делает любое действие идемпотентным.

Неявная и явная модель времени

В каждом языке существует неявная модель времени. Эта модель описывает два аспекта выполнения: упорядочение и повторение.

В JavaScript модель времени очень проста.

1. Последовательные команды выполняются по порядку.
 2. Шаги двух разных временных линий могут выполняться в порядке «сначала левый» или «сначала правый».
 3. Асинхронные события вызываются на новых временных линиях.
 4. Действие выполняется столько раз, сколько раз вы его вызываете.
-  Упорядочение
Повторение

Такая модель хорошо подходит для использования по умолчанию, потому что она понятна. Мы использовали эту модель для рисования временных диаграмм:

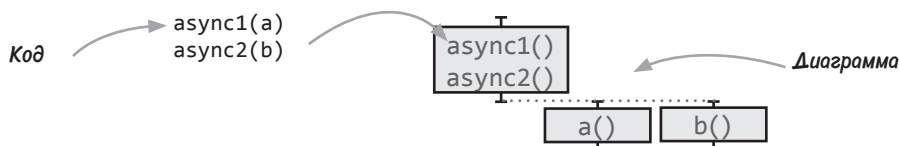
1. Последовательные команды выполняются по порядку



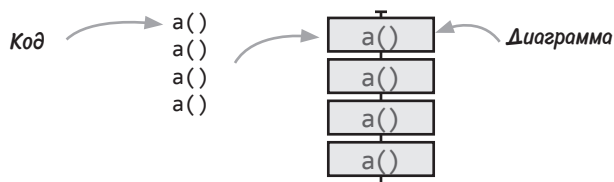
2. Шаги двух временных диаграмм могут происходить в двух вариантах упорядочения



3. Асинхронные события вызываются на новых временных диаграммах



4. Действие выполняется столько раз, сколько вы его вызываете



Неявная модель — хорошая отправная точка. Тем не менее это только один из возможных вариантов выполнения. Более того, он редко совпадает с тем, что нам нужно. Функциональные программисты строят новую модель времени, более близкую к тому, что им нужно. Например, мы создаем очередь, которая не создает новые временные линии для асинхронных обратных вызовов. Или мы создаем примитив `JustOnce()`, который выполняет действие только один раз, даже если будет вызван многократно.



Отдых для мозга

Это еще не все, но давайте сделаем небольшой перерыв для ответов на вопросы.

В: Три примитива синхронизации, которые мы написали, были функциями высшего порядка. Все примитивы пишутся в этой форме?

О: Хороший вопрос. Действительно, мы написали три примитива синхронизации в виде функций высшего порядка. Тем не менее не все примитивы являются функциями высшего порядка. В JavaScript вследствие его асинхронности функции высшего порядка используются достаточно часто из-за необходимости передачи обратных вызовов. В следующей главе будет представлен другой примитив синхронизации — ячейки. Он предназначен для совместного использования состояния. Ячейки используют функции высшего порядка, но формально функциями высшего порядка не являются.

В функциональных языках примитивы синхронизации имеют одну практически универсальную характеристику: использование первоклассных значений. Первокласность какой-либо сущности означает, что для работы с ней могут использоваться все средства языка. Заметим, что мы используем первоклассные действия в своем коде, чтобы вызывать их в другом контексте: например, как часть рабочего процесса очереди. Другой пример: мы берем первоклассное действие, которое отправляет текст, и заворачиваем его в новую функцию, которая вызовет его только один раз. Это стало возможным, потому что мы сделали действие первоклассным.

В: Принцип 5 говорит о «манипуляциях со временем». Вы не преувеличиваете?

О: Ха! Возможно. На самом деле мы строим новую явную модель упорядочения и повторения, важных аспектов времени при программировании. Далее мы можем использовать эту явную модель вместо того, чтобы полагаться на неявную модель выбранного языка. В общем, конечно, мы не манипулируем со временем: мы манипулируем с *моделью* времени.



Ваш ход

Ниже приведены примитивы синхронизации, построенные в книге. Как каждый из них строит новую модель времени? Опишите их кратко и нарисуйте временные диаграммы. Помните: основными аспектами времени являются упорядочение и повторение.

- `Queue()`
- `Cut()`
- `JustOnce()`
- `DroppingQueue()`

Запишите здесь
свои ответы



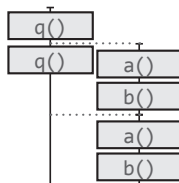
Ответ

Queue()

Элементы, добавленные в очередь, обрабатываются на одной отдельной временной линии. Каждый элемент обрабатывается до завершения, после чего начинается следующий:

```
var q = Queue(function() {
  a();
  b();
});
```

```
q();
q();
```



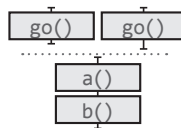
Cut()

Обратный вызов на новой временной линии активизируется только после завершения всех временных линий:

```
var done = Cut(2, function() {
  a();
  b();
});
```

```
function go() { done(); }
```

```
setTimeout(go, 1000);
setTimeout(go, 1000);
```



JustOnce()

Действие, упакованное в JustOnce(), будет выполнено только один раз, даже если упакованная функция вызывается многократно:

```
var j = JustOnce(a);
```

```
j();
j();
j();
```

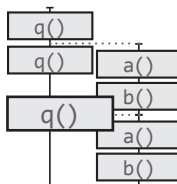


DroppingQueue()

Аналог Queue(), но пропускает задачи, если они накапливаются слишком быстро:

```
var q = DroppingQueue(1, function() {
  a();
  b();
});
```

```
q();
q();
q();
```



Резюме: манипулирование временными линиями

Рассмотрим все аспекты, в которых мы усовершенствовали использование временных линий. Они перечисляются в порядке важности, начиная с самых важных.

Сокращение количества временных линий

Упростите свою систему, чтобы в ней создавалось меньше потоков, асинхронных вызовов или запросов к серверу. Тем самым вы уменьшите количество временных линий.

Сокращение длины временной линии

Используйте меньше действий на каждой временной линии. Преобразуйте действия в вычисления (которые не находятся на временных линиях). Исключите неявный ввод и вывод.

Исключение совместного использования ресурсов

Сократите количество совместно используемых ресурсов. Две временные линии, у которых нет общих ресурсов, избавлены от проблем, связанных с упорядочением. Обращайтесь к общим ресурсам из одного потока, если это возможно.

Используйте примитивы синхронизации при совместном использовании ресурсов

Замените совместный доступ к небезопасному ресурсу совместным доступом к безопасным ресурсам (например, очередям, блокировкам и т. д.), чтобы совместная работа с ресурсами стала безопасной.

Координируйте выполнение с помощью примитивов синхронизации

Используйте обещания, срезы и т. д. для координации временных линий, чтобы ограничить упорядочение и повторение их действий.

Итоги главы

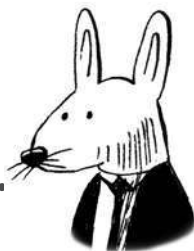
В этой главе мы диагностировали ситуацию гонки, связанную с порядком выполнения веб-запросов. Если запросы возвращают ответы в том порядке, в котором они вызывались, — все хорошо. Но гарантировать это невозможно, поэтому иногда мы получали ошибочный результат. Мы создали примитив, который позволял двум временным линиям работать вместе для получения постоянного результата. Этот пример демонстрировал координацию между временными линиями.

Резюме

- Функциональные программисты строят новую модель времени поверх неявной модели, предоставляемой языком. Новая модель обладает свойствами, помогающими в решении задач, над которыми они работают.
- Явная модель времени часто строится с первоклассными значениями. Первоклассный статус означает, что для манипуляций со временем вам доступны все средства языка.
- Мы можем строить примитивы синхронизации, координирующие две временные линии. Такие примитивы ограничивают возможные упорядочения и помогают добиться того, чтобы в любом случае достигался правильный результат.
- Нарезка временных линий — один из способов координации. Нарезка позволяет нескольким временным линиям дождаться завершения всех этих линий, чтобы одна линия могла продолжить выполнение.

Что дальше?

В этой части книги мы рассмотрели множество первоклассных функций и функций высшего порядка. Вы узнали, как объединять их в цепочки и как построить новую модель времени. Часть II завершается обсуждением проектирования в контексте многослойной архитектуры.



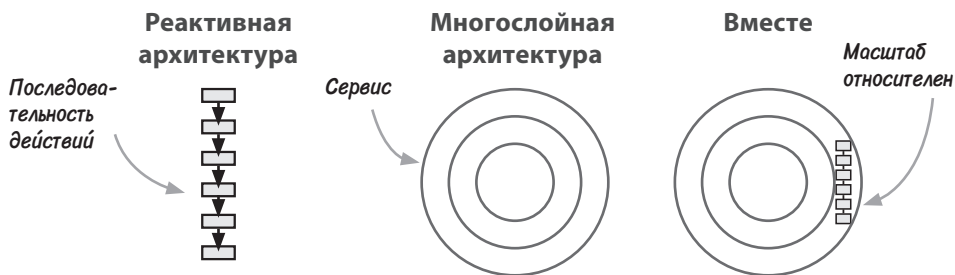
В этой главе

- ✓ Построение конвейеров действий с использованием реактивной архитектуры.
- ✓ Построение примитива для общего изменяемого состояния.
- ✓ Построение многослойной архитектуры для взаимодействия предметной области с окружающим миром.
- ✓ Применение многослойной архитектуры на многих уровнях.
- ✓ Сравнение многослойной архитектуры с традиционной многоуровневой архитектурой.

Ранее в главах этой части было представлено достаточно много практических применений первоклассных функций и функций высшего порядка. Пришло время сделать шаг назад и завершить эти главы обсуждением вопросов, связанных с проектированием и архитектурой. В этой главе мы рассмотрим два распространенных паттерна. *Реактивная архитектура* рассматривает упорядочение действий с обратной стороны. *Многослойная архитектура* представляет собой высокоуровневый взгляд на структуру функциональных программ, которые должны работать в реальных условиях. Итак, за дело!

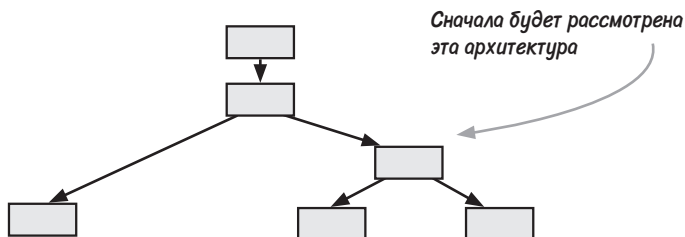
Два архитектурных паттерна

В этой главе мы рассмотрим два разных архитектурных паттерна: *реактивную* и *многослойную* архитектуру. Каждая архитектура работает на своем уровне. Реактивная архитектура используется на уровне отдельных последовательностей действий. Многослойная архитектура работает на уровне целых сервисов. Два паттерна хорошо дополняют друг друга, но могут применяться и по отдельности.



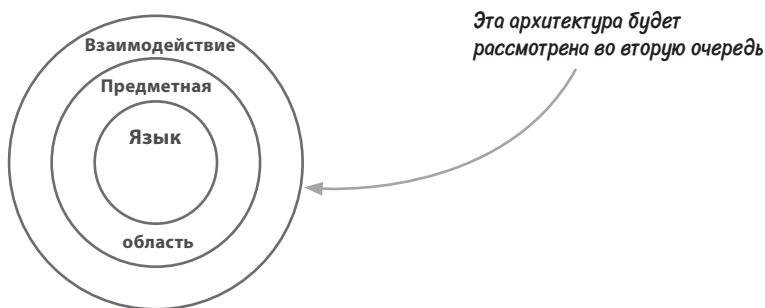
Реактивная архитектура

Реактивная архитектура меняет способ выражения порядка действий в коде. Как вы вскоре увидите, она позволяет отделить причину от эффекта, что помогает понять некоторые запутанные части вашего кода.



Многослойная архитектура

Многослойная архитектура определяет структуру для сервисов, которые должны взаимодействовать с внешним миром, будь то веб-сервис или термостат. Архитектура естественным образом следует из применения функционального мышления.

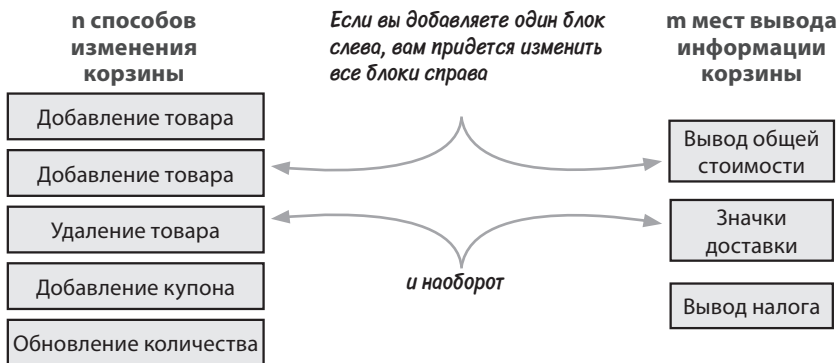


Связывание причин и эффектов изменений



Дженна: Каждый раз, когда я хочу добавить элемент пользовательского интерфейса (UI), который выводит какую-то информацию о корзине, мне приходится вносить изменения в десять местах. А еще пару месяцев назад таких мест было всего три.

Ким: Да, я вижу проблему. Это классическая проблема $n \times m$.



Ким: Чтобы добавить что-то в один столбец, необходимо изменить или продублировать все блоки в другом столбце.

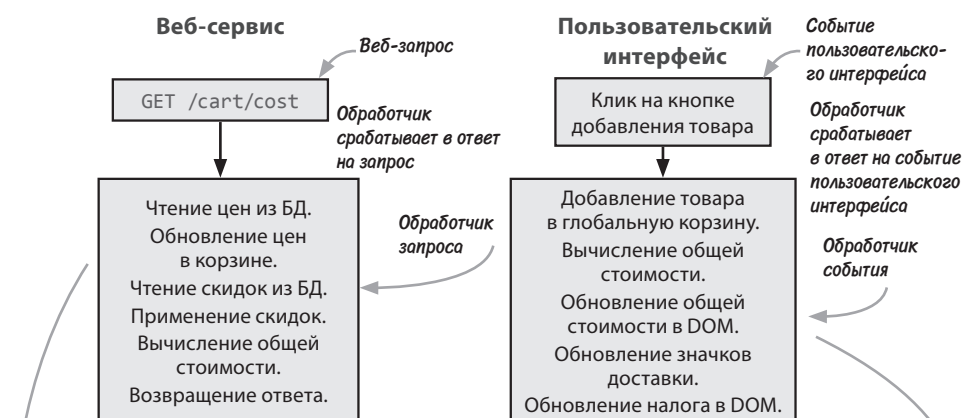
Дженна: Да! Та самая проблема. Есть какие-то мысли, как ее решать?

Ким: Думаю, можно воспользоваться реактивной архитектурой. Она отделяет действия в левом столбце от действий справа. На следующей странице я покажу, как это делается.

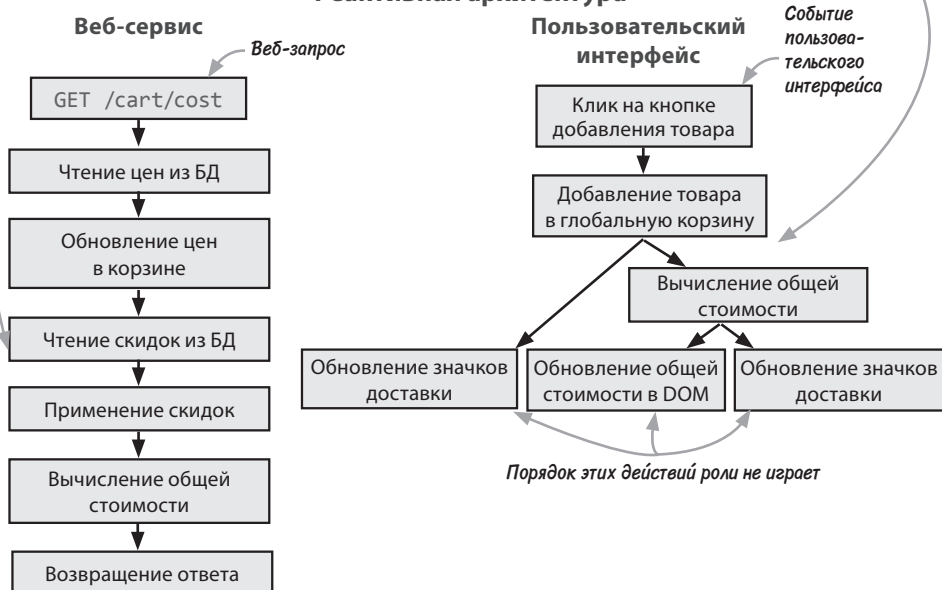
Что такое реактивная архитектура

Реактивная архитектура — еще один способ структурирования приложений. Согласно ее главному организационному принципу, вы определяете, что происходит в ответ на события. Она особенно эффективна в веб-сервисах и пользовательских интерфейсах. В веб-сервисах вы указываете, что происходит в ответ на веб-запросы. В пользовательском интерфейсе вы указываете, что происходит в ответ на события UI (например, клики кнопкой мыши). Обычно такие реакции называются обработчиками событий.

Примеры обработчиков событий



Реактивная архитектура



Обработчики событий позволяют указать: «Когда происходит X, выполнить Y, Z, A, B и C». В реактивной архитектуре этот принцип просто доводится до логического завершения. В предельном выражении все происходит в ответ на что-то. «Когда происходит X, выполнить Y. Когда происходит Y, выполнить Z. Когда происходит Z, выполнить A, B и C». Типичная пошаговая функция-обработчик разбивается на серию обработчиков, каждый из которых реагирует на предыдущий.

Плюсы и минусы реактивной архитектуры

Реактивная архитектура инвертирует типичный способ выражения упорядочения в коде. Вместо «Сделать X, потом сделать Y» реактивный стиль указывает: «Делать Y каждый раз, когда происходит X». Иногда это упрощает написание, чтение и сопровождение кода. Но не всегда! Реактивная архитектура — это не панацея. Вы должны руководствоваться здравым смыслом, чтобы определить, когда и как следует ее применять. При этом желательно хорошо понимать, на что способна реактивная архитектура. Затем вы сравниваете две архитектуры (типичную и реактивную) и решаете, достигает ли какая-либо из них ваших целей.

Отделение эффектов от причин

Отделение причин от эффектов иногда затрудняет чтение кода. Тем не менее иногда оно снимает ограничения и позволяет выражать ваши намерения намного точнее. Вскоре будут рассмотрены примеры обеих ситуаций.

Последовательность шагов рассматривается как конвейер

Вы уже знаете, какие возможности открывает построение конвейеров из этапов преобразования данных. Также мы пользовались конвейерами, объединяя функциональные инструменты в цепочки, — чрезвычайно эффективный способ объединения вычислений в более сложные операции. Реактивная архитектура позволяет аналогичным образом объединять действия с вычислениями.

Гибкость временных линий

Обратное выражение упорядочения обеспечивает гибкость временных линий. Конечно, как было показано ранее, эта гибкость может оказаться нежелательной, если она ведет к нежелательным возможным вариантам упорядочения. Но при разумном применении та же гибкость может сокращать временные линии.

Чтобы изучить эту тему, мы разработаем очень мощную первоклассную модель состояния, которая встречается во многих веб-приложениях и функциональных программах. Состояние является важной частью любых приложений, в том числе и функциональных. Мы займемся моделью состояния на следующей странице, а на примере написанного нами кода будет продемонстрирован каждый из приведенных выше пунктов.

Ячейки как первоклассное состояние

Корзина — единственный представитель глобального изменяемого состояния в нашем примере: все остальные были исключены. Требуется указать: «Делать Y при каждом изменении корзины».

На данный момент мы не знаем, когда изменится корзина. Это просто обычная глобальная переменная, и для ее изменения используется оператор присваивания. Одно из возможных решений — преобразование состояния к первоклассному статусу. Переменную можно преобразовать в объект для управления ее операциями. Первая попытка создания первоклассной изменяемой переменной:

```
function ValueCell(initialValue) {
  var currentValue = initialValue;
  return {
    val: function() {
      return currentValue;
    },
    update: function(f) {
      var oldValue = currentValue;
      var newValue = f(oldValue);
      currentValue = newValue;
    }
  };
}
```

Содержит одно неизменяемое значение (может быть коллекцией)

Получаем текущее значение

Изменяем значение, применяя функцию к текущему значению (паттерн «перестановка»)



Дженна из команды разработки

Имя ValueCell происходит от электронных таблиц, в которых также реализуется реактивная архитектура. При обновлении одной ячейки электронной таблицы формулы пересчитываются соответствующим образом.

ValueCell просто упаковывает переменную с двумя простыми операциями. Одна читает текущее значение (`val()`), а другая обновляет текущее значение (`update()`). Эти две операции реализуют паттерн, который использовался нами при реализации корзины. Пример его использования:

До

Паттерн «чтение, изменение, запись» (перестановка)

```
var shopping_cart = {};

function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart = add_item(shopping_cart, item);

  var total = calc_total(shopping_cart);
  set_cart_total_dom(total);
  update_shipping_icons(shopping_cart);
  update_tax_dom(total);
}
```

После

Ручная перестановка заменяется вызовом метода

```
var shopping_cart = ValueCell({});

function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart.update(function(cart) {
    return add_item(cart, item);
  });

  var total = calc_total(shopping_cart.val());
  set_cart_total_dom(total);
  update_shipping_icons(shopping_cart.val());
  update_tax_dom(total);
}
```

При таком изменении чтение и запись в `shopping_cart` становятся явными вызовами методов. Продолжим работу на следующей странице.

Переменную ValueCell можно сделать реактивной

На предыдущей странице мы определили новый примитив для представления изменяемого состояния. Мы все еще хотим иметь возможность указать: «Когда состояние изменяется, сделать X». Давайте реализуем эту возможность. Мы изменим определение ValueCell и добавим концепцию *наблюдателей* (watchers). Наблюдатели представляют собой функции, которые вызываются при каждом изменении состояния.

Оригинал

```
function ValueCell(initialValue) {
  var currentValue = initialValue;

  return {
    val: function() {
      return currentValue;
    },
    update: function(f) {
      var oldValue = currentValue;
      var newValue = f(oldValue);

      currentValue = newValue;
    }
  };
}
```

Для хранения списка наблюдателей

При изменении значения вызываются наблюдатели

Добавляем нового наблюдателя

С наблюдателями

```
function ValueCell(initialValue) {
  var currentValue = initialValue;
  var watchers = [];

  return {
    val: function() {
      return currentValue;
    },
    update: function(f) {
      var oldValue = currentValue;
      var newValue = f(oldValue);

      if (oldValue !== newValue) {
        currentValue = newValue;
        for (var i = 0; i < watchers.length; i++) {
          watchers[i](newValue);
        }
      }
    },
    addWatcher: function(f) {
      watchers.push(f);
    }
  };
}
```

Наблюдатели позволяют указать, что должно происходить при изменении корзины. Теперь мы можем использовать формулировки вида: «Когда корзина изменяется, обновлять значки доставки».



Загляни в словарь

У концепции наблюдателей существуют и другие названия. Ни одно имя нельзя считать более правильным, чем остальные. Возможно, вам встречались другие термины:

- *слушатели;*
- *обработчики событий;*
- *обратные вызовы.*

Все эти термины правильны, и все они представляют сходные концепции.

Теперь, когда у нас появился механизм отслеживания ячеек, посмотрим, как это выглядит в обработчике добавления товара в корзину.

Обновление значков доставки при изменении ячейки

На предыдущей странице мы включили в `ValueCell` метод для добавления наблюдателей. Кроме того, мы заставили наблюдателей выполняться каждый раз, когда изменяется текущее значение. Теперь мы можем добавить `update_shipping_icons()` как наблюдателя для `shopping_cart ValueCell`. В результате значки будут обновляться при каждом изменении корзины по какой-либо причине.

*Упрощаем обработчик события,
исключая последующие действия*

До

```
var shopping_cart = ValueCell({});

function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart.update(function(cart) {
    return add_item(cart, item);
  });
  var total = calc_total(shopping_cart.val());
  set_cart_total_dom(total);
  update_shipping_icons(shopping_cart.val());
  update_tax_dom(total);
}
```

После

```
var shopping_cart = ValueCell({});

function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart.update(function(cart) {
    return add_item(cart, item);
  });
  var total = calc_total(shopping_cart.val());
  set_cart_total_dom(total);

  update_tax_dom(total);
}
```

*Достаточно написать этот код один
раз, и он будет выполняться после
любых обновлений корзины*

```
shopping_cart.addWatcher(update_shipping_icons);
```

Здесь следует обратить внимание на два важных обстоятельства. Во-первых, наша функция-обработчик стала меньше. Она меньше делает. Ей уже не нужно вручную обновлять значки. Эта обязанность переместилась в инфраструктуру наблюдателей. Во-вторых, мы можем устранить вызов `update_shipping_icons()` из всех обработчиков. Он будет выполняться при любых изменениях в корзине, будь то добавление товара, удаление товара, обновление количества и т. д. Это именно то, что требовалось: значки всегда соответствуют текущему состоянию корзины. Из обработчика было исключено одно обновление DOM. Другие два зависят от корзины только косвенно. Непосредственно они зависят от *общей стоимости* — производного значения, вычисляемого на основании корзины. На следующей странице мы реализуем другой примитив, который поддерживает производное значение.

FormulaCell и вычисление производных значений

На предыдущей странице мы сделали `ValueCell` реактивным, добавив наблюдателей. Иногда требуется вычислить значение по существующей ячейке и поддерживать его актуальность при изменении ячейки. Именно это делает функция `FormulaCell`: она наблюдает за другой ячейкой и пересчитывает свое значение при изменении целевой ячейки.

```
function FormulaCell(upstreamCell, f) {
  var myCell = ValueCell(f(upstreamCell.val()));
  upstreamCell.addWatcher(function(newUpstreamValue) {
    myCell.update(function(currentValue) {
      return f(newUpstreamValue);
    });
  });
  return {
    val: myCell.val,
    addWatcher: myCell.addWatcher
  };
}
```

Используем механику ValueCell

Добавляем наблюдателя для пересчета текущего значения ячейки

val() и addWatcher() делегируют работу myCell

У FormulaCell нет средств для прямого изменения значения

Обратите внимание: напрямую изменить значение `FormulaCell` невозможно. Изменить его можно только изменением целевой ячейки, за которой оно наблюдает. `FormulaCell` работает по принципу: «Когда изменяется целевая ячейка, пересчитать мое значение на основании нового значения целевой ячейки». При этом для ячеек `FormulaCell` также могут назначаться наблюдатели.

Из-за того что им могут назначаться наблюдатели, мы можем добавить действия, происходящие в ответ на изменения общей стоимости:

До

```
var shopping_cart = ValueCell({});

function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart.update(function(cart) {
    return add_item(cart, item);
  });
  var total = calc_total(shopping_cart.val());
  set_cart_total_dom(total);
  update_tax_dom(total);
}
```

```
shopping_cart.addWatcher(update_shipping_icons);
```

После

```
var shopping_cart = ValueCell({});
var cart_total = FormulaCell(shopping_cart,
  calc_total);
```

cart_total будет изменяться при каждом изменении shopping_cart

```
function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart.update(function(cart) {
    return add_item(cart, item);
  });
}
```

Обработчик клика стал очень простым

```
shopping_cart.addWatcher(update_shipping_icons);
cart_total.addWatcher(set_cart_total_dom);
cart_total.addWatcher(update_tax_dom);
```

Теперь при каждом изменении корзины обновляются три части DOM. Более того, наш обработчик более прямолинейно сообщает, что он делает.

DOM будет обновляться в ответ на изменение cart_total

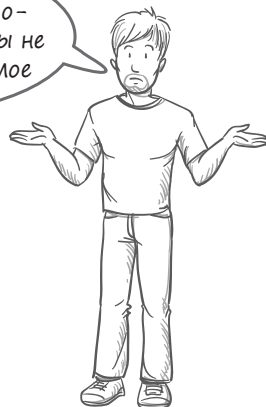
Изменяемое состояние в функциональном программировании

Возможно, вы слышали от функциональных программистов, что они не используют изменяемое состояние и избегают его любой ценой. Скорее всего, это преувеличение, потому что большинство программных продуктов злоупотребляет изменяемым состоянием.

Поддержание изменяемого состояния становится важной частью всех программных продуктов, включая написанные с применением функционального программирования. Все программные продукты должны получать информацию от изменяющегося мира и запоминать какую-то ее часть. Независимо от того, хранится ли информация во внешней базе данных или в памяти, что-то в коде должно как минимум узнавать о новых пользователях и действиях пользователей с программой. Здесь важна относительная безопасность используемого состояния. И хотя ячейки изменяемы, они чрезвычайно безопасны по сравнению с обычными глобальными переменными, если вы решите использовать их для хранения неизменяемых значений.

Метод `update()` объекта `ValueCell` позволяет легко обеспечить действительность текущего значения. Почему? Потому что `update()` вызывается с вычислением. Вычисление получает текущее значение и возвращает новое значение. Если текущее значение является действительным для предметной области и если вычисление всегда возвращает действительные значения при получении действительного ввода, то и новое значение всегда будет действительным. Объект `ValueCell` не может гарантировать порядок обновлений или чтений из разных временных линий, но может гарантировать, что любое хранящееся в нем значение будет действительным. Во многих ситуациях этого более чем достаточно.

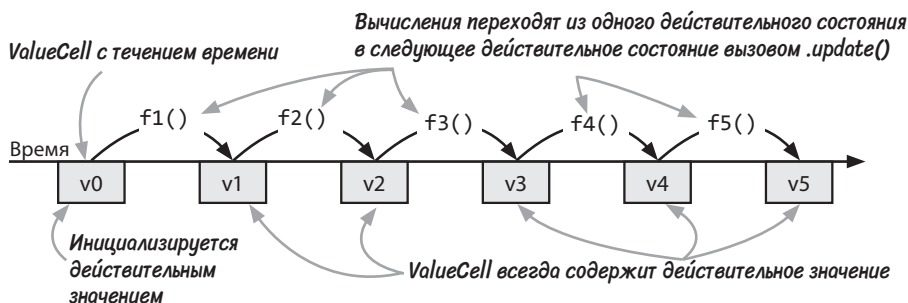
Секундочку!
Я думал, что функциональные программисты не используют изменяемое состояние!



Джордж из отдела тестирования

update() всегда передается вычисление

`ValueCell.update(f)`





Загляни в словарь

Эквиваленты `ValueCell` встречаются во многих функциональных языках и фреймворках:

- в Clojure: `Atom`;
- в Elixir: `Agent`;
- в React: `Redux store` и `Recoil atom`;
- в Haskell: `TVar`.

Правила целостности данных `ValueCell`

- Инициализируйте действительным значением.
- Передавайте `update()` вычисление (не действие!).
- При получении действительного ввода это вычисление должно возвращать действительное значение.

Как реактивная архитектура изменяет конфигурацию систем

Мы только преобразовали свой код в предельную разновидность реактивной архитектуры. Все компоненты стали обработчиками изменений других компонентов:

Типичная архитектура

```
var shopping_cart = {};
```

```
function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart = add_item(shopping_cart, item);
  var total = calc_total(shopping_cart);
  set_cart_total_dom(total);
  update_shipping_icons(shopping_cart);
  update_tax_dom(total);
}
```

Вся последовательность действий выражается в обработчике



Реактивная архитектура

```
var shopping_cart = ValueCell({});
var cart_total = FormulaCell(shopping_cart, calc_total);
```

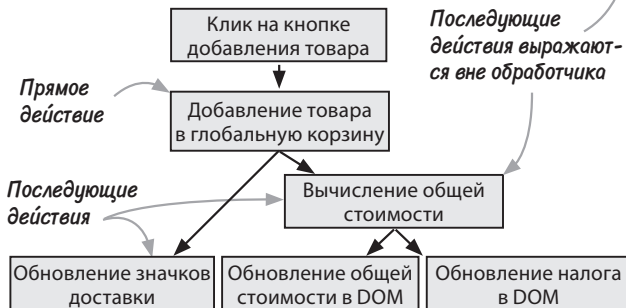
```
function add_item_to_cart(name, price) {
  var item = make_cart_item(name, price);
  shopping_cart.update(function(cart) {
    return add_item(cart, item);
  });
}
```

```
shopping_cart.addWatcher(update_shipping_icons);
cart_total.addWatcher(set_cart_total_dom);
cart_total.addWatcher(update_tax_dom);
```

Последующие действия выражаются вне обработчика

Прямое действие

Последующие действия



Необходимо исследовать последствия столь фундаментального изменения архитектуры. Как было показано ранее, реактивная архитектура имеет три основных последствия для кода.

1. Она отделяет эффекты от их причин.
2. Она интерпретирует последовательности шагов как конвейеры.
3. Она повышает гибкость временной линии.

Эти последствия будут рассмотрены на нескольких ближайших страницах.



Ваш ход

Чем являются `ValueCell` и `FormulaCell` — действиями, вычислениями или данными?

Ответ: Действиями, потому что они представляют изменяемое состояние. Результат вызова `val()` или `update()` зависит от того, код или сколько раз он вызывается.

Отделение эффектов от причин

Иногда вы хотите реализовать в своем коде некоторое правило. Для примера возьмем правило, которое было реализовано в этой книге: «Значки бесплатной доставки должны показывать, будет ли распространяться бесплатная доставка при включении товара в текущую корзину». Мы реализовали это сложное правило. Тем не менее в нем используется концепция текущей корзины. Оно подразумевает, что при изменении корзины также может возникнуть необходимость в обновлении значков.

Корзина может измениться по разным причинам. Мы уделяли основное внимание клику на кнопке добавления в корзину. Но как насчет кнопки удаления из корзины? Как насчет кнопки очистки корзины? Любая операция, выполняемая с корзиной, потребует выполнения практически одного и того же кода.

Вы находитесь
здесь

Реактивная архитектура

1. Отделяет эффекты от их причин.
2. Интерпретирует последовательности шагов как конвейеры.
3. Повышает гибкость временной линии.

Типичная архитектура



Один эффект записывается
три раза

В типичной архитектуре нам пришлось бы включать один и тот же код в каждый обработчик события UI. Когда пользователь кликает на кнопке добавления товара в корзину — обновить значки. Когда пользователь кликает на кнопке удаления товара из корзины — обновить значки. Когда он кликает на кнопке очистки корзины — обновить значки. Мы связываем причину (клик на кнопке) с эффектом (обновление значков). Реактивная архитектура позволяет отделить причину от эффекта. Фактически мы говорим: «При любом изменении корзины, независимо от причины, — обновить значки».

Реактивная архитектура



Достаточно один раз указать, что значки доставки должны обновляться. И правило можно сформулировать точнее: «Каждый раз, когда глобальная корзина изменяется *по любой причине*, — обновить значки доставки». На следующей странице вы увидите, какую проблему решает эта архитектура.

Центр связей между причинами и эффектами

Вы только что видели, что реактивная архитектура позволяет отделить причину от ее эффектов. Это мощный прием для решения довольно неприятной проблемы. В нашем случае эта проблема проявляется в виде многочисленных способов изменения корзины и многочисленных операций, выполняемых при изменении корзины.

Способы изменения корзины

1. Добавление товара.
2. Удаление товара.
3. Очистка корзины.
4. Обновление количества.
5. Применение кода скидки.

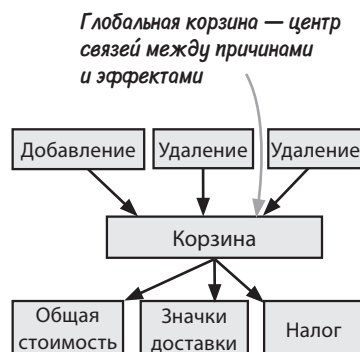
Действия, выполняемые при изменении корзины

1. Обновление значков доставки.
2. Вывод налога.
3. Вывод общей стоимости.
4. Обновление количества товаров в корзине.

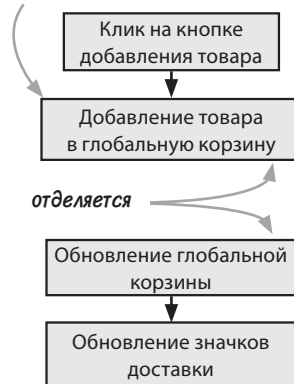
Существует много других способов изменения корзины и много других возможных действий. И они изменяются со временем. Представьте, что нам придется добавить одну операцию, которая должна выполняться при изменении корзины. В каком количестве мест нам придется ее добавить? В пяти, по одному для каждого способа изменения корзины. Аналогичным образом при добавлении одного способа изменения корзины необходимо добавить все действия в обработчик. И при добавлении новых возможностей на каждой из сторон проблема усугубляется.

Получается, что мы должны поддерживать 20 связей: пять возможностей изменения (причин), умноженные на четыре действия (эффекты). При добавлении новых причин или эффектов умножение только растет. Можно сказать, что глобальная корзина является центром связей между причинами и эффектами. Мы хотим управлять этим центром так, чтобы количество поддерживаемых связей не росло так быстро.

Быстрый рост — та самая проблема, которую решает отделение причин от эффектов. С ним умножение заменяется сложением: необходимо написать пять причин и отдельно написать четыре эффекта. Получается $5 + 4$ мест вместо 5×4 . При добавлении эффекта не нужно изменять причины, а при добавлении



Обработчик клика



Обработчик изменения корзины

причин не нужно изменять эффекты. Вот что я имел в виду, когда говорил об отделении причин от эффектов.

Если вы сталкиваетесь с этой проблемой в своем коде, такое решение будет исключительно мощным. Оно позволяет мыслить понятиями изменений корзины при программировании обработчиков событий. И оно позволяет мыслить понятиями обновлений DOM при выводе информации в DOM.

Если же такой проблемы нет, отделение ничем не поможет, а может только навредить. Иногда самым понятным способом выражения последовательности действий становится их последовательная запись, строка за строкой. Если центра нет, то нет и причины для отделения.

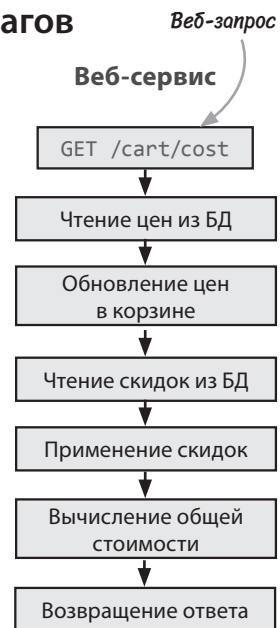
Интерпретация последовательности шагов как конвейера

В главе 13 было показано, как объединять вычисления с помощью цепочек функциональных инструментов, которые позволяют писать очень простые (легко реализуемые) функции, которые могут использоваться для создания сложного поведения. Кроме того, эти простые функции обеспечивают большие возможности повторного использования.

Реактивная архитектура позволяет строить сложные действия из более простых действий и вычислений. Составные действия воплощаются в форме конвейера. Данные поступают на вход и переходят от одного шага к другому. Конвейер может рассматриваться как действие, состоящее из меньших действий и вычислений.

Если у вас имеется серия шагов, которые должны быть выполнены программой, и данные, генерируемые одним шагом, используются как входные для следующего шага, конвейер может оказаться именно тем, что вам нужно. Возможно, подходящий примитив поможет вам реализовать эту функциональность на вашем языке.

Конвейеры чаще всего реализуются с использованием реактивных фреймворков. В JavaScript обещания (promises) предоставляют механизм построения конвейеров из действий и вычислений. Обещание работает для отдельного значения, переходящего между фазами конвейера.



Вы находитесь здесь

Реактивная архитектура

1. Отделяет эффекты от их причин.
2. **Интерпретирует последовательность шагов как конвейеры.**
3. Повышает гибкость временной линии.

Если вместо одного события вам нужен поток событий, семейство библиотек ReactiveX (<https://reactivex.io>) предоставит вам необходимые инструменты. Потоки событий предоставляют средства для отображения и фильтрации событий. Реализации существуют для многих языков, включая RxJS для JavaScript.

Также существуют внешние потоковые сервисы, такие как Kafka (<https://kafka.apache.org>) или RabbitMQ (<https://www.rabbitmq.com>). Они позволяют реализовать реактивную архитектуру в более крупном масштабе между отдельными сервисами вашей системы.

Если шаги не следуют паттерну передачи данных, либо измените их структуру, либо подумайте над тем, чтобы отказаться от использования паттерна. Если данные не передаются, то последовательность не является конвейером. Возможно, реактивная архитектура в данном случае не подходит.

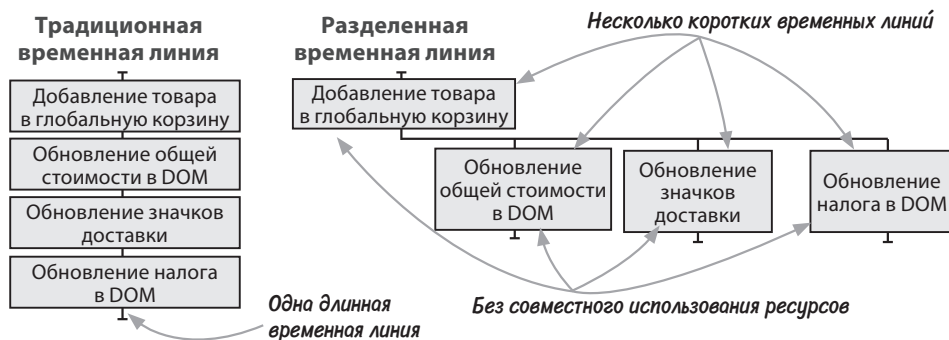


Глубокое погружение

Реактивная архитектура набирает популярность для построения микросервисов. Ее преимущества отлично объясняются в «Реактивном манифесте» (<https://www.reactivemanoifesto.org>).

Гибкость временной линии

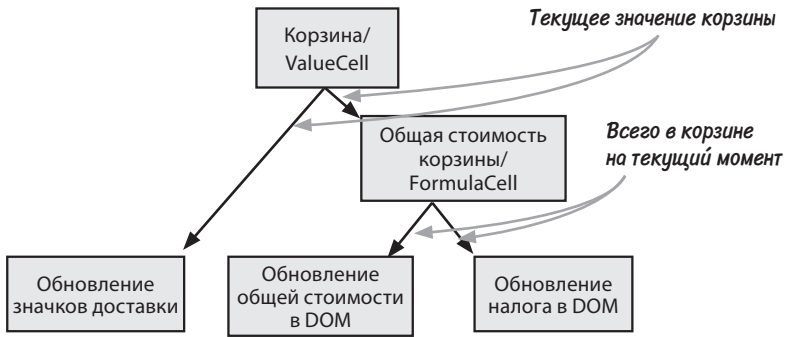
Реактивная архитектура также может обеспечить желаемую гибкость временной линии. Так как она инвертирует наш типичный подход к определению упорядочения, это способствует естественному разделению временных линий на меньшие части:



Как было показано начиная с главы 15, с короткими временными линиями удобнее работать. Тем не менее работать с большим количеством временных линий

обычно труднее. Фокус в том, чтобы разбить временные линии с исключением совместно используемых ресурсов.

Распространение событий



Объект **ValueCell** корзины вызывает свои функции-наблюдатели с текущим значением корзины. Функциям-наблюдателям не нужно читать **ValueCell** корзины, поэтому они не используют глобальную корзину как ресурс. Аналогичным образом объект **FormulaCell** для общей стоимости вызывает свои функции-наблюдатели с текущей общей стоимостью. Обновления DOM также не используют **FormulaCell** для общей стоимости. Каждое обновление DOM изменяет отдельную часть DOM. Их можно безопасно считать разными ресурсами; следовательно, никакие из этих временных линий не имеют общих ресурсов.

*Вы находитесь
здесь*

Реактивная архитектура

1. Отделяет эффекты от их причин.
2. Интерпретирует последовательности шагов как конвейеры.
3. **Повышает гибкость временной линии.**



Ваш ход

Необходимо спроектировать систему уведомлений, которая будет извещать пользователей об изменениях в их учетной записи, изменении условий обслуживания и появлении специальных предложений. Возможно, в будущем появятся и другие причины для уведомлений.

В то же время уведомления должны передаваться пользователю разными способами. Они могут отправляться по электронной почте, выводиться в баннерах на веб-сайте или размещаться в разделе сообщений на сайте. И в будущем также могут появиться другие способы уведомления. Разработчик из команды предложил использовать реактивную архитектуру. Будет ли это уместным применением реактивной архитектуры? Почему?

*Запишите здесь
свой ответ*



Ответ

Да, похоже, что в данном случае реактивная архитектура будет уместной. Есть несколько причин (поводов для уведомления) и эффектов (способов уведомления пользователей). Реактивная архитектура позволяет отделить причины от эффектов, чтобы они могли изменяться независимо.



Ваш ход

Новая система обработки документов использует чрезвычайно прямолинейную последовательность шагов, необходимую для выполнения стандартных операций. Документ проверяется, снабжается криптографической подписью, сохраняется в архиве и регистрируется в журнале. Будет ли это уместным применением реактивной архитектуры? Почему?

*Запишите здесь
свой ответ*

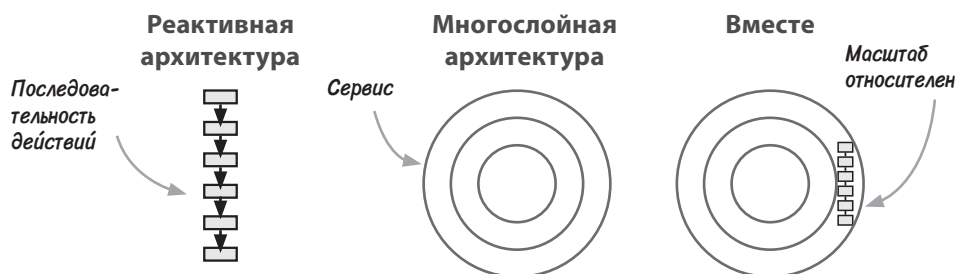


Ответ

Вероятно, нет. У этой последовательности нет центра связей между причинами и эффектами, с которым реактивная архитектура особенно полезна. Вместо этого шаги всегда выполняются последовательно, и ни один шаг не является очевидной причиной другого. Здесь уместна более прямолинейная последовательность.

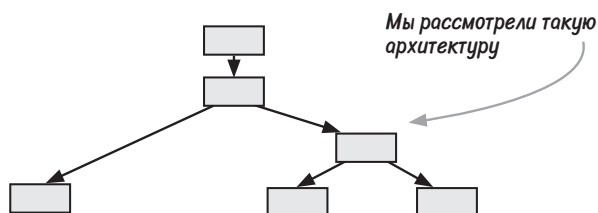
Второй архитектурный паттерн

Мы только что рассмотрели реактивную архитектуру. Теперь мы перейдем к совершенно другой архитектуре, которая называется *многослойной*. Многослойная архитектура применяется в более крупном масштабе, чем реактивная. Многослойная архитектура используется для построения целых сервисов, чтобы они могли взаимодействовать с окружающим миром. При совместном использовании двух архитектур реактивная архитектура часто вкладывается в многослойную, хотя ни одна из них не требует использования другой.



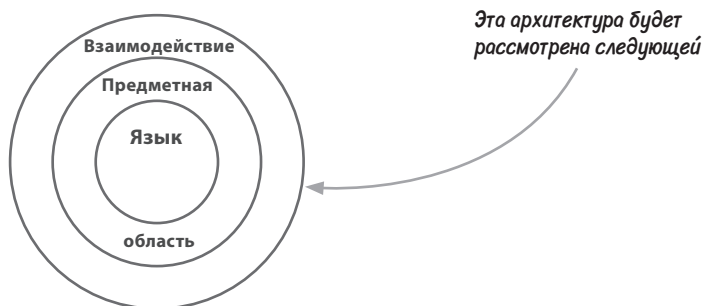
Реактивная архитектура

Реактивная архитектура инвертирует способ выражения порядка действий в вашем коде. Как вы увидите, он помогает отделить причину от эффекта, который может разобратся в некоторых запутанных частях вашего кода.



Многослойная архитектура

Многослойная архитектура определяет структуру для сервисов, которые должны взаимодействовать с внешним миром, будь то веб-сервис или термостат. Архитектура естественным образом следует из применения функционального мышления.



Что такое многослойная архитектура

Многослойная архитектура определяет структуру для сервисов, которые должны взаимодействовать с внешним миром. Как подсказывает само название, архитектура изображается в виде набора концентрических слоев, как луковица:



Слой взаимодействия

- Действия, которые влияют на окружающий мир или находятся под его влиянием.

Слой предметной области

- Вычисления, определяющие правила бизнеса.

Слой языка

- Язык и вспомогательная библиотека.

Многослойная архитектура не определяет конкретный набор уровней, но обычно они делятся на три большие группы. Даже этот простой пример демонстрирует главные правила, из-за которых они хорошо работают в функциональных системах. Вот эти правила.

1. Взаимодействие с окружающим миром осуществляется исключительно в слое взаимодействия.
2. Слои направляют вызовы к центру.
3. Слои не знают о других слоях, находящихся вне их самих.

Многослойная архитектура очень хорошо сочетается с делением на действия/вычисления и многоуровневым проектированием, о котором говорилось в главе 1. Мы кратко рассмотрим эти комбинации, а затем посмотрим, как многослойная архитектура применяется в реальных сценариях.

Краткий обзор: действия, вычисления и данные

В части I мы узнали о различиях между действиями, вычислениями и данными. Здесь мы возвращаемся к этой теме, потому что эти различия сильно влияют на наш выбор для построения архитектуры.

Данные

Начнем с данных, потому что это самая простая категория. Данные представляют собой факты, относящиеся к событиям: числа, строки, коллекции и т. д. Данные инертны и прозрачны.

Вычисления

Вычисления представляют собой преобразования входных данных в выходные. Для постоянных входных данных они всегда выдают один и тот же результат. Это означает, что результат вычислений не зависит от того, когда или сколько раз они выполняются. По этой причине они не наносятся на временные линии, потому что порядок их выполнения не важен. Большая часть того, что делалось в части I, была связано с вынесением кода из действий в вычисления.

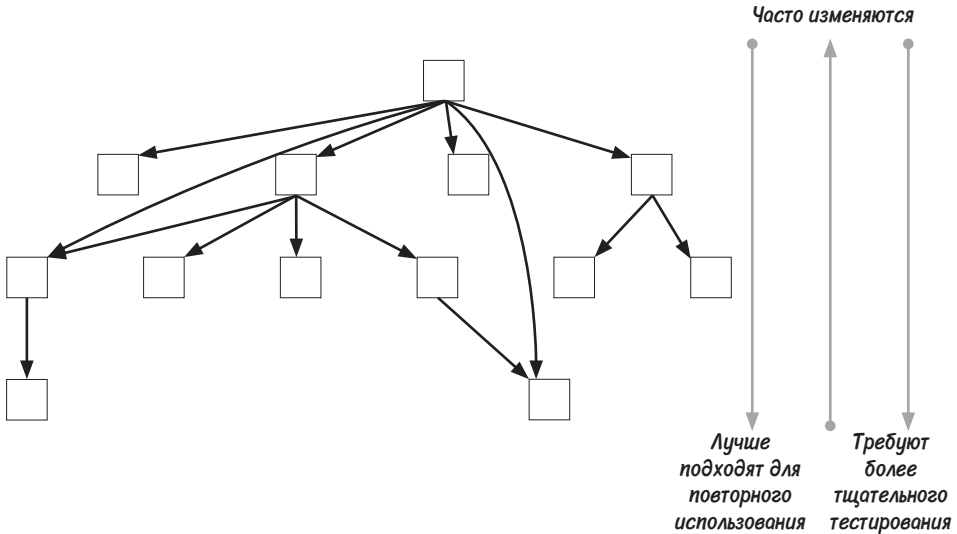
Действия

Действия представляют собой исполняемый код, который влияет на окружающий мир или находится под его влиянием. Это означает, что они зависят от того, когда и сколько раз они выполняются. Значительная доля части II посвящена управлению сложностью действий. Так как взаимодействие с базой данных, API и веб-запросы являются действиями, мы будем часто обращаться к этим темам в этой главе.

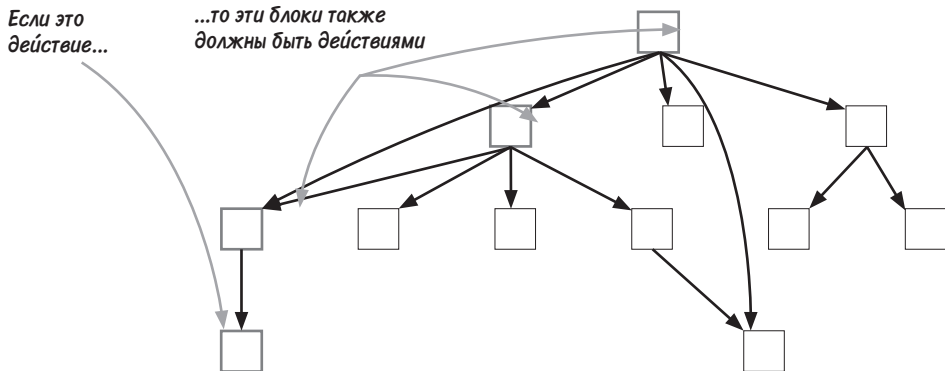
Следуя рекомендациям главы 4, которая предлагала нам извлекать вычисления из действий, мы естественным образом приходим к чему-то сильно напоминающему многослойную архитектуру. По этой причине функциональные программисты могут считать многослойную архитектуру чем-то настолько очевидным, что это даже не заслуживает специального термина. Однако это название используется (и поэтому его важно знать), и оно полезно для получения высокоуровневого представления о возможной структуре сервисов при использовании функционального программирования.

Краткий обзор: многоуровневое проектирование

Многоуровневое проектирование — представление, в котором функции распределяются по уровням в зависимости от того, какие функции они вызывают и какие функции вызывают их. Такое представление помогает прояснить, какие функции лучше подходят для повторного использования, проще изменяются и требуют более тщательного тестирования.



Эта диаграмма также хорошо демонстрирует правило распространения: если один из блоков является действием, каждый блок на пути до верха также является действием:



Если на графе имеется хотя бы одно действие, то вершина графа также будет действием. Часть I в значительной мере была посвящена отделению действий от вычислений. На следующей странице показано, как выглядит построение графа с действиями, отделенными от вычислений.

Традиционная многоуровневая архитектура

Традиционные веб-API тоже часто называются многоуровневыми. Тем не менее в них используются другие уровни. Типичная структура многоуровневого веб-сервера выглядит так:

Веб-интерфейс
Предметная область
База данных

Уровень веб-интерфейса

- Преобразует веб-запросы в концепции предметной области, а концепции предметной области — в веб-ответы.

Уровень предметной области

- Специализированная логика приложения, часто преобразует концепции предметной области в запросы к БД и команды.

Уровень базы данных

- Хранит информацию, изменяющуюся со временем.

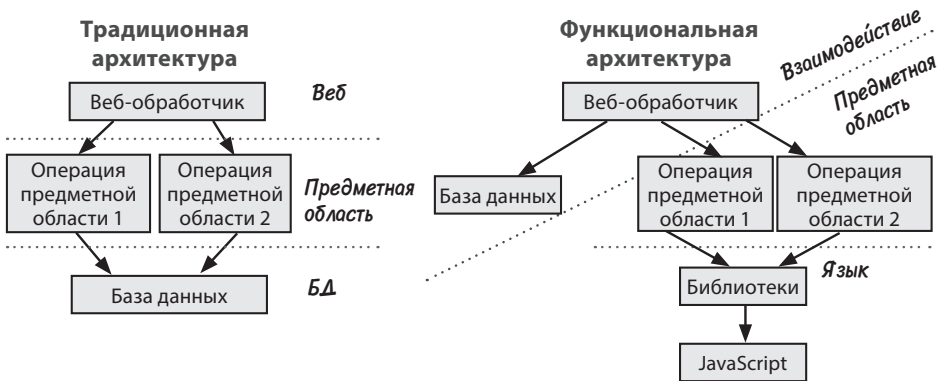
В этой архитектуре база данных (БД) становится фундаментом, лежащим в основе всего. Уровень предметной области строится (среди прочего) из операций с БД. Веб-интерфейс преобразует веб-запросы к операциям с предметной областью.

Такая архитектура встречается достаточно часто. Мы видим ее в таких фреймворках, как Ruby on Rails, которые строят модель предметной области (М в MVC) с использованием объектов активных записей, осуществляющих выборку и сохранение в базе данных. Конечно, успех этой архитектуры невозможно оспаривать, но она не функциональна. Дело в том, что размещение базы данных в основании всего означает, что все на пути до верха является действием. В данном случае это весь стек! Любое использование вычислений является чисто случайным. В функциональной архитектуре важная роль должна принадлежать как вычислениям, так и действиям.

Сравним ее с функциональной архитектурой на следующей странице.

Функциональная архитектура

Сравним традиционную (нефункциональную) архитектуру с функциональной. Главное различие заключается в том, что в традиционной многоуровневой схеме база данных находится в самом низу, тогда как в функциональной база данных располагается сбоку. База данных изменяема, поэтому доступ к ней является действием. Затем мы можем провести линию, отделяющую действия от вычислений, и другую линию, отделяющую наш код от языка и используемых библиотек:



База данных является изменяемой. В этом вся ее суть. Но в результате любое обращение к ней становится действием. Все, что находится на пути к вершине графа, неизбежно становится действием, включая все операции предметной области. Как вы узнали в главе 1, функциональные программисты предпочитают отделять вычисления от действий. Они стремятся к четкому разделению вплоть до построения всей бизнес-логики и логики предметной области в форме вычислений. База данных существует отдельно (хотя и играет важную роль). Действие на вершине связывает правила предметной области с состоянием в базе данных.

Если свернуть пунктиры функциональной архитектуры в круги, мы получим исходную диаграмму многослойной архитектуры:



Правила многослойной архитектуры

1. Взаимодействие с окружающим миром осуществляется исключительно в слое взаимодействия.
2. Слои направляют вызовы к центру.
3. Слои не знают о других слоях, находящихся вне их самих.

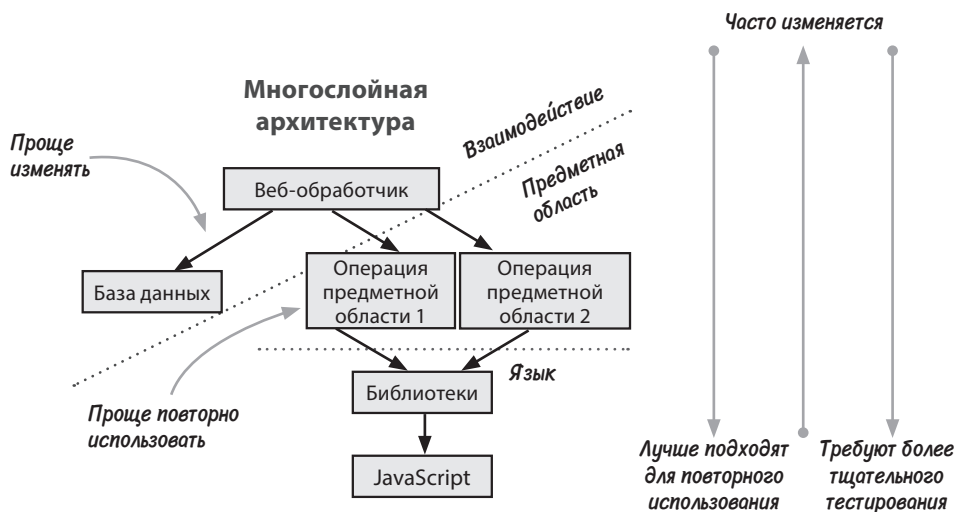
Упрощение изменений и повторного использования

В каком-то смысле сутью программной архитектуры является упрощение изменений. Какие изменения вы хотите упростить? Если вы сможете ответить на этот вопрос, значит, вы уже на половине пути к выбору архитектуры.

Мы сейчас рассматриваем многослойную архитектуру, поэтому резонно задаться вопросом: «Какие изменения упрощает многослойная архитектура?»

Многослойная архитектура позволяет легко изменять уровень взаимодействия. На вершине находится слой взаимодействия, который, как мы видели, изменяется проще всего. Поскольку слой предметной области ничего не знает о базах данных и веб-запросах, мы можем легко изменить базу данных или перейти на другой протокол обслуживания. Можно также использовать вычисления в слое предметной области вообще без баз данных или сервисов. Это одно из изменений, которые упрощаются правильным выбором архитектуры:

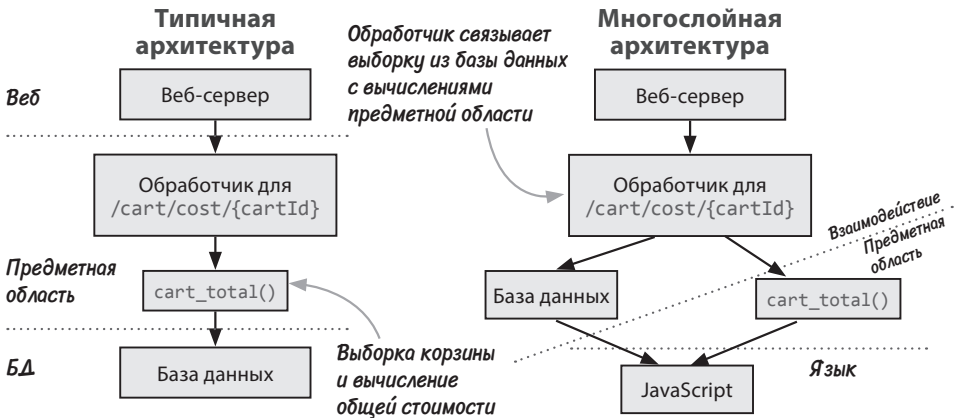
Многослойная архитектура упрощает изменение слоя взаимодействия. Она упрощает повторное использование слоя предметной области.



Это важный момент, и его стоит повторить: внешние сервисы (такие, как базы данных и вызовы API) проще всего изменяются в такой архитектуре. К ним обращается только самый верхний слой. Все, что находится в слое предметной области, проще тестировать, потому что он не содержит обращений к внешним сервисам. Многослойная архитектура подчеркивает ценность хороших моделей предметной области относительно выбора другой инфраструктуры.

В типичной архитектуре правила предметной области обращаются к базе данных. Но в многослойной архитектуре это невозможно. В многослойной архитектуре выполняется та же работа, но с другой структурой графа вызовов. Рассмотрим пример: веб-сервис для вычисления общей стоимости товаров в корзине. Веб-запрос выдается для пути `/cart/cost/123`, где 123 — идентификатор корзины. Идентификатор может использоваться для чтения корзины из базы данных.

Сравним две архитектуры.



В типичной архитектуре существует четкая иерархия уровней. Веб-запрос передается обработчику. Обработчик обращается к базе данных. Затем он возвращает ответ верхнему уровню, который пересылает его клиенту.

В этой архитектуре правило предметной области для вычисления общей стоимости корзины осуществляет выборку из базы данных и вычисляет сумму. Это не является вычислением, потому что информация читается из базы данных.

В многослойной архитектуре нам приходится основательно потрудиться, чтобы разглядеть уровни, потому что разделительная линия наклонена. Веб-сервер, обработчик и база данных принадлежат слою взаимодействия. `cart_total()` представляет вычисление, которое описывает, как цены товаров в корзине должны суммироваться в общую стоимость заказа. Функция не знает, откуда берется корзина (из базы данных или откуда-то еще). Задача веб-обработчика — предоставить корзину, загрузив ее из базы данных. Таким образом, выполняется одна и та же работа, но на разных слоях. Выборка выполняется в слое взаимодействия, а суммирование — в слое предметной области.

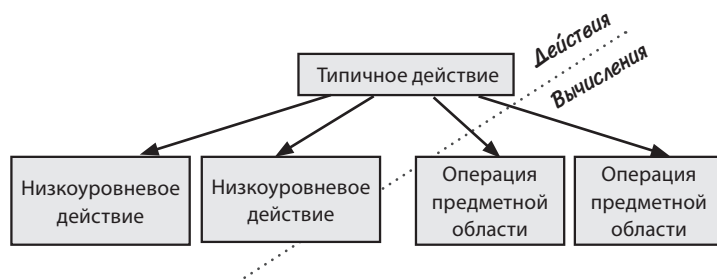
Дженна задала отличный вопрос.

Короткий ответ: вы всегда можете составить свою предметную область из вычислений. В части I мы рассмотрели немало примеров того, как это делается. Вычисления выделяются из действий. Вычисления и действия упрощаются до того момента, когда низкоуровневые действия содержат минимум логики. Таким образом, как было показано в этой главе, высокоуровневые действия связывают воедино действия и вычисления предметной области.

Я вижу закономерность, но разве не встречаются ситуации, в которых правила предметной области должны быть действиями?



Дженна
из команды
разработки



Полный ответ не столь однозначен. Когда вы решаете, должно ли конкретное правило предметной области быть вычислением или действием, необходимо учитывать два фактора.

1. Исследуйте понятия, используемые для размещения правила в слое.
2. Проанализируйте удобочитаемость и громоздкость решения.

Эти два фактора будут рассмотрены на нескольких ближайших страницах.

Понятия, используемые для размещения правила в слое

Мы часто рассматриваем всю важную логику программы как *правило предметной области* (иногда оно также называется *бизнес-правилом*). Тем не менее не вся логика относится к предметной области. Обычно понятия, используемые в коде, помогают решить, является ли этот код правилом предметной области. Например, код может решать, какую базу данных следует использовать. Если в новой базе данных хранится изображение товара, используйте ее. В противном случае используется старая база данных. Заметим, что в этом коде задействованы два действия (чтения из базы данных):

```
var image = newImageDB.getImage('123');
if(image === undefined)
    image = oldImageDB.getImage('123');
```

.....

Правила домена сформулированы в терминах домена. Посмотрите на термины в коде, чтобы узнать, является ли это правилом предметной области или относится к уровню взаимодействия.

.....

И хотя этот выбор жизненно важен для работы приложения, он на самом деле не является правилом предметной области. Он не выражается в понятиях предметной области. Понятия предметной области — *товар, изображение, цена, скидка* и т. д. Понятие «база данных» не описывает предметную область, а понятия новой и старой базы данных — в еще меньшей степени.

Этот код — техническая подробность, которая должна справиться с тем фактом, что некоторые изображения товаров еще не были перенесены в новую базу данных. Будьте внимательны: эту логику не следует путать с правилом предметной области. Код безусловно принадлежит слою взаимодействия. Он очевидным образом предназначен для взаимодействия с изменяющимся миром.

Другой пример: логика повторения неудачных веб-запросов. Допустим, у вас имеется код, который повторяет несколько попыток в случае неудачи веб-запроса:

```
function getWithRetries(url, retriesLeft, success, error) {
    if(retriesLeft <= 0)
        error('No more retries');
    else ajaxGet(url, success, function(e) {
        getWithRetries(url, retriesLeft - 1, success, error);
    });
}
```

Этот код также не является бизнес-правилом, даже при том, что повторные попытки важны для работы приложения. Он не выражается в понятиях предметной области. Предметная область интернет-коммерции не имеет прямого отношения к *запросам* AJAX. Это просто логика для преодоления проблем ненадежных сетевых подключений. Следовательно, она принадлежит слою взаимодействия.

Анализ удобочитаемости и громоздкости решения

Кажется, мы переходим от абстракций к конкретике. Я должен однозначно заявить: иногда преимущества некоторой парадигмы не оправдывают затрат на ее реализацию. Это относится и к реализации частей предметной области в виде вычислений. Хотя абсолютно ничто не мешает реализовать предметную область из одних вычислений, приходится учитывать, что иногда в определенном контексте действие читается лучше эквивалентного вычисления.

Удобочитаемость зависит от ряда факторов. Самые важные из них следующие:

- язык, на котором вы пишете;
- используемые библиотеки;
- унаследованный код и стиль программирования;
- привычки ваших программистов.

Представление многослойной архитектуры, которое приведено здесь, является *идеализированным* представлением реальной системы. В попытках достичь 100%-ной чистоты видения многослойной архитектуры можно вывернуться наизнанку. Тем не менее ничто не идеально. Одной из составляющих вашей роли как проектировщика архитектуры становится поиск компромисса между соблюдением архитектуры и соображениями реализма.

Удобочитаемость кода

Хотя функциональный код обычно очень хорошо читается, время от времени язык программирования делает нефункциональную реализацию намного более понятной. Будьте внимательны и обращайтесь внимание на такие случаи. Иногда в краткосрочной перспективе бывает лучше принять нефункциональный подход. Тем не менее также постарайтесь найти более ясный и удобочитаемый способ четкого отделения вычислений слоя предметной области от действий слоя взаимодействия (обычно основанный на извлечении вычислений).

Скорость разработки

Иногда нам приходится выпускать функциональность быстрее, чем нам хотелось бы по соображениям методологии. Спешка никогда не приводит к идеальным результатам, и когда вы спешите, приходится идти на многочисленные компромиссы. Будьте готовы позже заняться чисткой кода, чтобы привести его в соответствие с архитектурой. При этом можно использовать стандартные приемы, которые были представлены в книге: отделение вычислений, преобразование в цепочки функциональных инструментов и манипуляции с временными линиями.

Быстродействие системы

Нам часто приходится идти на компромисс ради быстродействия системы. Например, изменяемые данные несо-
мненно работают быстрее неизменяемых. Обязательно изолируйте эти компромиссы.

А еще лучше, рассматривайте оптимизацию как часть слоя взаимодействия и посмотрите, нельзя ли использовать вычисления слоя предметной области другим, более быстрым способом. Пример такого рода был приведен на с. 81, где мы оптимизировали генерирование электронной почты за счет выборки меньшего количества записей из базы данных. Вычисления в предметной области при этом вообще не изменились.

Применять новую архитектуру всегда непросто. С ростом квалификации ваша команда научится применять архитектуру так, чтобы код с первой попытки получался удобочитаемым.

Джордж задал хороший вопрос. Это вполне реальный сценарий, с которым вы можете столкнуться. Допустим, вы хотите построить отчет обо всех товарах, которые были проданы за последний год. Вы пишете функцию, которая получает товары и строит отчет:

```
function generateReport(products) {
  return reduce(products, "", function(report, product) {
    return report + product.name + " " + product.price + "\n";
  });
}
```

```
var productsLastYear = db.fetchProducts('last year');
var reportLastYear = generateReport(productsLastYear);
```

Пока все хорошо и функционально. Но затем приходит новое требование, и отчет приходится изменять. Теперь в него нужно включить данные скидок. К сожалению, запись товара включает только необязательный идентификатор скидки, а не полноценную запись с информацией о скидке. Запись с информацией о скидке также необходимо загрузить из базы данных:

<pre>{ name: "shoes", price: 3.99, discountID: '23111' }</pre>	<i>Товар с discountID</i> 	<pre>{ name: "watch", price: 223.43, discountID: null }</pre>	<i>Товар без discountID</i> 
--	--	---	--

Как получить необязательные данные на уровне предметной области?



Джордж из отдела тестирования

Самое простое решение — загрузить скидку, заданную идентификатором, в обратном вызове свертки. Но тогда `generateReport()` станет действием. Действия должны выполняться на верхнем уровне — на том же уровне, что и код загрузки товаров из БД:

```
function generateReport(products) {
  return reduce(products, "", function(report, product) {
    return report + product.name + " " + product.price + "\n";
  });
}

var productsLastYear = db.fetchProducts('last year');
var productsWithDiscounts = map(productsLastYear, function(product) {
  if(!product.discountID)
    return product;
  return objectSet(product, 'discount', db.fetchDiscount(product.discountID));
});
var reportLastYear = generateReport(productsWithDiscounts);
```

*Информация
товара
дополняется
на верхнем
уровне*



Помните: всегда возможно построить предметную область из вычислений и четко отделить слой взаимодействия от слоя предметной области.

**Ваш ход**

Вы работаете над программной системой для общественной библиотеки, которая следит за тем, кому из читателей была выдана та или иная книга. Пометьте каждый из следующих блоков функциональности буквой В, П или Я в зависимости от того, к какому слою он относится: взаимодействие, предметная область или язык.

1. Импортированная вами библиотека для работы со строками.
2. Функции для запроса записей пользователей из базы данных.
3. Обращение к API Библиотеки Конгресса.
4. Функции для определения того, на какой полке находятся книги по заданной теме.
5. Функции для вычисления суммы штрафа по списку просроченных книг.
6. Функция для сохранения нового адреса посетителя.
7. Библиотека JavaScript Lodash.
8. Функции для вывода экрана выдачи книг посетителю.

Условные обозначения**В** Слои взаимодействия**П** Слои предметной области**Я** Слои языка**Ответ**

1. Я. 2. В. 3. В. 4. П. 5. П. 6. В. 7. Я. 8. В.

Итоги главы

В этой главе были представлены высокоуровневые описания двух архитектурных паттернов: *реактивной* архитектуры и *многослойной* архитектуры. Реактивная архитектура кардинально меняет способ упорядочения действий: вы указываете, какие действия выполняются в ответ на другие действия.

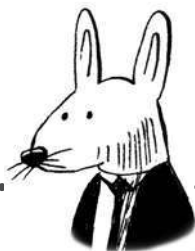
Многослойная архитектура — паттерн, который естественным образом возникает при применении практик функционального программирования. Эта точка зрения на архитектуру чрезвычайно полезна, потому что она проявляется на всех уровнях кода.

Резюме

- Реактивная архитектура инвертирует способ выражения упорядочения действий. Вместо «Сделать X, потом сделать Y» вы указываете: «Делать Y каждый раз, когда происходит X».
- Реактивная архитектура в предельном выражении организует действия и вычисления в конвейеры. Конвейеры представляют собой совокупности простых действий, которые происходят в определенной последовательности.
- Мы можем создать первоклассное изменяемое состояние, которое позволяет управлять операциями чтения и записи. Одним из примеров такого рода является концепция ячеек `ValueCell`, которая берет за образец концепции электронных таблиц и реализует реактивный конвейер.
- Многослойная архитектура в общих чертах делит программный код на три слоя: взаимодействия, предметной области и языка.
- Внешний слой взаимодействий содержит действия. Он координирует действия на основании вызовов к слою предметной области.
- Слой предметной области содержит логику предметной области и операции вашей программы, включая бизнес-правила. Этот уровень формируется исключительно из вычислений.
- Слой языка содержит язык и вспомогательные библиотеки, с которыми строится ваш программный продукт.
- Многослойная архитектура может проявляться на всех уровнях абстракции в действиях вашего кода.

Что дальше?

Часть II подошла к концу. В следующей главе наше путешествие в мир функционального программирования завершится обзором того, что вы узнали. Также вы узнаете, где следует искать информацию, чтобы узнать еще больше.



В этой главе

- ✓ Отработка и применение новых навыков без проблем с начальством.
- ✓ Выбор нового языка для погружения в функциональное программирование.
- ✓ Глубокое изучение математических аспектов функционального программирования.
- ✓ Другие книги по функциональному программированию.

Да, у вас получилось! Вы дочитали книгу до конца. За это время вы освоили много полезных фундаментальных навыков функционального программирования. Эти навыки принесут огромную пользу в любых проектах, над которыми вы будете работать. И еще они закладывают прочную основу для дальнейшего обучения. В этой главе вы найдете практические советы для применения новых навыков, а также узнаете некоторые возможности для продолжения обучения после того, как вы дочитаете книгу.

План главы

Считайте эту главу своего рода обрядом посвящения. Она поможет вам перейти от теоретических знаний к реальному программированию. Ниже приведен план такого перехода.

Припомните все, что вы узнали

Мы кратко опишем то, что вы узнали в книге. Это поможет вам оценить, какой большой путь вы проделали.

Сформируйте модель перехода к уровню мастера

Вы овладели навыками. Скорее всего, вам не терпится начать применять их на практике. В процессе освоения навыков мы проходим примерно похожий путь. В этом разделе приводится метаописание этого пути, чтобы вы могли двигаться к вершинам мастерства. Заодно вы познакомитесь с моделью двух путей.

Сформируйте путь 1: песочница

Необходимо создать безопасное место для экспериментов с новыми навыками. Мы подробно рассмотрим две разновидности песочниц:

- побочные проекты;
- упражнения.

Сформируйте путь 2: реальный код

Чтобы новообретенные навыки закалились, желательно подвергнуть их давлению условий реальной эксплуатации. Вы можете применять их так, как считаете нужным, но я могу предложить пару хороших отправных точек:

- устранение ошибок в реальном коде;
- пошаговое совершенствование структуры.

Продолжите путешествие в область функционального программирования

Вы прошли долгий путь и заложили прочный фундамент. Когда вам захочется узнать больше, перед вами откроются следующие пути:

- изучение функционального языка;
- математическая подготовка;
- чтение других книг.

Полученные профессиональные навыки

Поскольку книга подходит к концу, оглянемся назад, чтобы вы поняли, какой долгий путь был пройден. Я приведу высокоуровневый список навыков, которые вы приобрели в книге. Эти навыки, входящие в арсенал профессиональных функциональных программистов, были выбраны за свою мощь и глубину.

Часть I. Действия, вычисления и данные

- Выявление самых проблематичных частей кода с проведением различий между действиями, вычислениями и данными.
- Расширение возможностей повторного использования и тестирования кода за счет извлечения вычислений из действий.
- Улучшение структуры действий посредством замены неявного ввода и вывода явным.
- Реализация неизменяемости для превращения чтения данных в вычисление.
- Организация и усовершенствование кода с помощью многоуровневого проектирования.

*Вы находитесь
здесь*

План главы

- **Список навыков.**
- Два пути к мастерству.
- Путь 1: песочница.
- Путь 2: реальный код.
- Дальнейшее путешествие.

Часть II. Первоклассные абстракции

- Преобразование синтаксических операций в первоклассные сущности для абстрагирования в коде.
- Рассуждения на более высоком уровне с использованием функциональных средств перебора и других функциональных инструментов.
- Объединение функциональных инструментов в конвейеры преобразования данных.
- Анализ распределенных и параллельных систем с использованием временных диаграмм.
- Операции с временными диаграммами для устранения ошибок.
- Безопасное изменение состояния с использованием функций высшего порядка.
- Применение реактивной архитектуры для разрыва связей между причинами и эффектами.
- Применение многослойной архитектуры для проектирования сервисов, взаимодействующих с внешним миром.

Главные выводы

Возможно, лет через десять вы будете помнить только три вывода из этой книги. Даже если вы забудете все остальное, вот три самые важные вещи, которые следует помнить:

В действиях часто скрываются вычисления

Идентификация и извлечение вычислений требуют работы, но дело того стоит. Вычисления проще в тестировании и повторном использовании, чем действия, и их проще понять. Они не влияют на длину временных линий. Различия между действиями, вычислениями и данными лежат в основе многих навыков функционального программирования.

Вы можете разделить действия, вычисления и данные на уровни в соответствии с частотой изменения. В глобальном масштабе такое деление становится источником архитектурной информации и приводит к многослойной архитектуре.

Функции высшего порядка помогают достичь новых высот абстракции

Функции высшего порядка (функции, которые получают функции в аргументах и/или возвращают функции) освобождают вас от необходимости писать один и тот же низкоуровневый код снова и снова. Представьте себе свою будущую карьеру — сколько циклов `for` вы еще напишете? Сколько команд `try/catch`? Функции высшего уровня позволяют написать их раз и навсегда, освобождая вас для написания кода предметной области. Функции высшего порядка очень часто встречаются в функциональном программировании.

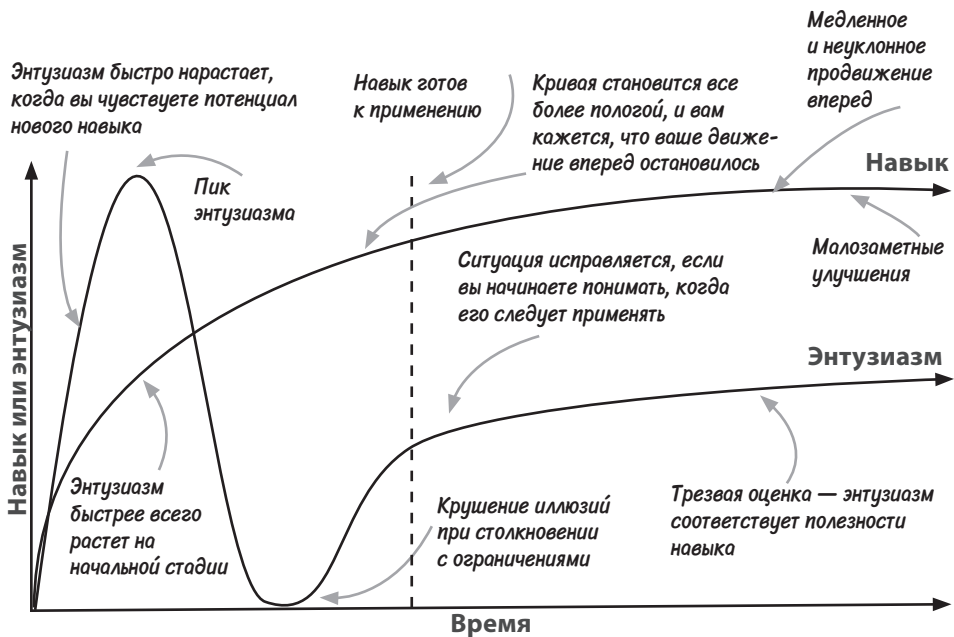
Вы можете управлять временной семантикой вашего кода

Многие ошибки возникают из-за выполнения кода в нежелательном порядке из-за того, что он выполняется на нескольких временных линиях. В наши дни большинство программных продуктов выполняется на нескольких временных линиях, поэтому очень важно понимать, как выполняется ваш код.

Временные диаграммы помогают наглядно представить выполнение вашего кода во времени. Они отражают последовательное и параллельное выполнение действий. Помните: действия особенно зависят от того, когда они вызываются (упорядочение) и сколько раз они вызываются (повторение). Функциональные программисты понимают необходимость управления упорядочением и повторением. Вы можете изменить семантику выполнения, создавая примитивы, которые предоставляют разную семантику упорядочения и повторения.

Приобретение навыков: победы и разочарования

Каждый раз, когда мы изучаем новый навык, мы проходим похожий процесс закрепления навыка. На первых порах мы горим энтузиазмом от новообретенной силы. Мы стараемся применять навык повсюду, чтобы поэкспериментировать с новой игрушкой. Но вскоре возводимая башня становится слишком высокой и начинает обваливаться. Раз за разом мы сталкиваемся с ограничениями нового навыка. Постепенно мы понимаем, когда его следует применять. Мы узнаем, как он взаимодействует с другими сущностями, которые нам известны, а также узнаем, когда его применять не стоит.



Этот процесс — естественный путь, по которому проходим мы все в процессе изучения функционального программирования. Постарайтесь понять, в какой точке этого пути вы находитесь. Получайте удовольствие от каждого его момента. И когда вы дойдете до конца, вы по-настоящему освоите этот навык. Вы сможете оглянуться и припомнить весь теоретический материал, все эксперименты (успешные и неуспешные), все тупики и правильные пути, которые привели вас к мастерству.

План главы

- Список навыков.
- Два пути к мастерству.
- Путь 1: песочница.
- Путь 2: реальный код.
- Дальнейшее путешествие.

Настоящая проблема этой учебной кривой — излишне усердное применение новых навыков, которые еще не были нормально освоены. На первых порах ваш энтузиазм растет гораздо быстрее, чем ваша квалификация. Появляется риск того, что вы будете злоупотреблять новыми знаниями в реальном коде, с ущербом для удобочитаемости и сопровождения. Но в какой-то момент вы осознаете его ограничения и тогда сможете применять этот навык в реальном коде. Рассмотрим стратегию дальнейшего совершенствования ваших навыков и их применения в текущей работе.

Параллельные пути к мастерству

На предыдущей странице была представлена модель развития наших навыков с ростом нашего энтузиазма. Проблема в том, что наш энтузиазм растет быстрее мастерства. Мы хотим применять новые навыки повсеместно, даже там, где они применяться не должны. Не стоит рисковать, применяя непроработанные навыки в реальном коде, где они могут ухудшить удобочитаемость и сопровождаемость. Как же отработать навык, чтобы преодолеть пик энтузиазма? И как определить, что мы достигли нужной квалификации и ей можно доверять?

Можно воспользоваться моделью с двумя параллельными путями к мастерству. На одном пути вы радостно экспериментируете и практикуетесь в применении навыка. На другом пути навыки трезво и осмысленно применяются в реальном коде.

Вы находитесь
здесь

План главы

- Список навыков.
- **Два пути к мастерству.**
- Путь 1: песочница.
- Путь 2: реальный код.
- Дальнейшее путешествие.

Путь 1: песочница

Пока фаза разочарования еще не пройдена, нам понадобится безопасное место для экспериментов с навыками. Примеры безопасных мест:

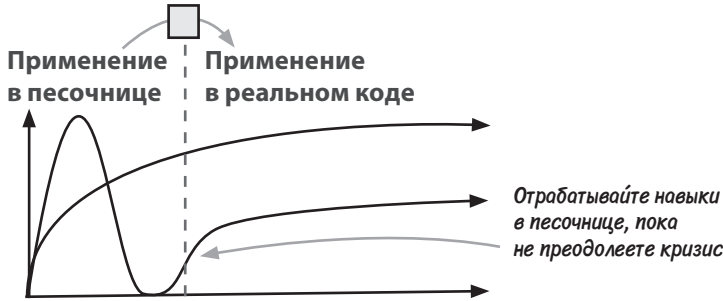
- практические упражнения;
- побочные проекты;
- отвергнутая ветвь рабочего кода.

Путь 2: реальный код

Когда вы достигнете трезвого понимания ситуации и почувствуете, что навык хорошо отработан, вы можете начать пользоваться им в реальных проектах,

зависящих от хороших практик программирования. Возможные места для применения:

- рефакторинг существующего кода;
- новая функциональность в существующем продукте;
- новый проект, создаваемый с нуля;
- обучение.



Песочница — изолированная среда, безопасная для экспериментов, а в реальном коде проявляются все последствия. Заметим, что обе фазы важны для освоения навыков. Поначалу улучшения будут наиболее заметными. Затем вы достигнете точки, в которой все определяется постепенным совершенствованием и адекватностью оценки. Суть начальной стадии заключается в экспериментах и проверках ограничений. Затем давление условий реальной эксплуатации заставляет вас совершенствовать свои навыки.



Пища для ума

На с. 591 приведен список навыков, которые вы изучили в этой книге. Оцените, где вы находитесь на этой кривой для каждого навыка. Какие навыки уже готовы для применения в реальном коде? Каким навыкам еще необходима песочница?

Песочница: создание побочного проекта

Побочные проекты — неплохой способ отработки новых навыков. Они предоставляют идеальную возможность учиться без серьезных последствий в случае неудачи. Но как выбрать побочный проект? Четыре критерия помогут вам остановиться на проекте, который будет интересным и контролируемым:

Ограничьте масштаб проекта

Ни в коем случае не запускайте проект настолько большой, что он никогда не будет до конца реализован. Ограничьтесь небольшим побочным проектом. Вы всегда сможете расширить его, когда ваши навыки улучшатся.

Задайте себе следующие вопросы

- Как выглядит эквивалент приложения Hello, World для веб-приложения?
- Как выглядит эквивалент приложения Hello, World для твиттерного бота?

Выберите экстравагантный побочный проект

Если проект будет слишком серьезным, может оказаться, что он не удовлетворяет произвольным целям, которые вы для него поставили. Но экстравагантный проект допускает свободные исследования и смену направления, если вы найдете нечто интересное, над чем вам захочется работать. Интерес и увлеченность помогают в обучении.

Задайте себе следующие вопросы

- Как бы найти нечто несерьезное, что даже не воспринимается как работа?
- Над чем вам будет интересно работать даже в случае неудачи?

Используйте знакомые навыки плюс один новый навык

Пока вы учитесь работать с временными линиями, вероятно, изучение нового веб-фреймворка лучше отложить на будущее. Сформируйте набор навыков, с которыми вы уже чувствуете себя достаточно уверенно, и добавьте в него один новый навык.

Задайте себе следующие вопросы

- Что я умею хорошо делать прямо сейчас?
- Как мне отработать еще один навык в дополнение к этому набору?

*Вы находитесь
здесь*

План главы

- Список навыков.
- Два пути к мастерству.
- **Путь 1: песочница.**
- Путь 2: реальный код.
- Дальнейшее путешествие.

Расширяйте проект так, как считаете нужным

Лучший побочный проект будет дожидаться того момента, когда вы будете готовы потренироваться в отработке того или иного навыка. Он будет содержать код, нуждающийся в работе по проектированию, а реализованные в нем базовые возможности могут расширяться с применением новых навыков. Например, вы можете реализовать очень простой блог, а на выходных к нему можно добавить аутентификацию пользователей.

Задайте себе следующие вопросы

- Что может стать надежным фундаментом для дальнейших исследований?
- Какие базовые возможности можно добавить позднее?

Песочница: практические упражнения

Для усвоения навыков необходима практика. А иногда лучшей разновидностью практики являются изолированные упражнения. Такие упражнения существуют вне контекста и определяются четкими требованиями. В сущности, это мишень для отработки навыков, чтобы вы могли потренироваться в их применении без последствий. Несколько отличных источников упражнений:

Edabit (<https://edabit.com/challenges>)

Edabit предлагает множество упражнений из области программирования. Они компактны. Они хорошо объясняются, и им назначаются оценки от «Очень просто» до «Для знатоков». Используйте эти задачи для отработки навыков функционального программирования. Посмотрите, удастся ли вам решить одну и ту же задачу разными способами с применением разных навыков.

*Вы находитесь
здесь*

План главы

- Список навыков.
- Два пути к мастерству.
- **Путь 1: песочница.**
- Путь 2: реальный код.
- Дальнейшее путешествие.

Project Euler (<https://projecteuler.net>)

На сайте Project Euler собрано множество задач из области программирования. Часто они сильно связаны с математикой, но все четко объясняется. В этих упражнениях важно то, что они заставляют вас сталкиваться с реальными ограничениями. Например, вычислить первые 10 простых чисел легко. С другой стороны, найти 1 000 000-е простое число до того, как сядет солнце, — непростая задача! Вы столкнетесь с ограничениями памяти, ограничениями по быстродействию, ограничениями на размер стека и т. д. Все эти ограничения заставят вас применять свои навыки в практическом, а не сугубо теоретическом ключе.

CodeWars (<https://codewars.com>)

CodeWars предлагает большую подборку упражнений — достаточно сложных, чтобы вы могли проверить свои навыки, но достаточно простых, чтобы их можно было решить за несколько минут. Эти упражнения также прекрасно подходят для применения разных навыков к одной задаче.

Code Katas (<https://github.com/gamontal/awesome-katas>)

Code Katas — вид упражнений, в которых одна и та же задача решается многократно. Упражнения выполняются скорее для тренировки процесса программирования, нежели для решения конкретной задачи. Они полезны еще и тем, что позволяют интегрировать новые навыки функционального программирования с другими навыками разработки (например, тестированием).

Реальный код: устранение ошибок

Каждый навык в книге был выбран с таким расчетом, чтобы вы могли немедленно применить его на практике. Но когда вы разглядываете кодовую базу на 100 тысяч строк, может возникнуть вопрос: с чего начать? Не беспокойтесь, это распространенная ситуация. Здесь нужно начать с малого... и вообще хотя бы с чего-то начать. В будущем вы сможете постепенно совершенствовать свой код.

Некоторые из навыков, которые вы узнали, можно сразу же применить для прямого устранения основных источников ошибок. Такие места оказываются наилучшим вариантом для ощутимых улучшений в базе данных и признательности коллег.

Уменьшите количество глобальных изменяемых переменных на 1

В главах 3–5 вы научились выявлять неявные входные и выходные значения у функций. Некоторые из них являются глобальными изменяемыми переменными. Они позволяют разным частям кода совместно использовать данные. Тем не менее совместное использование изменяемых данных становится колоссальным источником ошибок. Устранение даже всего одной глобальной изменяемой переменной может принести огромную пользу. Найдите глобальные переменные, выберите одну из них и проведите рефакторинг функций, в которых она используется, пока глобальная переменная не станет лишней. А затем переходите к следующей! Ваши коллеги будут вам благодарны.

Вы находитесь
здесь

План главы

- Список навыков.
- Два пути к мастерству.
- Путь 1: песочница.
- **Путь 2: реальный код.**
- Дальнейшее путешествие.

Уменьшите количество аномальных временных линий на 1

В главах 15–17 вы научились пользоваться временными диаграммами для анализа поведения кода. Временные диаграммы помогают выявлять ситуации гонки и другие проблемы упорядочения. А навыки изоляции, совместного использования и координации позволяют исключать нежелательные упорядочения. Когда вы программируете в кодовой базе достаточно долго, у вас появляется интуитивное ощущение о возможных причинах ошибок. Обусловлены ли они гонками? Выберите одну из них, нарисуйте диаграмму и примените навыки для исключения неправильных вариантов упорядочения.

Реальный код: постепенное улучшение проекта

Некоторые навыки, представленные в книге, можно немедленно использовать для пошагового улучшения структуры кода. Здесь самое главное — «пошагового». Структура очень важна, но выигрыш не будет очевиден. Тем не менее со временем улучшения будут накапливаться и преимущества от улучшения структуры начнут проявляться.

Выделите одно вычисление из действия

Исключать действия из кода очень трудно. У большинства действий есть реальное предназначение. Лишние действия встречаются редко. Тем не менее одной из возможных мер может стать уменьшение количества действий. Найдите действие, содержащее большой объем логики, и выделите часть логики в вычисление. Действия должны быть простыми и прямолинейными.

Преобразуйте один неявный ввод или вывод в явный

Стоит еще раз подчеркнуть, что полностью исключить конкретное действие непросто. Более перспективный путь — исключить один неявный ввод или вывод. Если действие имеет четыре неявных ввода, уменьшение их числа до трех станет серьезным улучшением. Хотя действие остается действием, оно немного улучшается. Измененное действие в меньшей степени привязано к состоянию системы.

*Вы находитесь
здесь*

План главы

- Список навыков.
- Два пути к мастерству.
- Путь 1: песочница.
- **Путь 2: реальный код.**
- Дальнейшее путешествие.

Замените один цикл for

В главах 12–14 были представлены некоторые полезные функции, заменяющие цикл `for`. Несмотря на то что исключение одного цикла `for` не кажется таким уж

выдающимся достижением, циклы `for` формируют значительную часть логики ваших алгоритмов. Начните заменять циклы такими функциями, как `forEach()`, `map()`, `filter()` и `reduce()`. Замена циклов `for` станет промежуточным шагом на пути к более функциональному стилю. Возможно, на этом пути вы обнаружите новые функциональные инструменты, специфические для вашего кода.

Популярные функциональные языки

Ниже перечислены функциональные языки, изучение которых принесет практическую пользу. Хотя существует много других языков, не включенных в список, эти языки популярны, и для них разработаны библиотеки, с которыми можно сделать практически все. Любой из этих языков — отличный кандидат на роль языка программирования общего назначения. На нескольких ближайших страницах эти языки сгруппированы по своим целям: для получения работы, запуска на различных платформах или изучения различных аспектов ФП.

Clojure (<https://clojure.org>)

Clojure работает в Java Virtual Machine и JavaScript (в форме of ClojureScript).

Elixir (<https://elixir-lang.org>)

Elixir работает в Erlang Virtual Machine. Для управления параллелизмом в нем используются акторы (actors).

Swift (<https://swift.org>)

Swift — язык программирования компании Apple, распространяемый с открытым кодом.

Kotlin (<https://kotlinlang.org>)

Kotlin объединяет объектно-ориентированное программирование с функциональным в один язык JVM.

Haskell (<https://haskell.org>)

Haskell — язык со статической типизацией, используемый в академических и корпоративных средах, а также в стартапах.

Erlang (<https://erlang.org>)

Erlang создавался для обеспечения устойчивости к отказам. Для управления параллелизмом в нем используются акторы.

Elm (<https://elm-lang.org>)

Elm — язык со статической типизацией для построения интерфейсных веб-приложений, компилируемый в JavaScript.

План главы

- Список навыков.
- Два пути к мастерству.
- Путь 1: песочница.
- Путь 2: реальный код.
- **Дальнейшее путешествие.**

Вы находитесь
здесь

Scala (<https://scala-lang.org>)

Scala объединяет объектно-ориентированное программирование с функциональным. Работает в Java Virtual Machine и JavaScript.

F# (<https://fsharp.org>)

F# — язык со статической типизацией, работающий в Microsoft Common Language Runtime.

Rust (<https://rust-lang.org>)

Rust — системный язык с мощной системой типов, спроектированный для предотвращения утечек памяти и ошибок параллелизма.

PureScript (<https://www.purescript.org>)

PureScript — похожий на Haskell язык, компилируемый в JavaScript для выполнения в браузере.

Racket (<https://racket-lang.org>)

Racket имеет богатую историю, большое и энергичное сообщество.

Reason (<https://reasonml.github.io>)

Reason — язык со статической типизацией, компилируемый в JavaScript и платформенные сборки.

Функциональные языки с наибольшим количеством вакансий

Возможно, вы решили изучить новый язык, чтобы найти работу в области функционального программирования. Считается, что вакансии для функциональных программистов относительно редки, но они существуют. Чтобы обеспечить себе максимальные шансы на получение работы, выберите один из этих языков. Несмотря на то что все языки, перечисленные выше, могут использоваться для практического программирования, по количеству вакансий лидируют четыре языка:

Elixir — Kotlin — Swift — Scala — Rust

Упорядочены по возрастанию сложности изучения

Следующие три языка уступают им по количеству вакансий, но и они открывают немало возможностей:

Clojure — Erlang — Haskell

Упорядочены по возрастанию сложности изучения

К сожалению, другие языки нельзя рекомендовать для изучения исключительно с целью получения работы.

Функциональные языки на разных платформах

Также функциональные языки можно упорядочить по доступности на выбранной вами целевой платформе.

Браузер (ядро JavaScript)

Эти языки компилируются в JavaScript. Кроме браузера, они также могут выполняться в Node:

Elm — ClojureScript — Reason — Scala.js — PureScript

Упорядочены по возрастанию сложности изучения

Серверы веб-приложений

Эти языки обычно используются для реализации серверов для веб-приложений:

Elixir — Kotlin — Swift — Racket — Scala — Clojure — F# — Rust — Haskell

Мобильные платформы (iOS и Android)

Упорядочены по возрастанию сложности изучения

Платформенные: **Swift**.

Через JVM: **Scala — Kotlin**.

Через Xamarin: **F#**.

Через React Native: **ClojureScript — Scala.js**.

Встроенные устройства

Rust

Возможность получения знаний

Погружение в язык помогает в учебе. Можно выбрать язык на основании того, чему он может вас научить. Ниже приведен список языков, сгруппированных по отличительным признакам. Такие языки могут создать великолепную учебную среду.

Статическая типизация

Самые мощные системы типов в наше время встречаются в функциональных языках. Такие системы типов базируются на математической логике, а их логическая целостность была доказана. Типы не ограничиваются предотвращением ошибок: они также помогают вам в проектировании программных продуктов. Хорошая система типов напоминает логика, который сидит у вас на плече и помогает в разработке хорошего ПО. Если вы захотите узнать больше по теме, следующие языки вам в этом помогут:

Elm — Scala — F# — Reason — PureScript — Rust — Haskell

Упорядочены по возрастанию сложности изучения

В Swift, Kotlin и Racket тоже существуют системы типов, хотя и не столь мощные.

Функциональные инструменты и преобразования данных

Во многих функциональных языках существуют эффективные инструменты для преобразования данных, но перечисленные ниже языки особенно хороши в этом отношении. Вместо того чтобы подталкивать вас к определению новых типов, эти языки работают с небольшим набором типов данных и многочисленными операциями с ними:

Kotlin — Elixir — Clojure — Racket — Erlang

Упорядочены по возрастанию сложности изучения

План главы

- Список навыков.
- Два пути к мастерству.
- Путь 1: песочница.
- Путь 2: реальный код.
- **Дальнейшее путешествие.**

Вы находитесь здесь

Параллелизм и распределенные системы

Многие функциональные языки хорошо работают в условиях многопоточности в основном из-за неизменяемых структур данных. Тем не менее некоторые языки особенно преуспевают в этой области, потому что они изначально концентрируются на этой задаче. Эти языки обладают выдающимися средствами для простого и прямолинейного управления несколькими временными линиями. Их можно классифицировать по категориям используемых средств.

С примитивами синхронизации: **Clojure — F# — Haskell — Kotlin**.

С использованием модели акторов: **Elixir — Erlang — Scala**.

Через систему типов: **Rust**.

Математическая основа

Функциональные языки заимствуют многие идеи из математики. Именно это привлекает в функциональном программировании многих разработчиков. Если вы склонны к продвинутой математической теории, ниже приведен список областей функционального программирования, которые вам могут прийти по вкусу.

Лямбда-исчисление

Лямбда-исчисление — простая и мощная математическая система, включающая определения и вызовы функций. Так как в нем используются многочисленные функции, ФП может свободно использовать идеи лямбда-исчисления.

Комбинаторы

Одним из интересных уголков лямбда-исчисления является идея *комбинаторов*. Комбинаторами называются функции, которые изменяют и комбинируют другие функции.

Теория типов

Теория типов — другой аспект лямбда-исчисления, который обрел свое место в функциональном программировании. По сути теория типов представляет собой логику, примененную к вашему коду. Она отвечает на вопрос: что можно вывести и доказать без введения логических противоречий? Теория типов формирует основу статических систем типов в функциональных языках.

Теория категорий

Рискуя чрезмерным упрощением, можно сказать, что теория категорий является ветвью абстрактной математики, которая исследует структурные сходства между разными типами. Применительно к программированию она открывает настоящую сокровищницу идей из области проектирования и реализации программных продуктов.

Системы эффектов

Системы эффектов заимствуют из теории категорий математические объекты (например, монады и аппликативные функторы). Эти концепции использовались для моделирования различных действий: изменяемого состояния, выдаваемых исключений и других побочных эффектов. Действия моделируются с использованием неизменяемых данных — таким образом расширяются границы того, что возможно сделать с использованием только вычислений и данных.

План главы

- Список навыков.
- Два пути к мастерству.
- Путь 1: песочница.
- Путь 2: реальный код.
- **Дальнейшее путешествие.**

Вы находитесь здесь

Литература

Ниже приведена подборка книг по функциональному программированию. Из всех книг, которые я прочитал, эти я рекомендую для применения на следующих шагах.

«Функционально легкий JavaScript»

(Кайл Симпсон)

(Functional-Light JavaScript (Kyle Simpson))

В книге сочетается руководство по стилю программирования на JavaScript с введением многих концепций функционального программирования. В ней доступно объясняются многие концепции, которые, как мне казалось, невозможно объяснить без погружения в теоретическую сторону ФП. Настоятельно рекомендуется к прочтению.

План главы

- Список навыков.
- Два пути к мастерству.
- Путь 1: песочница.
- Путь 2: реальный код.
- Дальнейшее путешествие.

Вы находитесь здесь

«Доменное моделирование в функциональном стиле» (Скотт Влашин)

(Domain Modeling Made Functional (Scott Wlaschin))

Книга показывает, как перейти от разговоров с заказчиком к полноценной функциональной реализации схемы работы. Вы узнаете, как использовать типы для моделирования предметной области. Также в ней приводится лучшее объяснение проектирования, ориентированного на предметную область, из всех мне встречавшихся.

«Структура и интерпретация компьютерных программ»

(Джеральд Джей Сассман, Гарольд Абельсон, Джули Суссман)

(Structure and Interpretation of Computer Programs (Harold Abelson, Gerald Jay Sussman, Julie Sussman))

Классическая книга, которая использовалась в качестве учебника на курсах компьютерной теории в Массачусетском технологическом институте. Большинству рядовых читателей она может показаться слишком сложной. Но сколько бы трудов вы ни приложили к тому, чтобы разобраться в материале, книга остается наиболее доступным источником для изучения многих важных идей. А если вы окажетесь в тупике, не огорчайтесь. Это одна из тех книг, к которым вы возвращаетесь через годы с ростом вашей квалификации.

«Грокаем функциональное программирование» (Михал Плахта)

(Grokking Functional Programming (Michał Płachta))

Эта книга станет хорошим введением в функциональное программирование, представленным с другой точки зрения. Если вам понравилась эта книга, «Грокаем функциональное программирование» расширит ваше понимание чистых функций, функциональных инструментов и неизменяемых данных. В ней

моделирование данных рассматривается глубже, чем мы могли себе позволить в этой книге.

Кроме этих общих книг по программированию, рекомендую выбрать пару популярных книг по языкам программирования, на которых вы работаете или которые собираетесь изучать. Книги о функциональном программировании написаны для многих языков. И о функциональных языках такие книги тоже существуют.

Итоги главы

В этой главе мы воздали должное навыкам, которые вы изучили в этой книге. Теперь вы знаете достаточно для применения функционального программирования в своем коде. Также был представлен план, который позволит вам продолжить практиковаться и оттачивать свои навыки. Наконец, когда придет время, вы всегда можете узнать больше, и в этой главе приведены возможные источники информации.

Резюме

- Вы овладели рядом важных навыков. Подумайте, как довести их до совершенства.
- Мы с неоправданным энтузиазмом относимся к новым навыкам до того, как будем готовы к их разумному применению. Найдите безопасное место для экспериментов.
- Функциональное программирование может улучшить ваш реальный код. Давление условий реальной эксплуатации поможет вам отточить приобретенные навыки.
- Есть много функциональных языков, которые могут использоваться как для коммерческих продуктов, так и для побочных проектов. Еще никогда не было более подходящего момента для построения карьеры в функциональном программировании.
- Функциональное программирование используется для применения математических концепций. Если это вам по душе — изучайте! Здесь есть много такого, что стоит узнать.
- Есть целый ряд неплохих книг, которые помогут вам больше узнать о функциональном программировании. Ознакомьтесь с ними.

Что дальше?

Ничего! Будьте здоровы!

Эрик Норманд
Грокаем функциональное мышление

Перевел с английского Е. Матвеев

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Ведущий редактор	<i>Н. Гринчик</i>
Литературные редакторы	<i>Ю. Зорина, Н. Хлебина</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>С. Беляева, Н. Викторова</i>
Верстка	<i>Л. Егорова</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».
Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,
Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 02.2023. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 01.12.22. Формат 70×100/16. Бумага офсетная. Усл. п. л. 49,020. Тираж 1200. Заказ 0000.

А. Бхаргава

ГРОКАЕМ АЛГОРИТМЫ. ИЛЛЮСТРИРОВАННОЕ ПОСОБИЕ ДЛЯ ПРОГРАММИСТОВ И ЛЮБОПЫТСТВУЮЩИХ



Алгоритмы — это всего лишь пошаговые инструкции решения задач, и большинство таких задач уже были кем-то решены, протестированы и проверены. Можно, конечно, погрузиться в глубокую философию гениального Кнута, изучить многостраничные фолианты с доказательствами и обоснованиями, но хотите ли вы тратить на это свое время? Откройте великолепно иллюстрированную книгу, и вы сразу поймете, что алгоритмы — это просто. А грокать алгоритмы — веселое и увлекательное занятие.

КУПИТЬ